OPEN-TEXT EDITION

How much to order, and when to order it

Analysis of Inventory Systems

with **spreadsheets** and the **Kotlin Simulation Library** (KSL)



Manuel D. Rossetti

University of Arkansas

Open Educational Resource

Free & open source

Analysis of Inventory Systems

Manuel D. Rossetti

2026-10-30

Table of contents

Preface **1**
Approach . . . 1
How this book is organized . . . 2
Software . . . 2
A note on notation . . . 3
How this book was written . . . 3
Acknowledgments . . . 5
License . . . 6

About the Author **7**

1. Introduction to Inventory Systems **8**
1.1. Why Hold Inventory? . . . 9
1.1.1. Pipeline Inventory . . . 9
1.1.2. Cycle Stock . . . 10
1.1.3. Safety Inventory . . . 10
1.1.4. Anticipation Inventory . . . 11
1.1.5. Speculative Inventory . . . 11
1.1.6. Different Causes, Different Levers . . . 11
1.1.7. The Cost of Not Holding It . . . 14
1.2. Classifying Inventory Systems . . . 14
1.2.1. Where the Demand Comes From . . . 15
1.2.2. Known or Uncertain Demand . . . 16
1.2.3. Constant or Time-Varying Demand . . . 16
1.2.4. How the System Is Reviewed . . . 17
1.2.5. The Lead Time . . . 17

1.2.6. What Happens to Unmet Demand 18
1.2.7. The Planning Horizon 19
1.2.8. One Item or Many 19
1.2.9. One Location or Several 20
1.2.10. The Dimensions Together 20
1.2.11. Which Costs the Classification Implies 21
1.2.12. Choosing a Classification 23
1.3. The State of an Inventory System 23
1.3.1. On Hand, On Order, and Backordered 24
1.3.2. Lost Sales 25
1.3.3. Why Ordering Decisions Use the Inventory Position 26
1.3.4. Little's Law and the Averages It Connects 29
1.3.5. Stockout Indicators 31
1.3.6. The Demand Process 32
1.4. Cost Parameters 33
1.4.1. What an Inventory Model Charges 33
1.4.2. The Marginality Test 35
1.4.3. Unit Cost, c 37
1.4.4. Holding Cost, h 37
1.4.5. Fixed Order Cost, k 40
1.4.6. Stockout and Backorder Costs, π and b 40
1.4.7. Stocking Position Cost, f 42
1.4.8. How Accurate Do These Need to Be? 43
1.5. Estimating Cost Parameters from Standard Data 45
1.5.1. Where the Numbers Come From 46
1.5.2. Unit Cost 46
1.5.3. The Carrying Charge 47
1.5.4. The Fixed Order Cost 50
1.5.5. Stockout and Backorder Costs, and What to Do Instead 52
1.5.6. Avoiding Double Counts 53
1.5.7. When a Point Estimate Is Not Defensible 55
1.6. Performance Measures 56
1.6.1. Cost as a Performance Measure 56
1.6.2. Service Measures 58

1.6.3. The Problem the Rest of the Book Solves 63
1.7. Summary 65
1.8. Exercises 67

2. Managing Inventory **71**
2.1. The Manager's Problem 71
2.1.1. What Scale Changes 72
2.1.2. What the Manager Decides 73
2.1.3. What This Chapter Does Not Cover 75
2.2. Grouping and Segmentation 76
2.2.1. Two Reasons to Group 76
2.2.2. Pareto Analysis and ABC Classification 77
2.2.3. What One Criterion Misses 81
2.2.4. Deriving the Criterion Instead of Choosing Weights 83
2.2.5. Which Attributes, and Which Forbid Grouping . . . 83
2.2.6. Demand Pattern Segmentation 85
2.2.7. Clustering for Policy 88
2.2.8. Evaluating a Grouping 89
2.2.9. Combining Schemes 91
2.2.10. Using a Classification 91
2.3. Inventory Record Analysis 93
2.3.1. What a Record Claims, and Why It Drifts 93
2.3.2. Why Accuracy Comes First 95
2.3.3. What Error Actually Costs 96
2.3.4. Measuring Record Accuracy 97
2.3.5. Physical Inventory and Cycle Counting 100
2.3.6. Cycle Counting Methods 101
2.3.7. Setting Count Frequency 104
2.3.8. Monitoring Accuracy Over Time 109
2.3.9. Accuracy Is Not a Local Property 112
2.3.10. Correction Is Not the Point 113
2.4. Other Responsibilities 115
2.4.1. Excess and Obsolete Stock 115
2.4.2. Performance Reporting 117

2.5. Summary 118
2.6. Exercises 119

I. Deterministic Models **125**

3. Deterministic Lot Sizing **126**
3.1. The General Cycle 127
3.1.1. Assumptions 127
3.1.2. One Cycle, Four Segments 128
3.2. Average Inventory and Backorders 131
3.3. The Total Cost Rate 133
3.4. Optimizing the General Model 135
3.5. The Classical Models as Special Cases 136
3.5.1. No Charge Per Unit Short 137
3.5.2. No Backorders 137
3.5.3. Instantaneous Replenishment 138
3.5.4. The Classical Economic Order Quantity 139
3.6. Lead Time and the Reorder Point 143
3.7. Sensitivity of the Order Quantity 144
3.8. Quantity Discounts 145
3.8.1. All-Units Discounting 146
3.8.2. Incremental Discounting 150
3.9. Performance Measures for the Cycle 153
3.10. Building These Models in a Spreadsheet 155
3.10.1. The Four-Block Layout 155
3.10.2. The Reorder Point Cell 159
3.10.3. Reducing the Production Model to the Order Model 162
3.10.4. Discount Schedules as Columns 166
3.10.5. What the Worksheets Do Not Do 171
3.11. Designing the Software 172
3.11.1. What Changes, and What Must Not 172
3.11.2. One Pipeline, Six Models 174
3.11.3. Sealed Types for Closed Sets 177

3.11.4. An Answer That Carries Its Reasoning 179
3.12. The Same Models in Code 181
3.12.1. The Four Blocks as Objects 182
3.12.2. The Reorder Point 184
3.12.3. The Same Substitution 185
3.12.4. Discount Schedules as Candidates 187
3.12.5. Doing the Three Things a Worksheet Does Not . . . 189
3.13. Summary 190
3.14. Exercises 192

4. Multi-Item and Constrained Systems 198
4.1. When Items Stop Being Independent 199
4.2. A Constrained Lot Sizing Problem 200
4.2.1. First Check Whether the Constraint Binds 202
4.2.2. Moving the Constraint into the Objective 202
4.2.3. Finding the Multiplier 204
4.2.4. When the Limit Is on Ordering, Not on Stock 207
4.3. What the Multiplier Means 209
4.4. The Exchange Curve 211
4.4.1. One Item First 212
4.4.2. The Aggregate Curve 213
4.4.3. Reading a Cost Ratio from Practice 215
4.4.4. When Ordering Costs Differ Across Items 218
4.5. Joint Replenishment 224
4.5.1. The Coordinated Problem 224
4.5.2. Power-of-Two Policies 226
4.5.3. What a Power-of-Two Policy Costs 227
4.6. Multi-Echelon Systems 233
4.6.1. Echelon Inventory and Echelon Holding Cost 234
4.6.2. Serial Systems 239
4.6.3. Distribution Systems 245
4.7. Building These Models in a Worksheet 249
4.7.1. The Constrained Problem 249
4.7.2. The Exchange Curve 256

4.7.3. Intervals and Rounding . . . 259
4.7.4. The Two Echelon Structures . . . 264
4.7.5. What a Worksheet Can and Cannot Do . . . 267
4.8. Designing the Software . . . 269
4.8.1. What the Software Must Do . . . 269
4.8.2. Finding the Nouns . . . 273
4.8.3. What Each Thing Knows and Does . . . 276
4.8.4. The Classes . . . 279
4.8.5. Measuring, and Limiting a Measure . . . 282
4.8.6. Alternatives Are Types, Settings Are Values . . . 287
4.8.7. Three Couplings and No Common Base . . . 290
4.8.8. How the Objects Solve It . . . 292
4.8.9. Solving the Chapter's Examples . . . 296
4.8.10. Checking the Design Against the Mathematics . . . 301
4.9. Summary . . . 303
4.10. Exercises . . . 306

5. Dynamic Lot Sizing 311
5.1. When the Demand Rate Changes . . . 312
5.1.1. The Requirements Schedule . . . 313
5.1.2. The Assumptions . . . 314
5.1.3. Where This Chapter Sits . . . 315
5.2. What a Plan Costs . . . 315
5.2.1. The Purchase Term . . . 316
5.2.2. Three Plans Without Any Theory . . . 316
5.3. Two Properties That Make the Problem Small . . . 319
5.3.1. Zero-Inventory Ordering . . . 320
5.3.2. The Window Cost . . . 321
5.3.3. Property 2: A Requirement That Forces an Order . 323
5.4. Choosing the Order Periods . . . 324
5.4.1. The Problem Is a Shortest Path . . . 324
5.4.2. The Recursion . . . 326
5.4.3. The Same Problem for Solver . . . 330
5.4.4. How Much Work It Is . . . 331

5.5. Three Heuristics and One Shape . . . 332
5.5.1. One Procedure . . . 332
5.6. What a Heuristic Costs . . . 336
5.6.1. When None of This Is Worth Doing . . . 337
5.6.2. Sensitivity to the Cost Ratio . . . 338
5.6.3. What the Aggregate Studies Say . . . 339
5.7. The Rolling Horizon . . . 340
5.7.1. What the Truncation Costs . . . 340
5.7.2. Does the Heuristic Ever Win? . . . 341
5.7.3. Nervousness . . . 342
5.8. Building These Models in a Worksheet . . . 343
5.8.1. The Data Sheet . . . 344
5.8.2. Costing a Plan . . . 346
5.8.3. The Recursion and the Traceback . . . 347
5.8.4. The Three Rules . . . 349
5.8.5. The Mixed Integer Program . . . 358
5.8.6. What the Worksheet Cannot Do . . . 358
5.9. Designing the Software . . . 358
5.9.1. What the Software Must Do . . . 360
5.9.2. Finding the Nouns . . . 360
5.9.3. What Each Thing Knows and Does . . . 361
5.9.4. The Schedule . . . 363
5.9.5. Three Rules Are One Rule . . . 366
5.9.6. The Algorithm Is Not a Heuristic . . . 370
5.9.7. How the Objects Solve It . . . 372
5.9.8. Rolling the Horizon . . . 374
5.9.9. Solving the Chapter's Examples . . . 375
5.10. Summary . . . 378
5.11. Exercises . . . 379
5.11.1. Terminology and Concepts . . . 380
5.11.2. Working the Methods by Hand . . . 382
5.11.3. Modeling Problems . . . 384
5.11.4. The Rolling Horizon and the Solver . . . 386
5.11.5. Using the Workbook . . . 387

5.11.6. Using the Software 388
5.11.7. From the Literature 389

6. Material and Distribution Requirements Planning 390
6.1. Demand You Should Not Forecast 391
6.1.1. Why Forecasting It Is a Mistake 391
6.1.2. Where This Chapter Sits 392
6.2. The Bill of Material 392
6.2.1. Low-Level Codes 394
6.2.2. The Explosion Calculation 395
6.3. The MRP Record 395
6.3.1. The Maximum Is the Whole Idea 397
6.3.2. Safety Stock and Allocations 398
6.3.3. How Long the Horizon Has to Be 399
6.4. Running the Explosion 399
6.5. What Lot Sizing Does to the Level Below 404
6.5.1. One Rule Changes, and Everything Moves 404
6.5.2. What the Variability Coefficient Does Not Tell You . 407
6.5.3. What Is Actually Done 407
6.6. Distribution Requirements Planning 408
6.6.1. It Is the Same Calculation 409
6.6.2. What the Regions Do to the Center 410
6.7. Nervousness 412
6.8. What Requirements Planning Assumes 413
6.9. Building These Models in a Worksheet 414
6.9.1. The Record, Computed Twice 415
6.9.2. The Explosion 415
6.9.3. The Lumpiness Sheet 415
6.9.4. The Network 420
6.10. Designing the Software 422
6.10.1. What the Software Must Do 422
6.10.2. Finding the Nouns 422
6.10.3. The Join with Chapter 5 424
6.10.4. One Interface for Two Kinds of Rule 425

6.10.5. Two Readings of "Net Requirement" 427
6.10.6. A Network Is a Bill of Material 428
6.10.7. Running the Chapter's Examples 430
6.11. Summary 434
6.12. Exercises 435
6.12.1. Terminology and Concepts 436
6.12.2. Working the Records by Hand 437
6.12.3. Modeling Problems 439
6.12.4. Using the Workbook 440
6.12.5. Using the Software 441
6.12.6. From the Literature 442

II. Stochastic Models **443**

7. Single-Period Stochastic Inventory **444**
7.1. One Order, One Chance 444
7.1.1. Demand Is Not Sales 446
7.2. Working the Decision by Hand 447
7.3. Getting a Demand Distribution 450
7.3.1. Look Before You Model 451
7.3.2. Summarize It 453
7.3.3. Hypothesize, Estimate, Test 455
7.3.4. What the Test Does Not Settle 459
7.4. Intermittent and Slow-Moving Demand 459
7.5. The Newsvendor Model 461
7.5.1. The Same Model as a Profit 462
7.6. The Critical Ratio 463
7.6.1. Reading the Critical Ratio 465
7.7. Continuous Demand 466
7.8. Discrete Demand 469
7.9. Extensions 472
7.9.1. A Penalty Beyond the Lost Margin 473
7.9.2. Stock Already on Hand 473

7.9.3. When the Model Refuses 474
7.9.4. Is It Really a Single Period? 475
7.10. Estimating the Answer by Simulation 475
7.10.1. When It Is Worth It 477
7.11. Building These Models in a Worksheet 478
7.11.1. The Five Sheets 478
7.11.2. The Cells That Carry the Mathematics 482
7.11.3. Reading the Sheets 487
7.11.4. Using the Sheets on Your Own Item 489
7.11.5. What the Worksheet Does Not Do 490
7.12. Designing the Software 490
7.12.1. What the Software Must Do 490
7.12.2. Finding the Nouns 491
7.12.3. The Classes 492
7.12.4. The Two Routes Must Agree 495
7.12.5. Running the Chapter's Examples 495
7.13. Summary 499
7.14. Exercises 501
7.14.1. Terminology and Concepts 502
7.14.2. Working the Model by Hand 502
7.14.3. Modeling Problems 504
7.14.4. Several Items and a Shared Limit 506
7.14.5. Using the Software 513
7.14.6. Using the Workbook 514
7.14.7. From the Literature 515

8. Continuous and Periodic Review Systems 517
8.1. One Item, Watched Continuously 518
8.1.1. Working the Policy by Hand 519
8.2. What to Measure 524
8.2.1. Five Ways to State a Service Requirement 525
8.2.2. The Ready Rate Is Not the Fill Rate 526
8.2.3. A Third Measure, and Why It Is the Same as the Second 527

8.2.4. Service Targets Are Costs You Declined to Estimate 528
8.3. Lead Time Demand 528
8.3.1. Two Ways to Build It 529
8.3.2. The Moments, and the Three Cases 530
8.3.3. Getting the Distribution in Practice 533
8.4. The Base-Stock Model 534
8.4.1. Three Lines of Derivation 535
8.4.2. Choosing the Level 537
8.5. General Batch Sizes 539
8.5.1. Where the Position Sits 540
8.5.2. The Four Measures 540
8.5.3. The Variance of the Backorder Level 541
8.5.4. When the Lead Time Is Random 544
8.6. The Classical Approximation 546
8.6.1. Twelve Assumptions 546
8.6.2. Which Continuous Family 548
8.6.3. The Cost Model 549
8.6.4. Two Conditions, Solved Together 550
8.6.5. The Backorder-Cost Variant 551
8.6.6. When It Does Not Do So Well 554
8.7. Service Levels Instead of a Shortage Cost 557
8.7.1. Type 1: the Cycle Service Level 558
8.7.2. Type 2: the Fill Rate 558
8.7.3. The Same Number Means Two Different Things . . 559
8.7.4. Solving the Problem the Target Poses 560
8.7.5. A Shortcut That Usually Works 561
8.7.6. Reading the Target Back into a Cost 562
8.7.7. Safety Stock and the Safety Factor 564
8.8. The Cost Function and Its Shape 566
8.8.1. The One-for-One Cost, and Why It Is Convex 566
8.8.2. The General Cost Is an Average of One-for-One Costs 567
8.9. What You Know Before You Solve 568
8.9.1. The Optimal Band Straddles the Optimal Base-Stock Level 569

8.9.2. The Cost Is Bounded Above and Below 569
8.9.3. Choosing the Wrong Quantity Is Cheap 570
8.10. An Exact Algorithm 571
8.10.1. The Idea 571
8.10.2. The Algorithm 573
8.10.3. When the Batch Is Large 575
8.10.4. Two Cautions 576
8.11. Lumpy Demand 577
8.11.1. What the Lumpiness Does to the Lead Time Demand 578
8.11.2. The Policy Needs Adapting, and Two Ways Are Used 579
8.11.3. Almost Nothing Else Changes 579
8.11.4. The Fill Rate Is the One Measure That Changes . . 580
8.11.5. What Is Not Covered 585
8.12. The (s, S) Policy 585
8.12.1. Undershoot Is Why the Policy Exists 585
8.12.2. Approximating It with an (r, Q) Policy 587
8.12.3. Where This Goes Next 589
8.13. When Review Is Periodic 590
8.13.1. The Protection Interval 591
8.13.2. The Four Measures 596
8.13.3. Choosing the Level 600
8.13.4. Choosing the Interval 604
8.13.5. The (R, s, S) Policy 606
8.13.6. The Price of Not Watching 608
8.14. Many Items at Once 610
8.14.1. Two Formulations 611
8.14.2. Two Constraints, Two Loops 611
8.14.3. Equal Service Is Not Efficient 612
8.14.4. Where This Connects 614
8.15. Updating r and Q in a Running System 614
8.15.1. What the Forecast Supplies 615
8.15.2. Carrying Them Over a Lead Time 616
8.15.3. Which Parameters Move, and How Often 617

8.16. Building These Models in a Worksheet 618
8.16.1. The Ledger 619
8.16.2. Evaluating a Policy 620
8.16.3. The Algorithm 621
8.16.4. The Approximation 623
8.16.5. Service Levels 625
8.16.6. Reviewing on a Calendar 625
8.16.7. Demand in Lots 627
8.16.8. Using the Sheets on Your Own Item 628
8.16.9. What the Worksheet Cannot Do 631
8.17. Designing the Software 632
8.17.1. What the Software Must Do 632
8.17.2. Finding the Nouns 633
8.17.3. The Window Is the Unit of Work 635
8.17.4. Two Searches, One Answer 636
8.17.5. Where the Conventions Live 637
8.17.6. Running the Chapter's Examples 638
8.18. Summary 641
8.19. Exercises 646
8.19.1. Working the Model by Hand 646
8.19.2. Using the Software 650
8.19.3. Using the Workbook 653

9. Multi-Echelon Inventory Systems 657
9.1. Where Should the Stock Sit? 658
9.1.1. Why Two Locations Are Enough 659
9.1.2. What Changes When Stock Moves Forward 660
9.2. Ordering One Unit at a Time 661
9.2.1. The Item Comes Back 661
9.2.2. The Two Levels 663
9.2.3. Notation 665
9.2.4. What Is Being Measured 666
9.2.5. Where This Leaves Us 667

9.3. Palm's Theorem 667
9.3.1. Why Distribution-Free Is the Whole Point 668
9.3.2. What the Theorem Requires 669
9.3.3. The Pipeline at a Storeroom 670
9.3.4. Where This Leaves Us 673
9.4. A Single Location Under One-for-One Ordering 673
9.4.1. The Substitution 674
9.4.2. The Measures 674
9.4.3. What One More Unit Buys 675
9.4.4. Where This Leaves Us 678
9.5. Working Two Levels by Hand 678
9.5.1. The System and the Rules 679
9.5.2. The Ledger 680
9.5.3. What the Ledger Establishes 690
9.5.4. Where This Leaves Us 692
9.6. The Delay the Depot Imposes 693
9.6.1. The Depot Is a Single Location 693
9.6.2. From Backorders to a Wait 694
9.6.3. What a Unit at the Depot Buys 695
9.6.4. Where This Leaves Us 698
9.7. METRIC 698
9.7.1. The Procedure 699
9.7.2. What METRIC Approximates 701
9.7.3. Where This Leaves Us 703
9.8. VARI-METRIC 704
9.8.1. Splitting the Depot's Backorders 704
9.8.2. The Pipeline's Two Moments 705
9.8.3. The Depot's Backorder Variance 706
9.8.4. Fitting a Distribution to Two Moments 707
9.8.5. The Procedure 711
9.8.6. Where This Leaves Us 714
9.9. Spending a Budget Across Items 714
9.9.1. What Is Being Chosen, and Against What 714
9.9.2. Marginal Analysis 715

9.9.3. Two Echelons Break the Convexity 716
9.9.4. Flushout 718
9.9.5. Across Items 719
9.9.6. The Procedure 723
9.9.7. Where This Leaves Us 724
9.10. Building These Models in a Worksheet 725
9.10.1. The Depot 725
9.10.2. One Storeroom 726
9.10.3. The Allocation 730
9.10.4. The Ledger 732
9.10.5. Solving a Problem on the Sheets 733
9.10.6. What the Worksheet Cannot Do 736
9.11. Designing the Software 736
9.11.1. What the Software Must Do 737
9.11.2. Finding the Nouns 737
9.11.3. What Each Thing Knows and Does 739
9.11.4. The Classes 740
9.11.5. What the Library Already Supplies 742
9.11.6. Choosing a Family 743
9.11.7. One Class, Two Models 744
9.11.8. Two Searches, Only One of Them Greedy 745
9.11.9. Pricing the Budget Instead of Enforcing It 746
9.11.10. Running the Chapter's Examples 749
9.11.11. Where This Leaves Us 750
9.12. Summary 751
9.13. Exercises 754
9.13.1. Working the Model by Hand 755
9.13.2. Using the Software 758
9.13.3. Using the Workbook 759
9.13.4. Reading the Literature 760

10. Multi-Echelon Systems with Batch Ordering 761
10.1. What Batching Changes 762
10.1.1. Where This Leaves Us 763

10.2. The Item and the Network 763
10.2.1. Where This Leaves Us 765
10.3. Working Two Locations by Hand 765
10.3.1. The System and the Rules 765
10.3.2. The Ledger 766
10.3.3. What the Ledger Establishes 781
10.3.4. Where This Leaves Us 781
10.4. The Hub Is a Chapter 8 Problem 782
10.4.1. Where This Leaves Us 783
10.5. The Delay the Hub Imposes 784
10.5.1. The Mean 784
10.5.2. The Distribution 784
10.5.3. The Second Moment 786
10.5.4. Where This Leaves Us 786
10.6. Exact Storeroom Measures 786
10.6.1. The One-for-One Recursion 786
10.6.2. The Hub Batches 787
10.6.3. Three Routes to a Storeroom's Lead Time Demand . 788
10.6.4. Where This Leaves Us 790
10.7. When the Storerooms Batch Too 790
10.7.1. One Storeroom 790
10.7.2. Several Storerooms 791
10.7.3. Where Exactness Ends 792
10.7.4. Where This Leaves Us 793
10.8. Orders in a Window 793
10.8.1. Working a Batching Storeroom by Hand 795
10.8.2. Where This Leaves Us 804
10.9. The Hub With Other Demand 804
10.9.1. A Two-Moment Recipe 805
10.9.2. Where This Leaves Us 807
10.10. The Storeroom Behind a Batching Hub 807
10.10.1. The Order's Wait 807
10.10.2. Where This Leaves Us 809

10.11 Random Lead Times 809
10.11.1 Where This Leaves Us 811
10.12 Setting Both Policies 811
10.12.1 What Each Location Minimizes 811
10.12.2 The Iteration 812
10.12.3 Where This Leaves Us 815
10.13 Validating the Iteration 815
10.13.1 Where This Leaves Us 816
10.14 The Cutout in Job Lots 816
10.14.1 Where This Leaves Us 818
10.15 Building These Models in a Worksheet 818
10.15.1 Part I on the Sheets 818
10.15.2 Part II on the Sheets 821
10.15.3 Part III on the Sheets 823
10.15.4 Solving a Problem on the Sheets 824
10.15.5 Where This Leaves Us 825
10.16 Designing the Software 826
10.16.1 What the Software Must Do 826
10.16.2 The Classes 826
10.16.3 Running the Chapter's Examples 828
10.16.4 Where This Leaves Us 831
10.17 Summary 831
10.18 Exercises 833
10.18.1 Working the Model by Hand 834
10.18.2 Using the Software 838
10.18.3 Using the Workbook 839
10.18.4 Reading the Literature 840

11. Simulating Inventory Systems **842**
11.1. When the Formulas Stop 842
11.1.1. Three Routes to One Number 843
11.1.2. Where the Exact Route Ends 844
11.1.3. What a Simulated Answer Costs 845
11.1.4. What This Chapter Assumes 845

11.2. The Inventory Processes as Events 846
11.2.1. What Changes the State 846
11.2.2. Averages Over Time, Accumulated at Events 847
11.2.3. The Storeroom's Thirteen Weeks as Events 849
11.3. Building the (r, Q) Model 856
11.3.1. Where This Leaves Us 860
11.4. Output Analysis for a Stock Point 860
11.4.1. A Stock Point Never Terminates 860
11.4.2. Where to Start the Run 861
11.4.3. Replications and Their Length 862
11.4.4. Three Fill Rates, and How to Average Each 863
11.4.5. How Many Replications 864
11.4.6. The Measure That Needs the Longest Run 865
11.5. Where the (r, Q) Formulas Stop 866
11.5.1. Orders That Overtake One Another 867
11.5.2. Job Lots and the (s, S) Policy 870
11.5.3. Demand That Is Lost 873
11.5.4. Where This Leaves Us 875
11.6. Periodic Review 876
11.6.1. Where This Leaves Us 881
11.7. Comparing and Choosing Policies 881
11.7.1. Common Random Numbers 882
11.7.2. Choosing the Best 882
11.7.3. Searching for a Policy 885
11.7.4. Where This Leaves Us 886
11.8. Multi-Echelon Systems 886
11.8.1. Where This Leaves Us 894
11.9. The KSL Supply Chain Package 894
11.9.1. What the Package Provides 895
11.9.2. The Package's Names for the Book's Quantities . . . 895
11.9.3. What the Package Does Not Do Yet 900
11.9.4. Where This Leaves Us 900
11.10Designing the Software 901
11.10.1.What the Software Must Do 901

11.10.2.The Nouns 902
11.10.3.The Classes 903
11.10.4.The Two Routes Must Agree 905
11.10.5.Running the Chapter's Examples 906
11.10.6.Where This Leaves Us 908
11.11Summary 909
11.12Exercises 912
11.12.1.Working the Model by Hand 912
11.12.2.Using the Software 914
11.12.3.Reading the Literature 920

Appendices **921**

A. Notation **921**
A.1. Conventions 921
A.2. State Variables 922
A.3. Demand and Time 923
A.4. Cost Parameters 925
A.5. Performance Measures 926
A.6. Policy Parameters 928
A.7. Demand Classification and Record Quantities 928
A.8. Deterministic Lot Sizing Quantities 930
A.9. Single Period Quantities 931
A.10.Continuous and Periodic Review Quantities 933
A.11.Multi-Item Quantities 935
A.12.Multi-Echelon Quantities 938
A.13.Repairable Item Quantities 941
A.14.Dynamic Lot Sizing Quantities 945
A.15.Requirements Planning Quantities 949
A.16.Distribution and Loss Function Quantities 953
A.17.Stochastic Process Quantities 956
A.18.Symbols That Carry More Than One Meaning 958
A.19.A Note on Other Sources 958

B. Probability and Stochastic Processes Review **960**
B.1. Random Variables and Expectation 961
B.1.1. Expectation, Variance, and Two Ratios 961
B.1.2. A Function of a Random Variable 962
B.1.3. Conditioning 964
B.2. Sums of Random Variables 965
B.2.1. The Mean and the Variance of a Sum 965
B.2.2. Averages, and Why Precision Is Expensive 967
B.2.3. Families Closed Under Addition 967
B.2.4. The Central Limit Theorem, and What It Does Not Say 968
B.3. The Poisson Process 970
B.3.1. Interarrival Times 971
B.3.2. Superposition, Thinning, and Conditioning 971
B.3.3. Palm's Theorem 972
B.3.4. PASTA 973
B.3.5. Demand Over a Random Lead Time 974
B.4. Renewal Processes 976
B.4.1. How Many Events, and How Variable 976
B.4.2. A Random Horizon 978
B.4.3. Little's Law 978
B.4.4. Demands That Arrive in Batches 979
B.5. Random Sums 980
B.5.1. The Mean 981
B.5.2. The Variance 981
B.5.3. Lead Time Demand 982
B.5.4. Batches, and the Compound Poisson 983
B.5.5. What This Does and Does Not Give You 984

C. Common Probability Distributions **985**
C.1. Discrete Distributions 985
C.1.1. The Poisson Distribution 986
C.1.2. The Geometric and Negative Binomial Distributions 986
C.1.3. The Binomial Distribution 987

C.1.4. The Discrete Empirical Distribution 988
C.2. Continuous Distributions 988
C.2.1. The Exponential Distribution 989
C.2.2. The Gamma and Erlang Distributions 989
C.2.3. The Normal Distribution 990
C.2.4. The Lognormal Distribution 991
C.3. Loss Functions 992
C.3.1. Discrete Random Variables 993
C.3.2. The Discrete Empirical Distribution 994
C.3.3. Continuous Random Variables 995
C.3.4. The Standard Normal Case 996
C.3.5. Loss Functions for Common Distributions 998
C.3.6. The Third Order Loss Function 1000
C.3.7. Closed Forms for the Third Order 1002
C.3.8. Mixtures 1004
C.3.9. Computing Them 1005
C.4. Choosing a Demand Distribution 1006
C.4.1. Fitting to a Sample 1006
C.4.2. Matching the First Two Moments 1008
C.4.3. When the Demand Is Intermittent 1013

D. Statistical Tables **1014**
D.1. Standard Normal Distribution 1014
D.2. Standard Normal Loss Functions 1016
D.3. Cumulative Poisson Probabilities 1019
D.4. Poisson Loss Functions 1020

E. Kotlin and KSL Setup **1024**
E.1. What You Need 1024
E.2. Getting the Course Code Project 1025
E.3. What Is in It 1025
E.4. Running Your First Model 1026
E.5. Adding Your Own Model 1027
E.6. The KSL Itself 1028

References **1029**

Preface

This book is intended as a textbook for an upper-level undergraduate or introductory graduate course on the modeling and analysis of inventory systems. It is written for engineering students, primarily industrial engineering, but is also suitable for students in operations management, supply chain management, and data science who have the necessary background in probability and statistics.

The material assumes that you have had a calculus sequence and a first course in probability and statistics. Familiarity with basic optimization concepts is helpful but not required; the optimization needed for the models developed here is introduced as it is used.

Approach

Inventory theory sits at the intersection of stochastic modeling and optimization. The organizing question throughout this book is simple to state and difficult to answer well:

How much should we order, and when should we order it?

Every model developed here is an answer to that question under a particular set of assumptions about demand, lead times, cost structure, and the review process. The emphasis is on understanding **which assumptions produce which policy**, and on what happens when the assumptions fail. Analytical models are developed first; simulation is then used to study systems that

resist closed-form analysis and to check how far the analytical results survive their assumptions.

How this book is organized

The development moves from deterministic to stochastic, and from single-location to multi-echelon systems:

- **Foundations**: the structure of inventory systems, the costs and service measures used to evaluate them, and the management of items and records.
- **Deterministic models**: lot sizing when demand is known, for one item and for many, for constant demand and for time-varying requirements, and requirements planning across a product structure.
- **Stochastic models**: single-period decisions, continuous- and periodic-review policies for repeated ordering under uncertainty, and repairable items circulating between two echelons.

Each chapter closes with exercises. Some are analytical, some require computation in a worksheet or in Kotlin, and one asks you to check a closed form by Monte Carlo simulation.

Software

Two tools carry the computational work in this book, and each is used where it is the right instrument.

Spreadsheets handle the numerical models. Most of what this book asks you to compute, such as a lot sizing schedule, an MRP record, or a policy's expected cost, is naturally tabular and time-phased, and a spreadsheet shows the whole calculation at once in a way that a printed formula cannot. Worked spreadsheets accompany the chapters, and several exercises ask you

to extend one. You are expected to be comfortable building and auditing a spreadsheet model.

Kotlin and the Kotlin Simulation Library (KSL) handle the simulation work, where a system's behavior has to be generated rather than solved for. Appendix E covers installing the tools and running your first model. The KSL is developed and documented separately; see *Simulation Modeling using the KSL* for a full treatment of the library.

Some figures and tables in the text are generated programmatically so that they stay consistent with the numbers around them. That tooling is an authoring convenience and is not shown, taught, or required of you.

A note on notation

Inventory theory has an unfortunate amount of notational variation across the literature. This book fixes a single notation and uses it consistently; where a symbol differs from the common usage in a widely cited source, the difference is noted so that you can move between this text and the literature without confusion.

How this book was written

This book began as course notes. I have taught inventory theory to industrial engineering students for more than thirty years, most recently as *Analysis of Inventory Systems* at the University of Arkansas, and over that time the notes accumulated lecture slides, worked spreadsheets, homework sets, exams, and the results of a long series of funded research projects on inventory and supply chain problems. The raw material was plentiful. What it lacked was a single notation, a single running example, and a consistent standard of proof that a number printed on the page is the number the model actually produces.

How this book was written

The book is written in Quarto. Each chapter is a Markdown file with mathematics in LaTeX, and one source renders to both the website and the PDF. The whole book, including its code, workbooks and build scripts, lives in a version-controlled repository, so every change to a chapter is recorded alongside the computation that supports it.

Every number is computed, not typed. The Kotlin code that accompanies the book, built on the KSL, is a Gradle project with automated tests, and the tests assert the values the chapters print. The spreadsheets are generated by scripts and then recalculated and checked against the same Kotlin results, so a worksheet and the code must agree before either is published. In addition, a set of scripts checks the cross-references, the citations, the spelling, and the consistency of the prose with a written style guide on every build. Thus, when an example, a figure, or an exercise solution quotes a value, some piece of code produced it and some test will fail if it changes.

I wrote this book with the help of an AI assistant, Claude, developed by Anthropic. That is, I used it the way I would use a fast and patient research assistant. It turned decades of notes into draft sections under the style guide derived from my earlier book; it wrote and tested much of the companion code and the scripts that build the workbooks; it worked exercise solutions and pinned their numbers in code; and it read the papers the book cites, so that each citation could be checked against what its source actually says. For example, the exact two-echelon results of Chapter 10 were reproduced from the published tables before any approximation was measured against them. The decisions were mine: what the book covers, in what order, with which models and which examples, and what a student should be able to do at the end of each chapter. I reviewed every section, and the assistant's work was held to the same standard as my own, which is that a claim is accepted when a computation confirms it. Any errors that remain are my responsibility.

Acknowledgments

Much of what this book knows about inventory systems in practice was learned on sponsored projects, and I am grateful to the organizations that posed the problems and funded the work. The National Science Foundation supported this work directly and, through its Industry/University Cooperative Research Centers program, through the Center for Engineering Logistics and Distribution (CELDi), which I had the privilege of directing at the University of Arkansas. The Defense Logistics Agency sponsored more than a decade of projects on inventory record accuracy, sourcing, lead time variability, item classification, bulk petroleum supply chains, and multi-echelon inventory modeling. The Naval Supply Systems Command, the Air Force Research Laboratory, and the Air Force Office of Scientific Research supported work on readiness-based sparing, spare parts segmentation, stockage policy, and forecasting for military logistics, and the Pine Bluff Arsenal and the Red River Army Depot supported work on materials management and depot logistics. The Arkansas Science and Technology Authority funded early work on the simulation of supply chain networks. In industry, Walmart, Invistics, Covidien, and Medtronic brought problems in inventory accuracy, intermittent demand, service levels, continuous replenishment, and healthcare supply chains, and the Center for Innovation in Healthcare Logistics at the University of Arkansas supported work on the healthcare supply chain.

Many of the models in this book were first worked out, tested, or taken apart by my students. I thank the doctoral students whose dissertations on inventory and supply chain systems I chaired: Mohammad H. Al-Rifai, Vijith Varghese, Yasin Unlu, Mohammad Shbool, Anvar Abaydulla, Payam Parsa, and Alireza Sheikh-Zadeh; and those on whose dissertation committees I served: Ghazi Magableh, Mehmet Miman, and Yisha Xiang. I thank the students whose master's theses on these problems I chaired: Yeu-San Tee, Ashish Achlerkar, Vijith Varghese, Vikram Desai, Seda Gumrukcu, Yanchao Liu, Server Apras, and Tanvir Sattar; those on whose thesis committees I served: James Oldham, Derek Malstrom, Yisha Xiang,

Ronald Walker, and John Sophabamixay; the students whose master's projects I directed: Ravi Kurgund, Hin-Tat Chan, Soncy Thomas, Kiran Chittoori, Josh McGee, and Amit Bhonsle; David Cox, whose honors thesis I chaired; and Julianna Bright, for research on (r, Q) optimization and bulk petroleum supply chains. Their questions made the notes better long before the notes became a book.

License

The online version of this book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License. The Kotlin code that accompanies it, in the `code` project beside the book, is licensed under the GNU General Public License, version 3, the same license as the KSL itself. To cite this book:

> Rossetti, M.D. (2026). *Analysis of Inventory Systems*, Online and Open Text Edition. Retrieved from https://rossetti.github.io/RossettiInventoryBook/, licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

About the Author

Dr. Manuel Rossetti, P.E. Dr. Rossetti is a University Professor of Industrial Engineering at the University of Arkansas. He served as the inaugural Director for the University of Arkansas Data Science program from 2019-2024. In addition, Dr. Rossetti served as the Director for the NSF I/UCRC Center for Excellence in Logistics and Distribution (CELDi) from 2013-2020. Dr. Rossetti has published over 125 journal and conference articles in the areas of simulation, logistics/inventory, and healthcare and has been the PI or Co-PI on funded research projects totaling over 6.4 million dollars. He was selected as a Lilly Teaching Fellow in 1997/98 and was voted Best IE Teacher by IE students in 2007, 2009, and 2017. He won the IE Department Outstanding Teacher Award in 2001-02, 2007-08, and 2010-11. He received the College of Engineering Imhoff Teaching Award in 2012 and was elected an IIE Fellow. In 2013, the UA Alumni Association awarded Dr. Rossetti the Charles and Nadine Baum Faculty Teaching Award, the highest award for teaching at the university. In 2015, Dr. Rossetti served as Program Chair for the Winter Simulation Conference. In 2024, Dr. Rossetti served as the General Chair for the Winter Simulation Conference. He is also the author of the book, Simulation Modeling and Arena, published by John Wiley & Sons.

Dr. Rossetti grew up in Canton, Ohio and is an ardent Cleveland sports fan. He received his Ph.D. and MSIE degrees in Industrial and Systems Engineering from The Ohio State University and his BSIE degree from the University of Cincinnati. He is a registered professional engineer in the State of Arkansas.

1. Introduction to Inventory Systems

Learning objectives

After reading this chapter you should be able to:

- explain why organizations hold inventory, and what it costs them
- classify an inventory system by its demand, review, and lead-time characteristics
- define the state variables of an inventory system (on hand, on order, backordered, inventory position) and the time averages built from them
- define the cost parameters used by inventory models, and state what each one multiplies
- decide which costs belong in a given parameter and reject those that do not
- estimate each parameter from standard accounting and transaction data
- distinguish the cost and service measures used to evaluate a policy
- state the general problem that the rest of the book solves

1.1. Why Hold Inventory?

Inventory is capital converted into a form that cannot be spent. An organization holding forty million dollars of stock has forty million dollars it cannot use for anything else, sitting in a building it must heat and insure, exposed to theft, damage, and the possibility that nobody will ever want it. Held long enough, some of it will be written off. Thus, on any ordinary accounting of costs, inventory looks like a mistake.

It is not a mistake. Inventory exists to **decouple**, i.e. to break a linkage between two processes that would otherwise have to operate in lockstep. Supply and demand are separated in space, in time, in rate, and in predictability, and each separation forces stock into existence somewhere. Take the inventory away without removing the separation and the two processes must synchronize, which usually means one of them stops.

Each kind of separation creates a distinct kind of inventory, with a distinct cause and therefore a distinct lever for reducing it. Confusing them is an expensive error. In the sections that follow we take the five kinds in turn.

1.1.1. Pipeline Inventory

Supply and demand are separated in **space and time**. Units ordered from a supplier do not arrive at the loading dock the instant they are ordered. They spend the lead time in transit, in production, or awaiting inspection, and during that time they are owned, paid for, and unavailable. Those units are *pipeline* or *in-transit* inventory.

Notice that its size is not a decision. Equation 1.5 gives $\overline{IO} = \lambda \overline{L}$, i.e. the demand rate times the average lead time and nothing else. For example, an organization selling a hundred units a week from a supplier six weeks away owns six hundred units in transit at all times, whatever its reorder point. The only ways to change that number are to sell less or to source closer.

1.1.2. Cycle Stock

Supply and demand are separated in **rate**. Consumption is more or less continuous, while replenishment arrives in discrete lots. Between one delivery and the next, the stock drawn down is inventory that exists only because the arrival was a lot and not a trickle. That is *cycle* inventory, and it averages half the lot size.

It exists because replenishing is not free. Something is incurred once per order regardless of size, e.g. the administrative cost k of Section 1.4.5, a production setup, or a truck that costs the same whether half full or full. That fixed charge makes ordering in batches cheaper than ordering continuously. Cycle inventory is the price paid to avoid it.

The lever is the fixed cost, not the ordering rule. Electronic ordering, blanket contracts, and setup time reduction all attack cycle stock at its cause. Instructing the buyer to order less at a time does not, because it only moves cost from holding into ordering.

1.1.3. Safety Inventory

Supply and demand are separated in **predictability**. Demand over a lead time is not known in advance, and neither, often, is the lead time. Stock held against that uncertainty, beyond what average demand over the average lead time requires, is *safety* inventory.

Safety stock is created by **variability, not by level**. If demand were a steady hundred units a week and the lead time exactly six weeks every time, then no safety stock would be needed at any service target, however demanding, because the reorder point could be set exactly. That is, it is the variance, not the mean, that has to be covered.

It is also the only kind of inventory that is a direct purchase of service. The others buy transportation or production economies, while safety stock

buys the probability of being able to fill a demand, which Section 1.6 measures.

1.1.4. Anticipation Inventory

Supply and demand are separated in **rate over a horizon**, in a way that is known in advance. For example, a plant with fixed capacity facing a seasonal peak must build ahead of the peak or fail to meet it, and a distributor expecting a promotion, a price increase, a plant shutdown, or a holiday closure stocks up beforehand.

What separates anticipation from safety inventory is that the variation is *foreseen*. That is, it is covered not because it might happen but because it will. Thus, the lever is capacity or flexibility, not the replenishment policy, and the decision is usually made outside the inventory system.

1.1.5. Speculative Inventory

Speculative inventory is held because the item is expected to cost more later, or to become unavailable. For example, a hedge against a commodity price, a buy-ahead against an announced increase, or a strategic reserve against a supply interruption.

This is a financial position denominated in goods, governed by a view about future prices, not by any mechanics in this book. It appears here because it turns up on real balance sheets, where it can dominate the operational categories and distort any attempt to benchmark them.

1.1.6. Different Causes, Different Levers

Table 1.1 collects the distinctions. Reading across a row gives the cause of that kind of inventory and the intervention that follows from it. Notice that no two rows share an entry in the last column.

Table 1.1.: The kinds of inventory, distinguished by what creates them. Because the causes differ, so do the interventions that work.

Type	What it decouples	Created by	Reduced by
Pipeline	Supply and demand in space and time	Lead time	Shorter lead time; closer sourcing
Cycle	Continuous consumption from discrete replenishment	A fixed cost per order or per setup	Cheaper ordering; faster setups
Safety	The system from uncertainty	Variability in demand and lead time	Less variability; shorter lead time; less service
Anticipation	Fixed capacity from foreseen variation in demand	A capacity constraint	More or more flexible capacity
Speculative	The purchase decision from the price	An expected price or supply movement	A change of view about the future

A directive to reduce inventory by twenty percent is not actionable, because it does not say which kind. Each of the five responds to a different intervention, most of those interventions take months, and exactly one of them, reducing safety stock, can be executed by decree.

That asymmetry makes the unspecified mandate dangerous. Faced with a target and a deadline, the only lever available on the deadline is the safety stock. So that is the lever that gets pulled, and inventory falls on schedule. The consequence arrives one lead time later as stockouts, by which time it

is attributed to demand, to suppliers, or to bad luck. The mandate was met, and the service was spent to meet it.

Figure 1.1 shows the same point for a representative item, using results derived in Chapter 3 and Chapter 8. Each bar is the total stock remaining after one lever has been pulled, with the components stacked so that the pipeline, cycle, and safety contributions can be read separately. Notice that halving the fixed order cost moves only the cycle component, and that halving the variability of demand moves only the safety component. Halving the lead time moves two of the three at once.

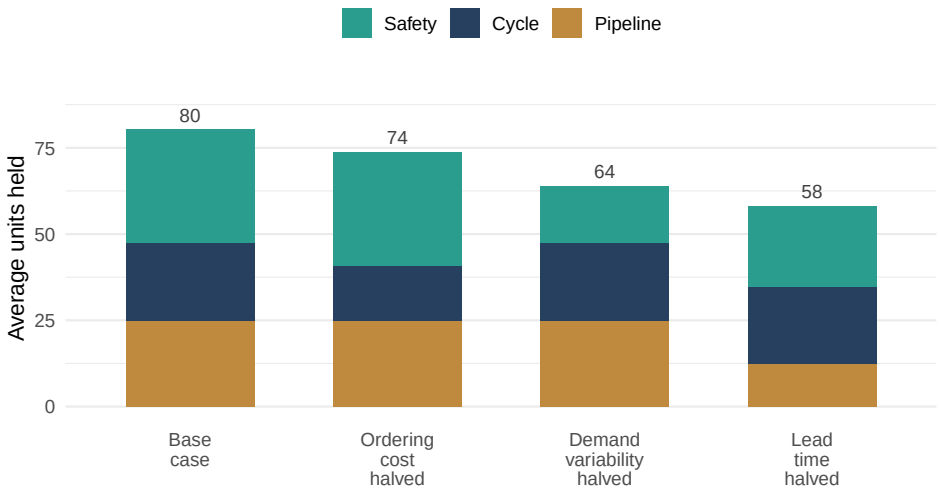


Figure 1.1.: Average inventory for a representative item, decomposed by cause, under three interventions. Cheaper ordering reduces only cycle stock; less variable demand reduces only safety stock (drawn here with a normal safety factor for illustration only; see Section 8.7.7); a shorter lead time reduces pipeline stock and safety stock at once, and so achieves the largest total reduction without any loss of service.

That is why lead time wins. It is the only one of the three levers that appears in two components, directly in the pipeline and through the

exposure window in the safety stock. Thus, lead time reduction is the lever most often underrated in inventory management.

1.1.7. The Cost of Not Holding It

All of this reads as an argument that inventory is a symptom of problems elsewhere, e.g. long lead times, expensive setups, or unpredictable demand. That much is correct. However, the conclusion sometimes drawn from it, that the right target is therefore zero, does not follow.

Inventory is not waste. It is a **purchase**, and what it buys is the ability to serve a demand from stock committed before anyone knew the demand would come. Thus, removing it without removing what created it converts a holding cost into a shortage cost, and shortage costs are the ones that appear on no ledger (Section 1.4.6).

The question is never whether to hold inventory. It is how much, and when to replenish, given what holding costs and what not holding costs. Every model in this book answers that pair of questions under a particular set of assumptions about demand, lead times, costs, and review. The assumptions change from chapter to chapter. The question does not. Section 1.2 sets out the dimensions along which they change.

1.2. Classifying Inventory Systems

There is no single inventory model, and there could not be. For example, a hospital stocking a drug that expires, a distributor replenishing a fast moving consumable from a supplier three weeks away, and a manufacturer scheduling components against next quarter's build plan are all managing inventory, and almost nothing carries from one to the next. What they share is the pair of questions from Section 1.1, how much and when. They do not share the machinery that answers them.

Classification is how you find the machinery. Each dimension below is a question whose answer eliminates some models and selects others, and the chapters of this book are organized around the answers. Notice that most modeling errors in practice are not arithmetic errors. They are a model applied to a system it does not fit.

> **A property of the system, not of the model**
>
> These are characteristics of the real system, in the sense of Section 1.3. For example, a supplier's lead time has a distribution whether or not your model represents it as a constant. Choosing to treat it as constant is an approximation you make deliberately, having judged that the decision at hand is insensitive to the difference. It is not a discovery that the lead time is constant.

1.2.1. Where the Demand Comes From

The first question, because it governs whether anything else in this book applies.

Independent demand arises outside the system. End customers want the item for its own sake, and their wanting it is not deducible from anything the organization has scheduled. It must be observed, characterized, and, where it is uncertain, described by a probability distribution.

Dependent demand is implied. Demand for a component follows from the production schedule for the assembly that consumes it, and that schedule is already known. Such demand should be *calculated*, not forecast; forecasting it independently discards information the organization already has. Chapter 6 is about the calculation.

Every other dimension below concerns independent demand, so this question comes first. Thus, if you answer it "dependent" the correct treatment is requirements planning, regardless of how the remaining dimensions fall.

1.2.2. Known or Uncertain Demand

This is the dividing line of the book, and the reason it has two parts.

Deterministic demand is known in advance. Known does not mean constant: demand may vary from period to period and still be deterministic, provided the variation is known before the decisions are made. It means no probability distribution is needed.

Stochastic demand is uncertain and must be described by a distribution. The question then changes character, from "what will demand be" to "how much protection against what it might be," and every result acquires a service dimension it did not have before.

No real demand is deterministic. We use deterministic models anyway, and legitimately, in two situations: when variability is small relative to the decision, so ignoring it changes nothing that matters, and when the model sits inside a larger deterministic plan, as lot sizing does inside a material requirements plan. Thus, the justification is the quality of the approximation, never a claim that demand is actually known.

1.2.3. Constant or Time-Varying Demand

Within either case, demand may proceed at a steady rate or change over the horizon.

A **constant** rate permits a single order quantity, repeated indefinitely. That is the setting of Chapter 3. A **time-varying** rate destroys that: with demand of 10 units this month and 400 next, no single lot size is sensible, and the problem becomes choosing a schedule of order sizes and timings. Chapter 5 treats the deterministic time-varying case.

The stochastic analogue separates **stationary** demand, whose distribution does not change over time, from **non-stationary** demand, whose does. Seasonality, product life cycles, and promotions all produce non-stationarity,

and it is substantially harder. The policies of Chapter 8 assume stationarity, and the usual treatment of a seasonal item is to re-solve the stationary problem periodically with updated parameters, and not to pretend the assumption holds.

1.2.4. How the System Is Reviewed

Continuous review means the inventory position is known at all times and an order may be placed at any instant. The policies of Chapter 8 trigger on the position crossing a threshold, which requires knowing the moment it crosses.

Periodic review means the position is observed at fixed intervals, and orders may be placed only at those moments. Section 8.13 develops the corresponding policies, in which the review interval becomes a parameter and adds its own exposure to risk: between reviews, the system cannot respond to anything.

Historically this was a technology constraint, since continuous review required a perpetual inventory record that manual systems could not maintain. That constraint is largely gone, and the distinction survives for a better reason. Review is often *logistically* periodic even when it is informationally continuous: if a truck calls weekly, knowing the position continuously confers no ability to order continuously. Classify by when an order can be placed, not by when the position can be seen.

1.2.5. The Lead Time

The interval between placing a replenishment order and having it available to satisfy demand.

A **zero** lead time is a simplification rather than an approximation. With replenishment instantaneous there is no exposure between ordering and

receiving, so no reorder point is needed and no safety stock. Several classical results assume it.

A **constant** lead time is the usual working assumption, and it makes the lead-time demand distribution of Chapter 8 tractable.

A **random** lead time compounds with demand uncertainty, since what matters is demand over an interval that is itself uncertain. That is, both the rate and the length of the exposure are unknown when the order is placed. It also raises a possibility usually assumed away: **order crossing**, where an order placed later arrives earlier. Most models assume orders arrive in the sequence placed. That is harmless when lead-time variability is modest relative to the time between orders, and not harmless for a system ordering frequently from a source with erratic transit.

1.2.6. What Happens to Unmet Demand

Whether a demand arriving to an empty shelf waits or departs, discussed as a state distinction in Section 1.3.2.

Under **backordering** the demand waits and is filled later, so the system carries an obligation, $B(t)$, and pays for the waiting through b. Under **lost sales** the demand departs, nothing is owed, and the failure is charged once through π. **Partial backordering**, where some customers wait and others do not, is the common reality and is usually modeled as one of the two extremes with the choice stated.

This is not only a costing convention. It changes what the system can do: under lost sales net inventory can never go negative, and the state description is smaller.

1.2.7. The Planning Horizon

A **single-period** problem is decided once. Stock is committed before demand is observed, demand occurs, and whatever is left has no further use. This is the newsvendor setting of Chapter 7, and it is not the multi-period problem with the number of periods set to one. Leftover stock has no future value, so the trade-off is purely one of overage against underage.

A **finite-horizon** problem has an end date that matters: a service part for equipment being retired, a component for a product with an announced end of life. The last order is a newsvendor problem in disguise.

An **infinite-horizon** problem assumes the system continues indefinitely, which licenses long-run averages as the objective. It is the setting of Chapter 8, and the assumption is usually harmless: what it requires is a horizon long relative to the replenishment cycle.

1.2.8. One Item or Many

If items are independent, a multi-item problem is many single-item problems and nothing is gained by treating them together.

Items become **coupled** in two ways. A **shared constraint** (a budget, a storage capacity, a supplier's minimum) means one item's order quantity restricts another's, so the items must be solved jointly. A **shared setup** (several items on one purchase order or one truck) means ordering them together costs less than ordering them separately, which is the joint replenishment problem. Chapter 4 treats both.

Most organizations manage tens of thousands of items and cannot make a considered decision about each. Classification by value and volume, with policies applied by class instead of by item, is how that is handled, and Section 2.2 treats it.

1.2.9. One Location or Several

A **single stocking point** faces external demand and an external supplier.

A **multi-echelon** system has stocking points that supply each other: a central warehouse replenishing branches, a depot supplying bases. This is not several single-location problems, and treating it as such is a standard and expensive error. The demand a warehouse sees is not customer demand. It is the *replenishment orders* of the locations it serves, which are shaped by those locations' own ordering policies and, when those locations order in batches, are lumpier than the demand underneath them. Stock held at one level substitutes for stock at another, so the levels cannot be optimized independently. Chapter 9 takes this up for locations that order one unit at a time.

1.2.10. The Dimensions Together

Table 1.2 collects the questions and where each answer is treated.

Table 1.2.: The dimensions along which inventory systems differ, and where each is treated.

Dimension	Alternatives	Where treated
Origin of demand	Independent / dependent	Chapter 6 for dependent
Certainty of demand	Deterministic / stochastic	Part I / Part II
Behavior over time	Constant / time-varying; stationary / non-stationary	Chapter 3, Chapter 5
Review	Continuous / periodic	Chapter 8, Section 8.13

Dimension	Alternatives	Where treated
Lead time	Zero / constant / random	Chapter 8
Unmet demand	Backordered / lost / partial	Section 1.3.2
Horizon	Single period / finite / infinite	Chapter 7; Chapter 8
Items	One / many independent / many coupled	Chapter 4
Locations	One / multi-echelon	Chapter 9

Read Table 1.2 as a questionnaire, not as a taxonomy. The left column is the question you ask of the system in front of you, the middle column is the set of answers the question admits, and the right column is where this book takes that answer up. Notice that the second row, the certainty of demand, is the only one that splits the book itself. Every row above it and below it recurs on both sides of that split, so review, lead time, and the treatment of unmet demand each appear twice in the book under different assumptions about demand.

Nine questions describe a large number of combinations, and this book does not address all of them. It addresses the ones that occur, and the ones whose answers illuminate the rest.

1.2.11. Which Costs the Classification Implies

Classifying a system also narrows which of the cost parameters of Section 1.4 matter. Estimation effort is expensive and should not be spread evenly.

Table 1.3.: What the classification implies about which cost parameters matter.

If the system is…	The costs that decide the answer are…
Deterministic, no shortages permitted	k and h only. Shortage costs never arise; unit cost enters only through h.
Deterministic, shortages permitted	k, h, and b. The backorder cost sets how deep the system is allowed to run negative.
Single period	The overage and underage costs: c, the salvage value, and π. Ordering cost is irrelevant because there is one order.
Stochastic, backordering	k, h, and b, or k, h and a service target standing in for b.
Stochastic, lost sales	k, h, and π.
Multi-location	All of the above, at every location. Where whether a location stocks the item is itself in question, f as well.

Notice that the first row of Table 1.3 names only two parameters. A deterministic system in which shortages are not permitted never realizes a shortage, so no amount of care spent estimating one changes the answer, and Chapter 3 derives its order quantity from k and h alone. Notice also that π and b never appear in the same row. A demand that is lost is not also backordered, and a model that charges both for the same event has double counted, which Section 1.5.6 takes up. Thus, you should settle the classification before any estimation begins, because it tells you which two or three numbers deserve the effort.

1.2.12. Choosing a Classification

Real systems do not present themselves neatly. Demand is neither purely independent nor purely dependent; some customers wait and some do not; lead times are usually stable and occasionally not. Every classification is therefore a decision, and it should be made on a single criterion:

> **!** The criterion
>
> Choose the simplest classification that preserves what the decision is sensitive to.

Simplicity repays real effort, because a simpler classification yields a model whose behavior can be understood and whose parameters can be estimated. But simplicity that discards what the answer depends on is not simplification. It is an error with a smaller model attached.

The criterion is testable. When it is unclear whether a distinction matters, whether lead-time variability needs representing or the occasional lost sale needs its own treatment, model it both ways and compare the recommendations. If they agree, the comparison is the justification for the simpler classification. If they disagree, the question has answered itself. This is one of the things simulation is for.

1.3. The State of an Inventory System

Before anything can be optimized it has to be described. This section defines the quantities that describe an inventory system at a point in time, and the averages built from them. They are properties of the physical system. That is, an observer with a clipboard, who knows nothing about how replenishment is controlled and nothing about probability, could walk into the warehouse and measure every one of them.

You should insist on that, because it locates what a model is for. The deterministic models of Chapter 3 and the stochastic models of Chapter 8 are not describing different systems, and they do not have different state variables. They are two ways of *computing* the same quantities under different assumptions about what is known. Everything in Section 1.4 and Section 1.6 is expressed in the variables defined here.

1.3.1. On Hand, On Order, and Backordered

Three counts describe the system at time t. Note that a policy does not read these counts from the shelf; it reads them from a record, and the two can differ. Section 2.3 is about that gap and how it is measured and closed.

Let $I(t)$ represent the **inventory on hand** at time t, meaning the units physically present and available to satisfy a demand. Let $IO(t)$ represent the **inventory on order**, meaning the units ordered from the supplier but not yet arrived, the replenishment pipeline of Section 1.1.1. Let $B(t)$ represent the **backorders**, meaning the units demanded but not yet supplied, which the system is still committed to supplying.

All three are counts of units and none of them can be negative. From them we build two composites. The **net inventory**

$$IN(t) \;=\; I(t) - B(t) \tag{1.1}$$

is positive when the system holds stock and negative when it owes units, and it is the single number that summarizes whether the system is ahead or behind. The **inventory position**

$$IP(t) \;=\; I(t) + IO(t) - B(t) \tag{1.2}$$

adds what is already coming.

One structural fact does a great deal of work throughout the book:

> **!** On hand and backordered are never both positive
>
> If $I(t) > 0$ then $B(t) = 0$, and if $B(t) > 0$ then $I(t) = 0$.

A system holding stock while owing units would be one that had received inventory and declined to ship it to a waiting customer. Under the ordinary assumption that arriving replenishments fill outstanding backorders first, that situation never arises. Thus, at every instant at most one of the two is positive, and $IN(t)$ determines both: $I(t) = [IN(t)]^+$ and $B(t) = [IN(t)]^-$.

This is also what makes the cost model of Section 1.4 coherent. The holding cost charges $I(t)$ and the backorder cost charges $B(t)$; because the two are never simultaneously positive, no unit is ever charged both ways, and the total is unambiguous.

1.3.2. Lost Sales

Everything above assumes an unsatisfied demand *waits*. That is one of two possibilities, and the other changes the state description and not merely the arithmetic.

Under **lost sales**, a demand that cannot be filled from stock departs and does not return. Nothing is owed, so

$$B(t) \equiv 0, \qquad IN(t) = I(t) \geq 0, \qquad IP(t) = I(t) + IO(t)$$

for all t. The state collapses to two counts, net inventory can never go negative, and the backorder cost b has nothing to multiply.

What takes its place is the **rate of lost demand**. Let λ_ℓ represent the rate, in units per unit time, at which demand arrives and departs unfilled,

and let λ represent the overall demand rate in the same units. Since a demand is lost exactly when it arrives to an empty shelf,

$$\lambda_\ell \;=\; \lambda\left(1 - \overline{FR}_u\right) \tag{1.3}$$

where $\overline{FR}_u$ is the fraction of demanded units filled from stock, defined in Section 1.6. This is the quantity the stockout cost π multiplies in Table 1.4, and writing it out shows why that term is a genuine rate while the backorder term is a level. That is, lost demand is a flow of units past the system, whereas backorders are a population sitting inside it.

The same quantity exists under backordering, since demand arrives to an empty shelf there too, but it is not what the cost model charges. There the harm is the waiting, not the loss.

Which regime applies is a fact about the system, not a modeling convenience. For example, a distributor whose customers will wait a week for a spare part is backordering, while a retail shelf whose customers walk to a competitor is losing sales. Many real systems are partially both, and the usual treatment is to model the dominant case and say so. This book develops the backorder case in detail, because it is the harder of the two and because the lost-sales results follow from setting $B(t) \equiv 0$.

1.3.3. Why Ordering Decisions Use the Inventory Position

Every replenishment policy in this book triggers on $IP(t)$, never on $I(t)$ alone, and you should understand the reason before the policies arrive.

Suppose a system reorders whenever on-hand inventory falls to some level. Stock runs down, the level is reached, an order is placed. The lead time has not yet elapsed, so on-hand inventory keeps falling, and the trigger level is still breached, so the rule places another order, and another, each for a shortfall the first order is already on its way to cover. Thus, by the time the shipments arrive the system is grossly overstocked.

Including $IO(t)$ is what prevents this. Because outstanding orders are counted in $IP(t)$, placing an order raises the position immediately, and the system orders again only when what is already outstanding is no longer enough. The position is what the system will *eventually* have; the on-hand inventory is only what it has now.

Figure 1.2 shows both for a system operating an (r, Q) policy, which orders Q units whenever the position falls to r. Let r represent the reorder point, in units, and let Q represent the fixed order quantity, also in units. The position sawtooths between r and $r + Q$. Net inventory follows the same pattern delayed by the lead time, so net inventory, and not the position, runs short.

Walk across Figure 1.2 from left to right. The dashed line is the inventory position and the solid line is net inventory. Both start high, just after a receipt, and both decline at the demand rate. Notice that the two lines lie on top of each other over the early part of each cycle. That is the stretch during which no order is outstanding, so $IO(t) = 0$ and Equation 1.2 reduces to Equation 1.1. The dashed line then reaches the dotted line at r, an order is placed, and the position jumps by Q to $r + Q$ while net inventory does not move at all, because nothing has arrived. The two lines run apart for exactly one lead time L, marked by the double arrow. Net inventory continues its decline through that interval, crosses zero, and the shaded region below the axis is the backorders $B(t)$ accumulating while the order is in transit. When the order arrives, net inventory jumps by Q, the backorders are filled first, and the cycle repeats. Thus, the reorder point r does not determine when the system runs out. What determines that is r measured against demand over the lead time. Chapter 8 is built on that calculation.

Each state variable is a **time-persistent** quantity: it holds a value over an interval instead of being observed at isolated instants. Thus, its average is a time average, computed by integrating over the observation interval and dividing by its length. Let T represent the length of that interval, in

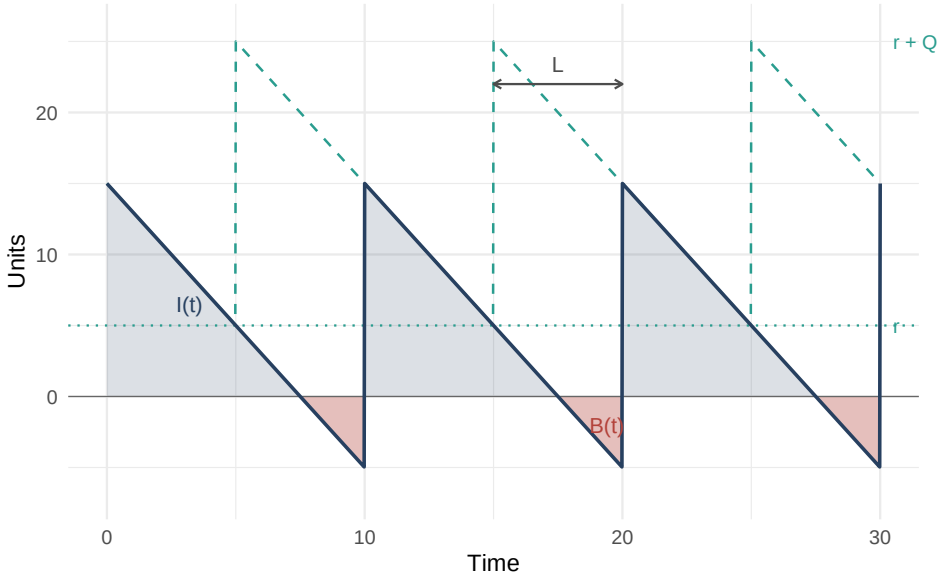


Figure 1.2.: Inventory position and net inventory for a system operating an (r, Q) policy with constant demand. The position (dashed) falls to r, an order is placed, and it jumps by Q; net inventory (solid) repeats the same pattern delayed by the lead time L, which is why it, and not the position, is what runs short. Where net inventory is positive it is inventory on hand $I(t)$; where it is negative its magnitude is the backorders $B(t)$. While no order is outstanding the two curves coincide, because $IO(t) = 0$ there.

the same time units as the demand rate, and let $\bar{I}$ and $\bar{B}$ represent the resulting averages, in units:

$$\bar{I} \;=\; \frac{1}{T}\int_0^T I(t)\,\mathrm{d}t, \qquad \bar{B} \;=\; \frac{1}{T}\int_0^T B(t)\,\mathrm{d}t \tag{1.4}$$

and likewise for $\overline{IO}$, $\overline{IN}$, and $\overline{IP}$. Because integration is linear and $IN(t) = I(t) - B(t)$ at every instant, the averages inherit the same relation, $\overline{IN} = \bar{I} - \bar{B}$. When the system is stationary, these averages converge as T grows to the system's steady-state performance.

These integrals are where the two halves of the book meet. In a deterministic model $I(t)$ is a known piecewise-linear function and the integral is the area of a triangle. For example, Chapter 3 computes $\bar{I} = Q/2$ this way, and the familiar result is nothing more than the area of the sawtooth in Figure 1.2 divided by its base. In a stochastic model the same integral is an expectation, computed from the distribution of the process or estimated from a simulation run. Thus, the quantity being computed does not change; only the method does.

1.3.4. Little's Law and the Averages It Connects

Several relationships hold regardless of the replenishment policy, and all of them are Little's law applied to a different flow of units. Little's law states that for any system in steady state, the average number of items inside equals the arrival rate times the average time each spends there. Thus, choosing what counts as "inside" is what generates the results below.

Pipeline inventory. Units enter the pipeline when ordered and leave it when received, spending the lead time inside. Applying Little's law to that flow,

$$\overline{IO} \;=\; \lambda \overline{L} \tag{1.5}$$

Let $\overline{L}$ represent the average lead time, in the same time units as the demand rate. Average on-order inventory is the demand rate times the average lead time, and nothing else. Notice that no ordering policy appears in Equation 1.5, so no ordering policy can reduce it. You should know this early, because pipeline stock is often mistaken for a target of improvement. It is a consequence of how fast you sell and how long your supplier takes, and it responds only to changing one of those.

Backorders and waiting. Units enter the backorder queue when demanded and unfilled, and leave when supplied, spending the customer's wait inside. The same argument gives

$$\bar{B} \;=\; \lambda \overline{W} \tag{1.6}$$

where $\overline{W}$ represents the average time a demanded unit waits, counting zero for units filled immediately. That is, average backorders and average customer waiting are two views of one quantity. This is why Section 1.5.5 can convert a statement management is willing to make, a tolerable wait, into the quantity that the backorder cost b multiplies.

Stocking time and turnover. Apply the same argument to the stock on the shelf. Units enter when a replenishment is received and leave when they are demanded, so if $\overline{T}_s$ denotes the average time a unit spends on hand,

$$\bar{I} \;=\; \lambda \overline{T}_s \qquad \Longleftrightarrow \qquad \overline{T}_s \;=\; \frac{\bar{I}}{\lambda} \tag{1.7}$$

Units that arrive to fill an outstanding backorder ship immediately and spend no time on the shelf; they enter the average as zeros, and the relation still holds.

Its reciprocal is a quantity every operations manager already knows. The **inventory turnover ratio**

$$TO \;=\; \frac{\lambda}{\bar{I}} \;=\; \frac{1}{\overline{T}_s} \tag{1.8}$$

counts how many times the average stock is consumed and replaced per unit time. Turnover is usually quoted annually and computed in dollars, as cost of goods sold divided by average inventory value. Multiplying numerator and denominator by the unit cost c changes nothing, so the dollar ratio and the unit ratio are the same number.

That equivalence is more useful than it looks. Turnover is reported by almost every organization and understood by people who would not sit through a derivation, and Equation 1.8 says it is nothing more than the reciprocal of the average time a unit sits on a shelf. "We turn this item four times a year" and "a unit sits here three months on average" are the same statement. It also supplies a sanity check available before any modeling begins: an item whose computed $\bar{I}$ implies a turnover far from what the business reports has been mis-specified somewhere.

1.3.5. Stockout Indicators

Whether the system is currently able to fill a demand is itself a quantity we can average. Define the **stockout indicator**

$$SO(t) = \begin{cases} 1 & I(t) \le 0 \\ 0 & I(t) > 0 \end{cases} \tag{1.9}$$

Since $I(t)$ is a count of units on hand it is never negative, so the condition $I(t) \le 0$ is the event $I(t) = 0$; it is written with the inequality because a demand arriving at the instant the shelf empties cannot be filled, and the convention should make that unambiguous.

The **ready indicator** is its complement, $RR(t) = 1 - SO(t)$. Averaging either over time gives a proportion:

$$\overline{SO} \;=\; \frac{1}{T}\int_0^T SO(t)\,\mathrm{d}t \tag{1.10}$$

is the proportion of time the system is out of stock, and $\overline{RR} = 1 - \overline{SO}$ is the proportion of time it has stock on hand. Section 1.6 turns these into service measures and explains why $\overline{RR}$ is *not* interchangeable with the fraction of demand that gets filled, even though the two are frequently reported as though they were.

1.3.6. The Demand Process

Demand arrives as a sequence of **transactions**. Let t_i represent the time at which the ith transaction arrives, and let D_i represent the number of units it requests, a positive integer. Over an interval of length T containing n transactions, the demand rate is

$$\lambda \;=\; \frac{1}{T}\sum_{i=1}^{n} D_i \qquad \text{units per unit time,}$$

while n/T is the **transaction rate**, in transactions per unit time. The two differ by the average transaction size, and keeping them distinct matters more than it first appears. For example, a location seeing 500 units a year in five orders of 100 is a different system from one seeing 500 units in 500 orders of one, even though λ is identical in both. The first will run a large order quantity and spend most of its time either well stocked or badly short; the second will behave far more smoothly.

The distinction also divides the service measures of Section 1.6. Some count units, some count transactions, and on a demand stream with $D_i > 1$ they give different answers about the same history.

The state variables defined here are what the next section prices. Nothing in Section 1.4 introduces a new quantity to be measured; it attaches a rate of money to each of the quantities already defined.

1.4. Cost Parameters

An inventory policy is a rule, and a rule has no cost. What costs money is the *behavior* the rule produces: orders placed at some rate, stock sitting on a shelf for some length of time, demands arriving when the shelf is empty. The purpose of a cost model is to price that behavior, so that two rules can be compared on something other than intuition.

1.4.1. What an Inventory Model Charges

Every model in this book computes a total cost that is a **rate**, dollars per unit time, and every term in it has the same shape:

$$\text{cost rate} \;=\; \text{price} \times \text{quantity the system exhibits} \tag{1.11}$$

The division of labor matters. The quantities on the right, how often orders are placed and how much stock is on hand and how many units are owed, are properties of the system, and computing them is the technical content of the chapters that follow. The *prices* on the left are what you supply, and no amount of modeling will produce them.

Table 1.4 sets out the division for the terms that appear throughout the book. The state variables in its right column are defined in Section 1.3.

Table 1.4.: The two halves of every cost term. Only the left column requires data about your organization; the right column is what the models in this book are for.

Term	Price (you supply)	Quantity (the model computes)
Ordering	k, dollars per order	order rate, orders per unit time
Holding	h, dollars per unit per unit time	average on-hand inventory $\bar{I}$, units
Backorder	b, dollars per unit per unit time	average backorders $\bar{B}$, units
Stockout	π, dollars per unit	rate of demand not filled from stock, units per unit time
Purchase	c, dollars per unit	demand rate λ, units per unit time
Stocking position	f, dollars per item per location per unit time	*none*

Read Table 1.4 one row at a time, checking the units as you go. The ordering row multiplies dollars per order by orders per unit time, and the orders cancel. The holding row multiplies dollars per unit per unit time by a number of units, and the units cancel. Every row in the table reduces to dollars per unit time, so the rows can be added. Notice that the last row has no quantity at all. Three features of the table shape everything that follows.

Some prices are themselves rates. The holding and backorder prices are quoted per unit *per unit time*, so they multiply a **level**, a number of units, and not a rate, and the product is still dollars per unit time. The ordering and stockout prices multiply genuine rates. Thus, this is the commonest

source of dimensional error in a first cost model, and the check is the one just performed: every term must reduce to dollars per unit time.

The purchase term is inert. It is $c\lambda$, and when demand is not itself a decision, which through Chapter 7 it is not, this term is identical under every policy and cannot influence the choice among them. Unit cost still matters, but it matters *through the holding cost*, as we are about to see, not on its own account. Including $c\lambda$ in a comparison is a common error, and its symptom is that the alternatives all look suspiciously alike, because a large constant has been added to both sides.

The stocking position term has no quantity at all. It is charged from the moment an item is carried at a location, identically whether one unit or ten thousand pass through it. That absence is not an oversight. It makes this the parameter that decides marginal cases. See Section 1.4.7.

> **A note on time units**
>
> The models are written in a generic time unit. Whether it is a day, a week, or a year is a matter of convention, and only two rules matter: every rate in a model must use the same unit, and every price quoted per period must be converted to that unit on entry. Accounting data arrives annually and simulations usually run in days, so a conversion is nearly always required. A missed one is the error most likely to survive review, because the answer remains plausible while being wrong by a factor of 365.

1.4.2. The Marginality Test

Almost every difficulty in cost estimation reduces to one question, and it has one answer.

> **!** The test
>
> A cost belongs in a parameter **only if it changes when the quantity that parameter multiplies changes.**

Recall from Equation 1.11 that every cost term is a price times a quantity the system exhibits. Thus, applying the test means naming the quantity first. The holding cost h multiplies average on-hand inventory $\bar{I}$, so for a cost to belong in h it must rise when one more unit is held for one more unit of time. The ordering cost k multiplies the order rate, so a cost belongs in k only if it rises when one more order is placed. A cost that does neither is a **period cost**: real, possibly large, and irrelevant to the decision, because it is incurred identically under every policy you are choosing among.

For example, the classic failure is a fully allocated warehouse cost per unit. The building's lease, its depreciation, its heating, and its management are divided by the units that passed through, producing a defensible-looking dollars-per-unit figure. That figure is correct as an accounting statement and wrong as a model input, because none of those costs change if you hold one more unit. Charging it as though it did will overstate holding cost, understate order quantities, and recommend ordering more often than is economical.

This is the point at which the approach is usually accused of ignoring real money. It is not. Period costs are real and must be paid; they simply do not depend on the decision at hand, so including them changes the answer without changing reality. An accountant computing the true cost of operating a warehouse and an analyst computing the cost parameters for a lot sizing model are answering different questions, and will legitimately produce different numbers from the same ledger. Neither is wrong. Confusing the two is.

1.4.3. Unit Cost, c

The value of one unit, in dollars per unit. What matters is the money tied up in the asset, which is normally the **latest acquisition cost**, what it would cost to replace the unit now, and not an internal transfer price or a customer-facing standard price.

Unit cost enters the models in two distinct ways, and only one of them affects the policy. It appears in the purchase term $c\lambda$, which is constant across policies and therefore inert. It also appears inside the holding cost, because the dominant cost of holding a unit is the capital it ties up, in proportion to what the unit is worth. Thus, the second route is the one that matters.

1.4.4. Holding Cost, h

The cost of holding one unit in inventory for one unit of time. It is almost never estimated directly. Instead it is built from unit cost and a dimensionless **carrying charge** i:

$$h \;=\; i\,c \tag{1.12}$$

Let i represent the carrying charge, expressed as dollars per dollar of inventory value per unit time, and let c represent the unit cost in dollars per unit, as defined in Section 1.4.3. That is, a carrying charge of 0.20 per year means it costs twenty cents per year to hold a dollar's worth of stock. Writing h this way is not only convenient. It says that the cost of holding is proportional to value, so a single carrying charge serves an entire portfolio of items with different prices.

Carrying charges are quoted annually by universal convention, so a model running in days uses $h = ic/365$. Section 1.5.3 builds i from accounting data, where the natural period is again the year.

Note that h multiplies **on-hand** inventory $\bar{I}$, not net inventory $\overline{IN}$. The two differ exactly when the system is short, and the difference is charged separately, by b. Treatments that write the holding term on net inventory are assuming backorders are negligible; Chapter 3 does not make that assumption, and Section 1.3 makes the distinction precise.

Which costs belong in i is the question the marginality test exists to answer, and the conventional list is not reliable. Table 1.5 applies the test to each component in turn.

Table 1.5.: Applying the marginality test to the components conventionally listed under carrying cost.

Component	Belongs in i?	Why
Cost of capital	**Yes**	Money tied up in stock is unavailable for anything else. Unambiguous, and usually the floor.
Obsolescence, deterioration, shrinkage	**Yes**	Exposure is proportional to the value held and to how long it is held.
Insurance and inventory taxes	**Yes, where assessed on inventory value**	Insurers rate on declared value; some jurisdictions levy an ad valorem inventory tax and many do not.
Storage and space	**Only where marginal**	In an owned facility below capacity, the marginal space cost of one more unit is zero.

Component	Belongs in i?	Why
Handling	**Rarely**	Receiving, picking, and shipping scale with *throughput*, which is set by customer demand, not by the policy.

Notice that the three rows answering an unqualified **Yes** are the three whose exposure grows with the value held and with the time it is held, the quantity h multiplies. The two remaining rows are qualified, and they are the ones that surprise people. We justify each of them.

Storage. In a facility you own, operating below capacity, the rack exists whether or not anything is on it. The lease is signed, the depreciation accrues, and holding one more unit costs nothing in space. Charging a percentage for space in that setting treats a fixed period cost as variable. Storage *is* marginal in three situations, and belongs in i in each of them: when space is rented from a third party and billed per pallet position per period; when the facility is at or near capacity, so that additional stock forces overflow leasing or displaces something else, in which case the right figure is the shadow price of capacity rather than the average cost per square foot; and when a lease scales with the space consumed.

Handling. Every unit that arrives is received and put away, and every unit that leaves is picked and shipped. That work is real, but its volume is set by how much customers buy, not by how the replenishment policy is configured. It does not change with the order quantity or the reorder point. Per-receipt handling belongs in the ordering cost k, where it does scale with the decision; per-unit handling belongs in unit cost. Only stock-driven handling, such as re-warehousing, relocation, and congestion caused by holding too much, belongs in i, and it is usually small and difficult to isolate.

1.4.5. Fixed Order Cost, k

The cost of placing one replenishment order, in dollars per order, independent of how large the order is. It covers the administrative work a replenishment triggers: creating and approving the requisition, transmitting it, receiving and inspecting the shipment, matching the invoice, and paying it.

The word *fixed* in "fixed order cost" means fixed **with respect to the order quantity**, not fixed with respect to the number of orders. This trips up nearly everyone once. That is, a cost that scales with the number of units ordered is not part of k and belongs in c, while a cost that is incurred once per order regardless of size is what k is.

Inbound freight is the usual complication. Where a replenishment order causes a shipment, the fixed portion of that shipment's freight behaves exactly like an ordering cost and belongs in k. That fixed portion is often larger than it appears: carriers charge a minimum, and for many items an order sits below the minimum across its entire realistic range, so the freight bill does not vary with order size at all. The variable portion, a rate per pound say, scales with the number of units and behaves like unit cost, with the same inertness as $c\lambda$, because it depends on the demand rate, which the policy does not change. Where a lane is *scheduled* instead of on demand, the truck departs whether or not you ordered, and no part of its cost is caused by your order. Where the choice of where to stock changes which lanes are used, the freight becomes part of that choice. In this chapter it is enough to know that the fixed component belongs in k and must be counted exactly once. See Section 1.5.6.

1.4.6. Stockout and Backorder Costs, π and b

When a demand arrives and cannot be filled from stock, something bad happens. What that something *is* depends on what becomes of the unsatisfied demand, and the two possibilities are priced differently.

If the demand is lost, meaning the customer buys elsewhere and does not return for this order, the harm is a discrete event incurred once per unit not supplied. It is priced by the **stockout cost**

$$\pi \;=\; \text{dollars per unit of demand not filled from stock,}$$

and the corresponding cost rate is π times the rate at which demand goes unfilled, the quantity λ_ℓ of Equation 1.3.

If the demand is backordered, meaning the customer waits and is supplied later, the harm is a *duration*, not an event. One unit owed for ten days is worse than one unit owed for one day, and a cost charged per incident cannot express that. It is priced by the **backorder cost**

$$b \;=\; \text{dollars per unit backordered per unit time,}$$

and the corresponding cost rate is $b\,\bar{B}$, where $\bar{B}$ is the average number of units backordered. This is the parameter that governs the general deterministic model of Chapter 3, in which the system is deliberately allowed to run short because carrying less stock is worth more than the waiting it causes.

The two are not interchangeable notation for one idea; they describe different physical situations. Some settings need both, a per-incident charge for the expedite a stockout triggers plus a per-unit-time charge for the waiting it imposes, and some models charge both. What must not happen is charging both for the *same* consequence, one of the double counts catalogued in Section 1.5.6.

Backorders and waiting are the same measurement

Charging b per unit backordered per unit time is the same as charging for customer waiting time. If demand arrives at rate λ and the average customer waits $\overline{W}$, then $\bar{B} = \lambda\overline{W}$, so $b\,\bar{B} = b\lambda\overline{W}$. A backorder cost and a delay cost are two views of one quantity, and Section 1.3 derives the relationship. This matters in practice: organizations that cannot price a backorder can often state a tolerable customer wait, and that statement is enough.

In practice, organizations rarely produce a defensible number for either π or b. The standard response is not to force one. Instead a **service target** is specified and the policy chosen to meet it at minimum cost. Thus, the two formulations are closely related, since any service target implies a shortage cost and any shortage cost implies a service level. Section 1.6 defines the service measures, and Section 1.5.5 shows how to recover an implied shortage cost from a target the organization already operates to.

1.4.7. Stocking Position Cost, f

The cost, per unit time, of carrying an item at a location at all: the catalog record, the bin reserved for it, the cycle counts it attracts, the planner attention it consumes. Conventionally quoted as dollars per item per location per year.

This is not holding cost

Holding cost scales with the quantity on hand. Stocking position cost is **independent of quantity held**, identical whether one unit or a thousand sits in the bin. The two must not be merged, and neither should be folded into the other.

This parameter does not appear in classical single-location treatments, and it is easy to dismiss as a detail. It is not. For a typical mid-value item it is frequently *larger* than the holding cost of the stock it houses, and because it multiplies nothing, it is the only term that penalizes stocking an item in more places. A model without it will forward-position marginal items that no practical organization would ever stock, because it sees no reason not to.

It matters whenever the decision is not only how much to stock but *whether* an item is stocked at a location at all. The models of this book take the stocking locations as given, so f does not enter their policy calculations; it is introduced here because it is a cost parameter of inventory systems, and leaving it out of the definitions invites the assumption that holding cost is the only cost of carrying an item.

1.4.8. How Accurate Do These Need to Be?

A reasonable worry, having seen how much judgment goes into these numbers, is whether the resulting model can be trusted at all. The answer has a reassuring half and an uncomfortable half, and you should know both before doing the work.

The reassuring half is that the order quantity depends on k/h under a square root, as Chapter 3 derives, so errors are compressed. Let θ represent the factor by which the estimated ratio k/h departs from its true value, so that $\theta = 1$ is a perfect estimate. A parameter wrong by a factor of four moves the order quantity by a factor of two and, as Figure 1.3 shows, costs far less than that in dollars. The curve in that figure is

$$\frac{C}{C^*} = \frac{1}{2}\left(\sqrt{\theta} + \frac{1}{\sqrt{\theta}}\right),$$

because the quantity actually ordered is $\sqrt{\theta}$ times the right one, and Equation 3.22 prices an order quantity that is wrong by a given factor.

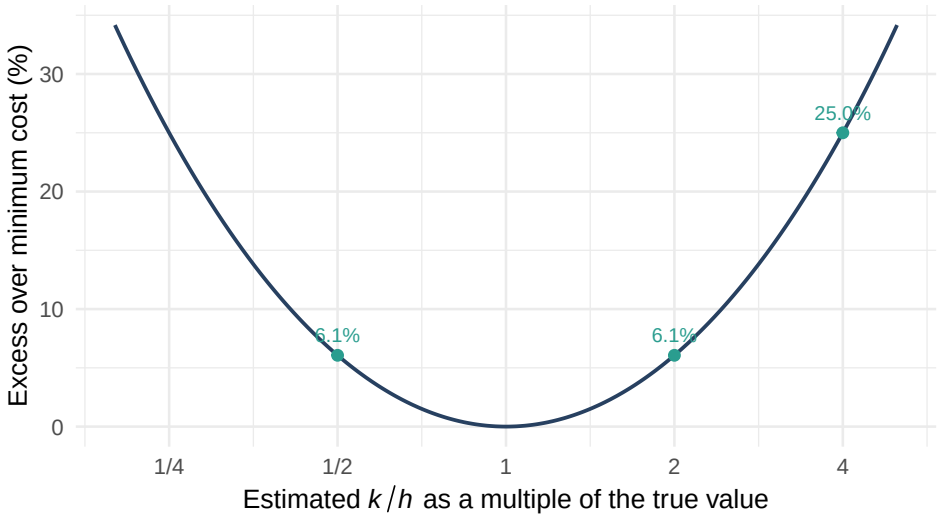


Figure 1.3.: Cost penalty from a misestimated parameter. If the ratio k/h is wrong by a factor θ, the resulting order quantity is wrong by $\sqrt{\theta}$, and the relevant cost rate exceeds its minimum by the amount shown. The curve is flat near the optimum: being wrong by a factor of two costs about 6%.

Read Figure 1.3 from its minimum outward. The horizontal axis is θ on a doubling scale, so equal distances left and right are equal factors of error, and the vertical axis is the percentage by which the resulting cost rate exceeds the minimum. The curve touches zero at $\theta = 1$, where the estimate is exact. Notice how flat it is on both sides of that point. Estimating k/h at half its true value or at twice its true value costs about 6%, and being wrong by a factor of four still costs only about 25%. Notice also that the curve is symmetric on the doubling scale, so overestimating the ratio by a factor and underestimating it by the same factor carry the same penalty. Thus, the square root in the order quantity is doing the protecting, and it protects a great deal.

The uncomfortable half is the same fact seen from the other side. Because the cost surface is flat, a badly wrong parameter produces a cost that looks perfectly acceptable while the *policy* it recommends is substantially wrong. The model will not complain. If the recommendation is a number of units to order, a 6% cost penalty may be tolerable. If the recommendation is which of two configurations to adopt, such as whether to stock an item at a location at all, then a parameter error costing 6% can flip the decision outright, and the flatness that protected the cost is what makes the comparison delicate.

Two practical consequences follow, and both shape the estimation work in Section 1.5. First, effort should go where the answer is sensitive, not uniformly across parameters. Second, when a parameter cannot be pinned down, the right move is often not to defend a point estimate but to report the value at which the recommendation changes.

1.5. Estimating Cost Parameters from Standard Data

The parameters in Section 1.4 are defined precisely enough to be estimated, and every one of them can be built from data an organization already keeps. This section shows how. The examples are notional, but the methods and

the magnitudes are representative of distribution practice. Every figure below is annual, following the convention of the accounting records these numbers come from; converting to the model's time unit is the last step, not the first.

1.5.1. Where the Numbers Come From

Four sources carry essentially everything needed:

- **The general ledger**: write-offs and disposals, insurance premiums, departmental budgets, average inventory value at cost.
- **ERP transaction counts**: purchase orders placed, receipts processed, lines picked. These supply the denominators.
- **The item master**: unit cost, unit of issue, weight and cube.
- **Contracts and instruments**: the insurance policy, the tax assessment, the carrier tariff, the published cost-of-capital rate. These are read, not estimated, and are the most defensible inputs available.

Notice what is absent from that list: no new measurement, no study, no instrumentation. Thus, the work is one of selection and arithmetic, and the selection is governed by the marginality test of Section 1.4.2.

1.5.2. Unit Cost

Take the **latest acquisition cost** from the item master, not the standard or list price.

The distinction matters whenever an internal price carries a cost-recovery surcharge, which is common in organizations that bill their own operations for material.

Example 1.1 (Acquisition cost against standard price)**.** An item's records show two prices: a standard price of \$7.35, which is what internal customers are billed, and a latest acquisition cost of \$6.11, which is what the organization paid. The ratio is

$$\frac{7.35}{6.11} = 1.203$$

so the standard price carries a 20.3% cost-recovery markup. Recall that c is meant to be the money tied up in the asset, what the organization paid and not what it bills internally. Thus, using the standard price as c overstates the capital tied up in each unit by 20.3%, and because $h = ic$ by Equation 1.12 it overstates the holding cost by the same fraction.

That error does not stay put. Holding cost is one of the two quantities the order quantity balances, so an inflated h produces order quantities that are too small, and it does so for every item priced this way.

If only the standard price is available, divide by one plus the cost-recovery rate in force for the period, and record the adjustment as an explicit assumption. It is a single multiplier and easy to revise later.

1.5.3. The Carrying Charge

We build i component by component, applying the test in Table 1.5. We include only the components that survive the test, and we compute each as a fraction of average inventory value.

Cost of capital. Read it off, do not estimate it. Use the published rate on the marginal use of funds: a Treasury or OMB rate in a government setting, the weighted average cost of capital commercially. This is the one component nobody argues about, and it sets a floor under i.

Obsolescence, deterioration, and shrinkage. Usually the largest component, and recoverable directly from accounting:

$$\text{obsolescence rate} = \frac{\text{annual write-offs} + \text{disposals} + \text{net negative inventory adjustments}}{\text{average inventory value at cost}} \tag{1.13}$$

Write-offs and disposals come from the general ledger; average inventory from monthly balances.

Compute this **by commodity class, not portfolio-wide**. Items supporting equipment near the end of its service life run far above consumables, and that spread is the single largest reason two organizations' carrying charges differ. A portfolio average charges the fast-moving consumable for the obsolescence risk of the spare part.

Insurance and taxes. Read off the instruments. Insurers rate on declared inventory value, so the component is the premium attributable to inventory divided by average inventory value. Ad valorem inventory tax applies only where the jurisdiction levies it, which is often nowhere. In practice this component is well under 1%, and it does not repay extended effort.

Storage, where it qualifies. Only in the three marginal cases of Section 1.4.4. Where it does qualify, it is the contracted rate times the positions consumed, divided by inventory value.

Example 1.2 (Building a carrying charge)**.** A depot carries $70 million of inventory at cost, in a building it owns and operates at about 60% of capacity. Its published cost of capital is 5.0%. Last year it wrote off and disposed of $4.2 million of stock, and its insurance premium attributable to inventory was $180,000. Receiving and picking labor cost $3.1 million.

We take the components that survive the marginality test, each as a fraction of the $70 million base:

$$\text{obsolescence} = \frac{4{,}200{,}000}{70{,}000{,}000} = 6.0\%, \qquad \text{insurance} = \frac{180{,}000}{70{,}000{,}000} = 0.26\%$$

Table 1.6.: A carrying charge built from components that survive the marginality test.

Component	Value
Cost of capital	5.00%
Obsolescence and shrinkage	6.00%
Insurance	0.26%
Storage	excluded: owned, below capacity
Handling	excluded: throughput-driven
Carrying charge i	**11.3% per year**

Notice that Table 1.6 lists five components and includes only three of them. We exclude storage because the building exists whether or not the rack is full, so one more unit costs nothing in space. We exclude handling because the \$3.1 million of receiving and picking scales with throughput, which customers set, not with how the replenishment policy is configured. Adding the three surviving components gives $5.00 + 6.00 + 0.26 = 11.26\%$, which we report as 11.3% per year. You should perform that addition every time, because a carrying charge that does not reconcile to the sum of its own table has usually picked up a component twice.

Now change one fact. Suppose the same inventory sits in third-party warehousing at \$18 per pallet position per month, occupying 8,400 positions. Space is now bought by the position and is unambiguously marginal. Carrying the units through the arithmetic,

$$\left(\frac{18 \text{ dollars}}{\text{position-month}}\right)\left(\frac{12 \text{ months}}{\text{year}}\right)(8{,}400 \text{ positions}) \;=\; \frac{1{,}814{,}400 \text{ dollars}}{\text{year}}$$

so that

$$\text{storage} = \frac{1{,}814{,}400}{70{,}000{,}000} = 2.6\%$$

which raises i from 11.3% to 13.9% per year. Notice that the physical inventory has not changed at all. What changed is whether holding one more unit costs anything in space, and that alone moves the carrying charge by a quarter.

We compare the 11.3% figure against the 20% to 25% often quoted as a rule of thumb. The gap is not an error in either direction; it is the storage and handling components, which the rule of thumb includes and the marginality test excludes. Which is right depends entirely on whether those costs are marginal in the setting at hand. In a third-party warehouse billed by the pallet position they certainly are, and i will be much closer to the rule of thumb. In an owned building at 60% occupancy they are not.

> ⚠ This choice is not neutral
>
> Because Q^* varies as $1/\sqrt{h}$, moving i from 18% to the 11.3% of Example 1.2 raises order quantities across the portfolio by $\sqrt{18/11.3}$, about 26%. An inflated carrying charge does not merely add noise: it systematically penalizes whichever alternative holds more inventory. In any comparison where the alternatives differ in how much stock they carry, the direction of the error is predictable, which makes an unjustified carrying charge worse than an uncertain one.

1.5.4. The Fixed Order Cost

We divide the annual cost of the functions that scale with transaction count by the number of transactions processed:

$$k \;=\; \frac{\text{annual transaction-driven cost}}{\text{orders placed per year}} \tag{1.14}$$

Let k represent the fixed order cost in dollars per order, as defined in Section 1.4.5. The numerator is where the judgment lives, because it includes only what moves with the number of orders.

Example 1.3 (Building a fixed order cost)**.** A purchasing organization placed 18,000 orders last year. We review its budget and keep only the costs that move with the number of orders placed:

Table 1.7.: Building the fixed order cost from a purchasing organization's budget.

Component	Annual
Purchasing staff time attributable to placing orders	\$570,000
Receiving and inspection, the per-receipt portion	\$280,000
Systems, transmission, invoice matching	\$50,000
Transaction-driven total	**\$900,000**

Then from Equation 1.14,

$$k = \frac{900{,}000}{18{,}000} = \$50 \text{ per order}$$

Notice what Table 1.7 excludes: management overhead, facility cost, and the contract-negotiation effort that does not vary with how many orders are placed. Each is a real cost of running a purchasing department, and none of them changes if one more order is placed this year. That is, they are period costs in the sense of Section 1.4.2.

Suppose we had included all of those, bringing the numerator to \$1.4 million and k to \$77.78 per order. Because the order quantity varies as

$\sqrt{k}$, the inflated figure would raise every order quantity in the portfolio by $\sqrt{77.78/50} = 1.25$, a 25% overstatement, and it would do so invisibly.

Note also that k carries no time unit. It is dollars per order, and the time dimension enters through the order *rate* it multiplies, so it is the one parameter needing no conversion.

Do not blend dissimilar transactions. An automated release against an existing contract, an EDI message or an automatic replenishment signal, may cost under $5 and sometimes a fraction of a dollar. A manual procurement action involving sourcing, approval, and inspection commonly runs $30 to $150. The gap is an order of magnitude, and a single average is wrong for both populations: it makes automated items order too much and manual items order too little. Where both exist, estimate two values and assign each item by its acquisition method. The same logic applies across echelons. An internal requisition from a branch to its own warehouse is usually far cheaper than an external purchase and should carry its own k.

1.5.5. Stockout and Backorder Costs, and What to Do Instead

There is no ledger account for a disappointed customer, so neither π nor b can be built the way i and k can. Three approaches, in increasing order of practicality:

Enumerate the consequences. Where the unsatisfied demand triggers a measurable event, such as an emergency shipment at a known premium, a production line idled at a known hourly cost, or a contractual late penalty, those costs can be priced directly. Notice that the natural unit falls out of the enumeration. An expedite fee is a one-time charge and belongs in π, while an idled line accrues at an hourly rate and belongs in b. This works in narrow settings and fails in most.

Substitute a service target. Specify the required fill rate, ready rate, or tolerable customer wait, and choose the cheapest policy meeting it. This is what most organizations actually do, and it is not a retreat: it moves the judgment from a dollar figure nobody can defend to a service statement management is already prepared to make. A stated tolerable wait is particularly convenient. Recall that $\bar{B} = \lambda \overline{W}$ from Equation 1.6, so a tolerable wait converts directly into the quantity b would have multiplied. Chapter 8 develops the machinery.

Invert an existing policy. If the organization already operates a rule, a service target it holds to or a stocking threshold such as "do not stock at a branch below 12 demands a year", then that rule embodies a shortage cost, whether or not anyone has written it down. Solving for the π or b that makes the model reproduce the existing rule recovers the organization's own revealed valuation. It is often the most defensible number available, precisely because it is not an outside opinion about what a stockout is worth. It also serves as a cross-check: if the implied value is absurd, the existing rule deserves scrutiny.

1.5.6. Avoiding Double Counts

Cost accounting does not partition costs the way a model needs them to be partitioned, and the same dollar can easily land in two parameters. The model will not detect this; it will simply charge twice and recommend accordingly. Four overlaps account for nearly all such errors.

Table 1.8.: The four overlaps that produce double counts, and how to resolve each.

Overlap	The test
k against inbound freight	Many organizations already bundle inbound freight into a standard cost per order. If k includes it, do not charge freight again as a separate term.
i against f	The holding term scales with **inventory value**; the stocking position term scales with **nothing**. If i already contains a flat per-item storage allocation, subtract it before setting f.
π against b	A backordered demand is charged for the *waiting* by b. Adding a per-unit π on top charges the same failure twice, unless π prices a separate one-time consequence such as an expedite fee.
Obsolescence	Belongs in i, not f, because write-off exposure scales with the value sitting there, not with the fact of the item being listed.

Notice that every row of Table 1.8 is resolved the same way, by asking which quantity the disputed dollar scales with. Thus, the general form of the test is the one that governs everything else in this section. Ask what quantity the cost scales with, and put it in the parameter that multiplies that quantity. A cost that scales with value goes in i. A cost that scales with orders goes in k. A cost that scales with units-owed-times-time goes

in b. A cost that scales with nothing goes in f. A cost that scales with throughput goes in none of them.

1.5.7. When a Point Estimate Is Not Defensible

Some parameters resist estimation no matter how the ledger is interrogated, and f is usually one of them, because the costs that make it up are recorded as departmental totals rather than per item.

The productive move is to stop defending a number and report the value at which the decision changes.

Example 1.4 (When the slot costs more than the stock)**.** A mid-value item has unit cost $c = \$6.11$ per unit, carrying charge $i = 0.18$ per year, and average on-hand inventory $\bar{I} = 10$ units. Using Equation 1.12, we compute its holding cost rate as

$$h\,\bar{I} = i\,c\,\bar{I} = (0.18)(6.11)(10) = \$11.00 \text{ per year}$$

A plausible stocking position cost of $f = \$20$ per item per year is 20/11.00, or 1.8 times that. That is, the cost of *having a slot* exceeds the cost of the inventory sitting in it.

Thus, the stocking decision for this item turns almost entirely on f, the parameter that is hardest to estimate, and hardly at all on the holding cost, which can be computed to the cent. Defending \$20 over \$15 would be false precision. Reporting the break-even is not.

The alternative is a statement of the form: *forward stocking this item pays as long as a stocking position costs less than \$X per year.* That is more defensible than a point estimate, more useful to whoever must act on it, and it converts an argument about an unknowable number into a question the organization can answer from its own experience. Where a parameter is both uncertain and decisive, report the break-even.

1.6. Performance Measures

A performance measure reduces the behavior of a system to a number, so that two policies can be compared. Inventory systems are judged on two families of them. **Cost measures** ask what the system consumes. **Service measures** ask how well it meets demand. Both are built from the state variables of Section 1.3 and the prices of Section 1.4, and neither is complete on its own.

1.6.1. Cost as a Performance Measure

The cost terms of Table 1.4 assemble into a single rate. Let $\overline{N}$ represent the average number of orders placed per unit time, in orders per unit time, the quantity the ordering price multiplies.

$$TC = \underbrace{k\overline{N}}_{\text{ordering}} + \underbrace{h\bar{I}}_{\text{holding}} + \underbrace{b\bar{B}}_{\text{backorder}} + \underbrace{\pi\lambda_\ell}_{\text{stockout}} + \underbrace{c\lambda}_{\text{purchase}} + \underbrace{f}_{\text{position}} \tag{1.15}$$

Every symbol on the right has been defined, the prices in Section 1.4 and the quantities in Section 1.3. Notice that each of the six terms reduces to dollars per unit time, the check of Section 1.4.1 applied to the assembled expression. Three features of Equation 1.15 matter before we use it.

The two shortage terms are alternatives. Under backordering nothing is lost, so $\lambda_\ell = 0$ and the π term vanishes. Under lost sales nothing is owed, so $\bar{B} = 0$ and the b term vanishes. Both appear only under partial backordering, where some demand waits and some departs.

Two terms are inert. Recall that Section 1.4.1 showed $c\lambda$ to be identical under every policy, and that f is charged for carrying the item at all rather than for how it is replenished. Both belong in TC, which is a total, and both drop out of any comparison between replenishment policies.

That last claim has one exception, and Section 3.8 is where it appears. When the supplier prices in tiers, c depends on the order quantity and the purchase term stops being common to the alternatives being compared. What remains,

$$C \;=\; k\,\overline{N} \;+\; h\,\bar{I} \;+\; b\,\bar{B} \;+\; \pi\,\lambda_{\ell} \tag{1.16}$$

is the **relevant cost**, and it is what Section 3.7 measures penalties against.

TC is a rate. It is dollars per unit time, so it can be compared across systems of different sizes only after normalizing, usually by dividing by λ to get cost per unit demanded.

Three other cost-based measures appear constantly in practice, and all are functions of quantities already defined:

- **Average inventory investment**, $c\bar{I}$, in dollars. This is what appears on a balance sheet and what a directive to reduce inventory usually refers to.
- **Cost per unit demanded**, TC/λ, which normalizes for volume.
- **Turnover**, $\lambda/\bar{I}$ from Equation 1.8, which normalizes for the size of the item.

Cost has one property that no service measure has. It is a single number, so it ranks alternatives without further judgment, and that is why Equation 1.15 is the objective for the rest of the book. It also has one weakness, and it is decisive. Two of its six terms require b or π, and Section 1.5.5 argued that most organizations cannot defend either. Thus, a measure that cannot be computed is not a measure.

Service measures are the response. They replace the term that cannot be priced with a constraint that can be stated.

1.6.2. Service Measures

Service measures differ from each other in one respect, and it explains nearly every disagreement about them in practice:

> **!** The distinction that matters
>
> Service measures differ in **what they average over**. Some average over *time*, some over *units demanded*, some over *demand transactions*, and some over *replenishment cycles*. Two measures from different families can be far apart on the same history, and neither is wrong.

1.6.2.1. The Ready Rate

Recall that Equation 1.9 defines $SO(t)$ to equal 1 whenever the shelf is empty. The **ready rate** is the proportion of time the system has stock on hand:

$$\overline{RR} \;=\; 1 - \overline{SO} \;=\; \frac{1}{T}\int_0^T RR(t)\,\mathrm{d}t \tag{1.17}$$

It averages over **time**, weighting every instant equally. That is its strength and its weakness. It is the natural measure when what matters is whether the item is available at an arbitrary moment, such as a spare part supporting equipment that could fail at any time. However, it counts the long quiet stretches when nothing was demanded as heavily as the busy ones, so a system can post a high ready rate while having failed the demands that actually arrived.

1.6.2.2. Fill Rates

A **fill rate** averages over demand rather than time, and there are two of them because demand can be counted in transactions or in units.

Let Y_i represent an indicator that takes the value 1 if the ith demand transaction is filled immediately and completely from stock, and 0 if any part of it is backordered. The **transaction fill rate** over n transactions is

$$\overline{FR} \;=\; \frac{1}{n}\sum_{i=1}^{n} Y_i \tag{1.18}$$

The **unit fill rate** counts units instead, giving partial credit when a transaction is filled in part:

$$\overline{FR}_u \;=\; \frac{\text{units filled from stock on arrival}}{\text{units demanded}} \tag{1.19}$$

When every transaction is for a single unit the two coincide. When transactions are larger they do not, and the gap is not small. For example, a ten-unit order filled to nine units scores 0.9 on the unit fill rate and 0 on the transaction fill rate. Which is right depends on the customer. If the ten units are ten independent issues from a stockroom, nine of them were served and the unit measure is fair. If the ten units are a kit and nine of them assemble nothing, the transaction measure is the right one.

1.6.2.3. Cycle Service Level

The **cycle service level** is the proportion of replenishment cycles in which no stockout occurs, equivalently the probability that demand over a replenishment cycle does not exceed the stock available to meet it. It averages over **cycles**.

This measure is the odd one in this section, and the reason matters. A *cycle* is defined by the replenishment policy: it is the interval between successive order arrivals. An observer who knew nothing about how the system was controlled could measure the ready rate, both fill rates, and the customer wait, but could not identify a cycle at all. The cycle service level is therefore a property of the system *together with* its policy, not of the system alone.

Thus, it is convenient for setting a reorder point, since it translates directly into a quantile of the lead-time demand distribution, and treacherous for comparing alternatives. Two systems at 95% cycle service can deliver very different service to customers if their cycles differ in length, because the one ordering more often is exposed to more cycles and therefore to more opportunities to fail.

1.6.2.4. Customer Wait

The average time a demanded unit waits to be supplied, $\overline{W}$, counting zero for units filled immediately. By Equation 1.6 it is $\bar{B}/\lambda$, so it requires no new measurement.

It is often the measure management can state most confidently. For example, a purchasing manager who cannot price a stockout can usually say whether a two-week wait is acceptable, the property Section 1.5.5 exploits. Reporting the wait *conditional* on having waited is also useful, and is a larger number, because it excludes the immediately-filled demands that pull the unconditional average toward zero.

1.6.2.5. The Measures Disagree

Example 1.5 (Three service measures from one demand history)**.** A location opens the window $[0, 5]$ with 10 units on hand and receives no replenishment during it. Demands arrive at $t = 1, 2, 3, 4, 5$ for the quantities

shown. Unfilled demand is backordered, and $Y_i = 1$ when transaction i is filled completely on arrival, as defined in Section 1.6.2.2.

We work the history one transaction at a time, carrying the on-hand balance forward from each row to the next. At $t = 1$ the shelf holds 10 units and 3 are demanded, so all 3 are filled and 7 remain. At $t = 2$ four more are demanded against 7 on hand, so all 4 are filled and 3 remain. At $t = 3$ five are demanded against 3 on hand, so 3 are filled, 2 are backordered, and the shelf empties. From that point the shelf stays empty, so the demands at $t = 4$ and $t = 5$ are backordered in full. Table 1.9 records the result.

Table 1.9.: A five-transaction demand history against an opening stock of 10 units.

t_i	D_i	On hand before	Filled now	Backordered	$I(t)$ after	Y_i
1	3	10	3	0	7	1
2	4	7	4	0	3	1
3	5	3	3	2	0	0
4	2	0	0	2	0	0
5	1	0	0	1	0	0
Total	**15**		**10**	**5**		**2**

Ready rate. On hand is positive over $[0, 3)$ and zero over $[3, 5]$, so by Equation 1.17

$$\overline{RR} = \frac{3}{5} = 60\%$$

Unit fill rate. Of the 15 units demanded, 10 were supplied from stock on arrival, so by Equation 1.19

$$\overline{FR}_u = \frac{10}{15} = 66.7\%$$

Transaction fill rate. Two of the five transactions were filled completely, so by Equation 1.18

$$\overline{FR} = \frac{1}{5}\sum_{i=1}^{5} Y_i = \frac{2}{5} = 40\%$$

Three numbers between 40% and 67%, from one history, none of them wrong. Notice that the transaction measure is harshest because it gives no credit for the three units of transaction 3 that were supplied. The ready rate is highest because it counts the quiet early periods, when the shelf was full and nothing much was being asked of it, as heavily as the busy later ones. That is, the three measures differ here for the reason Section 1.6.2 gave, which is that each averages over a different thing.

A report claiming "95% service" without saying which of these it means is not telling you very much.

1.6.2.6. When Do They Agree?

The ready rate and the unit fill rate coincide under a specific and important condition: when demand arrives one unit at a time in a Poisson stream. The reason is that Poisson arrivals see time averages: the fraction of arrivals finding the system out of stock equals the fraction of *time* it is out of stock, so averaging over demand and averaging over time give the same answer.

Thus, the two are so often used interchangeably, and doing so is usually safe for slow-moving items demanded singly. It stops being safe as soon as demand arrives in batches. With batch demand a stockout tends to be *caused* by a large transaction, so the arrivals that find the system short are systematically the big ones, and the demand-weighted measures fall

below the time-weighted one. That is the pattern in Example 1.5, where the transaction fill rate sits well below the ready rate.

1.6.2.7. Choosing a Measure

Two questions settle it in practice.

What does the customer actually experience? If a partially filled order is useless, the transaction fill rate is the measure. If units are consumed independently, the unit fill rate is. If the item's value is being available whenever it is needed, the ready rate is. If the customer will wait but cares how long, it is $\overline{W}$.

What can the organization state without inventing a number? The point of substituting a service target for a shortage cost is that management can commit to the target. A target nobody can defend is no better than the cost estimate it replaced. In practice a tolerable wait and a fill rate are the two statements organizations make most readily.

Whichever is chosen, state it precisely and use it consistently. Most of the disagreement about inventory performance in practice is not disagreement about the system. It is two people reporting different averages of the same history, each assuming the other means what they mean.

1.6.3. The Problem the Rest of the Book Solves

The pieces now assemble. A policy sets the values of a few parameters, those parameters determine the quantities in Equation 1.15 and the service measures above, and the problem is to choose the parameters. It takes one of two forms.

The **cost form** prices every consequence and minimizes:

$$\min_{\text{policy}} \quad k\,\overline{N} + h\,\bar{I} + b\,\bar{B} + \pi\,\lambda_\ell + f \tag{1.20}$$

The **service-constrained form** minimizes only the costs that can be defended, and requires a service level instead of pricing failure:

$$\min_{\text{policy}} \quad k\,\overline{N} + h\,\bar{I} + f \qquad \text{subject to} \qquad \overline{FR} \geq \beta \tag{1.21}$$

for a stated target β, with any of the measures of this section in place of $\overline{FR}$.

These are the same problem. A shortage cost implies the service level that optimizing against it produces, and a service target implies the shortage cost for which it is optimal, which is the inversion Section 1.5.5 uses to recover b from an existing policy. Thus, which form to use is a question about which input the organization can defend, not about which is correct.

What changes from chapter to chapter is the policy being chosen and which terms survive, as Table 1.10 shows.

Table 1.10.: The general problem specialized. Every chapter chooses different parameters against the same objective.

Chapter	Policy parameters	Terms that remain
Chapter 3	Q	$k\overline{N} + h\bar{I}$, plus $b\bar{B}$ when backorders are permitted
Chapter 5	An order quantity per period	$k\overline{N} + h\bar{I}$, with demand varying by period
Chapter 7	Q, chosen once	Overage against underage; k is irrelevant with one order

Chapter	Policy parameters	Terms that remain
Chapter 8	r and Q	All of Equation 1.15, or Equation 1.21
Section 8.13	R and S	The same, with R a decision
Chapter 9	Levels at every location	Backorders summed over the locations, under a budget on stock

Read Table 1.10 down its third column rather than across. Notice that the holding term $h\bar{I}$ appears in every row, and that the ordering term $k\overline{N}$ appears in every row but one, the single-period problem, where there is only one order and its cost is unavoidable. What varies is which shortage term survives, and whether a stocking position cost enters at all. Thus, the rest of the book is one objective, one set of state variables, and a different set of assumptions in each chapter about what is known and what may be decided.

1.7. Summary

Inventory exists because supply and demand are separated, and each separation is a lever. Pipeline stock covers the lead time, cycle stock covers the interval between orders, safety stock covers what demand does that the forecast did not say, and anticipation stock covers a season. Each is reduced by a different intervention, and the levers are not interchangeable.

A model is chosen by classifying the system, and the classification is a list of assumptions. Where demand comes from, whether it is known, how the system is reviewed, what the lead time is, and what happens to

unmet demand together decide which chapter applies. Every later chapter changes one or two of those assumptions and keeps the rest.

Three state variables carry the whole subject. On-hand $I(t)$, backorders $B(t)$ and on-order $IO(t)$ combine into the net inventory and the inventory position, and every policy in the book watches the position and not the shelf. Their time averages, and the law of Section 1.3.4 that connects a level to a rate and a wait, are what the performance measures are built from.

A cost belongs in the model only if it changes with the decision. The marginality test separates the holding cost, the ordering cost and the shortage costs from the period costs that are paid whatever the policy, and the stocking position cost enters only where whether to stock at all is in question. Estimating the parameters is harder than using them, and the double counts of Section 1.5.6 are the usual error.

Service is measured three ways, and the three disagree. The ready rate counts time, the fill rate counts demands, and the cycle service level counts replenishment cycles, so one item can report three different figures for the same policy. A stated target means nothing until it says which.

Every chapter minimizes the same objective under different assumptions. Equation 1.15 charges ordering, holding and shortage, and Table 1.10 says which terms survive in each chapter and which parameters are decided. That is the plan of the book.

Table 1.11 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 1.11.: Notation introduced in this chapter.

Symbol	Meaning
λ	Demand rate, in units per unit time
L	Replenishment lead time

Symbol	Meaning
$I(t)$, $B(t)$, $IO(t)$	Units on hand, backordered and on order at time t
$IN(t)$, $IP(t)$	Net inventory $I - B$, and inventory position $I + IO - B$
$\bar{I}$, $\bar{B}$, $\overline{N}$	Time averages of on-hand and backorders, and the average order rate
$\overline{W}$, $\overline{T}_s$	Average wait of a demanded unit, and average time a unit spends on hand
c, i, h	Unit cost, carrying charge, and holding cost $h = ic$
k	Fixed order cost
b, π	Backorder cost per unit per unit time, and stockout cost per unit
f	Stocking position cost
$\overline{RR}$, $\overline{FR}$, CSL	Ready rate, fill rate and cycle service level
TC, C	Total cost rate, and the part of it the policy changes

1.8. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year, and convert any figure quoted per month or per day to an annual basis before using it. Report dollar amounts to the nearest cent, carrying charges and service measures as percentages to one decimal place, and all other ratios and intervals to four significant figures. Assume unmet demand is backordered unless the exercise says it is lost, and assume that arriving replenishments

fill outstanding backorders before any of the units reach the shelf. Where an exercise asks which cost parameter a disputed cost belongs in, state the quantity the cost scales with, since that is what the marginality test of Section 1.4.2 turns on.

Exercise 1.1. A distributor stocks a single item with steady demand and a reliable two-week lead time. Classify this system along each of the dimensions introduced in this chapter, and identify which cost parameters you would expect to dominate.

Exercise 1.2. Explain, in your own words, why a fill rate of 95% and a ready rate of 95% are not the same requirement. Then construct a demand history of your own, in the style of Example 1.5, in which the two measures disagree by at least twenty percentage points, and report all three measures for it.

Exercise 1.3. A distribution center reports the following for the past year: average inventory at cost \$42,000,000; inventory write-offs and disposals \$1,850,000; insurance premium attributable to inventory \$96,000; warehouse operating cost \$3,100,000 for an owned building running at roughly 55% of capacity; receiving and picking labor \$2,400,000. The organization's cost of capital is 4.5%.

Following Example 1.2, determine the carrying charge i. State which components you excluded and justify each exclusion using the marginality test. Then state what your answer would become if the same operation used third-party warehousing billed at \$16 per pallet position per month for the 5,500 positions it occupies.

Exercise 1.4. A purchasing manager proposes the following fixed order cost: the purchasing department's total annual budget of \$1,400,000 divided by the 20,000 orders it placed, giving \$70 per order. The budget comprises buyer salaries of \$780,000, of which roughly two-thirds is spent placing and expediting orders and one-third on contract negotiation and supplier

development; departmental management of \$210,000; an allocated share of facility cost of \$160,000; and purchasing systems and transaction processing of \$250,000.

Following Example 1.3, determine a defensible value of k and explain each inclusion and exclusion. By what factor does the manager's figure overstate the order quantity the model will recommend?

Exercise 1.5. An analyst estimating parameters for a distribution network reports: a carrying charge of 22% per year, which includes a \$14 per item per year allocation for shelf space and cataloguing; a fixed order cost of \$85, which the analyst notes "includes the inbound freight, since we always pay a \$60 minimum on that lane"; and a stocking position cost of \$14 per item per year. The model will also charge inbound freight explicitly at \$60 per shipment. The average item carries \$350 of inventory value at the location.

Identify every double count in this set of parameters, state which parameter each disputed cost belongs in, and give a corrected set.

Exercise 1.6. An item is backordered when out of stock. Demand arrives at $\lambda = 40$ units per year, and under the current policy the average number of units on backorder is $\bar{B} = 0.8$.

Determine the average time a demanded unit waits, counting zero for units filled at once, and state the relationship you used. If management says a two-week wait is tolerable but a one-month wait is not, what does that imply about the backorder cost b relative to the holding cost h? Explain why a per-unit stockout cost π could not express the same requirement.

Exercise 1.7. A location operates an (r, Q) policy with $r = 8$, $Q = 24$, a constant demand rate of $\lambda = 3$ units per day, and a constant lead time of $L = 4$ days.

Sketch $IP(t)$ and $IN(t)$ over three replenishment cycles. Determine the cycle length, the minimum net inventory reached, the average on-hand

inventory $\bar{I}$, the average backorders $\bar{B}$, and the ready rate. Then determine the average on-order inventory $\overline{IO}$ two ways, from your sketch and from Equation 1.5, and confirm they agree. Finally, report the average stocking time $\overline{T}_s$ and the turnover ratio, and state the turnover in turns per year assuming the demand rate is in units per day.

Exercise 1.8. A stockroom opens a week with 12 units on hand and receives no replenishment during it. Demands arrive on successive days for 4, 3, 7, 2, and 5 units.

Compute the ready rate over the five-day window, the unit fill rate, and the transaction fill rate. Explain which of the three you would report to a customer whose orders are kits that are useless if incomplete, and why.

Exercise 1.9. A regional distributor has been told by its parent company to reduce inventory investment by 25% within the quarter. Its stock of a representative item decomposes roughly as: 30% pipeline, 25% cycle, 40% safety, and 5% anticipation held against a seasonal peak three months away.

Identify which interventions could plausibly deliver the reduction within a quarter and which could not, and explain why in terms of what creates each kind of inventory. Then state what you would expect to observe if the target were met on schedule with no other change, and when you would expect to observe it.

2. Managing Inventory

Learning objectives

After reading this chapter you should be able to:

- describe the recurring decisions an inventory manager faces across a large item population
- construct an ABC classification and defend its cut points
- explain what a single-criterion classification misses, and apply a multi-criteria alternative
- classify items by demand pattern, and say which model each pattern selects
- explain why inventory records drift, and measure record accuracy
- design a cycle counting program and set count frequencies by class

2.1. The Manager's Problem

Recall that Section 1.6.3 ends Chapter 1 with a single problem: choose the policy parameters that minimize Equation 1.15, or that meet a service target at least cost. Everything from Chapter 3 onward solves that problem under different assumptions about demand, lead times, and review.

All of it concerns one item. A manager does not have an item. In this chapter we take up the decisions that arise only when there are thousands of them, and none of those decisions is a lot sizing formula.

2.1.1. What Scale Changes

A distributor carries tens of thousands of stock keeping units. A hospital system carries more. Each one has a demand history, a lead time, a unit cost, a supplier, and a policy that has to come from somewhere.

Computation is not what breaks. A spreadsheet will produce forty thousand order quantities in a few seconds, and the models in this book are not expensive to evaluate. What breaks is everything around the computation. Someone has to decide which items deserve a considered decision instead of a default, keep the inputs current as demand and costs move, investigate the exceptions the system raises, and answer for the results. Those activities consume people, and people are finite in a way that arithmetic is not.

Example 2.1 (The attention budget)**.** A distributor stocks 12,000 items. One planner is responsible for them, and about 30 hours a week of that planner's time is available for reviewing items, the rest going to expediting, supplier problems, and meetings. A considered review of one item, looking at its recent demand, checking its parameters, and deciding whether to change anything, takes about six minutes. We work out what that buys.

The weekly capacity is

$$\frac{(30)(60)}{6} = 300 \text{ reviews per week}$$

or 15,000 reviews a year over 50 working weeks. Spread evenly across the portfolio that is

$$\frac{15{,}000}{12{,}000} = 1.25 \text{ reviews per item per year}$$

Now we try to allocate that budget sensibly. Suppose 8% of items are important enough to want monthly attention. Then 960 items at 12 reviews a year consumes 11,520 reviews and leaves 3,480. Give the next 17%, some 2,040 items, a quarterly review and the requirement is 8,160 more, for a total of 19,680 against a budget of 15,000. It does not fit.

Falling back to reviewing that middle group once a year costs 2,040 and leaves 1,440 reviews for the remaining 9,000 items. That works out to one review per item every six years.

Two conclusions follow, and neither depends on the particular numbers.

Most items will never be looked at. That is not through neglect but by arithmetic. Whatever the portfolio size and however diligent the planner, the great majority of items must be handled by a rule that runs without anyone examining the item.

The scarce resource is attention rather than computation. For example, giving all 12,000 items even a quarterly review would take 48,000 reviews a year, i.e. 3.2 planners doing nothing else. The order quantities, by contrast, could be recomputed nightly at no meaningful cost.

A manager's real decisions are not the ones the models answer. The models answer how much and when, for an item, once the inputs are known. The manager decides which items get a considered answer and which get a default, and whether the inputs can be trusted at all.

2.1.2. What the Manager Decides

Table 2.1 lists the recurring responsibilities and where each is treated.

Table 2.1.: The recurring responsibilities of inventory management. Two of them are the subject of this chapter.

Responsibility	The question	Treated in
Segmentation	Which items deserve attention, and which can share a policy?	Section 2.2
Record accuracy	Do the quantities the policies read match the shelf?	Section 2.3
Policy parameters	How much to order, and when?	Parts I and II
Service targets	What availability is promised, and measured how?	Section 1.6
Stocking decisions	How much of an item is held at each location?	Chapter 9
Excess and obsolete	What happens to stock that will not sell?	Section 2.4
Performance reporting	What is measured, and to whom?	Section 2.4

The first two are the subject of this chapter, and we place them before the models, not after, for the same reason.

Segmentation comes first because the models assume an allocation of attention that somebody has to make. Every result in Parts I and II is derived for an item whose parameters are known and current. Recall that Example 2.1 shows this condition cannot hold for the whole portfolio. Thus, deciding which items it will hold for is a prior decision, and it is not a technical one.

Record accuracy comes first because the models read a record rather than a shelf. Section 1.3 defines the state of an inventory system as though $I(t)$ were directly observable. It is not. Every policy in this book triggers on a number the system believes, and Section 2.3 is about the gap between that number and the truth. A correct policy applied to a wrong quantity is wrong, and nothing in its output says so.

Both are prerequisites in the strong sense. That is, a policy can be derived perfectly and still fail, either because it was applied to items that never should have had individual treatment, or because it was computed from quantities that were not true. Notice that neither failure is visible in the policy itself.

2.1.3. What This Chapter Does Not Cover

Two large topics are named here and treated elsewhere, because both need machinery this chapter does not have.

What to stock, and where, is a decision about the network, not about an item, and it turns on the stocking position cost f of Section 1.4.7 together with the structure of the supply network. This book takes the stocking locations as given; Chapter 9 decides how much each of them holds.

How much and when is the rest of the book.

What remains here is the work that has to be done before either of those questions can be answered sensibly: dividing the portfolio so that attention can be allocated, and establishing that the numbers everything else depends on deserve that reliance.

2.2. Grouping and Segmentation

A manager responsible for forty thousand items cannot make forty thousand decisions. Something has to reduce the problem, and that something is grouping. That is, we partition the item population so that decisions are made once per group instead of once per item.

2.2.1. Two Reasons to Group

Grouping is done for two different purposes, and confusing them causes most of the trouble in practice (Abaydulla 2016).

Grouping for attention. Management effort is scarce and should go where it matters. An importance-based scheme ranks items so that the few worth individual review can be identified and the many that are not can be left to a rule. ABC classification is the standard instrument, and Section 2.2.2 takes it up.

Grouping for policy. Setting and maintaining a policy costs effort whether the item is important or not. An operations-based scheme puts items with similar operating characteristics together so that one policy can serve them all without much loss. Clustering methods are the standard instrument, and Section 2.2.7 takes them up.

These are not the same problem. They use different attributes, produce different groups, and are judged by different criteria. An importance ranking says nothing about whether two items can share a reorder policy, and a scheme that groups items by operating similarity says nothing about which deserve a planner's attention.

Applying one where the other is needed is the commonest failure in practice. It is why Ernst and Cohen (1990) observe that classifications built on cost and volume alone "may provide unacceptable performance when evaluated with respect to cost and service measures in complex inventory environments."

Table 2.2 sets out the distinction. Reading down the two columns, notice that they share no row: not the question, not the attributes, not the method, not even the criterion by which the result is judged. Thus, a scheme built for one column is not evidence about the other.

Table 2.2.: The two purposes of grouping. A scheme built for one column is not evidence about the other.

	Grouping for attention	Grouping for policy
Question answered	Which items deserve review?	Which items can share a policy?
Typical attributes	Annual dollar usage, criticality	Demand rate, unit cost, lead time, demand pattern
Typical method	ABC classification	Clustering
Number of groups	Three to six	Whatever the data supports
Judged by	Whether attention lands on the right items	Cost penalty against individual policies

2.2.2. Pareto Analysis and ABC Classification

The observation behind ABC classification is that annual dollar usage is distributed very unevenly across items. A small fraction of items accounts for most of the money.

- Let λ_i represent the annual demand for item i, and c_i its unit cost, so that their product is the **annual dollar usage** of that item. Ranking by

$$\text{annual dollar usage} \;=\; \lambda_i \, c_i \tag{2.1}$$

and cutting the ranked list into classes concentrates attention where the money is. Class A gets individual review and tight control, class C gets a simple rule and infrequent attention, and class B sits between them.

Example 2.2 (Constructing an ABC classification)**.** A location carries twelve items with the annual demands and unit costs below. The third column is Equation 2.1, and the rows are already sorted by it. We construct the classification in two steps, first accumulating the value and then choosing cut points.

Item	Annual demand λ	Unit cost c	Annual dollar usage	Cumulative	Cumulative %
IT-01	1,200	\$75.00	\$90,000	\$90,000	51.4%
IT-02	60	\$520.00	\$31,200	\$121,200	69.3%
IT-03	9,000	\$2.40	\$21,600	\$142,800	81.6%
IT-04	300	\$38.00	\$11,400	\$154,200	88.1%
IT-05	40	\$180.00	\$7,200	\$161,400	92.2%
IT-06	2,500	\$1.80	\$4,500	\$165,900	94.8%
IT-07	150	\$22.00	\$3,300	\$169,200	96.7%
IT-08	18,000	\$0.15	\$2,700	\$171,900	98.2%
IT-09	25	\$64.00	\$1,600	\$173,500	99.2%
IT-10	800	\$1.10	\$880	\$174,380	99.7%
IT-11	5,000	\$0.08	\$400	\$174,780	99.9%
IT-12	90	\$2.20	\$198	\$174,978	100.0%

Total annual dollar usage is \$174,978. Cutting at 80% and 95% of cumulative value gives Table 2.3.

Table 2.3.: The resulting classification.

Class	Items	Share of items	Share of value
A	IT-01 to IT-03	25%	81.6%
B	IT-04 to IT-06	25%	13.2%
C	IT-07 to IT-12	50%	5.2%

Notice that three items out of twelve carry 82% of the annual expenditure, while half the items carry 5%. The technique depends on that concentration, because a planner who reviews only IT-01, IT-02, and IT-03 is watching most of the money.

Notice also that the cut points were chosen rather than derived. For example, moving the A boundary up one item would put 88% of the value in class A, at the cost of reviewing a third of the portfolio. Nothing in the data says where to stop. Thus, the conventional 80/15/5 split is a convention, not a result, and you should be prepared to defend the cut points you choose.

Figure 2.1 shows the same data as a cumulative curve, which is how the concentration is usually presented and how the cut points are usually chosen.

Read the curve from left to right. It rises steeply over the first quarter of the items, reaching 82% of the value by the third item, and then flattens as each further item adds less. The diagonal is what the curve would look like if every item carried the same annual dollar usage. Thus, the gap between the curve and the diagonal is the concentration itself.

Notice what that gap tells you before any cut point is chosen. An item file that plots close to the diagonal has no concentration to exploit, and ABC classification will not help there. The steeper the early rise, the more a small class A buys.

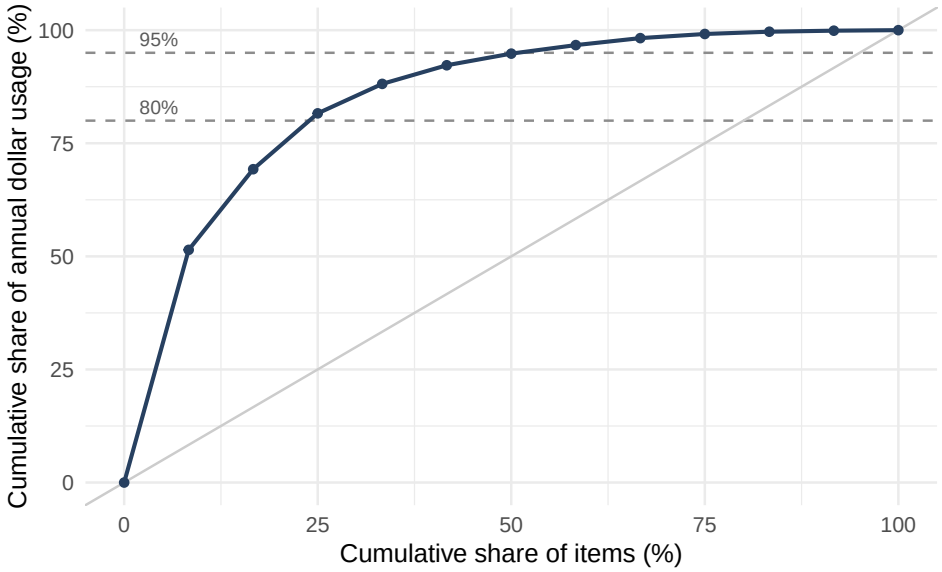


Figure 2.1.: Cumulative share of annual dollar usage against cumulative share of items, for the twelve items of Example 2.2. The curve's steepness at the left is the concentration that makes ABC classification useful. The dashed lines mark the 80% and 95% cuts.

2.2.3. What One Criterion Misses

Ranking by annual dollar usage answers one question, where the money is. That is not always the right question.

The failure runs in both directions (Flores and Whybark 1986). For example, a high dollar usage item may be entirely undemanding, i.e. readily available, substitutable, with a short lead time and no consequence to running out. It will be reviewed weekly because it is expensive. A low dollar usage item may be a fastener that halts an assembly line, available from one supplier on a four month lead time. It will be reviewed annually because it is cheap.

Flores and Whybark (1986) propose that more than one criterion be used, and the criteria proposed since then are numerous: lead time, criticality, commonality, obsolescence, substitutability, reparability, scarcity, durability, order size requirements, stockability, and stockout penalty cost (Ramanathan 2006). Criticality is the most important of these, because it captures the consequence of not being able to supply, what dollar usage cannot see.

Example 2.3 (Reclassifying on two criteria)**.** Recall the twelve items of Example 2.2, and add a criticality rating on a five-point scale, where 5 means an item whose absence halts an operation.

We normalize each criterion to the unit interval, dollar usage by its maximum and criticality by $(\text{rating} - 1)/4$, and then score each item with equal weights:

$$\text{score}_i \;=\; 0.5\left(\frac{\lambda_i c_i}{\max_j \lambda_j c_j}\right) \;+\; 0.5\left(\frac{\text{criticality}_i - 1}{4}\right)$$

Table 2.4.: The twelve items reranked, taking the top three as A and the next three as B.

Item	Dollar usage	Criticality	Score	New class	Old class
IT-01	$90,000	3	0.750	A	A
IT-08	$2,700	5	0.515	A	**C**
IT-11	$400	5	0.502	A	**C**
IT-05	$7,200	4	0.415	B	B
IT-03	$21,600	3	0.370	B	**A**
IT-07	$3,300	3	0.268	B	**C**
IT-04	$11,400	2	0.188	C	**B**
IT-02	$31,200	1	0.173	C	**A**
IT-06	$4,500	2	0.150	C	**B**
IT-09	$1,600	2	0.134	C	C
IT-10	$880	1	0.005	C	C
IT-12	$198	1	0.001	C	C

Seven of the twelve items change class. Two of the movements matter most.

IT-11 rises from C to A. It is an eight cent part with $400 of annual usage, ranked eleventh of twelve on money and third overall once criticality is counted. Thus, under the single-criterion scheme it would be counted once a year and reviewed never.

IT-02 falls from A to C. It is the second most expensive line in the file at $31,200 a year, and it is a criticality 1 item, i.e. substitutable, with no operational consequence to a stockout. It was receiving weekly attention because of its price tag alone.

Notice that the weights were chosen, and that a different pair would move different items. That is the weakness of this approach, and Section 2.2.4 is about avoiding it.

2.2.4. Deriving the Criterion Instead of Choosing Weights

The difficulty with Example 2.3 is the 0.5. Nothing justified it, a manager who preferred 0.7 would get different classes, and no experiment settles the question.

A better line of attack derives the ranking statistic from the cost model rather than assembling it from opinions. Zhang et al. (2001) formulate the inventory problem as minimizing investment subject to service and order frequency constraints, and extract from the resulting expression for the reorder point a single classifying statistic combining unit cost, lead time, and demand. Teunter et al. (2010) extend this with a cost criterion that includes shortage cost, so that criticality enters through the cost of failing to supply instead of through a subjective rating, and report that it outperforms both traditional ABC and the earlier criterion across several real datasets.

The appeal is that several parameters are folded into one statistic without anyone choosing weights, and that the statistic is the one the objective function implies. Thus, where a defensible cost model exists, this is the better approach. Where one does not, weighted scoring remains, and you should record its weights as the assumptions they are.

2.2.5. Which Attributes, and Which Forbid Grouping

Not every attribute plays the same role. Lenard and Roy (1995) separate three kinds, given in Table 2.5, and the distinction is the most practically useful in this literature.

Table 2.5.: Three roles an attribute can play in a grouping (Lenard and Roy 1995).

Role	Meaning	Examples
Prevent grouping	Items differing here must not share a group at all	Storage structure, strategic importance
Weaken grouping	Differences degrade a shared policy without forbidding it	Demand dispersion, lead time
Useful to the manager	Make the resulting groups actionable	Nature of the item, supplier, existing functional groups

The first row of Table 2.5 is the one that gets skipped. For example, two items stocked at different echelons serve different functions, so grouping them produces a policy correct for neither, however similar their demand rates. Thus, you should identify the preventing attributes first and partition on them before any statistical method is applied.

Abaydulla (2016) draws a related line between **structural** attributes, which describe an item's position in the supply network, and **non-structural** attributes, which do not. You should carry that finding forward. Structural attributes should not be fed into a clustering algorithm as ordinary numeric variables, because doing so raised both the cost penalty and the computation time. It was also unnecessary, since items sharing a network structure tended to cluster together on their other attributes anyway. Thus, partition on structure first, then cluster within.

For operational attributes, Kampen et al. (2012) offer four categories: **volume** (demand quantity and demand value), **product** (unit cost, lead time), **customer**, and **timing**. Timing is the least used of the four, and its most important member is the interval between demands. The next

section takes that up, and the criterion there selects the model instead of merely describing the item.

2.2.6. Demand Pattern Segmentation

Two items with identical annual demand can behave entirely differently. For example, one sells four units every week while the other sells two hundred units twice a year. Annual dollar usage cannot tell them apart, and no policy that suits one will suit the other.

We describe the pattern with two quantities.

- Let ADI represent the **average demand interval**, which measures intermittence, i.e. how often a demand occurs.
- Let CV^2 represent the squared coefficient of variation of the non-zero demand sizes, which measures lumpiness, i.e. how variable a demand is when it does occur.

The first is the average interval between non-zero demands,

$$ADI \;=\; \frac{\text{number of periods}}{\text{number of periods with non-zero demand}} \tag{2.2}$$

and the second is

$$CV^2 \;=\; \left(\frac{s}{\bar{x}}\right)^2 \tag{2.3}$$

where $\bar{x}$ and s are the mean and the sample standard deviation of the non-zero demands, with $n-1$ in the denominator of s, as Example 2.4 computes them. Cutting each at a threshold gives the four classes of Table 2.6.

Table 2.6.: Demand classes by intermittence and lumpiness, with the conventional cut points.

	Frequent ($ADI \leq 1.32$)	Infrequent ($ADI > 1.32$)
Variable ($CV^2 > 0.49$)	Erratic	Lumpy
Steady ($CV^2 \leq 0.49$)	Smooth	Intermittent

Example 2.4 (Classifying five demand histories)**.** Five items each have twelve periods of demand. We classify them by computing Equation 2.2 and Equation 2.3 for each.

Item	Demand by period
S-1	20, 22, 19, 21, 23, 20, 18, 22, 21, 19, 20, 21
E-1	5, 40, 2, 60, 10, 35, 1, 50, 8, 45, 3, 30
I-1	0, 10, 0, 0, 9, 0, 11, 0, 0, 10, 0, 0
L-1	0, 0, 45, 0, 0, 3, 0, 0, 0, 80, 0, 12
B-1	0, 15, 12, 0, 18, 14, 0, 16, 13, 0, 17, 15

Take I-1 first. Four of the twelve periods carry demand, so from Equation 2.2 $ADI = 12/4 = 3.00$. The non-zero demands are 10, 9, 11, and 10, with mean 10.0 and standard deviation 0.816, so from Equation 2.3 $CV^2 = (0.816/10.0)^2 = 0.007$. That is infrequent and steady, so I-1 is **intermittent**.

Repeating for all five gives Table 2.7.

Table 2.7.: The five histories classified.

Item	Non-zero periods	ADI	CV^2	Class
S-1	12	1.00	0.005	Smooth
E-1	12	1.00	0.798	Erratic
I-1	4	3.00	0.007	Intermittent
L-1	4	3.00	1.001	Lumpy
B-1	8	1.50	0.018	Intermittent

B-1 is the instructive one. It has demand in eight of twelve periods and very steady sizes, which reads as a well behaved item. However, $ADI = 1.50$ puts it just past the 1.32 cut and into the intermittent class. Thus, you should not let an item sitting that close to a boundary have its treatment decided by which side it fell on.

The classification matters because it selects the model and does not merely describe the item. The stochastic policies of Chapter 8 assume a demand distribution that a smooth item satisfies and a lumpy one does not. The fill rate computed for a lumpy item from a normal approximation can be badly wrong.

Two cautions apply. First, the thresholds are conventions, and the literature does not agree on when a series becomes intermittent. Proposals include at least 30% of periods with zero demand, no more than 60 to 70% of periods with demand, and a mean interval exceeding 1.25 review periods (Johnston and Boylan 1996).

Second, the class depends on the period length chosen. The same transactions bucketed daily rather than monthly will show more zero periods and a longer inter-demand interval. Thus, an item can be moved between classes by changing nothing but the reporting calendar. You should never allow such a result to drive a policy.

A portfolio is rarely mostly smooth. For example, in a hospital pharmacy study of roughly two thousand items, most items fell outside the smooth class (Varghese et al. 2012). That is the normal case and not a pathology, and it explains much of why fill rate estimates disappoint in practice.

2.2.7. Clustering for Policy

Classification schemes assign items to a small number of predetermined classes. Clustering instead lets the data determine the groups. That is, we place items in a space whose coordinates are their operational attributes and group those that lie close together.

The practical algorithm at scale is **k-means**, which partitions items into k groups by repeatedly assigning each item to the nearest group center and recomputing the centers. Alternatives exist, including genetic algorithms, simulated annealing, and tabu search. However, they are far slower on problems of this size, and k-means and its close relatives are essentially the only methods that have been applied to inventory populations of realistic scale (Abaydulla 2016).

Three practical points.

Attributes must be scaled. Demand rate in thousands and unit cost in dollars are not comparable distances, so an unscaled clustering is dominated by whatever attribute happens to carry the largest numbers.

Choosing k is a judgment. More groups reduce the penalty of a shared policy and increase the work of maintaining policies. For example, Abaydulla (2016) found seven groups outperformed three on penalty cost, which is unsurprising and is the trade Section 2.2.8 makes explicit.

Group first, then decide how far to share. Rossetti and Achlerkar (2011) distinguish two uses of the same clustering. Under **group policies**, every item in a cluster uses a single policy computed from the group's average attributes. Under **grouped individual policies**, the clustering organizes

the work but each item still receives its own policy. The first is fast and carries a real penalty, besides costing items their individual identity. The second lands close to individual optimization and costs computation. Both beat ABC on cost and service.

You should keep that distinction in view, because it separates two things that are often run together. Grouping is a way of organizing decisions. It does not oblige you to give every member of a group the same answer.

2.2.8. Evaluating a Grouping

A classification is not right or wrong. It is more or less expensive, and the cost is measurable.

The reference point is what the items would cost under individual policies. Any grouping that forces items to share a policy gives something up against that benchmark.

- Let the **grouping penalty** represent that loss, as a fraction of the individual policy cost.

That is,

$$\text{penalty} \;=\; \frac{C(\text{group policies}) - C(\text{individual policies})}{C(\text{individual policies})} \tag{2.4}$$

Notice that Equation 2.4 uses the relevant cost of Equation 1.16 rather than TC, and that the choice matters. Purchase cost is identical under every grouping, so it cancels in the numerator and only inflates the denominator. Dividing by TC would make the same grouping look cheap for an expensive item and dear for a cheap one, a statement about unit cost and not about grouping.

Set against the penalty is the effort saved, which is roughly the reduction in the number of policies to set, review, and maintain. A decision about grouping is made on those two numbers.

Example 2.5 (Is the grouping worth it?)**.** Six items have been costed three ways: under individual policies, under a two-group scheme, and under a three-group scheme. All figures are annual relevant cost, so purchase cost is excluded. We compute the penalty for each scheme.

Item	Individual	Two groups	Three groups
A	\$4,200	\$4,310	\$4,250
B	\$3,800	\$3,905	\$3,845
C	\$5,100	\$5,180	\$5,140
D	\$1,150	\$1,260	\$1,200
E	\$980	\$1,090	\$1,030
F	\$1,340	\$1,415	\$1,370
Total	**\$16,570**	**\$17,160**	**\$16,835**

From Equation 2.4, the two-group scheme costs

$$\frac{17{,}160 - 16{,}570}{16{,}570} = 3.56\%$$

and the three-group scheme costs

$$\frac{16{,}835 - 16{,}570}{16{,}570} = 1.60\%$$

The decision is worth \$590 a year against \$265 a year, set against maintaining two policies, three, or six.

For six items this is not a decision anybody needs to make, because anyone can maintain six policies. Scale it to forty thousand items and the same

3.56% is the price of reducing forty thousand policy calculations to two hundred, the difference between a procedure that runs and one that does not. Notice that the penalty is the same number either way. What changes is what it buys.

2.2.9. Combining Schemes

Nothing requires choosing one method. Importance classification and operational clustering answer different questions, so applying both yields groups that are identified by importance and homogeneous in operation. However, the order in which they are applied matters (Abaydulla 2016).

Importance first, then clustering. Classify into A, B, and C, then cluster within each class. Items keep a visible priority label, and the clusters inside each class support a shared policy. It is the natural choice when management reporting is organized around the classes.

Clustering first, then importance. Cluster on operational attributes, then rank within each cluster. The number of operational groups is no longer constrained to three or six, and the importance label is applied inside groups that are already homogeneous.

The simplest version of the first is a cross-tabulation, classifying items by dollar usage and by demand pattern and then reading the table. For example, an A item with lumpy demand and a C item with smooth demand are different management problems, and neither classification alone distinguishes them.

2.2.10. Using a Classification

A classification earns its place only through the decisions it changes. Table 2.8 lists the usual ones. Notice that a class appearing in none of those rows is changing nothing.

Table 2.8.: What a classification is for. A class that changes none of these is decoration.

Decision	How the class enters
Review frequency	A items reviewed individually and often, C items on a rule
Service target	Higher fill rates for critical or high value classes
Count frequency	More frequent cycle counts for A items (Section 2.3.7)
Stocking decision	Whether to hold the item at a location at all
Policy sharing	Whether a group policy is applied or each item is set individually

Three failures recur in practice.

Classes become ends in themselves. A quarterly report of how many items are in each class, with no decision attached, is a measurement that costs money and returns nothing.

Boundaries are treated as cliffs. The item just below the A cut and the item just above it are nearly identical, so giving them materially different treatment is arbitrary. Thus, where a decision is sensitive at a boundary, you should look at the items near it, not at the rule.

Classes go stale. Demand moves, products reach end of life, and criticality changes when equipment is retired. Thus, a classification built three years ago and never revisited is directing today's attention using yesterday's item file. Reclassification should be scheduled, and its frequency is a decision of the same kind as count frequency in Section 2.3.7.

2.3. Inventory Record Analysis

Nobody sets a reorder point by walking to the shelf and counting. The policy reads a **record**, and the record is a claim about the shelf, not the shelf itself. Claims decay. In this section we take up how far they decay, how to measure it, and what to do about it.

2.3.1. What a Record Claims, and Why It Drifts

An inventory record asserts, at a minimum, four things: a **stock number**, a **location**, a **quantity on hand**, and a **condition** (Rossetti et al. 2001). The record is inaccurate if any one of them is wrong. That is a stricter definition than it first appears, and it is the right one. For example, an item recorded in the correct quantity at the wrong location cannot be picked, and an item in the right place with the right count but the wrong condition code will be issued to a customer who cannot use it.

Table 2.9 lists where the errors come from.

Table 2.9.: Where record errors come from, and the signature each leaves. The signature matters because it is what makes a cause findable.

Source	What happens	How it shows up
Transaction error	A receipt, issue, or transfer is keyed with the wrong quantity, item, or location	A discrepancy with no offsetting entry
Unrecorded movement	Stock is taken, borrowed, or returned without a transaction	Record higher than shelf, drifting one way
Shrinkage	Theft, breakage, spoilage	Record higher than shelf

Source	What happens	How it shows up
Misidentification	Two similar items confused at receipt or pick	Paired errors, one item high and another low
Unit of issue	Each against case, pack against piece	Errors that are exact multiples
Timing	The count and the transaction fall on opposite sides of a cutoff	A discrepancy that resolves itself
Location error	Stock physically moved without a transfer	Correct total, wrong place

Those sources fall into two mechanisms, and we separate them because they call for different responses (Rossetti et al. 2006). **Transaction errors** occur when a movement is recorded incorrectly, e.g. a receipt keyed at the wrong quantity or a transfer entered against the wrong item. They are process defects and can be designed out. **Stock loss errors** are unrecorded losses through shrinkage, damage, or theft, where the movement was real and no transaction was ever generated. No amount of transaction discipline prevents them, and they drift in one direction, with the record always above the shelf.

Their relative weight depends on the setting. In retail store operations the errors concentrate at receiving and at the point of sale, followed by database processing, physical counting, ticketing, stocking, and shelf alignment (Rossetti et al. 2006). Knowing the order matters, because it says where process improvement effort returns the most.

Notice one feature of Table 2.9 that organizes the whole subject. Every source is a **transaction**, or the absence of one. That is, a record for an item that nothing has happened to cannot become wrong. Orozco (1993) pressed this point hard, arguing that counting effort spent on items

with no activity since the last count is wasted, which is the reasoning behind the transaction-based and opportunity-based counting methods of Section 2.3.6.

2.3.2. Why Accuracy Comes First

Recall that Section 1.3 defined the on-hand inventory $I(t)$ as though it were a single quantity. It is two.

- Let $I_a(t)$ represent the amount actually on the shelf at time t.
- Let $I_r(t)$ represent the amount the system believes is there.
- Let $D(t)$ represent the **discrepancy** between them.

That is,

$$D(t) \;=\; I_a(t) - I_r(t) \tag{2.5}$$

The record is accurate when $D(t) = 0$. Notice that shrinkage and unrecorded issues make $D(t)$ negative, with the record standing above the shelf.

The distinction matters because the two quantities are used for different things (Rossetti et al. 2008). Demand is filled from $I_a(t)$, because that is what physically exists. However, the reorder decision is made on the inventory position, and the position is computed from the record:

$$IP(t) \;=\; I_r(t) + IO(t) - B(t) \tag{2.6}$$

Every policy in this book compares that quantity against a reorder point. When $D(t) \neq 0$ the comparison is made against the wrong number. DeHoratius and Raman (2004) note that the error runs both ways, since it can trigger an order that is not needed and it can suppress one that is.

A policy computed from a wrong record is wrong, and it fails in a way that is difficult to detect.

Consider a continuous review system holding a reorder point r chosen to give some service level. The policy triggers when the recorded position reaches r. If the record overstates what is on the shelf, then the trigger fires late, the effective reorder point is lower than the one designed, and the item runs short more often than the design allows. Notice that nothing in the system reports this. The model reports the service it was designed to deliver, and the warehouse delivers something worse.

Kumar and Arora (1992) studied exactly this at a service parts operation and found that service levels were not being met, with record inaccuracy the cause. We dwell on that for a moment. The organization had a defensible policy, correctly derived, and the policy was not delivering, because the quantity it was reading was not the quantity it had.

There is a second way to see it. Chapter 8 will show that safety stock exists to absorb uncertainty in demand over the lead time. Record error is a second source of uncertainty, and the safety stock absorbs it too, whether or not anyone intended that. Morey (1985) formalized this by defining a minimum actual protection level, i.e. the service a policy really achieves once record error is accounted for alongside demand variability. Thus, an organization with poor records is paying for safety stock twice, once for the demand uncertainty it knows about and again for the record uncertainty it does not.

2.3.3. What Error Actually Costs

Simulation of a two-echelon chain, a supplier feeding a distribution center feeding a retailer, each running an (r, Q) policy with backordering, separates the responses available to an organization whose records are wrong (Rossetti et al. 2008). There are three cases, and the comparison between them is the argument for this whole section.

Leave the policy alone. Running the parameters that were correct for accurate records, against records that are not, gives up service. For example, in the reported experiments the high error case lost roughly half its fill rate, and system backorders rose from about ten items to about three hundred and seventy.

Re-optimize the policy around the error. Recomputing r and Q to hit the same 90% fill rate target does restore the service, and pays for it in stock. The system carries substantially more inventory purely to mask the discrepancy, and backorders still rise sharply, by roughly 300% in the high error case for fast moving items.

This second case is the one that explains why record inaccuracy survives for years without being raised. The customer facing measure is back at target, while the cost has moved into inventory, transportation, and handling, where nobody attributes it to record keeping (Rossetti et al. 2006). That is, the symptom that would have prompted an investigation has been bought off.

Count. Adding cycle counting raised fill rates while holding inventory the same or slightly lower. Notice that this is not a trade. The first two responses trade service against inventory, while counting improves service without buying it with stock, because it attacks the discrepancy instead of compensating for it. This section therefore comes before the policies. There is no point tuning a reorder point to the second decimal place against a record that is right eight times in ten.

2.3.4. Measuring Record Accuracy

The standard measure is the fraction of records examined that are accurate:

$$\text{record accuracy} \;=\; \frac{\text{number of accurate records}}{\text{number of records examined}} \tag{2.7}$$

That is simple to state. However, it hides three choices that change the answer.

What counts as accurate. Requiring the count to match the record exactly is one convention. The other is a **tolerance**, under which a count within some percentage of the recorded quantity is treated as accurate. Tolerances exist because exact agreement on a bin of ten thousand washers is not worth what it costs to obtain. APICS (1980) recommends tolerances of 0.5%, 1%, and 5% for A, B, and C items respectively. Stahl (1998) argues the tolerance should be set from value, usage, lead time, method of counting, criticality, and bill of material level, not from value alone, on the reasoning that a cheap item that halts an assembly deserves a tight tolerance whatever it costs.

The choice is not cosmetic. The same warehouse reports very different accuracy under a strict match rule and under a 5% tolerance. Thus, comparisons across organizations that do not state their tolerance are meaningless.

What is weighted. Equation 2.7 weights every record equally. An alternative weights by value, the thing a financial auditor cares about. These diverge sharply, and the divergence has a direction. A warehouse can be accurate in dollars while being inaccurate in records, because the large errors fall on cheap items and the expensive items are counted carefully. Aggregate dollar value therefore masks individual errors, the central criticism of accuracy programs built on financial sampling. Picking an item to fill an order requires that item's record to be right, and no amount of dollar accuracy supplies it.

Example 2.6 (Three accuracy figures from one count sheet)**.** A cycle count of ten records returns the results below, where the error column is the absolute difference as a percentage of the recorded quantity. We compute the accuracy three ways.

Item	Unit cost	Recorded	Counted	Difference	Error
A-100	$80.00	120	120	0	0.0%
A-101	$150.00	45	44	-1	2.2%
A-102	$2.00	500	495	-5	1.0%
B-200	$400.00	12	12	0	0.0%
B-201	$0.40	3,000	2,970	-30	1.0%
B-202	$25.00	8	0	-8	100.0%
C-300	$6.00	250	250	0	0.0%
C-301	$0.75	1,000	990	-10	1.0%
C-302	$3.00	60	66	+6	10.0%
C-303	$95.00	15	15	0	0.0%

Strict match. Four records agree exactly, so

$$\text{accuracy} = \frac{4}{10} = 40\%$$

Within a 2% tolerance. The four exact records qualify, and so do A-102, B-201, and C-301 at 1.0% each. However, A-101 misses at 2.2%, C-302 at 10%, and B-202 at 100%. Thus,

$$\text{accuracy} = \frac{7}{10} = 70\%$$

Weighted by value. Recorded value is $80(120) + 150(45) + \cdots + 95(15) = \$27{,}405$, and the absolute dollar discrepancy is $150(1) + 2(5) + 0.40(30) + 25(8) + 0.75(10) + 3(6) = \$397.50$. Thus,

$$\text{accuracy} = 1 - \frac{397.50}{27{,}405} = 98.55\%$$

Notice that the same shelf, counted once, reports 40%, 70%, or 98.6% depending only on which definition is used. The value-weighted figure is

highest because the errors sit on cheap items. Recall the masking effect described in Section 2.3.3; this is that effect seen at the level of a single count sheet.

Notice also what the dollar figure does to B-202. The item is missing entirely, with eight units recorded and none on the shelf, and it moves the value-weighted accuracy by $200 out of $27,405. Thus, the next customer who wants B-202 will not care that the warehouse is 98.6% accurate by value.

Which fields. Quantity accuracy alone is the usual reported figure. Location accuracy and condition accuracy are separate measures. A program that reports only quantity is reporting on one of the four fields a record asserts.

Benchmarks. A survey of 410 manufacturing companies found roughly 20% of firms using cycle counting achieving 98% accuracy or better, with more than 60% between 90% and 97% (Rossetti et al. 2001). Materials requirements planning is usually said to require 95% or better in order to function, because Chapter 6 explodes a schedule through records and an error at one level propagates to every level below it. Best in class is 99% and above.

2.3.5. Physical Inventory and Cycle Counting

There are two ways to find out what is really there.

An **annual physical inventory**, counting everything at once, is the older practice and is often required for financial reporting. As a way of managing accuracy it has three defects. It is disruptive, usually requiring operations to stop. It is infrequent, so an error introduced in February is corrected in December after eleven months of decisions made on it. And it diagnoses nothing, because a wall to wall count produces a corrected balance and no information about how the balance became wrong.

Cycle counting counts a portion of the population continuously, on a schedule, without stopping operations. It corrects records as a by-product. Its actual purpose, as Tersine (1994) sets out, is a list in a deliberate order:

1. Identify the causes of errors.
2. Correct the conditions causing the errors.
3. Maintain a high level of record accuracy.
4. Provide a correct statement of assets.

Notice that accuracy is third on that list and the balance sheet is fourth. A cycle counting program judged only on the number of records it corrected is being judged on its by-product.

The two are not exclusive, and in practice most organizations run both, using cycle counting to manage accuracy and a physical inventory to satisfy an auditor. Where cycle counting is well established the physical inventory can often be dispensed with, which is a large part of its financial justification.

2.3.6. Cycle Counting Methods

The methods differ in how the sample is formed, and the choice has consequences for cost, coverage, and statistical validity. Table 2.10 summarizes the main ones (Rossetti et al. 2001; Brooks and Wilson 1995).

Table 2.10.: Cycle counting methods. No method dominates, and large operations normally combine several.

Method	How the sample is formed	Strengths and weaknesses
Random sample	Items drawn at random from the population, with or without replacement	Statistically clean and easy to defend; ignores everything known about which items are likely to be wrong
ABC	Population stratified by class, each class counted at its own frequency	Focuses effort where value is; workload scales with the number of items; classifies on financial criteria that may not track error risk
Process control	The counter counts what is easy to count and skips the rest	Fast, many counts per hour; the sample is biased, but biased toward the items most likely to be wrong
Location based	An area is chosen and every item in it is counted	Large economies of movement; the sample is formed by geography, so item characteristics play no part

Method	How the sample is formed	Strengths and weaknesses
Opportunity based	Counts triggered by events: a reorder, a put away, a balance near the reorder point, or a demand that exposes a discrepancy	Counts at the moment the count is cheapest and most informative; coverage is uneven
Transaction based	An item is counted after every n transactions	Targets the mechanism that creates errors; items with no activity are never counted

Two entries in Table 2.10 deserve comment, because they look wrong at first.

Process control counting is deliberately biased, and its advocates know it. The counter is free to skip an item that is hard to count, so the sample is not random and the resulting accuracy figure is not an unbiased estimate of population accuracy. Brooks and Wilson (1995) recommend it anyway, on the argument that the bias runs toward items with the greatest probability of containing errors, where the counting effort should go. That is, if the purpose is to find and fix causes rather than to estimate a population parameter, then a biased sample aimed at the problem beats an unbiased one that is mostly clean records. The counter is also not performing a blind count, since the record is visible, which is a separate and more serious objection.

Transaction and opportunity based counting follow from the observation in Section 2.3.1 that errors enter through transactions. Counting an item that has not moved since it was last counted correct is close to guaranteed to find nothing.

Opportunity counting is more specific than a general instruction to count at convenient moments. Ordinary operation exposes discrepancies for free, and each exposure is a trigger (Rossetti et al. 2006):

- A demand arrives, the record says there is not enough to fill it, and the stock is visibly on the shelf. The record is understating and can be corrected on the spot.
- A demand arrives, the record shows a positive balance, and the shelf cannot fill it. The record is overstating, and the stockout just made that visible.
- The recorded balance approaches the reorder point, so a count is about to matter for a decision, not for a report.

Notice that the third trigger is the most useful of the three, because it verifies the record precisely when a wrong one would cause a wrong order.

2.3.7. Setting Count Frequency

Under ABC counting the conventional frequencies are four counts per year for A items, two for B, and one for C (Rossetti et al. 2001). Those numbers are a starting convention and not a result. We consider two ways to do better.

Work back from the error rate.

- Let p represent the probability that a record, once verified correct, becomes wrong in any one period, and assume it stays wrong until the next count finds it.
- Let n represent the number of periods between counts, and $A(n)$ the average accuracy over the count cycle.

A record counted every n periods is correct for the first few periods and increasingly likely to be wrong later. Thus,

$$A(n) \;=\; \frac{1}{n}\sum_{t=0}^{n-1}(1-p)^{t} \;=\; \frac{1-(1-p)^{n}}{n\,p} \tag{2.8}$$

Equation 2.8 is the simplest useful version of the two-state Markov model Neeley (1987) develops, in which a record is either accurate or inaccurate and counting returns it to the accurate state.

Figure 2.2 inverts the relation, so that given an error rate and a target accuracy it returns the count interval required. Read it by fixing a target accuracy and following that curve from left to right. As the error rate rises the required interval falls steeply at first and then flattens, so an item that goes wrong twice as often needs counting roughly twice as often. Notice how far apart the curves for different targets lie at low error rates and how they converge at high ones. That is, once a record is going wrong often enough, the target accuracy barely changes the schedule, because the record must be counted almost every period whatever the target is.

Notice that Figure 2.2 makes the case for differentiated frequencies without appealing to value at all. Items differ in p, and it is p, not annual dollar volume, that determines how often a record needs verifying. Thus, where the two disagree, counting by value alone spends effort in the wrong place.

They do disagree. The discriminant analysis of a field experiment by Sheppard and Brown (1993) found item value, dollar value of stock, and quantity on hand all predictive of error, with low value high volume items showing the *lowest* accuracy. That is precisely the population an ABC scheme counts least often.

Example 2.7 (Choosing a count interval from an error rate)**.** An item's record goes wrong with probability $p = 0.03$ in any month, and the target is 95% average accuracy. We evaluate Equation 2.8 at successive intervals.

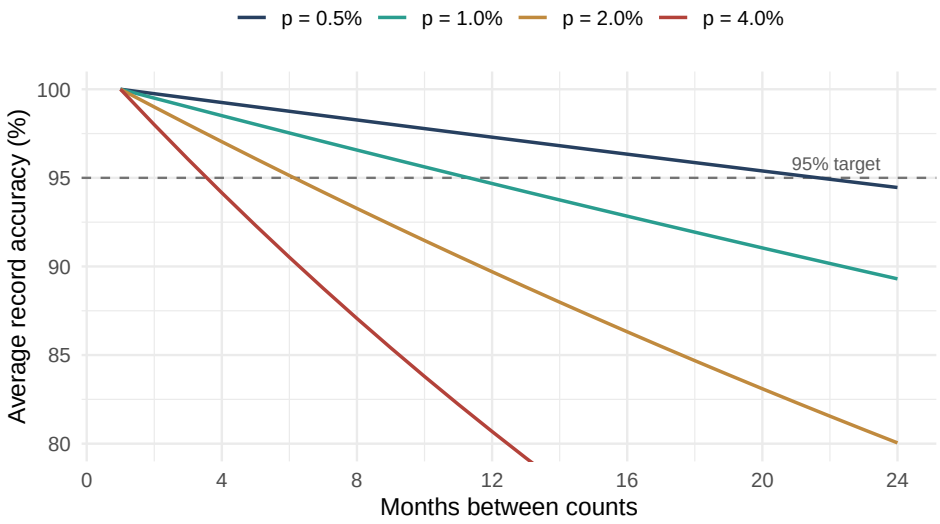


Figure 2.2.: Average record accuracy over a count cycle, from Equation 2.8, against the interval between counts. Each curve is one monthly error rate p. The dashed line marks a 95% target. Holding that target needs a count roughly every six months at $p = 2\%$ and every quarter at $p = 4\%$, against every eleven months at $p = 1\%$.

n months	1	2	3	4	5	6
$A(n)$	100.0%	98.5%	97.0%	**95.6%**	94.2%	92.8%

Counting every four months holds 95.6%, while every five months falls to 94.2%. Thus, the item should be counted every four months, i.e. three times a year.

Now suppose a second item is twice as error prone, with $p = 0.06$. Then $A(2) = 97.0\%$ and $A(3) = 94.1\%$, so it needs counting every two months, six times a year, in order to hold the same target. Notice that doubling the error rate roughly doubles the required count frequency, the flattening seen in Figure 2.2.

Notice also that the first answer lands near the conventional four counts a year for an A item. The difference is that it was derived from a measured error rate rather than assumed, and that it applies whatever the item's annual dollar volume happens to be.

Example 2.8 (Sizing a sample to estimate accuracy)**.** Accuracy is believed to be near 92%, and the goal is to estimate it within two percentage points at 95% confidence. From Equation 2.9 with $z_{0.025} = 1.96$, $\hat{a} = 0.92$, and $e = 0.02$, we have

$$m = \frac{(1.96)^2(0.92)(0.08)}{(0.02)^2} = \frac{(3.8416)(0.0736)}{0.0004} = 706.9$$

Thus, 707 records must be counted. Notice that tightening the margin to one percentage point quadruples the requirement, to 2,828 records, because m varies as $1/e^2$. For that practical reason, accuracy is usually reported to the nearest percentage point and no finer.

Ask which items counting pays for. Accuracy and the value of accuracy are different things, so the item classes that score worst on one may not be

the ones worth counting. For example, simulation of a retail supply chain found cycle counting essential for **low demand, high cost** items, where it produced substantial savings while holding fill rates, and close to worthless for **high demand, low cost** items, where the savings were negligible and sometimes negative (Rossetti et al. 2006). For fast moving cheap items it is usually cheaper to hold more stock than to count, and where counting them is worthwhile at all, opportunity counting outperforms a schedule.

Set that beside Sheppard and Brown (1993), who found low value high volume items had the *lowest* accuracy. Both results hold. The cheap fast items have the worst records and are the least worth correcting, because the consequence of an error on them is small and a little extra stock covers it. Thus, counting effort should follow the cost of being wrong, not the probability of being wrong.

Optimize against cost. The alternative is to price counting and to price its absence. Young and Nie (1992) build a total cost model with stockout cost, counting cost, ordering cost, and holding cost, and study frequency by simulation. Iglehart and Morey (1971) go further and choose the counting policy and the stocking policy together, minimizing total cost per unit time subject to a bound on the probability of a stock denial. Both frame the count frequency as one more term in Equation 1.15, the right way to see it. Counting is a cost incurred in order to protect the accuracy that every other term depends on.

Sizing a sample. Estimating population accuracy is a separate problem from maintaining it.

- Let m represent the number of records to count, e the margin of error, and $\hat{a}$ a prior estimate of accuracy.

To estimate a proportion within a margin e at confidence $1 - \alpha$, the usual expression applies:

$$m \;=\; \frac{z_{\alpha/2}^{2}\,\hat{a}(1-\hat{a})}{e^{2}} \tag{2.9}$$

Notice the awkward consequence of high accuracy. As $\hat{a}$ approaches one the variance falls, and yet demonstrating that accuracy is 99% rather than 97% still requires a large sample. That is one reason mature programs stop trying to measure accuracy precisely and concentrate on causes.

Stratified sampling reduces the sample needed, and Buck and Sadowski (1983) give a method for choosing strata and allocating counts to them for a target confidence. However, their approach and its successors have been criticized for resting on aggregate dollar value, which returns to the masking problem of Section 2.3.4.

2.3.8. Monitoring Accuracy Over Time

Everything above treats accuracy as a quantity to be measured and a schedule to be set. Neither answers the operational question a manager actually has, which is whether accuracy is holding steady, drifting, or responding to a change that was made last month.

Record accuracy is an attribute. That is, each record sampled is either accurate or it is not, so a sample of n records yields a proportion. A p **chart** consumes that data. We plot the sampled accuracy period by period against a center line and control limits, and the chart then separates ordinary sampling noise from a real change. A point inside the limits means the process is behaving as it has been, whatever its level. A point outside means something has changed, and you should investigate it.

Perpetual inventory record sampling is built on that, combining control group sampling with statistical process control (Rossetti et al. 2006). Rather than counting every record on a cycle, which is infeasible where a single department may carry twenty to fifty thousand items, a sample is drawn each period from a prioritized control group and charted.

The recommendation from that work is a sample of 200 to 300 records per week, enough to make the chart informative without consuming a counting team.

Figure 2.3 shows such a chart. The center line is the process average and the two outer lines are the control limits, set three standard errors away. Following the points from left to right, the first eighteen weeks scatter within the limits with no pattern, which is what a stable process looks like however high or low its level. Notice that they range from 88% to 95%. That is, a week four points below the center line is ordinary, and on its own it means nothing.

From week 19 the points drop, and nine of the last twelve fall below the lower limit. That run is the signal. Notice that it is a run and not a point. The chart is reporting a level that has moved and stayed moved, which is what separates a process change from the scatter above it.

Example 2.9 (Reading a control chart)**.** A program samples $n = 250$ records a week, and accuracy has been running at $\bar{p} = 0.92$. We compute the limits and then read two weeks against them.

The standard error of a sampled proportion is

$$s = \sqrt{\frac{\bar{p}(1-\bar{p})}{n}} = \sqrt{\frac{(0.92)(0.08)}{250}} = 0.0172$$

Thus, the control limits are $0.92 \pm 3(0.0172)$, giving a lower limit of 0.869 and an upper limit of 0.972.

This week's count finds 214 of 250 records accurate, i.e. 85.6%. That falls below the lower limit, so the process has changed and the cause should be investigated.

Contrast a week returning 220 of 250, i.e. 88.0%. That is four percentage points below the center line and looks alarming. However, it sits inside the limits. A sample of 250 drawn from a process running at 92% produces

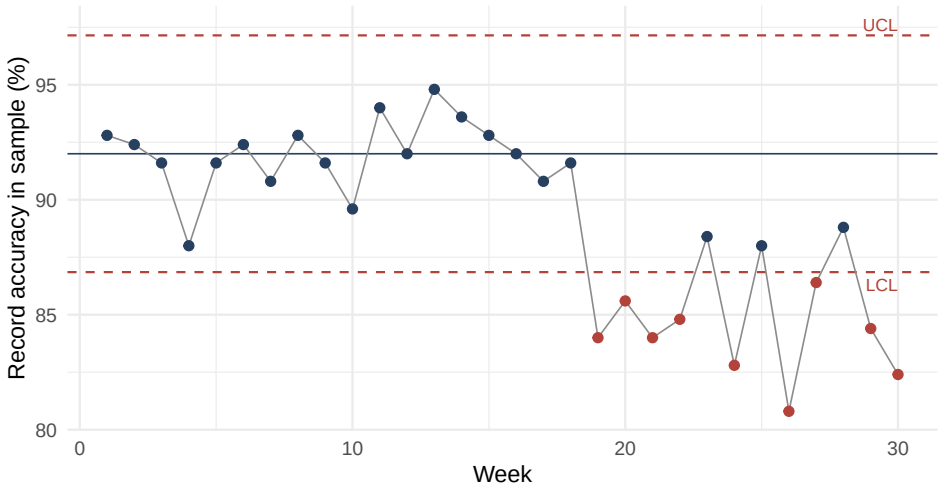


Figure 2.3.: A p chart of weekly record accuracy, 250 records sampled per week. Accuracy runs at 92% for eighteen weeks and then drops to 86% after a process change. Sampling alone moves the weekly reading between 88% and 95% over the stable period, so no single low week means anything on its own. The chart separates that noise from the shift, which breaks the lower control limit in week 19 and stays out for most of the weeks that follow.

readings that low often enough that one of them means nothing on its own. Thus, acting on it would send a team looking for a cause that does not exist.

Two cautions come with the chart, and you should keep both in view.

The population is not homogeneous. A p chart assumes every unit sampled comes from the same process with the same probability of conforming. Inventory records do not satisfy that assumption. Each item has its own propensity to go wrong, for the reasons in Section 2.3.1, so the sample mixes items with different underlying accuracies, so the false alarm rate is inflated. Simulation of the effect found the inflation slight, small enough that the chart remains usable in practice (Rossetti et al. 2006). However, the assumption is violated, and the chart's stated false alarm rate is optimistic.

A signal is a prompt to investigate, not to adjust. An out of control point says the process producing records has changed. Correcting the sampled records addresses none of that. The response is a root cause investigation, the subject of Section 2.3.10.

The chart's value is not primarily in detecting deterioration. It is in demonstrating stability, and in showing whether a process improvement actually moved the level. Without a baseline and control limits, an improvement of two percentage points is indistinguishable from a good week.

2.3.9. Accuracy Is Not a Local Property

Everything so far treats record accuracy as something a location manages for itself. Across a supply chain it is not.

A location's replenishment orders are the demand its supplier sees. Thus, a retailer ordering against bad records sends distorted demand upstream, and the distribution center serving it responds to orders that were triggered

when they should not have been, or that failed to trigger when they should have.

The same two-echelon experiments measured this directly. A single retailer declining to cycle count, while every other location did, cost that chain roughly 6% of fill rate, added about 2% to inventory, and raised backorders by roughly 400% (Rossetti et al. 2008). That is, the cost of one partner's records lands on partners who have no control over them.

Two consequences follow. First, record accuracy belongs in the performance measures a supply chain agrees on, and not only in each location's internal reporting. Brooks and Wilson (1995) make tying inventory managers' evaluation to record accuracy a basic tenet of a working program. Second, where locations are separate firms, the natural instrument is contractual, i.e. performance clauses specifying the accuracy each partner will maintain, on the same footing as delivery or quality terms (Rossetti et al. 2008).

Chapter 9 takes up the analysis of these systems, where the assumption that each location knows its own stock is doing more work than it appears to.

2.3.10. Correction Is Not the Point

A count that finds a discrepancy and adjusts the record has closed a symptom. If the condition that produced the discrepancy is still in place, then the same record will be wrong again, and the program will count the same errors indefinitely at a steady cost.

Control group counting is the standard instrument for finding causes. A small representative set of items is counted repeatedly and frequently, often before a full program is deployed. Because the same items are counted again and again, the errors that appear are process errors, not history, and the short interval between counts localizes when each one entered (Stahl 1998). Control group counting is therefore diagnostic, not corrective, and it is the part of a program most often skipped.

The most useful finding in the practitioner literature is a negative one. In benchmarking more than twenty companies, Rossetti et al. (2001) report that best in class accuracy, 99% and above, was achieved by organizations that dedicated resources to counting, had strong system support, and concentrated on finding and eliminating common process errors. Those organizations did **not** emphasize statistical methods. Stratified sampling and control charting were not what carried them past 99%. Their attention went to enforcing procedures and resourcing the work.

You should read that carefully, because it is easy to over-read. You should know the statistical material above, since it tells you how often to count, what a measured accuracy figure does and does not support, and how large a sample an estimate requires. What the benchmarking says is that statistical sophistication is not sufficient and is not the binding constraint. That is, an organization counting diligently on crude rules and fixing what it finds will outperform one sampling optimally and adjusting records without asking why they were wrong.

The same conclusion is reached from the other direction in Rossetti et al. (2006), whose recommendation for a sampling program is explicit that a program producing only counts and adjustments delivers no real benefit. The counting is the instrument. Finding and preventing the causes is the work.

The difference is also measurable. The two-echelon experiments included a **learning effect**, in which counting not only corrects records but reduces the rate at which errors are subsequently introduced. Its benefits ran both locally and across the chain, and they were larger than those from correction alone (Rossetti et al. 2008). That states the difference between the two activities quantitatively. Correcting a record improves one number once, while removing its cause improves every future period, at every location downstream.

2.4. Other Responsibilities

Two responsibilities from Table 2.1 remain. Neither gets a chapter of its own, and both arise constantly enough to state here.

2.4.1. Excess and Obsolete Stock

Excess stock is more than will be used in any reasonable horizon. **Obsolete** stock will not be used at all, e.g. the product it served is gone, the equipment it repaired is retired, or the specification changed.

It accumulates because every model in this book assumes demand continues. A replenishment policy has no mechanism for noticing that an item's demand has stopped. It sees inventory position fall below a reorder point and orders, as designed, until someone intervenes. Thus, excess is not a failure of the policy. It is the absence of a decision the policy was never asked to make.

Disposing of it is a forward-looking decision, and the difficulty is that it does not feel like one.

Example 2.10 (Keeping stock that will never be used)**.** A location holds 400 units of an item that cost \$28 each. Annual demand has fallen to 12 units, so the stock represents more than 33 years of supply. The carrying charge is $i = 0.20$ per year and the stocking position cost is $f = \$30$ per year. A liquidator offers \$6 per unit. We compare keeping the excess against disposing of it.

Suppose two years of supply is retained, i.e. 24 units, leaving 376 units in excess. The holding cost per unit per year is $h = ic = (0.20)(28) = \$5.60$, so keeping that excess costs

$$(376)(5.60) = \$2{,}105.60 \text{ per year}$$

The stocking position cost f does not appear. The 24 units are kept either way, so the item stays stocked whatever is done with the excess, and by the test of Section 1.4.2 a cost that is incurred under both alternatives is not part of the decision.

Disposing of it returns $(376)(6.00) = \$2{,}256$ now. Notice that one year of holding costs very nearly the entire salvage value, so the break-even is about 1.07 years. Since the stock represents three decades of demand, disposal dominates by a wide margin.

What makes this decision hard is a number that does not appear above. The excess cost $(376)(28) = \$10{,}528$ to buy, and disposing of it at \$6 realizes a write-off of over \$8,000 that someone must sign. That \$10,528 is sunk. It is gone whether the stock is sold, held, or scrapped, and it is not a reason to keep anything. However, the write-off is visible and the \$2,106 a year is not, so the common outcome is to spend the larger amount quietly rather than recognize the smaller one publicly.

Notice where the pieces of this decision came from. Holding cost is Equation 1.12, the stocking position cost of Section 1.4.7 was tested for marginality and left out, and obsolescence risk is already inside the carrying charge through Example 1.2. Excess disposition is not a separate subject. It is the same cost model applied to an item whose demand has ended.

Detecting it is harder than it looks, and Section 2.2.6 explains why. The obvious rule flags items with no movement in some number of months. However, an intermittent item with $ADI = 4$ has demand in one month of four, so three consecutive zero months is its normal behavior, not evidence of death. Thus, a no-movement rule applied without regard to demand pattern will flag healthy slow movers as obsolete and will be slow to catch a smooth item that has stopped. You should set detection thresholds by demand class.

2.4.2. Performance Reporting

What an organization measures is what it manages, and inventory reporting is usually built from quantities already defined: fill rate and ready rate from Section 1.6, average inventory and turnover from Section 1.3, and backorders, inventory investment, and record accuracy from Section 2.3.

Two cautions apply to all of them.

Aggregation hides. For example, a portfolio fill rate of 96% is consistent with a class of critical items running at 60%, because the fast movers that dominate the average are the easy ones. Any measure reported for a portfolio should also be reported by class, which is one more thing the segmentation of Section 2.2 is for.

Measures drive behavior, including behavior nobody wanted. Recall that Section 1.1 described what happens when inventory reduction is mandated without saying which kind. Safety stock is the only component that can be cut on a deadline, so it is the one that gets cut, and the consequence appears a lead time later. Every inventory measure has an equivalent failure mode. Thus, the defense is to pair each with a counterweight that moves the other way, as in Table 2.11.

Table 2.11.: Inventory measures and their counterweights. A measure reported alone will eventually be met alone.

Measure	Improved cheaply by	Counterweight
Inventory turnover	Cutting safety stock	Fill rate, backorders
Inventory investment	Deferring replenishment	Fill rate, stockout frequency
Fill rate	Holding more of everything	Turnover, investment
Order frequency	Ordering in larger lots	Average inventory
Record accuracy	Counting only easy items	Accuracy on a random sample

The last row of Table 2.11 is the one Section 2.3.6 already anticipated. Process control counting is defensible as a way of finding errors precisely because it is biased toward items likely to be wrong, and it is indefensible as a way of reporting a population accuracy figure, for the same reason.

2.5. Summary

The models in the rest of this book take an item and produce a policy. A manager has neither one item nor the attention to treat forty thousand of them individually. In this chapter we settled the two things that have to be in place before those models can be used at scale.

Attention is the binding constraint, not computation. Example 2.1 shows that a realistic review capacity covers a small fraction of a portfolio. Thus, most items must be handled by a rule that runs without anyone examining them, and deciding which items escape the rule is a prior decision. It is a management decision rather than a technical one.

Grouping serves two purposes that should not be confused. Grouping for attention ranks items so effort lands where it matters, and ABC classification by annual dollar usage is the standard instrument. Grouping for policy puts operationally similar items together so one policy can serve them, and clustering is the standard instrument. Notice that a ranking says nothing about whether two items can share a policy.

Single-criterion ABC misses in both directions, promoting expensive items that nothing depends on and burying cheap items that halt operations. Example 2.3 demonstrates the problem, and multi-criteria schemes and cost-derived criteria address it. Whatever the method, a grouping is judged by the cost penalty it carries against individual policies, set against the policies it saves maintaining.

A policy reads a record, not a shelf. Section 1.3 defines on-hand inventory as though it were observable, while Section 2.3 separates what

is on the shelf from what the system believes and shows that the reorder decision uses the second.

Record accuracy is not one number. The same count sheet in Example 2.6 gives 40%, 70%, or 98.6% depending on the definition, so a figure reported without its definition says little. Errors enter through transactions, which is why counting effort belongs where activity is and why opportunity counting works.

Counting is not a trade against inventory. An organization with bad records can give up service or hold more stock in order to mask the problem. The second is why inaccuracy survives for years unremarked, because the customer facing measure returns to target and the cost moves somewhere nobody attributes to record keeping.

Counting raises service without buying it with stock, because it removes the discrepancy instead of compensating for it. Notice also that correcting records is the by-product, not the purpose. A program that only adjusts will find the same errors indefinitely.

Both subjects are prerequisites in a strong sense. A policy derived correctly can still fail, either because it was applied to items that should never have had individual treatment, or because it was computed from quantities that were not true. Neither failure is visible in the policy itself, which is why both belong before Part I rather than after it.

In the next chapter we take up the first of the models themselves, under the strongest assumption available: that demand is known and constant.

2.6. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year. Report dollar usage and costs to the nearest cent, percentages and accuracy figures to one decimal place, and indices, ratios, and intervals to four significant

figures. Where an exercise asks for a classification, state the cut points you used, since nothing in the data supplies them.

Exercise 2.1. A regional distribution center stocks 28,000 items. Three planners are responsible for them, each with about 26 hours a week available for item review over 48 working weeks a year. A considered review takes eight minutes.

Determine the annual review capacity, and the average number of reviews per item per year if the capacity were spread evenly.

Management proposes reviewing the top 6% of items monthly and the next 14% quarterly. Show that this proposal cannot be met with the present staffing, and determine by how much it exceeds capacity. Then find a feasible allocation that preserves monthly review for the top 6%, and report how often a typical item in the remaining group would be reviewed under it.

Finally, determine how many planners would be needed to review every item at least twice a year, and comment on whether that is the right response to the finding.

Exercise 2.2. Table 2.12 gives twenty items from a distribution center's file.

Table 2.12.: Twenty items from a distribution center's file. Criticality is the storeroom's own judgment of what a stockout stops.

Item	Annual usage	Unit cost	Criticality	Lead time, weeks
AW-9930	70	$43.00	low	2
BR-1035	2,200	$50.00	low	1
CT-6650	110	$500.00	high	8
DM-3358	14	$500.00	high	6
GK-7702	40	$1,000.00	low	3
HX-4120	130	$2,000.00	high	6

Item	Annual usage	Unit cost	Criticality	Lead time, weeks
JH-7794	1,200	$5.00	low	1
KY-2143	90	$56.00	low	3
LB-4477	180	$100.00	low	2
MN-5518	1,600	$20.00	low	1
NE-5087	260	$15.00	med	2
PL-2207	600	$300.00	med	2
QP-1209	1,400	$10.00	low	1
RD-2846	350	$80.00	med	2
SV-8814	50	$3,000.00	low	10
TF-9163	25	$800.00	high	12
UT-4462	600	$3.00	low	1
VC-8035	30	$400.00	high	9
WS-3391	900	$50.00	med	2
ZN-6621	450	$20.00	med	2

Following Example 2.2, construct an ABC classification by annual dollar usage. Report the fraction of items and the fraction of annual dollar volume falling in each class, sketch the cumulative curve, and defend your cut points. State what would change if the cuts were moved by five percentage points in either direction.

Exercise 2.3. Following Example 2.3, reclassify Table 2.12 using at least two criteria, one of which is not dollar usage. The table carries two: the storeroom's criticality judgment and the supplier's lead time. Identify the items whose class changes, explain why each moved, and say which classification you would give to a planner.

Exercise 2.4. Four items have the following demand histories over twelve months.

Item	Demand by month
P-1	0, 0, 8, 0, 0, 0, 12, 0, 0, 9, 0, 0
Q-1	4, 6, 5, 7, 4, 6, 5, 6, 4, 7, 5, 6
R-1	0, 30, 0, 2, 0, 45, 0, 1, 0, 38, 0, 3
S-1	12, 1, 20, 3, 15, 2, 18, 1, 22, 2, 14, 3

Following Example 2.4, compute ADI and CV^2 for each item and assign it to a demand class.

Now suppose the same transactions are reported in two-month buckets rather than monthly, so that each item's history becomes six periods. Recompute both statistics and reclassify. Two of the four items change class. Identify which, explain why the change occurred, and state what this implies about reporting a demand classification without stating the period length it was computed on.

Exercise 2.5. Eight items have been costed under individual policies and under two candidate groupings, one forming two groups and one forming four. All figures are annual.

Item	Individual	Two groups	Four groups
A	\$6,400	\$6,610	\$6,490
B	\$5,900	\$6,090	\$5,980
C	\$7,200	\$7,395	\$7,280
D	\$2,100	\$2,290	\$2,185
E	\$1,850	\$2,040	\$1,930
F	\$2,400	\$2,585	\$2,480
G	\$980	\$1,145	\$1,050
H	\$1,120	\$1,290	\$1,195

Using Equation 2.4, compute the grouping penalty for each scheme, as in Example 2.5.

Then recommend a scheme in each of two settings, justifying each recommendation in terms of what the penalty buys. In the first, these eight items are the entire portfolio. In the second, they are a representative sample of a portfolio of 30,000 items, and a planner can maintain roughly 150 distinct policies.

Exercise 2.6. A cycle count of 200 item records finds 34 with a quantity discrepancy. Of those, 11 are within a 2% tolerance of the recorded quantity and the rest are not.

Following Example 2.6, report record accuracy under a strict count-match definition and under the tolerance-based definition, and explain which you would report to management and why. Then, following Example 2.8, determine the sample size needed to estimate accuracy within two percentage points at 95% confidence, using your tolerance-based figure as the prior estimate.

Exercise 2.7. Three items have monthly record error rates of $p = 0.01$, $p = 0.02$, and $p = 0.05$.

Using Equation 2.8, determine the longest count interval that holds average record accuracy at 98% for each, as in Example 2.7. Express each answer as counts per year. Then determine the total annual counting workload for a population of 400 items split evenly among the three error rates, and compare it with the workload under a conventional schedule of four counts a year for A items, two for B, and one for C, assuming the population splits 20/30/50 across those classes.

Exercise 2.8. Take the twenty items of Table 2.12 as a representative sample of a population of 1,200 items in the same class proportions, and a counting team able to perform 60 counts per week over 52 weeks.

State a count frequency for each class, show that the schedule fits the available capacity, and report the slack. Justify the frequencies in terms of what a count is for, and say what you would do with the slack.

Exercise 2.9. A location holds 900 units of a component that cost $14.50 each. The product it supported was discontinued eighteen months ago, and demand over the past year was 30 units, all from a single customer still running the old equipment. The carrying charge is 0.22 per year and the stocking position cost is $25 per year. A broker offers $2.75 per unit for the entire quantity.

Determine the annual cost of retaining the stock beyond a three-year supply, the proceeds of disposing of that excess, and the break-even holding period. State your recommendation.

Then identify the sunk cost in this problem, state its correct role in the decision, and explain why a manager might nonetheless act as though it mattered.

Part I.

Deterministic Models

3. Deterministic Lot Sizing

Learning objectives

After reading this chapter you should be able to:

- describe one cycle of a general deterministic inventory system and identify its four segments
- derive the average on-hand and backorder levels from the geometry of the cycle
- assemble the total cost rate and optimize it over the order quantity and the backorder level
- obtain the classical lot sizing models as special cases of the general one
- compute the reorder point when the lead time is shorter or longer than a cycle
- assess how much a departure from the optimal order quantity costs
- determine order quantities under all-units and incremental quantity discounts
- compute the performance measures of Section 1.6 in closed form for this model
- build and audit a spreadsheet implementation of each of these models
- reproduce the same analysis in code, and say what code does that a worksheet cannot

Recall the pair of questions from Section 1.1: how much to order, and when. In this chapter we answer them for the simplest interesting case, in which demand is known and constant, and so is everything else. That assumption is never exactly true, and Section 1.2.2 explained when it is nonetheless the right one to work under.

Our approach is to build one model that contains all the classical results, and then to obtain each of them by switching an assumption off. That is more work at the start and much less work afterwards, because the cycle geometry is derived once instead of four times.

3.1. The General Cycle

3.1.1. Assumptions

A single item is stocked at a single location. We need notation for the item before anything can be derived.

- Let λ represent the demand rate, which is known and constant, so that demand occurs continuously at that rate.
- Let p represent the replenishment rate, also known and constant, with $p > \lambda$, so that stock builds while replenishment is in progress and depletes when it is not.

Unsatisfied demand is backordered, not lost, and everything repeats, i.e. each cycle is identical to the last.

Two quantities are ours to choose.

- Let Q represent the **order quantity**, the amount brought in each time replenishment occurs.
- Let $\hat{B}$ represent the **maximum backorder level**, the depth to which the system is allowed to run short before replenishment arrives.

Two symbols for peaks

Recall that Section 1.3 writes $I(t)$ for on-hand inventory and $B(t)$ for backorders at an instant, with $\bar{I}$ and $\bar{B}$ for their time averages. In this chapter we also need their **peaks** within a cycle, which are written with hats:

$$\hat{I} = \max_t I(t), \qquad \hat{B} = \max_t B(t)$$

Bar for average, hat for maximum. The peak $\hat{B}$ is a decision, while $\bar{B}$ is a consequence of that decision.

3.1.2. One Cycle, Four Segments

Figure 3.1 shows one cycle, and we follow it from left to right. Net inventory $IN(t)$ starts at its lowest point, $-\hat{B}$. Replenishment begins, and since production runs at p while demand continues at λ, net inventory climbs at the net rate $p - \lambda$. Notice that the line crosses zero partway up the first rise. At that instant the accumulated backorders have all been filled, and every unit arriving after it becomes stock, not a shipment to a waiting customer. Net inventory continues to its peak $\hat{I}$.

Replenishment then stops and demand alone acts, so net inventory falls at rate λ. It passes through zero a second time, at which point the shelf is empty and backorders begin to accumulate, and it reaches $-\hat{B}$, where the cycle repeats. The two rising segments have the same slope, and the two falling segments have the same slope, because only two rates act anywhere in the cycle.

Two relations set the scale of the cycle.

- Let T_p represent the replenishment period, the length of time during which replenishment is in progress.

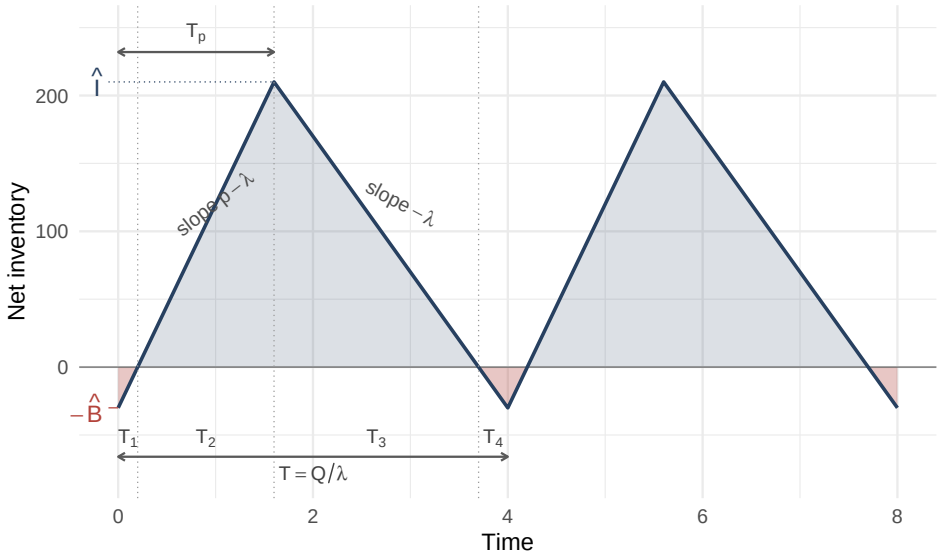


Figure 3.1.: One cycle of the general deterministic model, with $\lambda = 100$, $p = 250$, $Q = 400$, and $\hat{B} = 30$. Net inventory rises at the net rate $p - \lambda$ while replenishment is in progress and falls at λ when it is not. The four segments are the times to clear the backorders, build to the peak, deplete the peak, and accumulate the backorders again.

The quantity Q is produced at rate p, so that

$$Q = p\,T_p \qquad \Longrightarrow \qquad T_p = \frac{Q}{p} \tag{3.1}$$

During T_p the net inventory must climb the whole distance from $-\hat{B}$ to $\hat{I}$ at the net rate $p - \lambda$. That is,

$$\hat{I} + \hat{B} = (p - \lambda)T_p = (p - \lambda)\frac{Q}{p}$$

Solving for the peak in terms of the two decisions, we obtain

$$\hat{I} = Q\left(1 - \frac{\lambda}{p}\right) - \hat{B} \tag{3.2}$$

Notice the factor $(1 - \lambda/p)$, which recurs throughout this chapter. It is the fraction of the order quantity that survives to become stock, the rest having been consumed by demand while it was still being produced. For example, an item produced at twice its demand rate keeps half of each order as stock.

The four segment lengths follow from the same two rates.

- Let T_1, T_2, T_3, T_4 represent the lengths of the four segments, in the order they occur.

Over T_1 the backorders are cleared at the net rate, over T_2 the peak is built at the net rate, over T_3 the peak is consumed at the demand rate, and over T_4 the backorders accumulate at the demand rate. Therefore,

$$T_1 = \frac{\hat{B}}{p - \lambda}, \qquad T_2 = \frac{\hat{I}}{p - \lambda}, \qquad T_3 = \frac{\hat{I}}{\lambda}, \qquad T_4 = \frac{\hat{B}}{\lambda} \tag{3.3}$$

The cycle length is their sum, and it must also be the time to consume Q at rate λ. That is,

$$T = T_1 + T_2 + T_3 + T_4 = \frac{Q}{\lambda} \tag{3.4}$$

Section 3.2 turns the four segments into the two averages the cost function charges.

3.2. Average Inventory and Backorders

Section 1.3 defines the averages as time integrals over the cycle. Here the integrands are triangles, so the integrals are areas and no calculus is needed.

Notice in Figure 3.1 that the on-hand inventory is positive over T_2 and T_3, forming a triangle of height $\hat{I}$ on a base of $T_2 + T_3$. The backorders are positive over T_4 and T_1, forming a triangle of height $\hat{B}$ on a base of $T_1 + T_4$, so that

$$\bar{I} = \frac{1}{T}\int_0^T I(t)\,dt = \frac{1}{T}\cdot\frac{\hat{I}}{2}(T_2+T_3), \qquad \bar{B} = \frac{1}{T}\int_0^T B(t)\,dt = \frac{1}{T}\cdot\frac{\hat{B}}{2}(T_1+T_4) \tag{3.5}$$

Substituting Equation 3.3 and Equation 3.4, the on-hand base is

$$T_2+T_3 = \frac{\hat{I}}{p-\lambda}+\frac{\hat{I}}{\lambda} = \frac{\hat{I}\,p}{\lambda(p-\lambda)}, \qquad \text{so} \qquad \bar{I} = \frac{\lambda}{Q}\cdot\frac{\hat{I}}{2}\cdot\frac{\hat{I}\,p}{\lambda(p-\lambda)} = \frac{\hat{I}^2}{2Q\left(1-\frac{\lambda}{p}\right)},$$

and the backorder base $T_1 + T_4$ simplifies the same way with $\hat{B}$ in place of $\hat{I}$. With Equation 3.2 for $\hat{I}$, we obtain closed forms in the decision variables:

$$\bar{I} = \frac{\hat{I}^2}{2Q\left(1 - \frac{\lambda}{p}\right)} = \frac{\left[Q\left(1 - \frac{\lambda}{p}\right) - \hat{B}\right]^2}{2Q\left(1 - \frac{\lambda}{p}\right)} \tag{3.6}$$

$$\bar{B} = \frac{\hat{B}^2}{2Q\left(1 - \frac{\lambda}{p}\right)} \tag{3.7}$$

Example 3.1 (Working one cycle)**.** An item is demanded at $\lambda = 100$ units per year and replenished at $p = 250$ units per year. The order quantity is $Q = 400$ units and the policy permits a maximum backorder level of $\hat{B} = 30$ units. We work the cycle through.

The peak on-hand level, from Equation 3.2, is

$$\hat{I} = 400\left(1 - \frac{100}{250}\right) - 30 = 400(0.6) - 30 = 210 \text{ units}$$

The four segments follow from Equation 3.3, with $p - \lambda = 150$ units per year:

$$T_1 = \frac{30}{150} = 0.2, \quad T_2 = \frac{210}{150} = 1.4, \quad T_3 = \frac{210}{100} = 2.1, \quad T_4 = \frac{30}{100} = 0.3$$

Notice that these sum to 4.0 years, which is $Q/\lambda = 400/100$, as Equation 3.4 requires. You should perform that check every time, because it catches most algebra errors immediately.

We compute the averages twice, first from the areas directly:

$$\bar{I} = \frac{1}{4} \cdot \frac{210}{2}(1.4 + 2.1) = \frac{1}{4}(105)(3.5) = 91.875$$
$$\bar{B} = \frac{1}{4} \cdot \frac{30}{2}(0.2 + 0.3) = \frac{1}{4}(15)(0.5) = 1.875$$

and then from the closed forms of Equation 3.6 and Equation 3.7, with $Q(1-\lambda/p) = 240$:

$$\bar{I} = \frac{210^2}{2(240)} = \frac{44{,}100}{480} = 91.875, \qquad \bar{B} = \frac{30^2}{2(240)} = \frac{900}{480} = 1.875$$

The two routes agree, the second check we make.

The system holds about 92 units on average and owes about 2. Notice the asymmetry. A backorder allowance of 30 units, a seventh of the peak, produces an average backorder level of under two units. The reason is visible in the segments: the system spends $T_1 + T_4 = 0.5$ years of every four in a shortage position, so the backorder triangle is short as well as low. Thus, you should not read a maximum backorder level as though it described the usual state of the system.

3.3. The Total Cost Rate

Equation 1.15 gives the shape of the total cost rate. Each term is a price from Section 1.4 times a quantity the cycle produces, so we take them one at a time.

Ordering. One order per cycle at k dollars each, and $1/T = \lambda/Q$ cycles per unit time, giving an ordering cost rate of $k\lambda/Q$.

Purchasing. Q units per cycle at c each, giving $c\lambda$. Recall from Section 1.4.1 that this term does not depend on Q or $\hat{B}$, and so it cannot affect the optimization. It is part of TC and not part of the relevant cost of Equation 1.16.

Holding. h per unit per unit time on the average on-hand level, $h\bar{I}$.

Backordering. b per unit per unit time on the average backorder level, $b\bar{B}$.

Stockout. The price π is charged on every unit of demand that arrives to an empty shelf, the λ_ℓ of Equation 1.15. Demand arrives at rate λ throughout the shortage, and the shortage lasts $T_4 + T_1$. Thus, the count in a cycle is $\lambda(T_4 + T_1)$ and not $\hat{B}$.

We dwell on the distinction, because it is easy to get wrong. The peak $\hat{B}$ is the *depth* the queue of waiting demand reaches, while units keep arriving as it drains. Over T_1 another λT_1 of them join the queue and wait. From Equation 3.3,

$$\lambda\left(T_4 + T_1\right) = \lambda\left(\frac{\hat{B}}{\lambda} + \frac{\hat{B}}{p-\lambda}\right) = \frac{\hat{B}}{1-\frac{\lambda}{p}} \tag{3.8}$$

At λ/Q cycles per unit time, the cost rate is therefore $\pi\hat{B}\lambda/[Q(1-\lambda/p)]$.

The backorder term needs no such correction. The average $\bar{B}$ integrates the queue over both segments, so the waiting of the units that arrive during T_1 is already counted there.

Collecting the five terms, we obtain

$$TC(Q,\hat{B}) = \frac{k\lambda}{Q} + c\lambda + h\,\frac{\left[Q\left(1-\frac{\lambda}{p}\right) - \hat{B}\right]^2}{2Q\left(1-\frac{\lambda}{p}\right)} + b\,\frac{\hat{B}^2}{2Q\left(1-\frac{\lambda}{p}\right)} + \frac{\pi\hat{B}\lambda}{Q\left(1-\frac{\lambda}{p}\right)} \tag{3.9}$$

Notice that both shortage prices appear in Equation 3.9. Recall that Section 1.4.6 described π and b as belonging to different physical situations. In a backordering model with an expedite charge both are present, because π prices the incident of being short and b prices the waiting that incident causes. Setting either to zero is a modeling choice, and Section 3.5 makes several of them.

3.4. Optimizing the General Model

We minimize Equation 3.9 over the two decisions. Setting the partial derivatives to zero,

$$\frac{\partial TC}{\partial Q} = 0, \qquad \frac{\partial TC}{\partial \hat{B}} = 0$$

and solving the pair, we obtain

$$Q^* = \sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h\left(1-\frac{\lambda}{p}\right)} - \frac{(\pi\lambda)^2}{h(h+b)\left(1-\frac{\lambda}{p}\right)^2}} \tag{3.10}$$

$$\hat{B}^* = \frac{hQ^*\left(1-\frac{\lambda}{p}\right) - \pi\lambda}{h+b} \tag{3.11}$$

Every classical lot sizing result in this chapter is Equation 3.10 and Equation 3.11 with something switched off.

Two structural features can be read off before we specialize. Notice first that the factor $\sqrt{(h+b)/b}$ is greater than one and increases as b falls. Thus, cheaper backordering means larger orders, because the system is willing to run short for longer between replenishments. Notice second that $\hat{B}^*$ is proportional to $h/(h+b)$ through Q^*. That is, it depends on the ratio of the cost of holding to the cost of holding plus the cost of owing, the same trade-off that the newsvendor's critical ratio expresses in Chapter 7.

Neither formula is unconditionally usable, and you should see the reason now rather than discover it later. A large π drives the second term under the inner root above the first, leaving no real root. A little below that threshold, Equation 3.11 returns a negative $\hat{B}^*$. In both cases the stationary point lies outside the feasible region, so the optimum is on the boundary $\hat{B} = 0$, where the model reduces to Equation 3.14.

Neither case is exotic. For example, with $\lambda = 100$, $p = 250$, $k = 20$, $h = 4$, and $b = 6$, the backorder level turns negative by $\pi = 1$ and the root disappears by $\pi = 1.55$. A high enough price on being short means the right answer is never to be short. Section 3.5 now recovers the classical models as special cases of Equation 3.10 and Equation 3.11.

3.5. The Classical Models as Special Cases

Four assumptions can be relaxed or imposed: whether replenishment is instantaneous, whether backorders are permitted, and whether each of the two shortage prices is charged. Table 3.1 collects the results, and we derive each of them in turn.

Table 3.1.: The classical lot sizing models, each obtained from the general model by imposing an assumption.

Assumptions	Model	Order quantity
p finite, backorders, both π and b	General	Equation 3.10
p finite, backorders, $\pi = 0$	General, no expedite charge	$\sqrt{\frac{h+b}{b}}\sqrt{\frac{2k\lambda}{h\left(1-\frac{\lambda}{p}\right)}}$
p finite, no backorders	Economic production quantity	$\sqrt{\frac{2k\lambda}{h\left(1-\frac{\lambda}{p}\right)}}$
$p \to \infty$, backorders, $\pi = 0$	EOQ with planned backorders	$\sqrt{\frac{h+b}{b}}\sqrt{\frac{2k\lambda}{h}}$
$p \to \infty$, no backorders	Classical EOQ	$\sqrt{\frac{2k\lambda}{h}}$

3.5.1. No Charge Per Unit Short

Setting $\pi = 0$ removes the subtracted term in Equation 3.10, which leaves

$$Q^* = \sqrt{\frac{h+b}{b}}\sqrt{\frac{2k\lambda}{h\left(1-\frac{\lambda}{p}\right)}}, \qquad \hat{B}^* = Q^*\,\frac{h\left(1-\frac{\lambda}{p}\right)}{h+b} \tag{3.12}$$

Equation 3.12 can be reached directly, and the route shows where the factor $\sqrt{(h+b)/b}$ comes from. Let $a = 1-\lambda/p$, so that with $\pi = 0$ the relevant part of Equation 3.9 is $k\lambda/Q + [h(Qa-\hat{B})^2 + b\hat{B}^2]/(2Qa)$. Then

$$\frac{\partial TC}{\partial \hat{B}} = \frac{-h(Qa-\hat{B}) + b\hat{B}}{Qa} = 0 \quad \Longrightarrow \quad \hat{B} = \frac{hQa}{h+b}, \qquad Qa - \hat{B} = \frac{bQa}{h+b},$$
$$TC(Q) = \frac{k\lambda}{Q} + \frac{hb^2+bh^2}{2(h+b)^2}\,Qa = \frac{k\lambda}{Q} + \frac{hb}{h+b}\cdot\frac{Qa}{2},$$

which is the order model with holding rate $ha\,b/(h+b)$, and its minimum is Equation 3.12. That is, the best backorder level for any Q is a fixed share $h/(h+b)$ of the surviving part of the order, and what remains is an ordinary economic order quantity.

Equation 3.12 is the usual formulation when shortages are paid for by the waiting they cause and nothing else. That is the common case when backorders are filled from the next scheduled replenishment and not by an expedited shipment.

3.5.2. No Backorders

Forbidding shortages means $\hat{B} = 0$, so both $\bar{B}$ and the stockout term vanish, and Equation 3.6 collapses to

$$\bar{I} = \frac{\left[Q\left(1-\frac{\lambda}{p}\right)\right]^2}{2Q\left(1-\frac{\lambda}{p}\right)} = \frac{Q}{2}\left(1-\frac{\lambda}{p}\right)$$

This is the familiar half-of-the-peak result, adjusted for the fraction of the order that survives production. The cost rate becomes

$$TC(Q) = \frac{k\lambda}{Q} + c\lambda + \frac{hQ}{2}\left(1-\frac{\lambda}{p}\right) \tag{3.13}$$

Minimizing Equation 3.13 over Q, we obtain the **economic production quantity**

$$Q^* = \sqrt{\frac{2k\lambda}{h\left(1-\frac{\lambda}{p}\right)}} \tag{3.14}$$

> ⚠ How backorders are forbidden
>
> You may be tempted to obtain Equation 3.14 by setting $b = 0$, and that is wrong. A zero backorder cost makes shortages free, not forbidden, and Equation 3.12 then makes Q^* unbounded.
> Forbidding shortages is instead the limit $b \to \infty$. In Equation 3.12, $\sqrt{(h+b)/b} \to 1$ and $\hat{B}^* \to 0$, which recovers Equation 3.14 exactly. Thus, charging an infinite price for owing a unit is how a model is told never to owe one.

3.5.3. Instantaneous Replenishment

If a replenishment arrives all at once instead of being produced over time, then $p \to \infty$ and $\lambda/p \to 0$. The factor $(1-\lambda/p)$ becomes one and T_p

becomes zero. Thus, the cycle loses its build-up segments, and the sawtooth of Figure 3.1 becomes vertical on the way up.

With backorders still permitted and $\pi = 0$,

$$Q^* = \sqrt{\frac{h+b}{b}}\sqrt{\frac{2k\lambda}{h}}, \qquad \hat{B}^* = Q^* \, \frac{h}{h+b} \tag{3.15}$$

3.5.4. The Classical Economic Order Quantity

Imposing both instantaneous replenishment and no backorders leaves the oldest result in the subject. The cost rate is

$$TC(Q) = \frac{k\lambda}{Q} + c\lambda + \frac{hQ}{2} \tag{3.16}$$

Notice the three terms: an ordering term falling in Q, a holding term rising in Q, and a purchase term that does not depend on Q at all. Differentiating,

$$\frac{d\,TC}{dQ} = -\frac{k\lambda}{Q^2} + \frac{h}{2} = 0 \qquad \Longrightarrow \qquad Q^* = \sqrt{\frac{2k\lambda}{h}} \tag{3.17}$$

Substituting back, we obtain a total cost with a memorable form:

$$TC^* = \sqrt{2k\lambda h} + c\lambda \tag{3.18}$$

Notice that at the optimum the ordering and holding terms are equal, each contributing $\sqrt{2k\lambda h}/2$. That is not a coincidence, and it is the fastest way to check a numerical answer. If the two terms differ, then Q is not optimal.

Figure 3.2 shows the three terms for an item with $\lambda = 2400$ units per year, $k = \$75$ per order, and $h = \$4$ per unit per year. Reading it from left to right, the ordering curve falls steeply at small quantities, because a small order means many orders a year. The holding line rises steadily, since average stock is proportional to the order quantity. Notice that the two cross exactly at the dashed line marking Q^*, the equal-terms property stated above made visible.

Notice also the shape of the total curve near its minimum. It is flat over a wide range of quantities, so a quantity some way from Q^* costs very little more than Q^* itself. That flatness is the subject of Section 3.7 and it is relied on repeatedly later in the book.

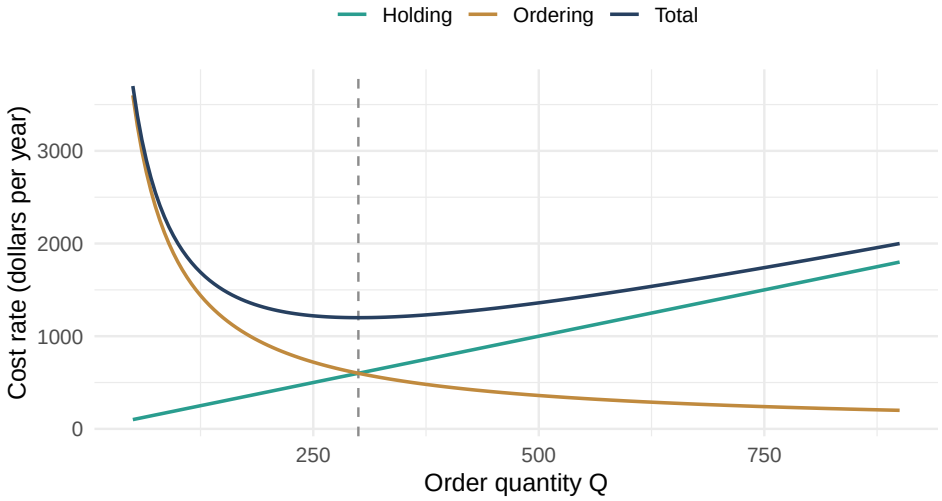


Figure 3.2.: Ordering, holding, and total relevant cost rate against the order quantity, for $\lambda = 2400$, $k = 75$, and $h = 4$. The dashed line marks Q^*, where the two curves cross and the total is at its minimum. The total is visibly flat near the minimum, which is the subject of Section 3.7.

Because Q^* is fixed and demand is constant, the optimal cycle length

follows immediately as

$$T^* = \frac{Q^*}{\lambda} = \sqrt{\frac{2k}{h\lambda}} \tag{3.19}$$

Choosing Q and choosing T are the same decision. Which one you present to an operating group depends on what they can act on. For example, a warehouse that ships on a weekly cycle finds a time far easier to use than a quantity.

Example 3.2 (The economic order quantity)**.** A product is demanded at $\lambda = 220$ units per year and has a unit cost of $c = \$1{,}200$ per unit. The ordering cost is $k = \$800$ per order and the carrying charge is $i = 0.18$ per year. No shortages are allowed.

What is stocked? One item, bought from an outside supplier.

What is the demand process? Constant at $\lambda = 220$ units per year, and known in advance.

When is inventory reviewed? Continuously.

What triggers replenishment, and how much? The shelf reaching zero, and the order is for Q units, which is the decision.

What happens to unmet demand? None occurs; shortages are not allowed.

What costs are incurred, and when? $k = \$800$ each time an order is placed, and the carrying charge $i = 0.18$ per year on the value of the stock held.

We work the four parts in turn.

The holding cost, from Equation 1.12, is

$$h = ic = (0.18)(1200) = \$216 \text{ per unit per year}$$

a) The optimal order quantity. From Equation 3.17,

$$Q^* = \sqrt{\frac{2(800)(220)}{216}} = \sqrt{\frac{352{,}000}{216}} = 40.37 \text{ units}$$

b) The optimal time between orders. From Equation 3.19,

$$T^* = \frac{40.37}{220} = 0.1835 \text{ year} = 66.98 \text{ days}$$

c) The optimal total cost. From Equation 3.18,

$$TC^* = \sqrt{2(800)(220)(216)} + (1200)(220) = 8{,}719.63 + 264{,}000 = \$272{,}719.63$$

Notice the proportions. The purchase cost is 97% of the total and is untouched by the decision. The \$8,719.63 is the entire quantity the lot sizing decision can influence, and half of it is ordering cost and half is holding cost. That equal split is the check of Section 3.5.4, and you should perform it every time, because unequal halves mean Q is not optimal.

d) The reorder point for a 7 day lead time. Seven days is well under the 66.98 day cycle. Thus, Section 3.6 gives $r = \lambda L$:

$$r = 220\left(\frac{7}{365}\right) = 4.22 \text{ units}$$

An order is placed whenever the inventory position falls to about 4 units.

A closing observation. The optimal cycle $T^* = 66.98$ days is 9.6 weeks. If this item ships on a weekly truck, then ordering every ten weeks fits an operating rhythm the warehouse already has, and it costs almost nothing against the optimum, as Section 3.7 will confirm. You will usually find that the better answer.

3.6. Lead Time and the Reorder Point

Nothing so far has said when to order, only how much. With demand and lead time both known the timing is exact. We place the order so that it arrives at the moment stock would otherwise reach $-\hat{B}$.

- Let L represent the replenishment lead time, which is known and constant.
- Let r represent the **reorder point**, the inventory position at which the order is placed.

When $L \leq T$. Demand over the lead time is λL. Thus, ordering when the position reaches

$$r = \lambda L - \hat{B} \tag{3.20}$$

brings the replenishment in exactly on time. With no backorders permitted this reduces to $r = \lambda L$.

When $L > T$. A lead time longer than a cycle means one or more replenishment orders are outstanding when the next is placed. Thus, the reorder point must account only for the demand not already covered by those outstanding orders. The number of whole cycles inside the lead time is $\lfloor L/T \rfloor$, so the uncovered portion of the lead time is the remainder $L - \lfloor L/T \rfloor T$, which gives

$$r = \left(L - \left\lfloor \frac{L}{T} \right\rfloor T \right) \lambda - \hat{B} \tag{3.21}$$

Equation 3.21 reduces to Equation 3.20 when $L < T$, because the floor is then zero. Recall from Section 1.3.3 that what matters is the inventory **position**, which counts outstanding orders, and not the on-hand level. Equation 3.21 is the deterministic version of that point.

3.7. Sensitivity of the Order Quantity

The formulas above return a number like 40.37, and no warehouse orders 40.37 of anything. It matters how much is lost by rounding that number, or by ordering a case quantity, or by fitting a shipping schedule.

- Let $C(Q)$ represent the relevant cost of Equation 1.16 at an order quantity Q, which for this model is the part of Equation 3.16 that depends on Q.

That is,

$$C(Q) = \frac{k\lambda}{Q} + \frac{hQ}{2}, \qquad C^* = C(Q^*) = \sqrt{2k\lambda h}$$

The step is to eliminate $k\lambda$ using Equation 3.17 itself. Squaring it gives $Q^{*2} = 2k\lambda/h$, so $k\lambda = hQ^{*2}/2$, and the minimum is $C^* = \sqrt{2k\lambda h} = h\sqrt{2k\lambda/h} = hQ^*$. Substituting for $k\lambda$ and dividing by C^*, we obtain

$$\begin{aligned} C(Q) &= \frac{hQ^{*2}}{2Q} + \frac{hQ}{2} = \frac{h}{2}\left(\frac{Q^{*2}}{Q} + Q\right) \\ \frac{C(Q)}{C^*} &= \frac{1}{2Q^*}\left(\frac{Q^{*2}}{Q} + Q\right) = \frac{1}{2}\left(\frac{Q}{Q^*} + \frac{Q^*}{Q}\right) \end{aligned} \tag{3.22}$$

Notice that the penalty depends only on the **ratio** of the chosen quantity to the optimal one, and not on k, λ, h, or the size of the item. Recall that the same expression appeared in Section 1.4.8, where the ratio came from an error in the parameters and not from a deliberate rounding.

Example 3.3 (What a wrong order quantity costs)**.** Suppose the order quantity used is 50% larger than optimal, $Q/Q^* = 3/2$. From Equation 3.22,

$$\frac{C}{C^*} = \frac{1}{2}\left(\frac{3}{2} + \frac{2}{3}\right) = \frac{1}{2}\left(\frac{9+4}{6}\right) = \frac{13}{12} = 1.083$$

That is an increase of 8.3% in the relevant cost. For example, for the item of Example 3.2, whose relevant cost is \$8,719.63, it amounts to about \$727 a year against a total spend of \$272,720.

Ordering half the optimal quantity, $Q/Q^* = 1/2$, gives $\frac{1}{2}(0.5 + 2) = 1.25$, a penalty of 25%. Ordering double gives the same 25%, because Equation 3.22 is symmetric in Q/Q^* and its reciprocal.

You should read this as saying that the EOQ is a target to round toward, not a value to honor. Being wrong by half is expensive; being wrong by a fifth is not. That tolerance is the reason it is reasonable to order in case quantities, to fit a delivery schedule, and to group items onto a shared order, all of which Chapter 4 does.

3.8. Quantity Discounts

Everything so far has treated the unit cost c as a constant, which made the purchase term $c\lambda$ inert. Suppliers routinely price in tiers, and once c depends on Q the purchase term is no longer inert. The analysis changes.

Two schemes are in general use. Under **all-units** discounting the whole order is priced at the rate reached by its size. Under **incremental** discounting only the units above each break point receive the lower price. For example, on a schedule breaking at 500 units, an order of 600 units pays the lower price on all 600 units under all-units discounting and on only the last 100 under incremental discounting. They lead to different models and different answers, so we treat them separately.

The setting is common to both.

- Let m represent the number of discount levels.

- Let q_j represent the break point beginning level j, with

$$q_1 = 0 < q_2 < q_3 < \cdots < q_m$$

- Let c_j represent the unit price applying in interval j, i.e. for $q_j \leq Q < q_{j+1}$, with $c_1 > c_2 > \cdots > c_m$ so that larger orders are cheaper per unit.

We assume a single item at a single location, no shortages, and instantaneous replenishment, so that the underlying model is the classical EOQ of Section 3.5.4.

3.8.1. All-Units Discounting

If the entire order is priced at c_j, then the cost rate within interval j is Equation 3.16 with that price:

$$TC_j(Q) = c_j\lambda + \frac{k\lambda}{Q} + \frac{h_j Q}{2}, \qquad h_j = ic_j, \qquad q_j \leq Q < q_{j+1} \tag{3.23}$$

The important feature is that $TC(Q)$ is **not continuous**. Each interval carries its own curve, lower prices give lower curves, and at each break point the cost drops to the next curve down.

Figure 3.3 shows the shape for an item with $\lambda = 8000$ units per year, $k = \$30$ per order, $i = 0.30$ per year, and a price of \$10 below 500 units and \$9 at or above. Each price is drawn solid where it applies and dotted where it does not, so the cost function the buyer actually faces is the solid pieces only. Notice that the \$9 curve lies wholly below the \$10 curve, because a lower price lowers both the purchase term and the holding term. Notice also what happens at the dashed line marking the break point. The

solid path drops vertically from the upper curve to the lower one, and that drop is why the minimum cannot be found by differentiating.

The two marked points are the candidates the procedure below produces. The left one is the unconstrained minimum of the \$10 curve, feasible because it lies below 500. The right one sits exactly at the break point, because the \$9 curve is still falling when it becomes available.



Figure 3.3.: All-units discounting with $\lambda = 8000$, $k = 30$, $i = 0.30$, and a price of \$10 below 500 units and \$9 at or above. Each price has its own cost curve, drawn solid where that price applies and dotted where it does not. The realized cost function is the solid pieces, and it drops discontinuously at the break point.

Because the function is discontinuous, we cannot find the minimum by differentiating alone. There are three cases, collected in Table 3.2, and the procedure below handles all of them.

Table 3.2.: The three cases for all-units discounting. The third is why every candidate must be evaluated instead of the first feasible one being accepted.

Case	What happens	Why
Minimum at a break point q_j	The unconstrained Q_j^* lies below q_j, so the interval's best feasible point is its left endpoint	The discount reduces cost by more than the extra stock costs to hold
Minimum inside the interval	Q_j^* falls within $[q_j, q_{j+1})$	The extra carrying cost outweighs the price reduction
A local minimum only	The interval contains a minimum that a lower interval beats	Must compare across intervals, not within one

The procedure follows from Table 3.2:

1. For each price level j, compute $Q_j^* = \sqrt{2k\lambda/h_j}$.
2. Discard any Q_j^* that falls outside its own interval $[q_j, q_{j+1})$; that price is not available at that quantity.
3. For each discarded level whose price is lower than a feasible one, add the break point q_j as a candidate, since the interval's cost is decreasing up to its unconstrained minimum.
4. Evaluate Equation 3.23 at every candidate and take the smallest.

Example 3.4 (An all-units discount)**.** An item has $\lambda = 8000$ units per year, $k = \$30$ per order, and a carrying charge of $i = 0.30$ per year. The supplier prices it at \$10 per unit for orders below 500 and \$9 per unit for orders of 500 or more. This is the item drawn in Figure 3.3, and we now work its four steps.

Step 1, the candidate quantities. With $h_j = ic_j$,

$$Q_1^* = \sqrt{\frac{2(30)(8000)}{10(0.3)}} = \sqrt{160{,}000} = 400 \qquad Q_2^* = \sqrt{\frac{2(30)(8000)}{9(0.3)}} = \sqrt{177{,}778} = 422$$

Step 2, feasibility. The quantity $Q_1^* = 400$ lies in $[0, 500)$, so it is feasible at the \$10 price. However, $Q_2^* = 422$ does not lie in $[500, \infty)$, so 422 units cannot be bought at \$9. That candidate is discarded.

Step 3, the break point. Since the \$9 curve is still decreasing at 500, the best available quantity at that price is the break point itself. 500 joins the candidate list.

Step 4, evaluate.

$$TC(400) = 8000(10) + \frac{30(8000)}{400} + \frac{(0.3)(10)(400)}{2} = 80{,}000 + 600 + 600 = \$81{,}200$$

$$TC(500) = 8000(9) + \frac{30(8000)}{500} + \frac{(0.3)(9)(500)}{2} = 72{,}000 + 480 + 675 = \$73{,}155$$

Order 500. Buying a quarter more than the EOQ at the \$10 price saves \$8,045 a year, almost all of it on the purchase term. The holding cost does rise, from \$600 to \$675. However, \$75 of extra holding against \$8,000 of price reduction is not a contest.

This is the first case in Table 3.2, and it is the common one. When a discount is worth taking at all, it is usually worth taking exactly at the break point and no further.

3.8.2. Incremental Discounting

Under incremental discounting the unit price falls only on the units above each break point. An order of size Q in interval j pays c_1 for the first $q_2 - q_1$ units, c_2 for the next $q_3 - q_2$, and so on, with c_j applying only to the $Q - q_j$ units above the last break point crossed.

- Let R_j represent the cost of filling all the intervals below j completely.

That is,

$$R_j = \sum_{i=2}^{j} c_{i-1} \left(q_i - q_{i-1}\right), \qquad R_1 \equiv 0 \tag{3.24}$$

The purchase cost of an order of size Q in interval j is then

$$C(Q) = R_j + c_j \left(Q - q_j\right) \tag{3.25}$$

Unlike the all-units case, Equation 3.25 is continuous. Crossing a break point changes the slope of the purchase cost, not its level.

The consequence for the model is that the **average price per unit** now depends on Q:

$$\frac{C(Q)}{Q} = \frac{R_j}{Q} + c_j - \frac{c_j q_j}{Q}$$

Holding cost is charged on the value of what is held. The holding cost per unit becomes $i\,C(Q)/Q$ instead of ic_j. The purchase rate is $\lambda\,C(Q)/Q$ and the holding rate is $i\,C(Q)/2$, so

$$\frac{k\lambda}{Q} + \frac{\lambda C(Q)}{Q} + \frac{iC(Q)}{2} = \frac{k\lambda}{Q} + \frac{(R_j - c_j q_j)\lambda}{Q} + c_j\lambda + \frac{ic_j Q}{2} + \frac{i(R_j - c_j q_j)}{2},$$

and collecting terms in Q, we obtain

$$TC(Q) = \frac{k_j \lambda}{Q} + \frac{h_j Q}{2} + \underbrace{c_j \lambda + \frac{i\left(R_j - c_j q_j\right)}{2}}_{\text{independent of } Q} \tag{3.26}$$

where

$$k_j = R_j - c_j q_j + k, \qquad h_j = i c_j \tag{3.27}$$

This is the point of the manipulation. Notice that the Q-dependent part of Equation 3.26 has the form of Equation 3.16, with an **effective** ordering cost k_j in place of k. Thus, the ordinary square root formula applies level by level:

$$Q_j^* = \sqrt{\frac{2k_j\lambda}{h_j}} \tag{3.28}$$

The procedure is then to compute Q_j^* for each level, keep those that fall inside their own interval, evaluate Equation 3.26 at each survivor, and take the smallest. Unlike the all-units case, there are no break-point candidates, because the cost function is continuous and has no downward jumps to exploit.

Example 3.5 (An incremental discount)**.** An item has $\lambda = 3000$ units per year, $k = \$50$ per order, and $i = 0.30$ per year. The supplier's incremental schedule is given in Table 3.3, and we work the levels in turn.

Table 3.3.: An incremental discount schedule with three levels.

Interval	Break point q_j	Price c_j
1	0	\$3.00

Interval	Break point q_j	Price c_j
2	500	\$2.97
3	1500	\$2.95

The filled-interval costs. From Equation 3.24,

$$R_1 = 0, \qquad R_2 = 3.00(500-0) = 1500, \qquad R_3 = 1500+2.97(1500-500) = 4470$$

The effective ordering costs. From Equation 3.27,

$$k_1 = 0-3.00(0)+50 = 50, \quad k_2 = 1500-2.97(500)+50 = 65, \quad k_3 = 4470-2.95(1500)+50 = 95$$

The effective ordering cost **rises** with the discount level. That is the mechanism by which incremental discounting encourages larger orders. Reaching a deeper level means having already paid for everything below it, and that paid amount behaves exactly like a larger fixed charge.

The candidate quantities. From Equation 3.28 with $h_j = ic_j$,

$$Q_1^* = \sqrt{\frac{2(50)(3000)}{0.3(3.00)}} = 577.4, \quad Q_2^* = \sqrt{\frac{2(65)(3000)}{0.3(2.97)}} = 661.6, \quad Q_3^* = \sqrt{\frac{2(95)(3000)}{0.3(2.95)}} = 802.5$$

Feasibility. The quantity $Q_1^* = 577.4$ must lie in $[0, 500)$ and does not, and $Q_3^* = 802.5$ must lie in $[1500, \infty)$ and does not. Only $Q_2^* = 661.6$ lies in its own interval $[500, 1500)$.

The answer. One candidate survives. Thus, $Q^* = 661.6$ units, and from Equation 3.26

$$TC = \frac{65(3000)}{661.6} + \frac{0.891(661.6)}{2} + 2.97(3000) + \frac{0.3\,(1500 - 2.97(500))}{2} = \$9{,}501.73$$

Notice that the first two terms are \$294.74 each. The effective ordering cost substitution has turned this into an ordinary EOQ problem, so the balance property of Section 3.5.4 still holds. You should perform that check every time, because it confirms both the effective ordering cost and the quantity computed from it.

3.9. Performance Measures for the Cycle

Section 1.6 defined the service measures without reference to any model. Because the deterministic cycle is fully known, each of them has a closed form here, and computing them is a good way to see what the definitions mean.

Throughout this section we use the general cycle of Section 3.1 with its four segments.

Order frequency. One order is placed per cycle, so that

$$\overline{N} = \frac{1}{T} = \frac{\lambda}{Q} \tag{3.29}$$

Inventory turnover. From Equation 1.8, with $\bar{I}$ from Equation 3.6:

$$TO = \frac{\lambda}{\bar{I}}$$

Fraction of time out of stock. The system is short over T_4 and T_1. Thus, from the stockout indicator of Equation 1.9,

$$\overline{SO} = \frac{T_1 + T_4}{T} = \frac{\dfrac{\hat{B}}{p-\lambda} + \dfrac{\hat{B}}{\lambda}}{\dfrac{Q}{\lambda}} \tag{3.30}$$

Ready rate. Its complement, $\overline{RR} = 1 - \overline{SO}$.

Rate of unfilled demand. Demand arrives at a constant rate, so the rate at which it arrives to an empty shelf is $\lambda\overline{SO}$. That is λ_ℓ of Equation 1.3, computable exactly here.

Average customer wait. From Equation 1.6, $\overline{W} = \bar{B}/\lambda$.

Example 3.6 (Performance of the general cycle)**.** Recall the cycle of Example 3.1, with $\lambda = 100$, $p = 250$, $Q = 400$, and $\hat{B} = 30$, giving $T = 4$ years, segments $0.2, 1.4, 2.1, 0.3$, and averages $\bar{I} = 91.875$ and $\bar{B} = 1.875$ units. We compute each measure in turn.

$$\overline{N} = \frac{100}{400} = 0.25 \text{ orders per year}$$

$$TO = \frac{100}{91.875} = 1.09 \text{ turns per year}$$

$$\overline{SO} = \frac{0.2 + 0.3}{4} = 0.125, \qquad \overline{RR} = 0.875$$

$$\lambda\overline{SO} = 12.5 \text{ units per year arrive to an empty shelf}$$

$$\overline{W} = \frac{1.875}{100} = 0.01875 \text{ year} = 6.8 \text{ days}$$

We compare the last two. The system is out of stock one eighth of the time, which sounds poor, and a customer who is made to wait waits about a week. Whether that is acceptable is not a question the model answers. Section 1.6.2.7 is about who does answer it.

Notice also that the ready rate of 87.5% and the fill rate are the **same** here. Demand arrives continuously at a constant rate, so the fraction of demand arriving during a stockout equals the fraction of time in stockout. You should treat that equality as a consequence of the deterministic assumption and not as a general fact. Section 1.6.2.6 gives the condition under which it survives into stochastic models.

3.10. Building These Models in a Spreadsheet

Every result in this chapter is a formula in three or four inputs, so a spreadsheet is the natural place to put it. The workbook that accompanies this chapter, `Chapter3Models.xlsx`, carries one worksheet per model, and all of them are built to the same plan. That plan is worth more than any single worksheet, because it lets someone who did not write the implementation audit it. In this section we take the plan first and then read each worksheet against it.

3.10.1. The Four-Block Layout

Each worksheet is divided into blocks, in the order given in Table 3.4.

Table 3.4.: The blocks common to every worksheet in the workbook. The separation of the evaluator from the optimizer is what makes the sensitivity analysis of Section 3.7 a matter of typing a number.

Block	Contents	Role
Inputs	λ, c, i, k, L, and p where the model has one, each with its units beside it	the only cells anyone types into

Block	Contents	Role
Intermediate calculations	quantities derived from the inputs alone, such as $h = ic$ and $1 - \lambda/p$	names the pieces the formulas are assembled from
Outputs for any Q	frequency, cycle time, average inventory, and each cost component, all evaluated at a quantity that is typed in	the **evaluator**
Outputs for Q^*	the same rows recomputed at the optimum	the **optimizer**

Two conventions run through all of it. First, every input carries a units label in the adjacent cell. That is the dimensional discipline of Section 1.4.1 enforced by the layout, because a carrying charge quoted per year cannot be dropped silently into a model whose demand rate is per month when the two labels sit side by side. Second, every quantity has a rounded companion computed with `ROUNDUP`, so the integer that would actually be ordered is visible next to the number the formula produced.

Figure 3.4 shows the plan in place on the Classic EOQ worksheet. Reading it top to bottom, rows 1 to 7 are the input block, with every rate carrying its unit in the cell to its right. Rows 9 to 22 are the evaluator, which computes every quantity at whatever order quantity is typed into it. Rows 24 to 35 are the optimizer, the same list of rows recomputed at Q^*. Notice that column D repeats several quantities as `ROUNDUP` companions, so the integer that would actually be ordered sits beside the number the formula produced.

Notice the three rows that join the two output blocks. They are the whole of Section 3.7 built into the sheet: the ratio of the evaluated cost to the

	A	B	C	D
1	**Inputs**			
2	Demand per time (λ)	100	(units/time)	
3	Cost per unit (c)	8	($)	
4	Holding interest rate (i)	0.020833	($/$/time)	
5	Cost per order (K)	20	($/order)	
6	Holding cost per unit (h=i*c)	0.166667	($/unit/time)	
7	Lead time (τ)	0.25	(time)	
8				
9	**Outputs for any Q**			
10	Order Quantity (Q)	154.9193	(units)	155
11	Ordering Frequency (λ/Q)	0.645497	(orders/time)	
12	Cycle time (Q/λ)	1.549193	time	
13	Average Inventory Onhand (I)	77.45967	(units)	
14	Purchase cost per time (λ*c)	$800.00	($/time)	
15	Ordering cost per time	$12.91	($/time)	
16	Holding cost per time	$12.91	($/time)	
17	Ordering + Holding Cost per time	$25.82	($/time)	
18	Total Cost per time	$825.82	($/time)	
19	Reorder point (R)	25	(units)	25
20	Ratio of cost to optimal cost	1		
21	Cost difference from optimal	0	$/time	
22	Relative Cost Error from Optimal	0		
23				
24	**Outputs for EOQ (Q*)**			Rounded
25	Economic Order Quantity (Q*)	154.9193	(units)	155
26	Optimal OC + HC at Q*	$25.82	($/time)	
27	Optimal Ordering Frequency (λ/Q*)	0.645497	(orders/time)	
28	Optimal Cycle time (Q*/λ)	1.549193	time	
29	Optimal Average Inventory Onhand (I)	77.45967	(units)	
30	Purchase cost per time (λ*c)	$800.000	($/time)	
31	Optimal Ordering cost per time	$12.910	($/time)	
32	Optimal Holding cost per time	$12.910	($/time)	
33	Optimal Ordering + Holding Cost	$25.820	($/time)	
34	Total Cost per time	$825.820	($/time)	
35	Reorder point (R)	25	(units)	25

Figure 3.4.: The Classic EOQ worksheet. Rows 1 to 7 hold the inputs and their units, rows 9 to 22 the evaluator, and rows 24 to 35 the optimizer. Column D carries the rounded companions. The three rows below the evaluator report no penalty here because the evaluator has been given Q^* itself.

optimal cost, the difference, and the relative error. In Figure 3.4 they read 1, 0, and 0, because the evaluator has been given Q^* itself. Type any other quantity and the penalty appears immediately.

The worksheet labels are not identical to the notation of this book, mostly for reasons of what a spreadsheet can display. Table 3.5 gives the translation, and you should keep it at hand when reading any of the worksheet figures.

Table 3.5.: Worksheet labels and the corresponding symbols in the text.

Worksheet label	This book	Meaning
Demand per time (λ)	λ	demand rate
Cost per unit (c)	c	unit purchase or production cost
Holding interest rate (i)	i	carrying charge
Cost per order (K)	k	fixed cost of an order or setup
Holding cost per unit ($h = i * c$)	h	holding cost rate
Lead time (τ)	L	replenishment lead time
Production rate (P)	p	production rate
Average Inventory Onhand (I)	$\bar{I}$	average on-hand level
Max On Hand Inventory (H)	$\hat{I}$	peak on-hand level
Reorder point (R)	r	reorder point

Example 3.7 (Reading the classic EOQ worksheet)**.** The Classic EOQ worksheet of Figure 3.4 is set up in **months**, with $\lambda = 100$ units per month, $c = \$8$ per unit, a carrying charge of 25% per year entered as $i = 0.25/12$ per month, $k = \$20$ per order, and $L = 0.25$ month. We follow the sheet block by block.

The intermediate cell gives the holding cost rate,

$$h = ic = \frac{0.25}{12}(8) = \$0.16667 \text{ per unit per month}$$

The optimizer block then applies Equation 3.17, Equation 3.19, and Equation 3.18:

$$Q^* = \sqrt{\frac{2(20)(100)}{0.16667}} = \sqrt{24{,}000} = 154.92 \text{ units}, \qquad T^* = \frac{154.92}{100} = 1.5492 \text{ months}$$

$$\frac{k\lambda}{Q^*} = \frac{hQ^*}{2} = \$12.91 \text{ per month}, \qquad TC^* = 25.82 + (8)(100) = \$825.82 \text{ per month}$$

The two cost components agree to five figures. That is the balance check of Section 3.5.4, performed by the sheet itself and not by the reader.

The evaluator. No one orders 154.92 units. Typing the `ROUNDUP` value 155 into the evaluator returns a relative cost error of 1.4×10^{-7}, effectively nothing. Typing 150 or 160 returns 0.052% in either case, about a penny a month on a relevant cost of \$25.82. Typing 200 returns 3.3%. The three sensitivity rows turn Equation 3.22 into an experiment instead of an algebraic exercise.

Why the units column matters. Suppose the annual charge of 0.25 had been entered directly against a monthly demand rate. Then h would have been twelve times too large, and Q^* would have come out as 44.7 units, an error of a factor of $\sqrt{12}$. Nothing in the arithmetic would have signaled it. The units label beside the cell is the only guard, so you should never omit it.

3.10.2. The Reorder Point Cell

One cell in the worksheet deserves attention on its own. The reorder point is computed as

```
= MOD(lead time, cycle time) * demand rate
```

which is Equation 3.21 written in the one function that already does the work. `MOD(L, T)` returns $L - \lfloor L/T \rfloor T$, i.e. the part of the lead time not covered by orders already outstanding. When $L < T$ the floor is zero and the expression collapses to λL. Thus, the single cell handles both cases of Section 3.6 without an `IF`.

Figure 3.5 shows the same worksheet with its formulas displayed instead of its values, the view to use when auditing someone else's sheet. Reading down the optimizer block, every formula there is one of the closed forms of Section 3.5.4, written once and referring only to the input and intermediate cells above it. Notice the reorder point cell, which is the single `MOD` call discussed above.

Notice one further detail that only the formula view reveals. The order quantity cell of the evaluator holds a reference to the optimizer's Q^* and not a typed number. The evaluator opens at the optimum, and typing over that one cell is the intended way to use it.

Example 3.8 (A lead time longer than the cycle)**.** Recall the item of Example 3.7, whose cycle is $T^* = 1.5492$ months, and suppose the supplier now quotes a lead time of $L = 4$ months.

The naive calculation gives $\lambda L = 100(4) = 400$ units, more than twice the order quantity, so it cannot be a position at which to order. The worksheet cell instead gives

$$\left\lfloor \frac{4}{1.5492} \right\rfloor = \lfloor 2.582 \rfloor = 2, \qquad 4 - 2(1.5492) = 0.9016 \text{ month}, \qquad r = 100(0.9016) = 90.16 \text{ units}$$

Two replenishment orders are outstanding at the moment the third is placed, and they cover $2T$ of the four months. Only the remaining 0.9016 month has to be covered by stock on hand. Thus, the order is placed when the inventory **position** reaches 90.16 units. Recall that Section 1.3.3

	A	B	C	D
1	**Inputs**			
2	Demand per time (λ)	100	(units/time)	
3	Cost per unit (c)	8	($)	
4	Holding interest rate (i)	=0.25/12	($/$/time)	
5	Cost per order (K)	20	($/order)	
6	Holding cost per unit (h=i*c)	=B3*B4	($/unit/time)	
7	Lead time (τ)	0.25	(time)	
8				
9	**Outputs for any Q**			
10	Order Quantity (Q)	=B25	(units)	=ROUNDUP(B10,0)
11	Ordering Frequency (λ/Q)	=B2/B10	(orders/time)	
12	Cycle time (Q/λ)	=1/B11	time	
13	Average Inventory Onhand (I)	=B10/2	(units)	
14	Purchase cost per time (λ*c)	=B2*B3	($/time)	
15	Ordering cost per time	=B11*B5	($/time)	
16	Holding cost per time	=B6*B13	($/time)	
17	Ordering + Holding Cost per time	=B15+B16	($/time)	
18	Total Cost per time	=B17+B14	($/time)	
19	Reorder point (R)	=MOD(B7,B12)*B2	(units)	=ROUNDUP(B19,0)
20	Ratio of cost to optimal cost	=(B17/B33)		
21	Cost difference from optimal	=B17-B33	$/time	
22	Relative Cost Error from Optimal	=B21/B33		
23				
24	**Outputs for EOQ (Q*)**			Rounded
25	Economic Order Quantity (Q*)	=SQRT(2*B5*B2/B6)	(units)	=ROUNDUP(B25,0)
26	Optimal OC + HC at Q*	=SQRT(2*B5*B2*B6)	($/time)	
27	Optimal Ordering Frequency (λ/Q*)	=B2/B25	(orders/time)	
28	Optimal Cycle time (Q*/λ)	=1/B27	time	
29	Optimal Average Inventory Onhand (I)	=B25/2	(units)	
30	Purchase cost per time (λ*c)	=B2*B3	($/time)	
31	Optimal Ordering cost per time	=B27*B5	($/time)	
32	Optimal Holding cost per time	=B29*B6	($/time)	
33	Optimal Ordering + Holding Cost	=B31+B32	($/time)	
34	Total Cost per time	=B33+B30	($/time)	
35	Reorder point (R)	=MOD(B7,B28)*B2	(units)	=ROUNDUP(B35,0)

Figure 3.5.: The Classic EOQ worksheet with formulas displayed. The reorder point is a single `MOD` call, the order quantity in the evaluator is a reference to the optimum below it, and every formula in the optimizer block is one of the closed forms of Section 3.5.4.

distinguishes the position carefully from the on-hand level, and this is a case where the distinction decides the answer.

Sweeping L upward makes the point visible. For example, at $L = 2$ months the same cell returns $r = 45.08$ units, far less than $\lambda L = 200$. The reorder point is not monotone in the lead time. It climbs to Q and drops back to zero once per cycle length of lead time.

3.10.3. Reducing the Production Model to the Order Model

The Classic POQ worksheet is the clearest illustration of what Section 3.5 is claiming. Rather than coding Equation 3.14 as a new formula, the worksheet computes three intermediate cells,

$$\frac{\lambda}{p}, \qquad 1 - \frac{\lambda}{p}, \qquad h' = h\left(1 - \frac{\lambda}{p}\right)$$

and then applies the **ordinary** EOQ formula to h'. The production model is not implemented at all. It is the order model with a different holding cost rate. The same substitution turns Equation 3.18 into the production version, so one set of formulas serves both worksheets.

The worksheet also guards its own assumption. A cell beside the production rate reads

```
= IF(demand rate >= production rate,
     "Infeasible production rate", "Production rate feasible")
```

because the cycle of Section 3.1 exists only when $p > \lambda$. Without the guard, $p = \lambda$ produces $h' = 0$ and a division by zero, while $p < \lambda$ produces a negative holding cost and a square root of a negative number. Neither failure explains itself, so the sheet explains it first.

Figure 3.6 shows both the substitution and the guard. Reading the intermediate block downward, the three cells build λ/p, then $1-\lambda/p$, then h', each referring only to the cell above it. Notice that the optimizer block below contains no production formulas at all, only the ordinary EOQ formulas pointed at h'. The guard sits beside the production rate input, where someone typing a bad rate will see it.

	A	B	C	D
1	**Inputs**			
2	Demand per time (λ)	2500	(units/time)	
3	Production rate (P)	10000	(units/time)	=IF(B2>=B3, "Infeasible production rate", "Production rate feasible")
4	Cost per unit (c)	2	($)	
5	Holding interest rate (i)	0.3	($/$/time)	
6	Setup Cost per order (K)	50	($/order)	
7	Holding cost per unit (h=i*c)	=B4*B5	($/unit/time)	
8				
9	**Intermediate Calculations**			
10	Demand/Production Rate Ratio (λ/P)	=B2/B3		
11	1- (λ/P)	=1-B10		
12	h prime = h*(1-λ/P)	=B7*B11		
13				
14	**Outputs for any Q**			
15	Production Quantity (Q)	=B30	(units)	=ROUNDUP(B15,0)
16	Cycle time (Q/λ)	=1/B20	(time)	
17	Time spent in production (Q/P)	=B15/B3	(time)	
18	Time Spent using production	=B16-B17	(time)	
19	Max On Hand Inventory (H)	=B15*B11	(units)	
20	Ordering Frequency (λ/Q)	=B2/B15	(orders/time)	
21	Average Inventory Onhand (I)	=B19/2	(units)	
22	Ordering cost per time	=B20*B6	($/time)	
23	Holding cost per time	=B7*B21	($/time)	
24	Ordering + Holding Cost per time	=B22+B23	($/time)	
25	Ratio of cost to optimal cost	=(B24/B40)		
26	Cost difference from optimal	=B24-B40	$/time	
27	Relative Cost Error from Optimal	=B26/B40		
28				
29	**Outputs for POQ (Q*)**			Rounded
30	Economic Production Quantity (Q*)	=SQRT(2*B6*B2/B12)	(units)	=ROUNDUP(B30,0)
31	Optimal OC + HC at Q*	=SQRT(2*B6*B2*B12)	($/time)	
32	Cycle time (Q/λ)	=B30/B2	(time)	
33	Time spent in production (Q/P)	=B30/B3	(orders/time)	
34	Time Spent using production	=B32-B33	(time)	
35	Max On Hand Inventory (H)	=B30*B11	(units)	
36	Ordering Frequency (λ/Q)	=1/B32	(units)	
37	Average Inventory Onhand (I)	=B35/2	($/time)	
38	Ordering cost per time	=B36*B6	($/time)	
39	Holding cost per time	=B37*B7	($/time)	
40	Ordering + Holding Cost per time	=B38+B39		

Figure 3.6.: The production worksheet with formulas displayed. The three intermediate cells build h', the optimizer then applies the ordinary EOQ formulas to it, and the cell beside the production rate carries the feasibility guard.

Example 3.9 (Reading the production worksheet)**.** The worksheet is set up with $\lambda = 2500$ units per year, $p = 10{,}000$ units per year, $c = \$2$ per unit, $i = 0.30$ per year, and a setup cost of $k = \$50$. We read it block by block.

The intermediate block gives

$$h = ic = 0.60, \qquad \frac{\lambda}{p} = 0.25, \qquad 1-\frac{\lambda}{p} = 0.75, \qquad h' = 0.60(0.75) = \$0.45$$

The optimizer block then runs the ordinary EOQ formula on h':

$$Q^* = \sqrt{\frac{2(50)(2500)}{0.45}} = 745.36 \text{ units}, \qquad \frac{k\lambda}{Q^*} = \frac{h'Q^*}{2} = \$167.71 \text{ per year}$$

The cycle quantities then follow from Equation 3.2 with $\hat{B} = 0$ and from Equation 3.1:

$$\hat{I} = 745.36(0.75) = 559.02, \quad \bar{I} = 279.51, \quad T = \frac{745.36}{2500} = 0.2981, \quad T_p = \frac{745.36}{10{,}000} = 0.0745$$

Notice that the machine runs for 0.0745 year of every 0.2981 year cycle, i.e. a quarter of the time, which is λ/p as it must be. You should perform that check every time, because it catches an error in either the cycle length or the replenishment period.

Figure 3.7 shows the completed sheet with its values. Reading across the two output blocks, the ordering and holding costs agree at \$167.71, the balance check of Section 3.5.4 surviving the substitution intact.

For comparison, ignoring the finite production rate entirely gives $\sqrt{2(50)(2500)/0.60} = 645.50$ units. Spreading the receipt over time lowers

	A	B	C	D
1	**Inputs**			
2	Demand per time (λ)	2500	(units/time)	
3	Production rate (P)	10000	(units/time)	Production rate feasible
4	Cost per unit (c)	2	($)	
5	Holding interest rate (i)	0.3	($/$/time)	
6	Setup Cost per order (K)	50	($/order)	
7	Holding cost per unit (h=i*c)	0.6	($/unit/time)	
8				
9	**Intermediate Calculations**			
10	Demand/Production Rate Ratio (λ/P)	0.25		
11	1- (λ/P)	0.75		
12	h prime = h*(1-λ/P)	0.45		
13				
14	**Outputs for any Q**			
15	Production Quantity (Q)	745.3559925	(units)	746
16	Cycle time (Q/λ)	0.298142397	(time)	
17	Time spent in production (Q/P)	0.074535599	(time)	
18	Time Spent using production	0.223606798	(time)	
19	Max On Hand Inventory (H)	559.0169944	(units)	
20	Ordering Frequency (λ/Q)	3.354101966	(orders/time)	
21	Average Inventory Onhand (I)	279.5084972	(units)	
22	Ordering cost per time	$167.71	($/time)	
23	Holding cost per time	$167.71	($/time)	
24	Ordering + Holding Cost per time	$335.41	($/time)	
25	Ratio of cost to optimal cost	1		
26	Cost difference from optimal	0	$/time	
27	Relative Cost Error from Optimal	0		
28				
29	**Outputs for POQ (Q*)**			Rounded
30	Economic Production Quantity (Q*)	745.3559925	(units)	746
31	Optimal OC + HC at Q*	335.4101966	($/time)	
32	Cycle time (Q/λ)	0.298142397	(time)	
33	Time spent in production (Q/P)	0.074535599	(orders/time)	
34	Time Spent using production	0.223606798	(time)	
35	Max On Hand Inventory (H)	559.0169944	(units)	
36	Ordering Frequency (λ/Q)	3.354101966	(units)	
37	Average Inventory Onhand (I)	279.5084972	($/time)	
38	Ordering cost per time	$167.71	($/time)	
39	Holding cost per time	$167.71	($/time)	
40	Ordering + Holding Cost per time	$335.41		

Figure 3.7.: The production worksheet evaluated. The intermediate block reduces the model to an ordinary EOQ in $h' = 0.45$, and the ordering and holding costs agree at \$167.71, the balance check of Section 3.5.4 surviving the substitution.

the average inventory for a given lot, which makes larger lots affordable. Thus, the production quantity exceeds the order quantity, by the factor $1/\sqrt{1-\lambda/p} = 1/\sqrt{0.75} = 1.155$.

3.10.4. Discount Schedules as Columns

The discount models of Section 3.8 differ from the others in that they require a comparison across levels instead of a single formula. Both worksheets handle this the same way. The price schedule is entered as a table, one row per level, and each step of the procedure becomes a computed column.

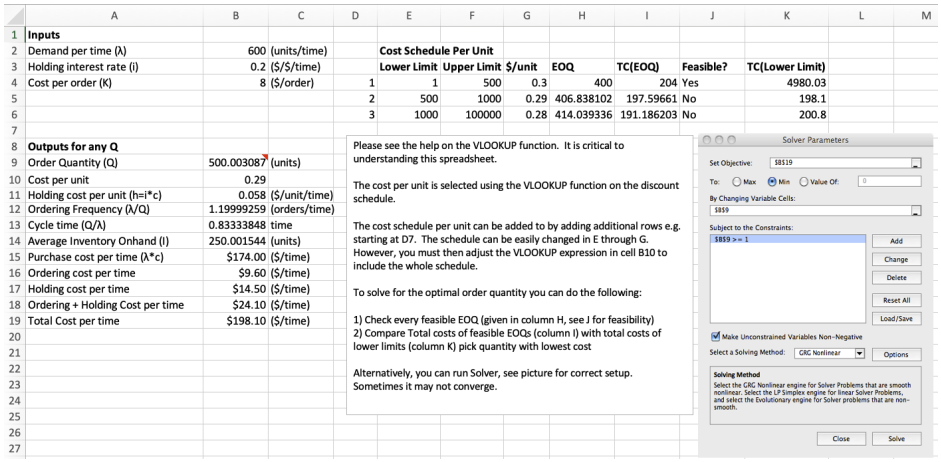
For all-units discounting the columns are those of Table 3.6.

Table 3.6.: The all-units worksheet's computed columns, one per step of the four-step procedure. The answer is the smallest cost among the feasible EOQ entries and the break points of the infeasible ones.

Column	Formula	Step of Section 3.8.1
Lower limit, Upper limit, \$/unit	typed in	the schedule q_j and c_j
EOQ	$\sqrt{2k\lambda/(ic_j)}$	step 1, the candidate at each price
TC(EOQ)	$c_j\lambda + \sqrt{2k\lambda ic_j}$	the cost of that candidate
Feasible?	`IF(AND(lower <= EOQ, EOQ <= upper), "Yes", "No")`	step 2, is the price available at that quantity
TC(Lower Limit)	$c_j\lambda + k\lambda/q_j + ic_jq_j/2$	step 3, the cost at the break point

Laid out this way the whole procedure is visible at once. Thus, the third case of Table 3.2, in which a lower interval beats a feasible candidate, cannot be missed by stopping early.

Figure 3.8 shows the result. The schedule occupies columns E through G, with one row per price level, and the four computed columns to its right carry out the procedure one step per column. Reading across a row gives that level's candidate quantity, its cost, whether the price is actually available at that quantity, and the cost at the break point if it is not. Notice the evaluator block at the left, which prices whatever quantity is chosen and is the same evaluator that appears on every other worksheet in the workbook.



	A	B	C
1	**Inputs**		
2	Demand per time (λ)	600	(units/time)
3	Holding interest rate (i)	0.2	($/$/time)
4	Cost per order (K)	8	($/order)
5			
6			
7			
8	**Outputs for any Q**		
9	Order Quantity (Q)	500.003087	(units)
10	Cost per unit	0.29	
11	Holding cost per unit (h=i*c)	0.058	($/unit/time)
12	Ordering Frequency (λ/Q)	1.19999259	(orders/time)
13	Cycle time (Q/λ)	0.83333848	time
14	Average Inventory Onhand (I)	250.001544	(units)
15	Purchase cost per time (λ*c)	$174.00	($/time)
16	Ordering cost per time	$9.60	($/time)
17	Holding cost per time	$14.50	($/time)
18	Ordering + Holding Cost per time	$24.10	($/time)
19	Total Cost per time	$198.10	($/time)
20			
21			
22			
23			
24			
25			
26			
27			

	D	E	F	G	H	I	J	K
1								
2		**Cost Schedule Per Unit**						
3		**Lower Limit**	**Upper Limit**	**$/unit**	**EOQ**	**TC(EOQ)**	**Feasible?**	**TC(Lower Limit)**
4	1	1	500	0.3	400	204	Yes	4980.03
5	2	500	1000	0.29	406.838102	197.59661	No	198.1
6	3	1000	100000	0.28	414.039336	191.186203	No	200.8

Please see the help on the VLOOKUP function. It is critical to understanding this spreadsheet.

The cost per unit is selected using the VLOOKUP function on the discount schedule.

The cost schedule per unit can be added to by adding additional rows e.g. starting at D7. The schedule can be easily changed in E through G. However, you must then adjust the VLOOKUP expression in cell B10 to include the whole schedule.

To solve for the optimal order quantity you can do the following:

1) Check every feasible EOQ (given in column H, see J for feasibility)
2) Compare Total costs of feasible EOQs (column I) with total costs of lower limits (column K) pick quantity with lowest cost

Alternatively, you can run Solver, see picture for correct setup. Sometimes it may not converge.

Figure 3.8.: The all-units worksheet. The schedule occupies columns E through G and the four computed columns to its right carry out the procedure of Section 3.8.1 one step per column. The evaluator at the left prices the chosen quantity.

Notice one detail of the schedule table. The first lower limit is entered as 1 and not 0, because a zero would leave the `VLOOKUP` without a match and would make the λ/Q terms infinite. The model of Section 3.8 sets $q_1 = 0$.

Nothing depends on the difference, because an order of size zero is never a candidate.

The worksheet offers a second route to the same answer. You can name the total cost cell as an objective, the order quantity cell as the variable, and minimize with a solver. It is the less reliable route, for a reason the model has already established. Equation 3.23 is discontinuous, dropping to a lower curve at each break point, and that a gradient method assumes it is not. Started inside one interval, such a method finds that interval's minimum and stops, the third case of Table 3.2. Thus, the columns enumerate the candidates and cannot miss one, while a solver searches and can.

Example 3.10 (Reading the all-units worksheet)**.** The worksheet has $\lambda = 600$ units per time, $k = \$8$ per order, and $i = 0.20$, together with the three level schedule below. The computed columns of Figure 3.8 fill in as shown.

Level	q_j	c_j	$h_j = ic_j$	Q_j^*	Feasible?	$TC(Q_j^*)$	$TC(q_j)$
1	0	\$0.30	0.060	400.00	Yes	204.00	
2	500	\$0.29	0.058	406.84	No		198.10
3	1000	\$0.28	0.056	414.04	No		200.80

The candidates are therefore $Q = 400$ at \$204.00, $Q = 500$ at \$198.10, and $Q = 1000$ at \$200.80. **Order 500.**

Level 3 is the instructive row. Its price is the lowest available, and evaluating its unconstrained candidate gives \$191.19, which is lower than any figure the table reports. However, 414 units cannot be bought at \$0.28, and buying the 1000 units required to reach that price costs more in holding than the price cut returns.

Notice what the two tempting shortcuts would have done. Evaluating candidates in price order and stopping at the first feasible one would have

chosen level 1 and \$204.00. Evaluating the lowest number in the table would have chosen a quantity that is not for sale. Every candidate must be evaluated, and the columns enforce that.

The incremental worksheet needs one more column, and the way it is built is neater than Equation 3.24. Rather than accumulating the filled-interval cost R_j and then subtracting $c_j q_j$, the sheet computes a single **fixed cost** column recursively.

- Let H_j represent that fixed cost column at level j.

That is,

$$H_1 = 0, \qquad H_j = \left(c_{j-1} - c_j\right) q_j + H_{j-1} \tag{3.31}$$

Equation 3.31 is exactly $R_j - c_j q_j$ from Equation 3.27, obtained without ever forming R_j. Each step adds the amount saved on the q_j units already bought by dropping to the new price, the same accounting seen from the other side. The effective ordering cost is then $k_j = H_j + k$.

Figure 3.9 shows the sheet. The fixed cost column sits at the right of the schedule table, and the price and fixed cost applying at a given Q are retrieved from that table with a `VLOOKUP`. Notice what is absent. Unlike the all-units sheet, this one carries no candidate columns, so the evaluator at the left is the whole of it, and the comparison across levels has to be made by the reader.

Example 3.11 (Reading the incremental worksheet, and a warning)**.** We keep the same inputs and the same break points as Example 3.10, now under an incremental schedule. The fixed cost column, from Equation 3.31, is

$$H_1 = 0, \qquad H_2 = (0.30{-}0.29)(500){+}0 = 5, \qquad H_3 = (0.29{-}0.28)(1000){+}5 = 15$$

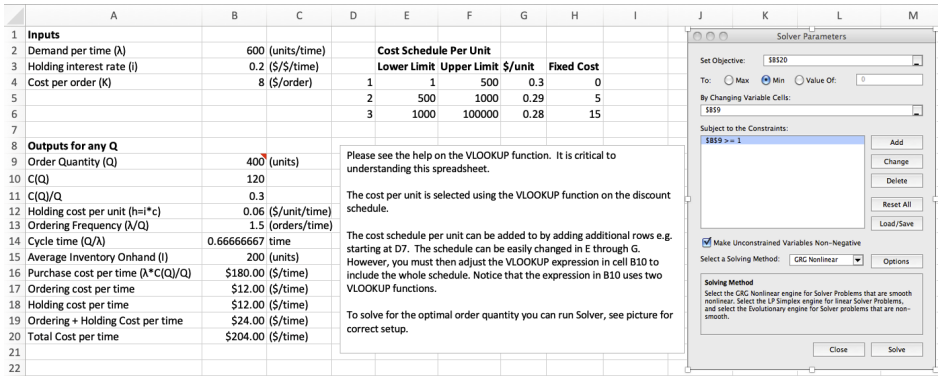


Figure 3.9.: The incremental worksheet. The fixed cost column at the right of the schedule accumulates $R_j - c_j q_j$ recursively. Unlike the all-units sheet, this one carries no candidate columns, so the evaluator at the left is the whole of it.

Thus, the effective ordering costs are $k_1 = 8$, $k_2 = 13$, and $k_3 = 23$, and Equation 3.28 gives

Level	k_j	h_j	Q_j^*	Own interval	Feasible?	TC
1	8	0.060	400.00	$[0, 500)$	Yes	204.00
2	13	0.058	518.62	$[500, 1000)$	Yes	204.58
3	23	0.056	702.04	$[1000, \infty)$	No	

Two candidates survive feasibility, and the smaller one wins. **Order 400 at a cost of $204.00 per unit time.**

⚠ An evaluator is not an optimizer

Either surviving candidate can be typed into the evaluator block, and both return a plausible number. That is, 400 returns $204.00 and 518.62 returns $204.58, because the block prices whatever quantity it is given.
Because the incremental worksheet carries no candidate columns, a solver run against the total cost cell is the only automated route on it, and that route is unsafe here, for a reason you should understand. The incremental cost function is continuous, so it has none of the jumps that defeat a solver in the all-units case. However, it is kinked at every break point, and each interval carries its own minimum. Both 400 and 518.62 are local minima. A gradient method started near either one reports that one and stops, so only enumeration distinguishes them.
The margin here is 58 cents per unit time against $204, i.e. about a quarter of a percent, the sort of difference that survives unnoticed. The check that catches it costs nothing. Level 2 is not the only feasible level, so its candidate cannot be accepted without evaluating level 1 as well.

3.10.5. What the Worksheets Do Not Do

Three limits need stating plainly, because each of them is a place where a correct spreadsheet still gives a wrong answer.

They evaluate, they do not choose. Every worksheet prices an entered quantity faithfully. Only the all-units sheet enumerates alternatives, and only because someone laid the enumeration out in columns. A solver can be pointed at any of them. However, the two discount models are precisely where a solver is least trustworthy, the case Example 3.11 makes.

They do not know their own assumptions. The production worksheet

guards $p > \lambda$ and nothing else. Nothing checks that demand is actually constant, that the lead time is actually known, or that backorders are actually permitted. Recall that those judgments belong to Section 1.2.2, and you make them before the workbook is opened.

They round after the fact. The `ROUNDUP` column reports the integer that would be ordered, but the cost figures are computed at the unrounded quantity. For the classic EOQ item that gap is 1.4×10^{-7} of the relevant cost and can be ignored. For an item ordered in cases of 48, as in Exercise 3.5, it cannot. Thus, the evaluator block exists precisely so that the rounded quantity can be entered and priced.

3.11. Designing the Software

Recall that Section 3.10.5 ended with three things a worksheet does not do. It evaluates and does not choose, it does not know its own assumptions, and it rounds after the fact. Those are reasons to write a program. They are not a design.

A worksheet argues for itself. The blocks sit in an order you can see, and a formula is a cell you can click on. A program's structure is a set of decisions about which classes exist and what each class is responsible for, and a reader handed the finished code sees only the result. The decisions for the six models of this chapter come first, and Section 3.12 then shows what to type.

Four decisions carry most of the design, and each one is a question that returns in any model you write. We take them in turn.

3.11.1. What Changes, and What Must Not

A worksheet is a thing you edit. You change the ordering cost, the sheet recalculates, and you compare what you now see against the numbers from

a minute ago. Both halves of that matter. The inputs have to be editable, and the answer you already took down has to stay where it was.

Those two requirements pull in opposite directions. We meet them with three types.

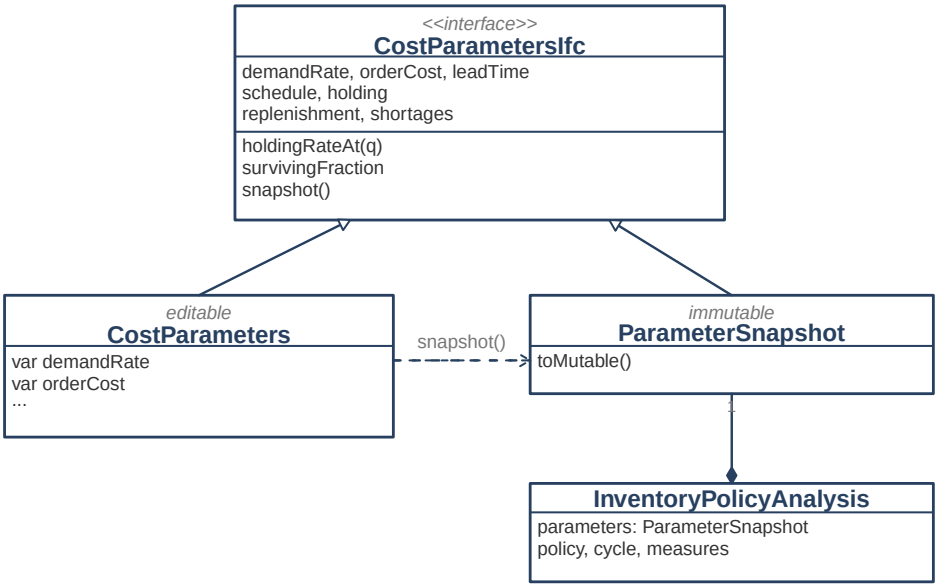


Figure 3.10.: One concept in three types. The interface is what everything downstream is allowed to see, the editable class is the sheet you change, and the snapshot is the printout. Notice that an analysis holds a snapshot by composition, so the record cannot outlive or drift from the result that carries it.

Figure 3.10 illustrates the three answers to one question. `CostParameters` is the sheet. Every property can be assigned, and every property validates on assignment with the same predicate the constructor applies, so no invalid parameter set ever exists. `ParameterSnapshot` is the printout, unchangeable once taken. `CostParametersIfc` is the read-only view, and

it is what every model, every cycle and every cost calculation depends on.

The read-only view is the decision doing the work. A model receives the interface, so a model cannot change the parameters it was given. Thus, the mutability is confined to the one place a reader utilizes it, and nothing below the input layer has to be written defensively.

The snapshot makes the arrangement safe.

```
val analysis = EconomicOrderQuantity.optimize(sheet)   //
 ↪  sheet.orderCost is 20.0
sheet.orderCost = 40.0
analysis.parameters.orderCost                          //
 ↪  still 20.0
```

Every result carries the values that produced it. Notice that the sweep in Section 3.12 assigns `sheet.leadTime = lead` inside a loop and keeps every analysis. That sweep is sound because each analysis froze its own inputs before the next assignment. Notice also that `ParameterSnapshot` is itself a `CostParametersIfc`, so a result can be re-analyzed from its own record.

You should take the general rule from this. When something must be editable and something must be stable, do not compromise between them on one type. Separate what can change from what has been recorded, and make the dependency run from the second to the first.

3.11.2. One Pipeline, Six Models

Section 3.4 derives one model and Section 3.5 obtains five more by imposing assumptions on it. The six therefore agree about almost everything. They agree about the geometry of Section 3.1, the cost assembly of Section 3.3,

the reorder point of Section 3.6, and the service measures of Section 3.9. They disagree about one thing, the policy.

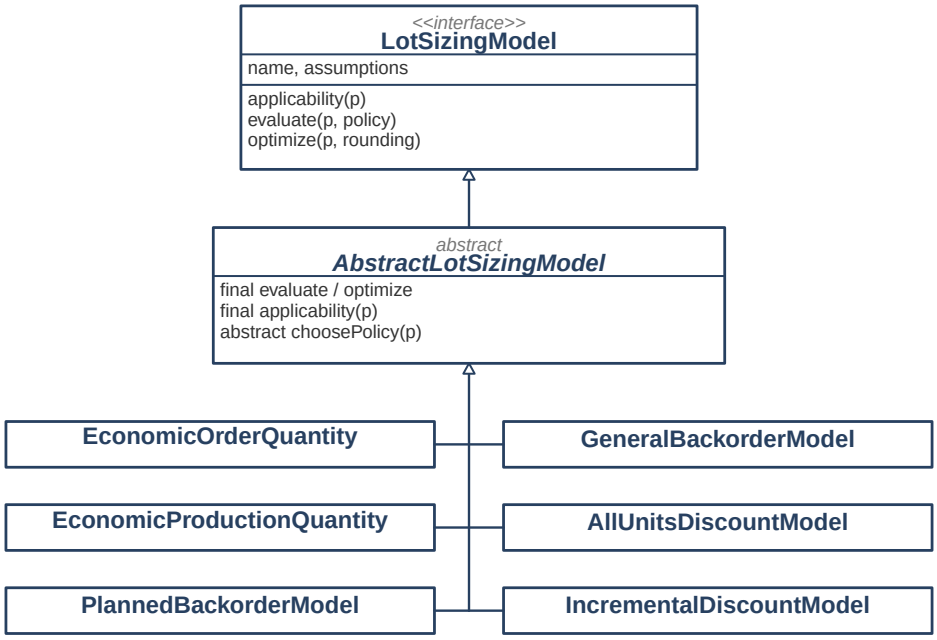


Figure 3.11.: One interface, one pipeline and six models. Everything the six share is written once in the abstract class, where `evaluate` and `optimize` are final, and each model supplies a single method that chooses a policy. Notice that no arrow joins the six to one another, which is the point of the next paragraph.

Figure 3.11 illustrates the arrangement. We write the shared part once and leave the varying part open.

```
abstract class AbstractLotSizingModel(override val name:
 ↪  String) : LotSizingModel {

    /** The policy this model chooses for these parameters,
     ↪  and how it chose it. */
```

```
    protected abstract fun choosePolicy(parameters:
     ↪  CostParametersIfc): ChosenPolicy

    final override fun optimize(
        parameters: CostParametersIfc,
        rounding: QuantityRounding,
    ): InventoryPolicyAnalysisIfc {
```

`optimize` and `evaluate` are `final`. That is, a model may decide what to order, and it may not decide what a cycle is, what it costs, or when to reorder. Those belong to the chapter and not to any one model, so a model that overrode them would contradict Section 3.1 instead of specializing it.

Table 3.1 obtains the classical economic order quantity from the general model as the replenishment rate and the backorder cost both grow without bound. The obvious move is to make `EconomicOrderQuantity` a subclass of `GeneralBackorderModel`. The code does not do so.

A subtype must be usable wherever its parent is, and the classical model is not. It has no backorder cost to be given, and handing it one is the case Section 3.5.2 warns about. What is true is a statement about numbers, that one model's answer approaches another's in a limit, and a statement about numbers belongs where numbers can be checked. Take the limit, watch the quantities converge, and the claim is settled. Thus, the relation between the six models is real, it is checkable, and it is not inheritance.

You should carry that distinction past this chapter. Before making B a subclass of A, ask whether every context that accepts an A would accept a B. In the mathematics, "B is a special case of A" is a different claim.

3.11.3. Sealed Types for Closed Sets

Four of the inputs are not numbers. Replenishment is instantaneous or at a rate. Shortages are forbidden or backordered. Holding cost is a charge on value or a rate. A schedule is flat, all-units or incremental. Each one is a small closed set of alternatives, and we write each as a sealed hierarchy.

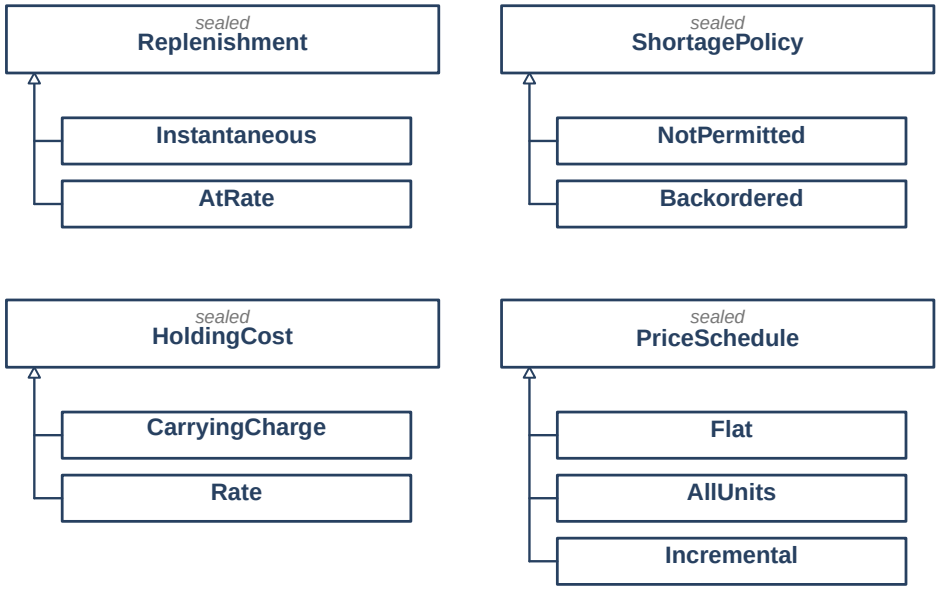


Figure 3.12.: The four inputs that are kinds rather than numbers. Each set is closed by the mathematics of this chapter, which is what makes sealing them honest: a seventh kind of price schedule would be a change to Section 3.8 before it was a change to any code.

Figure 3.12 illustrates the four. Sealing buys one thing. The compiler knows the whole list, so it can check a decision over the kinds for completeness. That pays when something dispatches on the kind, and here something does. Table 3.1 says which model matches which assumptions, and the

code is that table.

```
fun forParameters(parameters: CostParametersIfc):
 ↪  LotSizingModel {
    val schedule = parameters.schedule
    if (schedule is AllUnits) return AllUnitsDiscountModel
    if (schedule is Incremental) return
     ↪  IncrementalDiscountModel
    return when (parameters.replenishment) {
        is Replenishment.Instantaneous -> when
         ↪  (parameters.shortages) {
            is ShortagePolicy.NotPermitted ->
             ↪  EconomicOrderQuantity
            is ShortagePolicy.Backordered ->
             ↪  PlannedBackorderModel
        }
        is Replenishment.AtRate -> when
         ↪  (parameters.shortages) {
            is ShortagePolicy.NotPermitted ->
             ↪  EconomicProductionQuantity
            is ShortagePolicy.Backordered ->
             ↪  GeneralBackorderModel
        }
    }
}
```

There is no `else`, and the code will not compile if one is needed. For example, adding a third kind of replenishment stops the build at this function, the one place in the program that has to be revisited. The compiler enforces Table 3.1, and no comment asks anybody to remember it.

`ShortagePolicy` shows a second benefit. Recall from Section 3.5.2 that forbidding shortages is not the same as charging nothing for them, since a

backorder cost of zero makes shortages free and sends the order quantity to infinity. A design with one nullable backorder cost would have to catch that at run time. Here `NotPermitted` and `Backordered(b)` are different things, and the constructor of the second requires a positive `b`. Thus, the mistake the callout warns about cannot be written down at all.

We apply one criterion, the closed set and not the tidiness. Section 4.8.5 faces the same choice for the aggregate measures of Chapter 4 and answers it the other way, because nothing there dispatches on which measure it is holding and an organization may always find a fifth thing to run short of. You should seal when the mathematics closes the list and something must visit every case, and leave the hierarchy open otherwise.

3.11.4. An Answer That Carries Its Reasoning

We come to the last decision. It concerns what comes back, and an order quantity is a number that cannot be questioned. Three types let the caller question it.

A model asked to work outside its assumptions has three replies and not two.

```
sealed interface Applicability {
    data object Applicable : Applicability
    data class NotApplicable(val reason: String) :
     ↪ Applicability
    data class Approximating(val ignored: List<String>) :
     ↪ Applicability
}
```

The third reply is the interesting one. Notice that applying the order quantity to an item with a finite production rate is a deliberate approximation, which Exercise 3.4 asks you to make, so the model allows it and the

discarded assumption rides on the result. Asking the planned backorder model for a policy when shortages are forbidden is refused outright, since nothing is left for it to compute. A boolean would treat those two alike.

Each model declares its assumptions once, and the verdict is derived from that declaration in the one place all six share. Stating an assumption in a declaration and then testing for it again by hand would write the same rule twice, in two forms free to disagree.

`QuantityChoice` records how the quantity was reached, and the routes differ. A closed form produced an interior optimum; or the stationary point lay outside the feasible region, so the answer sits on a boundary and carries a reason; or a candidate list was ranked and retained; or an unrounded optimum was rounded and both neighbors evaluated. Section 3.8 is where the record pays, since a discount answer that does not show its candidates cannot be checked against the worksheet that produced the same number.

Section 3.1.2 says the four durations account for the whole cycle, and advises checking it because the check catches most algebra errors immediately. We write the advice into the constructor.

```
        checkClose(segments.total, length, "the segments sum
↪  to the cycle length")
        checkClose(maxOnHand + maxBackorder,
↪  policy.orderQuantity * survivingFraction,
            "the peaks account for the surviving part of the
             ↪  order")
```

Those checks run whenever a cycle is built, so an identity the chapter states cannot quietly fail to hold in the program. The averages computed from the triangle areas are checked against the closed forms beside them.

The same discipline applies to arguments. Every constructor validates, and so does every function that could divide by a quantity somebody passed as zero. The failure a guard prevents is usually not a crash. A rounding rule

asked for an interval of two to the seventieth base periods computes two to the sixth, because the shift that raises two to a power keeps only the low six bits of the exponent. It returns a plausible small number. A guard at the point of construction refuses the exponent.

We can now say what a seventh model would cost. It is one object with one `choosePolicy`, one `assumptions` declaration, and one branch that the compiler demands in `forParameters`. Nothing else in the package changes. Apply that test to any design and ask what the next requirement costs.

3.12. The Same Models in Code

The worksheets of Section 3.10 and the Kotlin code that accompanies this chapter compute the same quantities from the same inputs. Section 3.11 said why the code is shaped as it is; here we make the translation. Each part takes one piece of a worksheet, says what it is in code, and shows what to type in order to get the number the sheet shows. Appendix E covers obtaining and running the project.

Table 3.7 is the map, and you can find any equivalent in one lookup.

Table 3.7.: What each part of a worksheet is called in code.

In a worksheet	In code
The **Inputs** block	`CostParameters`, built once
The units column beside each input	`timeUnit`, required at construction
Intermediate calculations	Derived properties: `survivingFraction`, `holdingRateAt`
Outputs for any Q	`evaluate(parameters, orderQuantity)`
Outputs for Q*	`optimize(parameters)`
Ratio, difference, relative error	`penaltyAgainst(reference)`

In a worksheet	In code
The `ROUNDUP` column	`roundedOrderQuantity`, `roundedReorderPoint`
`=MOD( , T) *`	`reorderPoint`
`=IF( >= P, "Infeasible", ...)`	The constructor refuses the rate
`VLOOKUP` into the price schedule	`PriceSchedule.unitCostAt`
The `Fixed Cost` column	`Incremental.fixedChargeAt`
The candidate columns	`QuantityChoice.AmongCandidates`

3.12.1. The Four Blocks as Objects

The four-block layout of Section 3.10.1 survives the translation, with each block becoming something the language already has.

Inputs become an object. `CostParameters` holds what the Inputs block holds, and it takes the time unit as an argument instead of as a label in an adjacent cell. Notice the difference that makes. The worksheet's units column is a discipline the reader has to follow, while here it is a constructor argument. Thus, an annual carrying charge cannot be dropped silently beside a monthly demand rate.

Intermediate calculations become derived properties. They are computed on demand from the inputs and cannot go stale, the behavior a spreadsheet formula already has.

The two output blocks become two operations. `evaluate` prices a quantity that is handed to it, and `optimize` finds the best one. That is the split Section 3.10.1 describes, and it is why the sensitivity rows have something to join.

The joining rows become a method. `penaltyAgainst` compares two results and returns the ratio, the difference, and the relative error of Equation 3.22.

Example 3.12 (The classic EOQ sheet in code)**.** Recall the item of Example 3.7, with $\lambda = 100$ units per month, $c = \$8$ per unit, an annual carrying charge of 25% entered as a monthly rate, $k = \$20$ per order, and $L = 0.25$ month. We build it once and ask it for the optimum.

```
val sheet = CostParameters(
    demandRate = 100.0,
    unitCost   = 8.0,
    orderCost  = 20.0,
    holding    = HoldingCost.carryingCharge(0.25 / 12.0),
    leadTime   = 0.25,
    timeUnit   = TimeUnit.MONTH,
)

val best = EconomicOrderQuantity.optimize(sheet)
println(best.report())
```

```
Economic order quantity
  Order quantity        154.9193 units  (155 rounded up)
  Cycle length          1.5492 month
  Order frequency       0.6455 orders per month
  Average on hand       77.4597 units
  Reorder point         25.0000 units  (25 rounded up)
  Ordering cost         12.91 per month
  Holding cost          12.91 per month
  Relevant cost         25.82 per month
  Purchase cost         800.00 per month
  Total cost            825.82 per month
```

Every figure is the sheet's. Notice that the rounded companions in parentheses are the `ROUNDUP` column of Figure 3.4, and that the ordering and holding costs again agree at \$12.91, the balance check of Section 3.5.4.

The evaluator, and the three rows below it. Handing a quantity to `evaluate` and comparing it against the optimum reproduces the sheet's ratio, difference, and relative error:

```
val chosen = EconomicOrderQuantity.evaluate(sheet,
 ↪  orderQuantity = 155.0)
chosen.penaltyAgainst(best)
```

Table 3.8.: Typing a quantity into the evaluator, in code. The 150 and 160 rows carry the same penalty, Equation 3.22 depending only on the ratio.

Q	Ratio	Difference	Relative error
155	1.00000014	\$0.000003	0.000014%
150	1.00052070	\$0.013444	0.052%
160	1.00052070	\$0.013444	0.052%
200	1.03279556	\$0.846778	3.280%

3.12.2. The Reorder Point

Section 3.10.2 makes the case for `MOD( , T) *  `, which handles both cases of Section 3.6 in one function with no branch. Kotlin has the same function on `Double`. Thus, the translation is direct and `reorderPoint` is one expression.

Example 3.13 (The reorder point against the lead time)**.** We sweep the lead time on the item of Example 3.12, whose cycle is 1.5492 months.

```
for (lead in listOf(0.25, 1.0, 1.6, 2.0, 3.0, 4.0)) {
    sheet.leadTime = lead
    println(EconomicOrderQuantity.optimize(sheet).
 ↪  reorderPoint)
```

```
}
```

Table 3.9.: The reorder point is not monotone in the lead time. It climbs toward the order quantity and drops back once per cycle length of lead time.

Lead time, months	0.25	1.00	1.60	2.00	3.00	4.00
Reorder point	25.00	100.00	5.08	45.08	145.08	90.16

Notice the drop between 1.00 and 1.60 months, the second order becoming outstanding. Nothing in the code tests for it, because `mod` has already done so.

Notice also what the loop does with the parameters. `sheet.leadTime = lead` edits the object in place, the way a worksheet is used. The analyses already produced are unaffected, because each one records the inputs that produced it.

3.12.3. The Same Substitution

Recall from Section 3.10.3 that the production worksheet does not implement Equation 3.14 at all. It computes $h' = h(1 - \lambda/p)$ and applies the ordinary formula to it. The code does the same thing, and it puts the surviving fraction on the replenishment type so that no formula anywhere has to ask which case applies.

The worksheet's feasibility guard has a counterpart in code, and it is stricter. `=IF( >= P, "Infeasible production rate", ...)` prints a message beside a sheet that has already computed a wrong answer. In code the rate is refused at construction. Thus, no parameter set with $p \le \lambda$ ever exists.

Example 3.14 (The production sheet in code)**.** Recall the item of Example 3.9, with $\lambda = 2500$ and $p = 10{,}000$ units per year, $c = \$2$ per unit, $i = 0.30$ per year, and $k = \$50$ per setup.

```
val production = CostParameters(
    demandRate    = 2500.0,
    unitCost      = 2.0,
    orderCost     = 50.0,
    holding       = HoldingCost.carryingCharge(0.30),
    replenishment = Replenishment.atRate(10_000.0, demandRate
 ↪  = 2500.0),
)

production.baseHoldingRate                          // 0.6000
 ↪  h
production.survivingFraction                        // 0.7500
 ↪  1 - lambda/p
production.baseHoldingRate * production.survivingFraction
 ↪  // 0.4500   h'

val made = EconomicProductionQuantity.optimize(production)
made.orderQuantity                                  // 745.3560
made.cycle.maxOnHand                                //
 ↪  559.016994
made.cycle.averageOnHand                            // 279.5085
made.cycle.replenishmentTime                        // 0.0745356
made.measures.ordering                              // 167.71,
 ↪  equal to holding
```

The machine runs for 0.0745 year of every 0.2981 year cycle, a quarter of the time, which is λ/p as it must be.

Passing an infeasible rate does not produce a message beside an answer:

```
The replenishment rate must exceed the demand rate,
but 80.0 is not greater than 100.0
```

3.12.4. Discount Schedules as Candidates

Section 3.10.4 lays the discount procedure out as computed columns beside the price table. In code the price table is an object and the columns are a list of candidates. The result carries that list, so that the reasoning can be read and not only the answer.

Example 3.15 (The all-units sheet in code)**.** Recall the schedule of Example 3.10, with $\lambda = 600$ units per time, $k = \$8$ per order, $i = 0.20$, and three price levels. The first break point is 0 here and not the 1 the worksheet uses, for the reason Section 3.10.4 gives.

```
val allUnits = CostParameters(
    demandRate = 600.0,
    orderCost  = 8.0,
    holding    = HoldingCost.carryingCharge(0.20),
    schedule   = AllUnits(listOf(
        PriceLevel(breakPoint =    0.0, unitCost = 0.30),
        PriceLevel(breakPoint =  500.0, unitCost = 0.29),
        PriceLevel(breakPoint = 1000.0, unitCost = 0.28))),
)
val best = AllUnitsDiscountModel.optimize(allUnits)
```

Table 3.10.: The candidate list the result carries, the worksheet's computed columns.

Level	Break point	Price	Candidate	Total cost	
1	0	\$0.30	400.0000	204.0000	
2	500	\$0.29	500.0000	**198.1000**	chosen

Level	Break point	Price	Candidate	Total cost
3	1000	\$0.28	1000.0000	200.8000

Order 500 at \$198.10. Notice that the candidates are ranked on the *total* cost and not on the relevant cost of Equation 1.16. Recall that Section 1.4.1 sets the purchase term aside because it is identical under every policy. Section 3.8 is where that premise stops holding, because once the price depends on the quantity the purchase term is what the decision moves. For example, ranking these three candidates on ordering and holding alone chooses 400, wrong by \$5.90 a year.

Example 3.16 (The incremental sheet in code)**.** We keep the same inputs under an incremental schedule. The worksheet's recursive `Fixed Cost` column is the schedule's own calculation:

```
val schedule = Incremental(listOf(
    PriceLevel(0.0, 0.30), PriceLevel(500.0, 0.29),
 ↪  PriceLevel(1000.0, 0.28)))

schedule.fixedChargeAt(0)   // 0.00
schedule.fixedChargeAt(1)   // 5.00
schedule.fixedChargeAt(2)   // 15.00
```

With $k = \$8$ this gives the effective ordering costs 8, 13, and 23 of Equation 3.27. Optimizing then gives

Table 3.11.: Two candidates fall inside their own intervals, and the smaller wins.

Level	Price	Candidate	Total cost	
1	\$0.30	400.0000	**204.0000**	chosen

Level	Price	Candidate	Total cost
2	$0.29	518.6189	204.5799

Order 400 at $204.00. Recall from Section 3.10.4 that the worksheet has 518.62 entered in its evaluator, and that the sheet has no way to rank the two. Here the ranking is the operation itself. Thus, the 58 cents is not left to the reader to notice.

3.12.5. Doing the Three Things a Worksheet Does Not

Section 3.10.5 ends with three things the worksheets do not do. They are the three things the code is for, and we take them in the same order.

It chooses. A worksheet evaluates the quantity it is given. `optimize` instead returns the best one and carries the candidates it rejected, so the answer arrives with its reasoning attached. Example 3.16 is the case where that matters.

It refuses what it cannot model. A worksheet computes whatever is typed into it. Here the assumptions of Section 3.1.1 are checked where the value is made, and the message says what to do instead:

```
A backorder cost of zero makes shortages free rather than forbidden.
Use ShortagePolicy.NotPermitted, which is the limit as the cost grows
without bound. See @sec-eoq-nobackorders.
```

That is the callout of Section 3.5.2 enforced rather than printed. A model asked to work outside its assumptions says so too, and it distinguishes two cases. Applying the order quantity to an item with a finite production rate is a deliberate approximation and is allowed, with the discarded assumption recorded on the result. However, asking the planned backorder model for a policy when shortages are forbidden is refused outright.

It rounds as part of the decision. Section 3.7 observes that no warehouse orders 40.37 of anything, and Exercise 3.5 asks what a case quantity costs. Here rounding is an argument to `optimize`, which evaluates both admissible neighbors instead of rounding the answer afterwards:

```
EconomicOrderQuantity.optimize(item,
 ↪  QuantityRounding.multipleOf(48.0))
```

Table 3.12.: For the item of Exercise 3.2, whose optimum is 202.26. Rounding down is the cheaper of the two, which rounding the answer up would have missed.

Case quantity	Relevant cost	Penalty
192	\$1,782.30	1.001355
240	\$1,806.00	1.014671

Thus, the reported cost is the cost of the quantity that will actually be ordered, the gap Section 3.10.5 identifies in the worksheets.

3.13. Summary

In this chapter we answered the two questions of Section 1.1, how much to order and when, under the strongest assumption available. That is, everything is known and nothing varies.

One model, specialized four ways. Section 3.1 derives the geometry of a single cycle once, and Table 3.1 obtains every classical result from it by switching an assumption off. The economic production quantity is the general model with shortages forbidden, the classical order quantity is that model with replenishment made instantaneous, and the planned backorder model is the general one with no charge per unit short.

Deriving the cycle once instead of four times is the reason the chapter is arranged this way. It is also why Section 3.9 gets the service measures for nothing, since they are ratios of segment lengths that had been computed already.

Forbidding a shortage is a limit, not a price of zero. A zero backorder cost makes shortages free, not forbidden, and it sends the order quantity to infinity. The no-shortage models are instead the limit as b grows without bound. That is the distinction Section 3.5.2 is careful about, and it is the one that most often goes wrong.

Cost balances at the optimum, and the total is flat around it. Ordering and holding cost are equal at Q^*, the fastest arithmetic check available. The penalty for departing from the optimum depends only on the ratio Q/Q^* and on nothing else about the item. Thus, being wrong by a fifth costs almost nothing and being wrong by half costs 25%. That tolerance is the reason it is reasonable to order in case quantities and to fit a delivery schedule, and it is what Chapter 4 exploits when it forces items to share a cycle.

Timing is a separate decision from quantity, and it reads the position. The reorder point covers only the demand not already covered by orders outstanding. It is therefore not monotone in the lead time, and Equation 3.21 is written in terms of inventory position and not stock on hand.

A price schedule changes the problem in kind. While the unit cost is constant the purchase term is inert and can be set aside. Once the supplier prices in tiers it is what the decision moves. Thus, the cost rate is no longer smooth, and the answer is selected from a candidate list instead of obtained by differentiating. Section 3.8.1 and Section 3.8.2 differ in whether the break points are themselves candidates, and that difference follows from whether the cost function jumps.

Everything in this chapter rests on demand being known and constant. In the next chapter we keep that assumption and remove a different one, that

items can be decided independently, the subject of Chapter 4. Chapter 5 then keeps independence and lets demand vary over time. From Chapter 7 onward demand stops being known at all, and the quantities this chapter computed exactly become quantities that must be estimated.

3.14. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year, and convert any rate quoted on another basis before using it. Report order quantities and reorder points to two decimal places, cycle times to four, and costs to the nearest cent. Round an order quantity to an integer or to a case multiple only when the exercise asks for the quantity that would actually be ordered, and price the rounded quantity and not the unrounded one when you do.

Exercise 3.1. An item is demanded at $\lambda = 600$ units per year and replenished at $p = 1500$ units per year. The order quantity is $Q = 300$ and the policy permits a maximum backorder level of $\hat{B} = 40$ units.

Following Example 3.1, determine the peak on-hand level, the four segment lengths, and verify that they sum to Q/λ. Compute $\bar{I}$ and $\bar{B}$ both from the triangle areas and from the closed forms, and confirm they agree.

Then, following Example 3.6, report the order frequency, the fraction of time out of stock, the ready rate, and the average customer wait in days.

Exercise 3.2. An item is demanded at $\lambda = 1500$ units per year and costs \$40 per unit. The ordering cost is \$120 per order, the carrying charge is 22% per year, no shortages are permitted, and replenishment is instantaneous.

Following Example 3.2, determine the economic order quantity, the optimal cycle length in days, the optimal total cost, and the reorder point for a 10 day lead time.

Then determine what the reorder point would be if the lead time were 70 days instead, using Equation 3.21, and explain in one sentence why the answer is not $\lambda(70/365)$.

Exercise 3.3. Show that at $Q = Q^*$ the ordering cost rate and the holding cost rate are equal, and that each equals $\frac{1}{2}\sqrt{2k\lambda h}$.

Then use this to construct a check that requires no square roots: given a candidate Q, state a one-line test that determines whether Q is too large or too small.

Exercise 3.4. A component is used at $\lambda = 4000$ units per year and produced in house at $p = 10{,}000$ units per year. The setup cost is \$250 and the holding cost is \$3 per unit per year. Shortages are not permitted.

Determine the economic production quantity from Equation 3.14 and the economic order quantity from Equation 3.17, and report the ratio of the two.

Show algebraically that the ratio depends only on λ/p, and determine what happens to the production quantity as the production rate approaches the demand rate. Explain what that limit means physically.

Exercise 3.5. The item of Exercise 3.2 is supplied in cases of 48 units, so the order quantity must be a multiple of 48.

Determine the two case quantities nearest the economic order quantity, use Equation 3.22 to compute the cost penalty of each, and recommend one. State the annual dollar cost of your recommendation relative to the unrounded optimum.

Exercise 3.6. An item has $\lambda = 5000$ units per year, an ordering cost of \$40, and a carrying charge of 25% per year. The supplier offers an all-units discount schedule:

Order quantity	Unit price
$0 \le Q < 300$	\$20.00
$300 \le Q < 700$	\$19.00
$Q \ge 700$	\$18.00

Following Example 3.4, determine the optimal order quantity. Report which candidate quantities you evaluated and why each was or was not feasible, and give the total cost at each.

Then state which of the three cases in Table 3.2 applies, and explain why evaluating only the first feasible candidate would have given the wrong answer.

Exercise 3.7. Repeat Exercise 3.6 with the same break points and prices, but under an **incremental** discount schedule rather than an all-units one.

Following Example 3.5, compute R_j, the effective ordering cost k_j, and the candidate quantity for each level. Determine the optimal order quantity and total cost.

Compare your answer with Exercise 3.6 and explain, in terms of what each scheme charges, why the two schedules produce different orders and which one produces the larger.

Exercise 3.8. An item has $\lambda = 1200$ units per year, an ordering cost of \$100, a holding cost of \$6 per unit per year, and a backorder cost of \$15 per unit per year. Replenishment is instantaneous and there is no per-unit stockout charge.

Determine Q^* and $\hat{B}^*$ from Equation 3.15, and compute $\bar{I}$, $\bar{B}$, the ready rate, and the average customer wait.

Compare Q^* with the order quantity that would be used if backorders were forbidden, and explain why permitting backorders increases the order

quantity. Then determine what backorder cost b would be needed for the two order quantities to differ by less than 1%.

Exercise 3.9. A planner argues that because the total cost curve is flat near its minimum, as Figure 3.2 shows, the economic order quantity is not worth computing and any reasonable quantity will do.

Using Equation 3.22, determine the range of order quantities, expressed as a multiple of Q^*, over which the relevant cost stays within 2% of its minimum. Then state the two circumstances from Section 1.4.8 under which the planner's argument fails, and explain which of them applies to the quantity discount problems of Section 3.8.

Exercise 3.10. The Classic EOQ worksheet of Example 3.7 is set up in months, with $\lambda = 100$ units per month, $c = \$8$, a carrying charge of 25% per year, $k = \$20$ per order, and $L = 0.25$ month.

Reproduce the optimizer block by hand: h, Q^*, T^*, the two cost components, the total cost rate, and the reorder point.

Then change the carrying charge to 30% per year and recompute. Report the percentage change in Q^* and show algebraically that it depends only on the ratio of the two carrying charges, not on λ or k.

Exercise 3.11. For the item of Exercise 3.10, whose optimal cycle is $T^* = 1.5492$ months, compute the reorder point from Equation 3.21 for lead times of $L = 0.25$, 1.0, 1.6, 2.0, 3.0, and 4.0 months.

Plot r against L and describe the shape. Explain why r can be smaller at $L = 1.6$ than at $L = 1.0$, and state the largest value r can take for this item.

Then explain in one sentence why a worksheet that computed $r = \lambda L$ would be wrong for every lead time in the table beyond the first.

Exercise 3.12. The incremental discount worksheet of Example 3.11 evaluates a quantity that is entered instead of selecting among the candidates, so an analyst who computes 518.62 for level 2, finds it feasible, and enters it gets a plausible answer that is not the optimum.

Redesign it. Using Table 3.6 as the model, specify the computed columns that would turn the incremental schedule table into a self-selecting one: give the formula for each column, including the fixed cost recursion of Equation 3.31, and state the rule that picks the answer from the completed table.

Explain why your design needs no column corresponding to $TC(q_j)$, the break-point candidate that the all-units sheet requires.

Exercise 3.13. Take the item of Example 3.12 and, in code, reproduce every figure of Example 3.7: the holding cost rate, the order quantity, the cycle length, the two cost components, the total cost rate, and the reorder point.

Then set the order quantity to each of 150, 155, and 160 and report the ratio, difference, and relative error against the optimum. Explain in one sentence why two of the three carry the same penalty.

Exercise 3.14. The item of Example 3.14 is produced at $p = 10{,}000$ units per year against demand of $\lambda = 2500$.

In code, reduce the production rate toward the demand rate in steps and record the order quantity at each. Report what happens to it, and explain the limit in terms of Equation 3.14.

Then attempt to set the production rate to 2000 and report what happens. Contrast that with what the production worksheet of Example 3.9 does with the same value, and say which behavior you would rather have in a program that runs unattended.

Exercise 3.15. Example 3.15 ranks its three candidates on total cost. Rank them instead on the relevant cost of Equation 1.16, report the quantity that ranking selects, and compute what choosing it would cost per year.

Then state the condition under which the two rankings agree, and explain why the worked example of Example 3.4 does not reveal the difference while the schedule of Example 3.15 does.

4. Multi-Item and Constrained Systems

Learning objectives

After reading this chapter you should be able to:

- explain how a shared resource couples items whose models are otherwise independent
- formulate a lot sizing problem with an aggregate constraint and solve it with a Lagrange multiplier
- interpret the multiplier as a shadow price, and recognize when it is instead an implied carrying charge
- show that a budget constraint scales every order quantity by the same factor
- construct an exchange curve and read from current practice the cost ratio it implies
- use the variety index to say how much of aggregate performance is driven by the spread across items
- evaluate a power-of-two policy and state what it costs against the best coordinated policy
- define echelon inventory and echelon holding cost, and explain why installation accounting double counts
- find the optimal reorder intervals for a serial system and for a distribution system

Recall that Chapter 3 took one item at a time. Every model in it produced an order quantity from that item's own demand, prices, and replenishment rate, and nothing about any other item entered the calculation. That independence is what made the chapter tractable, and it is what this chapter removes.

In this chapter we develop the models that apply when items must be decided together. We begin with items that compete for something scarce, then turn to items that share the cost of an order, and end with stock that exists in order to replenish other stock. Each of the three couples the items differently, and each therefore needs its own machinery.

4.1. When Items Stop Being Independent

An item's lot sizing decision can be made on its own only if nothing it consumes is also consumed by something else. Two different things break that condition, and they lead to different mathematics.

A shared resource couples items through a constraint. Warehouse space, working capital tied up in stock, and the receiving department's capacity to process orders are all finite and shared. A larger order for one item leaves less for the others. The items' cost functions stay separate; what ties them together is a single inequality they must jointly satisfy. Four such constraints are common:

1. average total inventory investment, at most a budget
2. average number of replenishments per unit time, at most a fixed limit
3. maximum space used, at most what is available
4. maximum allowed backorder delay, at most a fixed limit

A shared setup couples items through a cost. For example, when several items come from one supplier on one truck, or run on one machine that is set up once for a family, part of the cost of ordering is incurred once regardless

of how many items are on the order. The cost functions themselves are no longer separate.

Section 4.2 through Section 4.4 treat the first case, where the coupling is a constraint. Section 4.5 treats the second, where the coupling is a cost. The techniques have almost nothing in common, so they occupy separate halves of the chapter. Section 4.6 then takes up a third kind of coupling, in which one stock exists in order to replenish another, and shows that the power-of-two machinery carries over once the holding cost is accounted for correctly.

Throughout this chapter, demand is still known and constant, and every item is still governed by the classical model of Section 3.5.4. What changes is only that the items are decided together. To begin, we take up the case where the items compete for a single scarce resource.

4.2. A Constrained Lot Sizing Problem

Consider a group of items that draw on one scarce resource. We need notation for the items before we can write the problem down.

- Let n represent the number of items sharing the resource, indexed by j.
- Let λ_j represent the demand rate for item j, which is known and constant.
- Let c_j represent the unit cost of item j, and k_j its ordering cost.
- Let h_j represent the holding cost rate for item j, which is ic_j when holding cost is charged as a carrying charge on value.
- Let Q_j represent the order quantity for item j, the quantity we are choosing.

The decision is the vector $Q_1, \dots, Q_n$, and the objective is the sum of the relevant costs of Equation 1.16:

$$\min\ C = \sum_{j=1}^{n} \left[\frac{k_j \lambda_j}{Q_j} + \frac{h_j Q_j}{2} \right] \tag{4.1}$$

Now consider the four constraints listed in Section 4.1. The first and third are **linear** in the order quantities.

- Let a_j represent what one unit of item j consumes of the shared resource.
- Let A represent how much of the resource there is.

Both take the form

$$\sum_{j=1}^{n} a_j Q_j \leq A \tag{4.2}$$

For example, under a **space** constraint a_j is the floor space one unit occupies and A is the space available, since immediately after a replenishment there are Q_j units on the floor. Under a **budget** constraint on average investment the average on-hand level is $Q_j/2$ and its value is $c_j Q_j/2$, so that $a_j = c_j/2$ and A is the budget. Both are Equation 4.2 with different coefficients, so one derivation covers them together.

The second constraint is not linear. An item ordered in quantity Q_j is replenished λ_j/Q_j times per unit time, so a limit on replenishments reads

$$\sum_{j=1}^{n} \frac{\lambda_j}{Q_j} \leq A \tag{4.3}$$

Consumption *falls* here as the quantities rise, instead of growing with them. Thus, this is a different problem, and Section 4.2.4 takes it up once the linear case is settled. The fourth constraint, on backorder delay, belongs with the models that permit shortages and is not treated in this chapter.

4.2.1. First Check Whether the Constraint Binds

Equation 4.1 is **separable**: it is a sum of terms, each involving only one item's quantity. Recall from Section 3.5.4 that each such term is convex, and a sum of convex functions is convex. Thus, in the absence of the constraint each item can be optimized on its own, and the answer is the ordinary order quantity of Equation 3.17:

$$Q_j^* = \sqrt{\frac{2k_j\lambda_j}{h_j}}$$

That gives a three-step procedure whose first step is free:

1. Solve the unconstrained problem, item by item.
2. If $\sum_j a_j Q_j^* \le A$, the constraint is **inactive** and the unconstrained answer is the answer. Otherwise it is **active** and step 3 applies.
3. Solve the problem with the constraint imposed as an equality.

Step 2 is easy to skip and expensive to skip. A constraint that does not bind changes nothing, and the machinery of the next two sections is needed only when the resource runs out. You should perform this check first every time.

Step 3 may treat the inequality as an equality because the objective is convex. That is, if the unconstrained optimum violates the limit, then the constrained optimum sits on the boundary of the feasible region, and on that boundary the inequality holds with equality. The problem we actually solve is an equality-constrained one.

4.2.2. Moving the Constraint into the Objective

The method of **Lagrange multipliers** moves a constraint into the objective in order that the unconstrained techniques of Chapter 3 apply to what is left.

- Let θ represent the **multiplier**, a single scalar attached to the constraint, whatever the number of items.

With this notation we form the Lagrange function

$$L(\vec{Q},\theta)=\sum_{j=1}^{n}\left[\frac{k_j\lambda_j}{Q_j}+\frac{h_jQ_j}{2}\right]+\theta\left[\sum_{j=1}^{n}a_jQ_j-A\right] \tag{4.4}$$

Notice that the bracketed term is zero whenever the constraint holds as an equality. At any feasible point on the boundary, L and C agree, so minimizing L is a legitimate substitute for minimizing C. Setting the partial derivatives to zero gives $n+1$ equations in the n quantities and θ:

$$\frac{\partial L}{\partial Q_j}=-\frac{k_j\lambda_j}{Q_j^2}+\frac{h_j}{2}+\theta a_j=0, \qquad j=1,2,\ldots,n$$

$$\frac{\partial L}{\partial \theta}=\sum_{j=1}^{n}a_jQ_j-A=0$$

Solving the first set for each quantity in terms of θ, we obtain

$$Q_j(\theta)=\sqrt{\frac{2k_j\lambda_j}{h_j+2\theta a_j}} \tag{4.5}$$

Notice that Equation 4.5 is Equation 3.17 with the holding cost rate h_j replaced by $h_j+2\theta a_j$. **The multiplier acts as a surcharge on holding**, charged in proportion to how much of the scarce resource a unit consumes. An item that occupies a lot of space, or ties up a lot of capital, is penalized more and its quantity falls further. Doubling a_j for one item while leaving everything else alone doubles the surcharge that item pays.

4.2.3. Finding the Multiplier

Substituting Equation 4.5 into the remaining equation, we are left with one equation in one unknown:

$$R(\theta) = \sum_{j=1}^{n} a_j \sqrt{\frac{2k_j\lambda_j}{h_j + 2\theta a_j}} - A \tag{4.6}$$

Let $R(\theta)$ represent the **resource function**, the amount of the shared resource the quantities consume at multiplier θ, less the amount available. Every term of the sum decreases as θ increases. R is strictly decreasing. At $\theta = 0$ it is positive, because that is precisely the case in which the unconstrained quantities violate the limit, and it tends to $-A$ as θ grows without bound. Thus, there is exactly one root $\theta^* > 0$, and any root-finding method will locate it. Bisection is enough: bracket the root, halve the interval, repeat.

The whole procedure is a one-dimensional search wrapped around a formula, so these problems are routinely solved in a spreadsheet with a nonlinear solver. The single root is also what makes the search dependable, a point we return to in Section 4.7.1. We now work a complete instance.

Example 4.1 (An investment limit on three items)**.** A distributor stocks three items that are funded out of one working capital account. Each costs \$50 to order and the carrying charge is 30% per year. Let $i = 0.30$ represent that carrying charge, so that $h_j = ic_j$ for every item.

What is stocked? Three items, each replenished from its own supplier.

What is the demand process? Constant and known, at the rates in Table 4.1.

When is inventory reviewed? Continuously, item by item.

What triggers replenishment, and how much? Each item's shelf reaching zero, and the order is for that item's Q_j, which are the three decisions.

What happens to unmet demand? None occurs; shortages are not allowed.

What costs are incurred, and when? \$50 each time any item is ordered, and the carrying charge on the value held, subject to management funding no more than \$10,000 of average investment across the three.

Table 4.1.: Three items and the quantities they would order if nothing were scarce.

Item	λ_j	c_j	$h_j = ic_j$	Q_j^* unconstrained
1	12,000	\$20	\$6.00	447.21
2	25,000	\$10	\$3.00	912.87
3	8,000	\$15	\$4.50	421.64

Step 1, the unconstrained solution. Applying Equation 3.17 to each item gives the quantities in Table 4.1, a relevant cost of \$7,319.26 a year, and an average investment of

$$\sum_j c_j \frac{Q_j^*}{2} = 20\frac{447.21}{2} + 10\frac{912.87}{2} + 15\frac{421.64}{2} = \$12{,}198.77$$

Step 2, does the constraint bind? Management will fund \$10,000. Since \$12,198.77 exceeds \$10,000, the constraint is active. The unconstrained answer is not available, and we continue to step 3.

Step 3, solve for the multiplier. Recall that under a budget constraint $a_j = c_j/2$. Substituting into Equation 4.5, the surcharge $2\theta a_j$ becomes θc_j, and we have

$$Q_j(\theta) = \sqrt{\frac{2k_j\lambda_j}{h_j + \theta c_j}}$$

Bisection on Equation 4.6 gives $\theta^* = 0.14643$. Evaluating Equation 4.5 at that multiplier gives the quantities in Table 4.2, together with the cost each item then incurs.

Table 4.2.: The solution under a \$10,000 limit on average investment.

Item	Q_j constrained	Ordering cost	Holding cost
1	366.61	\$1,636.64	\$1,099.82
2	748.33	\$1,670.39	\$1,122.50
3	345.64	\$1,157.28	\$777.69
Total		**\$4,464.30**	**\$3,000.00**

The arithmetic check. Notice that the holding cost total is exactly \$3,000, or $0.30 \times \$10{,}000$. The constraint fixed the investment, so it fixed the holding cost with it, and the two numbers must agree. You should perform that check every time a budget constraint is solved, because it catches an error in either the multiplier or the quantities immediately. The investment itself comes to exactly \$10,000, as an active constraint requires.

What the limit means for the distributor. The relevant cost is \$7,464.30, so the constraint raised the annual cost by \$145.04, or 1.98%. That is the price of holding \$2,199 less in stock. Thus, when someone proposes tightening or relaxing the working capital account, \$145.04 is the figure that belongs in the conversation, and you should be prepared to supply it instead of merely complying with the limit.

Notice also that the ordering and holding costs are no longer equal, at \$4,464.30 against \$3,000.00. Recall from Section 3.5.4 that at an unconstrained optimum the two are equal. That property fails here, as it must, because these quantities do not minimize the unconstrained cost. Thus, if you check a constrained solution by looking for balanced costs, you will conclude wrongly that something is broken.

4.2.4. When the Limit Is on Ordering, Not on Stock

Equation 4.3 limits how often the operation can act, not how much it can hold. For example, receiving docks, purchasing staff, and inspection capacity all impose limits of this kind. The mathematics turns out to run in the opposite direction from the linear case.

We form the Lagrange function as before, with Equation 4.3 in place of Equation 4.2:

$$L(\vec{Q},\theta)=\sum_{j=1}^{n}\left[\frac{k_j\lambda_j}{Q_j}+\frac{h_jQ_j}{2}\right]+\theta\left[\sum_{j=1}^{n}\frac{\lambda_j}{Q_j}-A\right]$$

The ordering term and the constraint both carry λ_j/Q_j. They combine when we differentiate:

$$\frac{\partial L}{\partial Q_j}=-\frac{(k_j+\theta)\lambda_j}{Q_j^2}+\frac{h_j}{2}=0\qquad\Longrightarrow\qquad Q_j(\theta)=\sqrt{\frac{2(k_j+\theta)\lambda_j}{h_j}}\quad(4.7)$$

Compare Equation 4.7 with Equation 4.5. There the multiplier was added to the holding rate; here it is added to the **ordering cost**. That is the whole of the difference, and the three consequences below all follow from it.

The multiplier is a price per order. Its units are dollars per order, i.e. the units of k_j, because that is the quantity it is added to. Thus, a limit on ordering capacity and a supplier who charges more per order are the same thing seen from two sides, as a budget limit and a higher cost of capital were in Section 4.3.

Quantities rise, they do not fall. Ordering less often means ordering more each time. Thus, every Q_j increases with θ and the replenishment count falls. The search still works for the reason given in Section 4.2.3, since

$\sum_j \lambda_j/Q_j(\theta)$ is again strictly decreasing in θ with a single root. Bisection applies unchanged.

The proportions do not survive. Recall that Equation 4.9 held because the budget multiplier entered as a factor $(i+\theta)$ common to every item. Here it enters as a *sum*, $k_j+\theta$, and adding a constant to unequal ordering costs changes them by unequal proportions. Thus, a frequency limit reallocates between items where a budget does not. The one exception is the case in which every item already has the same ordering cost, when $k+\theta$ is again common to all items and the portfolio rescales uniformly.

Example 4.2 (A limit on replenishments)**.** Return to the distributor of Example 4.1, with the same demands and unit costs and the same carrying charge of 30% per year. This time the items differ in what they cost to order, and the limit falls on the receiving dock, not on the capital account.

Table 4.3.: Three items whose unconstrained policies ask for 85 replenishments a year.

Item	λ_j	c_j	k_j	h_j	Q_j^*	Orders per year
1	12,000	\$20	\$50	\$6.00	447.21	26.83
2	25,000	\$10	\$20	\$3.00	577.35	43.30
3	8,000	\$15	\$80	\$4.50	533.33	15.00
Total						**85.13**

Unconstrained, the three items cost \$6,815.33 a year and require 85.13 replenishments. The receiving dock can process 50. Since 85.13 exceeds 50, the constraint is active.

Solving. Bisection on $\sum_j \lambda_j/Q_j(\theta) - 50$, with $Q_j(\theta)$ taken from Equation 4.7, gives $\theta^* = \$64.33$ per order.

Table 4.4.: The solution under a limit of 50 replenishments a year.

Item	Effective $k_j + \theta^*$	Q_j	Orders per year	Q_j/Q_j^*
1	\$114.33	676.25	17.74	1.5121
2	\$84.33	1,185.53	21.09	2.0534
3	\$144.33	716.36	11.17	1.3432
Total			**50.00**	

The arithmetic check. The replenishments total exactly 50.00, as an active constraint requires. Notice also that each quantity in Table 4.4 is the ordinary economic order quantity of Equation 3.17 computed with the effective ordering cost in the second column instead of with k_j. Item 1 at $k_1 + \theta^* = \$114.33$ gives $\sqrt{2(114.33)(12{,}000)/6} = 676.25$. You should perform that check every time, because it confirms the multiplier and the quantities were computed from the same value.

What the limit means. The annual cost rises to \$7,621.27, so the limit costs \$805.93, or 11.83%. Every quantity has grown, as ordering less often requires. Notice in the last column that they have not grown together. Item 2, whose \$20 ordering cost is most changed by an addition of \$64.33, more than doubles, while item 3 grows by only a third. Thus, the dock's capacity has redistributed the quantities as well as lengthened them.

The shadow price reading is unchanged in form. One more replenishment per year would save about \$64. That is the figure you should carry to whoever is deciding whether to add capacity at the dock.

4.3. What the Multiplier Means

The multiplier θ^* is not merely a device for getting an answer. It carries the economics of the constraint, and it has three readings you should keep

apart.

It is a shadow price. That is, θ^* is the rate at which the minimum cost falls as the limit is relaxed. For example, in Example 4.1 it is 0.14643, so one more dollar of working capital would save about 15 cents a year and a hundred more about \$14. Thus, it is the number you take to whoever controls the budget.

The rate is a local one. As the limit is relaxed the multiplier itself falls, reaching zero at \$12,198.77, where the constraint stops binding and the last of the \$145.04 has been recovered. Thus, you should not extrapolate the rate far from the current limit, and asking for more than \$12,198.77 buys nothing at all.

Under a budget constraint it is an increment to the carrying charge. Substituting $a_j = c_j/2$ and $h_j = ic_j$ into Equation 4.5, we obtain

$$Q_j = \sqrt{\frac{2k_j\lambda_j}{ic_j + \theta c_j}} = \sqrt{\frac{2k_j\lambda_j}{(i+\theta)c_j}} \tag{4.8}$$

That is, Equation 4.8 is the ordinary order quantity computed with a carrying charge of $i + \theta$ in place of i. For example, in Example 4.1 the effective charge is $0.30 + 0.14643 = 0.44643$ per year. A budget constraint and a higher cost of capital are the same thing seen from two sides. That is no coincidence, because both say that money tied up in stock is scarcer than the accounting rate admits.

Every quantity scales by the same factor. Equation 4.8 depends on θ only through the factor $(i + \theta)$, which is common to all items. Taking the ratio to the unconstrained quantity,

$$\frac{Q_j(\theta)}{Q_j^*} = \sqrt{\frac{i}{i+\theta}} \qquad \text{for every } j \tag{4.9}$$

For example, in Example 4.1 the factor is $\sqrt{0.30/0.44643} = 0.81975$. Notice that 366.61/447.21, 748.33/912.87, and 345.64/421.64 are all equal to 0.81975. Thus, **a budget constraint changes the scale of the whole portfolio and leaves the proportions between items untouched.** Nothing is reallocated; everything shrinks.

That is not what intuition suggests. Faced with a cut in working capital, a planner might expect to cut deepest on the items that consume the most of it. Equation 4.9 says otherwise. The unconstrained solution has already put each item in the right proportion, and the constraint asks only that everything be smaller.

You should note what the result depends on. It requires $h_j = ic_j$, so it holds when holding cost is charged as a carrying charge on value and not when holding cost is quoted directly. It also fails under a space constraint, where $a_j = v_j$ is unrelated to h_j, and under the frequency constraint of Section 4.2.4. Thus, uniform rescaling is a property of the budget case, not of constrained lot sizing in general.

Having established what one constraint does to a group of items, we now step back and ask what the group is doing in aggregate whether or not any constraint binds.

4.4. The Exchange Curve

A constraint is one way to make items interact. Another is to stop looking at items at all and ask what the whole portfolio is doing. Aggregate performance is what management reports on, and the relationship between its components turns out to be sharper than one would expect.

4.4.1. One Item First

Take a single item and let Q vary. Its average investment and its order frequency are

$$\bar{I} = \frac{cQ}{2}, \qquad \overline{OF} = \frac{\lambda}{Q}$$

Their product is

$$\bar{I} \cdot \overline{OF} = \frac{cQ}{2} \cdot \frac{\lambda}{Q} = \frac{c\lambda}{2} \tag{4.10}$$

and the order quantity has cancelled. Whatever quantity is used, the product of investment and order frequency is the same. The achievable combinations therefore lie on a hyperbola fixed by the item's own data, and choosing Q chooses a point on it and nothing more. Ordering more often is not a policy that can be adopted on its own: it is the same policy as holding less stock.

Which point does the order quantity of Equation 3.17 pick? Through this section and the next the carrying charge is taken to be one value shared by every item, and it is written γ and not i to keep that assumption visible in every formula it enters. Substituting $Q^* = \sqrt{2k\lambda/(\gamma c)}$,

$$\frac{\bar{I}}{\overline{OF}} = \frac{cQ^*}{2} \cdot \frac{Q^*}{\lambda} = \frac{c\,(Q^*)^2}{2\lambda} = \frac{c}{2\lambda} \cdot \frac{2k\lambda}{\gamma c} = \frac{k}{\gamma} \tag{4.11}$$

So the **product** of the two aggregates fixes the curve, and their **ratio** fixes the point on it. The ratio is k/γ, and nothing else about the item enters.

Example 4.3 (What a chosen operating point implies)**.** A planner decides to order an item twice a year and, doing so, carries an average of \$400 of it in stock. From Equation 4.11,

$$\frac{\bar{I}}{\overline{OF}} = \frac{400}{2} = 200 = \frac{k}{\gamma}$$

so the planner's choice implies $k = 200\gamma$, whether or not the planner knows it. If the firm's carrying charge is $\gamma = 0.20$ per year, the implied ordering cost is $k = \$40$.

That number can be checked, and the check is the point of the calculation. Suppose purchasing reports that placing an order costs \$120. Then Equation 4.11 requires $k/\gamma = 120/0.20 = 600$, and the planner is operating at 200. That is, the order quantity in use is too small and the item is being ordered far too often for the stated carrying charge.

Notice that the disagreement can be read the other way, and that reading is what makes it useful. If the ordering practice is taken as correct, then $\gamma = k/200 = 0.60$, a carrying charge of 60% per year. Thus, one of two things is true: the item is ordered too often, or the firm's carrying charge is three times what it says it is. Both are defensible claims and the calculation does not settle which, but it does force somebody to make one of them. Section 1.5.4 gives methods for estimating k directly. This gives a second reading of it, obtained from behavior and not from accounting.

4.4.2. The Aggregate Curve

Now take n items. Define the **aggregate investment in cycle stock** and the **aggregate number of replenishments** per unit time:

$$\bar{I}^a = \sum_{j=1}^{n} c_j \frac{Q_j}{2}, \qquad N^a = \sum_{j=1}^{n} \frac{\lambda_j}{Q_j} \tag{4.12}$$

The first is in dollars and appears on a balance sheet. The second is in orders per year and is what the receiving department feels. They trade against one another, and the question is exactly how.

Suppose that every item is ordered in its economic quantity, that the ordering cost is a common k for all items, and that the carrying charge is the common γ. Then $Q_j = \sqrt{2k\lambda_j/(\gamma c_j)}$. Substituting it into one item's terms of Equation 4.12 gives

$$c_j \frac{Q_j}{2} = \frac{c_j}{2}\sqrt{\frac{2k\lambda_j}{\gamma c_j}} = \sqrt{\frac{k}{\gamma}} \cdot \frac{\sqrt{\lambda_j c_j}}{\sqrt{2}}, \qquad \frac{\lambda_j}{Q_j} = \lambda_j \sqrt{\frac{\gamma c_j}{2k\lambda_j}} = \sqrt{\frac{\gamma}{k}} \cdot \frac{\sqrt{\lambda_j c_j}}{\sqrt{2}},$$

and summing over the items we obtain

$$\bar{I}^a = \sqrt{\frac{k}{\gamma}} \cdot \frac{1}{\sqrt{2}} \sum_{j=1}^{n} \sqrt{\lambda_j c_j}, \qquad N^a = \sqrt{\frac{\gamma}{k}} \cdot \frac{1}{\sqrt{2}} \sum_{j=1}^{n} \sqrt{\lambda_j c_j} \tag{4.13}$$

Two things follow at once. First, the product is

$$\bar{I}^a \cdot N^a = \frac{1}{2} \left(\sum_{j=1}^{n} \sqrt{\lambda_j c_j} \right)^2 \tag{4.14}$$

in which k and γ have both cancelled. Second, the ratio is

$$\frac{\bar{I}^a}{N^a} = \frac{k}{\gamma} \tag{4.15}$$

Equation 4.14 and Equation 4.15 are Equation 4.10 and Equation 4.11 again, one level up. That is, the product is fixed by the item data alone and defines a single hyperbola, called the **exchange curve**. The ratio k/γ is a ray from the origin, and where the ray crosses the exchange curve is where the portfolio operates.

Figure 4.1 draws both. The solid curve is the exchange curve for a portfolio whose data give a product of 120,000, so that every point on it satisfies $\bar{I}^a N^a = 120{,}000$. Following the curve from left to right, aggregate investment falls as the aggregate number of replenishments rises, the trade-off Equation 4.14 forces. Notice that the curve flattens toward the right, so that the stock given up for each additional order gets smaller as the portfolio orders more often.

The two dashed lines through the origin are cost ratios. The steeper one is $k/\gamma = 12$, and it meets the curve at 100 replenishments a year and \$1,200 of cycle stock. The shallower one is $k/\gamma = 3$, and it meets the curve at 200 replenishments and \$600. Notice that a firm does not choose a point on the curve directly. It chooses a cost ratio, and the ratio chooses the point.

The triangle marks 200 replenishments against \$1,200 of stock, which lies above the curve. Its product is 240,000 and not 120,000. No set of economic order quantities produces it, and a portfolio found at that point is ordering in quantities that are not economic. We return to what can be done about such a point in Section 4.4.3.

4.4.3. Reading a Cost Ratio from Practice

Figure 4.1 is more useful read backwards than forwards. Instead of choosing a ratio and finding the point, we observe a point and ask what it implies. Two readings matter.

A point off the curve means the quantities are not economic. The curve is the best that can be done. For any given number of orders no policy holds less stock, and for any given stock none orders less often. Thus, a portfolio operating above the curve can move onto it and improve on both measures at once, with no trade-off to argue about and no new money. That is the rare case in which an inventory recommendation costs nothing.

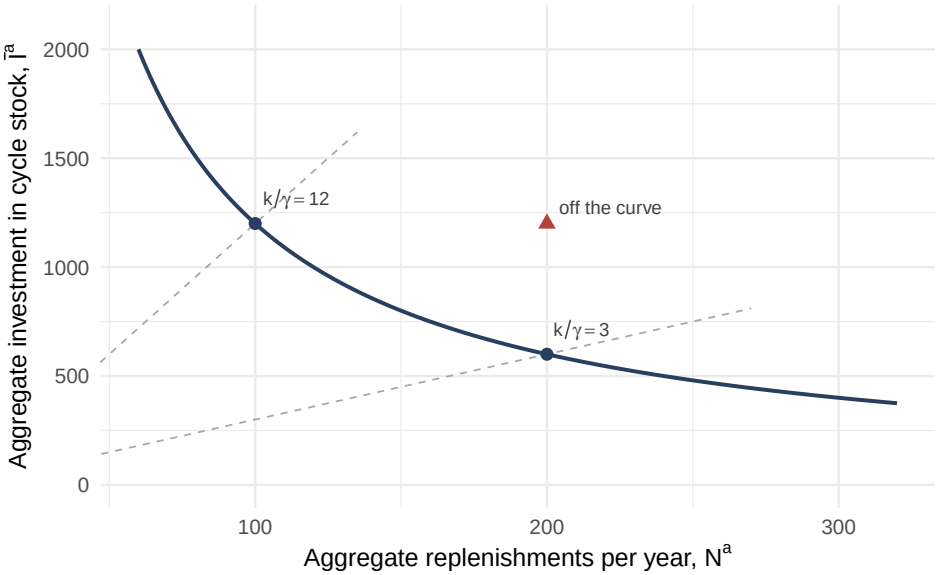


Figure 4.1.: The aggregate exchange curve, drawn for a portfolio whose data give a product of 120,000. Every combination of aggregate investment and aggregate order frequency reachable with economic order quantities lies on the curve. The two rays are cost ratios: $k/\gamma = 12$ meets the curve at 100 orders and \$1,200 of stock, and $k/\gamma = 3$ meets it at 200 orders and \$600. The marked point at 200 orders and \$1,200 of stock lies above the curve, which is only possible if the quantities in use are not economic ones.

A point on the curve reveals a cost ratio. Recall from Equation 4.15 that any operating point implies k/γ. If the firm's actual ordering cost and carrying charge are known, you can compare the implied ratio against them. A discrepancy says that current practice is optimal for costs the firm does not have.

Example 4.4 (What current practice implies)**.** A distributor places 100 replenishment orders a year across a group of items and carries \$1,200 of cycle stock, a point that lies on the exchange curve for that group. From Equation 4.15,

$$\frac{\bar{I}^a}{N^a} = \frac{1200}{100} = 12 = \frac{k}{\gamma}$$

Case 1, the ordering cost is known. Cost estimation of the kind described in Section 1.5.4 puts $k = \$4$ per order. Then

$$\gamma = \frac{k}{12} = \frac{4}{12} = 0.333 \text{ per year}$$

A charge of 33% is what would make current practice optimal. If the firm's actual carrying charge is near 33%, then the practice is defensible. If the true figure is 20%, the ratio that should govern is $4/0.20 = 20$ rather than 12, so the firm is ordering too often for the stock it holds and should move along the curve toward fewer orders and more stock.

Case 2, neither is known. Recall from Section 1.5.5 that firms often cannot defend a number for either cost. The ratio is still informative, because it is what the firm's own behavior asserts. For example, a manager who cannot produce k or γ but who rejects $k/\gamma = 12$ as absurd has learned something about current practice.

Case 3, the operating point is constrained. Suppose transportation arrangements fix the portfolio at 200 orders a year. The curve then gives

$\bar{I}^a = \$600$ and a required ratio of $k/\gamma = 3$. Reaching that ratio cheaply means reducing k, since γ is a property of the firm's cost of capital and is not easily moved. Figure 4.1 gives guidance on changing the environment, not merely on operating within it. It says which cost to attack, and by how much.

4.4.4. When Ordering Costs Differ Across Items

Recall that Equation 4.14 assumed a common k and a common γ. That assumption is what let the curve be drawn without solving anything, and it is often too strong. For example, an item ordered from overseas and an item picked from a local supplier do not cost the same to order.

Dropping the assumption requires the machinery of Section 4.2.

- Let $\overline{OC}^a$ represent the aggregate ordering **cost** rate, in dollars per unit time, and not the count of orders.

That is,

$$\overline{OC}^a = \sum_{j=1}^{n} k_j \frac{\lambda_j}{Q_j}$$

We then pose the problem of minimizing it subject to a fixed total investment:

$$\min \sum_{j=1}^{n} \frac{k_j \lambda_j}{Q_j} \qquad \text{s.t.} \qquad \sum_{j=1}^{n} \frac{c_j Q_j}{2} = \bar{I}^a \tag{4.16}$$

Notice what is absent from Equation 4.16. There is no holding cost in the objective at all. The investment is a constraint and not a cost, the situation when management sets a stock target and asks for the cheapest way to live

within it. Forming the Lagrange function and setting the derivatives to zero, we obtain

$$-\frac{k_j\lambda_j}{Q_j^2}+\frac{\theta c_j}{2}=0 \qquad \Longrightarrow \qquad Q_j=\sqrt{\frac{2k_j\lambda_j}{c_j\theta}} \tag{4.17}$$

Compare Equation 4.17 with the ordinary $Q^*=\sqrt{2k\lambda/(ic)}$. Notice that **the multiplier has taken the place of the carrying charge.** In Section 4.2 the multiplier was added to the holding rate because holding cost was in the objective. Here there is no holding cost to add to, and θ supplies it.

Thus, the same symbol has now carried three meanings, and the formulation decides which one applies: added to the holding rate under a limit on stock (Equation 4.5), added to the ordering cost under a limit on replenishments (Equation 4.7), and standing in for the carrying charge when there is no holding cost at all (Equation 4.17). In this last case it is precisely the implied carrying charge that Example 4.4 backed out of observed practice.

Substituting Equation 4.17 into the constraint and then into the objective, we obtain the general form of the curve:

$$\bar{I}^a\cdot\overline{OC}^a=\frac{1}{2}\left(\sum_{j=1}^{n}\sqrt{k_jc_j\lambda_j}\right)^2 \tag{4.18}$$

Equation 4.18 is Equation 4.14 with λ_jc_j replaced by $k_jc_j\lambda_j$ and the order count replaced by the order cost. The same structure survives, and the curve is traced by varying θ.

The variety index. Zipkin (2000) relates the right-hand side of Equation 4.18 to a measure of how alike the items are.

- Let J^* represent the **variety index**, defined as

$$J^* = \frac{\left(\sum_{j=1}^{n} \sqrt{k_j c_j \lambda_j}\right)^2}{\sum_{j=1}^{n} k_j c_j \lambda_j}, \qquad 1 \le J^* \le n \tag{4.19}$$

The variety index measures dispersion among the products $k_j c_j \lambda_j$. If every item contributes equally, then $J^* = n$. If one item dominates, then J^* approaches 1. The variety index is the **effective** number of items, in the sense that a portfolio of n items with an index of J^* behaves in aggregate like a portfolio of J^* identical ones. With it, Equation 4.18 can be written

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2} J^* \sum_{j=1}^{n} k_j c_j \lambda_j \tag{4.20}$$

Reading the constant. The remaining sum decomposes into quantities a firm already reports.

- Let $C^a = \sum_j c_j \lambda_j$ represent the aggregate purchase cost rate.
- Let $\lambda^a = \sum_j \lambda_j$ represent the aggregate demand rate.

Then $c_j \lambda_j / C^a$ is item j's share of purchasing, and λ_j / λ^a is its share of demand. Weighting by those shares, we obtain

$$K^w = \sum_{j=1}^{n} \left(\frac{c_j \lambda_j}{C^a} \right) k_j, \qquad c^w = \frac{C^a}{\lambda^a} \tag{4.21}$$

where K^w is the purchase-weighted average ordering cost and c^w is the demand-weighted average unit cost. Since $\sum_j k_j c_j \lambda_j = K^w C^a = K^w \lambda^a c^w$, we have

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2} J^* K^w \lambda^a c^w \tag{4.22}$$

Notice that four things move the whole curve, and that each is something a firm can act on: how varied the items are, what an order costs, how much is demanded, and what the goods are worth. Reducing the cost of placing an order shifts the entire frontier inward. Thus, Equation 4.22 is the quantitative version of the argument for simplifying purchasing. We now work a complete portfolio.

Example 4.5 (The exchange curve for a utility storeroom)**.** A municipal electric utility runs a storeroom carrying nine items. Order handling costs $\kappa = \$55$ per hour and item j takes w_j hours per order regardless of the order size, so $k_j = \kappa w_j$ differs across items and the simple curve of Equation 4.14 does not apply. The carrying charge is $\gamma = 0.25$ per year.

Table 4.5.: Item data for the storeroom, with the ordering cost derived from the workload and the rate. The last column is the only aggregation the curve requires.

Item	λ_j	c_j	w_j	k_j	$\sqrt{k_j c_j \lambda_j}$
Pad-mount transformer	45	\$3,800.00	4	\$220.00	6,133.51
Smart meter	1,200	\$95.00	2	\$110.00	3,541.19
Fuse cutout	800	\$115.00	1.5	\$82.50	2,755.00
Meter socket	900	\$48.00	1.5	\$82.50	1,887.86
Riser conduit, 4 in	600	\$62.00	1.5	\$82.50	1,751.86
Splice kit	1,500	\$28.00	1	\$55.00	1,519.87

Item	λ_j	c_j	w_j	k_j	$\sqrt{k_j c_j \lambda_j}$
Ground rod	2,400	\$14.00	0.75	\$41.25	1,177.29
Compression connector	6,000	\$3.40	0.5	\$27.50	749.00
Warning tape, roll	1,000	\$2.10	0.5	\$27.50	240.31
Total					**19,755.87**

The curve follows from the total of the last column:

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2}(19{,}755.87)^2 = 1.951 \times 10^8$$

Varying θ traces it. By Equation 4.17 and the constraint, $\bar{I}^a = \sum_j \sqrt{k_j c_j \lambda_j}/\sqrt{2\theta}$ and $\overline{OC}^a = \sqrt{\theta/2}\sum_j \sqrt{k_j c_j \lambda_j}$.

Table 4.6.: The curve traced by the multiplier. The last column is constant by Equation 4.18, and checking it is the arithmetic check for this section, in the way that the segment sum was for Section 3.1.

θ	$\bar{I}^a$	$\overline{OC}^a$	Order handling	Product
0.05	\$62,474	\$3,124	57 hours	1.951×10^8
0.10	\$44,175	\$4,418	80 hours	1.951×10^8
0.25	\$27,939	\$6,985	127 hours	1.951×10^8
0.50	\$19,756	\$9,878	180 hours	1.951×10^8
1.00	\$13,970	\$13,970	254 hours	1.951×10^8

θ	$\bar{I}^a$	$\overline{OC}^a$	Order handling	Product

By Equation 4.17 the firm's own carrying charge is the multiplier, so $\theta = \gamma = 0.25$ puts the storeroom on the third row: \$27,939 of cycle stock against \$6,985 a year of order handling, or 127 hours of a storekeeper's time. Halving the carrying charge to 0.125 would move it between the second and third rows, raising the stock and cutting the workload, and Equation 4.18 fixes the exchange rate between the two exactly.

Look again at the ratio column. Since $\bar{I}^a/\overline{OC}^a = 1/\theta$ in this formulation, the storeroom at $\theta = 0.25$ holds exactly four dollars of stock for every dollar a year it spends on ordering, and that ratio is the reading Section 4.4.3 takes from observed practice.

The variety index. The denominator of Equation 4.19 is $\sum_j k_j c_j \lambda_j = 6.870 \times 10^7$, so

$$J^* = \frac{(19{,}755.87)^2}{6.870 \times 10^7} = 5.68$$

Nine items behave in aggregate like fewer than six identical ones. The transformer alone contributes 6,133 of the 19,756 in the last column of Table 4.5, and that concentration is what Section 2.2.2 measures a different way.

The decomposition. The storeroom buys $\lambda^a = 14{,}445$ units a year at an aggregate purchase cost rate of $C^a = \$555{,}500$, so the demand-weighted unit cost is $c^w = \$38.46$ and the purchase-weighted ordering cost is $K^w = \$123.67$. Then Equation 4.22 gives

$$\frac{1}{2}(5.6813)(123.67)(14{,}445)(38.46) = 1.951 \times 10^8$$

the same constant reached from the item-by-item sum.

4.5. Joint Replenishment

The second kind of coupling is a shared cost, not a shared resource. Items that come from one supplier, travel on one truck, or run on one machine share part of the cost of replenishing, and that part is incurred once no matter how many items are on the order.

We split the ordering cost in two.

- Let K represent the **major** setup cost, incurred whenever an order is placed at all. For example, the truck, the paperwork, and the machine changeover to the family are all major setup costs.
- Let k_j represent the **minor** setup cost, incurred for each item included on that order. The line on the purchase order, the pick, and the receipt are all minor setup costs.

The minor cost keeps the symbol k_j that item j's ordering cost has carried all along. What is new is K, sitting outside the sum.

Ordering each item independently pays K every time any item is replenished, which for n items means paying it n times over. Coordinating the orders so that items share them pays it once. The saving is the whole of what this section is about.

4.5.1. The Coordinated Problem

Coordination is expressed by a base period and a set of integer multipliers.

- Let T represent the **base period**, the time between successive order opportunities.

- Let m_j represent the multiplier for item j, a positive integer, so that item j is replenished on every m_j-th order opportunity.

Item j's own cycle is m_jT, and its order quantity is $\lambda_j m_j T$. The major cost is incurred once per base period, and item j's minor cost once per m_j base periods. With average inventory $\lambda_j m_j T/2$, the relevant cost of Equation 1.16 is

$$C(T, \vec{m}) = \frac{K}{T} + \sum_{j=1}^{n} \left[\frac{k_j}{m_j T} + \frac{h_j \lambda_j m_j T}{2} \right] \tag{4.23}$$

The decisions are of two kinds. The base period is continuous and the multipliers are integers. The problem is harder than anything in Chapter 3, and the three policies below are ranked by how much structure they impose in order to make it tractable.

Independent ordering. Each item is ordered on its own schedule and pays the full setup. That is, item j faces an ordering cost of $K + k_j$, and its cost is $\sqrt{2(K + k_j)h_j\lambda_j}$, which is Equation 3.18 with the purchase term dropped. This is the baseline that coordination improves on.

A common cycle. Every item is replenished every base period, i.e. $m_j = 1$ for all j. Then Equation 4.23 reduces to a two-term function of T alone. By the argument of Section 3.5.4, its minimum is

$$T^* = \sqrt{\frac{2\left(K + \sum_j k_j\right)}{\sum_j h_j \lambda_j}}, \qquad C^* = \sqrt{2\left(K + \sum_j k_j\right)\sum_j h_j \lambda_j} \tag{4.24}$$

A common cycle is simple and it is often good. It is wasteful when the items differ widely in how often they want to be ordered, because a slow-moving expensive item is dragged onto the same frequency as a fast-moving cheap one.

Integer ratios. Allowing each m_j to be any positive integer is the general problem. Notice in Equation 4.23 that once T is fixed the terms separate, so each multiplier can be chosen on its own. The best continuous value is

$$m_j = \frac{1}{T}\sqrt{\frac{2k_j}{h_j\lambda_j}} \tag{4.25}$$

Notice that the square root in Equation 4.25 is the cycle item j would choose if it paid only its minor cost. Rounding to an integer and searching over T solves the problem to any accuracy wanted. However, the answer is then a set of arbitrary integers, and that turns out to matter.

4.5.2. Power-of-Two Policies

A **power-of-two policy** restricts each multiplier to $m_j = 2^{\ell_j}$ for a non-negative integer ℓ_j. Items are ordered every base period, or every second, fourth, eighth, and so on.

The restriction is imposed for two reasons, and only one of them is mathematical.

Such schedules nest. For example, if one item is ordered every 2 base periods and another every 8, then every time the slow item is due the fast one is due as well, because 2 divides 8. With arbitrary integers this fails. Multipliers of 3 and 5 coincide only every 15 base periods, and in between there are orders that carry one item and not the other. Thus, a nested schedule is one a receiving dock can actually run, and nesting is the reason power-of-two policies are used in practice at all.

They cost almost nothing. That is the subject of Section 4.5.3, and it is the result that makes the restriction respectable instead of merely convenient.

4.5.3. What a Power-of-Two Policy Costs

The bound is easiest to see one item at a time, so we fix the base period T and follow a single item, dropping its subscript on the item's own data. Its reorder interval is $T_j = m_j T$, which the power-of-two restriction writes as $2^\ell T$ for a non-negative integer ℓ.

We begin by putting the model of Section 3.5.4 in terms of the interval instead of the quantity.

- Let $C(T_j)$ represent the relevant cost rate written as a function of the reorder interval.
- Let $g = h\lambda/2$ represent the holding cost coefficient in that form.

Since $Q = \lambda T_j$, substituting into Equation 3.16 and dropping the purchase term gives

$$C(T_j) = \frac{k}{T_j} + \frac{h\lambda T_j}{2} = \frac{k}{T_j} + gT_j, \qquad g \equiv \frac{h\lambda}{2} \tag{4.26}$$

Equation 4.26 is the same two-term function as before with the decision variable renamed. It therefore has the same solution in the new variable:

$$T_j^* = \sqrt{\frac{k}{g}} = \sqrt{\frac{2k}{h\lambda}}, \qquad C(T_j^*) = 2\sqrt{kg} \tag{4.27}$$

Which power to use. Recall that C is convex. The best ℓ is the smallest one for which doubling again does not help. That is, we want the smallest ℓ satisfying $C(2^\ell T) \le C(2^{\ell+1} T)$. Writing that out,

$$\frac{k}{2^\ell T} + g2^\ell T \;\le\; \frac{k}{2^{\ell+1} T} + g2^{\ell+1} T$$

Collecting the k terms on the left and the g terms on the right, we obtain $\frac{k}{2^\ell T}\left(1-\frac{1}{2}\right) \le g 2^\ell T(2-1)$, hence $k/(2g) \le (2^\ell T)^2$. Taking square roots,

$$\ell^* = \text{the smallest non-negative integer with} \quad 2^\ell \ge \frac{T_j^*}{\sqrt{2}\,T} \tag{4.28}$$

Equation 4.28 is a formula, not a search. We take the base-2 logarithm of the right-hand side and round up.

Where the chosen interval sits. Notice that Equation 4.28 already says $T_j \ge T_j^*/\sqrt{2}$. The companion bound comes from running the same argument from the other side. Because ℓ^* is the *smallest* integer that works, the previous power was worse, i.e. $C(2^{\ell^*-1}T) > C(2^{\ell^*}T)$. Expanding that inequality as above yields $2k/g > (2^{\ell^*}T)^2$, so $T_j < \sqrt{2}\,T_j^*$. Together,

$$\frac{T_j^*}{\sqrt{2}} \;\le\; T_j \;<\; \sqrt{2}\,T_j^* \tag{4.29}$$

The worst case. The bracket in Equation 4.29 is the whole story, because C is convex and so takes its largest value over the bracket at one of the two ends. We evaluate it at the lower end, using $T_j^* = \sqrt{k/g}$:

$$C\left(\frac{T_j^*}{\sqrt{2}}\right) = \frac{k\sqrt{2}}{T_j^*} + \frac{gT_j^*}{\sqrt{2}} = \sqrt{2}\sqrt{kg} + \frac{1}{\sqrt{2}}\sqrt{kg} = \left(\sqrt{2}+\frac{1}{\sqrt{2}}\right)\sqrt{kg}$$

and at the upper end,

$$C\big(\sqrt{2}\,T_j^*\big) = \frac{k}{\sqrt{2}\,T_j^*} + g\sqrt{2}\,T_j^* = \frac{1}{\sqrt{2}}\sqrt{kg} + \sqrt{2}\sqrt{kg}$$

Notice that the two are the same expression. Since $\sqrt{2} + 1/\sqrt{2} = 3/\sqrt{2}$ and $C(T_j^*) = 2\sqrt{kg}$, both equal

$$\frac{3}{2\sqrt{2}}\,C(T_j^*) = 1.0607\,C(T_j^*) \tag{4.30}$$

Thus, **a power-of-two policy costs at most 6.07% more than the best interval, whatever the item's data and whatever the base period.** Rounding an interval up to the next power of two and rounding it down to the previous one cost the same in the worst case, so the bracket in Equation 4.29 is symmetric in $\sqrt{2}$.

Two qualifications limit the statement. First, the derivation assumes ℓ^* is non-negative, which requires $T \le \sqrt{2}\,T_j^*$. An item that wants to be ordered more often than the base period allows is stuck at $\ell = 0$, and for that item the base period, not the rounding, governs the cost.

Second, the base period may itself be a decision. Choosing it along with the multipliers improves the worst case from 6% to **2%** (Axsäter 2006). That sharper constant comes from a second route to the same bound. Rather than bracket one item's interval, it treats the rounding error as spread across the items and averages the resulting penalty, leaving the base period free to be optimized. It is also the constant behind Roundy's 98%-effective approximation for multi-stage systems (Roundy 1985). Muckstadt and Sapra (2010) develops power-of-two policies and their multi-stage extensions at length.

Recall from Section 3.7 that the total cost curve is flat near its minimum, and that this is why rounding to a case quantity is cheap. Equation 4.30 is the same fact applied to cycle lengths instead of order quantities. Thus, flatness is what licenses throwing away every multiplier that is not a power of two and keeping a schedule a warehouse can run.

Example 4.6 (Rounding two items to a weekly base period)**.** A plant can act on purchasing once a week, so the base period is $T = 1/52$ year. Two items are bought from the same supplier.

Table 4.7.: Two items facing a weekly base period.

Item	λ_j	h_j	k_j	$g_j = h_j\lambda_j/2$
Drive unit	8,000	\$5.00	\$237	20,000
Control arm	6,000	\$4.00	\$300	12,000

The unconstrained intervals. From Equation 4.27,

$$T^*_{\text{drive}} = \sqrt{\frac{237}{20{,}000}} = 0.10886 \text{ yr} = 5.66 \text{ weeks}, \qquad T^*_{\text{arm}} = \sqrt{\frac{300}{12{,}000}} = 0.15811 \text{ yr} = 8.22 \text{ weeks}$$

Neither is a whole number of weeks, let alone a power of two of them.

Rounding. Apply Equation 4.28 to the drive unit:

$$2^\ell \geq \frac{T^*_j}{\sqrt{2}\,T} = \frac{5.66}{\sqrt{2}} = 4.003 \quad \Longrightarrow \quad \ell \geq \frac{\ln 4.003}{\ln 2} = 2.001 \quad \Longrightarrow \quad \ell^* = 3$$

so the drive unit is ordered every $2^3 = 8$ weeks. For the control arm, $2^\ell \geq 8.22/\sqrt{2} = 5.81$, giving $\ell \geq 2.54$ and again $\ell^* = 3$, so it too is ordered every 8 weeks.

What the rounding cost. The drive unit was pushed from 5.66 weeks out to 8, a ratio of 1.4133, which is as far from 1 as Equation 4.29 permits. By Equation 3.22 its cost rises from \$4,354.31 to \$4,617.42, which is **6.04%**, essentially the worst case of Equation 4.30. The control arm was pulled from 8.22 weeks in to 8, a ratio of 0.9730, costing **0.04%**.

Two readings. The penalty is bounded but not evenly spread: one item paid almost the entire worst case and the other paid nothing, and which one pays depends only on where its unconstrained interval happens to fall

between two powers of two. And the two items ended up on the same eight-week cycle with no one coordinating them, which is the property Section 4.5.2 was after. Ordering them together is now free.

Example 4.7 (Coordinating four items from one supplier)**.** Four items are bought from one supplier. Placing an order costs $K = \$400$ whatever is on it, and each item added to an order costs a further k_j. The carrying charge is 25% per year.

Table 4.8.: Four items sharing a supplier, with the cycle each would choose on its minor cost alone in the last column.

Item	λ_j	c_j	h_j	k_j	Annual value $c_j\lambda_j$	$\sqrt{2k_j/(h_j\lambda_j)}$
Drive unit	8,000	\$20	\$5.00	\$25	\$160,000	0.0354
Gear-box	800	\$20	\$5.00	\$20	\$16,000	0.1000
Seal kit	200	\$8	\$2.00	\$15	\$1,600	0.2739
Name plate	40	\$4	\$1.00	\$15	\$160	0.8660

The last column is the cycle each item would want if it paid only its minor cost, and it spans a factor of 24. The annual values span a factor of a thousand, the ABC spread of Section 2.2.2, and it is the reason the multipliers will not all be one.

Independent ordering. Each item paying $K + k_j$ on its own costs \$8,422.39 a year.

A common cycle. From Equation 4.24, $T^* = 0.1462$ years, or 53 days, at a cost of \$6,497.54. Coordinating alone, with no differentiation between items, has already saved 23%.

The best integer-ratio policy. Searching over T with Equation 4.25 gives a base period of 0.14214 years, or 51.9 days, and multipliers $(1, 1, 2, 6)$, costing \$6,402.06.

The power-of-two policy. Restricting to powers of two gives multipliers $(1, 1, 2, 8)$ at a base period of 0.14192 years, or 51.8 days, costing \$6,403.34. Notice that the two policies do not share a base period. The search runs over T and the multipliers, so changing what multipliers are admissible moves the best T with them. That is **0.020%** above the best integer-ratio policy, against a worst case of 6.07%. The drive unit and the gearbox sit at $m_j = 1$ because the cycles in the last column of Table 4.8, 13 and 37 days, are shorter than the base period; nothing about them is rounded, and what sets their cost is the choice of base period.

The resulting schedule is

Table 4.9.: The power-of-two schedule on a base period of 51.8 days. Each interval is that base period times the multiplier, rounded to the nearest day, which is why the last row reads 414 and not eight times 52.

Item	Multiplier	Ordered every
Drive unit	1	52 days
Gearbox	1	52 days
Seal kit	2	104 days
Name plate	8	414 days

and it nests: every second order opportunity the seal kit joins an order the drive unit and gearbox were placing anyway, and every eighth the name plate joins one of those. No order is ever placed for one item alone.

Two things can be read off. Coordination saved 24% against ordering separately, and almost all of that came from sharing orders and not from choosing clever multipliers: the common cycle captured 23 of the 24 points. And the power-of-two restriction, which makes the schedule runnable, cost two hundredths of one percent. The bound of Equation 4.30 is a worst case that a real instance rarely approaches.

4.6. Multi-Echelon Systems

Everything so far has coupled items that sit side by side. Items compete for a budget, or share a truck, but none of them feeds another. A **multi-echelon** system couples them end to end. That is, stock at one location exists in order to supply the next, and a downstream order is possible only because one was placed upstream.

The decision is still a set of reorder intervals, and the tool is still the power-of-two policy of Section 4.5.2. What changes is the accounting. Holding cost can no longer be charged location by location without double counting, because a unit sitting at a regional warehouse has already been paid for by the central warehouse that shipped it. Section 4.6.1 therefore fixes the accounting first, and the two sections after it apply it to the two structures that matter.

One point of notation before we start. The echelon sections have no major setup cost, because each location places its own orders and nothing is shared across a family. Thus, the K of Section 4.5 does not appear here, and the ordering cost at a location is written k_i, the same k the book has used for a fixed order cost since Section 1.4.5. The index changes from j to i as well, since what is counted is now a location, not an item.

Demand is still known and constant throughout. Chapter 9 returns to these same structures once demand is random.

4.6.1. Echelon Inventory and Echelon Holding Cost

We number the stages so that stage i supplies stage $i-1$, with stage 1 facing the customer:

$$n \longrightarrow n-1 \longrightarrow \cdots \longrightarrow 2 \longrightarrow 1$$

Every stage sees the same demand rate λ, because a unit consumed at stage 1 must have passed through every stage above it.

The **echelon stock** of stage i is the total on-hand inventory at stages 1 through i. That is, it is everything at stage i together with everything that has already moved downstream of it. Table 4.10 gives an instance for a three stage chain.

Table 4.10.: On-hand and echelon stock for a three stage chain. Stage 3 owns six units in the sense that matters, even though only one is sitting at stage 3.

Stage	1	2	3
On hand	2	3	1
Echelon stock	2	5	6

Notice in Table 4.10 that stage 3 owns six units in the sense that matters, even though only one of them is sitting at stage 3. Echelon stock is the natural quantity because it is the one that behaves like the inventory in Chapter 3. Stage i's echelon stock falls at the demand rate λ and jumps when stage i orders, which is the sawtooth the EOQ was built for. The on-hand stock at stage i does not behave that way at all, because it jumps down whenever the stage below it orders, in steps and not continuously.

Echelon holding cost. If echelon stock counts a unit at several stages at once, then charging each stage its full holding cost rate would count the same money several times.

- Let h_i represent the conventional, or **installation**, holding rate at stage i, which measures the total value accumulated in a unit up to and including that stage.
- Let h'_i represent the **echelon holding cost** at stage i, which charges only the value that stage i itself adds.

$$h'_i = h_i - h_{i+1}, \qquad h'_n = h_n \tag{4.31}$$

The definition telescopes. Nothing is lost and nothing is double counted:

$$\sum_{j=i}^{n} h'_j = (h_i - h_{i+1}) + (h_{i+1} - h_{i+2}) + \cdots + h_n = h_i \tag{4.32}$$

The total value-added rate carried up through stage i is recovered by adding the increments from stage i upward. The rate h_i decreases as i increases, because value accumulates on the way down the chain. h'_i is non-negative, and a stage that adds no value has $h'_i = 0$.

Example 4.8 (The two accountings agree)**.** Take a two stage chain under a nested policy with $T_1 = \frac{1}{2}T_2$, so that stage 1 orders twice for every order stage 2 places. Lead times are zero. Installation holding rates are $h_1 = \$2.00$ and $h_2 = \$1.20$ per unit per year. By Equation 4.31, the echelon rates are $h'_1 = h_1 - h_2 = \$0.80$ and $h'_2 = h_2 = \$1.20$ per unit per year.

Figure 4.2 follows two full cycles, with $\lambda = 100$ units per year, $T_1 = 1/2$ year and $T_2 = 1$ year, so that $Q_1 = 50$ and $Q_2 = 100$ units. The three panels share one vertical scale, so you can add them by eye. The top panel is stage 1 on hand, an ordinary sawtooth falling from 50 to zero over each half year. The middle panel is stage 2 on hand, which is flat and not sloping, because stage 2 faces no external demand. Notice that it holds 50 units for the first half of the cycle and nothing for the second half. The bottom panel is stage 2 echelon stock, and it is the sum of the two panels

above it at every instant. We now cost the same system twice, once from the top two panels and once from the bottom one.

Following the on-hand stock. At the start of a cycle stage 2 orders $Q_2 = \lambda T_2$ and, because the policy is nested, stage 1 simultaneously orders $Q_1 = \lambda T_1 = \frac{1}{2}Q_2$, which ships immediately. Stage 2 is left holding $Q_2 - Q_1 = \frac{1}{2}\lambda T_2$. It holds that amount for T_1, until stage 1 reorders and takes the rest, after which stage 2 holds nothing for the remainder of the cycle. Its average on-hand cost rate is

$$h_2 \frac{\left(\frac{1}{2}\lambda T_2\right) T_1}{T_2} = \frac{1}{2} h_2 \lambda T_1$$

It is a rectangle of height $\frac{1}{2}\lambda T_2$ and width T_1, averaged over a cycle of length T_2, the shaded block in the middle panel of Figure 4.2. Stage 1 is an ordinary sawtooth averaging $\bar{I}_1 = Q_1/2$.

- Let $\bar{I}_i$ represent the average on-hand stock at stage i.
- Let $\bar{I}_i^e$ represent the average echelon stock at stage i.

Writing the two on-hand averages the way Figure 4.2 labels them,

$$\bar{I}_1 = \frac{Q_1}{2} = \frac{\lambda T_1}{2}, \qquad \bar{I}_2 = \frac{(Q_2 - Q_1)T_1}{T_2} = \frac{\lambda T_1}{2} \tag{4.33}$$

The system pays $h_1\bar{I}_1 + h_2\bar{I}_2$, which is

$$\frac{1}{2}h_1\lambda T_1 + \frac{1}{2}h_2\lambda T_1 = \frac{1}{2}(h_1 + h_2)\lambda T_1 \tag{4.34}$$

Following the echelon stock. Stage 2's echelon stock starts the cycle at $Q_2 = \lambda T_2$ and declines at rate λ to zero, i.e. a clean sawtooth over T_2, because units shipped to stage 1 stay inside stage 2's echelon. Notice in Figure 4.2 that the red transfer at T_1 drops the middle panel to zero

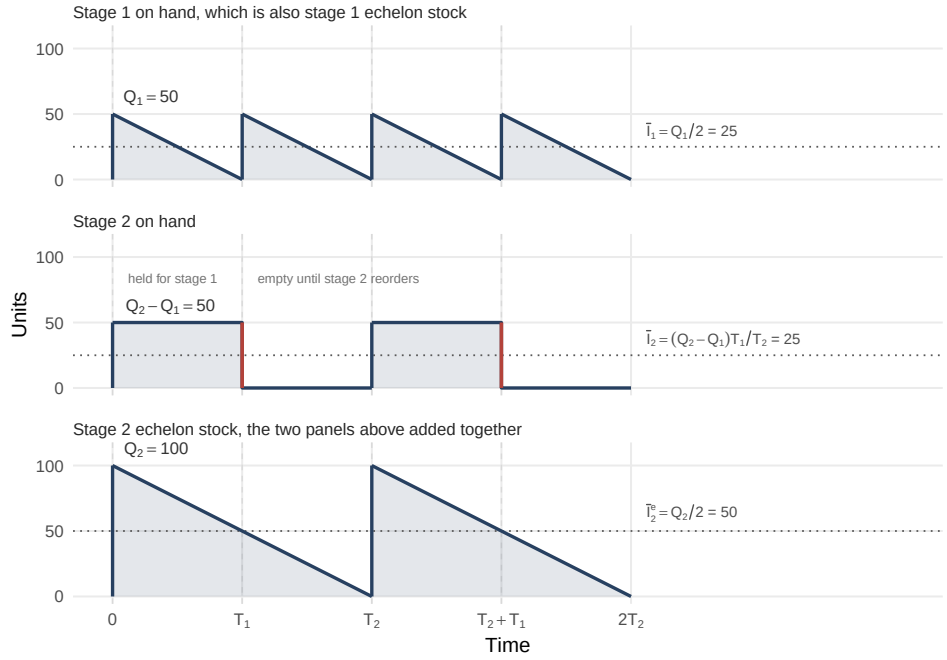


Figure 4.2.: On-hand and echelon stock for a two stage chain under a nested policy, drawn with $\lambda = 100$ units per year, $T_1 = 1/2$ year and $T_2 = 1$ year, so $Q_1 = 50$ and $Q_2 = 100$. Stage 1 is an ordinary sawtooth. Stage 2's on-hand stock is not: it receives Q_2, ships Q_1 downstream at once, holds the remainder until stage 1 reorders at T_1, and is then empty for the rest of the cycle. The red drops are transfers to stage 1, not demand. Adding the two upper panels gives the third, in which the transfers cancel, because a unit moved from stage 2 to stage 1 is still inside stage 2's echelon. What is left is a clean sawtooth of height Q_2. The dotted lines mark the averages that enter the cost.

and lifts the top panel by the same amount. Thus, the bottom panel passes through T_1 unbroken, and you should remember that mechanism. Its average is the third dotted line the figure marks:

$$\bar{I}_2^e = \frac{Q_2}{2} = \frac{\lambda T_2}{2} \tag{4.35}$$

Stage 1's echelon stock is its on-hand stock, so that $\bar{I}_1^e = \bar{I}_1$. The system pays $h_1'\bar{I}_1^e + h_2'\bar{I}_2^e$, and with $h_2' = h_2$, $h_1' = h_1 - h_2$, and $T_2 = 2T_1$, we obtain

$$\frac{1}{2}(h_1 - h_2)\lambda T_1 + \frac{1}{2}h_2\lambda T_2 = \frac{1}{2}(h_1 - h_2)\lambda T_1 + \frac{1}{2}h_2\lambda(2T_1) = \frac{1}{2}(h_1 + h_2)\lambda T_1 \tag{4.36}$$

the same total as Equation 4.34.

The arithmetic check. At the rates above, the three dotted lines in Figure 4.2 are 25, 25 and 50 units, and each cost term is a rate times one of them. Working both accountings side by side gives Table 4.11.

Table 4.11.: The same system costed both ways. Each average is a dotted line in Figure 4.2 and each rate comes from Equation 4.31.

Accounting	Term	Average	Rate	Cost rate
Installation	stage 1	$\bar{I}_1 = 25$	$h_1 = 2.00$	\$50.00
	stage 2	$\bar{I}_2 = 25$	$h_2 = 1.20$	\$30.00
	total			**\$80.00**
Echelon	stage 1	$\bar{I}_1^e = 25$	$h_1' = 0.80$	\$20.00
	stage 2	$\bar{I}_2^e = 50$	$h_2' = 1.20$	\$60.00
	total			**\$80.00**

Both totals are $\frac{1}{2}(h_1 + h_2)\lambda T_1 = \frac{1}{2}(3.20)(100)(0.5) = \$80$ per year, as Equation 4.34 and Equation 4.36 require. You should perform that check

every time echelon rates are introduced, because it catches a sign error in Equation 4.31 immediately.

Notice what differs. Installation accounting charges the two stages \$50 and \$30, while echelon accounting charges them \$20 and \$60. Echelon accounting moves cost upstream, because stage 2's echelon stock includes what stage 1 is holding. Thus, **echelon accounting reassigns cost between stages and preserves the total.** That is the license to use it. No money is invented or lost, and in exchange one awkward rectangle becomes a sawtooth, so that every stage looks like an EOQ.

4.6.2. Serial Systems

With the accounting settled, we can write the problem down.

- Let n represent the number of stages in the chain, indexed by i, so that stage 1 faces the customer and stage n orders from the outside supplier.
- Let k_i represent the fixed cost of placing an order at stage i.
- Let $g_i = \frac{1}{2}\lambda h_i'$ represent the holding cost coefficient at stage i, by analogy with Equation 4.26. Every stage sees the same λ, so no subscript is needed on it here; the distribution system of Section 4.6.3 is where that changes.
- Let T_i represent the reorder interval at stage i.

Stage i holding echelon stock on a cycle of T_i therefore costs $g_i T_i$ per unit time, and adding the ordering cost k_i/T_i at each stage gives

$$\min \sum_{i=1}^{n} \left[\frac{k_i}{T_i} + g_i T_i \right] \tag{4.37}$$

subject to two restrictions:

$$T_i = 2^{\ell_i} T, \qquad \ell_i = 0, 1, 2, ... \tag{4.38}$$

$$T_i \geq T_{i-1} \geq 0 \tag{4.39}$$

Equation 4.38 is the power-of-two restriction of Section 4.5.2. Equation 4.39 is **nestedness**. That is, whenever a stage orders, every stage below it orders too, so no stage can replenish more often than the one it supplies. Nestedness keeps work in process from piling up between stages. Notice that with power-of-two intervals it comes for free, because any two powers of two divide one another.

The relaxation. We drop Equation 4.38 and keep Equation 4.39:

$$(RP) \qquad \min \sum_{i=1}^{n} \left[\frac{k_i}{T_i} + g_i T_i \right] \qquad \text{s.t.} \qquad T_i \geq T_{i-1} \geq 0 \tag{4.40}$$

Without the nesting constraint each stage would separate and take $T_i = \sqrt{k_i / g_i}$, which is Equation 4.27 stage by stage. Thus, the constraint binds exactly when those intervals fail to increase up the chain. One feature of the answer makes the problem solvable: **stages that would violate the constraint are merged into a block and share one interval.**

Blocks. We partition the chain into consecutive blocks.

- Let G_r represent the rth block of consecutive stages, with G_1 holding the lowest-numbered stages, and let M represent the number of blocks.
- Let $T(r)$ represent the interval shared by every stage in G_r.
- Let $k(G_r) = \sum_{i \in G_r} k_i$ and $g(G_r) = \sum_{i \in G_r} g_i$ represent the block's totals.

If every stage in block G_r shares the common interval $T(r)$, then the terms for that block collapse into a single two-term function of $T(r)$. Its solution is Equation 4.27 again with the block's totals in place of one stage's:

$$T(r) = \sqrt{\frac{k(G_r)}{g(G_r)}} \tag{4.41}$$

A set of blocks solves Equation 4.40 when three conditions hold.

1. Each block's interval is given by Equation 4.41.
2. The intervals increase, $T(1) \le T(2) \le \cdots \le T(M)$, so that the solution is feasible.
3. No block can be split into a lower part and an upper part whose ratio of block totals, $k(\cdot)/g(\cdot)$, is larger in the upper part.

We dwell on the third condition. If such a split existed, then the upper part could lengthen its interval on its own and save money. Thus, a partition admitting such a split cannot be optimal, and Algorithm 4.1 is built to prevent one.

The algorithm. We walk up the chain, adding stages to the current block while doing so is forced, and starting a new block when the next stage wants a longer interval than the current block has.

Algorithm 4.1 Partitioning a serial chain into blocks that share a reorder interval. `<-` is assignment and `+` on blocks is set union. The ratio $k(G_r)/g(G_r)$ is the block total of Equation 4.41, so a block's own interval is always $\sqrt{k(G_r)/g(G_r)}$.

```
partition(stages 1 to n):
    r <- 1                      // index of the block being built
    G_1 <- {1}                  // stage 1 opens the first block

    for i = 2 to n:
        if k(G_r)/g(G_r) <= k_i/g_i:
            // stage i wants a longer interval: it stands alone
            r <- r + 1
            G_r <- {i}
        else:
            // stage i wants a shorter one, which is infeasible
            // upstream, so the current block absorbs it
            G_r <- G_r + {i}
            // absorbing i lowered this ratio, so it may now sit
            // below the block beneath and break condition 2
            r <- merge(r)

    return G_1 through G_r


merge(r):
    s <- r
    while s > 1 and k(G_{s-1})/g(G_{s-1}) > k(G_s)/g(G_s):
        // the lower block wants the longer interval, so the two
        // cannot stand apart: fold s into s-1 and retest
        G_{s-1} <- G_{s-1} + G_s
        s <- s - 1
    return s
```

Now consider what `merge` is for. Absorbing a stage changes the block's totals, which can push its ratio below the block beneath it and break condition 2. `merge` walks back down the chain repairing that, folding adjacent blocks together until the ratios increase again. Notice that a single absorption can cascade, merging several blocks at once.

Example 4.9 (A five stage chain)**.** A component passes through five stages, $5 \to 4 \to 3 \to 2 \to 1$, at $\lambda = 500$ units per year. The plant can act on replenishment once a week, so $T = 1/52$ year.

Table 4.12.: A five stage chain. The echelon rates come from Equation 4.31, with $h'_5 = h_5$.

Stage i	1	2	3	4	5
k_i	\$10	\$12	\$25	\$7	\$2
h_i	\$0.60	\$0.50	\$0.35	\$0.10	\$0.05
$h'_i = h_i - h_{i+1}$	0.10	0.15	0.25	0.05	0.05
$g_i = \frac{1}{2}\lambda h'_i$	25	37.5	62.5	12.5	12.5
k_i/g_i	0.400	0.320	0.400	0.560	0.160

Why the constraint binds. Taking each stage on its own gives intervals $\sqrt{k_i/g_i}$ of 0.632, 0.566, 0.632, 0.748, and 0.400 years. These are not nondecreasing, so the schedule is infeasible: stage 2 would order less often than stage 1 supplies, and stage 5 less often than anything below it.

Running the algorithm.

Table 4.13.: The forward pass.

i	Current block ratio	k_i/g_i	Action
2	$\frac{10}{25} = .400$	$\frac{12}{37.5} = .320$	$.400 > .320$, absorb: $G_1 = \{1, 2\}$
3	$\frac{22}{62.5} = .352$	$\frac{25}{62.5} = .400$	$.352 \le .400$, start $G_2 = \{3\}$
4	$\frac{25}{62.5} = .400$	$\frac{7}{12.5} = .560$	$.400 \le .560$, start $G_3 = \{4\}$
5	$\frac{7}{12.5} = .560$	$\frac{2}{12.5} = .160$	$.560 > .160$, absorb: $G_3 = \{4, 5\}$, then merge

The merge: G_3 now has ratio $\frac{9}{25} = .360$, below G_2's .400, so the two combine into $G_2 = \{3, 4, 5\}$ with ratio $\frac{34}{87.5} = .389$. That exceeds G_1's .352, so the walk stops. The partition is

$$G_1 = \{1, 2\}, \qquad G_2 = \{3, 4, 5\}$$

The relaxed intervals. From Equation 4.41,

$$T(1) = \sqrt{\frac{22}{62.5}} = 0.5933 \text{ yr} = 30.9 \text{ weeks}, \qquad T(2) = \sqrt{\frac{34}{87.5}} = 0.6234 \text{ yr} = 32.4 \text{ weeks}$$

which are nondecreasing, so the solution is feasible, at a cost of \$183.25 a year. The infeasible stage-by-stage solution would have cost \$181.81, so nesting the chain costs 0.8%.

Rounding to powers of two. Apply Equation 4.28 to each block. For G_1, $2^\ell \geq 30.9/\sqrt{2} = 21.8$, so $\ell \geq 4.45$ and $\ell^* = 5$. For G_2, $2^\ell \geq 32.4/\sqrt{2} = 22.9$, so $\ell \geq 4.52$ and again $\ell^* = 5$. Both blocks land on $2^5 = 32$ weeks, at a total of \$183.31, which is 0.03% above the relaxed solution.

The two blocks were distinguished by the relaxation and then rounded back together. That is not a failure of the method but a consequence of Equation 4.29: intervals within a factor of two of one another often round to the same power, and 30.9 and 32.4 weeks differ by 5%. When it happens the whole chain replenishes on one schedule, the strongest form of the coordination Section 4.5 was seeking.

4.6.3. Distribution Systems

Now we reverse the geometry. A central warehouse supplies several regional warehouses.

- Let index 0 denote the central warehouse, and let indices 1 through n denote the regional warehouses.
- Let λ_i represent the demand rate faced by regional warehouse i, so that $\lambda_0 = \sum_{i \geq 1} \lambda_i$ passes through the center.

Recall from Section 4.6.1 that the echelon stock of a location is everything at that location together with everything downstream of it. The echelon stock of the central warehouse is therefore everything in the system, i.e. its own on-hand plus all of the regional on-hand. Its echelon rate is its installation rate, $h'_0 = h_0$, and each regional warehouse is charged only what it adds:

$$h'_i = h_i - h_0, \qquad i = 1, 2, \dots, n \tag{4.42}$$

With $g_i = \frac{1}{2}\lambda_i h'_i$, which carries each location's own demand rate instead of the single λ a serial chain shares, the problem looks like Equation 4.37 with the index starting at 0. However, the nesting constraint runs the other way:

$$T_0 \geq T_i \geq 0, \qquad i = 1, 2, \ldots, n \tag{4.43}$$

Recall that in a chain a stage cannot order more often than the stage it supplies. Here the central warehouse supplies everyone, so nothing downstream may wait longer than the center. That is, a regional warehouse ordering less often than the center would leave stock sitting at the center with nowhere to go. Thus, Equation 4.43 caps each regional interval at T_0 instead of ordering the regions among themselves.

The shape of the answer. Relaxing the power-of-two restriction leaves a problem whose solution splits the regional warehouses into two groups. Those whose unconstrained interval $\sqrt{k_i/g_i}$ is short enough keep it and order on their own schedule. Those that would have wanted a longer interval than the center can offer are pinned to the center's interval and nested with it.

- Let $\mathcal{C}^0$ represent the **pinned set**, consisting of the central warehouse together with every region sharing its interval.

Building $\mathcal{C}^0$. We relabel the regional warehouses in ascending order of k_i/g_i, so that (1) wants the shortest interval and (n) the longest. We then work down from (n), adding a warehouse to $\mathcal{C}^0$ only when leaving it out would be infeasible. Adding warehouse i changes the center's interval to the pooled value

$$T_0 = \sqrt{\frac{\sum_{j \in \mathcal{C}^0} k_j}{\sum_{j \in \mathcal{C}^0} g_j}} \tag{4.44}$$

The test at each step compares warehouse i's own ratio against the pooled ratio of everything already in $\mathcal{C}^0$. If k_i/g_i is larger, then warehouse i standing alone would want a longer interval than the center has, violating Equation 4.43, and it must join. We stop at the first warehouse for which k_i/g_i is smaller, since that warehouse and everything below it can be left

alone. If even (n) fails the test at the start, then $\mathcal{C}^0$ is empty and every location takes $\sqrt{k_i/g_i}$.

Notice that this is a scan rather than a search. One pass down the list suffices only because the regions were sorted first, so an implementation of this algorithm sorts once and then never looks backward.

Example 4.10 (A central warehouse and six regions)**.** Six regional warehouses each face $\lambda_i = 200$ units per year, so the center sees $\lambda_0 = 1{,}200$. Replenishment decisions are made weekly, $T = 1/52$ year.

Table 4.14.: The distribution instance, with $h_i' = h_i - h_0$ and $g_i = \frac{1}{2}\lambda_i h_i'$.

i	0	1	2	3	4	5	6
k_i	\$500	\$100	\$125	\$110	\$150	\$75	\$90
h_i	\$10	\$15	\$12	\$20	\$18	\$20	\$17
h_i'	10	5	2	10	8	10	7
g_i	6,000	500	200	1,000	800	1,000	700
k_i/g_i	.0833	.200	.625	.110	.1875	.075	.1286

Relabel. Sorting the regions by k_i/g_i puts them in the order $5, 3, 6, 4, 1, 2$, which become (1) through (6):

Table 4.15.: The regions relabeled in ascending order of k_i/g_i.

New label	(1)	(2)	(3)	(4)	(5)	(6)
Was	5	3	6	4	1	2
k_i	\$75	\$110	\$90	\$150	\$100	\$125
g	1,000	1,000	700	800	500	200
k_i/g_i	.075	.110	.1286	.1875	.200	.625

Build $\mathcal{C}^0$, starting from $\mathcal{C}^0 = \{0\}$ with pooled ratio $500/6{,}000 = .0833$:

Table 4.16.: Constructing the pinned set.

Step	Candidate	Its k_i/g_i	Pooled $k(\mathcal{C}^0)/g(\mathcal{C}^0)$	Action
1	(6)	.625	$\frac{500}{6000} = .0833$	larger, add
2	(5)	.200	$\frac{625}{6200} = .1008$	larger, add
3	(4)	.1875	$\frac{725}{6700} = .1082$	larger, add
4	(3)	.1286	$\frac{875}{7500} = .1167$	larger, add
5	(2)	.110	$\frac{965}{8200} = .1177$	smaller, stop

So $\mathcal{C}^0 = \{0, (3), (4), (5), (6)\}$ and regions (1) and (2) are left alone. From Equation 4.44,

$$T_0 = \sqrt{\frac{965}{8{,}200}} = 0.3430 \text{ yr} = 17.8 \text{ weeks}$$

shared by the center and four regions, while (2) takes $\sqrt{.110} = 0.3317$ yr $= 17.2$ weeks and (1) takes $\sqrt{.075} = 0.2739$ yr $= 14.2$ weeks.

Rounding. For the center, $2^\ell \geq 17.8/\sqrt{2} = 12.6$, so $\ell \geq 3.66$ and $\ell^* = 4$, giving $2^4 = 16$ weeks. Region (2) needs $2^\ell \geq 12.2$ and region (1) needs $2^\ell \geq 10.1$; both give $\ell^* = 4$ as well.

Every location ends up on a sixteen-week cycle. As in Example 4.9, the relaxation distinguished three different intervals and the power-of-two rounding put them back together, because all three fell in the same rounding band. By Equation 4.28 the choice $\ell^* = 4$ covers every interval from $8\sqrt{2} = 11.3$ weeks to $16\sqrt{2} = 22.6$ weeks, and 14.2, 17.2 and 17.8 all sit inside it. The system is fully synchronized, and the construction of $\mathcal{C}^0$ was still what proved the schedule optimal rather than merely convenient.

4.7. Building These Models in a Worksheet

Recall that Section 3.10 built a worksheet for a single item, and that its layout followed the shape of the mathematics: inputs in one block, quantities derived from them in a second, and two output blocks evaluating the same rows first at a quantity someone typed and then at Q^*. That layout worked because the decision was one number.

Every model in this chapter breaks that assumption, and the workbook that accompanies it, `Chapter4Models.xlsx`, carries one sheet per model. The decision is a vector, the items are tied together by something that belongs to none of them, and two of the models are algorithms, not formulas. Each of those changes forces the implementation into a different shape.

In the sections that follow we work through the models one at a time, in the order the chapter introduced them, showing how each piece of the mathematics becomes a piece of a worksheet. We build a worksheet first because everything in it is visible: a formula is a cell you can click on, and a layout is an argument about the model that you can see. Section 4.8 then asks the same question of a program, where nothing is visible and the structure has to be designed instead of looked at.

4.7.1. The Constrained Problem

Recall what we have to compute. Each item has an unconstrained quantity from Equation 3.17. The resource those quantities consume is compared against the limit. If the limit binds, a multiplier is found by bisection on Equation 4.6, and every quantity is recomputed from Equation 4.5 at that multiplier.

The blocks. We take them in the order the sheet reads, which Table 4.17 gives.

Table 4.17.: The blocks of the constrained lot sizing worksheet, top to bottom. The first three are Section 4.2.1 and Section 4.2.3, and the item block is Equation 4.5 applied row by row.

Rows	Block	What it holds
5 to 7	Inputs	the carrying charge, the limit A, and a selector reading Budget or Space
10 to 11	Does the limit bind?	the resource the unconstrained quantities would consume, and the verdict
14 to 18	Solve	the multiplier, $R(\theta)$, and whether it has been driven to zero
22 to 27	Results	the same six quantities unconstrained and constrained
30 to 32	What the limit cost	a difference, a percentage, and the investment released
35 to 38	Check	what the shipped data should produce
43 onward	Item data	one row per item

Read Figure 4.3 from the top down and it follows the order of Section 4.2. The verdict sits above the multiplier and not below it, because Section 4.2.1 says an unconstrained solution already inside the limit is the answer and nothing further needs solving. Notice also that four columns hold the whole of it. Everything that is one number about the problem, and not one number about an item, lives here, and there are not many such numbers.

	A	B	C	D	E	F	G	H
1	**Constrained Lot Sizing**							
2	*Chapter 4, sections 4.2 and 4.3. One resource is shared by every item. Goal Seek prices it.*							
3								
4	**Inputs**							
5	Carrying charge (i)	30.0%	*per year*					
6	Resource limit (A)	$10,000.00	*dollars*					
7	Constraint type	Budget	*Budget or Space*					
8								
9	**Does the limit bind?**							
10	Resource the free qu	12,198.77						
11	Binds?	yes						
12								
13	**Solve for the multiplier**							
14	Multiplier (theta)	0.146429841	*Goal Seek changes this cell*					
15	R(theta), equation 4.	0.00000291						
16	Status	solved						
17	How	*Data > What-If Analysis > Goal Seek. Set cell B15 to value 0 by changing cell B14.*						
18	Why it is safe	*Section 4.2.3 proves R decreases strictly and has one root, which is what a numerical search needs and cannot verify for itself.*						
19								
20	**Results**							
21		**Unconstrained**	**Constrained**					
22	Ordering cost	$3,659.63	$4,464.30					
23	Holding cost	$3,659.63	$3,000.00					
24	Relevant cost	$7,319.26	$7,464.30					
25	Average investment	$12,198.77	$10,000.00					
26	Replenishments per	73.19	89.29					
27	Resource used	12,198.77	10,000.00					
28								
29	**What the limit cost**							
30	Extra cost per year	$145.04						
31	As a fraction of the fi	1.98%						
32	Investment released	$2,198.77						
33								
34	**Check**							
35	**Unconstrained releva**	$7,319.26, either constraint type						
36	**Budget, limit 10,000**	theta = 0.146429841, relevant cost $7,464.30						
37	**Budget arithmetic ch**	holding cost = i times the limit = $3,000.00						
38	**Space, limit 800**	theta = 2.953823172, relevant cost $7,723.58						

Figure 4.3.: The constrained lot sizing worksheet, rows 1 to 38. Inputs come first, then the question of whether the limit binds, then the multiplier with a cell saying whether it has been solved, then the results as two columns, unconstrained beside constrained. The last two blocks price the limit and record what the shipped data should produce.

The cells that carry the mathematics. We translate them cell by cell in Table 4.18. Read the last column first: every row of the table is one line of Section 4.2 rewritten in a language Excel evaluates.

Table 4.18.: The six cells of the constrained worksheet that implement a result from Section 4.2. Every one of them reaches the item block through a named range rather than a cell address.

Cell	Holds	Formula	Which result it is
`B10`	resource the free quantities consume	`=SUMPRODUCT(Con_a,Con_Qfree)`	the left side of Equation 4.2 at Q_j^*
`B11`	does the limit bind	`=IF(B10>B6,"yes","no")`	the test of Section 4.2.1
`B15`	$R(\theta)$	`=IF(B11="no",0,SUMPRODUCT(Con_a,Con_Qheld)-B6)`	the slack driven to zero
`C22`	constrained ordering cost	`=SUMPRODUCT(Con_K,Con_D/Con_Qheld)`	$\sum_j K_j D_j / Q_j(\theta)$
`C23`	constrained holding cost	`=SUMPRODUCT(Con_h,Con_Qheld/2)/2`	$\sum_j h_j Q_j(\theta)/2$
`C25`	constrained investment	`=SUMPRODUCT(Con_c,Con_Qheld)/2`	the quantity the budget limits

Notice what every formula in that table has in common. None of them names a cell in the item block. `Con_a`, `Con_Qheld` and the rest are named ranges spanning a hundred rows, so the summaries do not know, and do not need to know, how many items there are. Figure 4.4 shows the same block with its formulas displayed, the quickest way to see that the pattern holds everywhere and not only in the six cells quoted above.

One discipline that is not obvious. We make every summary either a `SUM` of one column or a `SUMPRODUCT` of several. It never divides one range by

	A	B	C
1	**Constrained Lot Sizing**		
2	*Chapter 4, sections 4.2 and 4.3. One resource is shar*		
3			
4	**Inputs**		
5	Carrying charge (i)	0.3	*per year*
6	Resource limit (A)	10000	*=IF(5B5?="Budget","dollars","cubic feet")*
7	Constraint type	Budget	*Budget or Space*
8			
9	**Does the limit bind?**		
10	Resource the free quantities would consun	=SUMPRODUCT(Con_a,Con_Qfree)	
11	Binds?	=IF(B10>B6,"yes","no")	
12			
13	**Solve for the multiplier**		
14	Multiplier (theta)	0.146429841	*Goal Seek changes this cell*
15	R(theta), equation 4.6, drive to zero	=IF(B11="no",0,SUMPRODUCT(Con_a,Con_Qheld)-B6)	
16	Status	=IF(B11="no","not needed, the limit does not bind",IF(ABS(B15)<0.000001*B6,"solved","STALE: re-run Goal Seek"))	
17	How	*Data > What-if Analysis > Goal Seek. Set cell B15 to value 0 by changing cell B14.*	
18	Why it is safe	*Section 4.2.3 proves R decreases strictly and has one root, which is what a numerical search needs and cannot verify for itself.*	
19			
20	**Results**		
21		**Unconstrained**	**Constrained**
22	Ordering cost	=SUMPRODUCT(Con_k,Con_NFree)	=SUMPRODUCT(Con_k,Con_NHeld)
23	Holding cost	=SUMPRODUCT(Con_h,Con_Qfree)/2	=SUMPRODUCT(Con_h,Con_Qheld)/2
24	Relevant cost	=B22+B23	=C22+C23
25	Average investment	=SUMPRODUCT(Con_c,Con_Qfree)/2	=SUMPRODUCT(Con_c,Con_Qheld)/2
26	Replenishments per year	=SUM(Con_NFree)	=SUM(Con_NHeld)
27	Resource used	=SUMPRODUCT(Con_a,Con_Qfree)	=SUMPRODUCT(Con_a,Con_Qheld)
28			
29	**What the limit cost**		
30	Extra cost per year	=C24-B24	
31	As a fraction of the free cost	=C24/B24-1	
32	Investment released	=B25-C25	
33			
34	**Check**		
35	**Unconstrained relevant cost**	$7,319.26, either constraint type	
36	**Budget, limit 10,000**	theta = 0.146429841, relevant cost $7,464.30	
37	**Budget arithmetic check**	holding cost = i times the limit = $3,000.00	
38	**Space, limit 800**	theta = 2.953823172, relevant cost $7,723.58	

Figure 4.4.: The same block with formulas displayed rather than values. Notice that no formula names a cell inside the item block. Every one of them refers to a named range, and that is what confines an added item to the item block.

another. Dividing would ask Excel to evaluate zero over zero on every empty row inside a hundred-row range, and the cell returns an error instead of a number. Thus, where a ratio per item is needed, and replenishments per year is the only such case here, the ratio gets its own column and the summary sums it. Thus, `C22` multiplies k_j by a column of frequencies instead of dividing $k_j\lambda_j$ by a column of quantities.

The search is a button, and you should know why that is safe. Rows 14 to 18 hold the multiplier, $R(\theta)$, and a status cell. Goal Seek drives the second to zero by changing the first.

Goal Seek is a numerical search that carries no guarantee of finding a root, or of finding the one you wanted. What makes it dependable here is not the software. Recall from Section 4.2.3 that $R(\theta)$ decreases strictly and therefore has exactly one root. A search cannot establish that for itself, and a reader who reaches for the button without the proof behind it is trusting a tool to be right about a function it has not examined.

One property of the design deserves the caution. Goal Seek does not re-run by itself, so changing a demand rate leaves a stale multiplier with every result below it confidently wrong. The status cell exists for that, and you should watch it and not the numbers.

The item data goes last. We come to the layout decision that matters most, and it is not about tidiness.

The decision in this model is a vector, $Q_1, \ldots, Q_n$, so the natural representation is a table whose rows are items. A table that rows are added to can only grow downward, and it can only grow downward if nothing sits beneath it. Put the item block in the middle, between the multiplier and the results, and every added item pushes two other blocks down and silently widens or breaks the ranges that point at it. Put it last and adding an item is typing in the first empty row.

Table 4.19 gives the columns.

Table 4.19.: The item columns. Every summary above the block is a `SUM` or a `SUMPRODUCT` over these, reached by name.

Column	Holds	Formula in row 43
A to E	name, λ_j, c_j, k_j, v_j	typed
F	$h_j = ic_j$	`=$B$5*C43`
G	a_j	`=IF($B$7="Budget",C43/2,E43)`
H	Q_j^*	`=SQRT(2*D43*B43/F43)`
I	$Q_j(\theta)$	`=SQRT(2*D43*B43/(F43+2*$B$14*G43))`
J, K	replenishments per year at each quantity	`=B43/H43`, `=B43/I43`
L	$Q_j(\theta)/Q_j^*$	`=I43/H43`

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
40	**Item data**													
41	*Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing: the named ranges already cover 100 rows.*													
42	**Item**	**Demand (lambda)**	**Unit cost (c)**	**Order cost (k)**	**Space per unit (v)**	**Holding (h = i*c)**	**Resource per unit (a)**	**Q free**	**Q held**	**Orders/yr free**	**Orders/yr held**	**Q held / Q free**		
43	Item 1	12,000.	20.	50.	0.8	$6.00	10.000	447.21	366.61	26.83	32.73	0.8198		
44	Item 2	25,000.	10.	50.	0.3	$3.00	5.000	912.87	748.33	27.39	33.41	0.8198		
45	Item 3	8,000.	15.	50.	1.2	$4.50	7.500	421.64	345.64	18.97	23.15	0.8198		

Figure 4.5.: The item block of the constrained worksheet. Columns A to E are typed and the rest are computed. Column I is equation 4.5 written out, and the reference to the multiplier in it is absolute while the references to the item's own data advance with the row.

Read column I of Figure 4.5 against Equation 4.5 and the correspondence is exact. `D43` and `B43` are k_j and λ_j, `F43` is h_j, `G43` is a_j, and `$B$14` is θ^*. Notice that only the last of those is written with dollar signs. Every formula in the column is the same formula copied down, and the dollar signs are what decide whether copying it changes it. Dollar signs are the spreadsheet's only mechanism for saying what belongs to an item and what belongs to the problem, and you should learn to read them as a statement about the model.

The multiplier belongs to no item. It appears in every row of column I and is not a property of any of them. It is one number describing the whole problem, and the layout says so by putting it in a single cell above the table.

You could instead give it a column of its own, repeated once per item. The sheet would still calculate correctly, and it would have stopped telling the truth about the model.

The coefficient is data rather than structure. Column G is the only thing that distinguishes a budget constraint from a space constraint. Recall that Equation 4.2 says both are $\sum_j a_j Q_j \le A$ with different coefficients. The worksheet holds a_j in a column and lets one cell choose what goes in it. The alternative, two worksheets differing in one formula, would have to be corrected twice every time anything else changed.

The evaluator and the optimizer, again. Rows 22 to 32 are the last two blocks of Section 3.10 in new clothing. There the sheet reported one quantity's cost at a typed Q and at Q^*, and the rows between them measured the penalty. Here the two columns are the portfolio unconstrained and constrained, and the same rows measure the price of the limit. The pattern survived because the question survived. What changed is only what is being compared.

4.7.2. The Exchange Curve

The constrained problem had one multiplier, fixed by the limit. The exchange curve of Section 4.4 has no limit at all, and sweeps the multiplier to trace what is reachable. That single change moves the multiplier from a cell to a column, and we take the why before the how.

What a sheet puts in a cell, and what it puts in a column. In Section 4.7.1 the multiplier described the problem, so it lived in one cell and every formula pointed at it absolutely. Here the multiplier is the thing being varied, so it runs down a column and every other quantity becomes a

column beside it. Thus, the layout is decided by a question about the model and not about convenience: is this quantity one fact about the problem, or is it the axis?

Get this backwards and the sheet still computes. For example, a multiplier repeated in a column in Section 4.7.1 would produce identical numbers while asserting, falsely, that each item has its own shadow price. A single multiplier cell here would produce one point and no curve.

	A	B	C	D	E
1	**Exchange Curve for a Group of Items**				
2	*Chapter 4, section 4.4. Aggregate investment and aggregate ordering cost cannot be chosen separately: their product is fixed by the item data.*				
3					
4	**Inputs**				
5	Order handling cost (kappa)	$55.00	*dollars per hour*		
6	Carrying charge (gamma)	25.0%	*per year*		
7	Operating point (theta)	0.25	*per year*	*set equal to gamma to price the firm's own position*	
8					
9	**Results**				
10	Aggregate demand rate	14,445			
11	Aggregate purchase cost rate	$555,500.00			
12	Demand-weighted unit cost (c-bar)	$38.46			
13	Purchase-weighted ordering cost (K-bar)	$123.67			
14	Variety index (J*)	5.6813			
15	Curve constant, equation 4.15	195,147,265.65			
16	The same constant from J*, K-bar, lambda-bar, c-bar	195,147,265.65			
17	Aggregate investment at theta	$27,939.02			
18	Aggregate ordering cost at theta	$6,984.76			
19	Their product	195,147,265.65			
20	Replenishments per year at theta	81.22			
21	Order handling hours per year	127.0			
22	Investment over ordering cost	4.000			
23					
24	**Check**				
25	**Sum of the SQRT column**	19,755.87			
26	**Curve constant**	195,147,266, by either route			
27	**Variety index J***	5.6813, so nine items behave like fewer than six			
28	**At theta = gamma = 0.25**	$27,939.02 of stock against $6,984.76 a year, or 127 hours			
29	**Investment over ordering cost**	exactly 4.000, because the ratio is 1/theta			

Figure 4.6.: The exchange curve worksheet, rows 1 to 29. Every figure in the results block is an aggregate over the item data, and the check block below records what the shipped items should produce.

Figure 4.6 shows the results. Notice that the constant of Equation 4.18 appears twice, at rows 15 and 16, computed two different ways. The first squares the sum of the item column, and the second multiplies the variety index by the three averages. They agree to the cent, and that agreement

is the arithmetic check for this sheet.

Table 4.20.: The six cells of the exchange curve worksheet that implement a result from Section 4.4. Rows 15 and 16 compute one quantity by two routes, which is a check.

Cell	Holds	Formula	Which result it is
B13	purchase-weighted ordering cost	=SUMPRODUCT(Exc_k,Exc_c,Exc_lambda)/B11	K^a, Equation 4.18
B14	variety index	=SUMPRODUCT(Exc_lambda)^2/SUMPRODUCT(Exc_k,Exc_c,Exc_la	L^a, Equation 4.18
B15	the curve constant	=SUMPRODUCT(Exc_lambda)^2/4	L^a, Equation 4.18
B16	the same constant, decomposed	=B14*B13*B10*B12/2	L^a, Equation 4.22
B17	aggregate investment	=SUMPRODUCT(Exc_$\bar{I}^a$,Exc_Q)/2	
B18	aggregate ordering cost	=SUMPRODUCT(Exc_k,Exc_$\overline{O}^a$)	

Figure 4.7 is the curve itself. Notice that it is not the output of a search. Equation 4.14 says the product of the two aggregates is a constant fixed by the item data, so each row evaluates a closed form and the shape is known before any row is filled in. Column D exists to show that: it is the same value, twelve times.

Notice also that we never divide one item column by another here. Each is a SUMPRODUCT of square roots, Equation 4.17 rearranged so the multiplier comes out as a scalar. The fifth column, replenishments per year, could not be written that way, since an item's order frequency divides by its own ordering cost. Thus, it takes the same route the constrained sheet took:

	A	B	C	D	E
32	**The curve**				
33	*One row per operating point. Investment times ordering cost is the same on every row, which is the content of equation 4.15.*				
34	**Multiplier (theta)**	**Aggregate investment**	**Aggregate ordering cost**	**Their product**	**Replenishments per year**
35	0.05	$62,473.56	$3,123.68	195,147,265.65	36.32
36	0.08	$49,389.68	$3,951.17	195,147,265.65	45.95
37	0.10	$44,175.48	$4,417.55	195,147,265.65	51.37
38	0.15	$36,069.12	$5,410.37	195,147,265.65	62.91
39	0.20	$31,236.78	$6,247.36	195,147,265.65	72.65
40	0.25	$27,939.02	$6,984.76	195,147,265.65	81.22
41	0.30	$25,504.72	$7,651.42	195,147,265.65	88.98
42	0.40	$22,087.74	$8,835.10	195,147,265.65	102.74
43	0.50	$19,755.87	$9,877.94	195,147,265.65	114.87
44	0.65	$17,327.05	$11,262.58	195,147,265.65	130.97
45	0.80	$15,618.39	$12,494.71	195,147,265.65	145.30
46	1.00	$13,969.51	$13,969.51	195,147,265.65	162.45

Figure 4.7.: The curve, one row per operating point. The multiplier runs down column A, the two aggregates are computed beside it, and their product is the same number on every row.

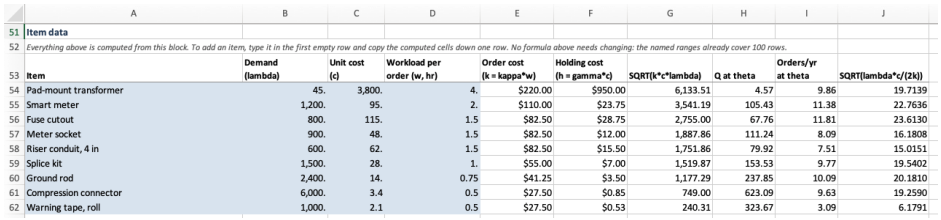
a per-item column doing the division, and a summary that multiplies a scalar by its sum.

Figure 4.8 carries nine items, against three on the constrained sheet, and nothing above it changed to accommodate them. The named ranges are doing that work.

4.7.3. Intervals and Rounding

Joint replenishment is the first model in this chapter whose answer is not a vector of quantities. Recall from Section 4.5.1 that it is a base period T, which is continuous, together with a multiplier m_j for each item, an integer. A worksheet has to carry both, and they behave differently: the base period is one cell an optimizer changes, and the multipliers are columns that recompute themselves every time that cell moves.

One continuous decision. The base period is the only continuous decision, because Equation 4.25 says the multipliers separate once T is



	A	B	C	D	E	F	G	H	I	J
51	**Item data**									
52	*Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing: the named ranges already cover 100 rows.*									
53	**Item**	**Demand (lambda)**	**Unit cost (c)**	**Workload per order (w, hr)**	**Order cost (k = kappa*w)**	**Holding cost (h = gamma*c)**	**SQRT(k*c*lambda)**	**Q at theta**	**Orders/yr at theta**	**SQRT(lambda*c/(2k))**
54	Pad-mount transformer	45.	3,800.	4.	$220.00	$950.00	6,133.51	4.57	9.86	19.7139
55	Smart meter	1,200.	95.	2.	$110.00	$23.75	3,541.19	105.43	11.38	22.7636
56	Fuse cutout	800.	115.	1.5	$82.50	$28.75	2,755.00	67.76	11.81	23.6130
57	Meter socket	900.	48.	1.5	$82.50	$12.00	1,887.86	111.24	8.09	16.1808
58	Riser conduit, 4 in	600.	62.	1.5	$82.50	$15.50	1,751.86	79.92	7.51	15.0151
59	Splice kit	1,500.	28.	1.	$55.00	$7.00	1,519.87	153.53	9.77	19.5402
60	Ground rod	2,400.	14.	0.75	$41.25	$3.50	1,177.29	237.85	10.09	20.1810
61	Compression connector	6,000.	3.4	0.5	$27.50	$0.85	749.00	623.09	9.63	19.2590
62	Warning tape, roll	1,000.	2.1	0.5	$27.50	$0.53	240.31	323.67	3.09	6.1791

Figure 4.8.: The exchange curve item block. Columns A to D are typed and the rest are computed. Column G is the only aggregation Equation 4.18 needs, and column J exists so that the replenishment column of the curve has something to sum.

fixed. That is, the whole problem is one-dimensional, so Solver has exactly one cell to change under each rule.

Notice in Figure 4.9 that the two rules have separate base period cells. They reach different optima, 51.882 days against 51.800, so one cell could not serve both.

Table 4.21.: The six cells of the joint replenishment worksheet that implement a result from Section 4.5. Notice that `C17` adds the major setup once and sums a per-item column, rather than dividing anything by a column of multipliers.

Cell	Holds	Formula	Which result it is
`B10`	independent ordering	`=SUMPRODUCT(SQRT(2*($B$5+Jnt_k)*Jnt_h*Jnt_lambda))`	each item paying $K + k_j$ alone
`B11`	common cycle T^*	`=SQRT(($B$5+SUM(Jnt_k))/SUM(Jnt_g))`	Equation 4.23
`C17`	cost at the integer-ratio base period	`=$B$5/(B17/$B$7)+SUM(Jnt_cost_int)`	Equation 4.25

Cell	Holds	Formula	Which result it is
`H56`	an item's integer multiplier	`=MAX(1,CEILING((-1+SQRT(1+4*(G56/($B$17/$B$7))^2))/2,1))`	the integer solution of Equation 4.25
`J56`	an item's power-of-two multiplier	`=2^MAX(0,CEILING(LOG(G56/($B$17/SQRT(2))*($B$18/$B$7)),2),1)`	Equation 4.26
`D18`	what nesting costs	`=C18/C17-1`	the bound of Equation 4.30, measured

The formula in `H56` looks worse than it is. The integer minimizing Equation 4.23 for one item is the m with $m(m-1) \le (T_j^*/T)^2 \le m(m+1)$, and solving that pair of inequalities for m gives the expression in the cell. Thus, no cell has to evaluate the cost at two candidate multipliers and keep the smaller.

Notice in Figure 4.10 that the curve is not how the answer is found. Solver finds it, by changing one cell. The curve is there because the objective is *piecewise* smooth: it kinks wherever a multiplier jumps, so a hill climber can halt at a kink instead of at the minimum. Thus, plotting the objective before optimizing it is not decoration here, it is the only evidence available that Solver stopped in the right basin.

We carry one contrast away from these two sections. In Section 4.7.1 the search is a button and nothing is plotted, because Section 4.2.3 proves the function has one root. Here the search is also a button, and the plot is mandatory, because no such proof exists. What decides whether you may trust an optimizer is a property of the function, and it is established in the chapter, not in the sheet.

Notice in Figure 4.11 that the drive unit and the gearbox both sit at a multiplier of one under both rules. Their preferred intervals, 13 and 37

	A	B	C	D	E	F	G	H	I
1	**Joint Replenishment: a base period and integer multipliers**								
2	*Chapter 4, section 4.5. The multipliers separate once the base period is fixed, so the only decision Solver has to make is the base period itself.*								
3									
4	**Inputs**								
5	Major setup cost (K)	$400.00	*dollars per order*						
6	Carrying charge (i)	25.0%	*per year*						
7	Days per year	365	*days*						
8									
9	**Two baselines, before any search**								
10	Independent orderin	$8,422.39							
11	Common cycle T (yea	0.146209							
12	Common cycle T (day	53.4							
13	Common cycle cost	$6,497.54							
14									
15	**Solve for the base period**								
16	**Rule**	**Base period (days)**	**Total cost**	**Excess over integer ratio**					
17	**Integer ratio**	51.882	$6,402.06						
18	**Powers of two**	51.800	$6,403.34	0.020%					
19	How	*Data > Solver. Set objective C17 to Min by changing B17, GRG Nonlinear, B17 >= 0.001. Then the same for C18 by changing B18.*							
20	Read the curve first	*The cost is piecewise smooth: it kinks wherever a multiplier jumps, so Solver can stop at a kink. The block below plots it, which is how yo*							
21									
22	**Check**								
23	**Independent orderir**	$8,422.39							
24	**Common cycle**	0.146209 years, 53.4 days, $6,497.54							
25	**Integer ratio**	51.882 days, $6,402.06, multipliers 1, 1, 2, 6							
26	**Powers of two**	51.800 days, $6,403.34, multipliers 1, 1, 2, 8, excess 0.020%							

Figure 4.9.: The joint replenishment worksheet, rows 1 to 26. Inputs, then the two baselines coordination is measured against, then the two rules with the base period Solver changes and what the power-of-two restriction costs against the best integer ratio.

	A	B	C	D	E	F	G	H	I	J	K
28	**The cost against the base period**										
29	*One row per candidate base period, so the shape can be seen before Solver is trusted. The multiplier columns are what kink the cost: watch one of them step up and the cost turn a corner in the same ro*										
30	**T (days)**	**m1, integer**	**m2, integer**	**m3, integer**	**m4, integer**	**Cost, integer ratio**	**m1, pow 2**	**m2, pow 2**	**m3, pow 2**	**m4, pow 2**	**Cost, powers of two**
31	26.7	1	1	4	12	$7,837.08	1	1	4	16	$7,838.66
32	29.4	1	1	3	11	$7,441.77	1	1	4	8	$7,443.87
33	32.0	1	1	3	10	$7,148.83	1	1	4	8	$7,152.87
34	34.7	1	1	3	9	$6,916.62	1	1	4	8	$6,922.78
35	37.4	1	1	3	8	$6,742.13	1	1	2	8	$6,746.04
36	40.0	1	1	3	8	$6,617.61	1	1	2	8	$6,618.50
37	42.7	1	1	2	7	$6,523.16	1	1	2	8	$6,523.21
38	45.4	1	1	2	7	$6,458.77	1	1	2	8	$6,459.11
39	48.0	1	1	2	7	$6,421.34	1	1	2	8	$6,421.94
40	50.7	1	1	2	6	$6,403.76	1	1	2	8	$6,404.82
41	53.4	1	1	2	6	$6,404.73	1	1	2	8	$6,406.31
42	56.0	1	1	2	6	$6,420.75	1	1	2	4	$6,422.76
43	58.7	1	1	2	5	$6,450.82	1	1	2	4	$6,452.27
44	61.4	1	1	2	5	$6,492.71	1	1	2	4	$6,493.80
45	64.0	1	1	2	5	$6,542.98	1	1	2	4	$6,543.75
46	66.7	1	1	2	5	$6,604.26	1	1	2	4	$6,604.71
47	69.4	1	1	2	5	$6,673.72	1	1	2	4	$6,673.87
48	72.0	1	1	1	4	$6,745.91	1	1	1	4	$6,745.91
49	74.7	1	1	1	4	$6,825.75	1	1	1	4	$6,825.75
50	77.4	1	1	1	4	$6,911.52	1	1	1	4	$6,911.52
51	80.0	1	1	1	4	$6,999.15	1	1	1	4	$6,999.15

Figure 4.10.: The cost against the base period, one row per candidate. The multiplier columns are the point: watch the last one step down from 6 to 5 and the cost column turn a corner in the same row.

	A	B	C	D	E	F	G	H	I	J	K
53	**Item data**										
54	*Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing: the named ranges already co*										
55	**Item**	**Demand (lambda)**	**Unit cost (c)**	**Minor setup (k)**	**Holding rate (h = i*c)**	**g = h*lambda/2**	**Preferred interval (yr)**	**m, integer ratio**	**Cost at that m**	**m, powers of two**	**Cost at that m**
56	Drive unit	8,000.	20.	25.	$5.00	20,000.0	0.035355	1	$3,018.73	1	$3,014.51
57	Gearbox	800.	20.	20.	$5.00	2,000.0	0.100000	1	$424.99	1	$424.76
58	Seal kit	200.	8.	15.	$2.00	200.0	0.273861	2	$109.62	2	$109.61
59	Name plate	40.	4.	15.	$1.00	20.0	0.866025	6	$34.65	8	$35.92

Figure 4.11.: The joint replenishment item block. The preferred interval in column G is what each item would choose paying only its minor setup, and the two multiplier columns move it to the nearest multiple of the base period under each rule.

days, are shorter than the base period, so nothing about them is rounded and what sets their cost is the choice of base period alone. It is the first qualification of Section 4.5.3, visible in two cells.

Where the item block cannot carry the work. Every summary above the item block on this sheet reaches it through a named range, so it does not care how many items there are. The curve cannot. Each row needs the multipliers *at that row's base period*, and a multiplier is a rounding, which no arithmetic over a whole column will produce.

Two ways out present themselves, and we state the choice between them because the rest of this section keeps making it. One is to compute the rounding over the whole column inside a single formula, compact and independent of the item count. The other is to give each item its own column and compute the rounding one item at a time, longer but showing the reader something the first hides.

The second is what the sheet does, and the multiplier columns of Figure 4.9 are the reason. Watch the last one step down from 6 to 5 as the base period lengthens, and the cost column turns a corner in the same row. The kink is made visible there, and it justifies plotting the curve at all. A formula that hid the multipliers would have hidden the very thing the block exists to show.

The cost is a column per item, so a fifth item means a column in each of the two groups. The distribution scan in Section 4.7.4 makes the same admission, and we state it plainly: the item block grows by a row, and a calculation that works item by item grows with it.

4.7.4. The Two Echelon Structures

The echelon models of Section 4.6 differ from everything before them in that we must fix the accounting before any optimization is possible. Recall from Equation 4.31 that an echelon rate is a difference between neighbors.

That is, no location can compute its own, and the sheet has to know the structure before it can charge anybody.

The distribution algorithm suits a worksheet and the serial one does not. The scan of Section 4.6.3 has a fixed number of steps known in advance, one per region, and each step depends only on the one before it. That is a table.

	A	B	C	D	E	F
1	**Distribution System: the pinned set**					
2	*Chapter 4, section 4.6.3. Sorting the regions turns the construction of C0 into running totals. No Solver, no iteration, no macros.*					
3						
4	**Inputs**					
5	Base period	1	*weeks*			
6	Weeks per year	52	*weeks per year*			
7	Base period in years (T)	0.019231	*years*			
8						
9	**Results**					
10	First pinned rank (i-bar)	3	*ranks at or above this share the central interval*			
11	Central warehouse interval (years)	0.343049				
12	Central warehouse interval (weeks)	17.84				
13	Rounded to a power of two (weeks)	16.0				
14	Pinned set C0	the central warehouse plus the regions at ranks 3 through 6				
15						
16	**Check**					
17	**Ranks, shortest interval first**	nodes 5, 3, 6, 4, 1, 2				
18	**First pinned rank (i-bar)**	3				
19	**Pinned set C0**	the central warehouse plus nodes 6, 4, 1 and 2				
20	**Left alone**	nodes 5 and 3, at 14.24 and 17.25 weeks				
21	**Central interval**	0.343049 years, or 17.84 weeks				
22	**Every power-of-two interval**	16 weeks				

Figure 4.12.: The distribution worksheet, rows 1 to 22. The results come first and the check block records what the shipped locations should produce. Nothing here is iterative.

Table 4.22.: The six cells of the distribution worksheet that implement a result from Section 4.6.3. Notice that the central warehouse is found with `INDEX(...,1)` rather than by a row number, so reordering the regions does not break the sheet.

Cell	Holds	Formula	Which result it is
`E42`	echelon rate h_i'	`=IF(A42=INDEX(Dis_node,1),D42,D42-INDEX(Dis_h,1))`	Equation 4.4
`G42`	the ratio k_i/g_i	`=C42/F42`	what the regions are sorted on
`F32`	pooled k of the set so far	`=INDEX(Dis_k,1)+SUMIF($A$32:$A$37,">"&A32,$C$32:$C$37`	total of the scan
`I32`	the verdict at this step	`=IF(E32>H32,"add","stop")`	the test of Section 4.6.3
`B10`	first pinned rank	`=MAX(J32:J37)+1`	where the scan stopped
`B13`	the rounded central interval	`=2^MAX(0,CEILING(LOG(B14/(SQRT(2)*$B$7),2),1))*$B$7*`	Equation 4.28

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
31	Rank	Node	k	g	k / g	Pooled k (C0 so far)	Pooled g (C0 so far)	Pooled k/g	Test	Stop index	Interval T (years)	T / (sqrt2*base)	I*	Power-of-two T (years)	T (weeks)
32	1	5	$75.00	1,000.0	0.075000	$1,075.00	9,200.0	0.116848	stop	1	0.273861	10.07	4	0.307692	16.0
33	2	3	$110.00	1,000.0	0.110000	$965.00	8,200.0	0.117683	stop	2	0.331662	12.20	4	0.307692	16.0
34	3	6	$90.00	700.0	0.128571	$875.00	7,500.0	0.116667	add	0	0.343049	12.61	4	0.307692	16.0
35	4	4	$150.00	800.0	0.187500	$725.00	6,700.0	0.108209	add	0	0.343049	12.61	4	0.307692	16.0
36	5	1	$100.00	500.0	0.200000	$625.00	6,200.0	0.100806	add	0	0.343049	12.61	4	0.307692	16.0
37	6	2	$125.00	200.0	0.625000	$500.00	6,000.0	0.083333	add	0	0.343049	12.61	4	0.307692	16.0

Figure 4.13.: The scan, one row per region. The candidate's own ratio sits beside the pooled ratio of everything already pinned, and the verdict is the comparison between them.

Figure 4.13 reproduces Table 4.16 row for row, and that is the point of the block. The pooled columns are running totals, so the sheet needs no iteration, no Solver and no macro. The serial partition has none of those

properties: the number of blocks is not known until the walk is finished, and a merge changes blocks already built. There is therefore no serial worksheet in the accompanying workbook, and the absence is an answer, not an omission.

One row per region is also one row to add. Every summary above the block reaches it by name and does not care how many locations there are, but the scan cannot be written that way, because each step needs the pooled totals of the step before it. Thus a seventh region means a row in the location block and a row in the scan, and the sheet says so instead of hiding it.

	A	B	C	D	E	F	G	H
39	**Item data**							
40	*Everything above is computed from this block. To add an item, type it in the first empty row and copy the co*							
41	**Node**	**Demand (lambda)**	**order (k)**	**Installation rate (h)**	**Echelon rate (h')**	**g = lambda*h'/2**	**k / g**	**Rank**
42	0 centr	1,200.	500.	10.	$10.00	6,000.0	0.083333	
43	1	200.	100.	15.	$5.00	500.0	0.200000	5
44	2	200.	125.	12.	$2.00	200.0	0.625000	6
45	3	200.	110.	20.	$10.00	1,000.0	0.110000	2
46	4	200.	150.	18.	$8.00	800.0	0.187500	4
47	5	200.	75.	20.	$10.00	1,000.0	0.075000	1
48	6	200.	90.	17.	$7.00	700.0	0.128571	3

Figure 4.14.: The distribution item block. The central warehouse is the first row, and every region is charged only the value it adds, which is column E.

Notice in Figure 4.14 that the rank column is blank on the central warehouse row. Ranking is over the regions alone, because the scan works down from the region wanting the longest interval and the center is what they are being compared against, not a candidate itself.

4.7.5. What a Worksheet Can and Cannot Do

Three questions were answered four times each in the preceding sections. Having seen every instance, we can state them.

Is this a property of an item, or of the problem? The multiplier is one cell. The volume v_j is a column. Getting this wrong costs nothing arithmetically, and that is what makes it dangerous: a multiplier repeated down a column computes the same numbers while asserting that each item has its own shadow price. Thus, the test is not whether the numbers come out, but whether the layout still describes the model.

Is this difference data, or a different sheet? A budget limit and a space limit differ in one column, so Equation 4.2 is implemented once and column G holds a_j. Two sheets differing in one formula would have to be corrected twice forever after. Notice that the same question has a harder answer for the coordination rules of Section 4.5.1, where the alternatives differ in what has to be done and not in a number, so the joint sheet carries two base period cells and two sets of multiplier columns instead of one of each with a switch.

When may a button be trusted? Twice in these sheets a search is a button, and the two cases are not alike. Goal Seek is safe in Section 4.7.1 because Section 4.2.3 proves $R(\theta)$ strictly decreasing with exactly one root. For example, Solver is not safe in Section 4.7.3, because the objective kinks wherever a multiplier steps, so that sheet plots the curve and the other does not. What decides whether an optimizer may be trusted is a property of the function, and it is established in the chapter, not on the sheet.

Where a worksheet stops. Three limits appeared, each in its place. It cannot bracket its own root or widen its own search, so it asks the reader to have established in advance what a program arranges in a loop. It cannot hold a structure whose size is not known until the computation finishes, so Algorithm 4.1 has no sheet. And it cannot be audited at three thousand items: every argument in this section about what the layout asserts depends on a person being able to see the layout.

None of that makes the worksheets inferior. They are better at the thing they are for, making a method visible. Figure 4.10 shows what a one-dimensional search over a base period is, and no loop shows that. What a worksheet cannot do is what Section 4.8 is for.

4.8. Designing the Software

In Section 4.7 we built each model as a worksheet. The layout of a worksheet is visible. A formula is a cell you can click on, a summary is a range you can highlight, and putting the multiplier in a single cell says that it belongs to the problem and not to any item.

A program shows none of that. Its structure is a set of decisions about which classes exist and what each class is responsible for, and those decisions leave no trace in the finished code. We therefore write them down first.

The analysis proceeds in the order such an analysis always does: what the software must do, what the domain calls things, what each thing knows and does, how the things are related, and how they collaborate to produce an answer. Code appears at the end of each step, by which point little is left to decide.

The effort pays off because the five models share more than they appear to. A programmer who writes the constrained problem, then the exchange curve, then joint replenishment, without pausing between them, will write Equation 4.5 three times. Thus, when the third one is wrong there are three places to correct.

4.8.1. What the Software Must Do

We begin with a list of requirements. Each entry names something a user would ask the software for, together with the section of this chapter that supplies the answer.

Table 4.23.: What the software has to do, and where in this chapter each requirement is worked out. Read the right-hand column as the specification: a requirement whose mathematics is not in the chapter is one nobody can check.

#	The software must	Established in
R1	compute each SKU's order quantity ignoring any coupling	Chapter 3
R2	report an aggregate measure of a set of decisions	Section 4.1
R3	decide whether a stated limit on a measure binds	Section 4.2.1
R4	find the cheapest decisions meeting a limit, and report its shadow price	Section 4.2.2
R5	price a limit against having none	Section 4.3
R6	trace the frontier of achievable aggregate positions	Section 4.4
R7	schedule SKUs sharing one setup, under a stated ordering practice	Section 4.5
R8	restrict intervals to what an operation can run, and price the restriction	Section 4.5.3

#	The software must	Established in
R9	account for holding cost across locations without double counting	Section 4.6.1
R10	find reorder intervals across a chain or a distribution network	Section 4.6.2

Two features of Table 4.23 matter more than the list itself.

Notice that R3, R4 and R5 do not mention a budget. They name a limit on a *measure*. Recall from Section 4.1 that an organization runs short of four different things, and that Section 4.2 then solved for one of them. A design that treats the budget as the problem, and the other three as variations on it, carries that assumption into every signature it writes.

A requirement is also not a class. R7 says to schedule SKUs sharing one setup. That sentence names two candidate things, a group of SKUs and a schedule, and it does not say whether either one is an object. For example, turning requirements straight into classes is how software acquires a `ScheduleManager`.

A requirement says what the software does. A scenario says who asks, and in what order.

Table 4.24.: The five scenarios the software is built to serve, each phrased as somebody's question. A scenario that no requirement supports, or a requirement no scenario ever uses, is a defect in one of the two lists.

#	Scenario	The question behind it
S1	solve under a limit	"Management will fund $10,000 of stock. What do I order?"
S2	price the limit	"What is that ceiling costing me?"
S3	trace the frontier	"If I want fewer orders, how much more stock is that?"
S4	coordinate a supplier's line	"These four come on one truck. How often do I order each?"
S5	plan a network	"The depot feeds six branches. What interval does each run?"

S1 and S3 of Table 4.24 repay attention. "What do I order under a ten thousand dollar limit" and "if I want fewer orders, how much more stock is that" sound like two programs. Section 4.4 showed that they are one piece of mathematics asked from two ends. If the analysis is sound, the two will use the same objects and nearly the same messages, and we check that in Section 4.8.8.

4.8.2. Finding the Nouns

The candidate classes are the nouns of the domain, and the test for keeping one is whether somebody who runs a stockroom would use the word. That test is not a matter of taste. A name taken from the domain survives changes in the mathematics, because the domain is what the mathematics describes. A name invented to suit an implementation has to be renamed every time the implementation moves.

Table 4.25.: The vocabulary the design is built from. Every term is one that appears in this chapter and would be understood in a warehouse, which is the only test a candidate class has to pass at this stage.

Term	What it means
SKU	a stock keeping unit: one item held at one location, and the thing that gets replenished
Portfolio	the SKUs that are decided together
Aggregate measure	a number computed over a whole plan: investment, space occupied, replenishment workload, relevant cost
Constraint	an aggregate measure together with the ceiling management has put on it
Replenishment plan	how much and how often, for every SKU in a portfolio
Order family	SKUs sharing a supplier, a truck or a machine setup, so that one order serves several
Interval rule	what intervals the operation can actually run

Term	What it means
Supply network	SKUs that replenish one another, whether in a chain or from a center
Echelon holding rate	the value a location adds, as against the value accumulated through it

Four other nouns suggest themselves. None of them survives the test.

The **shadow price** is a number that a solution reports. A class for it would know nothing beyond its own value and would do nothing, so it becomes a property of a replenishment plan. Recall from Section 4.3 that the price falls to zero exactly where the limit stops binding. Thus, the mathematics has no missing case and the property need not be nullable.

Echelon stock is defined in Section 4.6.1, and what it defines is a measure of a network. It belongs with investment and space, not with SKUs and plans.

Item and **stage** are one noun. A SKU already is an item at a location. Section 4.2 holds many SKUs at one location and Section 4.6 holds one item across many locations. That is, one entity under two topologies. Two classes would duplicate every field and force a conversion between the sections.

Shared resource is the instructive rejection, because Section 4.1 uses the phrase. A budget, a floor and a receiving dock are all things that several SKUs compete for. As a class name it fails the test. Nobody in a warehouse has a shared resource. They have a working capital account, a building with pallet positions, and a purchasing department that processes so many orders a week. The word that does pass is *measure*, since each of those is a number we can compute about a plan and put a ceiling on.

Figure 4.15 illustrates the concepts and how they relate. Read it as sentences. A portfolio is one or more SKUs. A constraint limits a portfolio, an order family coordinates one, and a supply network sequences one. The

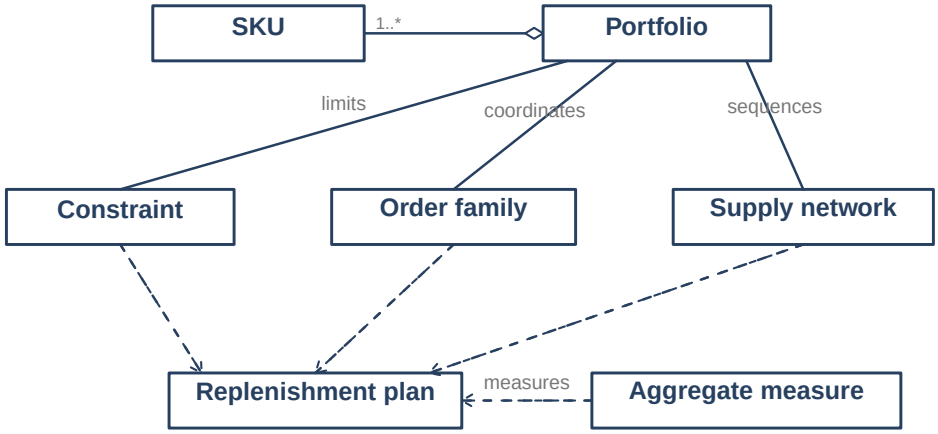


Figure 4.15.: The concepts of this chapter and how they relate, before any thought about implementation. A portfolio is one or more SKUs. A constraint limits a portfolio, an order family coordinates one, and a supply network sequences one. The dashed arrows show that all three produce a replenishment plan, and an aggregate measure measures that plan.

dashed arrows say that all three produce a replenishment plan, and an aggregate measure measures that plan.

Figure 4.15 carries no operations, no types and no direction of dependency. It is a conceptual model and not yet a design. Every argument in the rest of this section asks whether the code still says what this picture says.

4.8.3. What Each Thing Knows and Does

We describe each candidate class before drawing it. The traditional form is a class-responsibility-collaborator card, one index card per candidate, with three questions on it. What does it know? Who does it know? What does it do?

The size of the card does the work. A responsibility that will not fit on one card is usually two responsibilities, and a card with no collaborators is either a value or a mistake.

Table 4.26.: The responsibilities, one row per candidate class. The middle column is the more important one: a class with no collaborators does nothing for anybody, and a class that knows about most of the others has probably absorbed a responsibility that belongs elsewhere.

Class	Knows	Knows about	Does
`SKU`	label, demand rate, unit value, order cost, holding rate	nobody	its economic order quantity; its cost at a quantity; the interval it would prefer

Class	Knows	Knows about	Does
`Portfolio`	its SKUs, in a fixed order	`SKU`	hands out its SKUs; builds a plan from one quantity each
`AggregateMeasure`	its name and unit	`ReplenishmentPlan`	measure a plan
`ConstrainableMeasure`	whatever its kind knows	`SKU`	its marginal at a SKU; the quantity a SKU takes at a price
`Constraint`	a measure, a ceiling	`AggregateMeasure`	whether a plan meets it; the slack
`ReplenishmentPlan`	a quantity per SKU, and the price that produced it	`SKU`, `AggregateMeasure`	report any measure of itself; compare itself with another plan
`ConstrainedLotSizing`	a portfolio, a constraint, a method	all three	the cheapest plan meeting the limit; whether it binds
`LotSizingMethod`	whatever its kind needs	`Portfolio`, `Constraint`	produce that plan
`ExchangeCurve`	a portfolio	`Portfolio`, `AggregateMeasure`	the plan at a price; the curve over prices; the variety index

Class	Knows	Knows about	Does
`OrderFamily`	a portfolio, the major setup	`Portfolio`, `IntervalRule`	a plan under a rule; what independent ordering would cost
`IntervalRule`	its base period	nobody	turn a preferred interval into one that can be run
`SupplyNetwork`	its SKUs, and who supplies whom	`SKU`, `IntervalRule`	the echelon rate of a location; the plan

Three rows of Table 4.26 repay a second look.

`SKU` knows nobody. A SKU holding a reference to its portfolio could not be evaluated inside a sweep without the portfolio coming with it, and the exchange curve of Section 4.4 evaluates the same SKUs at a thousand prices. Notice also what a SKU does not know: its volume per unit. A SKU carrying a volume because a space limit exists would have to carry a weight when a weight limit arrives, so the class would grow once per constraint anybody writes.

`IntervalRule` knows nobody either. Given the interval a location would prefer, it returns one the operation can run, and it never asks which location. Thus, the same rule applies per SKU in an order family, per block in a serial chain, and per location in a distribution network.

The measure knows the plan and the plan knows the measure. Two rows pointing at each other should make us nervous. That is, they are one message seen from two sides. The measure owns the formula and the plan owns the data, so the measure is the receiver and `plan.measuredBy(m)` delegates to `m.measure(plan)`. If both carried real implementations we

would be starting on double dispatch, where adding a measure means editing every plan.

Reading the cards a second time, and asking who sends each message, finishes the analysis. One answer stands out. `marginalAt` is called by one kind of object, a lot sizing method, and by nothing that reports or displays. Thus, it belongs to the solving collaboration and not to the measure's public face. We act on that in Section 4.8.5.

4.8.4. The Classes

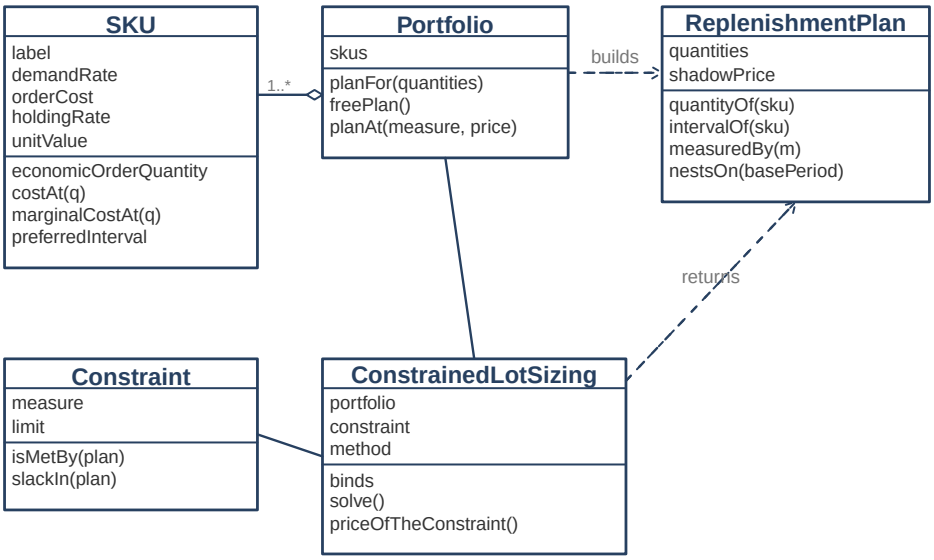


Figure 4.16.: The core classes and what they hold. A portfolio aggregates one or more SKUs and builds replenishment plans from them, and a constrained lot sizing study holds a portfolio, a constraint and a method. Notice that every arrow leaving the study points at something the study was handed, so nothing here constructs its own collaborators.

Figure 4.16 illustrates the classes that carry an answer, and we take them in turn.

The SKU holds its own data and computes its own mathematics.

```
class SKU(
    val label: String,
    val demandRate: Double,
    val orderCost: Double,
    val holdingRate: Double,
    val unitValue: Double = 0.0,
) {
    /** @eq-multiitem-objective, this SKU alone, ignoring
     ↪ anything it shares. */
    val economicOrderQuantity: Double
        get() {
            require(holdingRate > 0.0) {
                "$label has no holding cost, so it has no
                 ↪ unconstrained order quantity"
            }
            return sqrt(2.0 * orderCost * demandRate /
             ↪ holdingRate)
        }

    /** Ordering plus holding, per unit time, at a quantity
     ↪ someone else chose. */
    fun costAt(orderQuantity: Double): Double {
        require(orderQuantity > 0.0) { "An order quantity
↪ must be positive" }
        return orderCost * demandRate / orderQuantity +
         ↪ holdingRate * orderQuantity / 2.0
    }

    /** The derivative of costAt, which is what a
     ↪ stationarity condition needs. */
```

```
    fun marginalCostAt(orderQuantity: Double): Double =
        -orderCost * demandRate / (orderQuantity *
         ↪  orderQuantity) + holdingRate / 2.0
}
```

Notice that `costAt` takes a quantity instead of computing one. Every model in this chapter evaluates a SKU at a quantity chosen elsewhere, so a class that knew only its own optimum would be useless in four of the five. For example, the exchange curve evaluates every SKU at the quantity a price implies.

`marginalCostAt` looks like an implementation detail of `economicOrderQuantity`. It is in fact the piece every constrained model needs, because a stationarity condition is where a derivative is used.

One answer type serves all five models. The constrained problem and the exchange curve solve for quantities, and joint replenishment and both echelon structures solve for intervals. Recall that $Q_j = \lambda_j T_j$, so an object holding one can report the other.

```
class ReplenishmentPlan(
    val skus: List<SKU>,
    private val quantities: List<Double>,
    val shadowPrice: Double = 0.0,
) {
    fun quantityOf(sku: SKU): Double =
     ↪  quantities[indexOf(sku)]
    fun intervalOf(sku: SKU): Double =
     ↪  sku.intervalFor(quantityOf(sku))

    /** Delegates to the measure, which owns the formula
     ↪  while this owns the data. */
    fun measuredBy(measure: AggregateMeasure): Double =
     ↪  measure.measure(this)
```

```
}
```

This is not unification for its own sake. The chapter claims that every model in it answers how much and how often. If that claim is true, then one answer type is the right number, and a sixth model whose answer did not fit here would be evidence against the claim.

A constraint is a measure together with a ceiling. “At most ten thousand dollars of average investment” is one sentence in the domain, so we make it one object. Keeping the measure separate from the ceiling lets the same measure serve the constrained problem, the exchange curve, and a management report that constrains nothing at all.

```
class Constraint(val measure: ConstrainableMeasure, val
 ↪  limit: Double) {
    fun isMetBy(plan: ReplenishmentPlan): Boolean =
     ↪  measure.measure(plan) <= limit

    /** Positive when there is room left, negative when the
     ↪  plan overshoots. */
    fun slackIn(plan: ReplenishmentPlan): Double = limit -
     ↪  measure.measure(plan)
}
```

Recall that the worksheet of Section 4.7.1 put the bind test in `B11` and $R(\theta)$ in `B15`. Those two cells are `isMetBy` and `slackIn`. Both implementations put them on the constraint, because the comparison needs the measure and the limit together and needs nothing else.

4.8.5. Measuring, and Limiting a Measure

This is where the analysis pays for itself, so we go slowly.

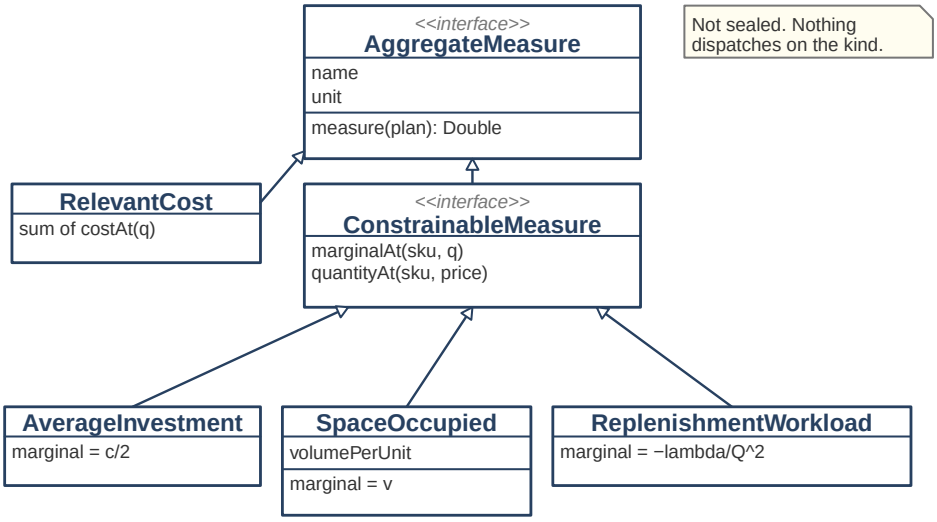


Figure 4.17.: The measures. What can be measured is one interface, and what a limit can be put on is a second interface extending it. Relevant cost implements only the first, since it is the objective and is never constrained. Notice that SpaceOccupied carries a volume per unit while the other two kinds carry nothing of their own.

Figure 4.17 illustrates two interfaces and four kinds. We take the split first.

Relevant cost is measured constantly and constrained never. Nobody puts a ceiling on the objective, because the objective is what everything else is traded against. Putting `marginalAt` on one interface with `measure` would force `RelevantCost` to implement a method that has no meaning for it, and a method implemented only to satisfy a compiler is a method somebody will eventually call. Thus, `AggregateMeasure` is what can be measured, and `ConstrainableMeasure` adds what a limit needs.

The marginal is the primitive and the quantity is derived from it. Recall from Section 4.2.2 that the stationarity condition for one SKU sets the derivative of the Lagrangian to zero.

- Let M represent any aggregate measure computed over a plan, so that $\partial M / \partial Q_j$ is how much the measure changes per unit of order quantity at SKU j.

With this notation the condition is

$$-\frac{k_j \lambda_j}{Q_j^2} + \frac{h_j}{2} + \theta \, \frac{\partial M}{\partial Q_j} = 0$$

which is Equation 4.5 before the algebra. Notice what the condition needs. It needs the SKU's own cost derivative, supplied by `SKU.marginalCostAt`, and the measure's marginal. Nothing else. Thus, any measure that can report a marginal can be solved numerically, and we write that once as a default implementation.

```
interface ConstrainableMeasure : AggregateMeasure {

    /** How much this measure changes per unit of order
     ↪  quantity at one SKU. */
    fun marginalAt(sku: SKU, orderQuantity: Double): Double
```

```
    fun quantityAt(sku: SKU, shadowPrice: Double): Double {
        val f = { q: Double -> sku.marginalCostAt(q) +
         ↪  shadowPrice * marginalAt(sku, q) }
        // bisect f to zero, widening the bracket until it
         ↪  changes sign
    }
}
```

A measure whose stationarity condition has a closed form overrides `quantityAt` and skips the search. Both of the measures that the chapter solves by hand do so.

```
object AverageInvestment : ConstrainableMeasure {
    override fun measure(plan: ReplenishmentPlan): Double =
        plan.skus.sumOf { it.unitValue * plan.quantityOf(it)
↪   / 2.0 }
    override fun marginalAt(sku: SKU, orderQuantity: Double):
     ↪  Double = sku.unitValue / 2.0
    override fun quantityAt(sku: SKU, shadowPrice: Double):
     ↪  Double =
        sqrt(2.0 * sku.orderCost * sku.demandRate /
↪   (sku.holdingRate + shadowPrice * sku.unitValue))
}
```

That is Equation 4.5 with $a_j = c_j/2$, and it is the only place in the program where the formula appears. The worksheet wrote it in column I of one sheet and again in the curve block of another, because a worksheet has no way to say the same thing once.

The third kind is the one that no coefficient can describe. A simpler interface suggests itself. Equation 4.2 writes every limit as $\sum_j a_j Q_j \le A$,

so a measure could report the single coefficient a_j and leave the rest to the solver. That interface serves a budget, where $a_j = c_j/2$, and it serves a floor, where $a_j = v_j$. Recall from Section 4.2.4 that a limit on replenishments is $\sum_j \lambda_j/Q_j \le A$. Consumption falls as the quantities rise, so the marginal is negative and depends on the quantity, and the multiplier joins the ordering cost.

```
object ReplenishmentWorkload : ConstrainableMeasure {
    override fun measure(plan: ReplenishmentPlan): Double =
        plan.skus.sumOf {
 ↪  it.orderFrequencyAt(plan.quantityOf(it)) }
    override fun marginalAt(sku: SKU, orderQuantity: Double):
     ↪  Double =
        -sku.demandRate / (orderQuantity * orderQuantity)
    override fun quantityAt(sku: SKU, shadowPrice: Double):
     ↪  Double =
        sqrt(2.0 * (sku.orderCost + shadowPrice) *
 ↪  sku.demandRate / sku.holdingRate)
}
```

No coefficient expresses that. An interface reporting only a coefficient would force every caller to ask which kind of resource it was holding, and that is the thing an interface exists to prevent. Thus, an abstraction drawn from two examples fits the third by luck or not at all, and the mathematics is what tells you which.

Why a hierarchy, and not one class holding two functions? In Kotlin we could write a single `AggregateMeasure` class with `measure` and `marginalAt` as function-valued properties, and the four kinds as four instances. That design would be better if the kinds differed only in a formula. They do not. `SpaceOccupied` knows a volume per unit, and a weight limit would know a weight per unit, while `AverageInvestment` reads the SKU's own value and knows nothing extra. That is, the kinds

differ in what they know, and holding state that varies by kind is what a subtype is for.

Notice also that this hierarchy is not sealed, while Section 3.11.3 seals four hierarchies in Chapter 3. Sealing buys exhaustive checking, and exhaustive checking pays only when something dispatches on the kind and the mathematics closes the list. Both conditions hold in Chapter 3, where Table 3.1 is a table of four rows. Neither holds here. A solver asks a measure for a number and never asks which measure it is holding, and the list of scarce things in Section 4.1 is open. Thus, a new measure needs two methods and no edit anywhere else.

4.8.6. Alternatives Are Types, Settings Are Values

Figure 4.18 illustrates two more hierarchies, and both exist because the domain offers alternatives.

Section 4.2.3 searches the multiplier until the measure meets its ceiling, which Goal Seek does on the worksheet and `LagrangianSearch` does here. Solver offers a second way, optimizing the quantities directly, which the code shown here does not implement. And Section 4.3 gives a third, a closed form under a budget when every SKU holds at $h_j = ic_j$. Every quantity then shrinks by the same factor, so we can invert Equation 4.9.

```
class ProportionalScaling(private val carryingCharge: Double)
 ↪  : LotSizingMethod {
    override fun solve(portfolio: Portfolio, constraint:
     ↪  Constraint): ReplenishmentPlan {
        val free =
         ↪  AverageInvestment.measure(portfolio.freePlan())
        val ratio = free / constraint.limit
        val price = carryingCharge * (ratio * ratio - 1.0)
        return portfolio.planAt(AverageInvestment, price)
    }
```

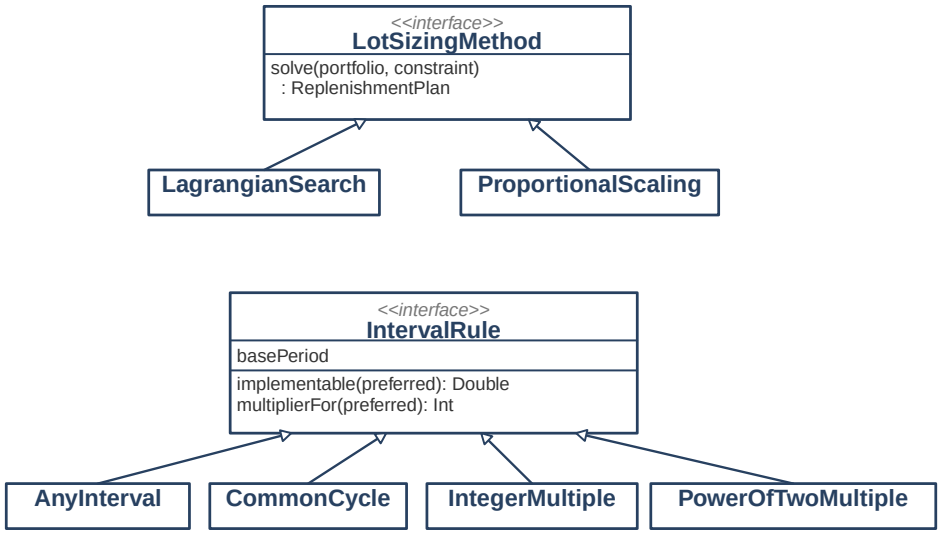


Figure 4.18.: The two strategy hierarchies. Each interface exists because the domain offers alternatives. Section 4.2 shows three ways to solve one constrained problem, of which two are implemented here, and Section 4.5.1 names four ordering practices. Notice that neither interface carries a setting that distinguishes its kinds.

```
}
```

Notice that no search appears in it. It reproduces the multiplier of Section 4.2 to every digit the search finds, and it refuses to run when its preconditions fail. An interface with one implementation is speculation. The second implementation is what justifies this one.

Section 4.5.1 does not present one model with a setting. It presents four practices: order whenever you like, order everything on every cycle, order on whole multiples of a cycle, and order on powers of two. Each one answers the same question. Given the interval a location would prefer, what interval can the operation run?

```
interface IntervalRule {
    val name: String
    val basePeriod: Double

    /** The runnable interval nearest the one this SKU or
     ↪  block would prefer. */
    fun implementable(preferredInterval: Double): Double

    fun multiplierFor(preferredInterval: Double): Int =
        max(1, Math.round(implementable(preferredInterval) /
 ↪  basePeriod).toInt())
}

class PowerOfTwoMultiple(override val basePeriod: Double) :
 ↪  IntervalRule {
    override fun implementable(preferredInterval: Double):
     ↪  Double =
        (1 shl exponentFor(preferredInterval)) * basePeriod
}
```

That question subsumes what look like two concepts. Coordinating a family and rounding an interval to a power of two appear to be different jobs, and they are one job with different answers. Thus, the arithmetic of Equation 4.28 is a function inside `PowerOfTwoMultiple` and not a class of its own, since it holds no state and there is only one way to do it. `multiplierFor` is written once on the interface, because under every rule the multiplier is the implementable interval divided by the base period.

We state the criterion behind both decisions once, since you will face the same question in every design you write.

> Make a class when it holds state that varies independently of its arguments, or when it is one of several interchangeable ways of doing the same thing. A computation with neither property is a function.

Measured against that criterion, `LotSizingMethod` and `IntervalRule` are interfaces because they name interchangeable alternatives, `SpaceOccupied` is a class because it carries a volume map, and the power-of-two exponent is a function because it is neither.

4.8.7. Three Couplings and No Common Base

The temptation here is a `Model` interface with a `solve()` on it, so that all three couplings can be held in one list. Recall that Section 4.1 says these are three different kinds of interaction, and the mathematics agrees. One is a Lagrangian relaxation, one is a one-dimensional search over a base period, and one is a combinatorial partition. Figure 4.19 shows what they do share, the SKU and the plan. A common supertype would assert a kinship the chapter denies, and every caller would then have to ask what it was really holding.

The two network topologies do share a supertype. They differ in the shape of one map, from a location to the location that supplies it. A chain points

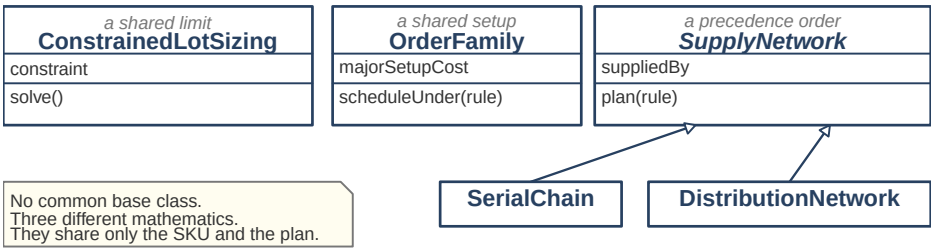


Figure 4.19.: The three couplings of this chapter as three unrelated classes. A shared limit, a shared setup and a precedence order are different mathematics, so they have no common supertype. The two network topologies do share one, since they differ only in the shape of a single map.

each location at the one above it, and a distribution network points every region at the center.

```
abstract class SupplyNetwork(
    val skus: List<SKU>,
    private val suppliedBy: Map<SKU, SKU>,
) {

    /** @eq-echelon-holding: the value this location adds,
     ↪  not the value accumulated through it. */
    fun echelonRate(sku: SKU): Double =
        sku.holdingRate - (suppliedBy[sku]?.holdingRate ?:
 ↪  0.0)

    /** The intervals that solve the relaxed problem, before
     ↪  any rounding. */
    abstract fun relaxedIntervals(): List<Double>

    /** The relaxed intervals made runnable, one rule applied
     ↪  to every location. */
```

```
    fun plan(rule: IntervalRule): List<Double> =
     ↪  relaxedIntervals().map { rule.implementable(it) }
}
```

One expression serves both structures. It gives $h_i - h_{i+1}$ down a chain and $h_i - h_0$ across a distribution network, and it gives a location with no supplier its own rate, which Equation 4.31 states as a separate case. Notice that `echelonRate` sits on the network and not on the SKU. An echelon rate is a difference between neighbors, and a SKU that knew its neighbor would record the topology twice.

The two subclasses override the one thing that differs, the algorithm. `SerialChain` implements Algorithm 4.1 and `DistributionNetwork` implements the scan of Section 4.6.3. Section 4.7 built a worksheet for the second and not for the first, and the reason survives the change of language. The scan has a step count known in advance and the partition does not.

4.8.8. How the Objects Solve It

A class model says what exists. It does not say what happens, so we trace S1 of Table 4.24 from the caller to the answer.

The four middle messages of Figure 4.20 repeat until the bracket closes.

```
class LagrangianSearch(
    private val precision: Double = 1.0E-12,
    private val maximumIterations: Int = 400,
) : LotSizingMethod {

    override fun solve(portfolio: Portfolio, constraint:
     ↪  Constraint): ReplenishmentPlan {
        val measure = constraint.measure
        var lo = 0.0
```

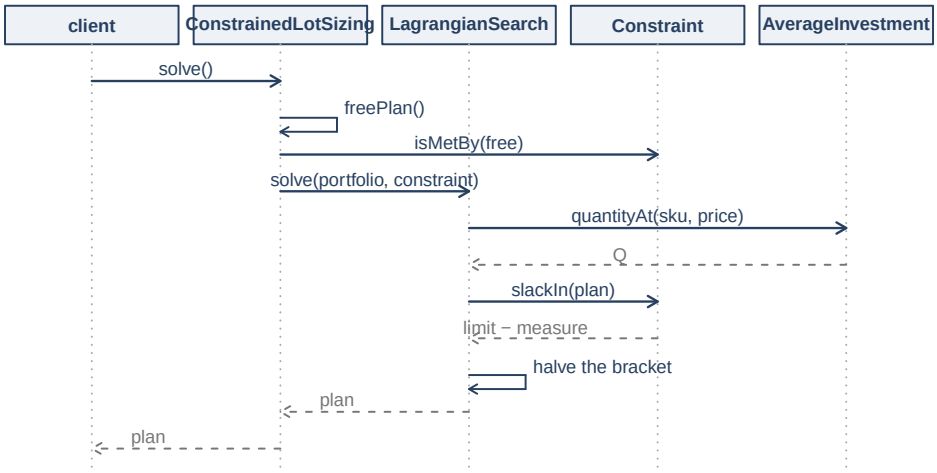


Figure 4.20.: Solving under a limit. The study computes the free plan and asks the constraint whether the limit binds, and the method takes over if it does. Notice that the search never computes an order quantity itself. It asks the measure, hands the plan back to the constraint for a slack, and halves its bracket on the sign of the answer. The dashed arrows are returns.

```
        var hi = 1.0
        while (constraint.slackIn(portfolio.planAt(measure,
         ↪ hi)) < 0.0) {
            hi *= 2.0
            check(hi < 1.0E12) {
                "No price high enough to meet the
                 ↪ ${measure.name} limit of
                 ↪ ${constraint.limit}"
            }
        }
        repeat(maximumIterations) {
            val mid = 0.5 * (lo + hi)
            if (hi - lo < precision) return@repeat
            if (constraint.slackIn(portfolio.planAt(measure,
             ↪ mid)) < 0.0) lo = mid else hi = mid
        }
        return portfolio.planAt(measure, 0.5 * (lo + hi))
    }
}
```

Notice what the search does not know. It never writes Equation 4.5, never names a budget, and never asks what the measure is. It knows that raising the price lowers consumption, which Section 4.2.3 proves, and it relies on nothing else. Thus, one search serves a working capital account, a warehouse floor and a purchasing department, and `quantityAt` belongs on the measure.

Notice also the first two lines of the loop. The search widens its bracket until the slack changes sign. Section 4.7.1 observed that Goal Seek does not take that step, since it starts from whatever is already in the cell. Neither tool is safe on its own, and both rest on the same proof. They differ in how much of the search they arrange for you.

Section 4.8.1 said that solving under a limit and tracing the frontier would turn out to be one thing. Compare the two.

```
    fun at(shadowPrice: Double): ReplenishmentPlan {
        val quantities = priced.skus.map {
            ↪ AverageInvestment.quantityAt(it, shadowPrice) }
        return portfolio.planFor(quantities, shadowPrice)
    }
```

That is the body of the search's inner step with the search removed, because the exchange curve is handed the price instead of finding it. Same objects, same message, one fewer participant. Had these two restated Equation 4.5 separately, the design would have failed at the exact place where Section 4.4 says the two problems meet.

One subtlety deserves a line. `priced` is the portfolio with every holding rate set to zero, because Section 4.4 has the multiplier replace the carrying charge instead of adding to it. Expressing that as a property of the SKUs keeps `AverageInvestment.quantityAt` the only place a quantity is computed.

A point on the curve is not a new kind of thing. `at` returns a `ReplenishmentPlan`, the type the constrained solve returns, because a point on the exchange curve is a portfolio evaluated at a price. It already has an investment and an ordering cost, the two axes of Figure 4.1. A `CurvePoint` class would add a noun the chapter never uses and force a conversion at every boundary. Before adding a type for a result, check whether the result is a familiar object under new conditions.

Two smaller decisions follow. The curve returns a `Sequence` and not a `List`, so a thousand-point sweep evaluates one point at a time and holds one. And the variety index J^* of Equation 4.19 is computed once on the curve, since it depends only on the items. Putting it on each point would compute the same number a thousand times and suggest that it varies along the curve.

4.8.9. Solving the Chapter's Examples

A design is judged by what it is like to use. We take the five scenarios of Table 4.24 in order and solve the chapter's own examples with it. Every figure below is produced by running the code, in the way Section 3.12 produces the figures of Chapter 3.

Table 4.27.: What to build for each model of this chapter, and what to ask it. Notice that the middle column never names a measure or a rule, because those are handed in, and are not chosen by the type.

To answer	Build	Then ask for
Section 4.2, a limit	`Portfolio`, `Constraint`, `ConstrainedLotSizing`	`binds`, `solve()`, `priceOfTheConstraint()`
Section 4.3, what it cost	the same study	`priceOfTheConstraint()`, `quantityRatios()`
Section 4.4, the frontier	`ExchangeCurve`	`constant`, `varietyIndex`, `at(price)`
Section 4.5, a shared setup	`OrderFamily`, an `IntervalRule`	`costOf(rule)`, `bestBasePeriod`, `scheduleUnder(rule)`
Section 4.6.2, a chain	`SerialChain`	`blocks()`, `relaxedIntervals()`, `plan(rule)`
Section 4.6.3, a network	`DistributionNetwork`	`ranked()`, `pinnedSet()`, `centralInterval()`, `plan(rule)`

Example 4.1 is three items, a carrying charge of 30% per year, and $10,000

of working capital. We build the portfolio and the constraint, then hand both to a study.

```
val skus = listOf(
    SKU.pricedAt("Item 1", demandRate = 12000.0, unitValue =
 ↪  20.0,
        orderCost = 50.0, carryingCharge = 0.30),
    SKU.pricedAt("Item 2", demandRate = 25000.0, unitValue =
 ↪  10.0,
        orderCost = 50.0, carryingCharge = 0.30),
    SKU.pricedAt("Item 3", demandRate = 8000.0, unitValue =
 ↪  15.0,
        orderCost = 50.0, carryingCharge = 0.30),
)
val storeroom = Portfolio(skus)
val budget = Constraint(AverageInvestment, limit = 10000.0)
val study = ConstrainedLotSizing(storeroom, budget)
```

Three lines of construction, and nothing has been computed yet. We now ask the study what it was built for.

```
study.binds                                  // true
val held = study.solve()
held.shadowPrice                             //     0.146430
held.measuredBy(AverageInvestment)           // 10000.0000
held.measuredBy(RelevantCost)                //  7464.2984
study.priceOfTheConstraint()                 // difference
 ↪  145.0375, 1.9816%
study.quantityRatios()                       // 0.819755
 ↪  three times
```

Every figure is Example 4.1's. Notice that we ask `measuredBy` for two different measures of one plan. One of them is the limited quantity and

the other is the objective, and the plan does not know which one was constrained.

Changing the constraint changes one line. Example 4.2 is the same distributor with ordering costs that differ across items and a dock that can take 50 receipts a year, so `dockSkus` holds the data of Table 4.3.

```
val dock = ConstrainedLotSizing(
    Portfolio(dockSkus),
    Constraint(ReplenishmentWorkload, limit = 50.0),
)
```

```
  free orders per year     85.1341
  free relevant cost     6815.3324
  shadow price             64.3293 per order
  Item 1   k+theta 114.3293   held  676.2523   ratio 1.5121
  Item 2   k+theta  84.3293   held 1185.5328   ratio 2.0534
  Item 3   k+theta 144.3293   held  716.3594   ratio 1.3432
  orders per year          50.0000
  relevant cost          7621.2652
  the limit cost          805.9328  (11.8253%)
```

Section 4.1 called a budget and a dock two of the four things an organization runs short of. Here that claim is discharged by one word in one argument. `ConstrainedLotSizing`, `LagrangianSearch` and `Portfolio` are untouched. The measure changed, and with it the sign of the marginal, the term the multiplier joins, and the fact that the quantities grow instead of shrinking.

Look next at the ratios. Under the budget every item shrank by the same 0.819755, and here they grow by 1.51, 2.05 and 1.34. Thus, the code reproduces the distinction of Section 4.3, that a budget rescales while a dock reallocates, and no type in the program knows that the distinction exists.

Where a closed form exists, we pass it as a third argument.

```
ConstrainedLotSizing(storeroom, budget,
 ↪  ProportionalScaling(carryingCharge = 0.30))
```

The search returns 0.14642984 and the closed form returns 0.14642984. For example, asking `ProportionalScaling` for a space limit raises an error instead of returning a wrong number, since Section 4.3 holds only under a budget with $h_j = ic_j$.

Example 4.5 is nine items in a utility storeroom, held in a `Portfolio` named `utility`.

```
val curve = ExchangeCurve(utility)
curve.constant                                  //
 ↪  195,147,265.65
curve.decomposedConstant                        //
 ↪  195,147,265.65
curve.varietyIndex                              //
 ↪  5.6813
curve.investmentAt(0.25)                        //     27,939.02
curve.orderingCostAt(0.25)                      //      6,984.76
```

The two routes to the constant, Equation 4.18 and Equation 4.22, agree to the cent. That is the arithmetic check the worksheet performs in rows 15 and 16. Sweeping the price illustrates the claim of Equation 4.14.

```
  price 0.10   investment    44175.48   ordering   4417.55   product   195147265.65
  price 0.25   investment    27939.02   ordering   6984.76   product   195147265.65
  price 0.40   investment    22087.74   ordering   8835.10   product   195147265.65
```

Example 4.7 is four items on one truck with a $400 major setup. Here the rule is what varies, so the rule is what we pass.

```
val family = OrderFamily(Portfolio(supplierSkus),
 ↪  majorSetupCost = 400.0)
family.independentCost                          // 8422.39
family.costOf(CommonCycle(family.commonCycleInterval))
val best = PowerOfTwoMultiple(family.bestBasePeriod {
 ↪  PowerOfTwoMultiple(it) })
family.costOf(best)
family.scheduleUnder(best).nestsOn(best.basePeriod)
```

```
  independent              8422.39
  common cycle             6497.54   T = 0.14621 year (53.4 days)
  integer multiple         6402.06   T = 0.14214 year (51.882 days)   m = [1, 1, 2,
  power of two             6403.34   T = 0.14192 year (51.800 days)   m = [1, 1, 2,
  nesting costs             0.0200%
  the schedule nests    true
```

Notice that `bestBasePeriod` takes a function from a base period to a rule, and not a rule itself. The search runs over base periods and every rule carries one, so the caller has to say how to build a rule at each candidate. That is the one place where the shape of the interface imposes something on a caller.

The last line checks the chapter's argument instead of accepting it. Section 4.5.2 justifies powers of two by nesting, and `nestsOn` asks the answer whether it nests.

Example 4.9 is five stages and Example 4.10 is a warehouse feeding six regions. We build them differently and ask them the same questions.

```
val chain = SerialChain(stages)
chain.blocks()                                  // {1,2} at
 ↪  0.5933, {3,4,5} at 0.6234
chain.costOf(chain.relaxedIntervals())          // 183.25
```

```
val weekly = PowerOfTwoMultiple(1.0 / 52.0)
chain.costOf(chain.plan(weekly))              // 183.31

val network = DistributionNetwork(warehouse, regions)
network.ranked()                              // 5, 3, 6, 4,
 ↪  1, 2
network.pinnedSet()                           // 6, 4, 1, 2
network.centralInterval()                     // 0.343049
network.plan(weekly)                          // every
 ↪  location at 0.307692
```

The same `weekly` object goes to a chain and to a network, and neither asks what it is. An interval rule turns a preferred interval into a runnable one, and a chain block and a warehouse both have a preferred interval.

No listing in this section branches on which constraint, which method, which ordering practice or which topology is in play. The `if` that a first implementation would write in each of those places is instead the choice of which object we constructed, made once, at the top.

4.8.10. Checking the Design Against the Mathematics

A design that cannot be checked is an opinion. Two things make this one checkable.

The first is Table 4.28, in which every result of this chapter has one home in the code.

Table 4.28.: Every result of this chapter and the one place that implements it. A row with no home, or a class with no row, is a defect in one of the two.

The mathematics	Where it lives
Equation 3.17	`SKU.economicOrderQuantity`
Equation 4.2	`AggregateMeasure.measure`, `Constraint.isMetBy`
Equation 4.5	`ConstrainableMeasure.quantityAt`
Equation 4.6	`LagrangianSearch`, over `Constraint.slackIn`
Equation 4.9	`ProportionalScaling`
Equation 4.14, Equation 4.19	`ExchangeCurve`
Equation 4.23, Equation 4.25	`OrderFamily`, `IntegerMultiple`
Equation 4.28, Equation 4.30	`PowerOfTwoMultiple`
Equation 4.31	`SupplyNetwork.echelonRate`
Algorithm 4.1	`SerialChain.blocks`

Read Table 4.28 in both directions. Downward it says where to look when a number is wrong. Upward it tests the design, because every class in the package appears in the table or collaborates with something that does. A class in neither would be doing something this chapter never asked for.

The second is that every quantity the chapter reports can be recomputed a second way. Two of those second ways are worth carrying past this chapter.

Check a marginal against a numerical derivative. Each `ConstrainableMeasure` supplies both a measure and its marginal, and the marginal is where a sign slip hides. Such a slip still produces order quantities and still converges, so the answer looks like an answer. Comparing `marginalAt` against a numerical derivative of the `measure` beside it is what catches that, and nothing else will. For the same reason, each closed-form `quantityAt`

is worth comparing against the general root finding default, which also demonstrates that a new measure really does need only two methods.

Check against an instance that can exhibit the thing being checked. A portfolio whose ordering costs are all equal cannot show the reallocation of Example 4.2, because a constant added to equal ordering costs changes them equally. A check written on such a portfolio agrees with the chapter and confirms nothing. Notice that it would look correct. Recall from Section 3.12 that this is not hypothetical: a ranking there came out identical under two different wrong objectives, and only a portfolio with unequal costs told them apart.

One question separates a design from an arrangement. What does the next requirement cost? A fifth measure, say a limit on handling hours, is a new class with two methods and no edit anywhere. A fifth ordering practice is a new `IntervalRule`. A third solution method is a new `LotSizingMethod`, and a third network topology is a subclass supplying one map and one algorithm. None of those changes an existing class.

4.9. Summary

Recall that Chapter 3 decided one item at a time. This chapter kept every assumption of that chapter except the one that made it easy.

Three kinds of coupling, three kinds of mathematics. A shared resource couples items through a constraint and leaves their cost functions separate. A shared setup couples their cost functions and leaves them free of constraints. An echelon structure couples them end to end, so that one stock exists in order to replenish another. Thus, the first is solved by moving the constraint into the objective, the second by imposing structure on the schedule, and the third by fixing the accounting before either technique can be applied.

The multiplier is the economics, not the arithmetic. The multiplier θ^* is the rate at which relaxing the limit reduces cost. Thus, it says what another dollar of budget or another square foot is worth, and when it stops being worth asking for.

Where it enters depends on the constraint, and the three cases should be kept separate. Under a limit on stock the multiplier is added to the holding rate, as a surcharge on whatever consumes the scarce resource. Under a limit on replenishments it is added to the ordering cost instead, and carries units of dollars per order. Where holding cost does not appear in the objective at all, as when management fixes an investment target, the multiplier takes its place and *is* the implied carrying charge.

A budget constraint rescales and does not reallocate. When holding cost is a carrying charge on value, every order quantity shrinks by the same factor $\sqrt{i/(i+\theta)}$. That is, the unconstrained solution already had the items in the right proportion, and the constraint asks only that everything be smaller. You should treat this as specific to a budget. A space constraint, whose coefficients have nothing to do with value, reallocates, and so does a limit on replenishments, because a constant added to unequal ordering costs changes them unequally.

Aggregate performance lies on a curve fixed by the item data. Investment and replenishment frequency cannot be chosen separately. Their product is determined by the item data and their ratio is k/γ. Thus, a portfolio above the curve can improve on both measures at once, and a portfolio on the curve is asserting a cost ratio. Comparing that assertion against estimated costs is a check on practice that needs no new data.

When ordering costs differ across items the same curve survives, in the form $\frac{1}{2}J^*K^w\lambda^a c^w$. Four things therefore set the frontier: how varied the items are, what an order costs, how much is demanded, and what the goods are worth.

Coordination is worth more than cleverness. In Example 4.7, sharing orders at a single common cycle captured 23 of the 24 percentage points

available, and differentiating the multipliers added one. Restricting those multipliers to powers of two, which is what makes a schedule nest and therefore what makes it runnable, cost two hundredths of a percent against a worst case of 6.07%. Recall that this worst case is Equation 3.22 applied to cycle lengths. Thus, the flatness that made case quantities affordable in Section 3.7 is the same fact that makes power-of-two schedules affordable here.

Echelons need their own accounting, and then behave like everything else. A unit held at a downstream stage has already been paid for upstream, so charging every stage its installation rate counts the same money twice. Echelon stock and the value-added rate $h'_i = h_i - h_{i+1}$ fix that. Once fixed, every stage looks like an EOQ in the interval form of Equation 4.26.

What remains is the ordering constraint. Stages in a chain must not replenish faster than what they supply, and locations in a distribution system must not wait longer than the warehouse that feeds them. Both are solved the same way, by merging the locations that would violate the constraint into blocks that share an interval and then rounding each block to a power of two. Notice that in Example 4.9 and Example 4.10 the rounding put every location on one schedule, because intervals within a factor of two often collapse to the same power.

Two implementations, and what each one makes visible. Section 4.7 built these models as worksheets and Section 4.8 built them as a program, and the two were kept apart because they teach different things. A worksheet makes a method visible: a formula is a cell and a layout is a claim about the model that anybody can see. A program makes a structure visible instead, and only to whoever wrote down the analysis behind it. Notice that the same four questions were answered on both sides. What belongs to an item and what belongs to the problem, whether a difference is a value or a kind, whether a result needs a new name, and when a search may be trusted.

Throughout this chapter demand has remained known and constant. In the next chapter we let it vary over time while keeping it known, which is the subject of Chapter 5. From Chapter 7 onward it stops being known at all, and Chapter 9 returns to the serial and distribution structures of this chapter once it does.

4.10. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year. Report order quantities and reorder intervals to four decimal places, multipliers and reorder intervals in weeks to two, costs to the nearest cent, and multipliers, ratios and indices to six significant figures. Where an exercise asks for a bisection, iterate until the constraint is met to within one part in a million of the limit. Round an order quantity to an integer only when the exercise asks for the quantity that would actually be ordered.

Exercise 4.1. Four items are stocked, each costing \$40 to order, with a carrying charge of 25% per year.

Item	λ_j	c_j
1	6,000	\$30
2	15,000	\$8
3	2,400	\$50
4	40,000	\$2

Determine the unconstrained order quantities and the average investment they require. Then impose a budget of \$6,000 on average investment, find θ^* by bisection on Equation 4.6, and report the constrained quantities and the annual cost of the constraint.

Verify Equation 4.9 by computing the ratio of each constrained quantity to its unconstrained value, and confirm that all four ratios agree.

Exercise 4.2. Repeat Exercise 4.1 with a **space** constraint instead of a budget. Item j occupies v_j cubic feet per unit, with $v_1 = 0.5$, $v_2 = 0.2$, $v_3 = 1.5$, and $v_4 = 0.05$, and 400 cubic feet of shelf are available at the moment a replenishment arrives, which is when the stock is at its peak.

Determine the constrained quantities, then compute the ratio of each to its unconstrained value as you did before.

Explain why the ratios are no longer all equal, and identify the property of the budget constraint that Equation 4.9 depends on.

Exercise 4.3. The multiplier is a shadow price. For the item set of Exercise 4.1, compute the minimum relevant cost at budgets of \$5,000, \$6,000, \$7,000, \$8,000, and at the unconstrained investment.

Plot the minimum cost against the budget, and estimate the slope at \$6,000 numerically. Compare it with $-\theta^{*}$ from Exercise 4.1 and explain any difference.

State the budget at which the multiplier reaches zero, and say what that budget is in terms of the unconstrained solution.

Exercise 4.4. A firm places 250 replenishment orders a year across a group of items and carries \$1,000,000 of cycle stock. Its accountants report a carrying charge of 22% per year, and a study of the purchasing department estimates the ordering cost at \$120 per order.

Assuming the firm is on its exchange curve, determine the cost ratio its practice implies and compare it with the ratio its stated costs imply.

State which direction along the curve the firm should move, whether that means ordering more often or less often, and what it would do to the stock it carries. Then state the one circumstance under which the comparison would tell you nothing.

Exercise 4.5. For the nine items of Example 4.5, verify by direct computation that $\bar{I}^a \cdot \overline{OC}^a$ is the same at $\theta = 0.10$ and at $\theta = 0.50$.

Then compute the variety index from Equation 4.19, and recompute it after deleting the pad-mount transformer from the portfolio. Report both values and explain what the change says about that item's contribution to aggregate performance.

Finally, compute λ^a, C^a, c^w, and K^w for the full portfolio and confirm that Equation 4.22 reproduces the constant you verified above.

Exercise 4.6. For the four items of Example 4.7, suppose the major setup cost falls from \$400 to \$50 because the supplier introduces electronic ordering.

Recompute the independent, common-cycle, and best integer-ratio policies. Report the new multipliers and say what has happened to the value of coordinating.

Explain, in terms of Equation 4.23, why a small major setup cost pushes the solution back toward independent ordering.

Exercise 4.7. Show that for a cost of the form $C(T) = k/T + hT/2$, choosing T to be $\sqrt{2}$ times its optimal value costs the same as choosing it to be $1/\sqrt{2}$ times that value, and that both cost the 6.07% of Equation 4.30.

Then explain why this is the worst case for a power-of-two policy, and why a policy whose base period may itself be chosen does better than 6.07%.

Exercise 4.8. The items of Example 4.7 are reordered under a common cycle policy, which Example 4.7 shows captures most of the available saving.

Compute, for each of the four items separately, the penalty its own cost incurs under the common cycle relative to the cycle it would choose under the power-of-two policy. Report which item bears the cost of the simplification and by how much.

Then state the management argument for adopting the common cycle anyway, and the argument against.

Exercise 4.9. A three stage chain has $\lambda = 1{,}000$ units per year and installation holding rates $h_1 = \$0.80$, $h_2 = \$0.55$, $h_3 = \$0.20$ per unit per year.

Compute the echelon holding rates h_i' and verify Equation 4.32 by showing that $h_1' + h_2' + h_3'$ recovers h_1.

Then, at an instant when the on-hand quantities are 40, 90, and 25 units at stages 1, 2, and 3, report the echelon stock at each stage and the total value-added holding cost rate being incurred, and confirm that it equals what installation accounting would charge.

Exercise 4.10. A four stage chain, $4 \to 3 \to 2 \to 1$, runs at $\lambda = 800$ units per year with a weekly base period.

Stage i	1	2	3	4
k_i	\$18	\$40	\$9	\$30
h_i	\$1.00	\$0.70	\$0.30	\$0.10

Compute h_i' and g_i, then show that the stage-by-stage intervals $\sqrt{k_i/g_i}$ violate Equation 4.39.

Run the algorithm of Section 4.6.2 to find the blocks, report $T(r)$ for each, and round each to a power of two. State the cost of the relaxed solution, the cost of the power-of-two solution, and the penalty between them.

Exercise 4.11. In Example 4.10 the ordering cost at the central warehouse is \$500. Suppose it falls to \$150.

Rebuild $\mathcal{C}^0$ and report which regions are now pinned to the central warehouse and which are left alone.

Explain, in terms of the test in Section 4.6.3, why lowering k_0 pulls regions into $\mathcal{C}^0$ rather than releasing them, and say what that means physically about a central warehouse that becomes cheap to replenish.

Then work the comparison in the other direction and identify the value of k_0 at or above which no region is pinned at all.

Exercise 4.12. Both Example 4.9 and Example 4.10 end with every location on a single power-of-two interval, even though the relaxed solution gave each block or region a different one.

Explain why this happens, and state exactly when two relaxed intervals round to the same power of two. Then show that a ratio below two is not sufficient, using your own answer to Exercise 4.10 as the counterexample.

Finally, modify the data of Example 4.10 so that the pinned set and region (1) round to different powers of two, and report the intervals that result.

5. Dynamic Lot Sizing

Learning objectives

After reading this chapter you should be able to:

- explain why the economic order quantity argument fails when the demand rate changes over time
- formulate the lot sizing problem for time-varying deterministic requirements
- state the zero-inventory ordering property and use it to reduce the problem to a choice of order periods
- derive the window cost of a replenishment and compute it for every pair of periods
- solve the problem to optimality with the Wagner-Whitin recursion and read the plan off the traceback
- formulate the same problem as a mixed integer program
- apply the Silver-Meal, least unit cost, and part-period balancing heuristics
- evaluate the cost penalty of a heuristic and say when a heuristic pays at all
- explain what a rolling horizon does to all of these methods

Chapter 3 decided one order quantity and used it forever. That was possible because the demand rate never changed. The stock fell at the same slope, the cycles were identical, and a cost per unit time could be written for a single cycle and minimized. Chapter 4 removed the assumption that items could be decided independently, but it left the constant rate in place.

In this chapter we remove the constant rate. Demand is still known in advance and still deterministic, but it now varies from period to period. We take up how to state the problem, how to solve it exactly, how to solve it quickly, and what happens to all of that when the horizon slides forward a period at a time.

5.1. When the Demand Rate Changes

A distributor of turf-equipment parts stocks a mower deck belt. The belt sells almost nowhere in the winter, very heavily in the spring, and then again in the autumn as dealers build up for the following season. The forecast for the coming twelve months is given in Table 5.1.

Table 5.1.: The monthly requirement for the mower deck belt.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220

The total is 1,840 belts, so the average requirement is 153.33 belts a month. It costs \$300 to place an order, the belt costs \$50, and the carrying charge is 2% per month, which by Section 1.4.4 makes $h = ic = \$1.00$ per belt per month.

Nothing stops us from applying Equation 3.17 to the average, and we get

$$Q^* = \sqrt{\frac{2(300)(153.33)}{1.00}} = 303.3 \text{ belts},$$

That is a supply of about two months. The trouble is that the answer means very little. Two months of supply is five months of stock in February and less than one month of stock in May. The quantity was derived from a picture of inventory that this item never has: a sawtooth of identical triangles.

Look at where the classical argument actually breaks. Recall from Section 3.5.4 that it minimizes cost per unit time over one cycle, and it can do that because every cycle is the same. Here there is no repeating cycle. An order placed in month 4 covers a different amount of demand, over a different length of time, at a different holding cost, than an order placed in month 9. There is no single cycle whose cost can stand for all of them, so there is nothing for the calculus to minimize.

What replaces it is a finite horizon and a decision in every period of it. We are no longer looking for a quantity. We are looking for a schedule.

5.1.1. The Requirements Schedule

The horizon runs over N periods, indexed $t = 1, 2, \ldots, N$. In each period we know

- d_t, the requirement in period t,
- k_t, the cost of placing an order in period t,
- c_t, the unit purchase cost in period t, and
- h_t, the cost of carrying one unit from the end of period t into period $t+1$.

Together these are the *requirements schedule* for the item. The subscripts are carried through the derivations because nothing in them requires the costs to hold still. In every example of this chapter $k_t = k$, $c_t = c$ and $h_t = h$, and the subscript is dropped whenever that is so.

It is convenient to name the total requirement over a stretch of periods. Write

$$d[t, u] = \sum_{s=t}^{u} d_s$$

for the requirement of periods t through u inclusive, with $d[t, u] = 0$ when $u < t$. For the belt, $d[1, 3] = 220$ and $d[5, 9] = 840$.

5.1.2. The Assumptions

The model of this chapter makes the following assumptions.

1. A single item is planned, independently of every other item.
2. The requirements d_t are known for all N periods and are not random.
3. The horizon is finite and ends at period N.
4. Demand within a period is satisfied from stock on hand at the start of that period, so a replenishment arriving in a period can cover that period.
5. Replenishment is instantaneous, and the lead time is zero or a known constant.
6. No shortages are allowed. Every requirement is met in the period it falls.
7. There is no limit on the order quantity and no limit on storage.
8. The starting inventory is zero, and nothing is left over at the end of the horizon.
9. The order cost k_t does not depend on the size of the order.
10. Carrying cost is charged on the inventory left at the end of a period.

Assumptions 1 through 3 place the chapter. Assumption 1 is what Chapter 4 removed and this chapter puts back, so that the time dimension can be studied on its own. Assumption 2 is the strong one, and Section 5.7 is about what to do because it is false.

Assumption 4 matters more than it looks. A unit that arrives in period t and is consumed in period t is never held, so it is charged nothing. Some treatments charge holding on the average of the beginning and ending balances instead, which gives a different answer to the same problem. We charge the ending balance, and that is the convention an MRP record uses and the one Chapter 6 will inherit.

Assumptions 8 and 10 are what make the horizon self-contained. Starting and ending empty means the plan neither inherits nor bequeaths a position,

so the cost we compute is the cost of the horizon and not a share of something longer.

5.1.3. Where This Chapter Sits

Chapter 3 produced one quantity that applied for all time. This chapter produces a quantity for each of N periods, and most of them are zero. Chapter 6 then runs the methods of this chapter once at every level of a bill of material, where the requirements are not forecast at all but computed from the level above. That is the setting in which these methods are most often used in practice, and it is the reason a chapter on deterministic time-varying demand is not a curiosity.

5.2. What a Plan Costs

Recall that a plan is a quantity $Q_t \geq 0$ for each period. Given the plan, the inventory follows from the balance

$$I_t = I_{t-1} + Q_t - d_t, \qquad t = 1, \dots, N, \tag{5.1}$$

with $I_0 = 0$. The no-shortage assumption is the requirement that $I_t \geq 0$ for every t, and starting and ending empty makes $I_0 = I_N = 0$.

The cost of the plan has three parts. Let $y_t = 1$ if an order is placed in period t and $y_t = 0$ otherwise. Then

$$TC = \underbrace{\sum_{t=1}^{N} k_t y_t}_{\text{ordering}} + \underbrace{\sum_{t=1}^{N} c_t Q_t}_{\text{purchase}} + \underbrace{\sum_{t=1}^{N} h_t I_t}_{\text{carrying}}. \tag{5.2}$$

5.2.1. The Purchase Term

Section 1.4.1 argued that the purchase term can be dropped, because every feasible policy buys the same total amount and so pays the same for it. That argument is exactly right, and it depends on something this chapter can remove.

Suppose $c_t = c$ for all t. Assumptions 6 and 8 force $\sum_t Q_t = \sum_t d_t$, so the purchase term is $c\sum_t d_t$ no matter how the orders are arranged. It is a constant, it cannot affect which plan is best, and we may as well leave it out. What is left,

$$TRC = \sum_{t=1}^{N} k_t y_t + \sum_{t=1}^{N} h_t I_t, \tag{5.3}$$

is the *relevant cost*, and every figure quoted in this chapter is a relevant cost unless it says otherwise.

Now suppose the price changes. If the belt costs \$50 through June and \$54 afterwards, then buying in June for a July requirement is worth \$4 a belt, and the purchase term is no longer the same under every plan. It has become part of the decision. This is the one place in the book where the argument of Section 1.4.1 does not hold, and the derivations below keep c_t so that the case is available when it is needed. Exercise 5.15 works an instance with period-varying costs, where the answer does depend on the purchase term.

5.2.2. Three Plans Without Any Theory

Before any method, ask what a planner would do with Table 5.1 and no inventory theory at all. Three answers suggest themselves, and all three are used in practice.

Lot-for-lot orders what each period needs: $Q_t = d_t$. Nothing is ever carried, because every unit arrives in the period it is consumed. It is the

plan an MRP system produces by default, and it is the right plan when ordering is nearly free.

An adjusted economic order quantity computes Q^* from the average requirement, as we did above, and then covers the whole number of periods whose requirement comes closest to Q^*. It keeps the quantity roughly steady and lets the timing float.

The adjustment is the whole of the difference between this rule and a stricter one that orders exactly Q^* every time, whatever the period boundaries are. That stricter rule is the one usually called a fixed economic order quantity, and it does not fit the model of this chapter. On the belt it orders 303.3150 belts seven times over, which is 2,123 belts against a requirement of 1,840, and it finishes the year holding 283 of them. Assumption 8 says the horizon ends empty, and no rule that orders in a quantity unrelated to the requirements can promise that. Exercise 5.11 works it out.

A periodic order quantity does the reverse. It converts Q^* into a time supply, $T_{EOQ} = Q^*/\bar{d}$, rounds it to a whole number of periods, and then orders that many periods of requirement every time. Here $T_{EOQ} = 303.3/153.33 = 1.98$, which rounds to two months, so the rule orders in months 1, 3, 5, 7, 9 and 11.

Example 5.1 (Costing a plan by hand)**.** The item is the mower deck belt of Table 5.1, and the modeling questions of the deterministic chapters have new answers here.

What is stocked? One item, the belt.

What is the demand process? Known in advance but different in every month: the twelve requirements of Table 5.1.

When is inventory reviewed? Once a month, at the month boundary.

What triggers replenishment, and how much? A plan, decided before the year starts, that says which months order and how much; a lot sizing rule is a way of writing that plan.

What happens to unmet demand? None occurs; every requirement is met in its month.

What costs are incurred, and when? $k = \$300$ each month an order is placed, and $h = \$1.00$ for every belt carried from one month into the next.

Let's begin with lot-for-lot on the belt. Eleven of the twelve months have a requirement, so eleven orders are placed, at \$300 each. Every order arrives in the month it is consumed, so $I_t = 0$ in every month and nothing is carried. The relevant cost is \$3,300, all of it setup.

Now we take the periodic rule with $T = 2$. The order in month 1 covers months 1 and 2, so $Q_1 = 40 + 60 = 100$. At the end of month 1 there are $100 - 40 = 60$ belts left, and at the end of month 2 there are none. The ledger runs

Table 5.2.: The periodic order quantity plan at a two month time supply.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220
Q_t	100	0	420	0	680	0	160	0	80	0	400	0
I_t	60	0	300	0	260	0	40	0	80	0	220	0

Six orders cost \$1,800. The ending balances sum to 960 belt-months, which at \$1.00 a belt-month costs \$960. The relevant cost is \$2,760.

Notice that the periodic rule pays \$960 to save \$1,500 of setup against lot-for-lot. That is a good trade on this item. Look also at where the money goes. Month 5 needs 420 belts and month 6 needs 260, so the order in month 5 carries 260 belts for a month, which alone costs \$260. An order in month 6 would have avoided that and cost \$300. The two are close, and that hints that the best plan lies near this one.

The adjusted economic order quantity is costed the same way once we know its plan, and the plan comes from comparing running totals with

$Q^* = 303.3$. From month 1 the totals are 40, 100, 220 and 520, and 220 is the closest, so the first order covers months 1 through 3. From month 4, 300 is closer than 720. From month 5, 420 is closer than 680. From month 6, 260 is closer than 380. From month 7 the totals run 120, 160, 160, 240 and 420, and 240 is the closest, so that order covers months 7 through 10. From month 11, 400 is closer than 180. Thus, the orders fall in months 1, 4, 5, 6, 7 and 11. The order in month 1 leaves $180 + 120 = 300$ belt-months, the order in month 7 leaves $120 + 80 + 80 = 280$, and the order in month 11 leaves 220, so the relevant cost is $\$1,800 + \$800 = \$2,600$.

Thus, we have three plans and no reason yet to prefer any of them. Their costs are collected in Table 5.3.

Table 5.3.: Three plans a planner would reach for without any theory.

Plan	Orders	Setup	Carrying	Relevant cost
Lot-for-lot	11	\$3,300	\$0	\$3,300
Adjusted economic order quantity	6	\$1,800	\$800	\$2,600
Periodic order quantity, $T = 2$	6	\$1,800	\$960	\$2,760

None of the three is optimal, and Section 5.4 will show that the best plan costs \$2,480. Before we can look for it we need to know how many plans there are.

5.3. Two Properties That Make the Problem Small

The plan is a vector of N non-negative numbers, so the search space is uncountable. Two properties cut it down to something a computer, and sometimes a person, can enumerate.

5.3.1. Zero-Inventory Ordering

An optimal plan never orders in a period that starts with stock on hand. That is, $I_{t-1}Q_t = 0$ for every t.

The argument is an exchange. Suppose a plan orders $Q_t > 0$ in a period that begins with $I_{t-1} > 0$ units on hand. Let $\delta = \min(I_{t-1}, Q_t)$, which is positive. Those δ units were bought in some earlier period $r < t$ and carried from r to t. We now consider the plan that buys them in t instead.

Nothing else changes. The same units are available in the same periods, so the plan is still feasible, and it still orders in t, so the setup in t is unchanged. What does change is that the δ units are no longer carried from r to t, which saves $\delta \sum_{s=r}^{t-1} h_s \geq 0$, and they are bought at c_t instead of c_r. With constant purchase costs the second change costs nothing and the first saves something whenever any h_s is positive, so the new plan costs no more. If the shift empties period r entirely, a setup is saved as well.

If the purchase cost varies, the exchange has to be allowed to go either way. Let x represent the number of the δ units bought in t rather than in r. The cost of the plan is linear in x on $0 \leq x \leq \delta$, because each unit moved changes the purchase term by $c_t - c_r$ and the carrying term by $-\sum_{s=r}^{t-1} h_s$, and a linear function on an interval is smallest at one of its ends. At $x = \delta$ period t no longer starts with those units on hand, and at $x = 0$ with $\delta = Q_t$ period t no longer orders and its setup is saved as well. Thus, one direction or the other costs no more, and the property holds whatever the prices do. This is Theorem 1 of Wagner and Whitin (1958), and it is why the window cost of Section 5.3.2 can keep c_t.

Repeating the exchange removes every violation, and we arrive at a plan that costs no more and orders only when empty. Thus, there is an optimal plan of that kind, and we may look only at plans of that kind.

The consequence is what matters. If a plan orders only when empty, then whenever it orders in period t it must order enough to cover periods t

through the period before its next order, and no more. That is, the quantity is determined by the timing:

$$Q_t = d[t, u] \quad \text{where } u + 1 \text{ is the next period with an order.}$$

Thus, a plan *is* a set of order periods. Period 1 always orders, because inventory starts at zero and d_1 must be met. Each of periods 2 through N either orders or does not. There are 2^{N-1} plans, which for the belt is 2,048.

That is a finite number, and it is also 2,048 only because $N = 12$. At $N = 52$ it is about 2×10^{15}. Enumeration is not the answer, but the fact that the plans can be counted at all is what makes the rest of the chapter possible.

5.3.2. The Window Cost

Since a plan is a set of order periods, its cost decomposes over the stretches between them. We define the *window cost* $w(t, u)$ as the cost incurred by a replenishment placed in period t that covers the requirements of periods t through u.

The order costs k_t. It buys $d[t, u]$ units at c_t each. The units for period t arrive and leave in period t, so they are never carried. The units for period $t + 1$ sit in stock at the end of period t, so they are charged h_t each. The units for period $t + 2$ are charged $h_t + h_{t+1}$, and so on. Collecting,

$$w(t, u) = k_t + \sum_{s=t}^{u} d_s \left(c_t + \sum_{r=t}^{s-1} h_r \right). \tag{5.4}$$

Thus, the bracket deserves a name of its own. Write

$$\hat{c}(t, s) = c_t + \sum_{r=t}^{s-1} h_r \tag{5.5}$$

for the cost of one unit bought in period t and held until it is used in period s, with $\hat{c}(t,t) = c_t$. Then Equation 5.4 is

$$w(t,u) = k_t + \sum_{s=t}^{u} d_s \, \hat{c}(t,s). \tag{5.6}$$

This is the step that makes everything afterwards cheap. A sum of carrying charges spread over many periods has been folded into a single price per unit, which depends only on where the unit was bought and where it is used. The cost of a window is then an ordinary inner product of requirements and prices.

Notice two things about Equation 5.6. First, $w(t,u)$ does not depend on any decision. It is a property of the data, so all of the window costs can be computed once, before any search begins. Second, $w(t,u)$ is increasing in u and the increments grow, because a unit added at the far end of the window has been held longer than any unit already in it.

With constant costs the relevant part of Equation 5.6 simplifies to

$$w(t,u) = k + h \sum_{s=t}^{u} (s-t) d_s, \tag{5.7}$$

where the purchase term has been dropped as Section 5.2.1 allows. The coefficient $s-t$ counts the periods a unit for period s waits. Every window cost quoted in this chapter is computed from Equation 5.7.

Example 5.2 (The window costs of the first half year)**.** Take the first six months of the belt, so that the table fits on a page. With $k = \$300$ and $h = \$1.00$,

$$w(1,3) = 300 + 1.00\,[0(40) + 1(60) + 2(120)] = 300 + 300 = \$600,$$

and the full table of windows is Table 5.4.

Table 5.4.: Window costs for the first six months of the belt. A blank cell is a window that would end before it began.

$w(t,u)$	$u=1$	2	3	4	5	6
$t=1$	300	360	600	1,500	3,180	4,480
2		300	420	1,020	2,280	3,320
3			300	600	1,440	2,220
4				300	720	1,240
5					300	560
6						300

Every diagonal entry is \$300, because a window covering one period carries nothing. Read across row 1 and the increments are 60, 240, 900, 1,680 and 1,300, which grow until month 6, whose requirement is smaller than month 5's. The increment from u to $u+1$ is $h(u+1-t)d_{u+1}$, so it grows with the distance and with the requirement, and either can dominate.

5.3.3. Property 2: A Requirement That Forces an Order

There is a second property that is often useful and costs nothing to check.

Suppose the requirement in period t satisfies $d_t > k_t/h_{t-1}$. Carrying those d_t units in from period $t-1$ would cost $h_{t-1}d_t > k_t$, which is more than a fresh order in period t costs. Thus, no optimal plan carries anything into period t, which means an optimal plan orders in period t.

For the belt, $k/h = 300$ belt-months. Month 5 needs 420 belts, and $420 > 300$, so every optimal plan orders in month 5. Nothing else in the year is that large.

The value of the property is that it splits the horizon. If period t must hold an order, then no window spans period $t-1$ to period t, and the problem over periods 1 through $t-1$ can be solved without reference to

the problem over periods t through N. That is, one problem of size 12 becomes two of size 4 and 8. Section 5.7 shows that this is also how a long rolling horizon can be cut short without losing anything, since the periods beyond a forced order have no influence on the decision being made now.

5.4. Choosing the Order Periods

Section 5.3 leaves us with a clean problem. A plan is a set of order periods, the periods between orders partition the horizon into windows, the cost of a window is $w(t, u)$, and the cost of a plan is the sum of its windows' costs. What remains is to choose the partition.

5.4.1. The Problem Is a Shortest Path

Draw $N + 1$ nodes, numbered 1 through $N + 1$. Node t means "the plan arrives at period t holding nothing." Node $N + 1$ means the horizon is finished. For every pair $t \leq u$, draw an arc from node t to node $u + 1$ and price it $w(t, u)$. The arc says: order in period t, cover through period u, and arrive at period $u + 1$ empty.

A path from node 1 to node $N + 1$ is a sequence of windows that tile the horizon, which by Section 5.3.1 is a plan. Its length is the sum of its arc costs, and that sum is the plan's cost. Thus, the lot sizing problem is a shortest path problem on an acyclic network of $N + 1$ nodes and $N(N + 1)/2$ arcs.

Figure 5.1 illustrates the network for the first four months of the belt.

Notice what the shortest path formulation buys. The arc costs were computed in Section 5.3.2 without reference to any decision, so the entire network is known before the search begins. A shortest path on an acyclic network with the nodes already in topological order is a single sweep.

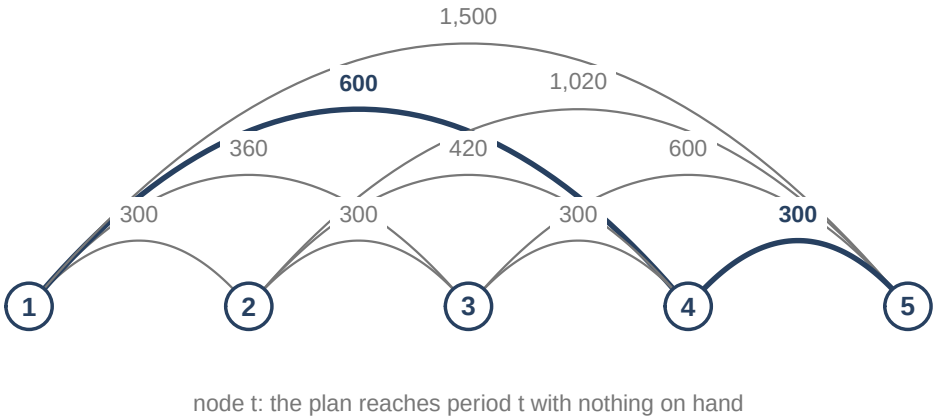


Figure 5.1.: The first four months of the belt as a shortest path problem. Node t means the plan reaches period t holding nothing, and the arc from t to $u+1$ is a replenishment in period t covering through period u, priced at $w(t,u)$. Every path from node 1 to node 5 is a plan. The heavy path, ordering in month 1 to cover months 1 through 3 and again in month 4, is the cheapest at \$900.

5.4.2. The Recursion

Let $V(u)$ be the least cost of covering periods 1 through u and arriving at period $u+1$ with nothing on hand. Then $V(0) = 0$, and for $u = 1, \ldots, N$,

$$V(u) = \min_{1 \le t \le u} \{V(t-1) + w(t,u)\}. \tag{5.8}$$

Read it as a question about the last order. Whatever the optimal plan for periods 1 through u is, it has a last order, and that order falls in some period t and covers periods t through u. Everything before period t is a plan for periods 1 through $t-1$ that arrives at t empty, and it had better be the cheapest such plan, or we could substitute the cheapest and do better. Thus, the cost is $V(t-1) + w(t,u)$, and we take the best t.

Let $S(u)$ record the minimizing t. Then $V(N)$ is the optimal cost and S holds enough information to recover the plan. The last order starts in period $S(N)$ and covers through period N. The order before it covers through period $S(N)-1$ and starts in period $S(S(N)-1)$. Walking backward this way ends when an order starts in period 1.

This is the algorithm of Wagner and Whitin (1958). It is stated here as a recursion over the window costs because that is what it is, and because Section 5.5 then needs only to change how far a window reaches.

Algorithm 5.1 The Wagner-Whitin algorithm. The three blocks are independent: the windows are a property of the data, the recursion is a shortest path, and the traceback reads the plan off the pointers.

```
wagnerWhitin(d, k, c, h over periods 1..N):

    // the windows, which depend on no decision
    for t = 1 to N:
        for u = t to N:
            w(t,u) <- k(t) + sum over s in t..u of d(s) * chat(t,s)

    // the recursion
    V(0) <- 0
    for u = 1 to N:
        V(u) <- infinity
        for t = 1 to u:
            if V(t-1) + w(t,u) < V(u):
                V(u) <- V(t-1) + w(t,u)
                S(u) <- t

    // the traceback
    orders <- empty list
    u <- N
    while u >= 1:
        t <- S(u)
        add (order in period t, quantity d[t,u]) to orders
        u <- t - 1

    return orders, V(N)
```

Example 5.3 (The recursion on the first half year)**.** Take the six-month instance of Example 5.2, whose window costs are Table 5.4. The recursion

fills one column at a time, and each column has one entry for each period that could hold the last order.

Table 5.5.: The recursion on the six month instance. Each entry is $V(t-1)$ from an earlier column plus a window cost from Table 5.4. The least entry in each column is in bold.

$V(t-1)+w(t,u)$	$u=1$	2	3	4	5	6
last order in period 1	**300**	**360**	**600**	1,500	3,180	4,480
2		600	720	1,320	2,580	3,620
3			660	960	1,800	2,580
4				**900**	1,320	1,840
5					**1,200**	**1,460**
6						1,500
$V(u)$	300	360	600	900	1,200	1,460
$S(u)$	1	1	1	4	5	5

Consider column 4. Covering periods 1 through 4 with one order costs $w(1,4) = 1{,}500$. Ordering in period 2 costs $V(1) + w(2,4) = 300 + 1{,}020 = 1{,}320$. Ordering in period 3 costs $V(2) + w(3,4) = 360 + 600 = 960$. Ordering in period 4 costs $V(3) + w(4,4) = 600 + 300 = 900$, the least of the four, so $V(4) = 900$ and $S(4) = 4$.

The optimum is $V(6) = \$1,460$. The traceback starts at $u = 6$: $S(6) = 5$, so the last order falls in period 5 and covers periods 5 and 6, or $d[5,6] = 420 + 260 = 680$ belts. Now $u = 4$. Since $S(4) = 4$, the next order back falls in period 4 and covers period 4 alone, 300 belts. Now $u = 3$, and

$S(3) = 1$, so the first order falls in period 1 and covers periods 1 through 3, 220 belts.

Table 5.6.: The optimal plan for the first six months.

t	1	2	3	4	5	6
d_t	40	60	120	300	420	260
Q_t	220	0	0	300	680	0
I_t	180	120	0	0	260	0

Three orders cost \$900 and the ending balances of 560 belt-months cost \$560, for \$1,460. Notice that the two checks agree, which they must: the traceback and the recursion are two readings of the same numbers, and if the ledger does not reproduce $V(N)$ then one of them was done wrong.

Example 5.4 (The full twelve months)**.** The same recursion over all twelve months of Table 5.1 gives

Table 5.7.: The recursion over the full horizon.

u	1	2	3	4	5	6	7	8	9	10	11	12
$V(u)$	300	360	600	900	1,200	1,460	1,620	1,700	1,700	2,000	2,180	2,480
$S(u)$	1	1	1	4	5	5	6	6	6	10	10	12

The optimum is \$2,480. Tracing back from $u = 12$ gives orders in months 12, 10, 6, 5, 4 and 1, so the plan is

Table 5.8.: The optimal plan for the year.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220

t	1	2	3	4	5	6	7	8	9	10	11	12
Q_t	220	0	0	300	420	420	0	0	0	260	0	220
I_t	180	120	0	0	0	160	40	0	0	180	0	0

Six orders cost $1,800 and 680 belt-months of stock cost $680. Notice that $V(9) = V(8)$. Month 9 has no requirement, so covering through month 9 costs exactly what covering through month 8 costs, and the recursion is indifferent to where the next order falls. Notice also that month 5 holds an order, as Section 5.3.3 promised it would.

Against the three plans of Table 5.3, the optimum saves $820 on lot-for-lot and $120 on the adjusted economic order quantity, which was within 4.84% of optimal. Thus, there is a real question about how much the rest of this chapter buys, and Section 5.6 takes it up directly.

5.4.3. The Same Problem for Solver

The recursion is the right way to solve this problem. It is not the only way, and we should see the alternative, because it generalizes where the recursion does not.

Written as a mathematical program, we have

$$\begin{aligned} \min \quad & \sum_{t=1}^{N} k_t y_t + \sum_{t=1}^{N} c_t Q_t + \sum_{t=1}^{N} h_t I_t & & \\ \text{subject to} \quad & I_t = I_{t-1} + Q_t - d_t, & & t = 1, \ldots, N \\ & Q_t \le M y_t, & & t = 1, \ldots, N \\ & I_0 = 0, \quad I_t \ge 0, \quad Q_t \ge 0, & & t = 1, \ldots, N \\ & y_t \in \{0, 1\}, & & t = 1, \ldots, N. \end{aligned} \tag{5.9}$$

Everything here is linear. That is not obvious, because the cost of ordering is not a linear function of the order quantity. It is zero at zero and jumps

to k_t the moment anything is ordered. A fixed charge of that kind cannot be written as a linear function of Q_t alone.

The device is the binary y_t and the constraint $Q_t \le M y_t$. If $y_t = 0$ then Q_t is forced to zero, and if $y_t = 1$ then Q_t may be anything up to M. Thus, the setup is charged exactly when something is ordered, and the objective is linear in the pair (Q_t, y_t). The constant M needs only to be large enough never to bind, and $M = d[1, N]$ serves, since no order in an optimal plan covers more than the whole horizon.

Take care with M, and keep it as small as is safe. A needlessly large M gives a weaker linear relaxation, and the branch and bound search that a solver runs will take longer for no benefit.

The formulation is in the `Solver` sheet of the workbook, where the problem is small enough for Excel's Solver to reach \$2,480 from any starting point.

Two reasons to know the formulation. The first is that it accepts constraints the recursion cannot. A limit on the order quantity, a shared capacity across several items, a minimum order size: none of these fit Equation 5.8, and all of them are one more row in Equation 5.9. The second is that Equation 5.9 is how the problem is actually solved inside commercial planning systems that must handle the extra constraints.

5.4.4. How Much Work It Is

The recursion computes $N(N+1)/2$ window costs and then takes $N(N+1)/2$ minima, so it runs in $O(N^2)$ time. For a twelve-month horizon that is 78 windows, about a page of arithmetic. For a 52-week horizon it is 1,378, nothing at all for a computer and out of the question by hand.

$O(N^2)$ was long thought to be the best possible. It is not. Federgruen and Tzur (1991) showed that the problem can be solved in $O(N \log N)$ time, and in $O(N)$ time in the constant-cost case, by exploiting the structure of the window costs so that most of the candidate minima never have to

be evaluated. The practical consequence is small, since N is rarely large enough for the difference to matter, but it settles a question that stood for thirty years.

The reason lot sizing is still done with heuristics has nothing to do with running time, and Section 5.5 takes it up next.

5.5. Three Heuristics and One Shape

An optimal algorithm exists, it is not slow, and it fits on half a page. So why is almost every planning system in the field running something else?

The answer is Section 5.1.2, assumption 2. The recursion needs d_1 through d_N before it can compute $V(N)$, because $V(N)$ depends on every window cost, and every window cost depends on the requirements out to period N. Those requirements are a forecast. The decision that actually gets made is the one for period 1, and it has been made to depend on a number twelve periods out that nobody believes.

A heuristic that looks only a few periods ahead makes the period 1 decision from data the planner can believe. That is the case for heuristics here, and it is a different case from the usual one. It is not an argument about running time.

5.5.1. One Procedure

The three heuristics in common use are one procedure. We start an order in period 1 and extend the window one period at a time. At each step we compute a figure of merit for the window so far, and we stop when it stops improving. Whatever period the window reached, the next order starts in the one after it, and we begin again.

The three rules differ only in the figure of merit.

Table 5.9.: Three stopping rules on one procedure.

Rule	Figure of merit for the window $[t, u]$	Stop when
Silver-Meal	$w(t,u)/(u-t+1)$, cost per period	it rises
Least unit cost	$w(t,u)/d[t,u]$, cost per unit	it rises
Part-period balancing	$w(t,u) - k_t$, what the window carries	it passes k_t

All three use the window costs of Section 5.3.2, which were computed once and are the same numbers the recursion used. That is what makes the comparison in Section 5.6 a comparison and not an accident of arithmetic.

Silver-Meal minimizes cost per period. The reasoning is an appeal to Section 3.5.4, where the quantity that minimizes cost per unit time is optimal because every cycle is identical. Here the cycles are not identical, so minimizing cost per period over one window is a local imitation of that argument and not a consequence of it.

Least unit cost minimizes cost per unit instead. It is the older rule and the more intuitive one, since a buyer thinks in dollars per piece. It tends to order larger quantities than Silver-Meal, because adding a period to the window always adds units to the denominator. That is, a long window is flattered by the arithmetic.

Part-period balancing looks different and is not. In Section 3.5.4 the optimum sets the setup cost equal to the holding cost, so this rule extends the window until the carrying it has accumulated is as close as it can get to k_t. Two versions are in circulation. The original, due to DeMatteis and Mendoza and given in Silver et al. (2016), takes the largest window whose carrying is at or under k_t and so never overshoots. The modification in Nahmias and Olsen (2015), which is the one used here

and in Table 5.9, compares the two windows either side of k_t and keeps whichever comes closer. Exercise 5.12 works both. Dividing through by h turns the accumulated carrying into a count of unit-periods, or *part-periods*, and that is where the name comes from. For the belt, $k/h = 300$, so the rule extends until the window carries about 300 belt-months.

One difference in the last rule matters when it is implemented. Silver-Meal and least unit cost walk a number to a minimum and keep the period before the first rise. Part-period balancing walks a number past a target, and then keeps whichever of the last two periods came closer. It can step back.

Example 5.5 (The three rules on the belt)**.** Start all three in month 1 and extend. The window costs come from Equation 5.7.

Table 5.10.: The first window under each of the three rules.

u	$w(1,u)$	Silver-Meal, $w/(u)$	Least unit cost, $w/d[1,u]$	Part-period, $w-k$
1	300	300.00	7.500	0
2	360	**180.00**	3.600	60
3	600	200.00	**2.727**	**300**
4	1,500	375.00	2.885	1,200

Silver-Meal stops at $u = 2$, because the cost per period rises from \$180.00 to \$200.00 when month 3 is added. It orders 100 belts.

Least unit cost keeps going to $u = 3$, where \$2.727 a belt is the least, and rises to \$2.885 at $u = 4$. It orders 220 belts.

Part-period balancing extends until the carrying passes $k =$ \$300. At $u = 3$ it carries exactly \$300. Nothing can come closer, so it stops there and also orders 220 belts.

Continuing each rule to the end of the horizon gives the three plans in Table 5.11.

Table 5.11.: The three heuristic plans and the optimum.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220
Silver-Meal	100	0	420	0	840	0	0	0	0	260	0	220
Least unit cost	220	0	0	720	0	380	0	300	0	0	0	220
Part-period balancing	220	0	0	720	0	420	0	0	0	260	0	220
Optimal	220	0	0	300	420	420	0	0	0	260	0	220

All three place five orders and the optimum places six. That is the first surprise. The heuristics are not ordering too often. They are ordering too seldom, and the carrying that buys is what costs them.

The 840 in month 5 is where Silver-Meal loses its money. Extending from month 5, the cost per period runs \$300.00, \$280.00, \$266.67, \$230.00, \$184.00, and only then rises to \$220.00 at month 10. It never rises through the whole run, so the rule never stops. Months 6 through 8 have falling requirements and month 9 has none at all, so each additional period adds almost nothing to the numerator while adding a full period to the denominator. The average keeps falling because the window is getting longer, not because the plan is getting better. That is the failure mode of

a cost-per-period rule. Notice that it is triggered by a declining tail and a dead period, the shape of a seasonal item going into its off season.

5.6. What a Heuristic Costs

Table 5.12 collects every method in this chapter on the belt.

Table 5.12.: Every method of this chapter on the mower deck belt.

Method	Orders	Setup	Carrying	Relevant cost	Penalty
Lot-for-lot	11	\$3,300	\$0	\$3,300	+33.06%
Least unit cost	5	\$1,500	\$1,540	\$3,040	+22.58%
Periodic order quantity, $T = 2$	6	\$1,800	\$960	\$2,760	+11.29%
Silver-Meal	5	\$1,500	\$1,160	\$2,660	+7.26%
Adjusted economic order quantity	6	\$1,800	\$800	\$2,600	+4.84%

Method	Orders	Setup	Carrying	Relevant cost	Penalty
Part-period balancing	5	\$1,500	\$1,100	\$2,600	+4.84%
Wagner-Whitin	6	\$1,800	\$680	\$2,480	optimal

Read the table carefully, because it does not say what a table like this is usually taken to say. Silver-Meal, which the literature reports as the best of the three heuristics on average, is beaten here by part-period balancing and also by an adjusted economic order quantity computed from the average requirement. One instance settles nothing. What it does show is how far apart the methods can land on a single item: part-period balancing gives away 4.84% and least unit cost 22.58%, so the choice among the three heuristics is worth almost eighteen percentage points here, and lot-for-lot is worse than any of them at 33.06%.

5.6.1. When None of This Is Worth Doing

If the requirements barely vary, then Chapter 3 already has the answer and this chapter is an elaborate way of reproducing it. The question we need answered is how much variation is enough, and Silver et al. (2016) gives a test.

The *variability coefficient* is the variance of the requirements divided by the square of their mean:

$$VC = \frac{\frac{1}{N}\sum_{t=1}^{N}\left(d_t - \bar{d}\right)^2}{\bar{d}^2} \quad \text{where} \quad \bar{d} = \frac{1}{N}\sum_{t=1}^{N} d_t. \tag{5.10}$$

Dividing by $\bar{d}^2$ makes it dimensionless, so it does not matter whether the item is counted in belts or in pallets. A perfectly level requirement gives $VC = 0$.

Thus, the rule of thumb is that $VC < 0.2$ calls for an economic order quantity and $VC \geq 0.2$ calls for one of the methods of this chapter. For the belt, $\bar{d} = 153.33$ and the variance is 14,555.6, so

$$VC = \frac{14{,}555.6}{153.33^2} = 0.619,$$

well past the threshold. Notice that the threshold is a rule of thumb and not a theorem. Table 5.12 shows the adjusted economic order quantity within 4.84% of optimal at $VC = 0.619$, which is better than the rule would lead you to expect.

5.6.2. Sensitivity to the Cost Ratio

Section 3.7 showed that the economic order quantity is forgiving: getting k/h wrong by a factor of two costs about 6% of the relevant cost. We can ask the same question here, and the answer has the same shape.

Solve the belt with a ratio k/h that is wrong by a factor m, then charge the resulting plan at the true costs. Table 5.13 gives the result.

Table 5.13.: The cost of solving the belt with the wrong ratio k/h.

m	0.25	0.5	0.75	1	1.5	2	4
Orders	9	9	7	6	4	4	3
Cost at true costs	\$2,800	\$2,800	\$2,540	\$2,480	\$2,660	\$2,660	\$3,280

m	0.25	0.5	0.75	1	1.5	2	4
Penalty	12.90%	12.90%	2.42%	0%	7.26%	7.26%	32.26%

A factor of two in either direction costs less than 13%. Thus, the flatness Section 3.7 found survives the move to discrete periods. The curve is not smooth here, because the number of orders is an integer and the plan does not change at all over a range of m. Doubling and halving are not symmetric either: at $m = 2$ the plan has four orders and costs 7.26% extra, and at $m = 0.5$ it has nine and costs 12.90%. Ordering too often is the more expensive mistake on this item, since every extra order costs a full \$300 while the carrying it saves is spread thin.

Put Table 5.13 beside Table 5.12 and something follows. The penalty for using Silver-Meal instead of the optimum is 7.26%. The penalty for getting the cost ratio wrong by a factor of two is also 7.26%. An organization that has not measured k to within a factor of two, and most have not, is not in a position to care which method it uses.

5.6.3. What the Aggregate Studies Say

One instance is one instance. Silver et al. (2016) reports comparisons over many generated problems, and the findings that hold up are these. All three heuristics land within a few percent of the optimum on average. Silver-Meal is usually the best of them. Least unit cost is usually the worst, for the reason Section 5.5 gave: the denominator grows with the window whether or not the window is a good one. Lot-for-lot is not competitive unless ordering is nearly free, and an economic order quantity adjusted to whole periods does better than its reputation whenever VC is small.

The averages hide a wide spread. Silver-Meal is best on average and can lose seven percent on a single item, as it does here. A heuristic is a claim about a population of problems, not about the problem in front of you.

Section 5.7 asks what happens when the problem in front of you keeps changing.

5.7. The Rolling Horizon

Everything so far has assumed a horizon that ends. The belt's year is planned, the plan is executed, and the problem is over. No item is planned that way.

What actually happens is that the planner solves a problem over the next N periods, places only the order for the current period, waits a period, and solves again over a horizon that has moved forward one period and acquired a new last period. The plan for periods 2 through N was never executed. It existed to inform the decision for period 1.

Two things follow immediately. The first is that only the first decision matters. Thus, the quality of a method is the quality of the sequence of first decisions it produces, and not the quality of the plans it produces. The second is that Wagner-Whitin is no longer optimal. It is optimal for the problem it was handed, and the problem it was handed is a truncation of the real one.

5.7.1. What the Truncation Costs

Let's run the belt through a rolling horizon. At each month the planner solves over the next W months, places that month's order, and moves on. The realized cost of the twelve months of orders that actually get placed is Table 5.14.

Table 5.14.: What the belt actually costs when the horizon rolls, one month committed at a time. The full-horizon optimum is \$2,480.

Planning window W	2	3	4	5	6
Silver-Meal	\$2,640	\$2,840	\$2,660	\$2,660	\$2,660
Wagner-Whitin	\$2,640	\$2,480	\$2,480	\$2,480	\$2,480

Wagner-Whitin reaches the full-horizon optimum at a window of three months and stays there. That is Section 5.3.3 doing its work. Month 5 forces an order, so the decision for month 4 cannot be improved by anything beyond month 5, and the three-month window already sees far enough.

Silver-Meal never reaches it, and its cost is not even monotone in the window. A three-month window costs it \$200 more than a two-month window does. Let's understand that instead of explaining it away. Lengthening the window gives the rule more periods to average over, and averaging over more periods is precisely what drives its cost-per-period down and its windows too long. A longer horizon is not always better information for a rule that was not built to use it.

5.7.2. Does the Heuristic Ever Win?

It is often said that under a rolling horizon a heuristic can beat the optimal algorithm, since neither is optimal for the real problem and the heuristic's myopia is no longer a handicap. The claim is true as stated. It is easy to read it as a claim about how the two typically compare. That is a different claim.

We can settle it by measurement. Generate 2,000 twelve-period schedules with requirements drawn at random from the eight values 0, 10, 20, 40, 80, 120, 200 and 300 units, with a non-zero first period, $k = \$200$, $c = \$50$ and $i = 0.02$ per period. Then we roll each one with a window of W periods, committing one period at a time, under both rules.

Table 5.15.: Silver-Meal against Wagner-Whitin over 2,000 random schedules under a rolling horizon.

Window W	Silver-Meal cheaper	Wagner-Whitin cheaper	Tied	Mean, Silver-Meal	Mean, Wagner-Whitin
3	360	704	936	\$1,440.91	\$1,429.90
4	246	886	868	\$1,395.74	\$1,375.08
6	71	1,062	867	\$1,383.20	\$1,350.14

Silver-Meal does win, and on a three-period window it wins 360 times out of 2,000. Thus, the claim is not empty. What the table adds is the rest of the picture. The two rules tie about 45% of the time, because on many schedules they make the same first decision every period. When they differ, the algorithm is ahead about two to one at a three-period window and about fifteen to one at six. On average the algorithm is cheaper at every window length, by 0.8% to 2.4%.

Thus, the rolling horizon does not rescue the heuristic. What it does is remove the guarantee, which turns a proof into an empirical question, and the empirical answer on these instances still favors the algorithm.

5.7.3. Nervousness

There is a cost the tables do not show. Each period the plan is recomputed, and the new plan may disagree with the old one about orders that have not been placed yet. Suppliers were told to expect 300 belts in month 7, and now the plan says 0. That instability is called *nervousness*, and it is expensive in ways that do not appear in Equation 5.3.

Part of it comes from the method. Table 5.14's first decision is instructive. Silver-Meal orders 100 belts in month 1 at every window length from two to six. Wagner-Whitin orders 100 at a two-month window and 220 at every

longer one. The heuristic is steadier here, because it never looks far enough ahead for a distant period to change its mind.

Three devices are in common use against nervousness, and each of them buys stability at a price.

A *freeze fence* fixes the plan for the first several periods and forbids the next solve from changing them. It converts instability into inflexibility, and that is sometimes the better problem. Committing F periods at a time instead of one is the same idea.

A *time fence* is softer. Inside it, changes need approval; outside it, the plan may move freely.

A *change penalty* adds a cost to the objective for every order that differs from the previous plan. That is, the previous plan becomes part of the data. It is the most direct treatment and the least used, for that reason.

Chapter 6 returns to all three, because nervousness at one level of a bill of material propagates downward and multiplies.

5.8. Building These Models in a Worksheet

Section 4.7 built a worksheet for each model of the previous chapter, and the layouts followed the shape of the mathematics. Here they follow the shape of the horizon. Every quantity in this chapter is indexed by period, so the periods run down the rows of one block and every sheet reads that block by name.

The workbook is `Chapter5Models.xlsx`. It has five working sheets, and the sections below take them in the order the chapter introduced the material. As in Section 4.7, blue cells are inputs and gray cells are computed, every sheet carries a check block giving what the shipped data should produce, and every sheet carries a text box to the right of its widest block saying what the sheet is for and what to watch. A third color appears here that

Section 4.7 did not need. Yellow marks a cell a method picked: a period an order starts in, or a period a stopping rule chose.

5.8.1. The Data Sheet

The requirements run down column B, one row per period, and the three cost columns beside them point at the input cells above. Writing them as columns instead of as three scalars costs nothing when the costs are constant and buys the period-varying case for free: type \$54 over one cell in the unit cost column and every window cost in the workbook accounts for the price rise.

Five named ranges carry the schedule to the rest of the workbook: `Dls_t`, `Dls_d`, `Dls_k`, `Dls_c` and `Dls_h`. A sub-range of one of them is written `INDEX(Dls_d,a):INDEX(Dls_d,b)`. That is the requirement of periods a through b, and it is Equation 5.7's inner sum in one expression.

Two helper columns sit to the right. One holds $(d_t - \bar{d})^2$, which totals to the numerator of Equation 5.10, and one holds the period number when $d_t h > k$ and nothing otherwise, so that the smallest number in it is Section 5.3.3's forced period. Both could be done with an array formula in a single cell. Both are done as columns because the reader can then see which period did it. They are tinted, and a rule down the right of the holding rate column says where the schedule stops and the figures derived from it start.

Figure 5.2 illustrates the top of the sheet. The three inputs in rows 5 through 7 are the only typed numbers on it, and everything below them follows. Notice that every figure the check block in rows 25 through 27 promises is on the screen above it: 1,840 belts, an average of 153.3333, a variability coefficient of 0.6191, $k/h = 300$, a first forced month of 5, and an economic order quantity of 303.3 belts that is a time supply of 1.98 months. Reading a check block against the sheet it sits on is the first thing to do with any workbook, and it takes ten seconds.

	A	B	C
1	**The item and its requirements**		
2	*Chapter 5, section 5.1.*		
3			
4	**INPUTS**		
5	Order cost, k	$300.00	*dollars per order*
6	Unit cost, c	$50.00	*dollars per belt*
7	Carrying charge, i	0.020	*per dollar per month*
8	Holding rate, h = ic	$1.00	
9	Periods in the horizon, N	12	
10			
11	**WHAT THE PATTERN LOOKS LIKE**		
12	Total requirement	1,840	
13	Average requirement per month	153.3333	
14	Variability coefficient	0.6191	
15	Use a heuristic?	yes, the pattern varies enough	
16	k / h, the carrying a single order buys	300	
17	First month whose requirement forces an order	5	
18	*Section 5.3: a requirement larger than k/h cannot be carried in from any earlier month.*		
19			
20	**THE ECONOMIC ORDER QUANTITY, FOR COMPARISON**		
21	EOQ on the average requirement	303.3	
22	As a time supply, months	1.98	
23	Rounded to whole months	2	
24			
25	**CHECK with the data as shipped**		
26	*Total 1,840 belts; average 153.3333; variability coefficient 0.6191; k/h = 300;*		
27	*the first forced month is 5; EOQ 303.3 belts, 1.98 months, rounding to 2.*		

Figure 5.2.: The `Data` sheet, rows 1 to 27. Three inputs, the figures derived from them, the economic order quantity of Chapter 3 for comparison, and the check block. Notice cell B15, which reads the variability coefficient against the threshold of Section 5.6.1 and answers the question in words rather than leaving the reader to compare two numbers.

The requirements themselves are Figure 5.3. Column B is the only blue block on the sheet, and columns C, D and E hold one cost per period.

	A	B	C	D	E	F	G
29	REQUIREMENTS type over column B to change the pattern						
30	Month t	Requirement d(t)	Order cost k(t)	Unit cost c(t)	Holding rate h(t)	(d(t) - mean)^2	Forced?
31	1	40	$300.00	$50.00	$1.00	12,844.4	
32	2	60	$300.00	$50.00	$1.00	8,711.1	
33	3	120	$300.00	$50.00	$1.00	1,111.1	
34	4	300	$300.00	$50.00	$1.00	21,511.1	
35	5	420	$300.00	$50.00	$1.00	71,111.1	5
36	6	260	$300.00	$50.00	$1.00	11,377.8	
37	7	120	$300.00	$50.00	$1.00	1,111.1	
38	8	40	$300.00	$50.00	$1.00	12,844.4	
39	9	0	$300.00	$50.00	$1.00	23,511.1	
40	10	80	$300.00	$50.00	$1.00	5,377.8	
41	11	180	$300.00	$50.00	$1.00	711.1	
42	12	220	$300.00	$50.00	$1.00	4,444.4	
43							
44	*The cost columns point at the inputs above, so a constant-cost problem is typed once.*						
45	*Type a number over one of them to make that month's cost differ, which is the case of Section 5.2.*						
46	*Columns F and G feed the variability coefficient and the forced month. They are not data.*						

Figure 5.3.: The `Data` sheet, rows 29 to 46, the requirements schedule. Columns C, D and E point at the input cells above, so a constant-cost problem is typed once and a period-varying one needs no second workbook. Columns F and G are tinted and stand to the right of a rule, because they are derived and not data. Notice that column G is empty except in month 5, where 420 belts exceed $k/h = 300$ and Section 5.3.3 forces an order.

5.8.2. Costing a Plan

The `Plans` sheet is the ledger of Example 5.1, made general. Four plans sit side by side, each with two columns: an order column the reader types into and an ending-inventory column that computes Equation 5.1 as `= previous end + order - requirement`.

The results block above them counts the orders with `COUNTIF` on the order column, charges carrying with `SUMPRODUCT(Dls_h, end column)`, and reports the penalty against the optimum on the next sheet. Four plans are shipped: lot-for-lot, the fixed economic order quantity, the periodic rule at $T = 2$, and a fourth to type over.

A negative number in an ending-inventory column is a shortage, which Section 5.1.2 forbids. Nothing on the sheet prevents one from appearing. The sheet evaluates whatever plan is typed into it, including infeasible ones, and it is the reader's job to look.

Figure 5.4 is the whole sheet. Read a plan by reading one pair of columns downward and then the block above it.

5.8.3. The Recursion and the Traceback

The `WagnerWhitin` sheet has four blocks, and the first three match the three blocks of Algorithm 5.1. Nothing on it is typed.

Figure 5.5 is what the sheet answers, and it is the block to read first.

The window costs occupy an N by N upper triangle. Rather than write Equation 5.7 out in each cell, each cell adds one period to the cell on its left:

$$w(t,u) = w(t,u-1) + d_u \sum_{r=t}^{u-1} h_r,$$

with $w(t,t) = k_t$ on the diagonal. That is one short formula, filled right and down, and it is also the increment Example 5.2 read off row 1. The triangle is Figure 5.6.

The recursion occupies a second triangle of the same shape, one entry for each t and u, holding $V(t-1) + w(t,u)$. Above it, $V(u)$ is the `MIN` down column u and $S(u)$ is `MATCH` of that minimum in the same column. Laying the candidates out instead of collapsing them into one formula per period is the point of the sheet. Table 5.5 is a table the reader is handed in the text, and Figure 5.7 is a range the reader can look at.

The traceback sits below them and is the only part that needs thought. It walks backward, so each step reads the pointer of the period before the one the previous step started in, and stops when a step starts in period 1. `INDEX` on the S row does the walk without a macro.

	A	B	C	D	E	F	G	H	I	J
1	**What a plan costs**									
2	*Chapter 5, section 5.2. Each plan owns an order column. Type over one and the results above it move.*									
3										
4	**RESULTS**									
5			**Lot-for-lot**		**Fixed EOQ**		**Periodic, T = 2**		**Your plan**	
6	Orders placed		11		6		6		11	
7	Setup cost		$3,300.00		$1,800.00		$1,800.00		$3,300.00	
8	Carrying cost		$0.00		$800.00		$960.00		$0.00	
9	Purchase cost		$92,000.00		$92,000.00		$92,000.00		$92,000.00	
10	Relevant cost		$3,300.00		$2,600.00		$2,760.00		$3,300.00	
11	Total cost		$95,300.00		$94,600.00		$94,760.00		$95,300.00	
12	Penalty against the optimum		33.06%		4.84%		11.29%		33.06%	
13										
14	**CHECK with the data as shipped**									
15	*Lot-for-lot: eleven orders, $3,300 of setup, nothing carried, 33.06% above the optimum.*									
16	*Fixed EOQ: six orders, $1,800 and $800, for $2,600 and 4.84%. Periodic: $1,800 and $960, for $2,760 and 11.29%.*									
17	**THE LEDGER one row per month**									
18	**Month**	**Requirement**	**Order**	**End**	**Order**	**End**	**Order**	**End**	**Order**	**End**
19	1	40	40	0	220	180	100	60	40	0
20	2	60	60	0	0	120	0	0	60	0
21	3	120	120	0	0	0	420	300	120	0
22	4	300	300	0	300	0	0	0	300	0
23	5	420	420	0	420	0	680	260	420	0
24	6	260	260	0	260	0	0	0	260	0
25	7	120	120	0	240	120	160	40	120	0
26	8	40	40	0	0	80	0	0	40	0
27	9	0	0	0	0	80	80	80	0	0
28	10	80	80	0	0	0	0	0	80	0
29	11	180	180	0	400	220	400	220	180	0
30	12	220	220	0	0	0	0	0	220	0
31	*End = previous end + order - requirement. A negative end is a shortage, which section 5.1 forbids.*									
32	*Carrying cost charges the end-of-month balance, so a plan that orders only when empty carries nothing.*									

Figure 5.4.: The `Plans` sheet, rows 1 to 32. Four plans sit side by side, each owning an order column and an ending-inventory column, with a rule between the pairs. The three shipped plans reproduce the three rows of Table 5.3 exactly, so the sheet can be checked against the chapter before it is trusted with a plan of your own. Notice the ending-inventory column under lot-for-lot: it is zero in every month, which is what "carries nothing" means when you look at it rather than read it. Notice also that the purchase cost is $92,000 under all four plans, which is Section 5.2.1 made visible.

	A	B	C
1	**The optimal plan**		
2	*Chapter 5, section 5.4. Nothing here is typed; everything reads the Data sheet.*		
3			
4	**RESULTS**		
5	Optimal relevant cost, V(N)	$2,480.00	
6	Orders placed	6	
7	Setup cost	$1,800.00	
8	Carrying cost	$680.00	
9	Setup plus carrying, which must equal V(N)	$2,480.00	
10			
11	**CHECK with the data as shipped**		
12	*V(12) = $2,480 in six orders: $1,800 of setup and $680 of carrying.*		
13	*The orders fall in months 1, 4, 5, 6, 10 and 12, for 220, 300, 420, 420, 260 and 220 belts.*		

Figure 5.5.: The `WagnerWhitin` sheet, rows 1 to 13. Nothing here is typed. Row 9 exists to be compared with row 5: the setup and carrying of the plan the sheet traced out must add to $V(N)$, because the recursion and the ledger are two readings of the same numbers, and a sheet that has been edited will usually break that first.

The fourth block converts the chain into an order quantity per period, with `COUNTIF` to ask whether this period starts an order and `INDEX(Dls_d,a):INDEX(Dls_d,b)` to total what it covers. Figure 5.8 is that block, and its two totals are the arithmetic check the sheet is built around.

5.8.4. The Three Rules

The `Heuristics` sheet illustrates Section 5.5.1's claim. Every rule on it reads the same window costs from the sheet before it, so the three answers in Figure 5.9 differ because the rules differ and for no other reason.

Each rule gets a triangle of scores over the same (t, u) pairs, and beside it a triangle of zeros and ones answering "does the rule stop here?" The stop period for an order placed in period t is then `MATCH(1, ...)` along row t, the first period whose answer is one. Figure 5.10 is Silver-Meal's scores and Figure 5.11 is the test that reads them.

	A	B	C	D	E	F	G	H	I	J	K	L	M
14	WINDOW COSTS w(t,u)		order in month t, cover through month u										
15	t \ u	1	2	3	4	5	6	7	8	9	10	11	12
16	1	$300	$360	$600	$1,500	$3,180	$4,480	$5,200	$5,480	$5,480	$6,200	$8,000	$10,420
17	2		$300	$420	$1,020	$2,280	$3,320	$3,920	$4,160	$4,160	$4,800	$6,420	$8,620
18	3			$300	$600	$1,440	$2,220	$2,700	$2,900	$2,900	$3,460	$4,900	$6,880
19	4				$300	$720	$1,240	$1,600	$1,760	$1,760	$2,240	$3,500	$5,260
20	5					$300	$560	$800	$920	$920	$1,320	$2,400	$3,940
21	6						$300	$420	$500	$500	$820	$1,720	$3,040
22	7							$300	$340	$340	$580	$1,300	$2,400
23	8								$300	$300	$460	$1,000	$1,880
24	9									$300	$380	$740	$1,400
25	10										$300	$480	$920
26	11											$300	$520
27	12												$300
28	*Each cell adds one month to the cell on its left: the new requirement, carried from the order month.*												

Figure 5.6.: The window costs, rows 14 to 28. This is Table 5.4 extended to twelve periods. Read row 1 across and the first six entries are the table the text gives. Notice columns 8 and 9: they are equal in every row, because month 9 requires nothing, so a window that reaches it costs what one ending at month 8 costs. Notice also how fast the upper right grows. Covering the whole year from month 1 costs $10,420 against $300 for a single month, and that ratio is the whole reason an optimal plan orders more than once.

	A	B	C	D	E	F	G	H	I	J	K	L	M
29	**THE RECURSION** V(u) = the least of V(t-1) + w(t,u) over the months t that could start the last order												
30													
31	**month u**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
32	**V(u), the least cost through month u**	$300	$360	$600	$900	$1,200	$1,460	$1,620	$1,700	$1,700	$2,000	$2,180	$2,480
33	**S(u), the month that last order starts in**	1	1	1	4	5	5	6	6	6	10	10	12
34	*MATCH returns the first month achieving the least cost, which breaks a tie toward the earlier order.*												
35													
36	**candidate V(t-1) + w(t,u)**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
37	**last order starts in month 1**	$300	$360	$600	$1,500	$3,180	$4,480	$5,200	$5,480	$5,480	$6,200	$8,000	$10,420
38	**last order starts in month 2**		$600	$720	$1,320	$2,580	$3,620	$4,220	$4,460	$4,460	$5,100	$6,720	$8,920
39	**last order starts in month 3**			$660	$960	$1,800	$2,580	$3,060	$3,260	$3,260	$3,820	$5,260	$7,240
40	**last order starts in month 4**				$900	$1,320	$1,840	$2,200	$2,360	$2,360	$2,840	$4,100	$5,860
41	**last order starts in month 5**					$1,200	$1,460	$1,700	$1,820	$1,820	$2,220	$3,300	$4,840
42	**last order starts in month 6**						$1,500	$1,620	$1,700	$1,700	$2,020	$2,920	$4,240
43	**last order starts in month 7**							$1,760	$1,800	$1,800	$2,040	$2,760	$3,860
44	**last order starts in month 8**								$1,920	$1,920	$2,080	$2,620	$3,500
45	**last order starts in month 9**									$2,000	$2,080	$2,440	$3,100
46	**last order starts in month 10**										$2,000	$2,180	$2,620
47	**last order starts in month 11**											$2,300	$2,520
48	**last order starts in month 12**												$2,480
49	*A blank means the window would have to end before it began.*												
50	**THE TRACEBACK** walk the pointers backwards from month N												
51	**Step**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
52	**Order starts in month**	12	10	6	5	4	1						
53	**Covers through month**	12	11	9	5	4	3						
54	*Step 1 is the last order. Each step reads the pointer of the month before the one it starts.*												

Figure 5.7.: The recursion and the traceback, rows 29 to 54. The V and S rows are Table 5.7 and sit above the candidate block they read. The candidate block is Table 5.5 with nothing collapsed: every entry is a $V(t-1)$ from an earlier column plus a window cost from the triangle above. Notice that $V(8)$ and $V(9)$ are both \$1,700 and that S is 6 at both, which is month 9 requiring nothing showing up a second time. The yellow row is the traceback, and reading it right to left gives orders in months 1, 4, 5, 6, 10 and 12.

	A	B	C	D	E	F
56	**THE PLAN one row per month**					
57	**Month**	**Requiremen**	**Order**	**End**	**Setup**	**Carrying**
58	1	40	220	180	$300	$180
59	2	60	0	120	$0	$120
60	3	120	0	0	$0	$0
61	4	300	300	0	$300	$0
62	5	420	420	0	$300	$0
63	6	260	420	160	$300	$160
64	7	120	0	40	$0	$40
65	8	40	0	0	$0	$0
66	9	0	0	0	$0	$0
67	10	80	260	180	$300	$180
68	11	180	0	0	$0	$0
69	12	220	220	0	$300	$0
70	**Total**				**$1,800**	**$680**
71	*These two totals are the setup and carrying reported at the top, and they add to V(N).*					

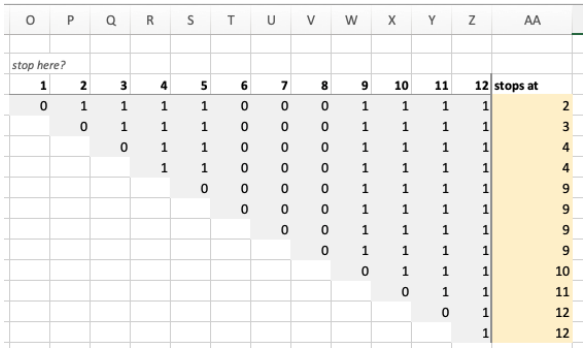
Figure 5.8.: The plan, rows 56 to 71. The chain above has become an order quantity per month, and the two totals are the $1,800 and $680 of Table 5.12. Notice month 6: it receives 420 belts and ends the month holding 160, which is the one place in the optimal plan where a month both orders and carries something forward.

	A	B	C	D	E
1	**Three heuristics, one procedure**				
2	*Chapter 5, section 5.5. Each rule extends a window from the order month until its own test says stop.*				
3					
4	**RESULTS**				
5		**Silver-Meal**	**Least unit co**	**Part-period balancing**	
6	Orders placed	5	5	5	
7	Relevant cost	$2,660.00	$3,040.00	$2,600.00	
8	Penalty against the optimum	7.26%	22.58%	4.84%	
9					
10	**CHECK with the data as shipped**				
11	*Silver-Meal $2,660, 7.26% above the optimum. Least unit cost $3,040, 22.58%.*				
12	*Part-period balancing $2,600, 4.84%. All three place five orders; the optimum places six.*				
13	*The optimum on the WagnerWhitin sheet is $2,480.*				

Figure 5.9.: The `Heuristics` sheet, rows 1 to 13. The three columns are the three heuristic rows of Table 5.12, computed on this sheet from the window costs of the last one.

	A	B	C	D	E	F	G	H	I	J	K	L	M
14	**SILVER-MEAL**	**the relevant cost of the window, divided by its length**											
15	*Extend while the cost per period falls. Stop at the first month that raises it.*												
16	**t \ u**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
17	**1**	300.00	180.00	200.00	375.00	636.00	746.67	742.86	685.00	608.89	620.00	727.27	868.33
18	**2**		300.00	210.00	340.00	570.00	664.00	653.33	594.29	520.00	533.33	642.00	783.64
19	**3**			300.00	300.00	480.00	555.00	540.00	483.33	414.29	432.50	544.44	688.00
20	**4**				300.00	360.00	413.33	400.00	352.00	293.33	320.00	437.50	584.44
21	**5**					300.00	280.00	266.67	230.00	184.00	220.00	342.86	492.50
22	**6**						300.00	210.00	166.67	125.00	164.00	286.67	434.29
23	**7**							300.00	170.00	113.33	145.00	260.00	400.00
24	**8**								300.00	150.00	153.33	250.00	376.00
25	**9**									300.00	190.00	246.67	350.00
26	**10**										300.00	240.00	306.67
27	**11**											300.00	260.00
28	**12**												300.00

Figure 5.10.: The Silver-Meal scores, rows 14 to 28. Every entry is a window cost from Figure 5.6 divided by the length of its window. Read row 1 across: \$300.00, \$180.00, then \$200.00. The rise at $u = 3$ is where the rule stops, and it is the first column of Table 5.10. Now read row 5, which is where the rule loses its money. The score falls from \$300.00 through \$280.00, \$266.67 and \$230.00 to \$184.00 at $u = 9$, and only then rises to \$220.00. Nothing in that run tells the rule to stop, so one order covers five months.



O	P	Q	R	S	T	U	V	W	X	Y	Z	AA
stop here?												
1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**stops at**
0	1	1	1	1	0	0	0	1	1	1	1	2
	0	1	1	1	0	0	0	1	1	1	1	3
		0	1	1	0	0	0	1	1	1	1	4
			1	1	0	0	0	1	1	1	1	4
				0	0	0	0	1	1	1	1	9
					0	0	0	1	1	1	1	9
						0	0	1	1	1	1	9
							0	1	1	1	1	9
								0	1	1	1	10
									0	1	1	11
										0	1	12
											1	12

Figure 5.11.: The stopping test beside the Silver-Meal scores, columns O through AA of the same rows. Each entry answers one question: does the score rise at the next period? The stop for an order placed in month t is the first one in row t, which `MATCH` finds, and the yellow column collects them. Notice that the column reads 2, 3, 4, 4, then 9 four times over. A rule that reaches month 9 from anywhere in months 5 through 8 stops in the same place, which is what a period requiring nothing does to a rule that divides by the length of the window.

Figure 5.12 is the same pair of blocks for least unit cost, and only the divisor has changed. That one change is enough to make the rule order 220 belts in month 1 where Silver-Meal orders 100.

	A	B	C	D	E	F	G	H	I	J	K	L	M
30	**LEAST UNIT COST**	**the relevant cost of the window, divided by the units it covers**											
31	*A window covering no units has no cost per unit, so it is left blank and stepped over.*												
32	**t \ u**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
33	**1**	7.50	3.60	2.73	2.88	3.38	3.73	3.94	4.03	4.03	4.31	4.94	5.66
34	**2**		5.00	2.33	2.13	2.53	2.86	3.06	3.15	3.15	3.43	4.06	4.79
35	**3**			2.50	1.43	1.71	2.02	2.21	2.30	2.30	2.58	3.22	3.95
36	**4**				1.00	1.00	1.27	1.45	1.54	1.54	1.84	2.50	3.25
37	**5**					0.71	0.82	1.00	1.10	1.10	1.43	2.18	2.98
38	**6**						1.15	1.11	1.19	1.19	1.64	2.53	3.38
39	**7**							2.50	2.13	2.13	2.42	3.10	3.75
40	**8**								7.50	7.50	3.83	3.33	3.62
41	**9**										4.75	2.85	2.92
42	**10**										3.75	1.85	1.92
43	**11**											1.67	1.30
44	**12**												1.36

Figure 5.12.: The least unit cost scores, rows 30 to 44. Every entry is the same window cost divided by the units the window covers instead of by its length. Row 1 runs 7.50, 3.60, 2.73, then 2.88, which is the second column of Table 5.10. Notice the blank on the diagonal at $t = u = 9$. That window covers no units at all, so it has no cost per unit, and the rule steps over it instead of dividing by zero. It is the only cell in the workbook that has to be guarded, and Section 5.9.5 shows the guard.

Part-period balancing needs a different test, and Figure 5.13 shows why. Its score is what the window carries, which only grows as the window grows, so the last period at or under k_t can be found with an ordinary ascending `MATCH` and no triangle of ones is needed. The two neighbors are then compared to see which comes closer. A score that can fall, as Silver-Meal's can, cannot be looked up that way.

Below the triangles, a chain row per rule starts at period 1 and reads each

	A	B	C	D	E	F	G	H	I	J	K	L	M
46	**PART-PERIOD BALANCING**			**what the window carries, against the order cost**									
47	*Carrying only grows as the window grows, so the last month at or under k is found by a lookup.*												
48	**t \ u**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
49	**1**	0.00	60.00	300.00	1,200.00	2,880.00	4,180.00	4,900.00	5,180.00	5,180.00	5,900.00	7,700.00	10,120.00
50	**2**		0.00	120.00	720.00	1,980.00	3,020.00	3,620.00	3,860.00	3,860.00	4,500.00	6,120.00	8,320.00
51	**3**			0.00	300.00	1,140.00	1,920.00	2,400.00	2,600.00	2,600.00	3,160.00	4,600.00	6,580.00
52	**4**				0.00	420.00	940.00	1,300.00	1,460.00	1,460.00	1,940.00	3,200.00	4,960.00
53	**5**					0.00	260.00	500.00	620.00	620.00	1,020.00	2,100.00	3,640.00
54	**6**						0.00	120.00	200.00	200.00	520.00	1,420.00	2,740.00
55	**7**							0.00	40.00	40.00	280.00	1,000.00	2,100.00
56	**8**								0.00	0.00	160.00	700.00	1,580.00
57	**9**									0.00	80.00	440.00	1,100.00
58	**10**										0.00	180.00	620.00
59	**11**											0.00	220.00
60	**12**												0.00

Figure 5.13.: The part-period balancing scores, rows 46 to 60. Here the entry is what the window carries, which is the window cost less the order cost. Row 1 runs 0.00, 60.00, 300.00, then 1,200.00, and the rule is looking for the entry closest to $k = \$300$. It finds it exactly at $u = 3$. Notice that every row increases from left to right without exception, which is the property that lets this rule use an ordinary ascending lookup and needs no stopping test beside it.

order's start from the previous order's stop, and a ledger per rule turns the chain into a plan and a cost. Figure 5.14 is both, and the three relevant costs it totals are the three heuristic rows of Table 5.12.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
63	**THE CHAINS**	**the first order falls in month 1, and each order starts where the last one stopped**												
64	**Order**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	
65	**Silver-Meal**	1	3	5	10	12								
66	**Least unit cost**	1	4	6	8	12								
67	**Part-period balancing**	1	4	6	10	12								
68	*A blank means the previous order already reached month N, so the plan needs no further order.*													
69														
70	**THE PLANS**	**one row per month, one group of columns per rule**												
71			**Silver-Meal**				**Least unit cost**				**Part-period balancing**			
72	**Month**	**Requirement**	**Order**	**End**	**Setup**	**Carrying**	**Order**	**End**	**Setup**	**Carrying**	**Order**	**End**	**Setup**	**Carrying**
73	1	40	100	60	$300	$60	220	180	$300	$180	220	180	$300	$180
74	2	60	0	0	$0	$0	0	120	$0	$120	0	120	$0	$120
75	3	120	420	300	$300	$300	0	0	$0	$0	0	0	$0	$0
76	4	300	0	0	$0	$0	720	420	$300	$420	720	420	$300	$420
77	5	420	840	420	$300	$420	0	0	$0	$0	0	0	$0	$0
78	6	260	0	160	$0	$160	380	120	$300	$120	420	160	$300	$160
79	7	120	0	40	$0	$40	0	0	$0	$0	0	40	$0	$40
80	8	40	0	0	$0	$0	300	260	$300	$260	0	0	$0	$0
81	9	0	0	0	$0	$0	0	260	$0	$260	0	0	$0	$0
82	10	80	260	180	$300	$180	0	180	$0	$180	260	180	$300	$180
83	11	180	0	0	$0	$0	0	0	$0	$0	0	0	$0	$0
84	12	220	220	0	$300	$0	220	0	$300	$0	220	0	$300	$0
85	**Total**				**$1,500**	**$1,160**			**$1,500**	**$1,540**			**$1,500**	**$1,100**
86	*Setup plus carrying for each rule is the relevant cost reported at the top.*													

Figure 5.14.: The chains and the three plans, rows 63 to 86. Each chain starts in month 1 and each order after it starts where the previous one stopped; a blank means the horizon is already covered. Reading the three chains down, the rules order in months 1, 3, 5, 10, 12; in 1, 4, 6, 8, 12; and in 1, 4, 6, 10, 12. All three place five orders and no two agree on where. The three ledgers below are Table 5.11, and their totals are $1,500 of setup in every case against carrying of $1,160, $1,540 and $1,100. Notice the 840 in month 5 of the Silver-Meal ledger, which Figure 5.10 explained.

5.8.5. The Mixed Integer Program

The `Solver` sheet is Equation 5.9 laid out one row per period: the order quantity and the binary flag as changing cells, the balance and the two cost terms as formulas, and a column holding $Q_t - My_t$, which Solver constrains to be at most zero. The sheet ships holding the optimal plan, so the objective already reads \$2,480. Clear the two changing columns and Solver finds its way back. Figure 5.15 is the whole sheet, instructions and all.

5.8.6. What the Worksheet Cannot Do

The rolling horizon of Section 5.7 is not in the workbook, and it is not an oversight. Table 5.14 is the cost of a sequence of twelve plans, eleven of which were computed, discarded, and never executed. A worksheet holds one instance of a model. Producing Table 5.15 needs 2,000 schedules solved twelve times each under two rules, or 48,000 solves, and a worksheet that is asked to do that stops being a thing anyone can read.

That is the boundary. A worksheet is the right tool for a model you want to look at, and the wrong tool for a model you want to run many times. That is where Section 5.9 begins.

5.9. Designing the Software

Section 5.8 built the models as worksheets, where the structure is visible. A program has no such thing. Its structure is a set of decisions about which classes exist and what each one is responsible for, and none of those decisions leaves a mark in the finished code. So we make them first, in the order such an analysis proceeds: what the software must do, what the domain calls things, what each thing knows and does, and how they collaborate.

	A	B	C	D	E	F	G	H
1	**The same problem for Solver**							
2	*Chapter 5, section 5.4.3.*							
3								
4	**OBJECTIVE**							
5	Total relevant cc	$2,480.00						
6	Largest shortage	0						
7								
8	**HOW**							
9	*Data > Solver. Set objective B5 to a minimum by changing C14:D25.*							
10	*Subject to: D14:D25 = binary, C14:C25 >= 0, E14:E25 >= 0, H14:H25 <= 0*							
11	*Choose the Simplex LP engine. Section 5.4.3 explains why the model is linear in these variables.*							
12								
13	**Month**	**Requirement**	**Order Q(t)**	**Order? y(t)**	**End I(t)**	**Setup**	**Carrying**	**Q - My**
14	1	40	220	1	180	$300	$180	-1,620
15	2	60	0	0	120	$0	$120	0
16	3	120	0	0	0	$0	$0	0
17	4	300	300	1	0	$300	$0	-1,540
18	5	420	420	1	0	$300	$0	-1,420
19	6	260	420	1	160	$300	$160	-1,420
20	7	120	0	0	40	$0	$40	0
21	8	40	0	0	0	$0	$0	0
22	9	0	0	0	0	$0	$0	0
23	10	80	260	1	180	$300	$180	-1,580
24	11	180	0	0	0	$0	$0	0
25	12	220	220	1	0	$300	$0	-1,620
26								
27	*Column H is the linking constraint: an order can be positive only in a month whose flag is one.*							
28	*M is 1840, the whole horizon's requirement, which is the most any single order would ever carry.*							
29								
30	**CHECK with the data as shipped**							
31	*The sheet ships holding the optimal plan, so B5 already reads $2,480 and B6 reads zero.*							
32	*Clear C14:D25 to zero and Solver should find its way back to $2,480.*							

Figure 5.15.: The `Solver` sheet, rows 1 to 32, with the constraints written out above the model. Column C is Q_t and column D is y_t, and the two together are the changing cells. Column H is $Q_t - My_t$, which Solver holds at or below zero, and that single column is the whole of what makes Equation 5.9 linear. The sheet ships holding the optimal plan, so the objective in B5 reads $2,480 and the shortage in B6 reads zero. Notice column H in the months that order: it is a large negative number, meaning the linking constraint is slack and doing nothing. It binds only at zero, which is its job, and a reader who expects a constraint to be tight at the optimum should look at this column and ask why this one is not.

This chapter is a good one to design, because at first glance it is six unrelated recipes and at second glance it is one recipe with six settings. Getting that right in the code is what makes Table 5.12 a comparison.

5.9.1. What the Software Must Do

Reading back over the chapter, we find that the software has to

1. hold a requirements schedule for one item, with costs that may vary by period,
2. price any window $w(t, u)$,
3. evaluate a plan someone else chose, reporting setup, purchase and carrying separately,
4. produce a plan under lot-for-lot, an adjusted economic order quantity, a periodic order quantity, Silver-Meal, least unit cost, part-period balancing, and Wagner-Whitin,
5. report the penalty of one plan against another, and
6. roll a horizon forward, committing part of each plan and discarding the rest.

Requirement 6 is the one Section 5.8 could not meet, and it is the reason for writing a program at all.

5.9.2. Finding the Nouns

Underline the nouns in the requirements, and three survive.

A **requirements schedule** is what the planner is handed. It is the item's requirements and its costs over a horizon, and it is the only thing in the chapter that is data and not decision.

A **plan** is what the planner produces. It is an order quantity per period, and it knows what it costs.

A **rule** is how one becomes the other. Seven of them appear in this chapter, and Section 5.6 ranks them, which means the software has to be able to hold one without knowing which.

Two candidate nouns are instructive rejections.

Window looks like a class. It has a start, an end, a cost, and the chapter talks about it constantly. It fails because a window has no behavior of its own and no life of its own. It exists for the length of one comparison inside one rule, and $w(t, u)$ is a function of the schedule and two integers. Thus, making it a class would put $N^2/2$ objects on the heap to hold a number apiece.

Order also looks like a class, and it is a real thing in a warehouse. Here it is Q_t, a number in a position. Every question the chapter asks about an order is a question about a plan: how many are there, what do they cost, and when do they fall.

Thus, the schedule owns the window cost. That is the decision the rest of the design rests on. Every rule in Table 5.12 prices the same windows, so computing them in one place means the heuristics and the algorithm are compared on identical numbers and not on two implementations of Equation 5.7.

5.9.3. What Each Thing Knows and Does

For each of the three we write a card, listing what it is responsible for and whom it needs in order to meet that responsibility.

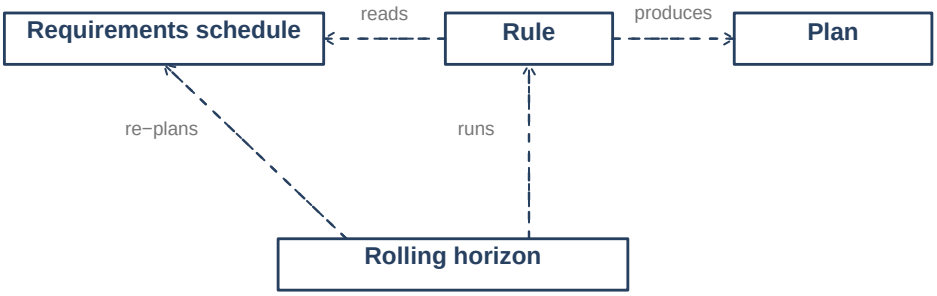


Figure 5.16.: The concepts of this chapter before any thought about implementation. A rule reads a requirements schedule and produces a plan. A rolling horizon runs a rule against one schedule repeatedly, keeping only part of each plan it gets back.

Table 5.16.: Responsibilities for the four classes.

Class	Knows and does	Collaborates with
`RequirementsSchedule`	the requirements and the three cost streams; the requirement over a span; the unit cost held from one period to another; the window cost; the variability coefficient; the forced order period	`LotSizingPlan`
`LotSizingPlan`	the order quantities; ending inventory; setup, purchase and carrying; whether it orders only when empty; its penalty against another plan	`RequirementsSchedule`

Class	Knows and does	Collaborates with
`LotSizingRule`	how far a replenishment should reach	`RequirementsSchedule`, `LotSizingPlan`
`RollingHorizon`	the true schedule, the planning window, the frozen span; what the committed orders actually cost	all three

Table 5.16 illustrates one thing by leaving it out. A plan does not know its ending inventory as stored data. Equation 5.1 determines the inventory from the orders, so storing both would allow a plan to exist whose inventory no sequence of orders produces. The plan stores the quantities and computes the rest.

Notice also that the plan validates itself. A plan that runs short in some period violates assumption 6, and the constructor refuses it. That is not defensive programming for its own sake. Thus, every plan anywhere in the program is feasible, and no method downstream has to ask.

5.9.4. The Schedule

```
class RequirementsSchedule(
    val label: String,
    requirements: List<Double>,
    orderCosts: List<Double>,
    unitCosts: List<Double>,
    holdingRates: List<Double>,
) {
    /** The number of periods in the planning horizon,
     ↪  written N in the text. */
```

```
    val horizon: Int get() = d.size

    /** The periods, 1 through N, as a range to iterate over.
     ↪  */
    val periods: IntRange get() = 1..horizon

    /** The requirement of periods [from] through [through],
     ↪  written d[t,u]. */
    fun requirementOver(from: Int, through: Int): Double {
        requireWindow(from, through)
        return (from..through).sumOf { d[it - 1] }
    }
}
```

Periods are numbered 1 through N on the outside and from zero on the inside, which is the ordinary arrangement. The alternative, exposing zero-based periods, would make every call site translate and every example in this chapter read differently from the text.

The costs arrive as four parallel lists instead of four scalars, so a period-varying instance needs no second class. For example, the constant-cost case that every worked example in this chapter utilizes is a factory method:

```
companion object {
    /**
     * The common case: one order cost, one unit cost and one
 ↪  carrying charge for the
     * whole horizon, with h = ic as in @sec-costest-i.
     */
    fun constantCosts(
        label: String,
        requirements: List<Double>,
        orderCost: Double,
```

```
        unitCost: Double,
        carryingCharge: Double,
    ): RequirementsSchedule = RequirementsSchedule(
        label = label,
        requirements = requirements,
        orderCosts = List(requirements.size) { orderCost },
        unitCosts = List(requirements.size) { unitCost },
        holdingRates = List(requirements.size) {
 ↪  carryingCharge * unitCost },
    )
}
```

The window cost is the schedule's central method, and it is memoized, since every rule asks for the same $N(N+1)/2$ values and Section 5.4.1 requires the arc costs to be settled before any decision is taken.

```
/**
 * The window cost: order in [from] enough to cover the
 ↪  requirements of periods
 * [from] through [through], and pay for the holding that
 ↪  implies.
 */
fun windowCost(from: Int, through: Int): Double {
    requireWindow(from, through)
    val key = (from - 1) * horizon + (through - 1)
    cached[key]?.let { return it }
    val value = k[from - 1] + (from..through).sumOf {
     ↪  unitCostHeld(from, it) * d[it - 1] }
    cached[key] = value
    return value
}

/**
```

```
 * Setup plus carrying for a window. This is the cost the
 ↪  heuristics compare. The
 * purchase term is excluded: it is the same under every plan
 ↪  while the
 * unit cost is constant, and a criterion that divides by the
 ↪  length of the window
 * would otherwise be dominated by it.
 */
fun relevantWindowCost(from: Int, through: Int): Double =
    orderCostIn(from) + carryingCostOver(from, through)
```

Two window costs, and the difference between them is not cosmetic. Equation 5.4 includes the purchase term and the recursion uses it, and that is right: adding the same constant to every plan changes no comparison. A heuristic that divides by the length of the window is a different matter. The purchase term grows with the window while the setup does not, so a cost-per-period built on Equation 5.4 falls simply because the window is longer, and Silver-Meal would extend to the end of the horizon on every instance. It has to divide the relevant cost, and `relevantWindowCost` supplies it.

5.9.5. Three Rules Are One Rule

Section 5.5.1 claims that Silver-Meal, least unit cost and part-period balancing are one procedure with three stopping rules. If that claim is true, the code should contain the procedure once.

```
interface LotSizingRule {
    val name: String
    fun plan(schedule: RequirementsSchedule): LotSizingPlan
}
```

```
abstract class WindowRule(override val name: String) :
 ↪  LotSizingRule {

    /** The figure of merit for covering [from] through
     ↪  [through], or null to skip. */
    protected abstract fun scoreOf(
        schedule: RequirementsSchedule,
        from: Int,
        through: Int,
    ): Double?

    final override fun plan(schedule: RequirementsSchedule):
     ↪  LotSizingPlan {
        val orders = mutableListOf<Int>()
        var from = 1
        while (from <= schedule.horizon) {
            orders.add(from)
            from = 1 + lastPeriodCovered(schedule, from)
        }
        return schedule.planFrom(orders)
    }

    /** How far a replenishment placed in [from] should
     ↪  reach. */
    fun lastPeriodCovered(schedule: RequirementsSchedule,
     ↪  from: Int): Int {
        var best = from
        var previous: Double? = null
        for (through in from..schedule.horizon) {
            val score = scoreOf(schedule, from, through) ?:
             ↪  continue
            if (previous != null && score > previous +
             ↪  TOLERANCE) break
```

```
            previous = score
            best = through
        }
        return best
    }
}
```

Then two of the three rules are three lines apiece.

```
/** Silver-Meal: the least cost per period. */
object SilverMeal : WindowRule("Silver-Meal") {
    override fun scoreOf(schedule: RequirementsSchedule,
     ↪  from: Int, through: Int): Double =
        schedule.relevantWindowCost(from, through) / (through
 ↪  - from + 1)
}

/** Least unit cost: the least cost per unit. */
object LeastUnitCost : WindowRule("Least unit cost") {
    override fun scoreOf(schedule: RequirementsSchedule,
     ↪  from: Int, through: Int): Double? {
        val units = schedule.requirementOver(from, through)
        return if (units <= 0.0) null else
         ↪  schedule.relevantWindowCost(from, through) /
         ↪  units
    }
}
```

That is Table 5.9, executable. The nullable return is the third column of that table's one irregularity: a window covering no units has no cost per unit, so the rule steps over it instead of dividing by zero. Month 9 of the belt is such a period.

The third rule does not fit, and forcing it in would be the mistake.

```
/**
 * Part-period balancing: extend until the carrying cost of
 ↪  the window is as close as
 * it can get to the order cost.
 *
 * It does not fit WindowRule because it does not walk a
 ↪  score to a minimum. It walks
 * a gap to its closest approach, and the period it keeps may
 ↪  be the one before the
 * test failed and not the one that failed.
 */
object PartPeriodBalancing : LotSizingRule {
    override val name: String = "Part-period balancing"

    fun lastPeriodCovered(schedule: RequirementsSchedule,
     ↪  from: Int): Int {
        val setup = schedule.orderCostIn(from)
        var best = from
        for (through in from + 1..schedule.horizon) {
            val carried = schedule.carryingCostOver(from,
             ↪  through)
            if (carried > setup) {
                val below = schedule.carryingCostOver(from,
                 ↪  through - 1)
                return if (abs(carried - setup) < abs(below
                 ↪  - setup)) through else through - 1
            }
            best = through
        }
        return best
    }
}
```

A template method earns its place when three things share a shape, and should be abandoned the moment a fourth thing only appears to. Part-period balancing can step back, and a stopping rule that returns "the one before the one that failed" cannot be expressed as a figure of merit walked to a minimum. Section 5.8.4 found the same boundary in the worksheet, where the two walking rules need a triangle of stopping tests and part-period balancing needs an ascending lookup. That two independent implementations drew the line in the same place is a sign the line is in the domain and not in either implementation.

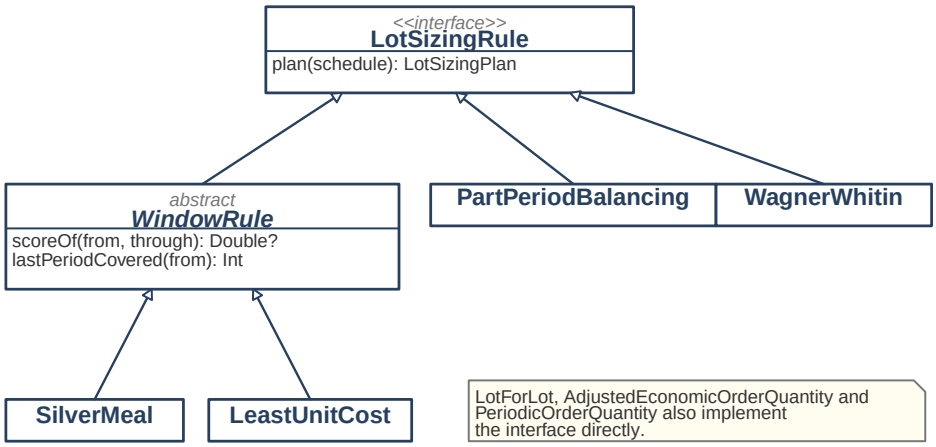


Figure 5.17.: The rule hierarchy. Silver-Meal and least unit cost share the window-walking procedure through WindowRule and differ only in the score. Part-period balancing and Wagner-Whitin implement the interface directly, because neither walks a score to a minimum.

5.9.6. The Algorithm Is Not a Heuristic

`WagnerWhitin` implements `LotSizingRule` and shares none of `WindowRule`. That is the arrangement Section 5.5 describes: an optimal algorithm and a

stopping rule are different kinds of thing, and the interface says only that both answer the same question.

```
object WagnerWhitin : LotSizingRule {
    override val name: String = "Wagner-Whitin"

    /** The value function, the predecessor pointers, and the
     ↪  plan they trace out. */
    fun solve(schedule: RequirementsSchedule): Solution {
        val n = schedule.horizon
        val value = DoubleArray(n + 1)
        val from = IntArray(n + 1)
        for (t in 1..n) {
            var best = Double.MAX_VALUE
            var argument = 1
            for (s in 1..t) {
                val candidate = value[s - 1] +
                 ↪  schedule.windowCost(s, t)
                // A tie keeps the earliest period achieving
                 ↪  it, the same one that
                // scanning s upward with a strict comparison
                 ↪  keeps, and that MATCH
                // finds on the worksheet. Both artifacts
                 ↪  must report the same S(t).
                if (candidate < best - TOLERANCE) {
                    best = candidate
                    argument = s
                }
            }
            value[t] = best
            from[t] = argument
        }
        ...
    }
```

```
}
```

`solve` returns more than a plan. It returns V and S, because Example 5.3 prints them and because a reader who wants to check the traceback needs the pointers. A method that returned only the plan would make Table 5.7 impossible to produce without solving the problem twice.

The two economic order quantity rules call into Chapter 3 instead of recomputing it.

```
class AdjustedEconomicOrderQuantity : LotSizingRule {
    override fun plan(schedule: RequirementsSchedule):
     ↪ LotSizingPlan {
        val quantity =
         ↪ EconomicOrderQuantity.orderQuantityFor(
            orderCost = schedule.orderCostIn(1),
            demandRate = schedule.averageRequirement,
            holdingRate = schedule.holdingRateIn(1),
        )
        ...
    }
}
```

This is the same `EconomicOrderQuantity` that Section 3.11 built, used unchanged. Equation 3.17 appears once in this book's code, so there is one place to correct when it is wrong.

5.9.7. How the Objects Solve It

Figure 5.18 illustrates one call. The caller hands a rule a schedule and gets a plan.

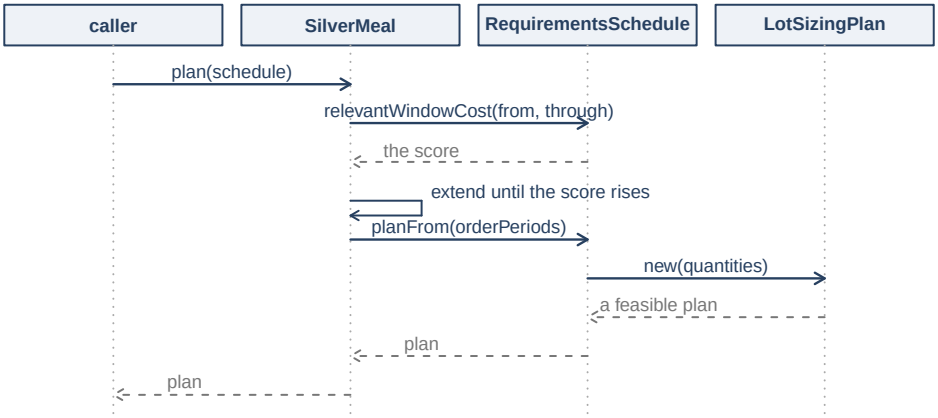


Figure 5.18.: Producing a Silver-Meal plan. The rule asks the schedule to price windows until its score rises, which fixes the order periods, and then asks the schedule to build the plan. Notice that the rule computes no cost of its own. It decides which costs to ask for. The dashed arrows are returns.

The schedule sits in the middle of every collaboration. That follows from giving it the window cost. A rule never computes a cost; it decides which costs to ask for. That is what lets Section 5.6 rank the rules: they are being compared on their decisions and not on their arithmetic.

5.9.8. Rolling the Horizon

`RollingHorizon` is the class that exists because a worksheet cannot do this.

```
class RollingHorizon(
    private val actual: RequirementsSchedule,
    private val planningHorizon: Int,
    private val freeze: Int = 1,
) {
    /**
     * Run [rule] on a window that advances by [freeze]
 ↪  periods at a time, implementing
     * only the frozen orders, and report what the
 ↪  implemented orders actually cost.
     */
    fun realizedPlan(rule: LotSizingRule): LotSizingPlan {
        val n = actual.horizon
        val placed = DoubleArray(n)
        var period = 1
        while (period <= n) {
            val onHand = carriedInto(placed, period)
            val committedTo = minOf(period + freeze - 1, n)
            val net = netRequirements(period, minOf(period +
             ↪  planningHorizon - 1, n), onHand)
            if (net != null) {
                val plan = rule.plan(net.window)
                for (t in period..committedTo) {
```

```
                    val offset = t - net.firstPlanned + 1
                    if (offset in 1..net.window.horizon)
                     ↪  placed[t - 1] = plan.orderIn(offset)
                }
            }
            period = committedTo + 1
        }
        return LotSizingPlan(actual, placed.toList())
    }
}
```

The subtle part is `netRequirements`. The planner at period t may be holding stock left over from an earlier order, and assumption 8 says a schedule starts empty. So the stock on hand is netted against the nearest requirements, as a requirements record nets it, and planning begins at the first period the stock does not cover. Getting this wrong produces a plan that orders on top of stock it already has, which every rule in the chapter is supposed to forbid.

Notice that `realizedPlan` returns a `LotSizingPlan` built against the *true* schedule and not against any of the windows. The plans the rule produced along the way were mostly discarded. What is reported is the cost of the orders that were actually placed, the only thing Table 5.14 could mean.

5.9.9. Solving the Chapter's Examples

Every figure in this chapter comes from running this code. The item of Table 5.1 is

```
val belt = RequirementsSchedule.constantCosts(
    label = "Mower deck belt",
    requirements = listOf(40.0, 60.0, 120.0, 300.0, 420.0,
 ↪  260.0,
```

```
                            120.0, 40.0, 0.0, 80.0, 180.0,
                             ↪  220.0),
    orderCost = 300.0, unitCost = 50.0, carryingCharge =
 ↪  0.02,
)

println(belt.variabilityCoefficient)   // 0.6191,
 ↪  @sec-dls-performance
println(belt.forcedOrderPeriod())      // 5,
 ↪  @sec-dls-properties
```

Table 5.12 is a loop, and that is what the design was for.

```
val optimum = WagnerWhitin.plan(belt)
val rules = listOf(
    LotForLot, AdjustedEconomicOrderQuantity(),
 ↪  PeriodicOrderQuantity(),
    LeastUnitCost, PartPeriodBalancing, SilverMeal,
 ↪  WagnerWhitin,
)
for (r in rules) {
    val p = r.plan(belt)
    println("%-32s %6d %9.0f %9.0f %9.0f %8.2f%%".format(
        r.name, p.orderCount, p.setupCost, p.carryingCost,
 ↪  p.relevantCost,
        p.penaltyAgainst(optimum).relativeError * 100.0))
}
```

Example 5.3 needs the recursion itself and not only its answer.

```
val quarter = RequirementsSchedule.constantCosts(
    label = "First half year",
```

```
    requirements = listOf(40.0, 60.0, 120.0, 300.0, 420.0,
 ↪  260.0),
    orderCost = 300.0, unitCost = 50.0, carryingCharge =
 ↪  0.02,
)
val solution = WagnerWhitin.solve(quarter)
for (t in 1..6) {
    val purchases = 50.0 * (1..t).sumOf {
     ↪  quarter.requirementIn(it) }
    println("V(%d) = %.0f, S(%d) = %d"
        .format(t, solution.valueAt(t) - purchases, t,
         ↪  solution.orderedFromAt(t)))
}
```

The subtraction is Section 5.2.1. Equation 5.4 carries the purchase term, so `valueAt` carries it too, and what Example 5.3 reports is the relevant part. Leaving the term in shifts every V by the same amount and changes no decision, which is the argument of Section 5.2.1 seen from the other side.

And Table 5.14 is two lines per row.

```
for (window in 2..6) {
    val sm = RollingHorizon(belt, window, freeze =
     ↪  1).realizedPlan(SilverMeal)
    val ww = RollingHorizon(belt, window, freeze =
     ↪  1).realizedPlan(WagnerWhitin)
    println("%d %.0f %.0f".format(window, sm.relevantCost,
 ↪  ww.relevantCost))
}
```

Those four fragments produce every number the chapter prints. Section 5.10 collects what it argued with them.

5.10. Summary

The chapter removed the constant demand rate and everything that depended on it. With no repeating cycle there is no average cost per cycle to minimize, so the economic order quantity has nothing to be derived from. What replaces it is a finite horizon with a decision in every period.

Two properties make the problem small enough to solve. Zero-inventory ordering says an optimal plan never orders into a shelf that is not empty, which means a plan is nothing but a choice of order periods. The window cost $w(t, u)$ prices any such choice, and it depends on no decision, so all of the window costs can be computed once.

Those two together turn the problem into a shortest path on an acyclic network, and Equation 5.8 solves it in $O(N^2)$. The same problem is a mixed integer program, slower, but it accepts constraints the recursion cannot.

The heuristics exist because Equation 5.8 needs the whole horizon and the whole horizon is a forecast. Three of them are one procedure with three stopping rules. On the mower deck belt they land between 4.84% and 22.58% above the optimum, which is the same order of magnitude as the penalty for getting k/h wrong by a factor of two.

The rolling horizon is where all of this is actually used, and it removes Wagner-Whitin's guarantee. It does not, on the evidence of Table 5.15, hand the advantage to a heuristic. What it does is make the question empirical.

Chapter 6 takes these methods up again with the requirements computed from a bill of material instead of forecast. That is where they do most of their work.

Table 5.17.: Notation introduced in this chapter.

Symbol	Meaning
N	number of periods in the horizon
d_t	requirement in period t
$d[t,u]$	requirement of periods t through u
k_t, c_t, h_t	order cost, unit cost, holding rate in period t
Q_t, I_t, y_t	order quantity, ending inventory, order indicator
$\hat{c}(t,u)$	cost of a unit bought in t and used in u
$w(t,u)$	cost of a replenishment in t covering through u
$V(u)$, $S(u)$	least cost through period u, and the period its last order starts in
VC	variability coefficient

5.11. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Periods are numbered from one. Inventory starts at zero and ends at zero. Carrying cost is charged on the balance at the END of a period, so a unit that arrives and is consumed in the same period is charged nothing. Report costs to the nearest cent, penalties as percentages to two decimal places, and the variability coefficient to four. Report relevant cost, setup plus carrying, unless the exercise asks for the total. Where two plans tie, report the one whose last order starts in the earlier period. That is what Equation 5.8 returns when the minimum is scanned upward in t.

5.11.1. Terminology and Concepts

Exercise 5.1. Define each of the following in one sentence, and say which section of this chapter introduced it.

a. requirements schedule
b. window cost
c. zero-inventory ordering
d. relevant cost
e. variability coefficient
f. part-period
g. rolling horizon
h. nervousness

Exercise 5.2. State whether each of the following is true or false, and give one sentence of justification. A one-word answer earns nothing.

a. An optimal plan never places an order in a period that begins with stock on hand.
b. Because the Wagner-Whitin algorithm is optimal, it is the method a planner should use when the horizon rolls forward one period at a time.
c. The window cost $w(t, u)$ depends on which periods the plan chooses to order in.
d. Two plans that place the same number of orders cost the same.
e. Under the convention of this chapter, a unit that arrives in a period and is consumed in that period is charged no holding cost.
f. The variability coefficient is measured in units per period.
g. Adding the same constant to every window cost cannot change which plan the recursion selects.
h. When ordering is free, lot-for-lot is optimal.

Exercise 5.3. Choose the best answer.

1. The window costs are computed before any search begins because

   (a) it saves memory; (b) they do not depend on any decision; (c) the heuristics need them before the algorithm does; (d) a worksheet cannot compute them on demand.

2. Silver-Meal's failure on the mower deck belt is triggered by

   (a) a run of declining requirements followed by a period with none; (b) a large requirement early in the horizon; (c) an order cost that is too high; (d) a holding rate that is too low.

3. Put a ceiling on the size of any single order and Equation 5.8 no longer applies, because (a) the window costs become wrong; (b) the network acquires a cycle; (c) zero-inventory ordering no longer holds; (d) the horizon is no longer finite.
4. The purchase term may be dropped from the objective when

   (a) the horizon is short; (b) never, and the chapter is wrong to do it; (c) the holding rate is small relative to the order cost; (d) the unit cost is the same in every period.

5. A heuristic is used instead of the Wagner-Whitin algorithm mainly because

   (a) the algorithm needs requirements out to period N and those are a forecast; (b) the algorithm is too slow; (c) the algorithm cannot be done in a worksheet;
   (b) the algorithm requires constant costs.

Exercise 5.4. An analyst argues that since the Wagner-Whitin algorithm is optimal and runs in $O(N^2)$, the heuristics of Section 5.5 are of historical interest only and should be dropped from the course.

Write a reply of at most 150 words. Your reply should name the assumption the argument depends on, say where in Section 5.1.2 it appears, and cite one number from this chapter.

Exercise 5.5. Section 5.3.1 proves that an optimal plan orders only into an empty shelf, and every method in this chapter rests on it.

a. State where the proof would fail if backorders were permitted at a cost per unit per period.
b. State what has to change in the proof if the unit cost fell partway through the horizon, and whether the property survives.
c. State where it would fail under a ceiling on the size of any single order.
d. For each, say whether the problem is still a shortest path over some network, and if so what the nodes would have to be.

5.11.2. Working the Methods by Hand

Exercise 5.6. An item has requirements of 30, 50, 10, 90, and 40 units over five periods. Ordering costs \$120, the item costs \$8, and the carrying charge is 2.5% per period.

a. Compute h and the variability coefficient. Does Section 5.6.1 call for a heuristic?
b. Cost the lot-for-lot plan.
c. Cost the plan that orders in periods 1 and 4 only.
d. Which is cheaper, and by how much?

Exercise 5.7. For the item of Exercise 5.6, compute the full 5×5 table of window costs $w(t, u)$ using Equation 5.7. Verify your entry for $w(1, 4)$ by costing the corresponding plan directly.

Exercise 5.8. Solve Exercise 5.6 to optimality with the recursion of Equation 5.8. Report $V(u)$ and $S(u)$ for every period, the traceback, the optimal plan, and its relevant cost.

Exercise 5.9. Apply Silver-Meal, least unit cost and part-period balancing to Exercise 5.6. Report each plan and its penalty against the optimum of Exercise 5.8. Explain why the answers come out the way they do.

Exercise 5.10. Take the last six months of the mower deck belt, months 7 through 12 of Table 5.1, and treat them as a horizon of their own numbered 1 through 6. The costs are unchanged: $k = \$300$, $c = \$50$, $i = 0.02$ per month.

t	1	2	3	4	5	6
d_t	120	40	0	80	180	220

a. Compute the variability coefficient and check Section 5.3.3. Does any period force an order?
b. Build the 6×6 table of window costs.
c. Solve the recursion, report V and S, and give the optimal plan and its cost.
d. Apply all three heuristics. Report each plan and its penalty.
e. Period 3 requires nothing. Say precisely what each of the three rules does when its window reaches that period, and which of them has to be told about it in code.

Exercise 5.11. Section 5.2.2 explains why the chapter compares an *adjusted* economic order quantity and not a fixed one. This exercise makes the reason concrete. Take the belt of Table 5.1 with $k = \$300$, $c = \$50$ and $i = 0.02$ per month, and apply the strict rule: order exactly $Q^* = 303.3150$ belts whenever the balance would otherwise go negative, and order twice in a month if once is not enough.

a. Build the ledger and report the order quantities and the ending balances.
b. How many belts are ordered in total, against a requirement of 1,840?
c. What is left at the end of month 12?

d. Which of Section 5.1.2 does the plan break, and why can no rule of this shape avoid breaking it?
e. The relevant cost comes to \$3,792.49. Explain why comparing that with the \$2,600 of Table 5.12 is not a fair comparison, and say what you would have to do to the plan to make it one.

Exercise 5.12. Section 5.5 notes that part-period balancing comes in two versions. The original takes the largest window whose carrying is at or under k_t, and so never overshoots. The modification compares the two windows either side of k_t and keeps whichever comes closer.

a. Using the carrying figures of Table 5.10 and the rest of the belt, work both versions over the full twelve months and report the stop period for each starting month.
b. Report each version's plan, its relevant cost and its penalty.
c. Which is better here? Is the answer what you expected?
d. Give the shape of a requirements pattern on which the original must do at least as well as the modification, and say why.

5.11.3. Modeling Problems

Exercise 5.13. The same distributor stocks a spindle assembly for the same line of mowers. It costs \$250 to place an order, the assembly costs \$25, and the carrying charge is 3% per month. The requirements for the coming year are

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	90	70	60	50	140	240	300	180	60	40	40	110

a. Compute h, the average requirement, the variability coefficient, k/h, the economic order quantity on the average requirement, and its time supply in months.

b. Does Section 5.3.3 force an order in any month?
c. Cost the lot-for-lot plan and the periodic order quantity plan at the time supply from part (a).
d. Solve the problem to optimality. Report V, S, the plan and its relevant cost.
e. Apply all three heuristics and report the penalty of each.
f. Compare your ranking of the three heuristics with Table 5.12. Does the belt's ranking survive?

Exercise 5.14. Table 5.12 ranks the methods on the belt at $k = \$300$. This exercise asks whether the ranking is a property of the methods or of that number.

Keep the requirements of Table 5.1, $c = \$50$ and $i = 0.02$ per month, and solve the problem again at $k = \$120$ and at $k = \$900$.

a. For each, report k/h and the periods Section 5.3.3 forces.
b. For each, report the optimal plan and its relevant cost.
c. For each, report the penalty of lot-for-lot, Silver-Meal, least unit cost and part-period balancing.
d. Tabulate the three settings side by side and say what happens to the ranking of the heuristics as k rises.
e. At $k = \$120$, use your answer to part (a) to say how many window costs the recursion actually has to compute, against the 78 it would need without Section 5.3.3.

Exercise 5.15. An item has requirements of 10, 2, 12, 4 and 14 units over five periods. Ordering costs \$40, the unit cost is \$2, and the holding rate is \$1 per unit per period.

a. Compute the window costs including the purchase term, as in Equation 5.4.
b. Solve the recursion and report the optimal plan.

c. Separate the answer into relevant cost and purchase cost. Confirm that the purchase cost is the same under every feasible plan, and explain why.
d. Now suppose the unit cost rises to $3 beginning in period 3 and stays there. Recompute the window costs and re-solve. Does the plan change, and is the purchase cost still the same under every plan?

Exercise 5.16. The variability coefficient of Equation 5.10 is the test for whether the methods of this chapter are worth using.

a. Construct a twelve-period schedule with VC just under 0.2 and one with VC near 1.0, both with the same total requirement.
b. For each, compute the penalty of an adjusted economic order quantity against the Wagner-Whitin optimum.
c. Does the 0.2 threshold do what it claims on your two instances?

Exercise 5.17. Prove that if $d_t h_{t-1} > k_t$ then every optimal plan orders in period t. Then explain how the property can be used to split a twelve-period problem into two smaller ones, and say how much arithmetic that saves for the belt.

5.11.4. The Rolling Horizon and the Solver

Exercise 5.18. Take the mower deck belt of Table 5.1 and a planner who sees only three months ahead. Starting in month 1, solve the three-month problem, place that month's order, advance one month, and repeat. Report the twelve orders actually placed and their cost, and compare with Table 5.14.

Exercise 5.19. Open the `Solver` sheet of `Chapter5Models.xlsx`, clear the order and flag columns to zero, and run Solver. Confirm that it reaches $2,480.

a. Now add the constraint that no single order may exceed 400 belts. What does the plan cost?
b. Explain why Equation 5.8 cannot answer part (a) without modification, and describe what would have to change.

Exercise 5.20. Section 5.7.3 describes three devices against nervousness. For each, say what it costs and under what circumstances you would recommend it. Use the first-month commitments reported in Section 5.7.3 to support your answer.

5.11.5. Using the Workbook

Exercise 5.21. Open `Chapter5Models.xlsx` and work entirely within it.

a. On the `Data` sheet, replace the requirements with those of Exercise 5.13 and set k, c and i to that item's values. Read the optimal cost off the `WagnerWhitin` sheet and the three heuristic costs off the `Heuristics` sheet, and check them against your answers to Exercise 5.13.
b. Restore the belt. On the `Plans` sheet, type the Wagner-Whitin plan of Table 5.8 into the fourth order column and confirm that the penalty reads 0.00%.
c. Now type a plan of your own into the same column that costs less than \$2,600 and is not the optimum. Report it, or explain how the `WagnerWhitin` sheet tells you in advance how many such plans there are.
d. On the `Heuristics` sheet, find the row of the Silver-Meal block that produces the order of 840 belts. Read off the six scores along it from $u = 5$ to $u = 10$ and say which one the rule stopped at and why.
e. Change the requirement in month 9 from 0 to 200. Report what happens to the six scores of part (d), to the Silver-Meal plan, to the optimum, and to the penalty. Two of those four move in a direction most students do not predict.

5.11.6. Using the Software

Exercise 5.22. The code of Section 5.9 is in `code/`, package `inventory.dynamiclotsizing`. Build it with `./gradlew build` and write a `main` of your own. Report the code you wrote alongside each answer.

a. Build the spindle assembly of Exercise 5.13 with `RequirementsSchedule.constantCosts`. Print `averageRequirement`, `variabilityCoefficient` and `forcedOrderPeriod()`, and compute k/h from `orderCostIn(1)` and `holdingRateIn(1)`. Check all four against your answers to Exercise 5.13, and say what it means that `forcedOrderPeriod()` returns `null`.

b. Put the seven rules in a list and loop over it, printing each rule's `name`, `orderCount`, `setupCost`, `carryingCost`, `relevantCost` and `penaltyAgainst(WagnerWhitin.plan(spindle))`. Report the table. Which rule is worst, and does Section 5.6.3 predict it?

c. Run `RollingHorizon(spindle, w, freeze = 1).realizedPlan(rule)` for $w = 2$ through 6, under both `SilverMeal` and `WagnerWhitin`, and report the two rows as Table 5.14 is reported. Two things happen on this item that do not happen to the belt. Identify both and say which claim of Section 5.7.2 they instantiate.

d. You worked both versions of part-period balancing over the belt by hand in Exercise 5.12. The package implements only the modified one. Implement the original as a `LotSizingRule`: extend the window while the carrying cost of the window is at or under k, and stop before the period that would carry it past k. Use `carryingCostOver(from, through)` and `planFrom(orderPeriods)`. Check it against your hand answer on the belt, then run both versions on the spindle assembly. The two items do not rank the two versions the same way, so say which is the better rule and defend the answer.

e. The study of Table 5.15 draws its schedules as follows.

```
fun schedules(count: Int, seed: Long, orderCost:
 ↪  Double): List<RequirementsSchedule> {
    val draw = java.util.Random(seed)
    val choices = listOf(0.0, 10.0, 20.0, 40.0, 80.0,
     ↪  120.0, 200.0, 300.0)
    return (1..count).map { n ->
        var d: List<Double>
        do { d = (1..12).map {
         ↪  choices[draw.nextInt(choices.size)] } }
        while (d.first() <= 0.0 || d.sum() <= 0.0)
        RequirementsSchedule.constantCosts("Trial $n",
 ↪  d, orderCost, 50.0, 0.02)
    }
}
```

Call it as `schedules(2000, 20240501L, 200.0)` and reproduce Table 5.15. Then raise the order cost to \$600 and run it again. Report both tables and state the conditions under which Section 5.7.2's conclusion holds.

5.11.7. From the Literature

Exercise 5.23. Wagner and Whitin (1958) is eight pages long and is the origin of everything in Section 5.4. Read it.

a. The paper states its result as a planning-horizon theorem rather than as a shortest path. State the theorem in the notation of this chapter.
b. The paper's recursion and Equation 5.8 are the same recursion. Identify the correspondence between its symbols and ours.
c. The paper was published in 1958 and the method was not widely adopted. Using Section 5.5 and Section 5.7, give two reasons that have nothing to do with the speed of computers.

6. Material and Distribution Requirements Planning

Learning objectives

After reading this chapter you should be able to:

- distinguish dependent from independent demand, and explain why dependent demand should be calculated and not forecast
- read a bill of material, assign low-level codes, and say why an item must be planned at its deepest occurrence
- complete an MRP record: netting, lot sizing, and lead-time offsetting
- explode a master production schedule through a product structure, level by level
- say how long a planning horizon has to be before it can produce a plan at all
- explain how a parent's lot sizing rule reshapes the demand its components see, and why minimizing cost level by level does not minimize cost
- apply the same time-phased logic to a distribution network
- recognize how lot sizing, lead times, and schedule changes produce nervousness
- state what requirements planning assumes, and which of those assumptions the rest of this book removes

Chapter 3 decided one order quantity at a constant demand rate. Chapter 5 let the rate vary and planned one item over a horizon whose requirements were given. This chapter is about where those requirements come from.

For most of the items in a manufacturing business they are not forecast at all. They are calculated, from the schedule for whatever is built out of them. In this chapter we do that calculation for every item at every level of a product structure, which is material requirements planning, and doing it across a network of stocking locations is distribution requirements planning. The two are the same arithmetic, and this chapter says so more than once.

6.1. Demand You Should Not Forecast

An item's demand is *independent* when it arises outside the firm and *dependent* when it arises from the firm's own plans. A distributor selling mower deck belts over the counter faces independent demand for them. The manufacturer who puts one belt into every deck it builds does not: the demand for belts is whatever the deck schedule says it is.

The distinction is not about the item. It is about the position. The same belt is independent demand at the distributor and dependent demand at the manufacturer, and the right way to plan it differs accordingly.

6.1.1. Why Forecasting It Is a Mistake

Suppose we forecast belt demand from belt history, as Chapter 5's distributor would. Two things go wrong.

The first is that the forecast throws away information. The manufacturer knows the deck schedule. Every belt it will need for the next year is implied by a document it wrote itself. Replacing that with an extrapolation from past belt usage replaces a known quantity with an estimate of it, and the

estimate can only be worse. That is, the forecast is not merely unnecessary; it is a way of making the answer less certain than it already was.

The second is worse and less obvious. Dependent demand is *lumpy*. A deck assembly built in lots of 300 consumes belts in lots of 300, so the belt sees nothing for several periods and then 300 at once. A forecast fitted to that history produces an average, and an average is the one value the demand never takes. For example, the belt requirement in Table 6.5 is 420 in one month and nothing at all four months later, and its mean of 153 occurs in no month at all. Section 6.5 measures the lumpiness and shows where it comes from. That is not the item and not the customer but the lot sizing rule one level up.

Recall that Section 5.6.1 gave a test for when a pattern varies enough to need the methods of Chapter 5. Dependent demand almost always passes it, and it passes it for a reason that is manufactured and not observed.

6.1.2. Where This Chapter Sits

Chapter 5 planned one item, and this chapter plans an item for every node of a product structure. What it hands each of them is the requirements schedule of Section 5.1.1, and what it does with the answer is pass it down to the level below. The lot sizing of Section 5.5 is used here unchanged and is not restated.

The stochastic chapters that follow remove the assumption that any of this is known. Section 6.8 collects what requirements planning takes for granted, and it is the bridge to them.

6.2. The Bill of Material

A *bill of material* records what goes into what, and how many. Figure 6.1 illustrates the manufacturer's mower deck assembly.

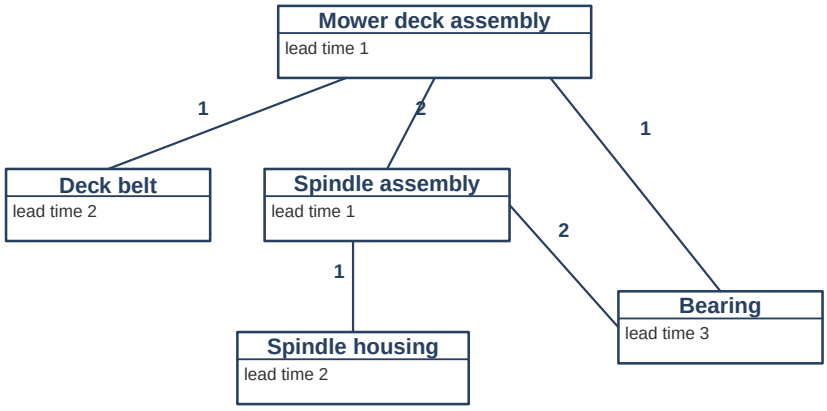


Figure 6.1.: The mower deck assembly. The number on an arc is the quantity per. Notice that the bearing hangs from two places: one is fitted directly to the deck and two more sit inside each spindle assembly. It therefore appears at level 1 and at level 2, and its low-level code is the deeper of them.

The same information is often written as an indented list, easier to type and harder to read:

```
Mower deck assembly (DA), lead time 1
    Deck belt (B), 1 each, lead time 2
    Spindle assembly (S), 2 each, lead time 1
        Spindle housing (H), 1 each, lead time 2
        Bearing (BR), 2 each, lead time 3
    Bearing (BR), 1 each, lead time 3
```

Each indentation is a *level*. The deck assembly is level 0, the belt and the spindle assembly are level 1, and the housing is level 2. The number in parentheses is the *quantity per*: one deck assembly takes two spindle assemblies, and each of those takes two bearings.

Notice that the bearing appears twice. An idler bearing is fitted directly to the deck and two more sit inside each spindle assembly. That is the usual case, not an awkward one, and it is the reason for the next idea.

6.2.1. Low-Level Codes

An item must be planned once, and it must be planned after every parent that consumes it. The bearing appears at level 1 and again at level 2. If it were planned at level 1, its requirements would be netted against the stock on hand before the spindle assembly had asked for any, and the spindle assembly would then find the stock already spoken for.

Thus, each item carries a *low-level code*: the deepest level at which it appears anywhere in the structure. The bearing's is 2. Table 6.1 collects them.

Table 6.1.: Low-level codes for the deck assembly structure.

Item	Appears at level	Low-level code
Mower deck assembly, DA	0	0
Deck belt, B	1	1
Spindle assembly, S	1	1
Spindle housing, H	2	2
Bearing, BR	1 and 2	2

Thus, planning in low-level-code order, shallowest first, guarantees that every parent of an item has a plan before the item does. For this structure the order is DA, then B and S, then BR and H.

6.2.2. The Explosion Calculation

Before any record, we need the arithmetic that connects two levels. Let item i be a parent of item j, and q_{ij} of j go into one i, then the requirement j inherits in period t is

$$G_{jt} = \sum_{i \in \text{parents}(j)} q_{ij}\, Rel_{it}, \tag{6.1}$$

where Rel_{it} is the quantity of i that the plan *releases* in period t.

That is, releases and not receipts. A parent needs its components in hand when it starts building, and the release is when it starts. Using the receipt row would ask for the components on the day the parent is finished, a whole lead time too late. This is the single most common error in a hand-worked explosion.

For the bearing, Equation 6.1 has two terms, because it has two parents:

$$G_{BR,t} = 1 \cdot Rel_{DA,t} + 2 \cdot Rel_{S,t}.$$

The two arrive in different periods, because the two paths through the structure have different lead times. Section 6.3 builds the record that turns a requirement into a release.

6.3. The MRP Record

We give every item a record: six rows over a common horizon of periods. The periods run across the columns. That is how a requirements record is laid out everywhere it is used, and the reverse of Chapter 5's worksheets. Nothing in the arithmetic depends on it.

Let G_t represent the gross requirement in period t, SR_t the scheduled receipt, I_t the projected on hand at the **end** of period t, NR_t the net

requirement, POR_t the planned order receipt, and Rel_t the planned order release. Let L be the lead time and SS the safety stock.

Gross requirements are what is needed. For an end item they come from the master production schedule. For everything else they come from Equation 6.1.

Scheduled receipts are what is already on order. These are commitments made in some earlier planning cycle, and they arrive whether or not this plan wants them.

Projected on hand is what is left at the end of a period:

$$I_t = I_{t-1} + SR_t + POR_t - G_t. \tag{6.2}$$

That is Equation 5.1 with the scheduled receipts added, and it is an end-of-period figure for the same reason: Section 5.1.2 charges carrying on what crosses a period boundary.

Net requirements are what is short once the opening stock and the scheduled receipts have been counted:

$$NR_t = \max\left(G_t + SS - SR_t - I_{t-1},\ 0\right). \tag{6.3}$$

Planned order receipts are what the lot sizing rule decides should arrive. Under lot-for-lot, $POR_t = NR_t$. Under anything else it is larger and less frequent, and Section 6.5 is about what that costs.

Planned order releases are the receipts moved back by the lead time:

$$Rel_{t-L} = POR_t. \tag{6.4}$$

This row is the record's output. That is, it is what Equation 6.1 reads at the next level down, and it is the only row that says anything the planner will act on.

6.3.1. The Maximum Is the Whole Idea

Equation 6.3 has a max in it, and a reader meeting the record for the first time usually leaves out the net requirement row altogether, and the max with it. See what happens.

Example 6.1 (A record without netting)**.** Let's take the deck assembly's master schedule and carry the on-hand balance forward without netting anything, so that Equation 6.2 runs with $POR_t = 0$ throughout.

Table 6.2.: The deck assembly with nothing netted. Every negative entry is a shortage the plan has not dealt with.

Period	6	7	8	9	10	11	12	13	14	15	16	17
Gross requirements	40	60	120	300	420	260	120	40	0	80	180	220
Projected on hand	-40	-100	-220	-520	-940	-1,200	-1,320	-1,360	-1,360	-1,440	-1,620	-1,840

The balance reaches $-1,840$, the whole year's requirement. That is correct arithmetic and a useless plan: it reports that if nothing is ordered then everything is short, which the reader knew.

Netting turns each of those shortages into an order. Because this item starts with nothing on hand and nothing on order, Equation 6.3 gives $NR_t = G_t$ in every period, lot-for-lot has $POR_t = NR_t$, and Equation 6.2 then holds I_t at zero throughout.

What the max itself does shows only when there is stock to spare. Suppose the deck assembly had started with 60 on hand. In period 6 the quantity inside the max is $40 - 60 = -20$, and without the max lot-for-lot would plan a receipt of -20: it would cancel stock that period 7 is going to use. The max sets the net requirement to zero instead, and Equation 6.2 carries the 20 units into period 7, whose net requirement falls to 40. That is, netting turns each shortage into an order, and the max stops a surplus being treated as a requirement.

6.3.2. Safety Stock and Allocations

We meet two modifications to Equation 6.3 constantly, and neither needs new machinery.

Safety stock is the SS term. It enters every period's netting, but its effect is to raise the *first* net requirement by SS; after that the projected balance already carries it, and the plan holds SS units permanently and plans around what is left. For example, take an item ordered lot-for-lot with nothing on hand and nothing on order, whose first requirement falls in period t. Then Equation 6.3 and Equation 6.2 give

$$NR_t = \max\left(G_t + SS - 0 - 0,\ 0\right) = G_t + SS, \qquad I_t = 0 + 0 + (G_t + SS) - G_t = SS,$$
$$NR_{t+1} = \max\left(G_{t+1} + SS - 0 - SS,\ 0\right) = G_{t+1}, \quad I_{t+1} = SS + 0 + G_{t+1} - G_{t+1} = SS,$$

and every later period repeats the second line. Whether it belongs in a system whose entire premise is that demand is known is a real question, and the field has never fully settled it. Section 6.8 returns to it.

Allocated stock is inventory that is physically present but already promised to something else. It is handled by reducing I_0, the opening balance, in order to keep the stock that is promised elsewhere out of the calculation. Nothing else changes.

6.3.3. How Long the Horizon Has to Be

A record cannot release an order before period 1, and we have to allow for that. If a receipt is due in period t and the lead time is L, then the release falls in period $t - L$, and when that is less than 1 the order is *past due*: the plan has discovered a requirement it no longer has time to meet.

Past-due orders at the bottom of a structure are usually not a data problem. That is, they are a horizon problem. The lead times accumulate down every path, and the horizon must lead the first requirement by the longest of those accumulations.

For the deck assembly the deepest path runs assembly, spindle assembly, bearing, with lead times 1, 1 and 3, so the cumulative lead time is 5 months. Table 6.3 shows what happens on either side of that.

Table 6.3.: What a horizon that is too short produces. Five months of lead-in is exactly enough, and that is the cumulative lead time through the deepest branch.

Empty months in front of the schedule	3	4	5	6
Units the plan cannot release in time	520	160	0	0

Thus, the horizon used throughout this chapter is 17 months: five empty months and then the twelve-month schedule.

6.4. Running the Explosion

With the record defined and the order of the items settled, we produce the plan by doing one and then the other, level by level.

Algorithm 6.1 The explosion. The sort is what makes the sum over parents safe: by the time an item is reached, every parent of it already has a release row.

```
plan(bill of material, master production schedule):

    assign every item its low-level code
    sort the items by low-level code, shallowest first

    for each item in that order:
        if a master schedule speaks about this item:
            G <- the master schedule
        else:
            G(t) <- sum over parents i of q(i, item) * Rel(i, t)

        // the record of section 6.3
        NR(t) <- max(G(t) + SS - SR(t) - I(t-1), 0)
        POR   <- the lot sizing rule applied to NR
        I(t)  <- I(t-1) + SR(t) + POR(t) - G(t)
        Rel(t - L) <- POR(t)

    return every item's record
```

Example 6.2 (The deck assembly, all five items)**.** *What is stocked?* Five items of one product structure: the deck assembly and what goes into it.

What is the demand process? Known, and computed rather than forecast: the master schedule for the deck assembly, and for every other item the releases of the items above it.

When is inventory reviewed? Once a month, over a common horizon.

What triggers replenishment, and how much? A net requirement in a month, and lot-for-lot orders that amount.

What happens to unmet demand? None occurs; a requirement that cannot be met inside the horizon is a past-due release, which Section 6.3.3 treats.

What costs are incurred, and when? None enter this example; Section 6.5 prices the plan.

We plan every item lot-for-lot, with nothing on hand and nothing on order. The master schedule carries the twelve-month pattern in months 6 through 17.

The deck assembly nets to its own schedule, and Table 6.4 reads its releases off as that schedule pulled back one month.

Table 6.4.: The deck assembly's record, months 5 through 17.

Period	5	6	7	8	9	10	11	12	13	14	15	16	17
DA gross requirements	0	40	60	120	300	420	260	120	40	0	80	180	220
DA planned order releases	40	60	120	300	420	260	120	40	0	80	180	220	0

Now we apply Equation 6.1. The belt takes one per deck, so its gross requirements are the deck assembly's releases unchanged, and its own lead time of two months pulls them back again. The spindle assembly takes two per deck, so its gross requirements are twice that row.

Table 6.5.: The two level-one items.

Period	3	4	5	6	7	8	9	10	11	12	13	14	15	16
B gross requirements	0	0	40	60	120	300	420	260	120	40	0	80	180	220
B planned order releases	40	60	120	300	420	260	120	40	0	80	180	220	0	0
S gross requirements	0	0	80	120	240	600	840	520	240	80	0	160	360	440
S planned order releases	0	80	120	240	600	840	520	240	80	0	160	360	440	0

The bearing is the interesting one, because Equation 6.1 has two terms for it. One bearing per deck assembly and two per spindle assembly, and Table 6.6 keeps the two apart before adding them, because the two parents release in different periods.

Table 6.6.: The bearing's gross requirements, which two levels feed.

Period	4	5	6	7	8	9	10	11	12	13	14	15	16
One per deck assembly	0	40	60	120	300	420	260	120	40	0	80	180	220
Two per spindle assembly	160	240	480	1,200	1,680	1,040	480	160	0	320	720	880	0
BR gross requirements	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220

Notice the shape of the last row. It is not the shape of the master schedule, and it is not the shape of either contribution taken alone. Two streams arriving on different offsets add up to a pattern that looks nothing like the demand that caused it, and nobody forecast any of it.

Lot-for-lot throughout costs $14,500, every dollar of it setup, because no item ever carries anything from one period to the next. That is the plan the next section improves on, and the way it improves on it is not the obvious one.

6.5. What Lot Sizing Does to the Level Below

Example 6.2 ordered lot-for-lot at every level. Thus, it is the plan an MRP system produces when nobody has told it otherwise. Recall from Section 5.5 that every rule there can be used instead, applied to the net requirement row of whichever record it is planning. None of them is restated here. What this section is about is something Chapter 5 could not see, because Chapter 5 had only one item.

A parent's lot sizing rule does not only decide what the parent costs. By Equation 6.1 it decides *when and in what sizes the components are asked for*, and so it decides what their problem looks like before they have made any choice at all.

6.5.1. One Rule Changes, and Everything Moves

We hold every component at lot-for-lot and change only the rule used on the deck assembly. Nothing below level 0 decides anything, so whatever we see move is the end item's doing.

Table 6.7.: What the end item's rule does to the whole structure. The variability coefficient is measured on what the deck assembly has arrive, which by Equation 6.1 is what the belt is asked for.

Rule on the deck assembly	Orders	VC at level 1	Cost at DA	Cost below	Total
Lot-for-lot	11	0.6191	\$5,500	\$9,000	\$14,500

Rule on the deck assembly	Orders	VC at level 1	Cost at DA	Cost below	Total
Periodic order quantity	11	0.6191	\$5,500	\$9,000	\$14,500
Wagner-Whitin	9	0.6701	**\$4,860**	\$7,500	\$12,360
Silver-Meal	8	0.9650	\$4,936	\$6,700	\$11,636
Least unit cost	7	1.0274	\$5,588	\$6,100	\$11,688
Part-period balancing	7	1.5463	\$5,444	\$6,100	\$11,544
Adjusted economic order quantity	7	0.8459	\$5,588	\$5,900	**\$11,488**

We draw three things out of that table.

Lot-for-lot is the only rule that changes nothing. Its receipts are the master schedule itself, so the components see the schedule, and $VC = 0.6191$ is the schedule's own figure. Every rule that batches raises it, as far as

1.5463 under part-period balancing. The periodic order quantity happens to agree with lot-for-lot here, because the deck assembly's holding rate is high enough that Equation 3.17 gives a time supply of 1.35 months, which rounds to one.

The cost below a level rises with how often the level above orders. Read the Orders column against the Cost below column and the relation is exact: seven orders cost between \$5,900 and \$6,100 below, eight cost \$6,700, nine cost \$7,500, and eleven cost \$9,000. The reason is Equation 6.1 and nothing else. Every release a parent makes is a requirement its components have to meet, and meeting a requirement costs a setup. For example, lot-for-lot releases eleven times and the components below it place their own orders eleven times each.

That is, each release of the deck assembly costs the belt, the spindle assembly and the housing one setup apiece, $\$300 + \$250 + \$150 = \$700$. The bearing orders once in every period in which either of its parents releases, at \$100 an order, so a release adds one or two bearing orders, according to whether a neighboring release has already claimed those periods. Under lot-for-lot,

$$11 \times \$700 + 13 \times \$100 = \$7,700 + \$1,300 = \$9,000,$$

which is the Cost below entry. Every other row decomposes the same way.

Thus, the rule that is best at a level is not the rule that is best for the product. Wagner-Whitin is optimal at the deck assembly, and it is optimal there by construction, since Section 5.4.2 proves no rule can beat it on that item's own requirements. It costs \$4,860 where the adjusted economic order quantity costs \$5,588, and it is \$872 worse over the structure as a whole. Choosing the rule by the level you can see costs 7.59% here.

6.5.2. What the Variability Coefficient Does Not Tell You

You will be tempted to read the VC column as a cost, and it is not one. Part-period balancing produces much the lumpiest pattern in Table 6.7 and is not the most expensive plan; Wagner-Whitin produces nearly the smoothest and is the second most expensive. Notice that the order count explains the cost below and the variability coefficient does not.

The variability coefficient is still the right thing to look at when you are asking whether a component's own planner needs the methods of Chapter 5. Section 5.6.1 set the threshold at 0.2, and every row of Table 6.7 clears it comfortably. That is the point made in Section 6.1.1, now with a number attached: dependent demand does not merely happen to be variable, it is made variable, by a decision taken one level up.

6.5.3. What Is Actually Done

Most installed systems run lot-for-lot below the top level or two, and reading Table 6.7 it is easy to see that as laziness. It is not, quite.

Lot-for-lot at a component is the rule that leaves the level beneath it alone, and in a structure several levels deep that matters more at each level down. Thus, it also produces the smallest projected balances, and that is what a planner wants when the requirements above are going to be revised anyway, and Section 6.7 is about how often that happens. And it needs no cost parameters, which matters because a bill of material with ten thousand items has ten thousand order costs that nobody has ever measured.

The defensible position is that batching pays where the setup is large and known, usually the end item and the major subassemblies, and that below that the information needed to do it well does not exist. Section 6.6 turns the same explosion on its side, to a network of regions feeding a center.

6.6. Distribution Requirements Planning

The same arithmetic plans a distribution network, and notice how little has to change.

Chapter 5's distributor does not build anything. It holds belts at three regional warehouses, each serving its own dealers, and replenishes all three from a central warehouse. Figure 6.2 illustrates the network.

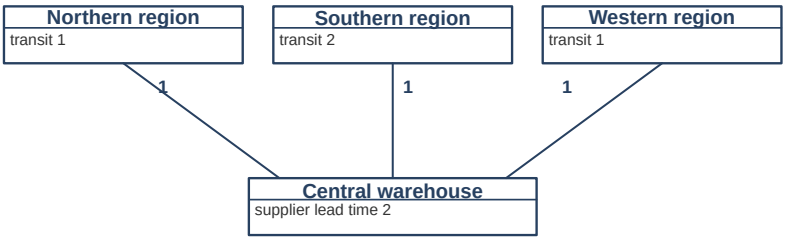


Figure 6.2.: The distributor's network. Three regions sell to dealers and one warehouse supplies all three. The arrows point the way a requirement travels, which is the same direction they point in Figure 6.1. Every quantity per is one, which is the whole of what separates this from a bill of material.

A regional warehouse's demand is independent: dealers buy what they buy. The central warehouse's demand is not. It is whatever the three regions order, a consequence of their own plans, as a component's demand is a consequence of its parent's. Thus, the central warehouse is planned the same way a component is.

6.6.1. It Is the Same Calculation

Let's write Equation 6.1 for the central warehouse. Its parents are the three regions, and the quantity per is one belt per belt:

$$G_{CW,t} = \sum_{r \in \text{regions}} 1 \cdot Rel_{rt}.$$

That is Equation 6.1 with every $q_{ij} = 1$. Transit time plays the part of lead time, the regions are the level-0 items, the warehouse is their component, and Algorithm 6.1 runs unchanged. That is, nothing in the algorithm has to know which kind of graph it is walking. The software of Section 6.10 uses one class for both, and Section 6.10.6 explains why that is not a trick.

Two differences are real and neither is arithmetic.

A bill of material has a quantity per and a distribution network does not, or rather has one everywhere. For example, a parent needs two spindle assemblies per deck, and a region needs one belt per belt.

The direction of the relationship also reads backwards. In a bill of material the parent is what the child goes into. In a network the parent is the *customer* and the child is the *supplier*. The arrow points the way the requirement travels, upstream in both cases, and that is why the same code carries them.

Example 6.3 (The three regions and the center)**.** The three regions sell 1,840 belts between them over the twelve months, split 830 to the north, 640 to the south and 370 to the west. Transit is one month to the north and west and two to the south, and the central warehouse's own supplier takes two months. The cumulative lead time is therefore four months, so the horizon leads the first sale by four.

Every location orders lot-for-lot. In Table 6.8 the regions' releases are their own sales pulled back by their transit times, and the center's gross requirements are the sum of the three.

Table 6.8.: The roll-up. The center's row is the three above it added down the column.

Period	3	4	5	6	7	8	9	10	11	12	13	14	15
North releases	0	20	30	60	140	180	110	50	20	0	40	80	100
South releases	10	20	40	100	160	100	50	10	0	20	60	70	0
West releases	0	10	10	20	60	80	50	20	10	0	20	40	50
Center's gross requirements	10	50	80	180	360	360	210	80	30	20	120	190	150

Notice that the south is a month out of step with the other two, because its transit time is longer. The center's requirement in period 3 is the south's alone.

6.6.2. What the Regions Do to the Center

Section 6.5's result has a counterpart here, and it is the more famous of the two. We change the rule the regions use and watch what the center sees.

Table 6.9.: What the regions' own rule does to the demand the center sees. The regions sell a pattern whose variability coefficient is 0.6191; the center's coefficient is measured over periods 3 through 15, the periods in which it has a requirement under lot-for-lot, and a rule that empties one of those periods is charged the zero.

Rule at every region	VC at the center	Regions	Center	Total
Lot-for-lot	0.6334	\$6,600	\$5,200	\$11,800
Part-period balancing	0.8692	\$3,860	\$3,200	\$7,060
Wagner-Whitin	1.1403	\$3,830	\$3,200	\$7,030
Silver-Meal	1.3715	\$3,980	\$3,600	\$7,580
Periodic order quantity	1.5896	\$4,310	\$3,200	\$7,510

Under every batching rule the demand the center faces is between 1.4 and 2.6 times as variable as the demand the regions actually sell, and the regions' customers did nothing to cause it. This is the *bullwhip effect*, and Table 6.9 is its mechanism in one table: batching at one stage is amplification at the next.

Notice two things before we let the conclusion run away.

Under lot-for-lot the center sees $VC = 0.6334$, within three percent of the 0.6191 the regions sell. Three locations ordering on different transit times spread the same 1,840 belts over thirteen periods instead of twelve, with two small periods at the front, and that spreading is the whole of the difference. Amplification is caused by batching and not by the network.

Every batching row in Table 6.9 is also cheaper than lot-for-lot, at the regions and at the center both. In a world where demand is known, amplification costs nothing: the center can see the lumpy pattern coming and plan for it, as it just did. The bullwhip effect is expensive when demand is uncertain, because then the center cannot tell an amplified pattern from a change in the market and must hold stock against both. Pricing that needs a model of demand uncertainty, which this chapter does

not have, and it is the reason this chapter can report the amplification without reporting a cost for it.

6.7. Nervousness

Recall from Section 5.7.3 what nervousness is with one item: the horizon rolls, the plan is recomputed, and the new plan disagrees with the old about orders not yet placed. In a product structure the disagreement propagates, and the propagation is the problem.

For example, a change to the master schedule in a distant period changes the deck assembly's releases. Those are the belt's requirements, so the belt replans. The belt's releases move, and with a two-month lead time they move into periods nearer than the change that caused them. Two levels down, a revision to a requirement eleven months out can move an order that was going to be placed next week.

Three things make it worse than the single-item case of Section 5.7.3.

Lot sizing amplifies it. A batching rule has thresholds, and a small change in requirements moves a plan across one. Section 5.7.3 measured the first-period commitment for two rules and found that Silver-Meal held steady while Wagner-Whitin did not. Here that instability is multiplied by every level beneath.

Lead times concentrate it. A change far out in the schedule arrives at the bottom of the structure much sooner, because each level's lead time pulls it earlier.

Low-level codes spread it. An item with several parents is replanned when any of them moves.

We have the devices of Section 5.7.3, with one addition. A *firm planned order* is a planned order the system is forbidden to move. It is the freeze fence applied to a single order instead of to a span of periods, and it is how

a planner overrides the calculation without turning it off. A *time fence* does the same for a span. A *change penalty* is rare here for the reason Section 5.7.3 gave, and rarer still because it would have to be defined for every item at every level.

Safety *lead time* is the device this chapter adds. Instead of holding extra stock, the item is planned to arrive a period early. It buffers against a late delivery in a way that safety stock does not, and it costs a period of carrying on everything instead of a permanent balance on one item. Which you should use depends on whether the uncertainty is in the quantity or in the timing, and requirements planning gives you no way to tell. Section 6.9 builds the records of this chapter in a worksheet, where the effect of each device can be tried on the belt.

6.8. What Requirements Planning Assumes

The arithmetic of this chapter is exact, and every assumption we made to get it is questionable. We collect them here, and they are the bridge to the rest of the book.

1. **Capacity is infinite.** Nothing in Algorithm 6.1 asks whether the shop can build 1,980 bearings in period 8. Thus, a plan that cannot be executed is not a plan, and capacity requirements planning takes the releases and compares them with what the work centers can do. When it finds a conflict the master schedule is changed and the explosion is run again. That loop, with capacity and the financial side attached, is what manufacturing resource planning means. It is outside this book, which takes up inventory and not scheduling.
2. **Lead times are fixed and known.** They are treated as properties of an item. In a real shop a lead time is mostly queueing, so it depends on how loaded the shop is, which depends on the lot sizes, which are what the plan is choosing. Planning with a fixed lead time

and then releasing work that makes the true lead time longer is a well-documented way to make a plan worse by following it.

3. **Requirements are known.** The master production schedule is a forecast for the end items, and everything below it inherits that forecast's error with the amplification of Section 6.5 applied.
4. **The records are right.** Section 2.3 showed what record accuracy actually is in the field. An explosion nets against on-hand balances. Thus, an error in one balance becomes a wrong order for every item beneath it. Requirements planning is less tolerant of bad records than an order-point system, because it computes rather than reacts.
5. **Everything is deterministic.** No demand distribution appears anywhere in this chapter, and the only concession to uncertainty is a safety stock that Section 6.3.2 could not justify from inside the model.

Assumption 5 is the one the rest of this book removes. Chapter 7 begins with a single period and a random demand, and Chapter 8 through Chapter 9 build the machinery that assumption 3 and assumption 5 need.

6.9. Building These Models in a Worksheet

The workbook is `Chapter6Models.xlsx`. It has four working sheets, and one thing about it will look wrong to a reader coming from Section 5.8: the periods run across the columns and not down the rows.

That is deliberate and it is not this book's choice. A requirements record is laid out with periods across the columns everywhere it is used, because a planner reads one item's six rows at a time and compares them with each other. Chapter 5 ran the periods downward because there the thing that grows is the horizon and every calculation was indexed by it. Here the thing that repeats is the item. The layout follows the field.

6.9.1. The Record, Computed Twice

The `Record` sheet computes the same gross requirements twice. The upper block carries the balance forward without netting and produces Table 6.2; the lower block nets and produces a plan. The two blocks differ in one formula, and putting them one above the other is the point of the sheet.

Below them, three cells: the lowest balance each block reaches, and the units the lead time pushed off the front of the horizon. Notice what that last one measures. It is the difference between what the plan wants to receive and what it managed to release, and it is zero only when the horizon is long enough. Figure 6.3 is the sheet.

6.9.2. The Explosion

The `Explosion` sheet holds the structure in three columns at the top and then one record per item, in low-level-code order. A component's gross requirement row is Equation 6.1 written out: a sum of terms, one per parent, each a quantity per times a release.

The quantities in the structure block are the cells those formulas read. Type a 3 over the 2 and every record below the spindle assembly moves. A workbook that showed the structure as a decoration while the formulas carried the numbers would be worse than one that did not show it at all, because it would look as though it worked. Figure 6.4 is the top of the sheet and Figure 6.5 the rest.

6.9.3. The Lumpiness Sheet

`Lumpiness` illustrates Table 6.7 with the arithmetic exposed. Each row is what one rule has the deck assembly receive, and the columns to the right count the orders and compute the variability coefficient over the twelve scheduled months.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	**One item's record**																	
2	*Chapter 6, section 6.3. The same gross requirements, computed twice.*																	
3																		
4	**WITHOUT NETTING what a reader writes the first time**																	
5	*Projected on hand is carried forward and allowed to go negative. Every negative entry is a shortage the plan has not dealt with.*																	
6	**DA Mower deck assembly lead time 1**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
7	**Gross requirements**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
8	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	**Projected on hand**	0	0	0	0	0	-40	-100	-220	-520	-940	-1,200	-1,320	-1,360	-1,360	-1,440	-1,620	-1,840
10	**Planned order receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	**Planned order releases**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12																		
13	**WITH NETTING the record of section 6.3**																	
14	*A net requirement is what is short after the opening stock and the scheduled receipts have been counted, and it is never negative.*																	
15	**DA Mower deck assembly lead time 1**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
16	**Gross requirements**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
17	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	**Net requirements**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
20	**Planned order receipts**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
21	**Planned order releases**	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
22																		
23	**CHECK with the data as shipped**																	
24	*Without netting the projected on hand reaches -1,840 by the last period.*																	
25	*With netting it never goes below zero, and every net requirement equals the gross requirement, because nothing is on hand and nothing is on order.*																	
26	Lowest projected on hand, without netting	-1,840																
27	Lowest projected on hand, with netting	0																
28	Units due before they could be ordered, which	0																

Figure 6.3.: The `Record` sheet. The upper block has no net requirement row at all, because a reader writing a record for the first time has not met netting; what it has instead is a projected-on-hand row running to $-1,840$. The lower block nets, and its projected-on-hand row never leaves zero. Notice that the two blocks share a gross requirement row to the unit, so the only thing that differs between them is the arithmetic.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	**The explosion**																	
2	*Chapter 6, section 6.4. Five items, planned in low-level-code order, every one on lot-for-lot.*																	
3																		
4	**THE STRUCTURE type over the quantities and every record below moves**																	
5	**Parent**	**Child**	**Quantity per**															
6	DA	B	1															
7	DA	S	2															
8	DA	BR	1															
9	S	H	1															
10	S	BR	2															
11	*The bearing appears twice, under the deck assembly and under the spindle assembly, which is why its low-level code is 2.*																	
12																		
13	**CHECK with the data as shipped**																	
14	*Lot-for-lot everywhere costs $14,500 in setup and nothing in carrying.*																	
15	*Nothing is released before period 1, because the horizon leads the first requirement by the cumulative lead time of five months.*																	
16	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
17	**DA Mower deck assembly lead time 1**																	
18	**Gross requirements**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
19	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	**Net requirements**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
22	**Planned order receipts**	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
23	**Planned order releases**	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
24																		
25	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
26	**B Deck belt lead time 2**																	
27	**Gross requirements**	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
28	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
29	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
30	**Net requirements**	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
31	**Planned order receipts**	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
32	**Planned order releases**	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0	0	0

Figure 6.4.: The top of the `Explosion` sheet: the structure, the check block, and the first two records. The five blue cells in column C are the quantities per, and every component's gross requirement formula reads them; the other blue cells are the master schedule and the scheduled receipts. Notice the deck belt's gross requirement row against the deck assembly's release row above it. They are the same numbers, because Equation 6.1 multiplies by a quantity per of one.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
34	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
35	**S Spindle assembly lead time 1**																	
36	**Gross requirements**	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
37	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39	**Net requirements**	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
40	**Planned order receipts**	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
41	**Planned order releases**	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
42																		
43	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
44	**BR Bearing lead time 3**																	
45	**Gross requirements**	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
46	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
47	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
48	**Net requirements**	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
49	**Planned order receipts**	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
50	**Planned order releases**	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0	0	0	0
51																		
52	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
53	**H Spindle housing lead time 2**																	
54	**Gross requirements**	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
55	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
56	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
57	**Net requirements**	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
58	**Planned order receipts**	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
59	**Planned order releases**	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0	0	0
60																		
61																		
62																		
63	**TOTALS**																	
64	Orders placed, all five items	57																
65	Setup cost, all five items	$14,500																
66	Carrying cost, all five items	$0																
67	*Lot-for-lot never carries anything from one period to the next, so the carrying cost is zero and every dollar of the total is setup.*																	

Figure 6.5.: The rest of the `Explosion` sheet: the spindle assembly, the bearing, the housing, and the totals. The bearing's gross requirements are the largest numbers on the sheet, reaching 1,980 in period 8, although no deck assembly is ever built in a quantity approaching that. Five bearings per deck and two paths through the structure will do that. The three cells at the foot are the check block's claim made computable: 57 orders, $14,500 of setup, and nothing carried.

The variability coefficient is written as the mean of the squares less the square of the mean, over the square of the mean. The more obvious `SUMPRODUCT((range - mean)^2)` computes the same number and is an array expression, which a reader cannot follow by clicking on it.

The rows are blue, so a reader can type a pattern of their own. Anything that sums to 1,840 and covers the schedule is a plan, and the sheet will price the consequences. Figure 6.6 is Table 6.7 as a reader meets it.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	**What the end item's rule does to everything below it**																
2	*Chapter 6, section 6.5. Every component stays on lot-for-lot. Only the deck assembly's rule changes.*																
3																	
4	**WHAT EACH RULE HAS ARRIVE AT THE DECK ASSEMBLY**																
5	*The twelve scheduled months only. The lead-in of the Explosion sheet is a property of the horizon and not of this co*													**Orders**	**Largest**	**Variability**	**Total**
6	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**placed**	**receipt**	**coefficient**	**belts**
7	**Lot-for-lot**	40	60	120	300	420	260	120	40	0	80	180	220	11	420	0.6191	1,840
8	**Adjusted EOQ**	220	0	0	300	420	260	240	0	0	0	180	220	7	420	0.8459	1,840
9	**Periodic order quantity**	40	60	120	300	420	260	120	40	0	80	180	220	11	420	0.6191	1,840
10	**Least unit cost**	100	0	420	0	420	260	160	0	0	260	0	220	7	420	1.0274	1,840
11	**Part-period balancing**	100	0	120	300	680	0	160	0	0	260	0	220	7	680	1.5463	1,840
12	**Silver-Meal**	100	0	120	300	420	420	0	0	0	80	180	220	8	420	0.9650	1,840
13	**Wagner-Whitin**	100	0	120	300	420	260	160	0	0	80	180	220	9	420	0.6701	1,840
14																	
15	**CHECK with the data as shipped**																
16	*Lot-for-lot passes the schedule through unchanged, so its variability coefficient is the schedule's own, 0.6191.*																
17	*Every rule that batches raises it, as far as 1.5463 under part-period balancing. Every row totals 1,840 belts.*																
18	*Part-period balancing is also the only rule with a receipt larger than 420, and its single lot of 680 is why its coefficient is the highest on the sheet.*																

Figure 6.6.: The `Lumpiness` sheet. Seven rules, twelve months, and four columns of summary. The lead-in of the `Explosion` sheet is left out, because it is a property of the horizon and not of this comparison. Notice the Largest receipt column: six of the seven rules never exceed 420, and part-period balancing's single lot of 680 is why its variability coefficient is the highest on the sheet. Notice also the Total column, which reads 1,840 on every row and is there to be checked.

6.9.4. The Network

`DRP` is the `Explosion` sheet with three level-0 locations instead of one and every quantity per equal to one. It is a separate sheet instead of a note on the first, because seeing the two side by side is what makes Section 6.6.1 believable. Figure 6.7 and Figure 6.8 are that sheet.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	**The distribution network**																	
2	*Chapter 6, section 6.6. The same arithmetic as the Explosion sheet, over a network instead of a bill of material.*																	
3																		
4	**CHECK with the data as shipped**																	
5	*The three regions sell 1,840 belts between them, the same total the deck assembly schedule carries.*																	
6	*Nothing is past due: the horizon leads the first sale by four months, which is the southern region's two plus the centre's two.*																	
7	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
8	**N Northern region transit time 1**																	
9	**Gross requirements**	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
10	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	**Net requirements**	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
13	**Planned order receipts**	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
14	**Planned order releases**	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0	0
15																		
16	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
17	**S Southern region transit time 2**																	
18	**Gross requirements**	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
19	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	**Net requirements**	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
22	**Planned order receipts**	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
23	**Planned order releases**	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0	0	0

Figure 6.7.: The two nearer regions of the `DRP` sheet. Each is planned exactly as an end item is planned on the `Explosion` sheet, and the only word that changes is the one naming the offset: a region waits a transit time where an assembly waits a lead time. Notice that the southern region's releases start a period earlier than the northern region's, although both regions' sales begin in the same month, because its transit time is a month longer.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
25	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
26	**W Western region transit time 1**																	
27	**Gross requirements**	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
28	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
29	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
30	**Net requirements**	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
31	**Planned order receipts**	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
32	**Planned order releases**	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0	0
33																		
34	**Period**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
35	**CW Central warehouse supplier lead time 2**																	
36	**Gross requirements**	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
37	**Scheduled receipts**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38	**Projected on hand**	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39	**Net requirements**	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
40	**Planned order receipts**	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
41	**Planned order releases**	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0	0	0
42																		
43	*The centre's gross requirements are the sum of what the regions release. That is the explosion of section 6.4 with every quantity per equal to one.*																	

Figure 6.8.: The western region and the center. The center's gross requirement row is the three regions' release rows added down the column, and nothing else. Notice period 3, where the center needs ten belts: the southern region alone is asking, because the other two are a month behind it.

6.10. Designing the Software

Section 6.9 built the models as worksheets, and a worksheet shows its own structure. A program does not, so the structure has to be decided and written down. In Section 5.9 we did this for Chapter 5 and found that three heuristics were one procedure. This chapter's design has a different shape, because most of what it needs already exists.

6.10.1. What the Software Must Do

1. Hold a product structure, with quantities per, and assign low-level codes.
2. Hold what the planner knows about an item that is not its requirements.
3. Compute one item's record: net, lot size, offset.
4. Explode a master production schedule through the structure, in the right order.
5. Do the same over a distribution network.
6. Report what a plan costs, by item and in total.

Requirement 5 is the one to watch. It looks like a second feature and Section 6.10.6 argues that it is not one.

6.10.2. Finding the Nouns

A **planned item** is what the item master holds: lead time, opening stock, what is on order, the costs, and the rule. Not its requirements, because for every item but an end item those are computed and change whenever the level above is replanned.

A **bill of material** is the structure. It owns the low-level codes, which are a property of the graph and not of any item in it. An item cannot answer

"what is my low-level code" without seeing every path that reaches it, so the question belongs to the thing that holds the paths.

An **MRP record** is one item's six rows over a horizon.

A **requirements plan** is every item's record, and the order they were computed in.

A **receipt policy** is how much to have arrive, given what must arrive.

Level is the instructive rejection. It reads like a class and the chapter uses the word constantly. It fails because a level has no behavior: it is an integer attached to an item, and the only thing anyone does with it is sort by it.

Figure 6.9 illustrates the five and how they stand to one another. Notice that the bill of material holds the low-level codes and the planned item does not.

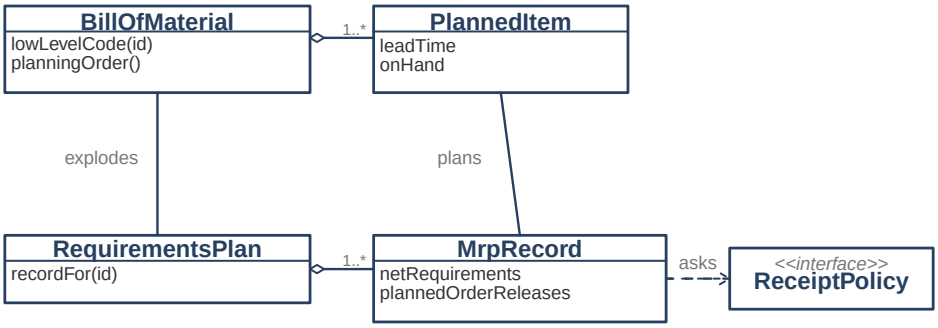


Figure 6.9.: The concepts of this chapter before any thought about implementation. A bill of material relates planned items to one another. A requirements plan holds one record per item, and each record asks a receipt policy how much should arrive. The low-level code sits on the bill of material and not on the item, because an item cannot answer what its own code is without seeing every path that reaches it.

6.10.3. The Join with Chapter 5

The design turns on one sentence: **the net requirement stream is a Section 5.1.1 requirements schedule**. Once that is said, none of Section 5.5 needs an MRP version.

```
/**
 * The net requirements of this item, as a Chapter 5
 ↪  requirements schedule.
 *
 * This is the join between the two chapters.
 ↪  @sec-dls-problem
 * took a requirements schedule as given. Chapter 6 computes
 ↪  one, for every item
 * at every level, and hands it back to @sec-dls-heuristics
 ↪  unchanged.
 */
fun scheduleFor(netRequirements: List<Double>):
 ↪  RequirementsSchedule =
    RequirementsSchedule.constantCosts(
        label = label,
        requirements = netRequirements,
        orderCost = orderCost,
        unitCost = unitCost,
        carryingCharge = carryingCharge,
    )
```

The adapter that uses it is the whole of what Section 5.5 needed to be reused.

```
class PlanTheHorizon(private val rule: LotSizingRule) :
 ↪  ReceiptPolicy {
    override fun receiptsFor(netRequirements: List<Double>,
     ↪  item: PlannedItem): List<Double> {
```

```
        val first = netRequirements.indexOfFirst { it > 0.0 }
        if (first < 0) return netRequirements.map { 0.0 }
        val plan = rule.plan(item.scheduleFor(
         ↪ netRequirements.drop(first)))
        return netRequirements.indices.map {
            if (it < first) 0.0 else plan.orderIn(it - first
             ↪ + 1)
        }
    }
}
```

The `drop` is part of the contract and not a convenience. Section 5.1.2 has inventory starting at zero and period 1 holding an order, so a rule handed a stream that opens with empty periods places a receipt in the first of them and carries it. The empty periods here are the lead-in of Section 6.3.3, and they are not part of the problem the rule is being asked to solve.

6.10.4. One Interface for Two Kinds of Rule

A fixed order quantity decides period by period and looks nowhere. Silver-Meal cannot decide anything until it has seen the whole horizon. They look as though they need two interfaces and they do not, because the net requirement stream is computed *before* any rule runs. That is what makes the whole of it available to a rule that needs the whole of it.

```
fun interface ReceiptPolicy {
    /** Planned order receipts, one per period, covering
     ↪ [netRequirements]. */
    fun receiptsFor(netRequirements: List<Double>, item:
     ↪ PlannedItem): List<Double>
}
```

Figure 6.10 illustrates what implements it. Two of the three are new, and the third is an adapter onto Chapter 5.

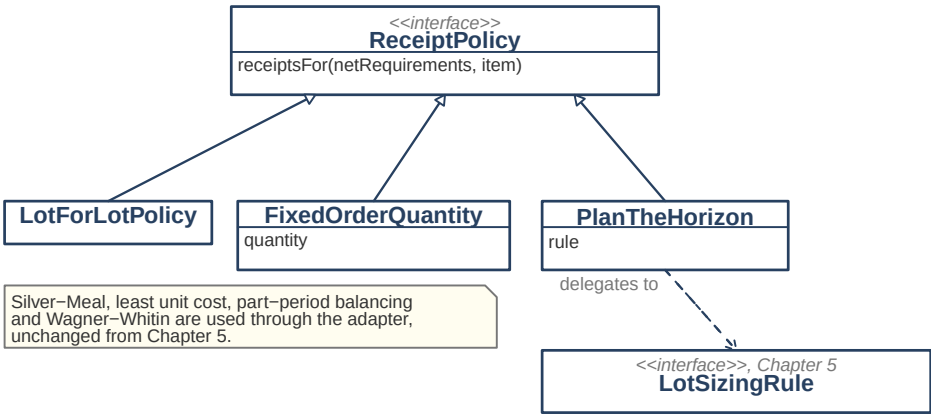


Figure 6.10.: The receipt policies. Lot-for-lot and the fixed order quantity are this chapter's own, and PlanTheHorizon is an adapter that hands the net requirement stream to any rule of Section 5.5. The dashed arrow is the whole of what Chapter 5 needed in order to be reused: no rule in that chapter has an MRP-specific version.

The fixed order quantity is the rule Section 5.2.2 had to exclude, and it is admissible here.

```
/**
 * Order [quantity], or the period's net requirement when
 ↪  that is larger.
 *
 * @sec-dls-threeplans excluded this rule, because a quantity
 ↪  unrelated to the
 * requirements cannot finish the horizon empty. Inside an
 ↪  MRP record it is
 * admissible, and widely used, precisely because the record
 ↪  is not required to
```

```
 * finish empty: what is left over is on hand at the start of
 ↪  the next planning
 * cycle. That is what the projected-on-hand row is for.
 */
class FixedOrderQuantity(private val quantity: Double) :
 ↪  ReceiptPolicy
```

6.10.5. Two Readings of “Net Requirement”

The record computes in two sweeps, and the reason matters, because this is the only place in the package where a name means two things.

The first sweep asks what would have to arrive in each period if nothing were carried beyond the opening stock and the scheduled receipts. That is the stream a policy is asked to cover, and it does not depend on the policy.

The second sweep works out what the policy’s receipts actually leave on hand, and recomputes the row the record *prints*. Those two are usually different. A fixed order quantity that covers three periods at once makes the printed net requirement zero in the two periods after it, and the stream it was given had positive numbers there.

```
val netRequirementsBeforeLotSizing: List<Double>   // what
 ↪  the policy is given
val netRequirements: List<Double>                  // what
 ↪  the record shows
```

Thus, the two names. One property meaning whichever the caller happened to need would put the package into a quiet disagreement with every worked example in the literature, and nothing in the arithmetic would say where the disagreement came from.

6.10.6. A Network Is a Bill of Material

Section 6.6.1 claimed that distribution requirements planning is the same calculation. The design's job is to make that claim structural and not rhetorical, and the way it does so is by not writing a second class.

```
BillOfMaterial(
    locations,
    regions.map { BillOfMaterial.Usage(parent = it, child =
 ↪  "CW", quantity = 1.0) },
)
```

That is the whole of the network's definition. `RequirementsPlan` then assigns low-level codes, which puts the regions at 0 and the center at 1, plans them in that order, and computes the center's gross requirements as the sum over its parents. Figure 6.11 traces one item through that, and nothing in the trace would change if the graph were a network instead of a structure.

The temptation is a `DistributionNetwork` class with `regions` and `center` on it, reading more naturally at the call site. It would duplicate the low-level coding, the planning order and the explosion, and it would then be possible for the two to disagree about a network that is also a structure, and that is a real thing: a warehouse supplying both dealers and an assembly plant.

Section 4.6.3 *does* have a `DistributionNetwork`, and it is not this one. It solves a different problem, the power-of-two reorder intervals of Section 4.6.2, and it holds no time-phased anything. Two classes with the same name in one book is a fair criticism; two classes for two problems is not.

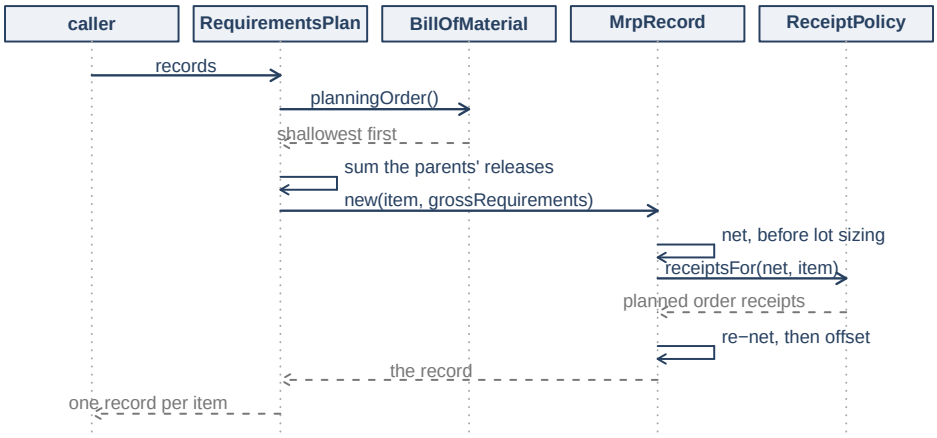


Figure 6.11.: Planning one component. The plan asks the bill of material for the order, then for each item asks the already-completed parents for their releases, builds the record, and the record asks the policy how much should arrive. Notice that the record computes its net requirements BEFORE the policy is consulted, which is what makes the whole stream available to a rule that needs all of it. The dashed arrows are returns.

6.10.7. Running the Chapter's Examples

Every figure in this chapter comes from running this code. The structure of Figure 6.1 is a list of items and a list of usages, and nothing else.

```
val bom = BillOfMaterial(
    listOf(
        PlannedItem("DA", "Mower deck assembly", leadTime =
↪  1, policy = LotForLotPolicy,
            orderCost = 500.0, unitCost = 180.0,
↪  carryingCharge = 0.02),
        PlannedItem("B", "Deck belt", leadTime = 2, policy =
↪  LotForLotPolicy,
            orderCost = 300.0, unitCost = 50.0,
↪  carryingCharge = 0.02),
        PlannedItem("S", "Spindle assembly", leadTime = 1,
↪  policy = LotForLotPolicy,
            orderCost = 250.0, unitCost = 25.0,
↪  carryingCharge = 0.03),
        PlannedItem("H", "Spindle housing", leadTime = 2,
↪  policy = LotForLotPolicy,
            orderCost = 150.0, unitCost = 12.0,
↪  carryingCharge = 0.02),
        PlannedItem("BR", "Bearing", leadTime = 3, policy =
↪  LotForLotPolicy,
            orderCost = 100.0, unitCost = 4.0, carryingCharge
↪  = 0.02),
    ),
    listOf(
        BillOfMaterial.Usage("DA", "B", 1.0),
        BillOfMaterial.Usage("DA", "S", 2.0),
        BillOfMaterial.Usage("DA", "BR", 1.0),
        BillOfMaterial.Usage("S", "H", 1.0),
        BillOfMaterial.Usage("S", "BR", 2.0),
```

```
    ),
)

println(bom.lowLevelCodes.toSortedMap())   // {B=1, BR=2,
 ↪  DA=0, H=2, S=1}
println(bom.planningOrder())               // [DA, B, S, BR,
 ↪  H]
```

Notice that no level was typed anywhere. `lowLevelCodes` is Section 6.2.1 and `planningOrder` is the order Section 6.4 plans in, and both are read off the usages, which is the argument of Section 6.10.2 made operational.

A master production schedule drives it. The five zeros in front are the lead-in of Section 6.3.3, and `RequirementsPlan` takes the schedule as a map because a structure may have more than one end item.

```
val schedule = List(5) { 0.0 } +
    listOf(40.0, 60.0, 120.0, 300.0, 420.0, 260.0, 120.0,
 ↪  40.0, 0.0, 80.0, 180.0, 220.0)
val plan = RequirementsPlan(bom, mapOf("DA" to schedule))

val bearing = plan.recordFor("BR")
println(bearing.grossRequirements.map { it.toInt() })
println(bearing.plannedOrderReleases.map { it.toInt() })
println("%d orders, %.0f setup, %.0f carrying"
    .format(bearing.orderCount, bearing.setupCost,
     ↪  bearing.carryingCost))
println("%.0f past due".format(plan.pastDueReleases))
```

```
[0, 0, 0, 160, 280, 540, 1320, 1980, 1460, 740, 280, 40, 320, 800, 1060, 220, 0]
[160, 280, 540, 1320, 1980, 1460, 740, 280, 40, 320, 800, 1060, 220, 0, 0, 0, 0]
13 orders, 1300 setup, 0 carrying
0 past due
```

That first row is Table 6.6 and the second is the bearing's release row in Figure 6.5. Notice `pastDueReleases`, which is how a horizon is checked without reading the record: it is zero here, and with four zeros in front instead of five it is 160, because the bearing's first release would fall before period 1.

Changing a rule means changing one item's policy, and `PlanTheHorizon` is what admits a rule of Section 5.5. Putting the deck assembly on Silver-Meal, with every component still on lot-for-lot, is one row of Table 6.7.

```
PlannedItem("DA", "Mower deck assembly", leadTime = 1,
    policy = PlanTheHorizon(SilverMeal),
    orderCost = 500.0, unitCost = 180.0, carryingCharge =
 ↪  0.02)
```

```
println("%.0f at the deck assembly, %.0f in total"
    .format(plan.recordFor("DA").relevantCost,
     ↪  plan.relevantCost))
// 4936 at the deck assembly, 11636 in total
```

That is against $14,500 with lot-for-lot everywhere, and Section 6.5.1 is that comparison run once per rule. Notice that `relevantCost` is available on the record and on the plan, which is what lets the experiment separate what the rule saved at the deck assembly from what it cost below.

The network of Section 6.6 uses the same two classes, with the regions as parents and every quantity per equal to one.

```
val demand = mapOf(
    "N" to listOf(20.0, 30.0, 60.0, 140.0, 180.0, 110.0,
     ↪  50.0, 20.0, 0.0, 40.0, 80.0, 100.0),
    "S" to listOf(10.0, 20.0, 40.0, 100.0, 160.0, 100.0,
     ↪  50.0, 10.0, 0.0, 20.0, 60.0, 70.0),
```

```
    "W" to listOf(10.0, 10.0, 20.0, 60.0, 80.0, 50.0, 20.0,
     ↪  10.0, 0.0, 20.0, 40.0, 50.0),
)
val regions = listOf("N" to 1, "S" to 2, "W" to 1)
val network = BillOfMaterial(
    regions.map { (id, transit) ->
        PlannedItem(id, "$id region", leadTime = transit,
 ↪  policy = LotForLotPolicy,
            orderCost = 200.0, unitCost = 50.0,
 ↪  carryingCharge = 0.02)
    } + PlannedItem("CW", "Central warehouse", leadTime = 2,
     ↪  policy = LotForLotPolicy,
        orderCost = 400.0, unitCost = 50.0, carryingCharge =
 ↪  0.02),
    regions.map { (id, _) -> BillOfMaterial.Usage(parent =
 ↪  id, child = "CW", quantity = 1.0) },
)
val drp = RequirementsPlan(network, demand.mapValues {
 ↪  List(4) { 0.0 } + it.value })

println(network.lowLevelCodes.toSortedMap())   // {CW=1, N=0,
 ↪  S=0, W=0}
println(network.planningOrder())               // [N, S, W,
 ↪  CW]
println(drp.recordFor("CW").grossRequirements.map {
 ↪  it.toInt() })
// [0, 0, 10, 50, 80, 180, 360, 360, 210, 80, 30, 20, 120,
 ↪  190, 150, 0]
```

The last row is the center's gross requirement row in Figure 6.8. Notice the lead-in is four here and five above, because Section 6.6 has a shallower graph. Notice also what is absent: no class in this package is told that the graph is a network, and the only thing that says so is the word `region` in

a label.

That is the whole of the interface. Section 6.11 collects what the chapter argued with it.

6.11. Summary

Most items in a manufacturing business have demand that is implied by the firm's own plans. Calculating it from those plans beats forecasting it, and the calculation is a record per item: net what is needed against what is already there, lot size the result, and move it back by the lead time.

Two ideas make the calculation run over a whole structure. A low-level code says when an item may be planned, and that is after every parent of it. And a component's gross requirements are its parents' planned order *releases* times the quantity per, and that is Equation 6.1.

The horizon has to lead the first requirement by the cumulative lead time down the deepest path, or the plan reports orders it has no time to place.

Chapter 5's rules run inside a record unchanged, and using them raises a question Chapter 5 could not ask. A parent's rule decides what its components are asked for, so it decides their problem. The cost below a level rises with how often the level above orders, and the rule that is cheapest at a level is not the rule that is cheapest for the product. On the deck assembly that error costs 7.59%.

A distribution network is the same calculation with every quantity per equal to one, and it has the same result: batching at one stage amplifies what the next stage sees, by a factor of 1.4 to 2.6 here. That amplification costs nothing while demand is known. Thus, it belongs to Chapter 9 and not to this chapter.

Everything here assumes infinite capacity, fixed lead times, accurate records and known demand. The last of those is what the rest of the book removes.

Table 6.10 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 6.10.: Notation introduced in this chapter.

Symbol	Meaning
G_t	gross requirement in period t
SR_t	scheduled receipt in period t
I_t	projected on hand at the end of period t
NR_t	net requirement in period t
POR_t	planned order receipt in period t
Rel_t	planned order release in period t
L	lead time, in periods
SS	safety stock
q_{ij}	units of j in one i

6.12. Exercises

Unless an exercise says otherwise, use these conventions. Periods are numbered from one. Projected on hand is an end-of-period figure. A net requirement is never negative. A planned order release falls in the period the receipt is due less the lead time, and a release that would fall before period 1 is past due and is reported instead of placed. Report costs to the nearest cent and the variability coefficient to four decimal places.

6.12.1. Terminology and Concepts

Exercise 6.1. Define each of the following in one sentence, and say which section introduced it.

a. dependent demand
b. quantity per
c. low-level code
d. gross requirement
e. net requirement
f. planned order release
g. past-due release
h. cumulative lead time
i. firm planned order
j. safety lead time

Exercise 6.2. State whether each is true or false and give one sentence of justification.

a. A component's gross requirements are its parents' planned order receipts times the quantity per.
b. An item that appears at two levels of a bill of material is planned twice.
c. Lot-for-lot at a parent leaves its components' demand pattern unchanged.
d. The variability coefficient of a component's demand tells you what the component's plan will cost.
e. If every item uses the rule that is cheapest for it, the plan is the cheapest plan for the product.
f. A past-due release means the inventory records are wrong.
g. The bullwhip effect is caused by the network having several stages.
h. Safety stock and safety lead time buffer against the same thing.

Exercise 6.3. Choose the best answer.

1. Low-level codes exist so that (a) items can be sorted for printing; (b) an item is planned only after every parent has asked for it; (c) the bill of material can be drawn as a tree; (d) lead times can be added up.
2. The max in Equation 6.3 is there to (a) keep the arithmetic positive; (b) stop a surplus in one period being treated as a requirement in the next; (c) round the order quantity; (d) prevent division by zero.
3. A planning horizon must lead the first requirement by at least (a) the longest single lead time; (b) the average lead time; (c) the cumulative lead time down the deepest path; (d) one period.
4. The cost below a level rises with (a) the variability coefficient of the demand the components see; (b) how often the level above orders; (c) the number of levels;

   (d) the quantity per.

5. Distribution requirements planning differs from material requirements planning in that (a) it uses a different netting formula; (b) every quantity per is one and the parent is a customer; (c) it cannot use lot sizing; (d) it has no lead times.

6.12.2. Working the Records by Hand

Exercise 6.4. The deck belt has the twelve-month gross requirements of Table 5.1, a lead time of two months, 60 belts on hand, and a scheduled receipt of 200 belts due in month 3.

a. Complete the record under lot-for-lot: net requirements, projected on hand, planned order receipts, and planned order releases.
b. How many belts are past due, and why?
c. Now use a fixed order quantity of 250, meaning that every receipt is 250 belts or the net requirement if that is larger. Complete the record again.

d. Compare the two on orders placed and on unit-months carried. Which would you use, and what would you need to know to decide?

Exercise 6.5. A product P is assembled from two A, one B and one W. Each A contains one C and three W. Each B contains two C. Each W contains two C. Lead times are P one period, A two, B one, W two and C one.

a. Draw the product structure and the indented list.
b. Assign every item its low-level code.
c. Give the order the items must be planned in.
d. C appears at three different levels. Say what would go wrong if it were planned at the shallowest of them.
e. Compute the cumulative lead time, and say how long a horizon must be to plan a twelve-period schedule for P.

Exercise 6.6. Using the structure of Exercise 6.5, suppose the master production schedule calls for 100 P in period 10 and 150 P in period 13, with nothing on hand or on order anywhere and lot-for-lot at every item.

a. Complete the record for P and read off its planned order releases.
b. Explode to A, B and W, and complete their records.
c. Explode to C. Show the three contributions separately before summing them.
d. In which period is the earliest order released, and for which item?

Exercise 6.7. Table 6.3 reports what happens to the deck assembly structure when the horizon is too short.

a. Reproduce the 160 units that are past due when the schedule is given four empty months instead of five. Which item are they, and which path through the structure produced them?
b. The bearing has a lead time of three months. If it were reduced to two, how many empty months would the horizon need?

c. A planner proposes to solve past-due orders by starting the schedule later rather than by lengthening the horizon. Say what is wrong with that.

6.12.3. Modeling Problems

Exercise 6.8. Table 6.7 holds every component at lot-for-lot and varies only the deck assembly's rule.

a. Explain in your own words why the Cost below column rises with the Orders column.
b. The periodic order quantity and lot-for-lot give identical results. Compute the deck assembly's economic order quantity from Equation 3.17 and its time supply, and explain the coincidence.
c. Wagner-Whitin is optimal at the deck assembly and third worst over the structure. State precisely what Section 5.4.2 does and does not guarantee.
d. Propose a procedure that would find the rule minimizing total cost over the structure, and say why it is not what an MRP system does.

Exercise 6.9. Take the three regions of Example 6.3.

a. Verify that the three regional demands sum to the 1,840 belts of Table 5.1.
b. Reproduce the center's gross requirements in periods 3 through 8.
c. The southern region's transit time is two months and the others' are one. Change the south to one month and recompute the center's gross requirements. Does the total change? Does the variability coefficient, measured as in Table 6.9 over the periods in which the center has a requirement?
d. Table 6.9 reports that every batching rule at the regions is cheaper than lot-for-lot, at the regions and at the center both. Explain why, and then say what would have to be true about the world for the amplification to cost something.

Exercise 6.10. The bearing has a lead time of three months and a supplier who is occasionally a month late.

a. Compare holding one month's average requirement as safety stock against planning the bearing with a safety lead time of one month. Which protects against a late delivery, and which against a larger requirement than planned?
b. Safety stock enters Equation 6.3 as an addition to the gross requirement. Write the corresponding modification for safety lead time and say which row it changes.
c. A colleague proposes safety stock at every level of the structure. Say what that does to the total stock held, and why buffering at one level is usually preferred.

6.12.4. Using the Workbook

Exercise 6.11. Open `Chapter6Models.xlsx`.

a. On the `Record` sheet, change the opening stock to 200 and describe what happens to each of the six rows.
b. On the `Explosion` sheet, change the quantity per for spindle assemblies from 2 to

    3. Report the new gross requirements for the bearing and explain the change.

c. On the `Lumpiness` sheet, type a plan of your own into the last row: any pattern that sums to 1,840 and covers the schedule. Can you find one with a lower variability coefficient than lot-for-lot's 0.6191? Explain your answer.
d. On the `DRP` sheet, give the southern region the one-month transit time of the other two by pointing each of its planned order releases at the planned order receipt one period later instead of two. In a spare cell, compute the center's variability coefficient over the periods in

which it has a requirement, before and after, with the formula the `Lumpiness` sheet uses. What happens to it, and what does that tell you about Section 6.6.2?

6.12.5. Using the Software

Exercise 6.12. The code of Section 6.10 is in `code/`, package `inventory.requirementsplanning`. Section 6.10.7 is the starting point: it builds the structure, plans it, and reads a record. Report the code you wrote alongside each answer.

a. Build the structure of Figure 6.1 and print `lowLevelCodes` and `planningOrder()`. Now suppose the belt runs over an idler pulley that carries one of the same bearing, which is `BillOfMaterial.Usage("B", "BR", 1.0)`. Add it and print both again, then print the bearing's gross requirements before and after. Finally, vary the number of empty months in front of the master schedule and report the shortest lead-in that leaves `pastDueReleases` at zero. It is not the five of Section 6.3.3. Explain why it moved when no low-level code did.

b. Reproduce Table 6.7. Give the deck assembly `PlanTheHorizon(rule)`, leave every component on `LotForLotPolicy`, and loop over the seven rules, reporting `orderCount` and `relevantCost` at the deck assembly, the same total over every other item, and the variability coefficient of the deck assembly's receipts over the twelve scheduled months. Which rule makes the deck assembly's receipts lumpiest, and is it the rule that costs the most?

c. Bearings are bought in boxes of 500. Implement a `ReceiptPolicy` whose receipt in each period is the smallest multiple of the box size that covers what is still short, where what is still short is that period's net requirement less anything left over from earlier receipts. Give it to the bearing, leave everything else on lot-for-lot, and report the bearing's receipts, `orderCount`, `setupCost`, `carryingCost`, and

the last entry of `projectedOnHand`. Is the plan cheaper? Which assumption of Section 5.1.2 does that last entry break, and why is breaking it admissible inside a record?

d. Section 6.5.1 holds every component on lot-for-lot. Put every component on `PlanTheHorizon(WagnerWhitin)` instead and run the comparison of part (b) again. Report the table. Does the cost below the deck assembly still rise with the deck assembly's order count? Does choosing the rule on the deck assembly alone still pick the wrong one, and what does the error cost now?
e. The distributor opens an eastern region whose transit time is three months and whose sales are those of the northern region, at the same costs as the other regions. Add it to the network of Section 6.6, report the center's gross requirements, and compare the variability coefficient of what the center sees against the 0.6334 of Section 6.6.2. Explain the direction it moved.

6.12.6. From the Literature

Exercise 6.13. Requirements planning was promoted in the 1970s as a replacement for order-point systems, and the claims made for it were larger than what it delivered.

a. Using Section 6.8, give two assumptions whose failure would account for a disappointing implementation.
b. Section 2.3 reports what inventory record accuracy actually is in practice. Explain why an order-point system tolerates a bad record better than a requirements planning system does.
c. Find one account of an MRP implementation in the literature and report what its authors say went wrong. Classify their explanation against the five assumptions of Section 6.8.

Part II.

Stochastic Models

7. Single-Period Stochastic Inventory

Learning objectives

After reading this chapter you should be able to:

- recognize a single-period problem and say why it is not a one-period version of a multi-period one
- work the ordering decision by hand as a marginal analysis
- characterize a demand history and fit a distribution to it
- formulate the newsvendor problem in both its cost and its profit forms
- derive and interpret the critical ratio
- solve the model for continuous and for discrete demand
- extend the model to a penalty cost and to stock already on hand
- estimate the answer by simulation, and say when that is worth doing
- build the model in a worksheet and in code, and check each against the other

7.1. One Order, One Chance

Every model so far has known its demand. Chapter 3 and Chapter 4 took a constant rate, Chapter 5 let the rate vary by period and still knew it, and Chapter 6 computed it from a production plan. Assumption 5 of

Section 6.8 is the one this chapter removes. From here on demand is a random variable, and the decision is made before it is observed.

We begin with the simplest such problem, in which the decision is made **once**.

Example 7.1 (The last order for the mower deck belt)**.** The distributor of Chapter 5 has stocked the mower deck belt for years. The deck it fits is now out of production, so the seasonal pattern of Table 5.1 is over and what remains is steady replacement demand from owners of machines already in the field. The supplier is discontinuing the belt and has offered one last order.

What is stocked? One item, the belt.

What is the demand process? Weekly demand is random and no longer seasonal. It will run for about fourteen more weeks, until a redesigned deck reaches the dealers and the belt becomes unsellable.

When is inventory reviewed? Once. There is one decision and no second chance.

What triggers replenishment, and how much? Nothing triggers it. The distributor names a quantity now.

What happens to unmet demand? It is lost. A dealer who cannot get the belt from this distributor gets it elsewhere.

What costs are incurred, and when? The belt costs \$50 and sells for \$95. Belts left when the deck is withdrawn go to a scrap dealer at \$12.

Three things about Example 7.1 distinguish it from everything before it.

There is no ordering cost in the model. Recall from Table 1.10 that k is the price of placing an order, and that the models of Chapter 3 through Chapter 6 trade it against holding. Here exactly one order is placed whatever its size, so k is the same under every decision. A constant added

to every alternative cannot change which alternative is least, which is the argument of Section 1.4.1. Thus, k drops out, and with it the entire question those chapters were about.

Leftover stock has no future value. Recall from Section 1.2.7 that a single-period problem is not a multi-period problem with the number of periods set to one. In a multi-period problem a unit left at the end of a period is available at the start of the next, so the cost of having too many is a holding charge for one period. Here it is the whole loss on the unit, \$50 paid against \$12 recovered. That is why the trade-off is overage against underage rather than holding against ordering.

The two errors are priced differently, and neither is free. Order too many and the distributor loses \$38 a belt. Order too few and it loses the \$45 margin on each belt it could have sold. Notice that both are opportunity costs measured against the decision that could have been made, and that neither appears in any ledger. Section 1.4.2 is why.

7.1.1. Demand Is Not Sales

One distinction has to be made before any data are used, because it is the commonest error in this material and it is invisible in the answer.

- Let **demand** represent what customers would have bought.
- Let **sales** represent what they did buy.

The two differ whenever the shelf was empty. A week that sold 40 belts because 40 were on the shelf and a week that sold 40 because 40 were wanted look identical in a sales record. That is, sales are demand **censored** at whatever was available, and a history of sales understates both the average and the spread. It understates them most in the weeks when the item mattered, because those are the weeks it ran out.

Recall from Section 2.3 that a stock record is only as good as the transactions behind it. This is the same point about a different record. An

organization that wants to model demand has to record the demand it could not fill, and most do not. Where that record does not exist, the fitted distribution of Section 7.3 is biased low and every quantity computed from it is too small. You should say so when you report the answer, rather than let the arithmetic imply a precision the data do not carry.

The next section makes the decision without any distribution theory at all.

7.2. Working the Decision by Hand

Before any calculus, we can make the decision with arithmetic a planner would do on a whiteboard. The method is worth more than the answer, because it produces the rule that the rest of the chapter derives formally.

Let's start with a sketch of demand rather than a fitted distribution. Suppose the distributor, asked for a judgment, offers five outcomes for total remaining demand and a probability for each.

Table 7.1.: A five-point sketch of remaining demand for the belt. The mean is 485 belts. The sketch is coarse: the argument below needs the shape of demand and not its detail.

Demand x, belts	300	400	500	600	700
$P\{X = x\}$	0.15	0.25	0.30	0.20	0.10
$P\{X \le x\}$	0.15	0.40	0.70	0.90	1.00

The decision is how many belts to buy, in hundreds. Rather than evaluate every candidate, we ask a smaller question. **Given that we have decided on some quantity, is one more hundred belts worth buying?**

Buying one more hundred pays off when demand turns out to exceed the quantity we already have, which happens with probability $P\{X \ge Q+100\}$,

and each of those belts earns the \$45 margin. It costs when demand falls at or below the quantity we already have, which happens with probability $P\{X \leq Q\}$, and each unsold belt loses \$38. The extra hundred is worth buying when

$$45 \times 100 \times P\{X \geq Q + 100\} \; > \; 38 \times 100 \times P\{X \leq Q\} \qquad (7.1)$$

Table 7.2 works Equation 7.1 one step at a time.

Step 1, from 300 to 400. Demand reaches 400 or more with probability 0.85, so the gain is $45(100)(0.85) = \$3{,}825$. Demand stops at 300 with probability 0.15, so the loss is $38(100)(0.15) = \$570$. The step is worth \$3,255. Take it.

Step 2, from 400 to 500. Demand reaches 500 or more with probability 0.60, giving $45(100)(0.60) = \$2{,}700$. Demand stops at 400 or below with probability 0.40, giving $38(100)(0.40) = \$1{,}520$. The step is worth \$1,180. Take it.

Step 3, from 500 to 600. Demand reaches 600 or more with probability 0.30, giving \$1,350. Demand stops at 500 or below with probability 0.70, giving \$2,660. The step **loses** \$1,310. Stop.

Table 7.2.: The marginal analysis, one hundred belts at a time. Notice that the net column changes sign once and never changes back, which is what makes stopping at the first negative step correct rather than merely convenient.

From	To	$P\{X \geq$ to$\}$	Gain	$P\{X \leq$ from$\}$	Loss	Net
300	400	0.85	\$3,825	0.15	\$570	**+\$3,255**
400	500	0.60	\$2,700	0.40	\$1,520	**+\$1,180**
500	600	0.30	\$1,350	0.70	\$2,660	-\$1,310
600	700	0.10	\$450	0.90	\$3,420	-\$2,970

Order 500 belts. Notice that 500 is *more* than the mean of 485. The distributor should plan to be left with belts more often than it plans to run out, because running out costs more per belt than scrapping does.

That the net column changes sign exactly once is not luck. The outcomes of Table 7.1 are a hundred belts apart, so demand at least $Q + 100$ is demand above Q, and $P\{X \geq Q + 100\} = 1 - P\{X \leq Q\}$. Dividing Equation 7.1 by 100 and substituting, the step is worth taking while $45\,(1 - P\{X \leq Q\}) > 38\,P\{X \leq Q\}$, that is, while

$$P\{X \leq Q\} < \frac{45}{45 + 38} = \frac{c_u}{c_u + c_o} = 0.5422 \tag{7.2}$$

and the left side is a cumulative distribution function, which never decreases. Thus, once the test fails it fails for every larger quantity, and the rule is to order the smallest quantity whose cumulative probability reaches 0.5422. From Table 7.1 that is 500 belts, since $F(400) = 0.40$ is short of it and $F(500) = 0.70$ clears it.

That ratio is the answer to the whole chapter, and Section 7.6 derives it properly.

A check, and a reason to trust the shortcut. Costing all five candidates directly should agree, and it does.

Table 7.3.: Expected profit at each candidate quantity, computed from Table 7.1 without any shortcut.

Q	300	400	500	600	700
Expected profit	\$13,500	\$16,755	**\$17,935**	\$16,625	\$13,655

Table 7.3 peaks at 500, which is where Table 7.2 stopped and where Equation 7.2 pointed. Three routes, one answer. That check is worth

performing the first time you use marginal analysis on a new problem, because the marginal argument is only valid when the expected profit has a single peak, and the direct calculation is what confirms it.

Notice what the hand method cost. Five outcomes, four steps, no distribution and no calculus. What it cannot do is tell the distributor anything between 400 and 500 belts, and the real decision is a number of belts rather than a number of hundreds. For that we need a distribution, and that is the next two sections.

7.3. Getting a Demand Distribution

Every model from here to the end of the book takes a demand distribution as an input, and the distribution carries more of the answer than the policy formula built on top of it does. This section fits one.

The subject has a book of its own. Appendix B of *Simulation Modeling using the KSL* (Rossetti 2026) gives the procedure in full, and Chapter 2 of the same text shows how to carry it out with the library this book's code uses. What follows is that procedure applied to an inventory problem, with the one difference that matters here made explicit: demand for a spare part is a **count**, so every tool below is a discrete one.

The data. The distributor has 104 weeks of demand for the belt, recorded after the deck went out of production. They are in `data/ch7-weekly-demand.csv` in the course code project, two columns and a header:

```
week,demand
1,94
2,67
3,31
```

Read them the way Section C.4 assumes they arrive, as a file whose contents you did not choose:

```
val df = DataFrame.readCsv(
    File("data/ch7-weekly-demand.csv"),
    colTypes = mapOf("week" to ColType.Int, "demand" to
 ↪  ColType.Int)
)
val history: IntArray = df.getColumn("demand").toList().map
 ↪  { it as Int }.toIntArray()
```

Notice that the column types are named rather than left to be inferred. That is the practice Section 2.4.4 of (Rossetti 2026) follows, and the reason is that a column of counts silently read as text, or as real numbers, fails later and somewhere else.

7.3.1. Look Before You Model

Appendix B of (Rossetti 2026) opens its fitting procedure by plotting the data. A summary statistic answers a question you already thought to ask. A picture shows you the thing you did not.

Three plots answer three different questions, and the KSL draws all three.

```
val h = Histogram(Histogram.createBreakPoints(0.0, 12, 10.0),
 ↪  "Weekly demand")
h.collect(history.toDoubles())
h.histogramPlot().showInBrowser()

ObservationsPlot(history.toDoubles()).showInBrowser()
ACFPlot(history.toDoubles()).showInBrowser()
```

What shape is it? Figure 7.1 is the histogram, on bins of ten belts. The break points are given rather than recommended, because ten belts to a bin is what the `Data` sheet of `Chapter7Models.xlsx` uses and a figure that disagrees with the spreadsheet beside it is worse than no figure.

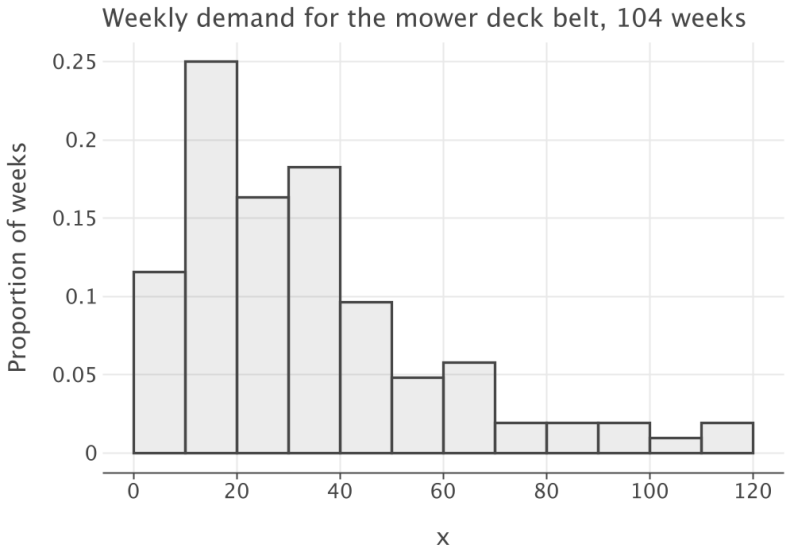


Figure 7.1.: Weekly demand for the belt over 104 weeks, in bins of ten. Each bin is closed below and open above, so the tallest bar counts the 26 weeks that sold 10 through 19 belts. Notice the shape: a peak well to the left, a long thin tail reaching past 110, and no suggestion of symmetry anywhere in it. That single picture has already ruled out the normal distribution.

Is it the same process throughout? Figure 7.2 plots the weeks in the order they occurred. What you are looking for is a trend, a level shift, or a season, and what is there is none of those: the series wanders about a constant level for two years. That matters because the season really is over, which is what Example 7.1 claimed and what this plot lets a reader check rather than accept.

Are the weeks independent? Figure 7.3 is the autocorrelation plot. Every lag falls inside the band, including the first, whose correlation is 0.1423 against a band of about $\pm 1.96/\sqrt{104} = \pm 0.19$. Recall from Section B.5.5 that autocorrelated demand breaks the variance of a sum. Section 7.7 is about to add fourteen of these weeks together and treat their variances as adding, so this is the plot that licenses the arithmetic two sections from now.

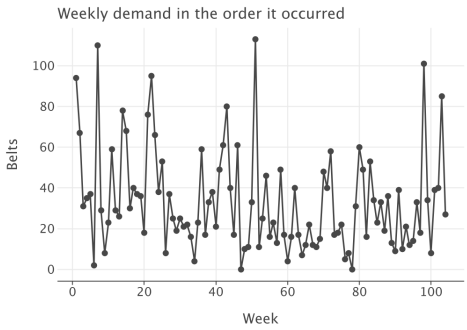


Figure 7.2.: The weeks in the order they occurred.

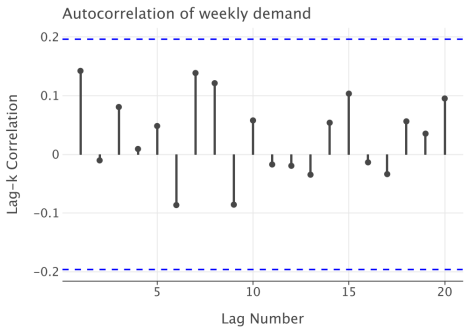


Figure 7.3.: Autocorrelation through twenty lags.

Taken together the three plots say the weeks are independent, identically distributed, and strongly right-skewed. That is the finding that lets the rest of the section treat the 104 numbers as 104 draws from one distribution.

7.3.2. Summarize It

Now the statistics, which the KSL accumulates as the data are collected.

```
val s = Statistic("Weekly demand", history.toDoubles())
val b = BoxPlotSummary(history.toDoubles(), "Weekly demand")
println(s)
println(b)
```

Table 7.4.: Summary of the 104 weeks. `Statistic` carries the first four moments and the lag one correlation; `BoxPlotSummary` carries the quartiles.

Statistic	Value
Weeks observed	104
Mean	33.2404
Variance	613.0387
Standard deviation	24.7596
Coefficient of variation	0.7449
Variance to mean ratio	18.4426
Skewness	1.2630
Smallest week	0
First quartile	16.0
Median	26.5
Third quartile	40.0
Largest week	113
Weeks with no demand	2

Read three things off Table 7.4 before going further.

The median sits well below the mean. A median of 26.5 against a mean of 33.24 says the distribution is right-skewed: most weeks are quiet and a few are very large. The skewness of 1.2630 is the same statement with a number attached, and Figure 7.1 is the same statement with a picture attached. Three routes, one finding.

Demand is a count. The weeks are whole belts, two of them are zero, and the quantity has no meaning between whole numbers. Thus, the families to consider are the discrete ones of Section C.1 and not the continuous ones of Section C.2.

For a count the diagnostic is the variance to mean ratio. Equation C.1 rather than the coefficient of variation, because the ratio is what each discrete family fixes or bounds. Here

$$VMR = \frac{613.0387}{33.2404} = 18.44$$

7.3.3. Hypothesize, Estimate, Test

Step 1, hypothesize. Equation C.1 does most of this step by itself. Recall from Section C.1.1 that the Poisson fixes $VMR = 1$, and from Section C.1.3 that the binomial forces $VMR < 1$. A ratio of 18.44 is near neither, and no choice of parameter repairs that, because in both families the variance is decided the moment the mean is estimated. What remains is Section C.1.2, the negative binomial, which admits any $VMR > 1$ and which is the Poisson with its constant-rate assumption relaxed. That is a plausible story for a spare part: the rate at which mower decks need belts is not the same in every week.

Step 2, estimate. The KSL estimates the parameters of every discrete family it knows in one call. `PMFModeler` is the discrete counterpart of the `PDFModeler` of Section 2.4.2 of (Rossetti 2026), and it is deliberately narrower: there are fewer discrete families, so it estimates parameters and leaves the choosing to you.

```
val modeler = PMFModeler(history)
for (result in
 ↪  modeler.estimateParameters(modeler.defaultEstimators)) {
    println(if (result.success) "${result.parameters}" else
 ↪  "FAILED: ${result.message}")
}
```

```
RV Type = NegativeBinomial
```

```
Double Parameters {probOfSuccess=0.05422232292974839, numSuccesses=1.905702484443969
RV Type = Poisson
Double Parameters {mean=33.24038461538462}
FAILED: Cannot match moments when sample average <= estimated variance
RV Type = Binomial
Double Parameters {probOfSuccess=0.29133420825260514, numTrials=114.09708737864078}
```

Read that output carefully, because the most useful line in it is the failure. The binomial estimator that matches moments **refuses to return an answer**, and the reason it gives is Equation C.1 in words: a binomial cannot have a variance above its mean. That is the software performing Step 1 rather than the analyst. The binomial that did return an answer did so by matching the largest observation instead of the moments, and it reports 114.1 trials, a fractional number of trials for a quantity that has no trials in it. Notice that an estimate being returned is not the same thing as a family being appropriate.

The negative binomial estimate is the method of moments of Section C.4, and it is worth confirming by hand. Matching Equation C.3 to the sample mean $\bar{x}$ and the sample variance s^2, we get

$$\hat{p} = \frac{\bar{x}}{s^2} = \frac{33.2404}{613.0387} = 0.054222, \qquad \hat{r} = \frac{\bar{x}^2}{s^2 - \bar{x}} = \frac{33.2404^2}{613.0387 - 33.2404} = 1.9057 \tag{7.3}$$

which is what the KSL reported. Notice that $1/\hat{p} = 18.44$ is the variance to mean ratio we started with, so the fitted $\hat{p}$ carries no information the summary did not already have. That is true of the method of moments generally: the fit is a restatement of two summary numbers in the parameters of a family.

The KSL uses the success convention of Equation C.3, so `probOfSuccess` is p and the mean is $r(1-p)/p$. That agrees with the check above. The

warning in Section C.1.2 is not hypothetical; you should confirm the convention every time before passing a parameter to a routine.

Step 3, test. For a discrete model the KSL performs a chi-squared goodness of fit test, forming bins of approximately equal probability.

```
val fitted = NegativeBinomial(0.054222, 1.9057)
val gof = DiscretePMFGoodnessOfFit(
    history.toDoubles(), fitted, numEstimatedParameters = 2,
    breakPoints =
 ↪  PMFModeler.makeZeroToInfinityBreakPoints(history.size,
 ↪  fitted)
)
println(gof.chiSquaredTestResults())
```

Table 7.5.: The chi-squared goodness of fit test on the 104 weeks, for the two families Equation C.1 left in contention. Notice that the Poisson is not rejected narrowly. Its test statistic is more than twenty times the negative binomial's.

Model	χ^2	Degrees of freedom	p-value	At the 0.05 level
Negative binomial, $\hat{p} = 0.0542$, $\hat{r} = 1.9057$	23.35	17	0.1382	do not reject
Poisson, $\hat{\lambda} = 33.2404$	571.98	16	0.0000	reject

Table 7.5 is the confirmation, not the decision. The decision was made by the variance to mean ratio, and a test that had contradicted it would have

been telling us that something is wrong with the data rather than with Equation C.1.

Figure 7.4 is the same conclusion drawn rather than tabulated. The KSL calls this a PMF comparison plot and Section 2.4.4 of (Rossetti 2026) uses one at the same point in the procedure.

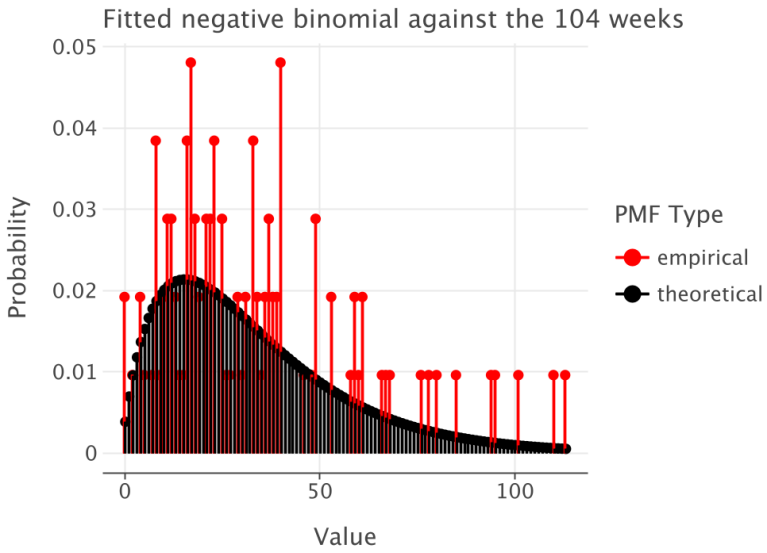


Figure 7.4.: The fitted negative binomial against the 104 weeks. The empirical mass function is spiky because it has to be: 104 observations spread over 54 distinct values put a mass of 1/104 or 2/104 on most of them. That is exactly why Table 7.5 bins before it tests. What to read here is whether the black curve runs through the middle of the red forest, which it does.

7.3.4. What the Test Does Not Settle

A goodness of fit test asks whether the data are *inconsistent* with a family. Failing to reject is not proof. With 104 observations spread over twenty bins, several of the bins carry an expected count below five, which is the condition under which the chi-squared approximation is least trustworthy, and the KSL flags each of them. The test and the ratio narrow the field, and the choice among what survives is yours to defend. Where two survivors disagree enough to change the decision, Section 7.7 shows what to do: compute the answer under both and report the spread.

What this section did not do. It did not deseasonalize, because Figure 7.2 shows nothing to remove and the season is over. It did not forecast, which is a different job from fitting a distribution and is outside this book. And it did not correct for censoring, because the distributor does not record lost sales, which is the limitation Section 7.1.1 warned about and which no amount of care here repairs.

It also did not have to deal with an item whose history is mostly zeros. The belt sold in 102 of its 104 weeks. Section 7.4 takes up the items that do not.

7.4. Intermittent and Slow-Moving Demand

The belt is demanded in almost every week: 2 of its 104 weeks were empty. Many items are not like that, and a history in which most periods are empty needs a different treatment.

Recall from Example 2.4 that a demand history is classified by two statistics: the average interval between demands, ADI, and the squared coefficient of variation of the non-zero sizes, CV^2. An item with $ADI > 1.32$ is **intermittent**, and one that is also variable in size, $CV^2 > 0.49$, is **lumpy**. Those two cut-offs are not conventions; Syntetos et al. (2005) derive them as the points at which one forecasting method overtakes another,

which is why they are the oddly specific numbers they are. For the belt, $ADI = 104/102 = 1.02$, comfortably under the threshold, which is the check that licenses everything Section 7.3 did.

Two things go wrong when an intermittent series is handed to Section 7.3 unchanged.

A single family has to explain two different things. The distribution of period demand for an intermittent item is a mixture of a point mass at zero and a distribution of sizes. Fitting one continuous family to it forces that family to account for both how often demand happens and how large it is, and no family in Section C.2 can do that well. Section C.4.3 gives the two repairs, a zero-modified family or a compound representation.

The coefficient of variation goes past where the normal is usable. Table C.1 gives a normal model at a coefficient of variation of 1.0 a sixth of its probability below zero. Intermittent items routinely run past 1.0, so the family that is most often reached for is the one most surely wrong. That is not a small error in a tail; it is a model asserting that demand is frequently negative.

Neither repair is needed for this chapter, because Section 7.8 handles the slow-moving case directly with a discrete distribution and no mixture. Both matter from Section 8.11 onward, where the quantity being modeled is demand over a lead time and the zeros no longer aggregate away.

One topic this book does **not** cover is forecasting an intermittent series, as opposed to fitting a distribution to it.

That literature begins with Croston (1972). Croston's insight was that exponentially smoothing an intermittent series directly is wrong, because the estimate jumps at every demand and decays through every empty period. Thus, it is **highest immediately after a demand**, which is exactly when the stock has been drawn down and a replenishment is most likely to be triggered. An estimator that looks unbiased across the whole series is biased upward at the only moments anyone reads it. His method smooths two series separately instead, the size of a demand when one

happens and the interval between demands, and combines them into a demand rate.

Syntetos and Boylan (2001) then showed that the obvious way of combining them is biased. The ratio of two independent smoothed estimates does not estimate the ratio of their means, so Croston's estimator runs high, and Syntetos and Boylan (2005) gave the correction that now carries their names. The same authors' categorization scheme (Syntetos et al. 2005) is the source of the ADI and CV^2 cut-offs above, and it exists to say which of these estimators to use.

Fitting asks what the distribution of demand is. Forecasting asks what next period's demand will be. They are different questions, both are needed in practice, and this chapter asks only the first.

7.5. The Newsvendor Model

We can now state the problem formally. It carries the name of a vendor buying newspapers for the day, and it applies to any resource committed before the need for it is known.

- Let X represent demand over the period, a random variable with cumulative distribution function $F(x) = P\{X \leq x\}$.
- Let Q represent the quantity ordered, the decision.
- Let c_o represent the **overage cost**, the loss on each unit left at the end of the period, in dollars per unit.
- Let c_u represent the **underage cost**, the loss on each unit of demand that could not be met, in dollars per unit.

Both are opportunity costs measured against the decision that could have been made, and both are strictly positive. Section 7.9 says what it means when one of them is not.

At the end of the period exactly one of two things has happened. Either $Q > X$ and $Q - X$ units are left over, or $Q < X$ and $X - Q$ units of demand went unmet. Using the notation $(y)^+ = \max(y, 0)$ of Section C.3, both cases are covered by one expression:

$$C(Q, X) = c_o(Q - X)^+ + c_u(X - Q)^+ \tag{7.4}$$

Notice that exactly one of the two terms is non-zero for any realized demand. Equation 7.4 is a random variable because X is, and we choose Q to minimize its expectation, $E[C(Q, X)]$.

7.5.1. The Same Model as a Profit

Much of the literature and all of the practice writes this as a profit rather than a cost, in terms of quantities that appear on an invoice.

- Let c represent the purchase cost, s the selling price, and u the salvage value per unit, with $s > c > u$.

The vendor sells $\min(X, Q)$ units, salvages $(Q - X)^+$, and pays for Q:

$$G(Q, X) = s\min(X, Q) + u(Q - X)^+ - cQ \tag{7.5}$$

The two formulations are the same model. Matching terms,

$$c_o = c - u, \qquad c_u = s - c \tag{7.6}$$

so the overage cost is what a unit loses between the counter and the scrap yard, and the underage cost is the margin forgone. For the belt, $c_o = 50 - 12 = \$38$ and $c_u = 95 - 50 = \$45$, which are the numbers Section 7.2 used.

Maximizing Equation 7.5 and minimizing Equation 7.4 give the same Q. Recall from Section 1.4.1 that a term which does not depend on the decision shifts an objective without moving its optimum. The two expressions differ by $(s-c)E[X]$, which is the profit the vendor would make if every unit demanded were sold, and that quantity does not depend on Q. It shifts the objective without moving its optimum. To see the difference exactly, write the units sold as $\min(X,Q) = X - (X-Q)^+$ and the order as $Q = X + (Q-X)^+ - (X-Q)^+$, and substitute both into Equation 7.5:

$$\begin{aligned} G(Q,X) &= s\left[X - (X-Q)^+\right] + u(Q-X)^+ - c\left[X + (Q-X)^+ - (X-Q)^+\right] \\ &= (s-c)X - (s-c)(X-Q)^+ - (c-u)(Q-X)^+ = (s-c)X - C(Q,X) \end{aligned}$$

by Equation 7.6. Taking expectations,

$$E[G(Q,X)] = (s-c)E[X] - E[C(Q,X)] \tag{7.7}$$

Use whichever formulation your audience speaks. This chapter derives in costs, because Equation 7.4 is shorter, and reports in profits, because that is what the distributor will ask for. Section 7.7 now works the model with a continuous demand distribution, where the critical ratio is a quantile.

7.6. The Critical Ratio

The expected cost has a closed form, and it is written in the first order loss function of Section C.3. Recall that $G^1(b) = E[(X-b)^+]$ is the expected amount by which demand exceeds b.

We take the expectation of Equation 7.4 term by term. The second term is $G^1(Q)$ by definition, so the expected shortage is

$$E\left[(X-Q)^+\right] = G^1(Q) \tag{7.8}$$

The first term needs one step. Notice the identity $(Q-X)^+ = (X-Q)^+ + Q - X$, which holds in both cases: when $X < Q$ the right side is $0 + Q - X$, and when $X \geq Q$ it is $X - Q + Q - X = 0$. Taking expectations, we have

$$E\left[(Q-X)^+\right] = G^1(Q) + Q - E[X] \tag{7.9}$$

which says the expected leftover is the expected shortage plus however far the order sits above the mean. Putting the two together, we have

$$E[C(Q)] = c_o\left(G^1(Q) + Q - E[X]\right) + c_u G^1(Q) = (c_u + c_o)G^1(Q) + c_o\left(Q - E[X]\right) \tag{7.10}$$

Equation 7.10 is the whole model in one line. The first term is the cost of being short, weighted by both prices because a unit short is both a sale lost and a unit not held. The second is the cost of the position itself.

Differentiating. Recall from Section C.3 that $dG^1(Q)/dQ = -G^0(Q)$, where $G^0(Q) = 1 - F(Q)$. Differentiating Equation 7.10 term by term, we get

$$\frac{d}{dQ}E[C(Q)] = -(c_u + c_o)G^0(Q) + c_o \tag{7.11}$$

Setting Equation 7.11 to zero, we get $G^0(Q) = c_o/(c_u + c_o)$, and since $G^0 = 1 - F$,

$$F(Q^*) = \frac{c_u}{c_u + c_o}, \qquad Q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right) \tag{7.12}$$

The quantity $c_u/(c_u + c_o)$ is the **critical ratio**. It is the rule Equation 7.2 produced by hand, and Equation 7.12 is why that rule was right. Notice that the optimum is a minimum: the second derivative is $(c_u + c_o)f(Q)$, which is non-negative everywhere, so the expected cost is convex and the stationary point is the bottom.

7.6.1. Reading the Critical Ratio

Equation 7.12 says something sharper than "solve for Q". The left side is $P\{X \leq Q^*\}$, the probability that demand falls short of the order. That is the probability of **having too much**. Thus, the critical ratio is not an input to the answer; it *is* the answer, expressed as a service level the costs imply.

The ratio depends only on the relative size of the two costs, so it can be read as odds. For example, if being short costs three times as much as being long, then $c_u/(c_u + c_o) = 3/4$ and it is optimal to run a 75% chance of leftovers. At four times it is 80%, and at nine times it is 90%.

Two consequences follow, and both contradict what an organization usually rewards.

Never having leftovers is bad. A buyer who ends every period with nothing left is not being careful. They are ordering at a critical ratio near zero, which is only correct when a lost sale costs nothing. For the belt the ratio is 0.5422, so the distributor should expect leftovers in about 54% of futures. A manager who treats scrapped belts as evidence of a mistake will push the buyer below the optimum, and the loss will never appear in any account because it is a sale that did not happen.

Never having unsatisfied customers is also bad. The mirror error costs just as much. At the belt's ratio the distributor should expect to run out about 46% of the time. A service target of 99% is a claim that c_u is ninety-nine times c_o, and if that is not true the stock is too large.

Thus, the useful question to ask of a stated service level is not whether it is high enough. It is what ratio of costs would make it optimal, and whether anyone believes those costs. Section 4.4.3 made the same move for the economic order quantity, reading a firm's behavior back into the parameters it implies.

7.7. Continuous Demand

With a continuous demand distribution, Equation 7.12 is a quantile and the work is arithmetic.

Example 7.2 (The final buy, solved)**.** Take Example 7.1 with the demand distribution of Section 7.3.

Notation. Let X represent total demand over the fourteen remaining weeks. Let X_i represent demand in week i, negative binomial with mean 33.2404 and variance 613.0387 from the fit of Equation 7.3.

The distribution of the total. Weekly demands are independent. Recall from Section B.2 that the mean and the variance of a sum then add:

$$E[X] = 14(33.2404) = 465.37 \text{ belts}, \qquad Var[X] = 14(613.0387) = 8582.54$$

so $\sigma = \sqrt{8582.54} = 92.64$ belts and the coefficient of variation is $92.64/465.37 = 0.1991$.

Which family. Section 7.3 fitted a **discrete** model to weekly demand, and demand over fourteen weeks is discrete too. Treating the total as continuous is an approximation, and it is a safe one here for the reason Section 7.8 gives at the end: one belt is 1/92.64, or 1.1%, of the standard deviation of the total, so rounding cannot decide the answer. The weekly data, where one belt is 4% of the standard deviation, would not have earned the same treatment.

The family to approximate with is the normal. Section 7.3 rejected the normal for **weekly** demand, whose coefficient of variation is 0.7449. The total over fourteen weeks has a coefficient of variation of 0.1991, and Table C.1 says a normal at that coefficient puts essentially nothing below zero. The normal is safe here and was not safe there. Recall from Section B.2

that a sum of many independent terms tends toward the normal whatever the terms look like.

The critical ratio. From Equation 7.6, $c_o = \$38$ and $c_u = \$45$, so

$$\frac{c_u}{c_u + c_o} = \frac{45}{83} = 0.542169$$

The quantity. Standardizing, we have $Q^* = \mu + z\sigma$ where $z = \Phi^{-1}(0.542169) = 0.1059$. That is,

$$Q^* = 465.37 + 0.1059(92.64) = 475.18 \text{ belts}$$

so the distributor orders **475 belts**.

The consequences. From Table C.2 the standard normal loss function at $z = 0.1059$ is $\Phi^1(0.1059) = 0.3482$, so by Equation C.20 we get an expected shortage of $92.64(0.3482) = 32.26$ belts. Reading Equation 7.9, the expected leftover is $32.26 + 475.18 - 465.37 = 42.07$ belts. The distributor expects to scrap about 42 belts and to turn away about 32 dealers.

The cost. From Equation 7.10,

$$E[C(Q^*)] = 38(42.07) + 45(32.26) = 1598.71 + 1451.72 = \$3{,}050.43$$

The check. The other form of Equation 7.10 must agree: $(45 + 38)(32.26) + 38(475.18 - 465.37) = 2677.62 + 372.81 = \$3{,}050.43$. The two agree, which confirms both the loss function and the quantity computed from it. That check is worth performing every time, because it catches a missing σ in the loss function immediately.

In business terms. By Equation 7.7 the expected profit is $(95-50)(465.37) - 3050.43 = \$17{,}891.01$. The distributor commits \$23,750 to buy 475 belts and expects to clear about \$17,900 on them.

Three things about Example 7.2 are worth more than the answer.

The optimum is flat. Ordering the mean rounded to whole belts, 465, costs \$3,068.88 against the optimum's \$3,050.43, which is 0.6% worse. The economic order quantity of Section 3.7 is flat in the same way and for the same reason: the objective is smooth and its derivative vanishes at the optimum. Thus, the value of this calculation is not the last belt. It is knowing that 475 is right and 700 is not, and 700 costs \$8,930, nearly three times the optimum.

The family matters less than the fit, and here that can be measured. Recall from Equation C.3 that a sum of independent negative binomials sharing p is negative binomial with the same p and the numbers of successes added. Thus, the fitted weekly model of Equation 7.3 gives the fourteen-week distribution **exactly**, as $NB(\hat{p} = 0.054222, 14\hat{r} = 26.6798)$, with the same mean and variance used above. Apply Equation 7.13 to it and the answer is 469 belts, because $F(468) = 0.5392$ is short of the ratio and $F(469) = 0.5434$ clears it. Now price the four candidate orders against that exact model.

Table 7.6.: What the choice of family costs, measured against the distribution the data were fitted to rather than against the other candidates. The gamma happens to land on the exact answer.

Model used to choose Q	Normal	Lognormal	Gamma	Exact negative binomial
Q, rounded to whole belts	475	466	**469**	**469**

Model used to choose Q	Normal	Lognormal	Gamma	Exact negative binomial
Expected cost under the exact model	\$3,061.28	\$3,057.19	**\$3,055.38**	**\$3,055.38**
Penalty against the exact answer	0.19%	0.06%	0.00%	0.00%

Read Table 7.6 as a bound rather than as a ranking. The worst of the three continuous families costs 0.19% more than choosing correctly, on a decision worth about \$3,050. That spread is small because the coefficient of variation of the total is 0.1991. It would not be small at the weekly coefficient of 0.7449, which is another reason Section 7.3 is worth doing carefully.

The normal's defect is real and did not bite. A normal with this mean and standard deviation puts $\Phi(-465.37/92.64) = \Phi(-5.02)$ below zero, which is about one in four million. Notice that this is a statement about *this* aggregation and not about the normal in general. The same family applied to the weekly data would have put about 9% of its mass below zero.

7.8. Discrete Demand

When demand is counted in small whole numbers, a continuous distribution is the wrong tool and the inverse in Equation 7.12 does not exist. A discrete cumulative distribution function is a staircase. Thus, a critical ratio almost never lands exactly on a step.

The rule that replaces it follows from the same marginal argument as Equation 7.2. We keep adding a unit while the expected gain exceeds the expected loss, which means while $F(Q) < c_u/(c_u + c_o)$. That is,

$$Q^* = \min\left\{Q : F(Q) \geq \frac{c_u}{c_u + c_o}\right\} \tag{7.13}$$

Equation 7.13 is not an inverse. It is the smallest quantity whose cumulative probability *reaches* the ratio, and at that quantity $F(Q^*)$ is usually strictly greater than the ratio. That gap is not an error. It is the staircase, and it means a discrete newsvendor generally achieves a service level a little better than the one its costs call for, because it cannot buy a fraction of a unit.

Evaluate a discrete second order loss function only at whole units

The first order loss function of a discrete distribution is defined for every real argument, and from version R1.7 of the KSL `firstOrderLossFunction` evaluates it correctly at any argument. Earlier versions were correct only at whole numbers, returned a value that was usually negative in between, and raised no error. The second order loss function is another matter. `secondOrderLossFunction` is correct at whole units and returns an approximation between them, in R1.7 as before, because the discrete definition in Equation C.10 is a statement about whole units.

That restriction costs nothing here, because Equation 7.13 only ever asks about whole units. It matters when a quantity that came from a continuous model is priced against a discrete one, which is what Table 7.6 does: round first, then evaluate.

Example 7.3 (A one-time buy of spare transformers)**.** Recall the municipal utility storeroom of Example 4.5, which carries a pad-mount transformer

at \$3,800. The utility is about to begin a capital project after which the specification changes, so transformers not used will be obsolete.

What is the demand process? Failures arrive independently at a low rate. Over the project window the utility expects 6 failures, and models the count as Poisson with $\lambda = 6$.

What costs are incurred? A transformer bought and not used scraps at \$900, so $c_o = 3800 - 900 = \$2{,}900$. A failure with no spare on the shelf is covered by an emergency purchase at \$9,500, so $c_u = 9500 - 3800 = \$5{,}700$.

The critical ratio. $5700/(5700 + 2900) = 5700/8600 = 0.662791$.

The quantity. We walk the Poisson distribution function until it reaches the ratio.

Table 7.7.: The Poisson newsvendor, walked one transformer at a time. Notice that the cost column confirms the rule rather than being used by it: the rule reads the $F(Q)$ row and stops at 7 because 0.6063 is short of 0.6628 and 0.7440 clears it.

Q	4	5	6	7	8
$F(Q)$	0.2851	0.4457	0.6063	**0.7440**	0.8472
Expected shortage $G^1(Q)$	2.2330	1.5181	0.9637	0.5700	0.3140
Expected cost	\$13,404	\$10,155	\$8,288	**\$7,802**	\$8,501

The answer. **Buy 7 spare transformers**, at a cost of \$26,600, with an expected total of \$7,802 in obsolescence and emergency purchases.

The check. Table 7.7 costs each candidate directly from Equation 7.10, using the Poisson loss function of Table C.3. The minimum is at 7, where the rule put it.

Does rounding the continuous answer work? Usually, and not always. Treating demand as normal with mean 6 and variance 6 gives $6 + \Phi^{-1}(0.6628)\sqrt{6} = 6 + 0.4201(2.4495) = 7.029$, which rounds to 7 and agrees.

Now change one number. Suppose the emergency purchase costs \$8,150 rather than \$9,500, so that $c_u = \$4{,}350$ and the critical ratio is exactly 0.6000. The exact rule reads $F(5) = 0.4457$ and $F(6) = 0.6063$, and stops at **6 transformers**. The normal approximation gives $6 + 0.2533(2.4495) = 6.621$, which rounds to **7**. The approximation buys one transformer too many, at \$3,800, which is a real error on a real decision.

The lesson is about when the approximation is safe rather than whether it is. A normal approximation to a discrete distribution is good when the mean is large enough that one unit is a small fraction of the standard deviation. At $\lambda = 6$ the standard deviation is 2.45 units, so a single transformer is 40% of it, and rounding decides the answer. At $\lambda = 600$ a single unit is 4% of the standard deviation and rounding decides nothing. Thus, you should use Equation 7.13 whenever the numbers are small, which is exactly when slow-moving items are involved and exactly when each unit is expensive.

7.9. Extensions

Four variations cover most of what a real single-period problem adds, and none of them changes Equation 7.12. They change what goes into it.

7.9.1. A Penalty Beyond the Lost Margin

Equation 7.6 sets $c_u = s - c$, the margin forgone. That is the whole cost only when a customer turned away this time returns next time as though nothing had happened. Where a stockout costs more than the sale, add it.

- Let π represent the stockout penalty of Section 1.4.6, in dollars per unit of unmet demand, so that $c_u = s - c + \pi$.

For the belt, suppose the distributor judges that a dealer refused a belt costs \$40 in future business beyond the margin lost today. Then $c_u = 45{+}40 = 85$ and the critical ratio rises from 0.5422 to $85/123 = 0.6911$. The quantity rises with it, from 475 belts to $465.37 + 0.4988(92.64) = 511.58$, so **512 belts**.

Notice the direction and the size. A penalty of \$40, which is under half the selling price, moved the order by 37 belts, which is 7.8%. Recall from Section 1.5.5 that π is the parameter no ledger contains and the one hardest to defend. The right way to report this decision is not a single number. It is that 475 belts is right if a stockout costs only the margin and 512 if it costs \$40 more, and that the organization should say which it believes.

7.9.2. Stock Already on Hand

Suppose the distributor already holds I belts when the final order is placed. The decision is now how many to *add*, but the model is unchanged in a way worth seeing.

What the model determines is the quantity that should be **available** for the period, which is the stock on hand plus the order. Call that the order-up-to level S^*, and notice that Equation 7.12 computes it exactly as before, because the cost of ending the period with $S - X$ units left does not care how they got there. That is,

$$S^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right), \qquad Q^* = (S^* - I)^+ \tag{7.14}$$

For the belt, $S^* = 475.18$, so with 120 belts already on hand the order is $475.18 - 120 = 355.18$, or **355 belts**. With 600 already on hand the order is zero, and the distributor is over-stocked before it begins.

Equation 7.14 is the first appearance of a structure the rest of the book uses constantly. The (R, S) policy of Section 8.13 builds its whole rule on an order-up-to level, and the (s, S) policy of Chapter 8 is the same idea with a trigger attached. Notice that the truncation at zero matters: a newsvendor cannot sell stock back, so an excess position is simply carried.

7.9.3. When the Model Refuses

Equation 7.12 requires $c_o > 0$ and $c_u > 0$, and the two ways that fails are both informative.

If $c_u \leq 0$ there is no penalty for being short, so the critical ratio is zero or negative and the answer is to order nothing. That is correct, and it usually means the problem was stated wrongly: an item nobody loses anything by running out of is an item nobody should stock.

If $c_o \leq 0$, which happens when the salvage value equals or exceeds the purchase cost, the critical ratio is one or more and the answer is to order without limit. That is also correct and also a sign of a mis-stated problem. A unit that can always be disposed of for at least what it cost carries no risk, so nothing stops you buying it. An unbounded answer is the model reporting that you have described a situation with no downside, and the right response is to find the cost you left out.

7.9.4. Is It Really a Single Period?

The strongest assumption in this chapter is the one in the title. Leftover stock must have no value after the period. That is exactly true for a perishable item, a newspaper or a flu vaccine, and exactly true for the final buy of Example 7.1 because the deck is withdrawn.

It is false for most inventory. A belt left over in an ordinary week is a belt available next week, worth its purchase cost less one week of holding, and that is a multi-period problem. Applying the newsvendor model there charges the whole value of the unit against a decision that costs only a week of carrying, so the order comes out far too small.

Thus, the test to apply before using this chapter is a single question. **What is a leftover unit worth at the end of the period?** If the answer is its scrap value, this is a newsvendor problem. If the answer is nearly what you paid, it is not, and Chapter 8 is where to go.

7.10. Estimating the Answer by Simulation

Everything so far has been a formula. This section computes the same answer by sampling, and it is the first simulation in this book.

The order matters. Equation 7.12 already gives the exact answer for Example 7.2, so simulating it proves nothing about the belt. What it proves is that the simulation is right, and that is worth having before the simulation is pointed at a problem where no formula exists.

The model is Equation 7.5 evaluated on sampled demand. Notice that the sampling is done from the **fitted** model of Equation 7.3 and not from the normal approximation to it. The simulation can sample counts as easily as it can sample real numbers, so there is no reason to hand it the approximation, and sampling counts removes the truncation at zero a normal would have needed.

```
val demand = NegativeBinomialRV(probOfSuccess = 0.054222,
 ↪  numSuccess = 26.6798, streamNum = 1)
val stat = Statistic("profit at Q")

demand.resetStartStream()
repeat(100_000) {
    val d = demand.value
    stat.collect(s * minOf(d, q) + u * maxOf(0.0, q - d) - c
 ↪  * q)
}
println("%.2f +/- %.2f".format(stat.average, stat.halfWidth))
```

Notice `resetStartStream`. Running it before each candidate makes every quantity face the **same** hundred thousand demands, which is what lets the rows of Table 7.8 be compared with one another rather than only with the exact answer. It also makes each row reproducible on its own, which a shared advancing stream would not. Section 7.12.3 packages this loop as `estimate`, priced by the same cost function the exact route uses.

Table 7.8.: Expected profit estimated by simulation, 100,000 replications at each quantity, every quantity facing the same demands. The closed form is Equation 7.10 evaluated on the exact negative binomial of Table 7.6.

Q	440	460	**469**	490	510
Estimated profit	\$17,737.18	\$17,877.57	**\$17,893.03**	\$17,818.04	\$17,610.98
Half-width	\$21.02	\$24.68	\$26.31	\$29.95	\$33.15
Closed form	\$17,732.96	\$17,871.00	**\$17,886.06**	\$17,811.30	\$17,606.01

Read Table 7.8 one column at a time. The estimate at $Q^* = 469$ is \$17,893.03 with a half-width of \$26.31, so the interval is $[\$17{,}866.72, \$17{,}919.34]$, and the exact \$17,886.06 lies inside it. So does the exact figure in every other column. Five agreements are a great deal more convincing than one, and they cost nothing extra to report. That agreement is the validation, and it is the reason to do the easy case first. Notice also that the simulation puts its peak at the same candidate the formula does, which the common demands make a fair comparison rather than a lucky one.

Notice what the simulation has that the formula does not. Every figure in Table 7.8 carries a half-width. A simulated answer is an estimate with a standard error, and reporting one without its interval is reporting a number that is not there. Recall from Section B.2 that the half-width shrinks with the square root of the replication count. Thus, ten times the precision costs a hundred times the run.

And notice what it costs. The exact answer took one walk up a distribution function. The estimate took half a million samples and is still uncertain in the third significant figure. Simulation is not how you solve the newsvendor problem.

7.10.1. When It Is Worth It

Simulation earns its place the moment any assumption behind Equation 7.12 fails, and four failures are common.

A demand distribution with no convenient inverse. A mixture, from Section C.3.8, has no closed-form quantile. The formula needs a search; the simulation does not care.

A salvage rule that is not linear. For example, suppose the scrap dealer takes the first 100 belts at \$12 and the rest at \$4. Equation 7.4 no longer has constant c_o, the derivation of Equation 7.12 breaks, and three lines change in the simulation. Example 7.5 runs it.

A constraint. A supplier minimum, a case quantity, a budget shared with another item. A budget is the easy case. The multiplier of Section 4.2.2 still works, because each item's expected cost is its own and the budget is a sum, and the exercises of Section 7.14.4 carry it through. A supplier minimum or a case quantity is harder. It makes the order jump, the derivative the multiplier relies on no longer exists, and a search over simulated costs is often the shortest route.

More than one item that interact. Two products competing for one display, or a substitution rule sending refused demand for one to the other. There is no newsvendor formula for that, and there is an obvious simulation.

In all four the pattern is the same. That is, the *policy* stays simple, a single quantity chosen once, and what changes is the cost function. A cost function is easier to sample than to integrate. Building and running models of systems that no formula reaches is the subject of Rossetti (2026), and Appendix E points the way in.

7.11. Building These Models in a Worksheet

The worksheets of Section 5.8 and Section 6.9 each laid out a procedure across rows. This one is different, because the decision is a single number and the work is in producing the distribution it is a quantile of.

7.11.1. The Five Sheets

`Chapter7Models.xlsx` has five working sheets, and the division between them is the argument of this chapter made visible. A sixth, `Notes`, says what the other five are for. Table 7.9 lists them in the order the chapter uses them.

Table 7.9.: The working sheets of `Chapter7Models.xlsx`, in the order the chapter uses them. The sheets are linked: `Fit` takes its demand from `Data`, `Newsvendor` takes its moments from `Fit`, and `Fit` and `Horizon` read the critical ratio back from `Newsvendor`. Only `Discrete` stands alone.

Sheet	What it does	Which part of the chapter
`Data`	holds the 104-week history, its summary statistics and its histogram	Section 7.3.1 and Section 7.3.2
`Fit`	rules families out by the variance to mean ratio, fits the survivor, aggregates it over fourteen weeks, and prices four candidate orders	Section 7.3.3 and Table 7.6
`Horizon`	walks the exact fourteen-week negative binomial one belt at a time	Section 7.7
`Newsvendor`	the decision for the belt, from the invoice prices to the expected profit	Example 7.2 and Section 7.9
`Discrete`	the decision for the spare transformer, walked one unit at a time	Example 7.3

`Data` holds the history and assumes nothing about it. The statistics block

computes the mean, the variance, the standard deviation, the coefficient of variation, and the median, and puts the median next to the mean because Section 7.3 reads them together.

Which family				
@sec-newsvendor-fitting. Demand for the belt is a COUNT, so the first question is which discrete family, and the diagnostic that answers it is the variance to mean ra				
THE DIAGNOSTIC one number, three verdicts				
Each family below either fixes the ratio or bounds it on one side, so the ratio rules families out before any of them is fitted. Equation C.1 of the appendix.				
Mean	33.2404			
Variance	613.0387			
Variance to mean ratio	18.4426			
	Poisson	**Binomial**	**Negative binomial**	
What the family requires	*ratio = 1*	*ratio < 1*	*ratio > 1*	
Does the history satisfy it?	no	no	yes	
Row 12 is @sec-newsvendor-fitting-fit's Step 2 done by the sheet. The KSL's binomial estimator refuses this data for the same reason row 12 says no: a binomial cann				
THE FIT method of moments, @eq-newsvendor-negbinfit				
The negative binomial is the one family left. Its two parameters come from the two moments above, and nothing else.				
p, probability of success	0.054222			
r, number of successes	1.9057			
Check: 1/p is the ratio	0.00000000			
Fitted mean, r(1-p)/p	33.2404			
Fitted variance, r(1-p)/p^2	613.0387			
Rows 20 and 21 must return the mean and variance the fit was made from. Row 19 must read zero. Neither is decoration: they are how you catch the other conventi				
OVER THE HORIZON fourteen weeks of it				
Weekly demands are independent, so the mean and the variance of the total both multiply by the number of weeks. The critical ratio comes from the Newsvendor she				
Weeks remaining	14	*weeks*		
Mean over the horizon	465.3654			
Variance over the horizon	8582.5424			
Standard deviation	92.6420			
Coefficient of variation	0.1991			
One belt, as a share of the deviation	1.08%			
Critical ratio	0.5422			

Figure 7.5.: The `Fit` sheet, rows 1 to 32. The diagnostic block computes the variance to mean ratio and answers yes or no for each discrete family. The fit block applies Equation 7.3 and checks it twice, and the horizon block multiplies the weekly moments by fourteen. Notice that row 19 reads zero and that rows 20 and 21 return the moments the fit was made from.

`Fit` does Steps 1 to 3 of Section 7.3, and Figure 7.5 shows it. Row 8 computes Equation C.1 as 18.4426, and row 12 answers each family's requirement in turn. The Poisson needs a ratio of one and the binomial a ratio below one, so both read no, which is the step the KSL's binomial estimator performs by refusing the data. Rows 17 and 18 then fit the one

family left standing, $\hat{p} = 0.054222$ and $\hat{r} = 1.9057$. Notice the two checks that follow. Row 19 computes $1/\hat{p}$ less the ratio and reads zero, and rows 20 and 21 rebuild the mean and the variance from the fitted parameters. Both catch the other convention for p, which is the commonest error in fitting this family. Rows 26 to 32 aggregate over the fourteen weeks and carry the critical ratio across from `Newsvendor`.

`Horizon` is the fitted model aggregated over the fourteen weeks and walked one belt at a time. Let $g(x)$ represent the mass function of the fourteen-week total, a negative binomial with $p = \hat{p} = 0.054222$ and $r = 14\hat{r} = 26.6798$. Excel's `NEGBINOM.DIST` will not take a number of successes that is not a whole number, so the sheet builds g from the recursion $g(x) = g(x-1)(x+r-1)(1-p)/x$, started from $g(0) = p^r$, and everything after it is a running sum. The walk stops at 700 belts, so the sheet reports the tail it did not reach, and you need not wonder why the last cumulative probability is not one.

465	4.30E-03	0.526338	37.0168	FALSE
466	4.29E-03	0.530625	36.5431	FALSE
467	4.28E-03	0.534903	36.0737	FALSE
468	4.27E-03	0.539170	35.6086	FALSE
469	4.26E-03	0.543428	35.1478	TRUE
470	4.25E-03	0.547674	34.6912	TRUE
471	4.24E-03	0.551909	34.2389	TRUE
472	4.22E-03	0.556132	33.7908	TRUE
473	4.21E-03	0.560344	33.3470	TRUE

Figure 7.6.: The `Horizon` sheet at belts 465 to 473. The columns are the mass function, the distribution function, the first order loss function and whether the distribution function has reached the critical ratio. Notice where the last column turns: at 469, which is the exact answer of Section 7.7.

Figure 7.6 shows the rows where the decision is made. Read the distribution function column down to the critical ratio of 0.5422. $F(468) = 0.539170$ falls short of it and $F(469) = 0.543428$ clears it, so Equation 7.13 stops at 469 belts. The loss function column is G^1 by the recursion of Equation C.13,

and at 469 it reads 35.1478 belts of expected shortage. That is the figure every other model on the workbook is priced against.

WHAT EACH MODEL WOULD HAVE YOU ORDER				
Three continuous approximations, and the exact answer from the Horizon sheet. Every quantity is rounded to whole belts before it is priced, because a fraction of a				
	Normal	**Lognormal**	**Gamma**	**Exact**
Order quantity	475.1761	466.0381	469.0364	469.0000
Rounded to whole belts	475	466	469	469
Expected cost under the exact model	$3,061	$3,057	$3,055	$3,055
Penalty against the exact answer	0.19%	0.06%	0.00%	0.00%
Row 41 is the price of choosing the wrong family, in the only currency that settles the question. Notice it is smaller than the rounding.				

Figure 7.7.: The comparison block at the bottom of `Fit`. Each column is one family's answer, rounded to a whole belt and then priced against the exact model on `Horizon`. Notice that the penalty in row 41 is never more than a fifth of a percent.

Figure 7.7 is Table 7.6 as the worksheet computes it. Row 38 holds each family's quantile at the critical ratio, and row 39 rounds it to whole belts. Row 40 prices the rounded order on the exact model, which is why the gamma and the exact columns agree exactly. Row 41 states each cost as a penalty over the exact answer. Notice that it never exceeds 0.19%. The rounding comes before the pricing for the reason Section 7.8 gives.

`Newsvendor` is the decision for the belt, and `Discrete` is the decision for the transformer. They carry the mathematics of Section 7.6, so we take them cell by cell.

7.11.2. The Cells That Carry the Mathematics

Figure 7.8 shows the sheet from the top down. The four blue cells, B5 to B8, are the item's inputs, and they are in the units of an invoice rather than of the model. The two blocks at the bottom of the sheet have one blue cell each, B40 and B48. Table 7.10 translates the rest of it, one row at a time. Read the last column first: every row is one result of this chapter rewritten in a language Excel evaluates.

The decision		
@sec-newsvendor-model to @sec-newsvendor-extensions. The final buy for the belt, in the profit quantities an invoice carries and the co		
WHAT AN INVOICE SAYS		
Selling price, s	$95.00	*$/belt*
Purchase cost, c	$50.00	*$/belt*
Salvage value, u	$12.00	*$/belt*
Stockout penalty, pi	$0.00	*$/belt beyond the lost margin*
WHAT THE MODEL USES		
Critical ratio	0.5422	
Overage cost, co = c - u	$38.00	
Underage cost, cu = s - c + pi	$45.00	
Both are strictly positive or the model has nothing to trade. Row 14 says so rather than leaving a reader to notice.		
Is the problem well posed?	admissible	
DEMAND OVER THE PERIOD		
Mean	465.3654	
Standard deviation	92.6420	
THE ANSWER		
z at the critical ratio	0.1059	
Order quantity, Q*	475.1761	
The loss function is arithmetic and not a macro. Phi1(z) = phi(z) - z(1 - Phi(z)), and G1 = sigma times that.		
phi(z)	0.3967	
1 - Phi(z)	0.4578	
Phi1(z)	0.3482	
Expected shortage, G1(Q*)	32.2605	
Expected leftover	42.0712	
WHAT IT COSTS, TWICE		
@eq-newsvendor-expectedcost has two forms and they must agree. Row 34 is that check wired into the sheet: it reads zero or something		
Expected cost, co(leftover) + cu(shortage)	$3,050.43	
Expected cost, (cu+co)G1 + co(Q-mean)	$3,050.43	
Difference, which must be zero	0.0000000000	
Expected profit	$17,891.01	
Cash committed at Q*	$23,758.80	

Figure 7.8.: The `Newsvendor` sheet, rows 1 to 36. The invoice prices come first and the costs the model uses second. Demand is read from `Fit`, then come the answer and its consequences, and last the expected cost computed twice. Notice that row 34, the difference between the two forms of Equation 7.10, reads zero.

Table 7.10.: The cells of the `Newsvendor` sheet that carry the mathematics, in the order the derivation of Section 7.6 produces them rather than the order the sheet lays them out.

Cell	Holds	Formula	Which result it is
B11	c_o	`=B6-B7`	Equation 7.6
B12	c_u	`=B5-B6+B8`	Equation 7.6, with the penalty of Section 7.9.1
B10	critical ratio	`=B12/(B12+B11)`	Equation 7.12
B14	well posed?	`=IF(OR(B11<=0,B12<=0),"REFUSED…","admissible")`	Section 7.8.3
B17, B18	μ, σ	`=Fit!B27`, `=Fit!B29`	the moments of a sum, Example 7.2
B21	z	`=NORM.S.INV(B10)`	the standard normal quantile
B22	Q^*	`=B17+B21*B18`	Equation 7.12 for the normal
B24, B25	$\phi(z)$, $1-\Phi(z)$	`=NORM.S.DIST(B21,FALSE)`, `=1-NORM.S.DIST(B21,TRUE)`	standard normal density and tail
B26	$\Phi^1(z)$	`=B24-B21*B25`	$\phi(z) - z\,(1-\Phi(z))$
B27	$G^1(Q^*)$	`=B18*B26`	Equation C.20 and Equation 7.8
B28	expected leftover	`=B22-B17+B27`	Equation 7.9
B32, B33	$E[C(Q^*)]$	`=B11*B28+B12*B27` and `=(B12+B11)*B27+B11*(B22-B17)`	the two forms of Equation 7.10

Cell	Holds	Formula	Which result it is
B35	expected profit	`=(B5-B6)*B17-B32`	Equation 7.7
B49	S^*	`=B22`	the order-up-to level of Equation 7.14
B50	the order	`=MAX(0,B49-B48)`	Equation 7.14, with I in B48

Notice two things in Table 7.10. First, the loss function in B26 is a formula and not a macro. Excel has no normal loss function, and it does not need one, because Φ^1 is two built-in functions and a subtraction. Second, B32 and B33 compute the same quantity two ways. Thus, the arithmetic check of Example 7.2 is wired into the sheet, and B34 reads zero or something above it is wrong.

TYPE A QUANTITY OF YOUR OWN		
The block above solves. This one prices whatever you type, so a reader can see how flat the optimum is by trying numbers either side of it.		
Quantity	465.0000	*belts*
z at that quantity	-0.0039	
Expected shortage	37.1418	
Expected cost	$3,068.88	
Penalty against the optimum	0.61%	
STOCK ALREADY ON HAND		
@eq-newsvendor-orderupto. What the model sets is the quantity that should be available, so the order is that less what is already there.		
On hand	0.0000	*belts*
Order-up-to level, S*	475.1761	
Order	475.1761	

Figure 7.9.: The bottom of `Newsvendor`, rows 38 to 50. The upper block prices any quantity typed into B40, and the lower one turns the order-up-to level into an order given the stock on hand in B48.

Figure 7.9 holds the two blocks that do not solve anything. B40 to B44 price whatever quantity is typed into B40. The sheet ships holding 465, the

mean in whole belts, so B44 shows the 0.6% penalty that Section 7.7 quotes for ordering the mean. B48 to B50 are Equation 7.14, and typing 120 into B48 returns the order of 355.18 belts that Section 7.9.2 computes.

When demand is counted						
@sec-newsvendor-discrete. The spare transformer. A staircase distribution function rarely lands on the critical ratio, so the answer is the smallest qua						
THE ITEM						
Expected failures over the window	6.0000	*transformers*				
Purchase cost	$3,800.00	*$ each*				
Scrap value	$900.00	*$ each*				
Emergency purchase price	$9,500.00	*$ each*				
Overage cost, co	$2,900.00					
Underage cost, cu	$5,700.00					
Critical ratio	0.6628					
Order quantity, Q*	7					
Row 12 reads the table below rather than repeating its logic. Change a price and watch both move.						
THE WALK one transformer at a time						
The loss function is the recursion of Equation C.6: G1(Q) = G1(Q-1) - (1 - F(Q-1)), starting from G1(0) = the mean. The cost column is not used by the ru						
Q	**g(Q)**	**F(Q)**	**G1(Q)**	**Expected cost**	**Reached?**	
0	0.0025	0.0025	6.0000	$34,200	FALSE	
1	0.0149	0.0174	5.0025	$28,521	FALSE	
2	0.0446	0.0620	4.0198	$22,971	FALSE	
3	0.0892	0.1512	3.0818	$17,803	FALSE	
4	0.1339	0.2851	2.2330	$13,404	FALSE	
5	0.1606	0.4457	1.5181	$10,155	FALSE	
6	0.1606	0.6063	0.9637	$8,288	FALSE	
7	0.1377	0.7440	0.5700	$7,802	TRUE	<--
8	0.1033	0.8472	0.3140	$8,501	TRUE	
9	0.0688	0.9161	0.1613	$10,087	TRUE	
10	0.0413	0.9574	0.0773	$12,265	TRUE	
11	0.0225	0.9799	0.0347	$14,799	TRUE	

Figure 7.10.: The `Discrete` sheet, rows 1 to 29. The item block holds the transformer's prices and the Poisson mean, and the walk below lays out one row per candidate quantity. Notice that the highlighted row is chosen by the `Reached?` column and not by the cost column beside it, and that the two agree.

`Discrete`, in Figure 7.10, walks Equation 7.13. B5 to B8 are the inputs and B9 to B11 derive the two costs and the ratio exactly as `Newsvendor` does. Each row of the walk holds a candidate Q, the Poisson mass and distribution functions from `POISSON.DIST`, the loss function by Equation C.13, and the expected cost. Column F asks whether $F(Q)$ has reached B11, and column G marks the first row where it has. B12 reads its answer from that column

with `INDEX` and `MATCH` rather than repeating the logic. Thus, the rule and the direct calculation of Table 7.7 sit side by side, and when you change a price you watch both move.

Two conventions hold on the workbook. It carries no macros, so every number on it can be traced by clicking on the cell that produced it. And it computes the expected cost of every candidate on `Discrete`, although Equation 7.13 does not need the costs, so that the rule can be checked.

7.11.3. Reading the Sheets

The workbook ships holding the belt and the transformer, so the first use of it is to confirm that it says what the chapter says. The second is to ask it something the chapter did not.

Example 7.4 (Reading the Newsvendor and Discrete sheets)**.** The distributor wants to confirm the workbook before trusting it with the final buy, and then to use it to price an offer from the scrap dealer.

The input block. On `Newsvendor`, B5 to B8 hold $s = \$95$, $c = \$50$, $u = \$12$ and $\pi = 0$. These are the cells you type the item into.

The derived block. B11 and B12 return $c_o = \$38$ and $c_u = \$45$, B10 returns the critical ratio 0.5422, and B14 reads "admissible".

Demand. B17 and B18 return $\mu = 465.3654$ and $\sigma = 92.6420$ belts, read from rows 27 and 29 of `Fit`. Notice that the sheet carries every digit, where Example 7.2 rounded to 465.37 and 92.64.

The answer. B21 returns $z = 0.1059$ and B22 returns $Q^* = 475.1761$ belts. B27 and B28 return an expected shortage of 32.2605 belts and an expected leftover of 42.0712 belts.

The check, by hand. Let's reproduce B27 from Table D.2, which steps z by tenths. The table gives $\Phi^1(0.1) = 0.3509$ and $\Phi^1(0.2) = 0.3069$. Interpolating to $z = 0.1059$,

$$\Phi^1(0.1059) \approx 0.3509 - 0.059(0.3509 - 0.3069) = 0.3509 - 0.0026 = 0.3483$$

and by Equation C.20 the expected shortage is $92.6420(0.3483) = 32.27$ belts. B27 holds 32.2605, from the exact $\Phi^1(0.1059) = 0.3482$ that Example 7.2 used. The two differ by a hundredth of a belt, which is the interpolation, because Φ^1 curves between the rows of the table. That check is worth performing once on any sheet you did not build. A loss function that has lost its σ is off by a factor of 92, and no interpolation error looks like that.

The cost, twice. B32 and B33 both return \$3,050.43, and B34 reads zero. B35 returns an expected profit of \$17,891.01. These are the figures of Example 7.2, now carried to the cent. B36 prices the unrounded 475.18 belts at \$23,758.80, where the example priced the 475 actually ordered, \$23,750.

Now a real question. The scrap dealer calls with an offer of \$20 a belt rather than \$12 for whatever is left when the deck is withdrawn. What is the offer worth, and should the order change? One cell answers both. Set B7 to 20 and read the sheet:

$$c_o = \$30, \qquad \frac{c_u}{c_u + c_o} = \frac{45}{75} = 0.6000, \qquad Q^* = 488.84 \text{ belts}, \qquad E[G(Q^*, X)] = \$18{,}257.08$$

What it is worth. The expected profit rises by \$366.06, from \$17,891.01 to \$18,257.08 before rounding. Now type the old order, 475.1761, into B40 and read B43. Under the new salvage value the old order costs \$2,713.86, so its expected profit is $45(465.3654) - 2{,}713.86 = \$18{,}227.58$. Thus, of the \$366.06, \$336.57 comes from the higher price on the belts that would have been left over anyway, and only \$29.49 comes from re-optimizing the order. Each figure is rounded from the sheet's full precision, which is why the printed values differ by a cent in the last place. In other words, the

offer is worth taking, and the 14 extra belts it calls for are worth having but are not where the money is. That is the flatness of Section 7.7 again, now measured on a decision someone has to make.

The transformer. On `Discrete`, the highlighted row is $Q = 7$ with an expected cost of \$7,802, which is Table 7.7. Set the emergency price in B8 to \$8,150 and the highlight moves up one row to 6, which is the change Section 7.8 made by hand. Exactly where it moves is a question for Exercise 7.21. Restore both sheets before going on.

7.11.4. Using the Sheets on Your Own Item

`Newsvendor` takes its demand from `Fit`, and `Fit` takes it from the history on `Data`. Thus, there are two ways to point the sheet at a new item. If you have a history of the same kind, paste it into column B of `Data` and every sheet but `Discrete` follows it, including the family verdicts on `Fit`. The length of the period is the number of weeks in B26 of `Fit`. The statistics read the named range `Demand`, which spans the 104 weeks shipped, so a longer history means widening that name. If you already know the mean and the standard deviation over the period, type them over B17 and B18 of `Newsvendor`. That breaks the link to `Fit` deliberately, and the comparison of families on `Fit` no longer describes your item.

`Discrete` stands alone. Its demand is a Poisson mean in B5, and the walk covers zero to 15 units. That is enough for the transformer and for any item whose mean is under about seven. When no row of the walk reaches the ratio, B12 returns `#N/A` rather than a wrong answer, which is the sheet saying the walk is too short. A larger mean belongs on `Newsvendor`, for the reason Section 7.8 gives at its end: once one unit is a small fraction of the standard deviation, rounding decides nothing.

7.11.5. What the Worksheet Does Not Do

Two things the workbook does not do, for different reasons.

The first is Section 7.10. A spreadsheet can sample, but a sampled spreadsheet recalculates every time anything is touched. It gives a different answer each time, which is the wrong tool for a number someone will be asked to defend.

The second is more than one item. Every sheet here holds one item, and nothing on it knows that a second item exists. When several items draw on one budget or one storage area, the order of each depends on all of the others, through a multiplier of the kind Section 4.2.2 introduced. Section 4.7.1 built that machinery for lot sizing, and the exercises of Section 7.14.4 build it for the newsvendor.

Section 7.12 now turns the same model into code, where neither limit applies.

7.12. Designing the Software

Section 5.9 and Section 6.10 each had an algorithm to organize. This chapter has a formula, so the design question is different and smaller: what are the pieces, and what keeps the two ways of computing the answer in agreement?

7.12.1. What the Software Must Do

Five requirements follow from the chapter, and each one names where in the chapter it comes from.

1. Solve exactly, by Equation 7.12 for a continuous model and by Equation 7.13 for a discrete one, for any family with a quantile and a first order loss function.
2. Evaluate any quantity and not only the best one, because Table 7.7, Table 7.6 and Table 7.8 all price a grid.
3. Estimate the profit of a quantity by sampling, priced by the same cost function the exact route uses, Section 7.10.
4. Refuse a problem with no finite, positive answer, Section 7.9.3.
5. Take stock already on hand, Section 7.9.2.

7.12.2. Finding the Nouns

A **demand model** is a distribution with a quantile and a first order loss function. Notice that this is not a new class. It is `LossFunctionDistributionIfc` from Section C.3.9, which the KSL already provides for every family this chapter uses, and the design's first decision is not to reimplement it.

A **single-period cost structure** is c_o and c_u, together with the mapping from s, c, u and π that produces them. It owns Equation 7.6, it owns the critical ratio, and it owns the refusal of Section 7.9.3: a cost structure with a non-positive c_o or c_u is rejected at construction rather than allowed to return an absurd quantity later.

A **solution** pairs a quantity with what it implies: the expected shortage, the expected leftover, the expected cost, the expected profit. It is computed for any quantity and not only the optimal one, because the tables of this chapter all need a grid.

Newsvendor is the instructive rejection. It reads like the central class and the chapter is named after it. It fails because the cost structure and the demand model already do everything between them. Given both, the optimal quantity is one line, and a class whose only method is one line is a function wearing a costume.

7.12.3. The Classes

The code is in `code/`, package `inventory.newsvendor`, file `SinglePeriod.kt`. The cost structure is a data class.

```
data class SinglePeriodCosts(
    val price: Double,
    val cost: Double,
    val salvage: Double,
    val penalty: Double = 0.0,
) {
    val overageCost: Double get() = cost - salvage
    val underageCost: Double get() = price - cost + penalty
    val criticalRatio: Double get() = underageCost /
     ↪  (underageCost + overageCost)

    fun cost(q: Double, d: Double): Double =
        overageCost * max(q - d, 0.0) + underageCost * max(d
 ↪  - q, 0.0)
    fun profit(q: Double, d: Double): Double =
        price * min(d, q) + salvage * max(q - d, 0.0) - cost
 ↪  * q - penalty * max(d - q, 0.0)
}
```

The four constructor parameters are s, c, u and π. `overageCost` and `underageCost` are Equation 7.6, `criticalRatio` is Equation 7.12, and `cost` and `profit` are Equation 7.4 and Equation 7.5 for one realized demand `d`, a value of X. An `init` block, not shown, refuses a non-positive c_o or c_u with a message naming Section 7.9.3. Notice that `price` is the value of a unit at the moment it is needed. For the belt that is the selling price, and for the transformer of Example 7.3 it is the emergency purchase price, because that is what a missing unit costs to replace. A data class also gives `copy`, so a variation is one line. For the belt's costs, `copy(penalty = 40.0)` is Section 7.9.1.

The exact route is two functions.

```
fun optimalLevel(costs: SinglePeriodCosts, demand:
 ↪  DistributionFunctionIfc): Double {
    val ratio = costs.criticalRatio
    if (demand !is DiscreteDistributionIfc) return
     ↪  demand.invCDF(ratio)
    var x = max(floor(demand.invCDF(ratio)), 0.0)
    while (x > 0.0 && demand.cdf(x - 1.0) >= ratio) x -= 1.0
    while (demand.cdf(x) < ratio) x += 1.0
    return x
}
```

`optimalLevel` is Equation 7.12 when `demand` is continuous. When it is discrete, the two loops make the answer Equation 7.13 exactly, the smallest whole unit whose distribution function reaches the ratio, whatever convention the library's quantile uses for a staircase. It takes any `DistributionFunctionIfc`, which is what lets Table 7.6 ask a lognormal and a gamma for a quantity without asking either for a loss function.

```
fun evaluate(costs: SinglePeriodCosts, demand:
 ↪  LossFunctionDistributionIfc,
              level: Double, onHand: Double = 0.0):
               ↪  NewsvendorSolution {
    val shortage = demand.firstOrderLossFunction(level)
    val leftover = shortage + level - demand.mean()
    val expectedCost = costs.overageCost * leftover +
     ↪  costs.underageCost * shortage
    return NewsvendorSolution(costs, level, onHand,
     ↪  demand.cdf(level), shortage, leftover,
        expectedCost, (costs.price - costs.cost) *
 ↪  demand.mean() - expectedCost)
}
```

```
fun solve(costs: SinglePeriodCosts, demand:
 ↪  LossFunctionDistributionIfc, onHand: Double = 0.0) =
    evaluate(costs, demand, max(optimalLevel(costs, demand),
 ↪  onHand), onHand)
```

`evaluate` prices any `level`, the stock available for the period, which is S of Section 7.9.2, and `onHand` is its I. Its lines are Equation 7.8, Equation 7.9, Equation 7.10 and Equation 7.7, in that order. `solve` evaluates at the optimal level, and `onHand` carries Equation 7.14: the level does not depend on what is already there, and the order is the difference. When more than the level is already on hand, the level is what is on hand and nothing is ordered. `NewsvendorSolution` holds the eight values and computes `orderQuantity` as $(S - I)^+$.

The sampled route is one function.

```
fun estimate(demand: RVariableIfc, q: Double, replications:
 ↪  Int,
             profit: (q: Double, d: Double) -> Double):
              ↪  Statistic {
    val stat = Statistic("profit at %.2f".format(q))
    demand.resetStartStream()
    repeat(replications) { stat.collect(profit(q,
 ↪  demand.value)) }
    return stat
}

fun estimate(costs: SinglePeriodCosts, demand: RVariableIfc,
 ↪  q: Double, replications: Int) =
    estimate(demand, q, replications, costs::profit)
```

This is the loop of Section 7.10 with its free variables given names.

`resetStartStream` is there for the reason that section gives: every quantity estimated with the same `demand` faces the same demands. The second form passes `costs::profit`, the very function the cost structure defines. The first form takes any profit function, which is what Section 7.10.1 needs.

7.12.4. The Two Routes Must Agree

The design's real content is the agreement Section 7.10 demonstrated. The same cost structure is evaluated two ways, exactly and by sampling, and the two must agree.

That agreement is not a convenience; it is what the design is for. A solver and an estimator that share a cost function can be checked against each other, and one that does not share it cannot. Both routes take their costs from `SinglePeriodCosts`. The exact route uses its `overageCost` and `underageCost` through the loss function in `evaluate`, and the sampled route calls its `profit` on each demand in `estimate`. The two are tied by Equation 7.7, which a test in the code checks.

Notice what that buys when the exact route stops applying. Under the nonlinear salvage rule of Section 7.10.1 the loss function integration is no longer valid, but the sampled evaluation still is. Only the profit function changes, and the machinery that samples and averages it is the machinery that was validated against the formula when the formula held. That is the whole argument for building it this way. Section 7.12.5 puts both routes to work on the belt.

7.12.5. Running the Chapter's Examples

`NewsvendorExamples.kt` holds everything below. Run it with `./gradlew run -PmainClass=inventory.newsvendor.NewsvendorExamplesKt`. The output shown is the program's, with its section headings left out.

Example 7.5 (The final buy in code, and a salvage rule no formula takes)**.** The distributor wants the final buy computed in code, checked against the workbook, and then priced under a salvage rule the workbook cannot take.

Building the item. `DemandFitting.fittedHorizon` returns the fourteen-week negative binomial of Section 7.7, fitted from the history file rather than typed in. We solve against the normal the `Newsvendor` sheet uses, and then against the exact model.

```
val belt = SinglePeriodCosts(price = 95.0, cost = 50.0,
 ↪  salvage = 12.0)
val horizon = DemandFitting.fittedHorizon()
val normal = Normal(horizon.mean(), horizon.variance())
println(solve(belt, normal))
println(solve(belt, horizon))
```

```
order 475.18 units, to a level of 475.18 with 0.00 on hand
  critical ratio        0.542169   F(level) 0.542169
  expected shortage       32.2605 units
  expected leftover       42.0712 units
  expected cost           3050.43
  expected profit        17891.01
order 469.00 units, to a level of 469.00 with 0.00 on hand
  critical ratio        0.542169   F(level) 0.543428
  expected shortage       35.1478 units
  expected leftover       38.7824 units
  expected cost           3055.38
  expected profit        17886.06
```

The check against the worksheet. The first block is B22, B27, B28, B32 and B35 of `Newsvendor`, to every digit printed. The second is the exact answer of Figure 7.6: 469 belts, $F(469) = 0.543428$, and an expected shortage of 35.1478 belts. Notice the line `F(level)`. Against the normal it equals the

critical ratio, because a continuous quantile lands on it exactly; against the negative binomial it is a little above, which is the staircase of Section 7.8.

The families. Table 7.6 is a short loop.

```
for ((name, model) in families) {
    val q = optimalLevel(belt, model).roundToInt().toDouble()
    println("%-10s Q = %4.0f   expected cost under the exact
 ↪  model %9.2f"
        .format(name, q, evaluate(belt, horizon,
         ↪  q).expectedCost))
}
```

```
normal     Q =  475   expected cost under the exact model   3061.28
lognormal  Q =  466   expected cost under the exact model   3057.19
gamma      Q =  469   expected cost under the exact model   3055.38
exact      Q =  469   expected cost under the exact model   3055.38
```

`families` pairs each name with a KSL distribution moment-matched to the horizon, or with `horizon` itself. `optimalLevel` asks each one for its quantile, and `evaluate` prices the rounded answer against `horizon` alone, which is row 40 of Figure 7.7.

Now the item the formula cannot take. Section 7.10.1 supposed that the scrap dealer takes the first 100 belts at $12 and the rest at $4. The profit function changes, so we write it and hand it to `estimate`:

```
val twoTier = { q: Double, d: Double ->
    val left = max(q - d, 0.0)
    belt.price * min(d, q) + 12.0 * min(left, tier) + 4.0 *
 ↪  max(left - tier, 0.0) - belt.cost * q
}
for (q in 420..500 step 10) {
```

```
    val stat = estimate(demand, q.toDouble(), 100_000,
     ↪  twoTier)
    ...
}
```

Here `tier` is 100 and `demand` is a `NegativeBinomialRV` with the parameters of `horizon`, on stream 1, as in Section 7.10. Notice that `twoTier` is Equation 7.5 with its salvage term replaced, and that nothing else in the call changes. The column headed `summed` is the check. It computes the same expected profit by summing `twoTier` against the negative binomial's mass function, which is possible here because demand has a mass function we can walk.

```
    Q     estimate half-width       summed
  420     17443.54      17.64     17440.64
  430     17598.94      19.50     17595.40
  440     17717.80      21.41     17713.30
  450     17799.34      23.33     17793.51
  460     17842.54      25.26     17835.67
  470     17847.14      27.18     17839.83
  480     17813.90      29.07     17806.45
  490     17743.39      30.91     17736.37
  500     17637.05      32.70     17630.75
best on the grid: Q = 470
best by the summed profit, one belt at a time: Q = 466, profit 17842.70; at 469 it i
```

Validation. Every summed value lies inside its interval, so the sampled route is right under the new rule as it was under the old one. Notice also that every estimate sits above its summed value, by between \$2.90 and \$7.45. That is not a bias. Every row faces the same hundred thousand demands, so the rows share one sampling error, and it happens to be

positive. Thus, the comparison across rows is far sharper than any one half-width suggests.

What it means for the belt. Summing one belt at a time, the best order is 466, against 469 under the linear salvage value, and keeping 469 costs \$1.59 over the close-out. In other words, the rule barely moves the order. It does lower the profit, by about \$45 at 469, because the second tier is reached in about one close-out in seven: the expected leftover at 469 is 38.8 belts, and more than 100 are left over with probability 0.145. The program repeats the run with the tier at 25 belts, and there the rule bites. The best order falls to 460, and staying at 469 costs \$14.69 over the close-out.

The limit. The `summed` column was available only because this is one item whose demand has a mass function. Make demand a mixture whose distribution cannot be walked one unit at a time, or let two items share a budget and substitute for one another, and the column cannot be computed. The `estimate` column still can, and it is the same function that has now agreed with exact answers twice.

7.13. Summary

The single-period problem is not a short multi-period problem. What distinguishes it is that leftover stock has no future value, so the cost of ordering one too many is the whole loss on the unit rather than a period of carrying. That is also the test for whether this chapter applies.

The ordering cost disappears. One order is placed whatever its size, so k is constant across every alternative and cannot affect which is best. The trade-off that remains is overage against underage, and both are opportunity costs that appear in no ledger.

The answer is a quantile, and the quantile is the answer. The critical ratio $c_u/(c_u + c_o)$ is the probability of having too much that the costs imply, and $Q^* = F^{-1}$ of it. Read as odds it says that if being short

costs three times as much as being long, it is optimal to run a 75% chance of leftovers. Thus, never having leftovers is not careful buying, and never disappointing a customer is not good service. Each is a claim about a ratio of costs, and the useful question about a service target is what ratio would make it optimal.

Discrete demand needs the rule and not the inverse. A staircase distribution function rarely lands on the critical ratio, so the answer is the smallest quantity whose cumulative probability reaches it. Rounding a continuous approximation usually gives the same answer and sometimes does not, and it fails exactly where the items are slow-moving and expensive.

The demand distribution carries more of the answer than the formula does. Characterizing a history is cheap and settles more than a goodness of fit test does. Demand arrives in whole units, and for a count the variance to mean ratio of Equation C.1 decides the family before any of them is fitted: it is one for a Poisson and below one for a binomial, so a ratio of 18 leaves only the negative binomial standing. The test then confirms what the ratio chose. And a sales history is not a demand history, so a fitted distribution built from sales is biased low in the periods that mattered.

Which family you pick costs less than you would guess, and it can be measured. Fitting a count model makes the aggregate exactly computable, because negative binomials sharing p add. Table 7.6 prices the normal, lognormal and gamma answers against that exact model and none of them costs more than a fifth of a percent. Thus, the argument for care in Section 7.3 is not that the family is delicate; it is that the moments are, and every family in the table was given the same two.

Simulation belongs after the closed form, not instead of it. The newsvendor is the one model in this book whose simulation can be checked against an exact answer, which makes it the right place to build the habit. What simulation buys is the cases the formula cannot reach: a distribution with no inverse, a salvage rule that is not linear, a constraint, or two items that interact.

Table 7.11 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 7.11.: Notation introduced in this chapter.

Symbol	Meaning
X	Demand over the single period, a random variable
$F(x)$	Cumulative distribution function of demand, $P\{X \le x\}$
Q	Order quantity, the decision
Q^*	The optimal order quantity
S^*	Order-up-to level, when stock is already on hand
c_o	Overage cost, the loss per unit left over
c_u	Underage cost, the loss per unit of unmet demand
c, s, u	Purchase cost, selling price, salvage value per unit
π	Stockout penalty beyond the lost margin, per unit
$G^1(Q)$	First order loss function, the expected shortage at Q

Chapter 8 removes the last assumption this chapter kept. There the period does not end, stock left over is worth what it cost, and the decision is not how much to buy once but when to buy and how much each time.

7.14. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.8. Report quantities to the nearest whole unit and state the unrounded value alongside. Where a demand history is called for, generate it with the code of Section 7.3 and the seed the exercise gives, so that your answer and the instructor's agree.

7.14.1. Terminology and Concepts

Exercise 7.1. Define each of the following in one sentence, and say what unit it carries: overage cost, underage cost, critical ratio, order-up-to level, censored demand.

Exercise 7.2. True or false, with one sentence of justification for each.

a. Increasing the salvage value raises the optimal order quantity.
b. A buyer who never has leftover stock is managing the item well.
c. The ordering cost k does not appear in the newsvendor model because it is small.
d. If the critical ratio is 0.5 then the optimal quantity is the mean of demand.
e. Rounding the normal-approximation answer always gives the discrete optimum.

Exercise 7.3. Table 1.10 says that k is irrelevant with one order. Explain why, and then describe a single-period situation in which it would *not* be irrelevant. State what would have to be true of the supplier for your situation to arise.

Exercise 7.4. A distribution manager states a target of 98% service for an item. Using Equation 7.12, determine what ratio of underage to overage cost would make that target optimal. The item costs \$40, sells for \$60, and salvages for \$25. What stockout penalty π would be needed to justify the target, and is it credible?

7.14.2. Working the Model by Hand

Exercise 7.5. A caterer must decide how many trays of a dish to prepare for one event. Each tray costs \$18 to make and sells for \$45. Unsold trays are donated and carry no salvage value. Demand is

Trays x	10	20	30	40	50
$P\{X = x\}$	0.10	0.20	0.35	0.25	0.10

a. Compute c_o, c_u and the critical ratio.
b. Build the marginal analysis table of Table 7.2, one step of ten trays at a time, and report where it stops.
c. Compute the expected profit at all five candidates directly and confirm the answer of part (b).
d. Report the expected number of trays donated and the expected number of customers turned away at the optimum.

Exercise 7.6. An airline holds one spare engine module at a maintenance base. Failures over the coming season are Poisson with a mean of 3. A module costs \$220,000 and is worth \$60,000 if the fleet is retired before it is used. A failure with no module on hand grounds an aircraft at a cost of \$500,000.

a. Compute c_o, c_u and the critical ratio.
b. Using Equation 7.13, determine how many modules to hold. Show the $F(Q)$ column you walked.
c. Compute the expected cost at $Q^* - 1$, Q^* and $Q^* + 1$ and confirm the minimum.
d. Compute the normal-approximation answer and compare. Explain the comparison in terms of the ratio of one unit to the standard deviation.

Exercise 7.7. For the final buy of Example 7.2, compute the expected shortage and expected leftover at $Q = 450$, $Q = 475.18$ and $Q = 510$, using Table C.2 and Equation C.20. Work with the normal approximation, $E[X] = 465.37$ and $\sigma = 92.64$.

a. Verify Equation 7.9 at each quantity, i.e. that the expected leftover equals the expected shortage plus $Q - E[X]$.

b. Compute the expected cost at each and confirm that 475.18 is the smallest.
c. Express the cost at 450 and at 510 as a percentage above the optimum, and compare the shape of that penalty with Equation 3.22.
d. Report the cost excess per belt of departure at each of the two quantities in part (c), dividing the excess by the distance from the optimum. Then do the same for two **symmetric** departures, 25 belts either side. The costs c_u and c_o differ by 18%, so being short ought to be the more expensive direction. Report by how much it actually is at 25 belts, and explain why the asymmetry is so much smaller than the ratio of the two costs suggests.

7.14.3. Modeling Problems

Exercise 7.8. A second item's history is in `data/ch7-exercise-demand.csv`, 104 weeks in the same two columns as the belt's. Work Section 7.3 on it from beginning to end, and report the code alongside each answer.

a. Draw the three plots of Section 7.3.1 and say what each one settles. Include them with your answer.
b. Report the summary statistics of Table 7.4 for this item. Which families does Equation C.1 rule out before anything is fitted, and why can no choice of parameter rescue them?
c. Run `PMFModeler` on the history. Report every estimate it returns, including any that fail, and say what the failures mean.
d. Compute $\hat{p}$ and $\hat{r}$ by hand from Equation 7.3 and confirm the KSL's estimates. Then compare $1/\hat{p}$ with the ratio from part (b) and explain why they must agree.
e. Test the fitted negative binomial with `DiscretePMFGoodnessOfFit` and report the statistic, the degrees of freedom and the p-value. Do the same for a Poisson with the same mean, and compare the two.

f. Suppose the item is being bought for a 20-week close-out with $c = \$30$, $s = \$55$ and $u = \$8$. Compute the mean and variance of 20-week demand, the critical ratio, and Q^* under a normal approximation. Then compute Q^* exactly from the aggregated negative binomial, as Table 7.6 does, and report what the approximation costs.

Exercise 7.9. Return to Example 7.2.

a. Plot or tabulate Q^* against the stockout penalty π for $\pi = 0, 20, 40, 60, 80, 100$ dollars per belt.
b. At what π does the optimal order reach 550 belts?
c. The distributor is unwilling to name π but is confident the order should not exceed 525 belts. What upper bound on π does that imply? This is the move of Section 4.4.3, applied here.
d. Your answer to part (a) should show the quantity rising steeply at first and then flattening. Explain that shape from the critical ratio rather than from the plot.

Exercise 7.10. A hospital pharmacy must decide a final order for a drug being withdrawn from the formulary. Demand over the remaining 90 days is normal with a mean of 400 doses and a standard deviation of 70. A dose costs \$22, is billed at \$58, and expired doses are destroyed at a disposal cost of \$3 each.

a. Note that the salvage value is negative. Compute c_o and explain the sign.
b. Compute S^* and the order quantity if 85 doses are already on hand.
c. Compute the order quantity if 450 doses are already on hand, and say what the pharmacy should do about the excess.

Exercise 7.11. For each of the following, determine whether the newsvendor model applies, and if it does not, identify the cost that has been left out or mis-stated.

a. A retailer can return all unsold units to the supplier for a full refund.
b. A publisher prints a commemorative program that has no value after the event and no shortage cost, because attendees who cannot get one simply do not.
c. A grocer stocks a staple with a two-year shelf life, ordering weekly.
d. A contractor buys a specialist part for one job; leftovers can be sold to another contractor at slightly above what was paid.

7.14.4. Several Items and a Shared Limit

The deck the belt fits is out of production, so the belt is not the only part facing a final buy. The distributor also stocks the deck's spindle assembly, its blade and its idler pulley, and all four last orders are placed together for the same fourteen-week close-out. Table 7.12 gives the parts. The exercises of this subsection all use it, and each one says which parts and which limits it takes.

Table 7.12.: The deck's end-of-life parts. Prices are per unit, and space is the shelf volume one unit takes in the distributor's bins. In code, take the belt's model from `DemandFitting.fittedHorizon`, whose parameters print as these.

Part	Demand over the close-out	s	c	u	Space per unit
Belt	negative binomial, $NB(0.054222, 26.6798)$, as in Section 7.7	\$95	\$50	\$12	0.05 ft^3
Spindle assembly	normal, mean 120, standard deviation 30	\$240	\$140	\$25	1.20 ft^3
Blade	normal, mean 300, standard deviation 60	\$34	\$18	\$2	0.15 ft^3
Idler pulley	Poisson, mean 9	\$120	\$65	\$10	0.40 ft^3

The notation is that of Section 4.2.2, with part j in place of item j. Let c_j represent part j's purchase cost and v_j the space one unit takes. Let θ represent the multiplier on a budget, in dollars of expected profit per dollar of budget, and η the multiplier on the bins, in dollars of expected profit per cubic foot. Under a limit that binds, each part is ordered at an **adjusted critical ratio**, below its own critical ratio, and Exercise 7.13 finds it.

Three conventions hold throughout. A multiplier is found from the unrounded quantities. Once it is found, a quantity computed from a continuous model is *rounded down* to a whole unit, so that a plan inside a limit stays inside it. A quantity from a discrete model is already whole, by Equation 7.13. Profits are expected profits by Equation 7.7, part by part, and a plan's profit is their sum.

Exercise 7.12. The distributor has a final-buy budget, and the four unconstrained orders of Table 7.12 cost more than it. Answer each part true or false, and give the reason in one sentence.

a. The best plan cuts every part's order by the same percentage, so that the total just fits the budget.

b. Under a budget that binds, every part's adjusted critical ratio is below its critical ratio by the same amount.

c. If the four unconstrained orders together cost less than the budget, the budget changes nothing, however close the two figures are.

d. The part whose unconstrained order costs the most to buy is cut first, and the others are cut only if that is not enough.

e. Multiple choice. The budget's multiplier is $\theta = 0.11$. Which statement is correct?

   i. One more unit of each part adds about \$0.11 of expected profit.
   ii. One more dollar of budget adds about \$0.11 of expected profit.
   iii. The budget should be raised by 11%.

iv. Each part's adjusted critical ratio is 0.11 below its critical ratio.

Exercise 7.13. Take two parts, $j = 1, 2$, with continuous demand. Let F_j represent part j's distribution function and $c_{o,j}$ and $c_{u,j}$ its overage and underage costs. Let A represent the budget, in dollars, so that the plan must satisfy $c_1 Q_1 + c_2 Q_2 \le A$.

a. Write the Lagrange function for minimizing $E[C_1(Q_1)] + E[C_2(Q_2)]$ subject to the budget, with a multiplier $\theta \ge 0$, following Section 4.2.2.

b. Differentiate it with the help of Equation 7.11 and show that at the optimum

$$F_j(Q_j) = \frac{c_{u,j} - \theta c_j}{c_{u,j} + c_{o,j}}, \qquad j = 1, 2.$$

The right side is part j's adjusted critical ratio.

c. Show that the same condition follows from Equation 7.6 if each purchase cost is raised from c_j to $(1 + \theta)c_j$ and nothing else changes. Say what the multiplier is a surcharge on, and compare it with the surcharge of Section 4.2.2.

d. What does the condition say about part j when $\theta c_j \ge c_{u,j}$? Relate your answer to Section 7.9.3.

e. Explain why the check of Section 4.2.1 comes before any of this.

Exercise 7.14. Take only the spindle assembly and the blade of Table 7.12, with a final-buy budget of \$18,000 for the two. Work by hand, with Table D.1 and Table D.2, and the result of Exercise 7.13.

a. Compute each part's unconstrained order and what the pair costs to buy. Does the budget bind?

b. Find θ by bisection. Start from the interval $[0, 1]$, try its midpoint, and keep the half in which the spend crosses \$18,000. Stop when the interval is narrower than 0.01. At each trial report θ, both adjusted critical ratios, both unrounded quantities and the spend.
c. At the last trial value of θ that keeps the spend within the budget, round both quantities down, and report the plan, its spend, and each part's expected profit.
d. Report the expected profit the budget costs, against the unconstrained pair. Which part gave up more units, and which gave up more profit?

Exercise 7.15. Two slow movers are left for a smaller close-out: idler pulleys with Poisson demand of mean 3, and spindle assemblies with Poisson demand of mean 2, both at the prices of Table 7.12. The budget for the two is \$450. Work by hand, with Table D.3.

a. For each part tabulate $F(k)$ for $k = 0$ to 5, and the expected gain from the $(k + 1)$st unit, $c_u - (c_u + c_o)F(k)$, which is the marginal argument of Section 7.2. What are the unconstrained orders, and do they fit the budget?
b. *The greedy rule.* Start with nothing. At each step, add one unit of whichever part has the larger expected gain per dollar, among those whose gain is positive and whose price fits in what is left of the budget. Stop when no unit qualifies. Keep a ledger with columns for the step, the pulleys, the spindles, each part's next gain per dollar, the unit added, the spend, and the budget left, and re-render it after every step.
c. *Enumeration.* Tabulate the expected profit of every pair that fits the budget, and find the best.
d. Compare (b) and (c). Explain why the greedy rule missed, and say under what condition on the prices it could not have.

Exercise 7.16. Add a sheet named `Budget` to `Chapter7Models.xlsx` for the belt, the spindle assembly and the blade, with the belt's demand taken as the normal of Example 7.2, mean 465.37 and standard deviation 92.64.

Lay it out as Section 4.7.1 lays out the constrained lot sizing sheet: an input block holding the budget, a block that says whether the budget binds, a multiplier cell, a results block showing each part unconstrained beside constrained, and one row per part below.

a. Implement the condition of Exercise 7.13 in the part rows, with `NORM.INV` for the quantity and the loss function of Table 7.10 for the expected profit.
b. With a budget of \$40,000, find the multiplier with Goal Seek, setting the spend of the unrounded quantities to the budget by changing the multiplier cell. Report θ, the three quantities rounded down, the spend, and the expected profit given up.
c. Add a check row that computes $F(Q)$ at each part's unrounded quantity and confirms it equals the adjusted critical ratio, $(c_u - \theta c)/(c_u + c_o)$.
d. Repeat (b) at budgets of \$44,000 and \$46,000, and explain what the sheet does at the second.

Exercise 7.17. Take all four parts of Table 7.12, a final-buy budget of \$43,000, and 195 ft^3 of bins. Use `SinglePeriodCosts`, `optimalLevel` and `evaluate` from Section 7.12, and report the code you wrote alongside each answer.

a. Compute the unconstrained plan, its spend and its space. Which limits does it break?
b. Impose the budget alone. Re-price each part with `copy`, raising its purchase cost by θc_j, and find θ by bisection on the unrounded quantities. Report θ, the plan, its spend and its space. Does it fit the bins?
c. Impose the bins alone. Raise each part's purchase cost by ηv_j, and find η the same way. Report η and the plan. Does it fit the budget?
d. Impose both. Alternate the two searches, each holding the other multiplier fixed, until neither changes in its sixth decimal place.

Report θ, η, the plan, its spend, its space and its expected profit, and say which limits bind.

e. Confirm the expected profit of each part in (d) by simulation: use `estimate` with 100,000 replications on stream 1 for each part, and show that the value `evaluate` gives lies inside each interval.

f. The multiplier on the budget is positive, but the plan's spend does not reach \$43,000. Explain why, naming the parts responsible.

Exercise 7.18. Use your answer to Exercise 7.17 (d).

a. A neighboring warehouse offers extra bin space for the close-out at \$10 per cubic foot. Another offers it at \$15. Using the multipliers alone, say which offer, if either, the distributor should take, and why.

b. Check your answer to (a) by solving again with 197 ft^3 and with 205 ft^3 of bins. Compare each gain in expected profit with what the extra space would cost at each price, and explain why the multiplier predicts the first comparison better than the second.

c. The treasurer offers to raise the budget by \$1,000 for a financing charge of \$150. Should the distributor accept? Answer from the multipliers, then check by solving again.

Exercise 7.19. Read Erlebacher (2000) and Lau and Lau (1996). Both study the problem of this subsection with a single limit, and Lau and Lau (1996) goes on to several.

a. Erlebacher (2000) writes the optimality condition as $F_i(Q_i) = \alpha_i - \lambda\beta_i$. State how he defines α_i and β_i. Then show that when the limit is a budget, so that the resource a unit uses is its purchase cost, his condition is the one you derived in Exercise 7.13, with $\lambda = \theta$.

b. Erlebacher (2000) proposes three heuristics, H1, H2 and H3, that need no search for the multiplier. Apply all three to the spindle assembly and the blade of Exercise 7.14 under the \$18,000 budget. Do not round, and price each plan by the pair's expected profit. Find the exact optimum the same way, by bisection on θ until the spend

is within a cent of the budget, and report each heuristic's shortfall from it as a percentage. Rank the three, and say whether the ranking agrees with his results for normal demand.

c. H2 is optimal when every part's demand is uniform. Say what H2 therefore assumes about the spindle and the blade, and explain why it does well here anyway. His Section 3 says when it does best.

d. Lau and Lau (1996) show an order falling to zero as the capacity tightens. In the notation of Exercise 7.13, state when that happens. Then explain, from their Section 3, why a numerical search for the multiplier can fail near that point when a demand distribution has a positive lower bound or a long left tail.

e. Lau and Lau (1996) handle several constraints by an active-set method that solves for the multipliers of the binding constraints together. Exercise 7.17 (d) alternated two one-dimensional searches instead. Say what their method does that alternation does not, and why alternation was enough there.

7.14.5. Using the Software

Exercise 7.20. The code of Section 7.12 is in `code/`, package `inventory.newsvendor`. Report the code you wrote alongside each answer.

a. Reproduce Table 7.7 with `evaluate` on a KSL `Poisson`. Then use `optimalLevel` and `copy` to find, to the nearest dollar, the emergency purchase price below which the order falls from 7 transformers to 6.

b. Reproduce Table 7.8 with `estimate`, and confirm that the value `evaluate` gives lies inside the confidence interval in every column, not only at Q^*. Then increase the replications tenfold and report how the half-width changes; state whether it changed by the factor Section 7.10 predicts.

c. Combine Section 7.9.1 and Section 7.9.2 against the exact model: a penalty of \$40 per belt and 120 belts already on hand. Use `solve`

with the fitted horizon of `DemandFitting`, and report the order-up-to level and the order. Compare them with what the normal answers of those two sections imply together, and explain the difference from the shape of the negative binomial.

d. The scrap dealer of Example 7.5 now takes at most 60 belts, at \$12 each, and the rest must be hauled away at a cost of \$2 each. Write the profit function, and estimate it with `estimate` at $Q = 400, 405, \ldots, 520$, using 100,000 replications and the stream of Example 7.5. Confirm the best quantity by summing the same function against the negative binomial's mass function, one belt at a time. Then name the step in the derivation of Equation 7.12 that fails under this rule.

7.14.6. Using the Workbook

Exercise 7.21. Open `Chapter7Models.xlsx` and work entirely within it.

a. On the `Newsvendor` sheet, reproduce Example 7.2 and confirm that the difference between the two forms of Equation 7.10 reads zero.
b. Raise the salvage value from \$12 to \$30 and report what happens to c_o, the critical ratio, Q^* and the expected cost. Explain the direction.
c. Raise the salvage value to \$50 and describe what the sheet does. Relate the result to Section 7.9.3.
d. The `Discrete` sheet ships with Example 7.3 already entered. Confirm that the highlighted row is the one Table 7.7 gives. Then lower the emergency purchase price until the highlight moves, and report the price at which it does, to the nearest dollar.
e. Restore the sheets, and set the stockout penalty on `Newsvendor` to \$40. Read z from B21, and reproduce the expected shortage in B27 by hand from Table D.2, interpolating as Example 7.4 does. How far apart are your figure and the sheet's, and what accounts for the gap?

7.14.7. From the Literature

Exercise 7.22. Section 7.4 states three results and derives none of them. This exercise works the first two by hand and reads the third.

a. **Croston's method.** The section claims that smoothing an intermittent series directly gives an estimate that is *highest immediately after a demand*, which is when a replenishment is most likely to be triggered. Demonstrate it. Take twelve periods in which demand is 10 units in periods 1, 6 and 11 and zero otherwise, and smooth it with $\alpha = 0.2$ from a starting estimate of 2.0 units. Report the estimate after every period, and say where in the cycle it is highest and where it is lowest.

b. Now apply Croston (1972) to the same twelve periods. Smooth the **size** of a demand and the **interval** between demands as two separate series, each with $\alpha = 0.2$, each updated only in a period that has a demand, starting from a size of 6 units and an interval of 5 periods. Report the demand rate the two give after period 11, and compare it with your answer to part (a) in the same period.

c. **The bias.** Syntetos and Boylan (2005) showed that the ratio of two independent smoothed estimates does not estimate the ratio of the means. Show it on the smallest example there is. Let $A = 10$ with certainty and let B take the values 4 and 6 with equal probability, independently of A. Compute $E[A]/E[B]$ and $E[A/B]$ and say which is larger. Then say in one sentence why Croston's estimator therefore runs high, and in which direction a reorder point built on it errs.

d. **Which family, and when it matters.** Table 7.6 measured what the choice of distribution family costs on the belt and found less than a fifth of a percent. Rossetti and Ünlü (2011) ask the same question for demand over a lead time and reach a sharper conclusion. Read it and report what they vary that Table 7.6 holds fixed, what they measure the error in, and which families they recommend. Then reconcile the two findings: name the regime in which the choice of

family is cheap and the regime in which it is not, and say which one the belt is in and why.

8. Continuous and Periodic Review Systems

Learning objectives

After reading this chapter you should be able to:

- trace the state variables of an (r, Q) policy through a sequence of demands
- explain why the reorder point is compared against the inventory position
- construct the lead-time demand distribution under various demand models
- distinguish the ready rate from the fill rate and say when they coincide
- analyze the base-stock policy as a newsvendor problem
- analyze the (r, Q) policy exactly under Poisson demand
- develop approximate cost models and measure what their assumptions cost
- relate safety stock to the safety factor and the service measure chosen
- account for variability in the lead time itself
- compute policy parameters that meet a stated service target

8.1. One Item, Watched Continuously

Chapter 7 removed the assumption that demand is known, and kept the assumption that there is one decision. This chapter removes the second one. The item is stocked permanently, demand arrives at random forever, and a replenishment decision is available at every instant.

> **Where the results in this chapter come from**
>
> The policy evaluation of Section 8.4 and Section 8.5, and the optimization that follows them, are drawn from Chapter 6 of Zipkin (2000), which is the definitive treatment. The approximation of Section 8.6 is the classical one, which descends from Hadley and Whitin (1963) and appears in Nahmias and Olsen (2015) and Silver et al. (2016). Results are stated here with their sources; the proofs are in Zipkin (2000) and are not reproduced. Where this book departs from that treatment, it says so.

Two numbers define the policy. Let r represent the **reorder point** and let Q represent the **order quantity**. The rule is one sentence: whenever the inventory position **reaches or falls below** r, order Q units. It is written (r, Q), and it is the policy most stock rooms actually run, under the name of a two-bin system or a min-max when the parameters are set by hand.

Three details of its operation matter later and are easy to skate over.

The position is checked after every demand, including a demand that could not be filled. A demand that is backordered changes the position as a filled one does, so it can trigger an order, and Table 8.2 below has an instance.

Unmet demand is backordered rather than lost. The backorders accumulate and are filled when a replenishment arrives, and we assume they are filled first come first served. Nothing in the performance measures depends on that order, but the waiting time distribution of Section 8.2 does.

The lead time is the interval from placing an order to having it available to fill demand, so it includes receiving and put-away and not only transit.

Recall from Section 1.3 that an inventory system carries more than one level, and that the distinction between them is what makes this policy work.

- $I(t)$, the units physically on hand.
- $IO(t)$, the units on order and not yet received.
- $B(t)$, the units of demand that arrived, were not filled, and are owed.
- $IN(t) = I(t) - B(t)$, the **net inventory**, which can be negative.
- $IP(t) = I(t) + IO(t) - B(t)$, the **inventory position**.

The policy compares r against the position and not against what is on the shelf. That choice is the whole of why the policy works, and a reader meeting it for the first time usually has to see it fail before believing it. If the rule watched $I(t)$, then a system whose shelf is empty and whose order is in transit would order again, and again at the next demand, until the first delivery arrived. Thus, it would order the same shortage several times over. The position counts what is already coming, so an order placed is an order not placed again.

8.1.1. Working the Policy by Hand

Before any expectation is taken, we run the policy on a sequence of demands and track every state variable through it. The example is the utility storeroom of Example 4.5.

Example 8.1 (Twelve events at the storeroom)**.** The storeroom carries a pad-mount transformer. Demand runs at $\lambda = 45$ units a year and arrives one unit at a time. The supplier quotes a lead time of two months, which the ledger carries as eight weeks. The storekeeper is running $r = 4$ and

$Q = 5$, and at the start of the observation there are 6 transformers on the shelf, none on order, and nothing owed.

What is stocked? One item, the transformer.

What is the demand process? Random arrivals of one unit each, about 45 a year.

When is inventory reviewed? Continuously. The position is known after every transaction.

What triggers replenishment, and how much? The position reaching $r = 4$ or below, and the order is for $Q = 5$.

What happens to unmet demand? It is backordered. A utility crew waits for the transformer rather than buying elsewhere.

What costs are incurred, and when? They are the subject of Section 8.2, and none of them is needed to run the ledger.

Notice that $r = 4$ is **not** the policy this chapter will end up recommending for this item. It is set low, so the ledger passes through every state the system can occupy, including one the recommended policy would reach rarely.

Notation. Let t represent the week, counted from the start of the observation. The ledger carries $I(t)$, $IO(t)$, $B(t)$, $IN(t)$ and $IP(t)$ as defined above, each recorded **after** the event on that row has been processed.

We begin. At $t = 0$ nothing has happened, so $I = 6$, $IO = 0$, $B = 0$, and both IN and IP are 6.

A demand arrives in week 1. There is stock, so it is filled from the shelf and I falls to 5. Nothing is on order and nothing is owed, so the position falls to 5 as well. Since $5 > 4$, no order is placed.

A second demand arrives in week 2. The shelf falls to 4 and the position falls to 4 with it. The position has now reached the reorder point, so the storekeeper orders 5 transformers. They will arrive eight weeks later, in

week 10. The moment the order is placed, IO becomes 5 and the position jumps to $4 + 5 - 0 = 9$.

The table has been updated as follows.

Table 8.1.: The ledger through the first order. Notice that the demand lowered the position to the reorder point and the order raised it again immediately, so the position is at the reorder point only for an instant.

Week	Event	$I(t)$	$IO(t)$	$B(t)$	$IN(t)$	$IP(t)$
0	start	6	0	0	6	6
1	demand	5	0	0	5	5
2	demand, **order 5**	4	5	0	4	**9**

Demands in weeks 3, 5, 6 and 7 are each filled from the shelf, which falls to 3, 2, 1 and then 0. The position falls with them, to 8, 7, 6 and 5.

Stop at week 7 and look at what the policy does not do. The shelf is empty. Nothing can be filled until the delivery arrives, and the policy does not order, because the position is 5 and $5 > 4$. That is the case promised above: a rule watching the shelf would order here, and it would be ordering a second five transformers to cover a shortage that the first five already cover. The position knows that and the shelf does not.

The demand in week 8 arrives to an empty shelf and is backordered. Thus, I stays at 0, B rises to 1, and the net inventory becomes $0 - 1 = -1$. The position falls to $0 + 5 - 1 = 4$, which reaches the reorder point, so a second order for 5 is placed. It will arrive in week 16. Now IO is 10, and the position returns to $0 + 10 - 1 = 9$.

The demand in week 9 is backordered too, so B rises to 2, the net inventory falls to -2, and the position falls to 8.

The first order arrives in week 10. Five units come in and the two backorders are filled first, which is what "backordered" means: the demand was recorded and is owed. Thus, B returns to 0 and I becomes $5 - 2 = 3$. The units still on order fall to 5, and the position is $3 + 5 - 0 = 8$, unchanged by the arrival.

Notice that last point. **A delivery does not change the inventory position.** It converts units on order into units on hand, and the position counts both.

Demands in weeks 11 and 13 are filled from the shelf. The completed ledger is Table 8.2.

Table 8.2.: The full ledger. Ten demands, two orders and one receipt.

Week	Event	$I(t)$	$IO(t)$	$B(t)$	$IN(t)$	$IP(t)$
0	start	6	0	0	6	6
1	demand	5	0	0	5	5
2	demand, **order 5**	4	5	0	4	9
3	demand	3	5	0	3	8
5	demand	2	5	0	2	7
6	demand	1	5	0	1	6
7	demand	0	5	0	0	5

Week	Event	$I(t)$	$IO(t)$	$B(t)$	$IN(t)$	$IP(t)$
8	demand backordered, **order 5**	0	10	1	-1	9
9	demand backordered	0	10	2	-2	8
10	**receipt of 5**	3	5	0	3	8
11	demand	2	5	0	2	7
13	demand	1	5	0	1	6

The check. Every row must satisfy $IP(t) = I(t) + IO(t) - B(t)$, and every row does. Week 8 is the one to verify by hand, since it is the first row on which a backorder enters the sum: $0 + 10 - 1 = 9$. Notice that no row has all three terms non-zero. A unit is owed only when the shelf is empty, so I and B are never positive together. That check is worth performing on every ledger, because it catches an order recorded in the wrong column immediately.

Interpretation. The storekeeper placed two orders in thirteen weeks, had an empty shelf for three of them, and owed at worst two transformers. Whether that is acceptable is a question about cost and service, and Section 8.2 is where it is asked.

Three properties of the policy are now visible in Table 8.2, and we name

them before any of them is used.

The position moves within a band of width Q. It falls one unit at a time from just above $r + Q$ down to r, at which point an order restores it. That is the structure Section 8.5 turns into a distribution, and it is the reason the exact analysis of this policy is possible at all.

Everything that can go wrong happens during a lead time. Between placing an order and receiving it, nothing the storekeeper does can change the outcome. Thus, the only random quantity the policy is exposed to is the demand over that interval, which is why Section 8.3 is a section of its own.

The shelf and the position answer different questions. The position decides when to order. The shelf decides whether a customer is served. A model that confuses them will get the ordering right and the service wrong, or the reverse.

Section 8.2 gives the quantities a policy is judged by, and they are all averages of the columns of Table 8.2.

8.2. What to Measure

A policy is judged by long-run averages of the quantities in Table 8.2. Each is a time average, and the definitions are those of Section 1.3, restated here because every formula in the chapter is one of them.

- $\bar{I}$, the average units on hand.
- $\bar{B}$, the average units backordered.
- $\overline{SO}$, the proportion of time the shelf is empty.
- $\overline{OF} = \lambda/Q$, the average number of orders placed per unit time.

$\overline{OF}$ needs no analysis. Every order is for Q units and all demand is eventually filled, so orders are placed at the rate demand consumes them. Thus, $\overline{OF}$ depends on Q alone, and none of the other three does anything so

simple. The technical conditions under which these time averages converge are treated in Chapter 6 of Zipkin (2000).

8.2.1. Five Ways to State a Service Requirement

Service means how well the system is meeting the customer's requirements, and there is no single measure of it. Five are in common use, and they are not interchangeable.

a) **The probability of no stockout during a lead time.** The system does not stock out during a replenishment lead time if the demand over that lead time is at most the inventory available at its start.
b) **The proportion of demands met from stock without delay.**
c) **The proportion of time that stock is on hand**, or equivalently the proportion of time that none is.
d) **The expected number of backorders.**
e) **The expected waiting time of a backordered demand.**

Notice that (a) counts **cycles**, (b) counts **demands**, (c) counts **time**, and (d) and (e) are not proportions at all. An organization that has agreed on a number without agreeing on which of the five it measures has agreed on nothing.

The rest of this section takes the two pairs that are most often confused: (b) against (c), and (d) against (e). Section 8.7 returns to (a) and (b) as targets a planner sets.

The cost rate follows from Equation 1.15. Let $C(r, Q)$ represent the relevant cost rate:

$$C(r, Q) = k \frac{\lambda}{Q} + h \bar{I} + b \bar{B} \tag{8.1}$$

Notice the units of each term. The ordering cost is dollars per order times orders per year. The holding cost is dollars per unit per year times a

number of units. The backorder cost is dollars per unit per year times a number of units, so b charges for **how long** a demand waits and not merely that it waited. A model that charges a flat π per unit short is a different model, and Section 8.6 gives it.

8.2.2. The Ready Rate Is Not the Fill Rate

Two service measures are used constantly, they are quoted interchangeably in practice, and they are not the same quantity.

- The **ready rate** $\overline{RR} = 1 - \overline{SO}$ is the proportion of **time** that stock is on the shelf.
- The **fill rate** $\overline{FR}$ is the proportion of **demands** filled from the shelf on arrival.

The first averages over the clock and the second averages over customers. In Table 8.2 the shelf was empty from week 7 to week 10 and two of the ten demands were backordered, and there is no reason those two figures should agree.

They agree under one condition. Recall from Section B.3.4 that PASTA says a Poisson arrival finds the system in its time-stationary state. Thus, when demand arrives in a Poisson process the proportion of demands meeting an empty shelf equals the proportion of time the shelf is empty, and

$$\overline{FR} = \overline{RR} = 1 - \overline{SO} \tag{8.2}$$

Under any other arrival process Equation 8.2 fails, and it fails in the direction that matters. Demand that arrives in bursts is more likely to find the shelf empty than a randomly chosen instant is, because the burst that emptied the shelf is the same burst that is still arriving. Thus, a system reporting a 95% ready rate may be filling well under 95% of its demands.

You should therefore ask of any quoted service figure which average it is, and whether the demand process justifies treating them as one. This book keeps them apart and says which one each formula computes.

8.2.3. A Third Measure, and Why It Is the Same as the Second

An organization that does not think in units thinks in time. How long does a customer wait? Recall from Equation 1.6 that Little's Law answers this with no distributional assumption at all. Taking the system to be the set of backordered demands,

$$\overline{W} = \frac{\bar{B}}{\lambda} \tag{8.3}$$

Thus, a target on the average backorder level and a target on the average wait are the **same target** in different units, and an organization that argues about the two separately is arguing about a unit conversion.

For example, an item with $\bar{B} = 0.12$ units at $\lambda = 45$ a year has $\overline{W} = 0.12/45 = 0.00267$ years, which is

$$0.00267 \text{ years} \times \frac{365 \text{ days}}{1 \text{ year}} = 0.97 \text{ days}$$

so the average demand waits about a day. Notice that this is an average over **all** demands and not over the backordered ones. Most demands wait no time at all, and the few that wait, wait much longer than a day.

8.2.4. Service Targets Are Costs You Declined to Estimate

Equation 8.1 needs b, and b is the parameter no ledger contains. The usual response is to drop it and impose a service target instead, which Section 8.7 takes up properly.

Imposing a service target does not remove b from the model. Recall from Section 4.4.3 that a firm's observed order quantities imply a ratio of its own cost parameters whether or not the firm has ever estimated them. The same argument runs here. A service target is not an escape from b; it is a statement about b made without saying so. Section 8.7 recovers the implied value and asks whether anyone believes it.

Section 8.3 takes up the one random quantity all four averages depend on.

8.3. Lead Time Demand

Section 8.1 observed that the policy is exposed to exactly one random quantity. Let $D(L)$ represent the **lead time demand**, the demand that arrives between placing an order and receiving it, and let $\theta = E[D(L)]$ represent its mean. Everything in the rest of this chapter is a functional of the distribution of $D(L)$, and nothing in it is a functional of anything else.

That is a stronger statement than it looks. Two systems with entirely different demand patterns and lead times, whose lead time demand distributions happen to coincide, have identical performance under the same policy. Thus, the modeling effort belongs on $D(L)$, and the choice of formula for $\bar{B}$ or $\overline{FR}$ is a detail by comparison.

8.3.1. Two Ways to Build It

The period based characterization divides time into buckets. Let D_i represent the demand in period i and let L be counted in whole periods. Then

$$D(L) = \sum_{i=1}^{L} D_i \tag{8.4}$$

which is the random sum of Equation B.26 with the lead time as the count. It is the natural form when the data arrive as a history of weekly or monthly totals, which is how Section 7.3 received them.

The event based characterization counts transactions. Let $N(L)$ represent the number of demand events in the lead time and let D_i be the size of the ith one. Then

$$D(L) = \sum_{i=1}^{N(L)} D_i \tag{8.5}$$

which is the compound process of Equation B.25. It is the natural form for a slow mover, where a bucket is mostly zeros and the informative quantities are how often a demand occurs and how large it is.

Notice that the two are the same object reached from different data. You should choose between them by asking which one the organization's records can actually support, and not by which is more elegant.

Most organizations hold demand statistics for some standard bucket, a week or a month, and not for a lead time. Let σ_D represent the standard deviation of demand in one period. We take Equation 8.4 with a constant lead time of L periods and independent period demands, and Equation B.11 gives

$$E[D(L)] = \lambda L, \qquad Var[D(L)] = \sigma_D^2 L, \qquad \sigma_{D(L)} = \sigma_D \sqrt{L} \tag{8.6}$$

The standard deviation scales with the square root of the lead time and the mean scales with the lead time itself. Thus, doubling a lead time doubles the expected requirement and multiplies its standard deviation by only 1.414, which is why a longer lead time raises the reorder point more than it raises the safety stock. It is also the most common place to go wrong, because the temptation is to multiply the standard deviation by L.

8.3.2. The Moments, and the Three Cases

Equation B.30 gives the moments of Equation 8.4 directly:

$$E[D(L)] = E[L]\,E[D], \qquad Var[D(L)] = E[L]\;Var[D] + Var[L]\,\left(E[D]\right)^2$$

and Equation B.23 gives them for Equation 8.5. Both reduce to the same three cases, which the literature treats as three separate models and which are one formula with terms switched off.

Table 8.3.: The three lead time demand cases. Notice that cases 1 and 2 are case 3 with one variance set to zero, so there is one result here and not three.

Case	What is random	$E[D(L)]$	$Var[D(L)]$
1	Demand only, lead time constant at L	$L\,E[D]$	$L\;Var[D]$

Case	What is random	$E[D(L)]$	$Var[D(L)]$
2	Lead time only, demand constant at rate λ	$\lambda E[L]$	$\lambda^2 Var[L]$
3	Both	$E[L]E[D]$	$E[L]Var[D] + Var[L](E[D])^2$

Compute the two terms of case 3 separately, every time. They say different things and they are acted on differently. The first is what the customer does and the second is what the supplier does, and a total that hides the split cannot tell you which one to spend money on.

Notice what Table 8.3 does not give you. It gives two moments. Every model in this chapter needs a *distribution*, and two moments do not determine one. Thus, the procedure in practice has three steps, in this order:

1. Compute $E[D(L)]$ and $Var[D(L)]$ from the case that applies.
2. Choose a family, using Equation C.1 and the selection rules of Section C.4.2. Poisson when the ratio is near one, negative binomial when it is above one, and the gamma when the counts are large enough that a continuous model is safe. Section 8.6.2 says why the normal is not on that list.
3. Compute the performance measures and set r and Q from that distribution.

Step 2 is a choice and not a finding, and Table 7.6 measured what such a choice costs on a decision of this kind. Where the answer matters, carry two families through step 3 and report the spread.

Example 8.2 (Lead time demand for the transformer)**.** Take the transformer of Example 8.1, with $\lambda = 45$ units a year arriving one at a time. We compute $D(L)$ under two descriptions of the lead time.

Notation. Let L represent the lead time in years and let $D(L)$ represent the demand over it. Demand events arrive in a Poisson process, so the count over a fixed interval is Poisson by Equation B.15, and each event is for one unit.

Case 1, the lead time is a constant two months. Then

$$\theta = \lambda L = \frac{45 \text{ units}}{1 \text{ year}} \times \frac{2 \text{ months}}{1} \times \frac{1 \text{ year}}{12 \text{ months}} = 7.5 \text{ units}$$

and since demand is Poisson and the interval is fixed, $D(L)$ is Poisson with mean 7.5. Thus, $Var[D(L)] = 7.5$ as well, and the standard deviation is $\sqrt{7.5} = 2.74$ units.

Case 3, the lead time averages two months with a standard deviation of one month. Now $E[L] = 1/6$ year and $Var[L] = (1/12)^2 = 1/144$. The mean is unchanged at 7.5 units, and

$$Var[D(L)] = \lambda E[L] + \lambda^2 Var[L] = 7.5 + 2025\left(\tfrac{1}{144}\right) = 7.5 + 14.0625 = 21.5625$$

so the standard deviation is 4.64 units.

Read the split. The demand contributes 7.5 of the 21.5625 and the lead time contributes 14.0625. Thus, **65% of the variance the safety stock must cover comes from the supplier**, on an item whose demand nobody can control and whose lead time a purchasing department could negotiate. That is the reading Section B.5.3 asked for, on this chapter's own item.

Which family. The variance to mean ratio is $21.5625/7.5 = 2.875$, which Equation C.1 sends to the negative binomial. Matching moments by Equation 7.3 gives $\hat{p} = 7.5/21.5625 = 0.3478$ and $\hat{r} = 7.5^2/(21.5625 - 7.5) = 4.00$.

The check, and it is a better one than a moment match. Suppose the lead time is not merely variable but is built of four independent stages

averaging half a month each, which is what a purchase order, a production slot, a shipment and a receiving inspection look like. Then L is Erlang with shape 4 and rate $\mu = 24$ a year, so $E[L] = 4/24 = 1/6$ year and $Var[L] = 4/24^2 = 1/144$, the figures above. By Equation B.19 the lead time demand is then **exactly** negative binomial with $r = 4$ and

$$p = \frac{\mu}{\lambda + \mu} = \frac{24}{45 + 24} = \frac{24}{69} = 0.3478$$

The moment match and the transform agree to four figures, and they are not the same calculation. Thus, the negative binomial is what the model gives here rather than an assumption.

8.3.3. Getting the Distribution in Practice

Step 2 above assumed a family was available to choose. Three routes produce one, and they are not equally practical.

Observe it directly. Record the demand over each realized lead time and fit the resulting sample as Section 7.3 fits any sample. It is the only route that needs no assumption about independence, and it needs a long history of lead times, which most organizations do not keep.

Simulate it. Sample period demands and lead times from their own fitted distributions and accumulate. It handles dependence between the two, which the moment formulas cannot, and it produces an empirical distribution whose loss functions come from Section C.3.2.

Match moments to a family. Compute the two moments from Table 8.3 and choose a family by Section C.4.2. This is what is actually done, because it needs only what an organization already has.

The third route is what the rest of this chapter assumes has been done. The demand process now enters every formula **only** through the distribution of $D(L)$, which is the point Zipkin (2000) makes at the head of his own

treatment: two systems whose lead time demand distributions agree have identical performance under the same policy, whatever produced them.

Section 8.4 now takes the simplest possible policy, in which the position never moves at all.

8.4. The Base-Stock Model

Set $Q = 1$. Every demand of one unit triggers an order for one unit, so the policy replenishes **one for one** and the inventory position never moves. This is the **base-stock** policy, and it is where Zipkin (2000) begins for the same reason we do: with the position held still, everything else follows in three lines.

Notation. Let S represent the **base-stock level**, the value at which the inventory position is held. Let $r = S - 1$ represent the reorder point, since the position reaches r exactly when one unit has been demanded and an order for that unit is then placed. Let $\theta = E[D(L)]$ represent the mean lead time demand and σ its standard deviation. Let

$$ss = r - \theta \tag{8.7}$$

represent the **safety stock**, the reorder point in excess of expected lead time demand.

> ⚠ Safety stock is often glossed as "the expected inventory on hand when a replenishment arrives". That reading is correct only when at most one order is outstanding, which is assumption (i) of Section 8.6. It is not correct here. Under one-for-one replenishment the expected number of outstanding orders is θ itself, which for the item below is 7.5, so there are almost always several. Treat Equation 8.7 as a definition and not as an interpretation.

Notice that S and r carry the same information and the literature uses both. This book states base-stock results in S and (r, Q) results in r, and Equation 8.7 is where the two meet.

It is not a curiosity. One-for-one replenishment is what stock rooms run for items that are expensive, slow moving and critical, which is the profile of the transformer, and it is the standard policy for military and utility spare parts. It is also the building block for everything that follows: Section 8.5 obtains the general case by averaging base-stock results over a range of S.

Why the ordering cost disappears is this. Under $Q = 1$ the order rate is $\overline{OF} = \lambda$ whatever S is, so $k\lambda$ is a constant added to every alternative. Recall from Section 1.4.1 that a constant added to every alternative cannot change which alternative is least. Thus, k drops out of the choice of S, as it dropped out of Chapter 7. It does **not** drop out of the comparison between this policy and one with $Q > 1$, and Section 8.5 is where that bill arrives.

8.4.1. Three Lines of Derivation

Under one-for-one replenishment the outstanding orders are the lead time demand. An order is placed at the instant of every demand, so the orders outstanding now are exactly those triggered by the demands of the last L time units. Thus,

$$IO = D(L) \tag{8.8}$$

We substitute that into $IP = I + IO - B$ and use $IP = S$ throughout, which gives

$$S = I + D(L) - B, \qquad \text{so} \qquad IN = I - B = S - D(L) \tag{8.9}$$

Equation 8.9 is the whole model. Everything else is read off it, using $I = (IN)^+$ and $B = (-IN)^+$ from Section 1.3, and the loss functions of Section C.3.

The **expected backorder level** is the expected amount by which demand exceeds the level, which is the first order loss function evaluated at S:

$$\bar{B}(S) = E\left[(D(L) - S)^+\right] = \sum_{x \geq S} (x - S) g(x) = G^1(S) \tag{8.10}$$

For hand work from a table of the distribution function, Equation C.13 gives the form to use:

$$\bar{B}(S) = \theta - \sum_{x=0}^{S-1} G^0(x) \tag{8.11}$$

Notice the upper limit. It is $S - 1$ and not S, because $G^1(S)$ is the tail sum of G^0 from S upward and the subtraction removes the terms strictly below it. Getting that limit wrong understates the backorder level, and on the item worked below it would report 0.0823 backorders where the answer is 0.1616.

The **expected on-hand** follows from Equation 7.9 without any new work:

$$\bar{I}(S) = E\left[(S - D(L))^+\right] = S - \theta + G^1(S) \tag{8.12}$$

and the **ready rate** is the probability the net inventory is positive, that is the probability demand over the lead time is at most $S - 1$:

$$\overline{RR}(S) = P\{D(L) \leq S - 1\} = G(S - 1) \tag{8.13}$$

Under Poisson demand Equation 8.2 makes that the fill rate too.

Notice that Equation 8.12 is not an approximation. The expected on-hand exceeds $S - \theta$ by the expected backorder level, which is the statement that stock you are short of is stock you are not holding.

8.4.2. Choosing the Level

We substitute into Equation 8.1 and drop the constant $k\lambda$, which gives

$$C(S) = h\left(S - \theta + G^1(S)\right) + b\,G^1(S) = h(S-\theta) + (h+b)G^1(S) \quad (8.14)$$

which is Equation 7.10 with $c_o = h$ and $c_u = b$. Thus, the base-stock problem **is** the newsvendor problem, with the holding rate playing the overage cost and the backorder rate playing the underage cost, and the answer is the critical ratio of Equation 7.12:

$$S^* = \min\left\{S : G(S) \geq \frac{b}{b+h}\right\} \quad (8.15)$$

using the discrete rule of Equation 7.13, since S is a whole number of units.

Chapter 7 solved a problem with one period and no future; this one has an infinite horizon and a permanent stock. They have the same answer because both trade one unit too many against one unit too few, and in a one-for-one system the trade is made afresh at every demand.

Example 8.3 (One for one at the storeroom)**.** The storeroom of Example 8.1 decides to stock the transformer one for one.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at $\lambda = 45$ units a year, one unit at a time.

When is inventory reviewed? Continuously.

What triggers replenishment, and how much? Every demand, for one unit.

What happens to unmet demand? It is backordered.

What costs are incurred, and when? From Table 4.5 the unit cost is $c =$ \$3,800 and the carrying charge is $\gamma = 0.25$ a year, so $h = 0.25(3800) = \$950$ per unit per year. The utility judges that a transformer owed to a waiting crew costs \$8,550 a year in rescheduled work and reliability penalties, so $b = \$8,550$ per unit per year.

Notation. Let S represent the base-stock level in units, and let $D(L)$ be Poisson with $\theta = 7.5$ units from case 1 of Example 8.2.

The critical ratio.

$$\frac{b}{b+h} = \frac{8550}{8550+950} = \frac{8550}{9500} = 0.90$$

The level. Walk the Poisson distribution function down the $\lambda = 7.5$ column of Table D.3. Notice that $G(10) = 0.8622$ falls short of 0.90 and $G(11) = 0.9208$ clears it. Thus, $S^* = 11$ transformers.

The consequences. From Equation 8.10, $\bar{B} = G^1(11) = 0.1616$ units. From Equation 8.12, $\bar{I} = 11 - 7.5 + 0.1616 = 3.6616$ units. From Equation 8.13 the fill rate is $G(10) = 0.8622$.

The cost. By Equation 8.14,

$$C(11) = 950(3.6616)+8550(0.1616) = 3478.48+1381.35 = \$4{,}859.83 \text{ a year}$$

The check. The critical ratio is a shortcut, so we cost the neighbors directly and confirm the minimum is where the rule put it.

Table 8.4.: The base-stock cost at five levels. Notice that the minimum is at 11, which is where Equation 8.15 pointed, and that the curve is flat near it: level 10 and level 12 cost 7.4% and 4.1% more.

S	9	10	11	12	13
$\bar{I}$	2.0229	2.7993	3.6616	4.5823	5.5397
$\bar{B}$	0.5229	0.2993	0.1616	0.0823	0.0397
$C(S)$	\$6,392.69	\$5,218.57	**\$4,859.83**	\$5,057.03	\$5,601.71

Two routes, one answer.

Interpretation. The storeroom holds about three and a half transformers on average, is out of stock about 14% of the time, and owes about 0.16 of a transformer at any moment. By Equation 8.3 the average demand waits $0.1616/45 = 0.0036$ years, or 1.3 days.

The assumption to examine. We dropped k because $Q = 1$ made it a constant. At $\overline{OF} = \lambda = 45$ orders a year and $k = \$220$ an order, that constant is $45(220) = \$9{,}900$ a year, which is **more than twice the cost we just minimized**. Thus, the policy is well chosen given $Q = 1$, and $Q = 1$ has not been justified at all.

That last paragraph is the argument for the next section. One-for-one replenishment is right when an order is nearly free or a unit is nearly priceless, and the transformer is neither.

8.5. General Batch Sizes

Now let Q be any positive whole number. The inventory position no longer sits still; it walks down through a band and jumps back to the top. What makes the policy solvable is that we know exactly how it is distributed.

8.5.1. Where the Position Sits

Look again at the position column of Table 8.2. It takes the values 9, 8, 7, 6, 5 and then returns to 9. With demands of one unit, the position visits each of the Q values $r+1, r+2, \ldots, r+Q$ once per cycle and spends the same expected time at each.

That is the result the whole section rests on. In steady state,

$$IP \sim \text{Uniform}\{r+1, r+2, \ldots, r+Q\}, \qquad \text{independent of } D(L) \quad (8.16)$$

The position at the moment an order is placed is determined by demands that have **already** happened, and $D(L)$ is demand that has **not yet** happened. Thus, the two are functions of disjoint stretches of the demand process, and a process with independent increments makes them independent.

We combine Equation 8.16 with the relation Equation 8.9 already used,

$$IN = IP - D(L) \quad (8.17)$$

and that gives everything. We condition on the position, apply the base-stock results of Section 8.4, and average over the Q values. That is Equation B.7 doing the work, and it is the same move Section B.1.3 described in general.

8.5.2. The Four Measures

We average Equation 8.10 over the position, which gives

$$\bar{B}(r,Q) = \frac{1}{Q}\sum_{j=r+1}^{r+Q} G^1(j) = \frac{1}{Q}\left[G^2(r) - G^2(r+Q)\right] \quad (8.18)$$

where the second form is Equation C.12 collapsing the sum. Averaging Equation 8.13 the same way, we obtain

$$\overline{RR}(r,Q) = \frac{1}{Q}\sum_{j=r+1}^{r+Q} G(j-1) = 1 - \frac{1}{Q}\left[G^1(r) - G^1(r+Q)\right] \tag{8.19}$$

The expected on-hand needs no averaging at all. By Equation 8.17 and Equation 8.16, $E[IN] = \frac{1}{2}(Q+1) + r - \theta$, and since $I = IN + B$,

$$\bar{I}(r,Q) = \frac{Q+1}{2} + r - \theta + \bar{B}(r,Q) \tag{8.20}$$

and $\overline{OF} = \lambda/Q$ as before. Four formulas, and every one of them is exact.

Notice what is not assumed. Nothing here supposes that at most one order is outstanding, that backorders are rare, or that lead time demand is continuous. Section 8.6 makes all three of those assumptions and Equation 8.18 is the yardstick that prices them.

Two sanity checks travel with these formulas. Setting $Q = 1$ collapses every formula to Section 8.4, since the sum has one term at $j = r + 1 = S$. The $\frac{1}{2}(Q+1)$ in Equation 8.20 is the average of the integers 1 through Q, which is the discrete counterpart of the $Q/2$ that Section 3.1 obtained for a deterministic cycle. Thus, the classical average cycle stock is still in there, raised by half a unit because units are whole and by $\bar{B}$ because shortages are possible.

8.5.3. The Variance of the Backorder Level

Equation 8.18 is a mean, and a mean of a quantity that is zero most of the time and occasionally large. Its spread deserves reporting, and averaging Equation C.21 over the position we obtain

$$E\left[B^2\right] = \frac{1}{Q}\left\{2\left[G^3(r) - G^3(r+Q)\right] + \left[G^2(r) - G^2(r+Q)\right]\right\} \qquad (8.21)$$

so that $Var[B] = E[B^2] - \bar{B}^2$. Notice that this is where the third order loss function of Section C.3.6 is needed and the first place in the book that needs it. The mean backorder level collapses a sum of first order functions into a difference of second order ones; the second moment collapses a sum of second order functions into a difference of third order ones.

Example 8.4 (Evaluating a policy for the transformer)**.** Example 8.3 left $Q = 1$ unjustified. We now choose both parameters.

Notation. As in Example 8.3: $\lambda = 45$ a year, $k = \$220$ an order, $h = \$950$ per unit per year, $b = \$8,550$ per unit per year, and $D(L)$ Poisson with $\theta = 7.5$.

A starting quantity. Equation 3.17 ignores uncertainty entirely, and is still the right place to start because Section 3.7 showed its cost curve is flat:

$$Q^* = \sqrt{\frac{2k\lambda}{h}} = \sqrt{\frac{2(220)(45)}{950}} = \sqrt{20.84} = 4.57 \to 5 \text{ units}$$

The reorder point. With $Q = 5$ fixed we evaluate Equation 8.1 across r, using Equation 8.18, Equation 8.19 and Equation 8.20. Every column is exact.

Table 8.5.: Policy evaluation across reorder points at $Q = 5$, computed from Equation 8.18, Equation 8.19 and Equation 8.20. The fill rate equals the ready rate by Equation 8.2, since demand is Poisson.

r	$\bar{B}$	$\overline{FR}$	$\bar{I}$	Ordering	Holding	Backorder	Total
7	0.3854	0.7492	2.8854	\$1,980	\$2,741	\$3,295	\$8,016
8	0.2212	0.8357	3.7212	\$1,980	\$3,535	\$1,891	\$7,406
9	0.1202	0.8990	4.6202	\$1,980	\$4,389	\$1,028	**\$7,397**
10	0.0619	0.9417	5.5619	\$1,980	\$5,284	\$529	\$7,793
11	0.0302	0.9683	6.5302	\$1,980	\$6,204	\$258	\$8,442

The answer. $r = 9$ and $Q = 5$, costing \$7,397 a year at a fill rate of 0.8990.

The check. At $r = 9$, Equation 8.20 must reproduce the table: $\bar{I} = (5 + 1)/2 + 9 - 7.5 + 0.1202 = 3 + 1.5 + 0.1202 = 4.6202$ units, which it does. That check is worth performing every time, because the $\frac{1}{2}(Q + 1)$ term is the one most often written as $Q/2$.

The spread. From Equation 8.21, $E[B^2] = 0.3533$, so $Var[B] = 0.3533 - 0.1202^2 = 0.3388$ and the standard deviation of the backorder level is 0.582 units. Notice that the standard deviation is nearly five times the mean. Thus, the average backorder level of 0.12 units is not a typical state; the typical state is zero backorders, punctuated by occasions of several. Reporting the mean alone would describe a situation that never occurs.

The comparison that matters. Example 8.3 held $Q = 1$ and cost \$4,859.83 a year in holding and backorders, plus $45(220) = \$9{,}900$ a year in ordering that the section set aside. Its true total is \$14,760.

Table 8.6.: What ordering five at a time is worth on this item.

Policy	Ordering	Holding	Backorder	Total
Base stock, $S = 11$	\$9,900	\$3,478	\$1,381	\$14,760
$(r, Q) = (9, 5)$	\$1,980	\$4,389	\$1,028	**\$7,397**

Ordering five at a time **halves the cost of running this item**, and it improves the fill rate from 0.8622 to 0.8990 while doing it. Thus, the base-stock policy was not merely suboptimal here, it was worse on both counts at once.

Interpretation, and the limit. One-for-one replenishment earns its place when k is small enough that $k\lambda$ is not the dominant term. Here $k\lambda = \$9{,}900$ against a total of \$14,760, so it dominated completely. An item with the same demand and a \$10 ordering cost would tell the opposite story.

8.5.4. When the Lead Time Is Random

Everything above took the lead time as a constant, and Section 8.3 did not. That section computed $D(L)$ with a random lead time and case 3 of Table 8.3 gives its moments, so the two have to be reconciled before the formulas are used that way.

The lead time enters Equation 8.18, Equation 8.19 and Equation 8.20 only through the distribution of $D(L)$. Nothing in the derivation asked whether L was random. Thus, a random lead time is the same model with a different lead time demand distribution rather than a new model, and every formula in this section stands unaltered once that distribution is supplied.

One condition makes that true. Equation 8.17 says the net inventory is the position less the demand over a lead time, and that holds because every order placed before the current one has already arrived. Thus, orders must

be received **in the sequence they were placed**. Where a supplier can deliver a later order ahead of an earlier one, Equation 8.17 fails and the analysis above does not apply.

Notice that this is a statement about the supply arrangement and not about the policy. A shipment on a fixed route preserves the sequence. A pool of suppliers any of whom may fill an order does not.

> **i** Under one-for-one replenishment with orders that **may** cross, Section B.3.3 shows the number outstanding is Poisson with mean $\lambda E[L]$ **whatever the lead time distribution is**, so its variance does not enter at all. Notice how far that is from the section below, where the variance does most of the damage. The two describe different kinds of randomness, and Section B.3.3 says which is which.

What the variability costs is measured next. Take the transformer with the two descriptions of the supplier from Example 8.2, both averaging two months, and optimize r and Q jointly under each.

Table 8.7.: What an uncertain lead time costs on the transformer, at the same mean.

The supplier	$Var[D(L)]$	$(r, Q)^*$	Cost
Two months exactly	7.5	$(8, 7)$	\$7,225.15
Two months on average, standard deviation one month	21.5625	$(10, 8)$	\$11,079.73

The uncertainty costs **\$3,854.58 a year**, which is 53% more than running

the item against a reliable supplier. Recall from Example 8.2 that 65% of the variance is the lead time's doing. Thus, Table 8.7 is the number to put in front of a purchasing department that is choosing between a cheaper supplier and a predictable one.

Notice also the direction the policy moves. Both r and Q rise, r because the tail of $D(L)$ is heavier and Q because a longer, more variable exposure makes each cycle more expensive to start. That is the same effect Equation 8.24 will make explicit.

Everything in this section is exact, computable by hand from Table D.4 and Table D.5, and free of the assumptions the classical treatment makes. Section 8.6 now gives that classical treatment, and uses Table 8.5 to price what its assumptions cost.

8.6. The Classical Approximation

Section 8.5 evaluates a policy without approximating anything, and it is not the model most textbooks give first. The classical treatment replaces it with a cost expression that can be differentiated, and that expression is what is implemented in nearly every spreadsheet and planning package in service. It descends from Hadley and Whitin (1963) and appears in Nahmias and Olsen (2015) and Silver et al. (2016). We give it here, and then we price it, which is the reason it comes second in this book and not first.

8.6.1. Twelve Assumptions

The approximation rests on the following. Several are so ordinary that they are never stated, which is why they are listed.

(a) The system is under **continuous review**, so that whenever a demand or a replenishment occurs the inventory position is immediately determined.
(b) There is a fixed ordering cost k, in dollars per order.
(c) There is a shortage cost π, measured **per unit of backordered demand**, in dollars per unit.
(d) The unit cost is c, in dollars per unit.
(e) The holding cost is h, in dollars per unit per unit time. If a carrying charge i is given instead, then $h = ic$.
(f) The lead time L is **fixed and positive**.
(g) Demand is stationary and characterized by an expected rate λ, in units per unit time.
(h) Lead time demand $D(L)$ is a **continuous** random variable with density $g(x)$ and distribution function $G(x)$, mean $\theta = \lambda L$ and variance σ^2. Which continuous family is a separate question, and Section 8.6.2 answers it.
(i) **There is never more than one outstanding order at any point in time.** This ensures that whenever the reorder point is reached there are no orders outstanding, and therefore that lead time demand never exceeds Q.
(j) A **cycle** is the time between the receipt of two consecutive replenishment orders.
(k) The reorder point r is **non-negative**, so it is reached while $I(t)$ is still positive. Thus, there are no backorders at the moment the reorder point is reached, and $IP(t) = I(t)$ there.
(l) **The expected number of backorders at a random point in a cycle is very small compared with the expected amount on hand.**

Most are harmless or are removed later. Assumption (f) is what Section 8.5.4 lifted, and (d) was the business of Section 3.8.

Three of them do the damage, and they do it in two different ways. Assumptions (i), (k) and (l) drop terms from a cost expression that Section 8.5

computes exactly. Assumption (h) replaces the distribution of $D(L)$ itself. Those are separate errors, they can run in opposite directions, and the examples below price them separately for that reason.

8.6.2. Which Continuous Family

Assumption (h) requires a continuous distribution for a quantity that is a count. The family it is most often given is the normal, and that choice is hard to defend. Table C.1 prices its central defect: the support is the whole real line, so a normal model assigns positive probability to negative demand, and the amount is governed by the coefficient of variation. Below about $c_X = 0.3$ the defect is invisible and past $c_X = 1$ a sixth of the distribution is nonsense. Lead time demand for a slow-moving item routinely sits in the second range. Rossetti and Ünlü (2011) examine the question directly and find the normal a poor model of lead time demand across the conditions that matter.

This book uses the gamma. It is strictly positive, it accommodates any coefficient of variation, and Table C.5 and Table C.6 give its loss functions in closed form. We match the two moments of Table 8.3, which gives

$$\alpha = \frac{\theta^2}{\sigma^2}, \qquad \beta = \frac{\sigma^2}{\theta} \tag{8.22}$$

which is Section C.4.2 applied to lead time demand.

Notice that this is a choice about assumption (h) and not a repair of it. A continuous family approximating a small count is still approximating a small count, and Section 8.5 does not have to.

Assumption (i) has a diagnostic you already have. The expected number of orders outstanding at any moment is the lead time demand divided by the order quantity, θ/Q. Thus, (i) asks for θ/Q comfortably below one. Read it against Equation 8.18: (i) says $D(L)$ never exceeds Q, which is the

claim that $G^1(r+Q)$ is zero, and $G^1(r+Q)$ is the term the approximation is about to drop.

8.6.3. The Cost Model

We take the three cost terms one at a time. The purchase cost is λc and does not depend on the policy, so by Section 1.4.1 it is omitted.

Ordering. Orders are placed at the rate demand consumes them, so the expected ordering cost per unit time is $k\lambda/Q$, as in Section 8.2.

Shortage. The shortage cost per unit time is the expected shortage per cycle, times cycles per unit time, times the cost of a unit short:

$$\frac{\text{units short}}{\text{cycle}} \times \frac{\text{cycles}}{\text{time}} \times \frac{\text{dollars}}{\text{unit short}} = \frac{\text{dollars}}{\text{time}}$$

By assumption (k), $I(t) = r$ when the reorder point is reached, so a shortage occurs in a cycle exactly when $D(L)$ exceeds r, and its size is $D(L) - r$. That expectation is the first order loss function, so the shortage cost per unit time is $\pi(\lambda/Q)G^1(r)$.

Holding. Assumption (l) sets $E[B] \approx 0$, so $E[I] \approx E[IN]$. The safety stock $ss = r - \theta$ of Equation 8.7 is what remains on hand when the order arrives, and Q more arrive with it, so the level cycles between ss and $ss+Q$ and averages $Q/2+r-\theta$. The holding cost per unit time is $h(Q/2+r-\theta)$.

Putting the three together, we have

$$C(r,Q) \approx k\frac{\lambda}{Q} + h\left(\frac{Q}{2} + r - \theta\right) + \pi\frac{\lambda}{Q}G^1(r) \tag{8.23}$$

Notice that Equation 8.23 is Equation 3.17 with two terms added, and that the $Q/2$ is the deterministic cycle stock of Section 3.1 unchanged. That

is the sense in which this is the classical model: the uncertainty has been pushed entirely into r.

8.6.4. Two Conditions, Solved Together

Equation 8.23 is differentiable in both arguments. We take the partial derivative in each, using $dG^1/dr = -G^0(r)$ from Equation C.23 for the second:

$$\frac{\partial C}{\partial Q} = -\frac{k\lambda}{Q^2} + \frac{h}{2} - \frac{\pi\lambda G^1(r)}{Q^2} = \frac{h}{2} - \frac{\lambda\left[k + \pi G^1(r)\right]}{Q^2}$$
$$\frac{\partial C}{\partial r} = h - \frac{\pi\lambda}{Q}\, G^0(r)$$

Setting the first to zero and solving for Q, and the second to zero and solving for $G^0(r)$, we obtain

$$Q = \sqrt{\frac{2\lambda\left[k + \pi G^1(r)\right]}{h}}, \qquad G^0(r) = \frac{Qh}{\pi\lambda} \tag{8.24}$$

Read the first one. It is the economic order quantity with the ordering cost inflated by $\pi G^1(r)$, the expected shortage cost per cycle. **Uncertainty raises the order quantity by making each cycle more expensive to begin**, which is the whole classical result in one sentence.

Read the second one. It says the probability of a stockout during a lead time is $Qh/(\pi\lambda)$, so for a given Q the reorder point is a quantile:

$$r = G^{-1}\left(1 - \frac{Qh}{\pi\lambda}\right) \tag{8.25}$$

Neither equation solves alone. Substituting the first into the second, we obtain the root equation

$$G(r) + \frac{h}{\pi\lambda}\sqrt{\frac{2\lambda\,[k + \pi G^1(r)]}{h}} - 1 = 0 \tag{8.26}$$

which can be solved directly, or the pair can be iterated: start from the economic order quantity, solve for r, put that r back, and repeat until Q stops moving.

8.6.5. The Backorder-Cost Variant

Where the shortage is charged as b per unit per unit time rather than π per unit short, the shortage term becomes $b\bar{B}$ with $\bar{B} \approx G^2(r)/Q$, which is Equation 8.18 with the far tail dropped. The same two derivatives then give

$$Q = \sqrt{\frac{2\,[k\lambda + b\,G^2(r)]}{h}}, \qquad G^1(r) = \frac{Qh}{b} \tag{8.27}$$

Use whichever matches the cost the organization can defend. Recall from Section 8.2 that b charges for how long a customer waits and π charges for the fact that they waited, and that these are different claims about the world rather than different units for one claim.

Example 8.5 (The approximation on a fast-moving item)**.** From Table 4.5 the storeroom's compression connector has $\lambda = 6{,}000$ a year, $c = \$3.40$ and $k = \$27.50$, so by assumption (e) and the storeroom's carrying charge of 0.25 a year, $h = 0.25(3.40) = \$0.85$ per unit per year. The supplier quotes two weeks.

What is stocked? One item, the compression connector.

What is the demand process? Poisson at 6,000 units a year.

When is inventory reviewed? Continuously.

What triggers replenishment, and how much? The position reaching r, and the order is for Q.

What happens to unmet demand? It is backordered, and a crew waits.

What costs are incurred, and when? $k = \$27.50$ an order, $h = \$0.85$ per unit per year, and the utility puts the cost of a connector short at $\pi = \$8.50$, two and a half times what the part is worth, because the crew is idle either way.

Notation. Let $D(L)$ represent demand over the two-week lead time, so

$$\theta = \lambda L = \frac{6{,}000 \text{ units}}{1 \text{ year}} \times \frac{2 \text{ weeks}}{1} \times \frac{1 \text{ year}}{52 \text{ weeks}} = 230.77 \text{ units}$$

and $\sigma = \sqrt{230.77} = 15.19$ units, since the count is Poisson. Assumption (h) asks us to treat $D(L)$ as continuous, and Equation 8.22 gives the gamma to treat it as:

$$\alpha = \frac{230.77^2}{230.77} = 230.77, \qquad \beta = \frac{230.77}{230.77} = 1.00$$

Notice that matching a gamma to a Poisson always returns $\beta = 1$ and $\alpha = \theta$, since the two moments are equal. The loss functions come from Table C.5.

The iteration. Start at $Q = \sqrt{2(6000)(27.50)/0.85} = 623.09$, and solve $G^0(r) = Qh/(\pi\lambda)$ for r by inverting the gamma distribution function.

Table 8.8.: The iteration on the connector. It has converged by the third pass.

Pass	$G^0(r) = Qh/(\pi\lambda)$	r	$G^1(r)$	new Q
1	0.010385	267.33	0.0599	628.83
2	0.010481	267.28	0.0606	628.89

Pass	$G^0(r) = Qh/(\pi\lambda)$	r	$G^1(r)$	new Q
3	0.010481	267.28	0.0606	628.89

The answer. $r = 267$ and $Q = 629$. Evaluating Equation 8.23 at those whole numbers, with $G^1(267) = 0.0635$ units recomputed at the rounded reorder point,

$$C = \underbrace{27.50\frac{6000}{629}}_{262.32} + \underbrace{0.85\left(\frac{629}{2} + 267 - 230.77\right)}_{298.12} + \underbrace{8.50\frac{6000}{629}(0.0635)}_{5.15} = \$565.59 \text{ a year}$$

Notice that G^1 is recomputed at the rounded reorder point rather than carried over from the last line of Table 8.8. Rounding r changes the shortage term, and recomputing after rounding is the habit to keep.

The check that matters. Evaluate the same policy with Equation 8.18, Equation 8.19 and Equation 8.20, which assume none of (h) to (l). The true cost is **$565.14**, so the approximation overstates by $0.45, or **0.08%**. A full search over all (r, Q) puts the best policy at $(267, 628)$, costing $565.14, which is what the recommended policy costs to the nearest cent. The **policy penalty is zero**.

Why it did so well, and the two errors separately. The model assumptions and the family choice are different approximations and both are small here.

Table 8.9.: The two approximations priced separately on the connector.

	Cost
Exact, from Section 8.5	$565.14
What (i), (k) and (l) alone would report	$564.71
What the gamma then adds	+$0.88
What the approximation reports	**$565.59**

Check the diagnostics before trusting any of it. $\theta/Q = 230.77/629 = 0.37$, so assumption (i) holds comfortably; $1/Q$ is 0.16%, so the half unit assumption (k) drops is negligible; and one unit is 6.6% of σ, so a continuous treatment under (h) is defensible. Notice also the fill rate, which is 0.9999. A cheap item that is expensive to be short of rationally runs at very nearly perfect service, and the arithmetic says so without anybody setting a target.

8.6.6. When It Does Not Do So Well

The storeroom holds items at the other extreme too.

Example 8.6 (The same procedure on a slow-moving item)**.** Take the transformer of Example 8.4: $\lambda = 45$ a year, $k = \$220$, $h = \$950$ per unit per year, and $D(L)$ with $\theta = 7.5$ over a two-month lead time. Recall from Example 7.3 that an emergency purchase of a transformer costs \$9,500 against a stock price of \$3,800, so $\pi = \$5{,}700$ per unit short.

> **i** This chapter has now costed the same item's shortage two ways: as $\pi = \$5{,}700$ per unit short here, and as $b = \$8{,}550$ per unit per year in Example 8.3. Those are not two units for one number. They are two different claims about what a stockout does, and choosing between them is the modeling decision Section 8.2 described.

The gamma. By Equation 8.22, $\alpha = 7.5$ and $\beta = 1.00$.

The iteration. Starting from $Q = 4.5653$, the pair of Equation 8.24 converges to $r = 13.81$ and $Q = 6.49$, so the recommendation is $r = 14$ and $Q = 6$. Notice that it takes twenty-one passes to settle the last decimal, against three for the connector. A slow iteration is itself a signal that the model is being asked to work outside its assumptions.

What it reports, and what is true. At $(14, 6)$ with $G^1(14) = 0.0351$,

$$C = \underbrace{220\frac{45}{6}}_{1{,}650.00} + \underbrace{950\left(\frac{6}{2} + 14 - 7.5\right)}_{9{,}025.00} + \underbrace{5700\frac{45}{6}(0.0351)}_{1{,}499.25} = \$12{,}174.25 \text{ a year}$$

and the evaluation formulas give **$11,922.90**, so the reported cost is **2.11% high**.

Two errors, and they run in opposite directions. This is the finding the connector was too well behaved to show.

Table 8.10.: The two approximations priced separately on the transformer.

	Cost	Against the truth
Exact, from Section 8.5	$11,922.90	
What (i), (k) and (l) alone would report	$11,448.30	−3.98%
What the gamma then adds	+$725.95	
What the approximation reports	**$12,174.25**	+2.11%

The model assumptions understate by $474.60 and the family choice overstates by $725.95, and the two nearly cancel. **A small net error is hiding two large errors of opposite sign**, which is a worse situation than a large net error, because nothing on the page says so.

Where each piece comes from.

Table 8.11.: Each assumption priced at the recommended policy.

Source	Term	Cost
(k), discreteness	$h \times \frac{1}{2}$	−$475.00
(l), backorders negligible	$h \times B = 950(0.0022)$	−$2.06

Source	Term	Cost
(i), $D(L)$ never exceeds Q	the $G^1(r+Q)$ term	+\$2.46
(h), a gamma in place of a Poisson	the tail of the shortage term	+\$725.95

Assumption (h) is the largest term by far, and the diagnostic for it is the one Section 7.8 gave: one unit is 1/2.74, or **36.5%**, of a standard deviation here, against 6.6% on the connector. A continuous family cannot represent a count that small, whatever family it is. The half unit assumption (k) drops is the next largest, and it matters because Q is small: it is $1/Q$ of the cycle stock, 0.16% on the connector and **17%** here.

Notice that assumption (i) barely registers, at \$2.46, although $\theta/Q = 7.5/6 = 1.25$ says it is violated. The reorder point landed high enough that $G^1(r+Q)$ is nearly zero. An assumption can be violated and still cost nothing, which is why the terms are priced rather than argued about.

The policy. A full search over all (r, Q) returns $(13, 6)$, costing \$11,892.91 against the recommended policy's \$11,922.90. **The policy penalty is 0.25%.**

Two items, one procedure, opposite verdicts. On the connector the approximation found the best policy and priced it to 0.08%. On the transformer it found a policy costing 0.25% more than the best and priced it 2.11% high, with the 2.11% concealing errors of −3.98% and +6.34% that happened to cancel.

Three diagnostics can be computed before anything else.

Table 8.12.: What to compute before trusting an approximate answer.

Ask	Tests	Connector	Transformer
One unit as a fraction of σ	(h), a continuous family for a count	6.6%	36.5%
$1/Q$	(k), the half unit of cycle stock	0.16%	17%
θ/Q	(i), at most one order outstanding	0.37	1.25

The rule to carry away is this. Use the approximation to choose a policy, and evaluate the policy you chose with Equation 8.18, Equation 8.19 and Equation 8.20 before quoting a cost or a service level to anyone. When all three diagnostics are small the two answers converge and the second step is a formality. When any of them is not, the second step is the only one that is true, and Section 8.5 costs no more to run.

Section 8.7 now removes the parameter that neither model has been able to defend.

8.7. Service Levels Instead of a Shortage Cost

Every model so far has needed π or b, and Section 1.5.5 said no ledger contains either. The universal response in practice is to refuse to estimate them and impose a service level instead. This section gives the two levels that are actually used, solves the problem properly once a level is imposed, and then reads the level back into the cost it was meant to avoid.

8.7.1. Type 1: the Cycle Service Level

Let α represent the probability that a replenishment cycle passes with no stockout. By assumption (k) of Section 8.6 the on-hand equals r when the reorder point is reached, so the cycle runs out only if lead time demand exceeds r:

$$\alpha = P\{D(L) \le r\} = G(r), \qquad r = \min\{r : G(r) \ge \alpha\} \tag{8.28}$$

Notice that Q does not appear. Thus, Type 1 service fixes r by itself, which is why it is the target most often met by hand, and a reasonable companion value for Q is the economic order quantity.

That independence is also its defect. Equation 8.28 counts the cycles in which a shortage occurred and says nothing about **how large** the shortage was. A cycle short by one unit and a cycle short by forty count alike. Worse, since Q is absent, an item ordered in large batches and one ordered in small batches get the same reorder point at the same α, although the large-batch item has far fewer cycles in which to fail.

For example, at $\alpha = 0.95$ the transformer of Example 8.4 needs $G(r) \ge 0.95$, which Table D.3 gives at $r = 12$, since $G(11) = 0.9208$ falls short and $G(12) = 0.9573$ clears it.

8.7.2. Type 2: the Fill Rate

Let β represent the fraction of demand filled from the shelf. Each cycle satisfies Q units of demand and falls short by $G^1(r)$ units on average, so

$$\beta = \frac{\text{ordered in a cycle} - \text{expected short in a cycle}}{\text{ordered in a cycle}} = \frac{Q - G^1(r)}{Q} = 1 - \frac{G^1(r)}{Q} \tag{8.29}$$

and the reorder point is the root of $G^1(r) - Q(1-\beta) = 0$. Notice that Q **is** present, and that the measure counts units rather than occasions, which is what an organization that ships to customers cares about. Equation 8.29 is Equation 8.19 with the far tail dropped, so it carries assumption (i) with it.

At $\beta = 0.95$ with Q set to the economic order quantity of 5, the transformer needs $G^1(r) \leq 0.05(5) = 0.25$, and Table D.4 gives $G^1(10) = 0.2993$ and $G^1(11) = 0.1616$, so $r = 11$.

8.7.3. The Same Number Means Two Different Things

Both targets were set to 0.95 and they differ by a transformer.

Table 8.13.: Two readings of "95% service" on the same item, at $Q = 5$. The last two columns are computed from Equation 8.19 and Equation 8.28 rather than from the targets.

Target	r	Safety stock	Holding cost of it	Exact fill rate	Exact cycle service
Type 1, $\alpha =$ 0.95	12	4.5 units	\$4,275 a year	0.9838	0.9573
Type 2, $\beta =$ 0.95	11	3.5 units	\$3,325 a year	0.9683	0.9208

Read the last two columns against the first. The policy set to a 95% **cycle service level** actually fills 98.4% of demand, and the policy set to a 95% **fill rate** runs a cycle service level of 92.1%. Thus, the two measures disagree by several points on the same item and in both directions, and an

organization reporting “95% service” without saying which has reported almost nothing.

You should ask which measure a stated target refers to before acting on it, and say which one you mean when you state one.

8.7.4. Solving the Problem the Target Poses

Setting Q to the economic order quantity and then choosing r is a convenience, not an answer. Once a service level replaces the shortage cost the problem is a constrained one, and it should be written down:

$$\min_{r,Q} \; k\frac{\lambda}{Q} + h\left(\frac{Q}{2} + r - \theta\right) \qquad \text{subject to} \qquad 1 - \frac{G^1(r)}{Q} \geq \beta \tag{8.30}$$

which minimizes ordering and holding subject to the fill rate. Notice that the shortage term has gone entirely: the constraint has replaced it.

For a fixed Q the best r is the smallest feasible one, since holding cost increases in r and the constraint is the only thing pushing it up. Finding Q is harder, and Zipkin (2000) is blunt that it requires a full search. The reason is visible in Table 8.14.

Table 8.14.: The constrained program on the transformer at $\beta = 0.95$.

Q	$Q(1-\beta)$	r	$G^1(r)$	Ordering	Holding	Total
4	0.20	11	0.1616	\$2,475.00	\$5,225.00	\$7,700.00
5	0.25	11	0.1616	\$1,980.00	\$5,700.00	\$7,680.00
6	0.30	10	0.2993	\$1,650.00	\$5,225.00	**\$6,875.00**
7	0.35	10	0.2993	\$1,414.29	\$5,700.00	\$7,114.29
8	0.40	10	0.2993	\$1,237.50	\$6,175.00	\$7,412.50
9	0.45	10	0.2993	\$1,100.00	\$6,650.00	\$7,750.00
10	0.50	10	0.2993	\$990.00	\$7,125.00	\$8,115.00

Q	$Q(1-\beta)$	r	$G^1(r)$	Ordering	Holding	Total
11	0.55	9	0.5229	\$900.00	\$6,650.00	\$7,550.00
12	0.60	9	0.5229	\$825.00	\$7,125.00	\$7,950.00

Notice that the total is not unimodal in Q. It falls to \$6,875 at $Q = 6$, rises steadily to \$8,115 at $Q = 10$, and then **falls again to \$7,550 at** $Q = 11$. The cause is in the third column: r is an integer, and a larger Q loosens the constraint $G^1(r) \le Q(1-\beta)$ until it slackens enough for r to drop a whole unit, which it does between $Q = 10$ and $Q = 11$. Each such drop saves h dollars a year at a stroke. Thus, a search that stops at the first local minimum can stop in the wrong place, and only a full search is safe.

Reading the table also prices the convenience. Setting Q to the economic order quantity gives $(11, 5)$ at \$7,680 a year, where solving the program gives $(10, 6)$ at \$6,875. **The shortcut costs \$805 a year, or 10.5%**, and it does so while meeting the same service target.

8.7.5. A Shortcut That Usually Works

Zipkin (2000) offers a way to avoid the constrained search. For a target $\underline{w}$ on the **ready rate**, set

$$b = \frac{h\underline{w}}{1-\underline{w}}, \qquad k' = \frac{k}{\underline{w}} \tag{8.31}$$

and minimize the ordinary unconstrained cost of Equation 8.1 with those values. The first substitution makes the critical ratio $b/(b+h)$ equal to $\underline{w}$, and the second compensates for the ordering term.

It is a heuristic, with no guarantee of minimizing the true cost, and it is exact only under a continuous approximation. On the transformer it lands close and consistently **short**:

Table 8.15.: The heuristic against the target it is aiming at.

Target $\underline{w}$	b	k'	(r, Q)	Achieved ready rate
0.90	\$8,550	\$244.44	$(8, 7)$	0.8781
0.95	\$18,050	\$231.58	$(9, 7)$	0.9258
0.98	\$46,550	\$224.49	$(11, 6)$	0.9733

Thus, the policy is close, and it is on the **wrong side**. If the target is a statement of intent, that is fine. If it is contractual, the heuristic gives an infeasible policy and you have to check the achieved rate with Equation 8.19 and raise r until it clears.

8.7.6. Reading the Target Back into a Cost

A service target does not escape the shortage cost. It asserts a value for it, and the value can be recovered.

From a solved program. At the optimum of Equation 8.30, the second condition of Equation 8.24 must hold at the shortage cost that would have produced the same answer without a constraint. Solving it for π gives the **imputed shortage cost**

$$\pi^* = \frac{Q^* h}{\lambda\, G^0(r^*)} \tag{8.32}$$

For the transformer at $\beta = 0.95$, with $(r^*, Q^*) = (10, 6)$ and $G^0(10) = 0.1378$,

$$\pi^* = \frac{6(950)}{45(0.1378)} = \frac{5{,}700}{6.20} = \$919 \text{ per unit short}$$

Look hard at that number. Recall from Example 7.3 that an emergency transformer costs \$9,500 against a stock price of \$3,800, so a stockout costs the utility about \$5,700. The 95% fill rate target implies \$919, which is **less than a sixth of it**. Thus, on this item the service target is holding far too little rather than being over-cautious, as service targets are usually accused of being, and Example 8.6 priced the policy the true shortage cost calls for at $r = 13$ against the target's $r = 10$.

From a ready rate. The same move for the base-stock model of Section 8.4 inverts Equation 8.15 instead. Setting $G(S) = b/(b + h)$ equal to a target γ,

$$b = \frac{\gamma h}{1 - \gamma} \tag{8.33}$$

Table 8.16.: What a ready-rate target asserts about the transformer's backorder cost, at $h = \$950$.

Target γ	0.90	0.95	0.98	0.99
Implied b, per unit per year	\$8,550	\$18,050	\$46,550	\$94,050

Notice the first column. The utility's stated backorder cost of \$8,550 implies a ready rate of exactly 0.90, so the cost-based answer of Example 8.3 and a 90% target are one policy reached from opposite directions. That is the check on Equation 8.33.

These two inversions answer different questions, and both questions matter. Equation 8.32 recovers a **per-unit** cost from a solved constrained program; Equation 8.33 recovers a **rate** from a stated ready rate. Each is the move Section 4.4.3 made for the economic order quantity, and the useful question is the same one: not whether the target is high enough, but what cost would make it optimal and whether anyone believes that cost.

8.7.7. Safety Stock and the Safety Factor

The quantity a planner actually discusses is neither r nor α. It is the safety stock $ss = r - \theta$ of Equation 8.7, now read against a service target rather than a cost. Let z represent the **safety factor**, the same quantity measured in standard deviations of lead time demand:

$$z = \frac{r - \theta}{\sigma_{D(L)}}, \qquad r = \theta + z\,\sigma_{D(L)} \tag{8.34}$$

Equation 8.34 is a change of units, and its value is that z is comparable across items where ss is not. Two items with three units of safety stock are not equally protected; two items at $z = 1.645$ are, at least under a common distributional assumption.

Notice that Equation 8.34 is arithmetic and carries no assumption. It divides by a standard deviation and nothing more. What does carry an assumption is the step a planner usually takes next, which is to look z up in a table, and that requires naming a distribution.

Do not name the normal. Section 8.6.2 gives the reason and Rossetti and Ünlü (2011) gives the evidence. The habit of reading z from a standard normal table is so widespread that the safety factor and the normal are often treated as the same idea, and they are not: z is a unit conversion, and the table is a modeling choice made silently.

Using the gamma of Equation 8.22 instead, which for the transformer is $\alpha = 7.5$ and $\beta = 1.00$, Table 8.17 compares the targets against the exact Poisson answers.

Table 8.17.: The gamma against the exact Poisson on the transformer, at $Q = 5$.

Target	Exact r	Gamma r	Rounds to
Type 1, $\alpha = 0.90$	11	11.15	11
Type 1, $\alpha = 0.95$	12	12.50	12
Type 1, $\alpha = 0.99$	15	15.29	15
Type 2, $\beta = 0.90$	10	9.21	9
Type 2, $\beta = 0.95$	11	10.58	11
Type 2, $\beta = 0.99$	13	13.42	13

Five of the six agree. The miss is a Type 2 target, and that is the pattern to expect: a Type 1 target reads a **quantile** and a Type 2 target reads a **loss function**, which averages over a tail, so it asks more of the family.

For an item like this one the conclusion is plain. One unit is 36.5% of this item's standard deviation, so no continuous family will reliably return the right whole number. Equation 8.28 and Equation 8.29 can be solved directly against the Poisson distribution function and its loss function, which is what Table D.3 and Table D.4 are for, and that is what the figures in Table 8.13 came from. The safety factor remains useful for **comparing** items; it is not the right instrument for **setting** a reorder point on a slow mover.

Everything to this point has either assumed a shortage cost or replaced it with a target, and in both cases has chosen r and Q by search or by shortcut. Section 8.8 now asks what the best policy looks like, Section 8.9 says what can be known about it before it is computed, and Section 8.10 computes it exactly without searching at all.

8.8. The Cost Function and Its Shape

Everything so far has **evaluated** policies. Section 8.5 computes what a given (r, Q) delivers, Section 8.6 computes an approximation to it, and Section 8.7 imposes a target and searches. None of them has asked what the best policy looks like.

This section and the two that follow do. They follow Chapter 6 of Zipkin (2000), and the results are stated with their sources rather than proved here.

The objective is Equation 8.1:

$$C(r, Q) = k\,\overline{OF}(r, Q) + h\,\bar{I}(r, Q) + b\,\bar{B}(r, Q)$$

and the problem is to choose whole numbers r and Q that minimize it. Notice that the purchase cost is absent, as Section 1.4.1 allows, because the average purchase rate is λc whatever the policy.

8.8.1. The One-for-One Cost, and Why It Is Convex

We start with $Q = 1$, where k drops out by the argument of Section 8.4, and write the base-stock cost as a function of S:

$$C(S) = h\,\bar{I}(S) + b\,\bar{B}(S) \tag{8.35}$$

There is a compact way to see its shape. We define

$$\mathcal{C}(y) = h(y)^+ + b(-y)^+ \tag{8.36}$$

which charges h per unit of positive net inventory and b per unit of negative. It is non-negative, piecewise linear and convex, with a kink at zero. Then, since $IN = S - D(L)$ by Equation 8.9,

$$C(S) = E\left[\mathcal{C}(S - D(L))\right] \tag{8.37}$$

An expectation of a convex function of S is convex in S. Thus, $C(S)$ is convex, and that single fact is what makes the rest of this chapter tractable.

Read the two ends of it. For small S the on-hand is negligible and the backorder level falls about one for one with S, so $C(S)$ is nearly linear with slope $-b$. For large S the backorders vanish and the on-hand rises one for one, so the slope approaches $+h$. In between it turns once.

Section 8.4 already located the turn. The optimal level is the smallest S whose distribution function reaches the critical ratio $b/(b+h)$, which is Equation 8.15, and the ratio is written w from here on:

$$w = \frac{b}{b+h} \tag{8.38}$$

For the transformer, $w = 8550/9500 = 0.90$ and $S^* = 11$, costing \$4,859.83 a year.

8.8.2. The General Cost Is an Average of One-for-One Costs

We now take any Q. Recall from Equation 8.18 and Equation 8.20 that $\bar{B}(r,Q)$ and $\bar{I}(r,Q)$ are simple averages of the base-stock quantities over $s = r+1, \ldots, r+Q$. Multiplying by h and b and adding, we find that the whole objective is an average of Equation 8.35:

$$C(r,Q) = \frac{k\lambda + \sum_{s=r+1}^{r+Q} C(s)}{Q} \tag{8.39}$$

Equation 8.39 is the engine of the next two sections. It says that an (r, Q) policy costs the average of Q consecutive base-stock costs, plus the fixed-cost term $k\lambda/Q$ spread over the same Q. The two-dimensional problem has become a question about which Q consecutive values of a convex sequence to average.

The check. For the transformer at $r = 8$ and $Q = 7$, the seven values of $C(s)$ from $s = 9$ to $s = 15$ sum to \$40,676.04, and $k\lambda = \$9{,}900$, so

$$C(8,7) = \frac{9{,}900 + 40{,}676.04}{7} = \$7{,}225.15 \text{ a year}$$

which is what Equation 8.18, Equation 8.19 and Equation 8.20 give directly. Two routes, one answer, and the second route is the one that can be optimized.

8.9. What You Know Before You Solve

Zipkin (2000) proves a group of results about the optimal policy that are useful even when you intend to compute it exactly, because they say what to expect and what to check. Three of them are practical and are given here. The proofs, and refinements of the bounds, are in that chapter.

Two reference quantities are needed, and both come from models the reader already has. Let

$$Q_k = \sqrt{\frac{2k\lambda}{hw}}, \qquad C_k = hwQ_k = \sqrt{2k\lambda hw} \tag{8.40}$$

be the order quantity and cost of the **deterministic** model with planned backorders, the fourth row of Table 3.1, and let

$$C_\infty = \sqrt{bh}\,\sigma \tag{8.41}$$

be a reference cost for the **base-stock** model, built from the standard deviation of lead time demand. Notice what each ignores: Q_k knows nothing about uncertainty, and C_∞ knows nothing about the fixed cost.

8.9.1. The Optimal Band Straddles the Optimal Base-Stock Level

$$r^* < S^* \le r^* + Q^* \tag{8.42}$$

The inventory position under the optimal policy ranges over the integers $r^* + 1$ through $r^* + Q^*$, and Equation 8.42 says that range **contains** the optimal base-stock level. That is the link between Section 8.4 and Section 8.5, and it makes sense of Equation 8.39: the best Q consecutive values of a convex sequence are the ones sitting around its minimum.

It is also the most useful check you can run on a computed answer. For the transformer the optimum is $r^* = 8$ and $Q^* = 7$, and $8 < 11 \le 15$.

It gives a starting value for r as well. Zipkin (2000) suggests $r \approx S^* - \beta Q$ with β between $1 - w$ and $\frac{1}{2}$, which for the transformer is r between 7.5 and 10.3, bracketing the answer of 8.

8.9.2. The Cost Is Bounded Above and Below

$$C_k \le C^* \le \sqrt{C_k^2 + C_\infty^2} = \sqrt{2k\lambda hw + bh\sigma^2} \tag{8.43}$$

The lower bound is the deterministic cost, which is what the item would cost if demand were known: uncertainty can only make things worse. The

upper bound adds the two reference costs in quadrature, and needs nothing about the lead time demand distribution beyond its variance.

For the transformer, $Q_k = 4.81$, $C_k = \$4{,}114.49$ and $C_\infty = \sqrt{8550(950)}\,(2.7386) = \$7{,}805.05$, so

$$\$4{,}114.49 \;\le\; C^* \;\le\; \sqrt{4114.49^2 + 7805.05^2} = \$8{,}823.14$$

and the true optimum of \$7,225.15 lies between them.

Read the bounds before dismissing them as loose. Both are proportional to $\sqrt{k}$, so the sensitivity analysis of Section 3.7 carries over: Q^* and C^* respond to the ordering cost as the economic order quantity does. Both are also bounded in σ, so lead time demand uncertainty can only raise the cost, and only by a limited amount. Everything the deterministic chapters established about what drives performance continues to hold here.

They also bracket Q^*. The lower bound is Q_k itself, which for the transformer is 4.81, against a true Q^* of 7. Theorem 6.5.1 of Zipkin (2000) supplies the upper bound: Q^* cannot exceed Q_k by more than $C(S^*)/(hw)$, the base-stock cost measured in units of the holding cost at the critical ratio, which for the transformer is $4{,}859.83/855 = 5.68$ units. Rounded to whole units the bracket is $5 \le Q^* \le 11$, and the code of Section 8.17 prints it.

8.9.3. Choosing the Wrong Quantity Is Cheap

Let $C^*(Q)$ represent the best cost achievable with Q held fixed. Then, under a continuous approximation,

$$\frac{C^*(Q)}{C^*} \le E\!\left(\frac{Q}{Q^*}\right), \qquad E(x) = \frac{1}{2}\left(x + \frac{1}{x}\right) \tag{8.44}$$

where E is the economic order quantity penalty function of Equation 3.22. Recall that the deterministic model satisfies that relation with **equality**. Here it is an inequality, so the insensitivity is if anything **stronger** than in the deterministic case: choosing some other Q, by error or by design, costs less than the deterministic penalty.

That is the result a practitioner should carry away from this section. For example, a batch size chosen for a pallet, a case quantity or a supplier minimum is not necessarily expensive, and Equation 8.44 bounds what it costs before any of it is computed.

> ⚠ Equation 8.44 is proved for the **continuous** approximation, and Zipkin (2000) is explicit that the discrete analogues of these results have not been investigated. They can fail, mildly. On the transformer the bound holds at $Q = 4, 5, 6, 7, 10, 12$ and 14, and is exceeded at $Q = 8$ and $Q = 9$, where the true ratio is 1.0152 against a bound of 1.0089 and 1.0355 against 1.0317. Thus, use Equation 8.44 to judge whether a deviation is worth worrying about, and not as a guarantee.

8.10. An Exact Algorithm

Equation 8.39 and the convexity of $C(s)$ give an algorithm that returns the optimal integer (r, Q) in a finite number of steps, with no search over two dimensions and no calculus. It is Algorithm Optimize_rq of Zipkin (2000).

8.10.1. The Idea

By Equation 8.39, an (r, Q) policy costs $k\lambda/Q$ plus the average of Q consecutive values of $C(s)$. For a fixed Q, then, the best r is the one that

makes those Q values the Q **smallest** values of $C(s)$. Because $C(s)$ is convex, its Q smallest values are always consecutive, so such an r exists and is easy to find.

We now increase Q by one. The set of smallest values gains exactly one member, and by convexity it must be at one end or the other of the current window: either $C(r)$, just below it, or $C(r+Q+1)$, just above. Taking the smaller of the two extends the window correctly. Thus, $r^*(Q)$ falls as Q rises, but never by more than one per step.

Finally, let $C^*(Q)$ represent the best cost at that Q. By Equation 8.39, $QC^*(Q)$ is $k\lambda$ plus the sum of the window. Adding one more value c to the window adds c to that numerator, so $(Q+1)C^*(Q+1) = QC^*(Q) + c$. Dividing by $Q+1$ and writing Q as $(Q+1)-1$, we obtain

$$C^*(Q+1) = C^*(Q) - \frac{C^*(Q) - c}{Q+1} \tag{8.45}$$

so the cost falls exactly when the value being added is below the current average. That is the stopping rule. When the next value to be added is not below the current average, no larger Q can help.

8.10.2. The Algorithm

Algorithm 8.1 Algorithm Optimize_rq. The cost function $C(s)$ is Equation 8.35, and the loop runs at most Q^* times.

```
optimize(r, Q):

    // Step 0, initialize
    find S*, the smallest S with G(S) >= b/(b+h)
    Q <- 1
    r <- S* - 1
    Cstar <- k*lambda + C(S*)

    // Step 1, the next smallest base-stock cost
    loop:
        c <- min( C(r), C(r+Q+1) )

        // Step 2, test for termination
        if c >= Cstar:
            return r, Q, Cstar

        // Step 3, extend the window
        Q     <- Q + 1
        Cstar <- Cstar - (Cstar - c)/Q
        if C(r) was the smaller:
            r <- r - 1
```

Example 8.7 (Optimizing the transformer)**.** Take the transformer once more: $\lambda = 45$ a year, $k = \$220$, $h = \$950$ per unit per year, $b = \$8{,}550$ per unit per year, and $D(L)$ Poisson with $\theta = 7.5$.

The input. One column of numbers, computed once from Equation 8.35

with G^1 read from Table D.4.

Table 8.18.: The base-stock cost for the transformer. Notice it falls, turns once at $S^* = 11$, and rises, which is the convexity of Equation 8.37 made visible.

s	8	9	10	11	12	13	14	15	16
$C(s)$	\$8,654	\$6,393	\$5,219	**\$4,860**	\$5,057	\$5,602	\$6,347	\$7,199	\$8,106

Step 0. $S^* = 11$, so $Q = 1$, $r = 10$, and $C^* = 9{,}900 + 4{,}859.83 = \$14{,}759.83$.

Table 8.19.: Algorithm Optimize_rq on the transformer.

Step	$C(r)$	$C(r + Q + 1)$	c	Against C^*	New Q	New C^*	New r
1	\$5,219	\$5,057	\$5,057.03	below \$14,759.83	2	\$9,908.43	10
2	\$5,219	\$5,602	\$5,218.57	below \$9,908.43	3	\$8,345.14	9
3	\$6,393	\$5,602	\$5,601.71	below \$8,345.14	4	\$7,659.28	9
4	\$6,393	\$6,347	\$6,346.84	below \$7,659.28	5	\$7,396.80	9
5	\$6,393	\$7,199	\$6,392.69	below \$7,396.80	6	\$7,229.45	8
6	\$8,654	\$7,199	\$7,199.37	below \$7,229.45	7	\$7,225.15	8

Step	$C(r)$	$C(r+Q+1)$	c	Against C^*	New Q	New C^*	New r
7	\$8,654	\$8,106	\$8,105.59	**not below** \$7,225.15		stop	

The answer. $r^* = 8$, $Q^* = 7$, $C^* = \$7{,}225.15$ a year, in seven steps and with no search.

The checks. Three are available and all three pass.

A full enumeration over every (r, Q) returns the same policy and the same cost.

Equation 8.42 requires $r^* < S^* \le r^* + Q^*$, and $8 < 11 \le 15$.

Equation 8.43 requires $\$4{,}114.49 \le C^* \le \$8{,}823.14$, and \$7,225.15 lies between them.

Look at the r column. It runs 10, 10, 9, 9, 9, 8, 8: falling as Q rises, never by more than one per step, as Section 8.10.1 said it must. Watch it while the algorithm runs, because a reorder point that jumps by two is a sign the cost column is not convex.

8.10.3. When the Batch Is Large

The loop runs once per unit of Q^*, so it is quick when the optimal batch is small and slow when it is not. An item whose optimal batch is in the hundreds would take hundreds of steps from $Q = 1$, and the storeroom holds several.

Zipkin (2000) gives the remedy, and Section 8.9 supplies the numbers it needs. Start from a plausible policy rather than from $Q = 1$: take Q near Q_k from Equation 8.40, and r near $S^* - \beta Q$ with β between $1 - w$ and $\frac{1}{2}$.

Find the best r for that Q by a short local search, then compare the next value to be added against $C^*(Q)$: if it is smaller, Q is below Q^* and the loop above continues from Step 3; if it is larger, Q is above Q^* and the same reasoning run backwards decreases it.

On the transformer the warm start would begin at $Q = 5$ rather than 1, since $Q_k = 4.81$, and at r between 7.5 and 10.3. Notice that this is not a heuristic: it reaches the same optimum and simply starts closer.

8.10.4. Two Cautions

Convexity is more than the algorithm needs, but something is. What the argument actually requires is that $C(s)$ be non-increasing up to S^* and non-decreasing after it. Convexity gives that and is easy to establish from Equation 8.37.

The algorithm is not guaranteed when the penalty is on the stockout probability. If the objective charges for the *probability* of being short rather than for the backorders themselves, the cost function becomes

$$C(s) = h\bar{I}(s) + b\,\overline{SO}(s) \tag{8.46}$$

and this need not have the shape the argument requires. It happens to have it under Poisson demand, and in general it does not. Zipkin (2000) notes that suggestions to the contrary appear in the literature and warns against them. Thus, if you are minimizing against a stockout probability rather than a backorder level, check the shape of the cost column before trusting any algorithm that assumes it.

8.11. Lumpy Demand

Every model so far has assumed demand arrives **one unit at a time**. For example, a crew that draws eight connectors, or a storm that takes out four cutouts on one pole, does not. Recall from Section 7.4 that the book set this aside there, and from Section C.4.3 that it promised to take it up once the quantity being modeled was demand over a lead time. This is where the zeros stop aggregating away.

Notation. Let a **demand epoch** represent an occasion on which a demand arrives, and let λ_e represent the rate at which demand epochs occur, in epochs per unit time. Let Y represent the **lot size**, the number of units demanded at one epoch. Assume that

- the demand epochs form a Poisson process at rate λ_e;
- the lot sizes are independent of one another and of the timing of the epochs;
- $Y \geq 1$, so a demand epoch is for at least one unit; and
- $P\{Y = 1\} > 0$, so a lot size of one is possible.

Notice that the demand epoch rate is not the demand rate. Epochs occur at λ_e and each brings $E[Y]$ units, so the **demand rate** in units per unit time is

$$\lambda = \lambda_e E[Y] \tag{8.47}$$

and λ is the same λ the rest of the chapter has used throughout. Demand over any interval is then the **compound Poisson** process of Equation B.25.

Two modeling choices are built into the treatment that follows.

A lot can be partly filled. A crew asking for a lot of five units when two are on the shelf takes the two and is owed three. A customer who insists on all or nothing gives a much harder model and rarely a different conclusion.

Service is measured in units rather than customers. $\bar{B}$ counts units owed, not crews waiting. That matches Section 8.2 and it is the measure the formulas produce. In particular, when this section speaks of the fill rate it means the **unit fill rate**, the proportion of units demanded that are filled from the shelf on arrival. Section 8.11.4 derives it, and shows that it is no longer the ready rate.

8.11.1. What the Lumpiness Does to the Lead Time Demand

Recall from Equation B.31 that a compound Poisson sum needs only the **second moment** of the lot size. Over a lead time L,

$$\theta = \lambda_e E[Y] L = \lambda L, \qquad \sigma^2 = \lambda_e E[Y^2] L \tag{8.48}$$

and dividing one by the other gives the diagnostic of Equation C.1:

$$VMR = \frac{E[Y^2]}{E[Y]} > 1 \tag{8.49}$$

The ratio exceeds one whenever the lot size is ever larger than one, and it does so regardless of how the lot size is distributed. Thus, lumpy demand is never Poisson, and Section C.1.2 is where the lead time demand distribution will usually be found. That is the same conclusion Section B.5.4 reached from the other direction.

Notice also what Equation 8.48 does **not** change. The mean is λL as before, so an organization looking only at averages will see nothing at all. Everything lumpiness does, it does to the variance.

8.11.2. The Policy Needs Adapting, and Two Ways Are Used

An (r, Q) policy assumed the position lands exactly on r. With lumpy demand it can jump straight past, so the rule has to say what to do. Two adaptations are in use.

The (r, S) policy. Order whenever the position drops to r or below, and order enough to bring it back up to an order-up-to level S. The order size is $S - IP$, which is at least $S - r$ and is **random**. This is the policy Section 8.12 takes up, written there as (s, S).

The (r, nQ) policy. Keep a basic batch Q and order whole multiples of it, adding batches until the position is back above r. The order size is a random multiple of a fixed quantity. This is the policy to use when there is a natural batch, a case, a reel or a truckload.

Both collapse to an ordinary (r, Q) policy when every demand is for one unit, and both collapse to base stock when $S - r = 1$ or $Q = 1$.

> **i** Zipkin (2000) shows that the best (r, S) policy is optimal among **all** policies under the usual cost assumptions, and that (r, nQ) is not. The best (r, nQ) policy is usually close, and it is the one to reach for when the batch is physically fixed.

8.11.3. Almost Nothing Else Changes

Here is the result that makes this section short. Under an (r, nQ) policy the inventory position is still uniform on the integers $r + 1$ through $r + Q$. That is neither obvious nor an approximation. Given it, the argument of Section 8.5 runs word for word: the position is independent of lead time demand, Equation 8.17 holds, and conditioning on the position and averaging gives Equation 8.18, Equation 8.19 and Equation 8.20 **exactly as written**. The only change is that G^0, G^1 and G^2 are now the loss

functions of a compound Poisson distribution rather than a Poisson one, and that θ is $\lambda_e E[Y]L$.

The optimization carries too. Equation 8.39, the bounds of Section 8.9 and Algorithm Optimize_rq all apply unchanged, with $\lambda_e E[Y]$ wherever λ appeared.

One quantity does need care, and it is the order frequency. When a single demand consumes more than one basic batch, several batches are ordered at the same instant, and whether that counts as one order or several depends on what the ordering cost pays for. If each batch needs its own vehicle or its own receiving transaction, they are separate orders and

$$\overline{OF} = \frac{\lambda_e E[Y]}{Q} \tag{8.50}$$

as before. If the paperwork is written once whatever the size, they are one order, and the rate is lower:

$$\overline{OF} = \frac{\lambda_e}{Q}\left[E[Y] - G^1_Y(Q)\right] \tag{8.51}$$

where G^1_Y is the first order loss function of the **lot size**, not of the lead time demand. Notice that the two agree whenever Q is at least as large as the biggest lot, since then $G^1_Y(Q) = 0$ and no epoch ever needs a second batch. Thus, the distinction bites only when the basic batch is small.

8.11.4. The Fill Rate Is the One Measure That Changes

Recall that Equation 8.2 rested on demand arriving one unit at a time in a Poisson process. Lots keep half of that assumption. The demand epochs are still Poisson, so by PASTA an arriving lot finds the net inventory in its time-stationary state, $IN = IP - D(L)$, with the position uniform on $r+1$ through $r+Q$ and independent of $D(L)$. That is, the ready rate of

Equation 8.19 is the proportion of **lots** that find at least one unit on the shelf. It is not the proportion of **units** filled.

A lot of Y units that finds IN on arrival takes $\min(Y, IN^{+})$ of them, since a lot can be partly filled, and leaves the rest owed. The unit fill rate is the expected number taken divided by the expected number asked for. We write the minimum as a sum of indicators,

$$\min(Y, IN^{+}) = \sum_{j \geq 1} 1\{Y \geq j\}\, 1\{IN \geq j\}$$

and, because the lot is independent of the state it finds, the expectation factors term by term. Averaging Equation 8.17 over the position, exactly as Equation 8.19 did, we obtain

$$P\{IN \geq j\} = \frac{1}{Q} \sum_{s=r+1}^{r+Q} G(s-j) = 1 - \frac{1}{Q}\left[G^{1}(r-j+1) - G^{1}(r+Q-j+1)\right] \tag{8.52}$$

and therefore

$$\overline{FR}(r,Q) = \frac{1}{E[Y]} \sum_{j=1}^{y_{\max}} P\{Y \geq j\}\, P\{IN \geq j\} \tag{8.53}$$

where $y_{\max}$ is the largest lot that can occur. Notice that the $j = 1$ term is $\overline{RR}(r,Q)$ itself, weighted by $P\{Y \geq 1\} = 1$. When every lot is one unit there is no other term and $E[Y] = 1$, so Equation 8.53 is Equation 8.2. When lots can be larger, every further term carries a probability $P\{IN \geq j\}$ no larger than the ready rate, and the weights $P\{Y \geq j\}$ sum to $E[Y]$. Thus, under lots the unit fill rate is **below** the ready rate, which is the direction Section 8.2.2 warned of.

A third figure is sometimes quoted, the **lot fill rate**, the proportion of lots filled complete on arrival:

$$\sum_{y} P\{Y = y\}\, P\{IN \geq y\} \tag{8.54}$$

In other words, a crew that leaves with two of the four cutouts it asked for counts as half a success in Equation 8.53 and as a failure in Equation 8.54. You should say which of the three a service target refers to, because on a lumpy item they are three different numbers.

Example 8.8 (Fuse cutouts drawn in job lots)**.** From Table 4.5 the storeroom's fuse cutout moves 800 units a year, at $c = \$115$ and $k = \$82.50$, so $h = 0.25(115) = \$28.75$ per unit per year. The utility puts the backorder cost at \$287.50 per unit per year, ten times the holding cost, since a cutout short holds up a restoration.

What is the demand process? Crews draw cutouts by the job, not by the unit. A single replacement takes one, a pair takes two, and a pole-top rebuild takes four. Over several years the storeroom's issue records give

Table 8.20.: The lot size distribution for the cutout.

Lot size Y	1	2	4
Probability	0.40	0.40	0.20

so $E[Y] = 2.0$ and $E[Y^2] = 5.2$ units. Job lots arrive at $\lambda_e = 400$ a year, and Equation 8.47 returns the familiar figure, $\lambda = 400(2.0) = 800$ units a year.

The lead time demand. The supplier quotes two weeks, so by Equation 8.48

$$\theta = 800\left(\tfrac{1}{26}\right) = 30.77 \text{ units}, \qquad \sigma^2 = 400(5.2)\left(\tfrac{1}{26}\right) = 80.00, \qquad \sigma = 8.94 \text{ units}$$

The comparison that matters. Had the same 800 units a year arrived **one at a time**, the lead time demand would be Poisson with the same mean of 30.77 and a standard deviation of $\sqrt{30.77} = 5.55$ units. The lots raise it to 8.94, a **61% increase in the standard deviation at an unchanged mean**. From Equation 8.49 the variance to mean ratio is $5.2/2.0 = 2.60$, which Section C.4 sends to the negative binomial.

The policy. The compound Poisson distribution of $D(L)$ is built by convolution, its loss functions follow from Equation C.13, and Algorithm Optimize_rq runs on them unchanged.

Table 8.21.: The cutout under lumpy demand and under one-at-a-time demand at the same average rate. The three service rows coincide one at a time, by Equation 8.2, and separate under lots.

	Lots of 1, 2 or 4	One at a time
σ of lead time demand	8.94	5.55
S^*	43	38
(r^*, Q^*)	$(25, 75)$	$(24, 73)$
C^*	\$2,010.22	\$1,921.64
Ready rate, Equation 8.19	0.9060	0.9038
Unit fill rate, Equation 8.53	0.8978	0.9038
Lot fill rate, Equation 8.54	0.8957	0.9038

Lumpiness costs \$88.58 a year, or 4.6%, on identical average demand. Notice where it goes: the reorder point rises by one and the batch by two, and almost all of the extra cost is carrying the stock that covers the larger tail.

The service rows. The ready rate of 0.9060 is the proportion of lots that find a cutout on the shelf. To turn it into a unit fill rate we need Equation 8.52 at $j = 1$ through 4, since the largest lot is four. At $(25, 75)$ it gives

Table 8.22.: The terms of Equation 8.53 for the cutout at $(25, 75)$. The first entry of the second row is the ready rate.

j	1	2	3	4
$P\{Y \geq j\}$	1.00	0.60	0.20	0.20
$P\{IN \geq j\}$	0.9060	0.8960	0.8855	0.8746

and Equation 8.53 sums the products of the two rows of Table 8.22 and divides by $E[Y] = 2.0$:

$$\overline{FR} = \frac{0.9060 + 0.60(0.8960) + 0.20(0.8855) + 0.20(0.8746)}{2.0} = \frac{1.7956}{2.0} = 0.8978$$

Notice that the weights of the first row sum to 2.0, which is $E[Y]$, and that is the check on the table. The lot fill rate of Equation 8.54 takes each lot size at its own column, $0.40(0.9060)+0.40(0.8960)+0.20(0.8746) = 0.8957$. Thus, a storeroom that reports a 90.6% service level on this item is filling 89.8% of the cutouts asked for and sending 10.4% of its crews away short. The gap is under a point here, because $Q^* = 75$ is large next to a lot of four and the shelf is rarely within four units of empty. It widens as the batch shrinks toward the lot size.

The checks. A full enumeration over every (r, Q) returns $(25, 75)$ and \$2,010.22. Equation 8.42 requires $r^* < S^* \leq r^* + Q^*$, and $25 < 43 \leq 100$.

A warning from running it. The algorithm took 74 steps, because it starts at $Q = 1$ and Q^* is 75. This is the case Section 8.10.3 describes, and the warm start is the one to use on any item whose batch runs to dozens.

A full enumeration used to confirm the result must range past Q^*. On this item a search capped at $Q = 60$ returns $(27, 59)$ at \$2,067.16, which is higher than the \$2,010.22 the algorithm reports, because the optimal batch of 75 lies outside the range searched.

8.11.5. What Is Not Covered

Optimizing an (r, S) policy is harder than optimizing (r, nQ), because the cost is generally not convex in the two parameters and a continuous approximation does not rescue it. Algorithms exist and Zipkin (2000) points to them. This book computes (r, nQ) policies, notes that the best (r, S) policy is at least as good, and observes that the bounds of Section 8.9 hold for both, so the conclusions about what drives performance do not depend on which class is used.

8.12. The (s, S) Policy

Section 8.11 named two adaptations of the (r, Q) policy to lumpy demand and developed the one that orders whole batches. This section takes the other, which is the one most systems actually run.

Let s represent the reorder point and S the **order-up-to level**, with $S > s$. The rule is: whenever the inventory position falls to s or below, order enough to bring it back to S. The order size is $S - IP$, and it is **random**, which is the whole difference from (r, Q).

Notice that this is the structure Section 7.9.2 met already. There the order-up-to level was computed once; here it is a standing rule.

8.12.1. Undershoot Is Why the Policy Exists

With demand one unit at a time the position lands exactly on r, and ordering a fixed Q returns it to exactly $r + Q$. With lumpy demand it can jump straight past. Let

$$U = s - IP \text{ just before the order} \tag{8.55}$$

represent the **undershoot**, the amount by which the position passes the reorder point. Ordering a fixed Q would then leave the position at $s-U+Q$, which is random, so successive cycles start from different places and the tidy structure of Section 8.5 is gone. Ordering **up to** S restores it: every cycle starts at S, whatever the undershoot was.

The undershoot has a distribution. When the spread $S-s$ is wide compared with a lot, the reorder point is crossed by a lot arriving at a random point in the sequence of lot sizes, and the standard result for that is the **equilibrium distribution**

$$P\{U = j\} = \frac{P\{Y > j\}}{E[Y]}, \qquad j = 0, 1, 2, ... \tag{8.56}$$

whose mean is

$$E[U] = \frac{1}{E[Y]} \sum_{j \geq 0} j\, P\{Y > j\} = \frac{E[Y(Y-1)]}{2E[Y]} = \frac{E[Y^2] - E[Y]}{2E[Y]} \tag{8.57}$$

The middle step counts: $\sum_{j \geq 0} j\, 1\{Y > j\} = 0+1+\cdots+(Y-1) = Y(Y-1)/2$, so the sum is half the second binomial moment of the lot size.

Notice that Equation 8.57 is zero exactly when Y is always one, which is the case where undershoot cannot happen. Notice also that it needs the second moment of the lot size, as everything about lumpy demand does. When the spread is as narrow as a single lot, a cycle from S down past s may take only one or two lots, the undershoot then depends on $S - s$, and Equation 8.56 is an approximation; Table 8.23 shows what that costs.

8.12.2. Approximating It with an (r, Q) Policy

An (s, S) system is not an (r, Q) system, and the exact analysis of it needs a renewal-theoretic distribution for the inventory position that Zipkin (2000) supplies and this book does not. But the approximation is simple and, with one correction, good.

The approximation treats the system as an (r, Q) policy with $r = s$ and $Q = S - s$, and use Equation 8.18, Equation 8.19 and Equation 8.20 unchanged.

The correction is the one that matters. The average order size is not $S - s$. Every order is placed after an undershoot, so its size is $S - (s - U) = (S - s) + U$, and averaging we obtain

$$E[\text{order size}] = S - s + E[U] \tag{8.58}$$

Orders are placed at the rate demand consumes them, so by the argument of Section 8.2 the order frequency is

$$\overline{OF} = \frac{\lambda}{S - s + E[U]} \tag{8.59}$$

and **not** $\lambda/(S - s)$. Using the uncorrected form overstates how often the system orders, and therefore overstates the ordering cost.

Example 8.9 (The cutout under an order-up-to rule)**.** Take the fuse cutout of Example 8.8, with lots of one, two or four at probabilities 0.40, 0.40 and 0.20, so $E[Y] = 2.0$ and $E[Y^2] = 5.2$ units, and 800 units a year over a two-week lead time. From Equation 8.57,

$$E[U] = \frac{5.2 - 2.0}{2(2.0)} = 0.8 \text{ units}$$

and from Equation 8.56 the undershoot is 0 half the time, 1 unit three times in ten, and 2 or 3 units one time in ten each.

How good is the approximation? Table 8.23 compares it against an exact calculation of the policy, at four spreads between s and S.

Table 8.23.: Approximating an (s, S) policy by an (r, Q) policy on the cutout.

(s, S)	$S - s$	Exact cost	Uncorrected	Error	With Equation 8.59	Error
$(40, 44)$	4	\$14,578.99	\$17,001.21	+16.61%	\$14,251.21	−2.25%
$(38, 48)$	10	\$6,635.86	\$7,117.18	+7.25%	\$6,628.30	−0.11%
$(36, 56)$	20	\$3,755.69	\$3,876.25	+3.21%	\$3,749.33	−0.17%
$(25, 100)$	75	\$2,010.10	\$2,010.22	+0.01%	\$2,000.93	−0.46%

Read the two error columns. Uncorrected, the approximation is excellent when the spread is wide and badly wrong when it is narrow, and the error is almost entirely the ordering term: at $S - s = 4$ it assumes 200 orders a year where the truth is 171. The correction removes that, and the remaining error is under half a percent at every spread down to ten.

The rule of thumb the table gives. The uncorrected approximation is safe when $S - s$ is large compared with the lot sizes, which on this item means large compared with 4. At a spread of 75 it makes no difference whether the correction is applied. At a spread of 4, which is the size of a single large lot, nothing about the policy resembles (r, Q) and only the exact treatment is trustworthy.

The check. Equation 8.59 predicts $800/(75 + 0.8) = 10.554$ orders a year at $(25, 100)$, and an exact calculation of the policy gives 10.554. At $(36, 56)$ it predicts 38.462 against an exact 38.462. The correction is not a fudge factor; it is the average order size.

8.12.3. Where This Goes Next

Two pointers close the section.

Zipkin (2000) shows that the best (s, S) policy is **optimal among all policies** under the cost assumptions of Section 8.8, which the (r, nQ) policy of Section 8.11 is not. Computing it is harder, because the cost is generally not convex in the two parameters, and algorithms for it are cited there. Thus, the (r, Q) machinery of this chapter gives a policy that is easy to compute and slightly worse than the best available, and Section 8.9 still bounds what that costs.

That claim deserves stating in the form it was proved, because it is the only optimality result in this chapter that covers a whole class of policies rather than the best member of one family.

Theorem 8.1 (Optimality of the (s, S) form)**.** *Consider a stock point reviewed at fixed intervals over a finite horizon, facing independent random demands, charging a fixed cost k for placing an order and convex holding and shortage costs per period. Then in every period there exist numbers s and S with $s \leq S$ such that ordering up to S when the inventory position is at or below s, and ordering nothing otherwise, minimizes the expected total cost over the horizon. (Scarf 1960)*

The proof turns on a property Scarf (1960) introduced for the purpose, called **K-convexity**, which is what survives of convexity once a fixed charge is added to the cost of ordering. Notice what the theorem does and does not deliver. It says the (s, S) **form** is right, so nothing is gained by searching over more complicated rules. It does not say what s and S are,

and it does not make the cost convex in them, which is why computing the best pair is the separate problem named above.

Notice also the condition the theorem carries that this chapter has not been assuming. Scarf's result is stated for a system reviewed at **fixed intervals**, not a system watched continuously. Thus, the policy reappears in a different setting, and it is the setting the theorem was written for. When review is periodic the same rule becomes the (R, s, S) policy: every R time units, look, and if the position is at or below s, order up to S. Section 8.13 takes that up next, and the undershoot of Equation 8.55 returns there for a second reason, because a review interval lets the position fall past s even when demand arrives one unit at a time.

8.13. When Review Is Periodic

Every model so far in this chapter has assumed that the inventory position is known at all times. That is assumption (a) of Section 8.6.1, it has never been examined, and it is the one assumption in the list that is a fact about the organization rather than about the item. This section removes it.

Let R represent the **review interval**, the time between consecutive examinations of the inventory position. Under **periodic review** the position is observed at the epochs $0, R, 2R, ...$ and an order may be placed only at those moments. Recall from Section 1.2.4 that this is usually imposed rather than chosen: a delivery calendar, a weekly buying meeting, a shared truck, or a planning run that executes overnight.

The policy that goes with it is the (R, S) **policy**. Let S represent the **order-up-to level**. At every review epoch, order the difference between S and the observed inventory position. Notice that this is the (s, S) rule of Section 8.12 with the trigger removed: there is no reorder point, because the calendar decides when to order and only the quantity is left to compute.

One substitution carries the entire chapter across. That claim is the whole of this section, and the rest of it is the substitution and its consequences.

8.13.1. The Protection Interval

Fix attention on the review at time t. The position is raised to S, and the order placed there arrives at $t + L$. The next order cannot be placed until $t + R$, and it does not arrive until $t + R + L$. Thus, the stock committed at time t is the only stock available to meet demand until $t + R + L$, and nothing decided in between can change that.

Let

$$\tau = R + L \tag{8.60}$$

represent the **protection interval**. Everything that can go wrong now happens during τ rather than during L, which is the periodic counterpart of the observation made after Table 8.2.

To see what that does to the model, let u represent the time elapsed since the most recent receipt, so that $0 \le u < R$. Every order placed at or before time t has arrived by $t + L$, and no later order arrives before $t + R + L$. Thus, for the whole of the interval $[t + L,\, t + R + L)$,

$$IN(t + L + u) = S - D(L + u) \tag{8.61}$$

Compare Equation 8.61 with Equation 8.9. They are the same statement, with the fixed lead time replaced by a lead time that grows through the review cycle. Let U represent the elapsed time at a randomly chosen instant, which is uniform on $[0, R)$ because the review epochs are equally spaced. Then

$$IN = S - D(L + U), \qquad U \sim \text{Uniform}[0, R) \tag{8.62}$$

Equation 8.62 is the dual of Equation 8.16, and the pairing deserves stating plainly. Under (r, Q) the net inventory is a fixed exposure subtracted from a position that is uniform over Q **levels**. Under (R, S) it is a fixed position with an exposure that is uniform over R of **time**. That is, one policy randomizes where the position sits and the other randomizes how long the position has been sitting. Thus, both models are the base-stock model of Section 8.4 averaged over a uniform quantity, and every result in this chapter follows from whichever average applies.

Two conditions are needed for Equation 8.61, and both were needed before. Orders must be received in the sequence they were placed, for the reason Section 8.5.4 gives, and the review must be accurate, so that the position observed is the position that exists, which is the whole subject of Section 2.3.

Example 8.10 (Ten months at the storeroom)**.** Before any expectation is taken, we run the policy by hand, as Example 8.1 did for (r, Q).

The storeroom of Example 4.5 carries the pad-mount transformer. Demand runs at $\lambda = 45$ units a year, the supplier quotes a lead time of two months, and the storekeeper has moved the item onto a monthly review. The order-up-to level is $S = 13$.

What is stocked? One item, the transformer.

What is the demand process? Random arrivals of one unit each, about 45 a year, which is 3.75 a month.

When is inventory reviewed? Every month, at the month end.

What triggers replenishment, and how much? The review itself, and the order is for S less the observed position.

What happens to unmet demand? It is backordered.

What costs are incurred, and when? They are the subject of Section 8.13.2, and none of them is needed to run the ledger.

Notice that $S = 13$ is **not** the level this section will end up recommending for this item. It is set one unit low, so that the ledger passes through a shortage.

Notation. Let t represent the month, counted from the start of the observation. Let $R = 1$ month and $L = 2$ months, so that $\tau = 3$ months by Equation 8.60. An order placed at the review ending month t is received at the start of month $t + 3$, and is therefore available to fill demand in month $t+3$. The ledger carries $I(t)$, $B(t)$, $IN(t)$ and $IO(t)$ as Example 8.1 defined them, each recorded at the month end **before** that month's order is placed.

We begin. At the end of month 0 the shelf holds 5 transformers, 4 are due in month 1 and 4 more in month 2, and nothing is owed. The position is therefore $5 + 8 - 0 = 13$, which is S, so the review at the end of month 0 orders nothing. That is the steady state the policy holds the system in.

Month 1 brings a receipt of 4 and a demand of 3. The shelf rises to 9 and falls to 6. Nothing is owed, and 4 remain on order, so the position at the review is $6 + 4 - 0 = 10$. The storekeeper orders $13 - 10 = 3$, and those 3 will arrive in month 4.

Month 2 brings a receipt of 4 and a demand of 5. The shelf rises to 10 and falls to 5. Now 3 are on order, so the position is $5 + 3 - 0 = 8$ and the order is for 5, arriving in month 5.

The table has been updated as follows.

Table 8.24.: The ledger through two reviews. Notice that the position is observed **before** the order and is raised to 13 by it, so the position immediately after every review is S, whatever the demand has been.

Month	Receipt	Demand	$I(t)$	$B(t)$	$IN(t)$	$IO(t)$	$IP(t)$	Order
0			5	0	5	8	13	0

Month	Receipt	Demand	$I(t)$	$B(t)$	$IN(t)$	$IO(t)$	$IP(t)$	Order
1	4	3	6	0	6	4	10	3
2	4	5	5	0	5	3	8	5

Month 3 brings no receipt at all, because the review at the end of month 0 ordered nothing. This is the first thing periodic review does that continuous review does not: a month passes with no replenishment, and the policy is working correctly. Demand of 2 takes the shelf from 5 to 3, and with 8 now on order the position is 11 and the order is for 2.

Months 4, 5 and 6 bring receipts of 3, 5 and 2 against demands of 6, 4 and 3. The shelf runs 0, 1, 0. **Stop at month 6 and look at the shelf.** It is empty, nothing is owed, and the position is 10, so the storekeeper orders 3. Nothing in the policy reacts to the shelf being empty, and nothing should, for the reason Section 8.1 gave: the position already counts the 10 units on their way.

Month 7 is the one to study. A receipt of 6 arrives and demand is 7. Six of the seven are filled and the seventh is backordered, so I stays at 0, B rises to 1, and the net inventory is -1.

Where that shortage was decided. It was not decided in month 7. The stock available in month 7 was fixed by the review at the end of month 4, whose order arrived at the start of month 7, and the next order after that one did not arrive until month 8. Thus, the position of 13 set at the end of month 4 had to cover months 5, 6 and 7, whose demands were 4, 3 and 7, totaling 14, and $13 - 14 = -1$, which is the net inventory the ledger reports for month 7. That is Equation 8.61, verified on a row.

The completed ledger is Table 8.25.

Table 8.25.: The full ledger. Ten reviews, ten orders, and one month in which a demand was backordered.

Month	Receipt	Demand	$I(t)$	$B(t)$	$IN(t)$	$IO(t)$	$IP(t)$	Order
0			5	0	5	8	13	0
1	4	3	6	0	6	4	10	3
2	4	5	5	0	5	3	8	5
3	0	2	3	0	3	8	11	2
4	3	6	0	0	0	7	7	6
5	5	4	1	0	1	8	9	4
6	2	3	0	0	0	10	10	3
7	6	7	0	1	-1	7	6	7
8	4	2	1	0	1	10	11	2
9	3	4	0	0	0	9	9	4
10	7	3	4	0	4	6	10	3

The check. Every row must satisfy $IP(t) = I(t) + IO(t) - B(t)$, and every row does. Month 7 is the one to verify by hand, since it is the only row on which a backorder enters the sum: $0 + 7 - 1 = 6$. That check is worth performing on every ledger, because it catches an order recorded in the wrong column immediately.

The second check, and it is the one this section is about. Take any month t and compare the position set at its review against the demand of the following three months. The end of month 4 set 13 against demands of 4, 3 and 7, giving -1. The end of month 5 set 13 against 3, 7 and 2, giving 1. The end of month 7 set 13 against 2, 4 and 3, giving 4. Read those against the $IN(t)$ column at months 7, 8 and 10: they are -1, 1 and 4. Thus, Equation 8.61 holds on every row of Table 8.25, and the model of the rest of this section is nothing more than taking its expectation.

Interpretation. The storekeeper placed an order in ten months out of ten, which is what periodic review does, and was short in one of them. Notice

the contrast with Table 8.2, where two orders covered thirteen weeks. Ordering on a calendar means ordering often and in small amounts, and Section 8.13.4 is where that bill is presented.

8.13.2. The Four Measures

Section 8.2 named four long-run averages, and all four follow from Equation 8.62 without new machinery. Let $G^1_v(x)$ represent the first order loss function of $D(v)$, the demand over an interval of length v, so that the $G^1(x)$ of Section 8.4 is $G^1_L(x)$ in this notation. Let

$$\bar{G}(x) = \frac{1}{R}\int_0^R P\{D(L+u) \le x\}\,du, \qquad \overline{G^1}(x) = \frac{1}{R}\int_0^R G^1_{L+u}(x)\,du \tag{8.63}$$

represent the distribution function and the first order loss function of $D(L+U)$, averaged over the review interval. The bar on $\overline{G^1}$ covers the whole loss function because it is the loss function that is averaged; the superscript is still its order, and not a power or an inverse. Notice that Equation 8.63 is doing for (R,S) what the sums in Equation 8.18 and Equation 8.19 do for (r,Q). One averages over Q levels and the other integrates over R of time, and both are Equation B.7 conditioning on a uniform quantity.

The order frequency needs one observation. Every order raises the position to S, so the next review finds the position below S, and places an order, exactly when some demand has arrived in between. A review that finds no demand since the last one has nothing to order; month 0 of Table 8.24 is such a review. Thus,

$$\overline{OF} = \frac{P\{D(R) \ge 1\}}{R} \tag{8.64}$$

and under Poisson demand $P\{D(R) \geq 1\} = 1 - e^{-\lambda R}$. For example, the transformer reviewed monthly has $\lambda R = 3.75$, so a month without a single demand has probability $e^{-3.75} = 0.0235$, and the storeroom orders $12(1-0.0235) = 11.72$ times a year rather than 12. When a review interval holds several units of demand the probability is near one and $\overline{OF}$ is nearly $1/R$; when it holds less than one, most reviews order nothing.

Notice how far that is from Equation 8.18 and its neighbors. Under (r, Q) the order frequency was λ/Q, which is proportional to the demand rate. Here it depends on the calendar almost entirely. For example, halving the transformer's demand takes λR to 1.875 and the monthly order frequency only from 11.72 to 10.16 a year. Thus, an item whose demand halves incurs nearly the same ordering cost under periodic review, which is the first hint of where this policy is expensive.

The expected backorder level is the time average of the base-stock result Equation 8.10 over the cycle:

$$\bar{B}(S) = \overline{G^1}(S) \tag{8.65}$$

The expected on-hand needs no integral at all. Taking expectations in Equation 8.62, we have $E[IN] = S - E[D(L+U)]$, and since U is uniform on $[0, R)$ the expected demand over the exposure is $\lambda(L + R/2)$. Using $I = IN + B$ as Section 8.5.2 did, we obtain

$$\bar{I}(S) = S - \lambda\left(L + \frac{R}{2}\right) + \bar{B}(S) \tag{8.66}$$

Read the $\lambda R/2$ in Equation 8.66, because it is the one new quantity in this section. We write Equation 8.66 as $\bar{I} = ss + \lambda R/2 + \bar{B}$, where $ss = S - \lambda\tau$ is the safety stock of Equation 8.7 carried over the protection interval. The middle term is stock that exists only because the system orders on a calendar. That is, reviewing every R time units forces the system to carry λR units of demand between deliveries, and it holds half of that on

average. Thus, the $\lambda R/2$ is the cycle stock of Section 3.1 with the order quantity written as λR, and it is the exact counterpart of the $\frac{1}{2}(Q+1)$ in Equation 8.20.

The ready rate is the time-averaged probability that the net inventory is positive, which by Equation 8.62 is the probability that $D(L+U)$ falls below S:

$$\overline{RR}(S) = \bar{G}(S-1) \tag{8.67}$$

The fill rate has its own derivation, and the two must agree. Demand is filled late during the cycle running from $t+L$ to $t+R+L$ exactly to the extent that the backlog grows across it. Thus, the expected units short in a cycle is the backlog at its end less the backlog at its start,

$$E[\text{units short per cycle}] = G^1_{R+L}(S) - G^1_L(S) \tag{8.68}$$

and the expected demand in a cycle is λR, so

$$\overline{FR}(S) = 1 - \frac{G^1_\tau(S) - G^1_L(S)}{\lambda R} \tag{8.69}$$

Notice the shape of Equation 8.69 against Equation 8.19. Both are one minus a difference of two first order loss functions divided by the expected demand in a cycle. Under (r, Q) the two loss functions are evaluated at levels separated by Q; under (R, S) they are evaluated at the same level over intervals separated by R. That is the level-against-time duality again, and it is the clearest place in the chapter to see it.

Equation 8.67 and Equation 8.69 are different quantities computed two different ways, and they agree here for the reason Equation 8.2 gives and no other: demand is Poisson, so PASTA makes the proportion of demands finding an empty shelf equal to the proportion of time the shelf is empty.

Under any other demand process they part company, and Section 8.2.2 says in which direction.

Four formulas again, and every one of them is exact. Nothing here supposes that at most one order is outstanding, which under periodic review would be a severe restriction: Table 8.25 never once had fewer than three units on order.

> ⚠ It is tempting to evaluate the measures at the **end** of the protection interval and be done, that is, to use $G^1_\tau(S)$ in place of $\overline{G^1}(S)$ in Equation 8.65. Do not. The end of the protection interval is the worst instant of the cycle, not a typical one, so $G^1_\tau(S)$ is an upper bound on $\bar{B}(S)$ and not an estimate of it. On the item worked below it overstates the backorder level by a factor of 2.8. The end-of-interval quantity is the right thing for a **cycle service level**, which asks about that instant on purpose, and the wrong thing for anything that is averaged over time.

Computing $\overline{G^1}$ by hand is easier than it looks. The integral in Equation 8.63 looks like an obstacle and is not one. The integrand is smooth in u, so Simpson's rule on three points is accurate to a fraction of a percent:

$$\overline{G^1}(S) \approx \frac{1}{6}\left[G^1_L(S) + 4\,G^1_{L+R/2}(S) + G^1_\tau(S)\right] \tag{8.70}$$

Thus, three evaluations of the first order loss function, at mean demands over L, $L+R/2$ and τ, give the backorder level. Notice that Table D.4 will usually not carry all three as printed columns, since $\lambda(L+R/2)$ and $\lambda\tau$ are whatever the calendar makes them. Compute them from Equation C.13, or from a worksheet, and use the table to check the one mean that does appear. The worked example checks Equation 8.70 against the exact integral.

The same three-point rule applies to $\bar{G}$, since Equation 8.63 integrates the distribution function over the same interval in the same way. Thus,

Equation 8.72 can be worked from three columns of Table D.3 whenever the calendar puts the three means on printed columns.

The rule has a range, however. Simpson's three-point rule is exact for a cubic and the integrand is not one, so the error grows with the curvature the interval spans. On the item worked below it is invisible at a monthly review, 3.9% at a quarterly one, 17% at a half-yearly one and 66% at an annual one, each at the level the rule itself selects. Thus, use Equation 8.70 when the review interval is short against the lead time, and integrate properly when it is not. Section 8.16.9 made the same kind of statement about the worksheet, and for the same kind of reason.

A cruder reading of Equation 8.70 travels with it, and needs no arithmetic at all. The exposure grows through the cycle, so $G^1_L(S) \le \overline{G^1}(S) \le G^1_\tau(S)$ always. Thus, the two endpoint values bracket the answer, and a bracket that is narrow enough settles the question without the weighted average.

8.13.3. Choosing the Level

With R fixed by the calendar, only S is left to choose. We substitute Equation 8.64, Equation 8.65 and Equation 8.66 into Equation 8.1, which gives

$$C(S) = k\,\overline{OF} + h\left(S - \lambda\left(L + \frac{R}{2}\right)\right) + (h+b)\,\overline{G^1}(S) \tag{8.71}$$

which is Equation 8.14 with $\overline{G^1}$ in place of G^1 and a constant ordering term that no choice of S can change, since Equation 8.64 does not involve S. Thus, the argument of Section 8.4.2 runs again without modification, and the answer is again a critical ratio:

$$S^* = \min\left\{S : \bar{G}(S) \ge \frac{b}{b+h}\right\} \tag{8.72}$$

Notice which distribution Equation 8.72 uses. It is $\bar{G}$, the distribution function averaged over the review interval, and not G_τ, the distribution function of demand over the full protection interval. The two are easy to confuse and they give different answers, because $\bar{G}$ averages the exposure over the cycle while G_τ takes it at its worst. Applying the critical ratio to G_τ on the item below returns a level two units too high, at 11% more cost. Thus, the periodic-review problem is still the newsvendor problem of Chapter 7, and the only thing that has changed since Equation 8.15 is which random variable the critical ratio is applied to.

Example 8.11 (The transformer reviewed monthly)**.** Example 8.10 ran the policy at $S = 13$ without asking whether 13 was the right level. We now choose it.

Notation. As in Example 8.3: $\lambda = 45$ units a year, $k = \$220$ an order, $h = \$950$ per unit per year and $b = \$8,550$ per unit per year. Let $R = 1$ month and $L = 2$ months, so $\tau = 3$ months by Equation 8.60. Demand is Poisson, so $D(v)$ is Poisson with mean λv for every interval length v.

The three means the calculation needs. Working the unit conversion out rather than asserting it,

$$\lambda L = \frac{45 \text{ units}}{1 \text{ year}} \times \frac{2 \text{ months}}{1} \times \frac{1 \text{ year}}{12 \text{ months}} = 7.5 \text{ units}$$

and the same chain at $\tau = 3$ months gives $\lambda\tau = 11.25$ units, and at $L + R/2 = 2.5$ months gives 9.375 units. Notice that the middle one is the mean exposure of Equation 8.66 and the last is the worst.

The level. The critical ratio is unchanged from Example 8.3 at $b/(b+h) = 8550/9500 = 0.90$, and Equation 8.72 applies it to $\bar{G}$.

Table 8.26.: The two distribution functions the level might be read from. Notice that $\bar{G}(13) = 0.8928$ falls short of 0.90 and $\bar{G}(14) = 0.9340$ clears it.

S	12	13	14	15	16
$\bar{G}(S)$	0.8339	0.8928	**0.9340**	0.9612	0.9782
$G_\tau(S)$	0.6611	0.7576	0.8352	0.8935	0.9344

Thus, $S^* = 14$ transformers.

The trap in Table 8.26. Read the second row instead of the first and the rule returns $S = 16$, because $G_\tau(15) = 0.8935$ misses 0.90 and $G_\tau(16) = 0.9344$ clears it. That level costs \$9,301.76 a year against \$8,397.22, which is 10.8% more for a service level nobody asked for. The error is not one of arithmetic; it answers the question "how much do I need at the worst instant" when the cost model asked "how much do I need on average".

The backorder level by hand. The three means are 7.5, 9.375 and 11.25 units, and only the first is a column of Table D.4, which gives $G^1_L(14) = 0.0181$ units. Computing the other two from Equation C.13 gives $G^1_{L+R/2}(14) = 0.1156$ and $G^1_\tau(14) = 0.4200$ units. By Equation 8.70,

$$\bar{B}(14) \approx \frac{0.0181 + 4(0.1156) + 0.4200}{6} = 0.1501 \text{ units}$$

The check on the hand rule. Evaluating the integral in Equation 8.63 numerically gives 0.150059, against 0.150061 from Equation 8.70. Thus, three evaluations reproduce the integral to five figures, and Equation 8.70 can be used without apology. Notice also that the bracket holds: 0.0181 and 0.4200 straddle 0.1501, though not tightly enough here to settle the level on their own.

The consequences. From Equation 8.66, $\bar{I} = 14 - 9.375 + 0.1501 = 4.7751$ units. From Equation 8.69 the fill rate is 0.8928, and Equation 8.67 gives

$\bar{G}(13) = 0.8928$ for the ready rate. Notice that those two agree to four figures, and that they were computed from different formulas along different routes. They agree because demand is Poisson, and Equation 8.2 is why.

The cost. By Equation 8.64 the storeroom orders $12(1 - e^{-3.75}) = 11.7178$ times a year, so the ordering term is $220(11.7178) = \$2{,}577.91$ a year, and by Equation 8.71

$$C(14) = 2577.91 + 950(4.7751) + 8550(0.1501) = 2577.91 + 4536.31 + 1283.00 = \$8{,}397.22 \text{ a year}$$

The check. The critical ratio is a shortcut, so we cost the neighbors directly and confirm the minimum is where the rule put it.

Table 8.27.: The periodic-review cost at five levels. Notice that the minimum is at 14, which is where Equation 8.72 pointed, and that the curve is flat near it: level 13 and level 15 cost 0.8% and 3.8% more.

S	12	13	14	15	16
$\bar{B}$	0.4233	0.2572	0.1501	0.0841	0.0453
$\bar{I}$	3.0483	3.8822	4.7751	5.7091	6.6703
$\overline{FR}$	0.7549	0.8339	0.8928	0.9340	0.9612
$C(S)$	\$9,093.07	\$8,465.46	**\$8,397.22**	\$8,720.37	\$9,301.76

Two routes, one answer.

Interpretation. The storeroom holds about 4.8 transformers on average, fills 89% of demands from the shelf, and owes about 0.15 of a transformer at any moment. By Equation 8.3 the average demand waits $0.1501/45 = 0.00333$ years, which is

$$0.00333 \text{ years} \times \frac{365 \text{ days}}{1 \text{ year}} = 1.22 \text{ days}$$

The comparison that matters. Table 8.7 optimized (r, Q) on this same item against this same supplier and got $(8, 7)$ at \$7,225.15 a year. A monthly review costs \$8,397.22, which is **\$1,172.07 a year more**, or 16.2%. Nothing about the item changed. The storeroom simply stopped watching it, and Section 8.13.6 is where that number is taken apart.

8.13.4. Choosing the Interval

Example 8.11 took R as given, which is usually right. When the review interval is open, Equation 8.71 decides it.

Hold the shortage term aside for a moment, as Section 8.6 does for the same reason, and take $P\{D(R) \geq 1\}$ as one, which it nearly is at any interval holding several units of demand. Then only the ordering and cycle-stock terms depend on R:

$$C(R) \approx \frac{k}{R} + \frac{h\lambda R}{2} \tag{8.73}$$

which is Equation 3.17 with λR in place of Q. We differentiate and set the derivative to zero, $-k/R^2 + h\lambda/2 = 0$, and solving for R gives

$$R^* = \sqrt{\frac{2k}{h\lambda}} = \frac{Q^*_{\text{EOQ}}}{\lambda} \tag{8.74}$$

The best review interval is the economic order quantity expressed as a time supply. That is, the calendar question and the batch question are the same question, asked in different units. Recall from Section 3.1 that the EOQ balances a fixed charge against the cost of carrying half a batch; here it balances a fixed charge against the cost of carrying half a review interval of demand, and Equation 8.66 is what makes the two identical.

For example, on the transformer, $R^* = \sqrt{2(220)/(950 \times 45)} = \sqrt{0.010292} = 0.1015$ years, which is

$$0.1015 \text{ years} \times \frac{52 \text{ weeks}}{1 \text{ year}} = 5.3 \text{ weeks}$$

so the rule recommends reviewing the transformer about every five weeks. Notice that this agrees with $Q^*_{\text{EOQ}}/\lambda = 4.57/45 = 0.1015$ years, as Equation 8.74 says it must.

The rule is better than its derivation deserves. Table 8.28 optimizes S at each of several intervals and reports the true cost, including the shortage term that Equation 8.73 dropped.

Table 8.28.: Cost against review interval on the transformer, with S optimized at each interval.

R	S^*	Ordering	Holding	Backorder	Total
2 weeks	12	\$4,707	\$3,630	\$1,590	\$9,926
4 weeks	13	\$2,770	\$3,797	\$1,944	\$8,512
1 month	14	\$2,578	\$4,536	\$1,283	\$8,397
5.3 weeks	14	\$2,146	\$4,213	\$1,863	\$8,222
6 weeks	15	\$1,896	\$4,823	\$1,477	\$8,196
8 weeks	16	\$1,429	\$5,001	\$1,932	\$8,361
13 weeks	20	\$880	\$6,749	\$1,955	\$9,584

Read Table 8.28 down the last column. The true optimum is at about 6.2 weeks, costing \$8,187. Equation 8.74 recommended 5.3 weeks, which costs \$8,222, or 0.43% more. Thus, a rule that ignores shortages entirely lands within half a percent of the answer, which is the same insensitivity Section 3.7 found for the EOQ and Section 8.9.3 found for Q.

Read the same column for how flat it is. Anything between four and eight weeks costs within 5% of the best. Thus, the review interval is almost never worth an argument, and an interval that fits the delivery calendar, the buying meeting or the truck should be preferred to one that

fits Equation 8.74. You should use the formula to find out whether the calendar is badly wrong, not to overrule it.

Notice the one column that does not behave. The backorder column falls and rises without pattern, because S is a whole number and each row rounds a different way. That is rounding noise and not a property of the policy.

8.13.5. The (R, s, S) Policy

The (R, S) policy orders at every review that follows any demand at all, however small the order. When the fixed charge k is large that is wasteful, and the repair is to attach a trigger to the calendar. Let s represent a reorder point with $s < S$. The (R, s, S) **policy** is: every R time units, observe the inventory position; if it is at or below s, order up to S; otherwise do nothing.

Notice that this is the (s, S) policy of Section 8.12 with the position examined on a calendar rather than continuously, and that Section 8.12.3 promised it here. Setting $s = S - 1$ recovers (R, S), since any demand at all in a review interval puts the position at or below $S - 1$. Setting $R \to 0$ recovers (s, S). Thus, the three policies are one policy with two limits.

The undershoot returns, for a second reason. Recall from Section 8.12.1 that the undershoot U of Equation 8.55 is the amount by which the position passes below the reorder point, and that under continuous review it is caused by demand arriving in lots: a lot of four can carry the position four units past s, while unit demands land on s exactly. Under periodic review it happens even when demand arrives one unit at a time, because a whole review interval of demand accumulates before anyone looks.

That is the more important source of the two. Let $D(R)$ represent the demand in one review interval. Applying Equation 8.57 with $D(R)$ in the role of the lot,

$$E[U] = \frac{E[D(R)^2] - E[D(R)]}{2E[D(R)]} \tag{8.75}$$

For example, take the transformer on its monthly review, where $D(R)$ is Poisson with mean $\lambda R = 3.75$ units. A Poisson random variable has $E[X^2] = \mu + \mu^2$, so Equation 8.75 gives

$$E[U] = \frac{(3.75 + 14.0625) - 3.75}{2(3.75)} = \frac{14.0625}{7.5} = 1.875 \text{ units}$$

Notice that $1.875 = \lambda R/2$, and that this is not a coincidence. For Poisson demand $E[X^2] - E[X] = \mu^2$, so Equation 8.75 collapses to $\mu/2$ for every mean. Thus, when the spread $S - s$ is wide compared with a review interval's demand, the expected undershoot under periodic review with Poisson demand is half a review interval of demand, which is the same $\lambda R/2$ that Equation 8.66 identified as cycle stock.

The qualification matters at the narrow end. Recall that $s = S - 1$ is the (R, S) policy itself, where any demand at all in a review interval triggers an order. There the undershoot is the review interval's demand less one, given that there was some, so

$$E[U] = \frac{\lambda R}{P\{D(R) \geq 1\}} - 1 = \frac{3.75}{1 - e^{-3.75}} - 1 = 2.8403 \text{ units}$$

for the transformer, half again the long-run figure. Thus, Equation 8.75 describes a policy that skips reviews often, which is the only kind for which the undershoot is worth computing. That is, the stock the calendar forces the system to carry and the amount by which the calendar lets the position drift past its trigger are the same number seen from two sides.

Compare that 1.875 units against the 0.8 units of Example 8.9, which was the undershoot a lot size of one, two or four produced under continuous review. Thus, on this item the review interval contributes more than twice

the undershoot that lumpy demand did, and a model that carries the lot-size undershoot while ignoring the review-interval one has kept the smaller term.

Evaluating the policy needs one correction and one warning. The correction of Equation 8.59 applies unchanged, with the order frequency reduced further by the reviews at which no order is placed. The exact analysis needs the renewal-theoretic position distribution that Zipkin (2000) supplies and this book does not, and the approximation in general use is Silver et al. (2016), which fits s and S from the moments of $D(\tau)$ and the cost parameters. You should reach for (R, s, S) only when k is large enough that skipping reviews is common. When almost every review produces an order, (R, S) is the same policy with less arithmetic.

8.13.6. The Price of Not Watching

Periodic review is nearly always imposed rather than chosen, but the choice is sometimes real, and when it is the question is what continuous review is worth. This section prices it on the transformer.

The comparison has to be fair, so each policy is optimized on its own terms against the same item, the same supplier and the same costs. Table 8.7 gave the continuous answer, $(r, Q) = (8, 7)$. The periodic answer comes from optimizing R and S together, which Table 8.28 located at about 6.2 weeks with $S = 15$. A search over R in steps of 0.001 year puts it at $R = 0.120$ year.

Table 8.29.: The two policies on the transformer, each optimized.

	Continuous, $(r, Q) = (8, 7)$	Periodic, $R = 0.120$ year, $S = 15$
Safety stock, units	0.50	2.10
Cycle stock, units	4.00	2.70
$\bar{I}$, units	4.6617	4.9897
$\bar{B}$, units	0.1617	0.1897

	Continuous, $(r, Q) = (8, 7)$	Periodic, $R = 0.120$ year, $S = 15$
$\overline{FR}$	0.8781	0.8819
Ordering	\$1,414.29	\$1,825.05
Holding	\$4,428.59	\$4,740.19
Backorder	\$1,382.28	\$1,621.68
Total	**\$7,225.15**	**\$8,186.92**

Read the fill rate row first. The two policies deliver 0.8781 and 0.8819, which is the same service to within half a percentage point. Thus, the \$961.77 between them is not buying anything, and the comparison is a like-for-like price rather than a trade.

Read the two stock rows next, because they move in opposite directions. The safety stock rises from 0.50 to 2.10 units, because the exposure lengthened from L to τ and the standard deviation of demand over it rose from 2.74 to 3.59 units, a factor of $\sqrt{\tau/L} = 1.31$. Recall from Equation 8.6 that the standard deviation scales with the square root of the interval, so a protection interval half again as long costs only 31% more spread, not 50%. Meanwhile the cycle stock **falls**, from 4.00 units to 2.70, because the periodic system reviews every 6.2 weeks and therefore orders 8.3 times a year where the continuous system ordered 6.4 times.

Thus, the periodic system pays for a longer exposure by ordering more often. That is why the premium is spread across all three cost terms, roughly \$411 in ordering, \$312 in holding and \$239 in backorders, rather than landing on one. An account of this comparison that mentions only the extra safety stock has described the largest physical change and missed most of the money.

The number to carry away is 13.3%. On this item, continuous review is worth about \$960 a year. If a perpetual inventory record, a scanner at the counter and the discipline to use them cost less than that on this item alone, they pay for themselves, and the storeroom carries nine items. If the item were cheaper, or the lead time longer relative to the review interval,

the figure would be smaller. Recall from Section 1.2.4 that this choice used to be made by what the technology allowed; it is now made by what the discipline costs.

Section 8.14 now returns to continuous review and asks the question this section has held fixed throughout: what happens when the storeroom must run all nine of its items at once, under a budget.

8.14. Many Items at Once

Every section of this chapter has decided one item on its own. A storeroom holds thousands, and Chapter 4 showed what changes when items are decided together. This section is the stochastic counterpart, and it is short because the argument is one you have already met.

The items do not couple through the physics. Each has its own demand, its own lead time and its own lead time demand distribution, and nothing in Section 8.5 refers to any other item. They couple through what they **share**: one storekeeper placing the orders, one budget, and one service figure that the organization reports.

Let j index the items. Two couplings are usual, and both were the subject of Chapter 4 in the deterministic case.

A shared ordering cost. The order handling capacity is fixed, so the average order frequency across the portfolio is constrained rather than priced. Recall from Section 4.2 that a shared resource is handled with a multiplier, and that the multiplier behaves as a price on the resource.

A shared service target. The organization commits to an average fill rate across items, not to a fill rate on each.

8.14.1. Two Formulations

Hopp and Spearman (2011) develops this problem for N items, and gives the two forms of it that correspond to the two shortage costs of Section 8.6.

With a common **backorder cost** b charged per unit per unit time, each item's reorder point satisfies the critical ratio of Equation 8.15 with its own holding cost:

$$G_j(r_j^*) = \frac{b}{b + h_j} \tag{8.76}$$

With a common **stockout cost** k charged per unit short, the order quantity enters and the condition becomes

$$G_j(r_j^*) = \frac{k\lambda_j}{k\lambda_j + h_j Q_j} \tag{8.77}$$

Notice that Equation 8.77 depends on Q_j and Equation 8.76 does not. Thus, the two reorder points cannot be set independently of the order quantities in the stockout formulation, which is what makes the search below run in the order it does.

In either case the order quantities are the multi-item economic order quantities of Chapter 4, $Q_j = \sqrt{2A\lambda_j/h_j}$, with A the shared ordering cost.

8.14.2. Two Constraints, Two Loops

When the ordering cost and the shortage cost cannot be defended, both are replaced by constraints, and the problem becomes

$$\text{min inventory investment} \quad \text{subject to} \quad \overline{OF} \le F, \quad \overline{FR} \ge S \tag{8.78}$$

which Hopp and Spearman (2011) solves by searching on the two multipliers in turn. Adjust A until the order frequency constraint binds. Then, holding A fixed, adjust the shortage cost until the service constraint binds.

The order matters. A sets the order quantities, the order quantities appear in Equation 8.77, and so the reorder points depend on A. The reverse is not true. Thus, settling the order frequency first and the service level second converges; the other order chases itself.

Notice also that the constraints are checked with the **exact** measures of Section 8.5 even though the parameters are computed from the approximation. That is the practice Section 8.6 recommended, arrived at here for the same reason.

8.14.3. Equal Service Is Not Efficient

The result to carry away from this section contradicts what most organizations do. An average service target does not imply an equal service target, and meeting it equally is the expensive way. In the worked example of Hopp and Spearman (2011) the search gives a fill rate of 99.5% to a cheap, fast-moving part and 74.9% to an expensive, slow-moving one, on the way to a 95% average. The algorithm buys service where it is cheap and where it moves the average, which means where the unit cost is low and the demand is high.

The storeroom shows why with no computation at all.

Example 8.8 costed a fuse cutout backorder at $b = \$287.50$ per unit per year, ten times that item's holding cost. Suppose the utility adopts that single figure across the storeroom, which is what Equation 8.76 assumes. Table 8.30 holds b at \$287.50, charges each of the nine items of Example 4.5

its own holding cost $h_j = 0.25c_j$, and reads the implied ready rate off Equation 8.76.

Table 8.30.: What one shared backorder cost of $287.50 per unit per year implies about service, item by item. Notice the cutout's own row returns 0.9091, since $287.50 is its own figure.

Item	c_j	$h_j = 0.25c_j$	$b/(b + h_j)$
Pad-mount transformer	$3,800.00	$950.00	0.2323
Fuse cutout	$115.00	$28.75	0.9091
Smart meter	$95.00	$23.75	0.9237
Riser conduit	$62.00	$15.50	0.9488
Meter socket	$48.00	$12.00	0.9599
Splice kit	$28.00	$7.00	0.9762
Ground rod	$14.00	$3.50	0.9880
Compression connector	$3.40	$0.85	0.9971
Warning tape	$2.10	$0.53	0.9982

The implied service runs from 23.2% to 99.8% across one storeroom, and nothing but the unit cost produced the spread. A common shortage cost is a strong assumption when unit costs span three orders of magnitude.

The chapter can show that the assumption fails, rather than assert it. Example 8.3 established the transformer's own backorder cost as $8,550 per unit per year, which is **thirty times** the cutout's $287.50. Reading Equation 8.76 at that figure gives $8550/9500 = 0.90$, against the 0.2323 the shared figure produced. Thus, the shared-cost model would run the transformer at a 23% ready rate where its own cost calls for 90%, and it would do so because the cutout's penalty was applied to an item worth thirty-three times as much.

That is the practical warning. A portfolio-wide shortage cost is defensible when the items are alike, and the storeroom's items are not. Where they are not, the constrained form of Equation 8.78 is the right one, because a

constraint on the **average** does not claim that every item has the same penalty. Section 8.7.6 is how to interrogate whichever figure ends up being used.

8.14.4. Where This Connects

Section 4.4 drew an exchange curve between aggregate investment and aggregate order frequency, and read a firm's implied cost ratio off its own practice. The same curve exists here, between aggregate investment and aggregate fill rate, with one curve per order frequency, and Hopp and Spearman (2011) draws it. It is read the same way and it carries the same warning: a point above the curve is a portfolio whose parameters are not the ones it thinks it is using.

This book computes single-item policies and leaves the portfolio search to the sources cited. What it does insist on is the finding of Section 8.14.3, because it is cheap to act on and almost never acted on: **before raising service on everything, find out which items are carrying the average.**

8.15. Updating r and Q in a Running System

Every parameter in this chapter has arrived as a given. The demand rate was λ, the lead time was L, and the costs were k, h and b. In a system that is actually running, none of those is given and none of them stands still. For example, the demand rate is an output of a forecasting system that revises it every period. The lead time belongs to the supplier and moves when the supplier does, and the holding cost moves whenever the item is repriced.

This section says where those numbers come from and how often the policy should be recomputed on them. Throughout it we assume **stationary demand**, by which we mean a level that drifts slowly rather than one

carrying a trend or a season. Silver et al. (2016) gives the smoothing recursions and their trend and seasonal extensions, and the trend form replaces $\hat{D}_t$ below without changing anything else here.

8.15.1. What the Forecast Supplies

Forecasts are produced period by period as new data is gathered. Let t_F represent the **basic forecast period**, the interval the forecasting system works in, and let $\hat{D}_t$ represent its forecast of the demand in one such period, revised at the end of period t. Under exponential smoothing $\hat{D}_t$ is a weighted average of the demand history with geometrically declining weights. Thus, the demand rate is

$$\lambda = \frac{\hat{D}_t}{t_F} \tag{8.79}$$

and this is the same λ that every formula in this chapter has used. Notice that it carries a period subscript in everything but name. It is the rate as of period t, and next period it is a different number.

Dispersion comes from the same system. Let MAD_t represent the **smoothed mean absolute deviation** of the one period ahead forecast error. Forecasting systems track the mean absolute deviation rather than the variance because updating it costs an absolute value instead of a square. Converting one to the other takes an assumption: for a normally distributed error, $E[|X - E[X]|] = \sigma\sqrt{2/\pi}$, and inverting that gives

$$\sigma_D = \sqrt{\frac{\pi}{2}}\, MAD_t \approx 1.2533\, MAD_t \tag{8.80}$$

where σ_D is the standard deviation of demand in one forecast period, in the sense of Equation 8.4 with the forecast period as the period.

The normal enters at Equation 8.80 and it enters nowhere else in this chapter. What it converts is a scale estimate of the *forecast error*. It is not a statement about the shape of $D(L)$, which is still chosen at step 2 of Section 8.3, and Section 8.6.2 says why the normal is not among the families offered there. Confusing the two is how the normal gets into a lead time demand model without anyone deciding to put it there.

8.15.2. Carrying Them Over a Lead Time

The policy is exposed to demand over a lead time and not to demand over a forecast period, so both quantities have to be carried across. Let η represent the **scaling exponent**. Then

$$\theta = \frac{\hat{D}_t}{t_F} L, \qquad \sigma_{D(L)} = \sigma_D \left(\frac{L}{t_F} \right)^{\eta} \tag{8.81}$$

where θ and $\sigma_{D(L)}$ are the mean and standard deviation of $D(L)$ as Section 8.3 defined them, and L/t_F is the number of forecast periods in a lead time, which is dimensionless.

Setting $\eta = \frac{1}{2}$ recovers Equation 8.6 exactly, with the period variance estimated from the mean absolute deviation instead of from a fitted distribution. That is case 1 of Table 8.3, and it is correct when the one period forecast errors are independent. In practice $\frac{1}{2} \leq \eta \leq 1$, and you raise η toward 1 as the errors become positively autocorrelated, which is what happens when the smoothing is slow to follow a change in level. Thus, η is where a forecast that lags gets paid for, and it is the consequential choice in Equation 8.81.

Where the lead time is random as well, Equation 8.81 is not enough. Case 3 of Table 8.3 applies instead, taking σ_D from Equation 8.80 as the period standard deviation and $Var[L]$ from the supplier's delivery history.

A check on the transformer closes the section. Suppose the storeroom forecasts monthly, so t_F is one month, and the smoothing system reports $\hat{D}_t = 3.75$ units and $MAD_t = 1.55$ units. Then

$$\lambda = \frac{3.75 \text{ units}}{1 \text{ month}} \times \frac{12 \text{ months}}{1 \text{ year}} = 45 \text{ units per year}$$

and $\sigma_D = 1.2533(1.55) = 1.94$ units per month. A two month lead time puts $L/t_F = 2$, so at $\eta = \frac{1}{2}$ we get $\theta = 7.5$ units and $\sigma_{D(L)} = 1.94\sqrt{2} = 2.75$ units. Example 8.2 computed 7.5 and 2.74 for the same item from the Poisson assumption, and the agreement is not an accident. A Poisson demand averaging 3.75 a month has a mean absolute deviation of 1.5502 units, and $\sqrt{\pi/2}(1.5502) = 1.9429$ against a true standard deviation of $\sqrt{3.75} = 1.9365$, which is 0.33% high. The conversion carries that 0.33% into $\sigma_{D(L)}$ and introduces nothing else. That check is worth performing whenever a forecasting system's numbers first replace a fitted distribution's, because it prices the conversion before the policy depends on it.

Notice what the exponent does to the same item. At $\eta = 1$ the standard deviation of lead time demand is $1.94(2) = 3.89$ units, 42% higher, and the safety stock of Equation 8.34 rises with it. A reader who takes $\eta = \frac{1}{2}$ because it is the familiar square root rule should be able to say why the forecast errors on this item are independent.

8.15.3. Which Parameters Move, and How Often

The remaining parameters move on their own schedules, and the schedules differ.

The fixed order cost is normally the same for a whole group. Recall from Section 8.14 that k is a property of the ordering process and not of the item, so it changes when the process changes and not when an item does. **The holding cost follows the item's value**, since $h = ic$ by Section 1.4.4, so a repriced item has a new h whether or not anyone recomputed one. **The**

lead time varies by item, and Example 8.2 showed that its variance can contribute more to $Var[D(L)]$ than the demand does.

With λ, θ and $\sigma_{D(L)}$ in hand, the policy is recomputed the way Section 8.6 computed it the first time. Take Q from Equation 3.17 using the current λ, and fit a family to θ and $\sigma_{D(L)}$ by Section C.4.2. Then set r from the fill rate target by Equation 8.31, or from the program of Equation 8.30 where the target has to be met exactly. Nothing here is a new model; it is the classical approximation of Section 8.6 with its inputs refreshed.

The two parameters do not deserve equal attention. Recall from Equation 3.22 that the order quantity is flat near its optimum, so a Q computed from a stale λ costs very little. The reorder point has no such tolerance, because it sits in the tail of $D(L)$ where a small shift in θ or $\sigma_{D(L)}$ moves the fill rate by a visible amount. Thus, recompute r on the forecast's own cycle and recompute Q when λ has moved enough to matter.

Section 8.16 puts the evaluation and the optimization on a worksheet, where a policy can be recomputed as these inputs move.

8.16. Building These Models in a Worksheet

The workbook is `Chapter8Models.xlsx`. The worksheets of Section 5.8 and Section 6.9 each laid a procedure across rows, because the answer was a schedule. This chapter has three different things a worksheet can carry, and the sheets divide on those lines: a policy traced through a sequence of events, a set of evaluation formulas, and an algorithm.

Seven working sheets, plus a `Notes` sheet that says what the others are for.

8.16.1. The Ledger

`Ledger` is Example 8.1 with nothing collapsed. The on-hand, on-order and backordered columns are inputs and everything to the right of them is a formula, so Equation 8.1's state identity is wired in rather than typed.

The (r, Q) policy, event by event								
Section 8.1. Reorder point 4 and order quantity 5, set low so every state appears.								
Policy								
reorder poir	4 *units*							
order quant	5 *units*							
week	**event**	**I(t)**	**IO(t)**	**B(t)**	**IN(t)**	**IP(t)**	**residual**	**order placed**
0	start	6	0	0	6	6	0.0000	0
1	demand	5	0	0	5	5	0.0000	0
2	demand, order 5	4	5	0	4	9	0.0000	1
3	demand	3	5	0	3	8	0.0000	0
5	demand	2	5	0	2	7	0.0000	0
6	demand	1	5	0	1	6	0.0000	0
7	demand	0	5	0	0	5	0.0000	0
8	demand backordered, order 5	0	10	1	-1	9	0.0000	1
9	demand backordered	0	10	2	-2	8	0.0000	0
10	receipt of 5	3	5	0	3	8	0.0000	0
11	demand	2	5	0	2	7	0.0000	0
13	demand	1	5	0	1	6	0.0000	0
Checks								
largest ident	0.0000							
orders place	2							

Figure 8.1.: The `Ledger` sheet. The three blue columns are the only ones typed. The net inventory, the position and the residual are formulas, and the residual is the position less on-hand plus on-order less backorders, so it reads zero on every row. Notice week 7: the shelf is empty, the position is 5, and the order flag is 0. A rule watching the shelf would order there, and the sheet shows why it must not.

Notice how the sheet counts orders. It does not search the event text. An order is placed exactly when the units on order **rise**, so the flag is `=IF(D10>D9,1,0)` and the count is a sum down that column. Searching

the description for the word "order" returns three orders on this ledger rather than two, because "backordered" contains it.

8.16.2. Evaluating a Policy

`Evaluate` is Section 8.5. The upper block holds the item and the policy, and the lower block holds the lead time demand distribution and its loss functions.

Evaluating a policy, exactly							
Section 8.5. None of assumptions (h) to (l) of section 8.6 is used here.							
Item							
demand rate lambda	45	*units per year*					
order cost k	$220.00	*dollars per order*					
holding cost h	$950.00	*dollars per unit per year*					
backorder cost b	$8,550.00	*dollars per unit per year*					
lead time L	0.1667	*years*					
theta = lambda L	7.5000						
Policy						**Cost**	
reorder point r	8	*units*				order frequency	6.4286
order quantity Q	7	*units*				ordering	$1,414.29
Bbar	0.1617	*units owed on average*				holding	$4,428.59
fill rate	0.8781	*of demand; the ready rate too, under PASTA*				backorder	$1,382.28
Ibar	**4.6617**	*units on hand on average*	**G0(s)**	**G1(s)**	**G2(s)**	**total C(r,Q)**	$7,225.15
0	0.000553	0.000553	0.999447	7.5000	28.1250		
1	0.004148	0.004701	0.995299	6.5006	21.6244		
2	0.015555	0.020257	0.979743	5.5053	16.1192		

Figure 8.2.: The `Evaluate` sheet, rows 1 to 20. The reorder point and the order quantity are the only policy inputs. Everything in the right-hand cost block, and the three measures beneath the policy, moves when either is changed. Notice that the total reads $7,225.15, which is the optimum Example 8.7 finds, because the sheet ships holding that policy.

The loss block is the part to read, because it needs no function Excel does not already have. G comes from `POISSON.DIST` with the cumulative flag, and the two loss functions come from the recursions of Equation C.13, one short formula filled down:

$$G^1(s) = G^1(s-1) - G^0(s-1), \qquad G^2(s) = G^2(s-1) - G^1(s)$$

started from $G^1(0) = \theta$ and $G^2(0) = \theta^2/2$.

Ibar	4.6617	*units on hand on average*	G0(s)	G1(s)	G2(s)
0	0.000553	0.000553	0.999447	7.5000	28.1250
1	0.004148	0.004701	0.995299	6.5006	21.6244
2	0.015555	0.020257	0.979743	5.5053	16.1192
3	0.038889	0.059145	0.940855	4.5255	11.5937
4	0.072916	0.132062	0.867938	3.5847	8.0090
5	0.109375	0.241436	0.758564	2.7167	5.2923
6	0.136718	0.378155	0.621845	1.9582	3.3342
7	0.146484	0.524639	0.475361	1.3363	1.9978
8	0.137329	0.661967	0.338033	0.8609	1.1369
9	0.114440	0.776408	0.223592	0.5229	0.6140
10	0.085830	0.862238	0.137762	0.2993	0.3147
11	0.058521	0.920759	0.079241	0.1616	0.1531
12	0.036575	0.957334	0.042666	0.0823	0.0708

Figure 8.3.: The loss block on the `Evaluate` sheet. One row per stock level, and each row reads the row above it. Notice the second order column. It steps down by $G^1(s)$ and not by $G^1(s-1)$, because G^2 is the tail sum of G^1 from $s+1$ upward.

Equation 8.18 and Equation 8.19 then read out of that block by position. The expected backorder level is

```
=(INDEX($F$18:$F$58,$B$13+1)-INDEX($F$18:$F$58,$B$13+$B$14+1))/B14
```

which is $\frac{1}{Q}[G^2(r) - G^2(r+Q)]$ with the offsets that turn a stock level into a row. Change r or Q and every figure moves; the block below never does. That is the separation of the evaluator from the optimizer that Table 3.4 settled in Section 3.10, and here it is what makes a sensitivity study a matter of typing a number.

8.16.3. The Algorithm

`Optimize` is Algorithm Optimize_rq of Algorithm 8.1. A worksheet cannot loop, and it does not need to.

The input is one column. Equation 8.35 evaluated at every base-stock level, which is Table 8.18 as a range rather than a table.

s	G(s)	G1(s)	C(s) = h*Ibar(s) +
0	0.000553	7.5000	$64,125.00
1	0.004701	6.5006	$55,580.25
2	0.020257	5.5053	$47,074.92
3	0.059145	4.5255	$38,717.35
4	0.132062	3.5847	$30,729.24
5	0.241436	2.7167	$23,433.82
6	0.378155	1.9582	$17,177.47
7	0.524639	1.3363	$12,219.94
8	0.661967	0.8609	$8,654.01
9	0.776408	0.5229	$6,392.69
10	0.862238	0.2993	$5,218.57
11	0.920759	0.1616	$4,859.83
12	0.957334	0.0823	$5,057.03
13	0.978435	0.0397	$5,601.71
14	0.989740	0.0181	$6,346.84

Figure 8.4.: The base-stock cost column on the `Optimize` sheet. This is Table 8.18 with more rows. It falls, turns once, and rises, which is the convexity of Equation 8.37 made visible, and it is the only property the algorithm needs. S^* above it is a `MATCH` of the cost ratio against the distribution function.

The algorithm below it runs at most Q^* times, so the sheet lays out one row per step and each row computes the next state from the row above. Three formulas carry it. The **latch**,

```
=OR(G61,MIN(E61,F61)>=D61)
```

is true once the stopping test has fired and stays true, and the other two defer to it:

```
Q       =IF(G62,B61,B61+1)
r       =IF(G62,C61,IF(E61<=F61,C61-1,C61))
C*      =IF(G62,D61,D61-(D61-MIN(E61,F61))/B62)
```

The third is Equation 8.45. Because the latch freezes everything, every row after termination repeats the answer and **the last row is always the**

result, whatever the data. No macros, no iterative calculation setting, no circular reference.

Algorithm Optimize_rq						
step	Q	r	C*	C(r)	C(r+Q+1)	stopped
0	1	10	$14,759.83	$5,218.57	$5,057.03	FALSE
1	2	10	$9,908.43	$5,218.57	$5,601.71	FALSE
2	3	9	$8,345.14	$6,392.69	$5,601.71	FALSE
3	4	9	$7,659.28	$6,392.69	$6,346.84	FALSE
4	5	9	$7,396.80	$6,392.69	$7,199.37	FALSE
5	6	8	$7,229.45	$8,654.01	$7,199.37	FALSE
6	7	8	$7,225.15	$8,654.01	$8,105.59	FALSE
7	7	8	$7,225.15	$8,654.01	$8,105.59	TRUE
8	7	8	$7,225.15	$8,654.01	$8,105.59	TRUE
9	7	8	$7,225.15	$8,654.01	$8,105.59	TRUE

Figure 8.5.: The algorithm on the `Optimize` sheet. This is Table 8.19 as a range a reader can edit. Read the reorder point column downward: 10, 10, 9, 9, 9, 8, 8. It falls as the batch grows and never by more than one per step, which is the property that makes the loop finite. Step 7 is where the stopped column turns TRUE, and every row below it repeats step 6's answer rather than drifting.

The sheet also runs the two results of Section 8.9, and it runs them as **tests** rather than as displays.

8.16.4. The Approximation

`Approximate` is Section 8.6. The gamma of Equation 8.22 is built from the two moments at the top, and the iteration of Equation 8.24 is unrolled one pass per row in the same way the algorithm is.

The gamma's first order loss function is Table C.5, $G^1(x) = \beta[(\alpha - x/\beta)G^0(x) + xg(x)]$, which on the sheet is

```
=$B$16*(($B$15-D21/$B$16)*(1-GAMMA.DIST(D21,$B$15,$B$16,TRUE))
        +D21*GAMMA.DIST(D21,$B$15,$B$16,FALSE))
```

28	7	8
29	7	8
30	7	8
Answer, read from the last row		
r*	8	
Q*	7	
C*	$7,225.15	
Checks from section 8.9		
r* < S* <= r* + Q*	holds	
Qk, the deterministic batch	4.8123	
lower bound Ck	$4,114.49	

Figure 8.6.: The checks on the `Optimize` sheet. Equation 8.42 and Equation 8.43 are evaluated against the answer the rows above produced, and each prints "holds" or "FAILS". Both are recomputed when a cost parameter changes.

The iteration of equation 8.29					
pass	**Q**	**G0(r) = Qh/(pi*lambda)**	**r**	**G1(r)**	**next Q**
0	4.5653	*start at the economic order quantity*			
1	4.5653	0.016909	14.4156	0.0271	5.9567
2	5.9567	0.022062	13.9611	0.0359	6.3434
3	6.3434	0.023494	13.8522	0.0384	6.4480
4	6.4480	0.023882	13.8238	0.0391	6.4761
5	6.4761	0.023986	13.8163	0.0393	6.4837
6	6.4837	0.024014	13.8143	0.0393	6.4857
7	6.4857	0.024021	13.8137	0.0393	6.4862
8	6.4862	0.024023	13.8136	0.0393	6.4864
9	6.4864	0.024024	13.8135	0.0393	6.4864
10	6.4864	0.024024	13.8135	0.0393	6.4864

Figure 8.7.: The iteration on the `Approximate` sheet. This is Table 8.8 on the transformer's numbers. Each row carries the previous row's order quantity into the stockout probability of Equation 8.24, inverts the gamma for a reorder point, and returns a new order quantity. Notice that the last decimal takes twenty-one passes to settle here against three on the connector.

`GAMMA.DIST` supplies g with its cumulative flag set to `FALSE` and G with it set to `TRUE`, and the rest is arithmetic.

What it recommends		
r, rounded	14	
Q, rounded	6	
G1 at the rounded r	0.0351	
Recomputed at the rounded reorder point, since rounding moves it.		
ordering	$1,650.00	
holding	$9,025.00	
shortage	$1,499.25	
total it reports	$12,174.25	
Diagnostics, section 8.6		
one unit as a fraction of sigma	36.51%	
1/Q	16.67%	
theta/Q	1.2500	
Small, small and below one is the case the approximation was built for.		

Figure 8.8.: The answer block on the `Approximate` sheet. Notice that the loss function is recomputed at the rounded reorder point rather than carried down from the iteration, because rounding r moves it. The three diagnostics of Table 8.12 sit below the cost.

8.16.5. Service Levels

`Service` is Section 8.7. Both targets sit above the loss block and read out of it, the Type 1 target with `MATCH` against the distribution function and the Type 2 target with a `COUNTIF` of the loss function above its threshold.

The constrained program of Equation 8.30 is laid out rather than solved, one row per order quantity, and the reason is Table 8.14.

8.16.6. Reviewing on a Calendar

`Periodic` is Section 8.13. The item block is the transformer's, and the review block beneath it holds the interval R and the order-up-to level

Q	(1-beta)Q	r	G1(r)	ordering	holding	total
1	0.0500	13	0.0397	$9,900.00	$5,700.00	$15,600.00
2	0.1000	12	0.0823	$4,950.00	$5,225.00	$10,175.00
3	0.1500	12	0.0823	$3,300.00	$5,700.00	$9,000.00
4	0.2000	11	0.1616	$2,475.00	$5,225.00	$7,700.00
5	0.2500	11	0.1616	$1,980.00	$5,700.00	$7,680.00
6	0.3000	10	0.2993	$1,650.00	$5,225.00	$6,875.00
7	0.3500	10	0.2993	$1,414.29	$5,700.00	$7,114.29
8	0.4000	10	0.2993	$1,237.50	$6,175.00	$7,412.50
9	0.4500	10	0.2993	$1,100.00	$6,650.00	$7,750.00
10	0.5000	10	0.2993	$990.00	$7,125.00	$8,115.00
11	0.5500	9	0.5229	$900.00	$6,650.00	$7,550.00
12	0.6000	9	0.5229	$825.00	$7,125.00	$7,950.00
13	0.6500	9	0.5229	$761.54	$7,600.00	$8,361.54
14	0.7000	9	0.5229	$707.14	$8,075.00	$8,782.14
15	0.7500	9	0.5229	$660.00	$8,550.00	$9,210.00
best total	$6,875.00					

Figure 8.9.: The constrained program on the `Service` sheet. This is Table 8.14 with more rows. Read the total column downward: it falls to a minimum, climbs for four rows, and then **falls again**. Notice the reorder point column beside it, which drops a whole unit where the total turns, because a larger order quantity slackens the constraint of Equation 8.30. A search that stopped at the first local minimum would stop in the wrong place.

S as the two inputs, with the protection interval, the mean exposure $\lambda(L + R/2)$, the safety stock and the cycle stock as formulas below them. Notice the label beside the mean exposure. It says the cell is not $\lambda\tau$, which is the exposure at the worst instant of the cycle, and that distinction is Equation 8.66 against Equation 8.61.

The loss block needs three Poisson means where the other sheets need one, because Equation 8.70 averages the base-stock measures over the review interval from three points: the start of the interval, its middle and its end. Nine columns carry the distribution function and the two loss functions at each of the three means, and two more combine them with Simpson's weights of one, four and one into $\bar{G}$ and $\overline{G^1}$. The on-hand and cost columns read the combined functions, so every row of the cost column is Equation 8.71 at one level, and the four measures at the typed level are read out of the block by `INDEX`.

The level and the interval are chosen at the bottom. `MATCH` reads S^* of Equation 8.72 off $\bar{G}$ against the critical ratio, and the caution beside it, that the level must be read off the averaged function and not off the column at τ, is the error Section 8.13.3 works through. The economic review interval of Equation 8.74 sits below it. Set R small and the level returns to the base-stock answer of Section 8.4, which is the check to run before trusting the sheet with an interval of your own.

8.16.7. Demand in Lots

`Lumpy` is Section 8.11. Lead time demand is compound Poisson, which is a sum of a random number of lot sizes.

It needs one column. **Panjer's recursion** builds the mass function upward, each value from the values below it:

$$g(0) = e^{-\nu}, \qquad g(x) = \frac{\nu}{x} \sum_{y \le x} y\, P\{Y = y\}\, g(x - y)$$

Lot size			
y	P{Y = y}	y P{Y = y}	y^2 P{Y = y}
1	0.4000	0.4000	0.4000
2	0.4000	0.8000	1.6000
4	0.2000	0.8000	3.2000
E[Y]	2.0000		
E[Y^2]	5.2000		
demand rate lambda	800		
epochs in a lead time	15.3846		
theta	30.7692		
variance of D(L)	80.0000		
variance to mean ratio	2.6000		
The ratio is E[Y^2]/E[Y] and exceeds one whenever a lot is ever bigger than one.			

Figure 8.10.: The lot size block on the `Lumpy` sheet. The lot sizes and their probabilities are the only inputs, and the first two moments follow. The variance to mean ratio is the diagnostic of Equation 8.49, and it is 2.6 here, which is what sends this item to the negative binomial rather than the Poisson.

where $\nu = \lambda_e L$ is the expected number of demand epochs in a lead time. The sum has one term per distinct lot size, three on this item, so each cell reaches three cells up its own column and nothing else.

8.16.8. Using the Sheets on Your Own Item

Every sheet carries its own copy of the item, in cells B5 through B9, and the copies are not linked. Pointing the workbook at a new item means typing it into each sheet you intend to use, and a policy that disagrees between two sheets is almost always one item entered twice and changed once.

Example 8.12 (Reading the Evaluate and Optimize sheets)**.** The workbook ships holding the transformer of Example 8.1, so the first use of it is to confirm that it says what the chapter says.

The input block. On `Evaluate`, cells B5 to B9 hold $\lambda = 45$ a year, $k = \$220$, $h = \$950$ per unit per year, $b = \$8{,}550$ per unit per year and $L = 0.1667$

Panjer's recursion for the compound Poisson mass function					
x	g(x)	G(x)	G0(x)	G1(x)	G2(x)
0	0.00000021	0.00000021	0.99999979	30.7692	497.9882
1	0.00000128	0.00000149	0.99999851	29.7692	468.2189
2	0.00000522	0.00000671	0.99999329	28.7692	439.4497
3	0.00001597	0.00002269	0.99997731	27.7692	411.6805
4	0.00004129	0.00006398	0.99993602	26.7693	384.9112
5	0.00009329	0.00015727	0.99984273	25.7693	359.1419
6	0.00019110	0.00034837	0.99965163	24.7695	334.3724
7	0.00036011	0.00070848	0.99929152	23.7698	310.6026
8	0.00063453	0.00134302	0.99865698	22.7705	287.8320
9	0.00105391	0.00239693	0.99760307	21.7719	266.0601
10	0.00166472	0.00406165	0.99593835	20.7743	245.2859
11	0.00251344	0.00657510	0.99342490	19.7783	225.5075
12	0.00364716	0.01022225	0.98977775	18.7849	206.7226
13	0.00510384	0.01532610	0.98467390	17.7951	188.9275
14	0.00691323	0.02223933	0.97776067	16.8105	172.1170
15	0.00908628	0.03132561	0.96867439	15.8327	156.2843
16	0.01161810	0.04294371	0.95705629	14.8640	141.4203
17	0.01447904	0.05742275	0.94257725	13.9070	127.5133
18	0.01762110	0.07504384	0.92495616	12.9644	114.5489

Figure 8.11.: The recursion on the `Lumpy` sheet. One row per unit of demand, and the mass column is the only thing computed; the four columns to its right are the same accumulations the `Evaluate` sheet uses. The two checks below are the reason to trust it. Neither is imposed: the mass sums to one and the mean it implies returns θ, and both come out of the recursion rather than being forced on it.

years. B10 is the only derived input, $\theta = \lambda L = 7.5$ units, and everything below it reads B10 and not B5 or B9.

The policy block. B13 and B14 hold $r = 8$ and $Q = 7$. These two cells are the only ones a reader types.

The three measures. B15, B16 and B17 return

$$\bar{B} = 0.1617 \text{ units}, \qquad \overline{FR} = 0.8781, \qquad \bar{I} = 4.6617 \text{ units}$$

The check, by hand. Read $G^2(8) = 1.1369$ and $G^2(15) = 0.0052$ from the $\lambda = 7.5$ column of Table D.5. Then Equation 8.18 gives

$$\bar{B} = \frac{1.1369 - 0.0052}{7} = \frac{1.1317}{7} = 0.1617 \text{ units}$$

which is B15. That check is worth performing once on any sheet you did not build, because it is the whole of Equation 8.18 in two table lookups.

The cost block. H13 to H17 return an order frequency of 6.4286 a year and

$$\underbrace{\$1{,}414.29}_{\text{ordering}} + \underbrace{\$4{,}428.59}_{\text{holding}} + \underbrace{\$1{,}382.28}_{\text{backorder}} = \$7{,}225.15 \text{ a year}$$

Typing a different policy. Enter $r = 9$ and $Q = 5$ into B13 and B14. The sheet returns $\bar{B} = 0.1202$, $\overline{FR} = 0.8990$, $\bar{I} = 4.6202$ and a total of \$7,396.80, which is the chosen row of Table 8.5. Restore $(8, 7)$ before going on.

Now a real question. Purchasing believes it can bring the transformer's lead time down from two months to one by ordering against a framework agreement, and wants to know what that is worth. **One cell answers it.** Set B9 to $1/12 = 0.0833$ years on the `Optimize` sheet and read the answer block:

$$r^* = 4, \qquad Q^* = 6, \qquad C^* = \$6{,}104.16 \text{ a year}$$

and both checks in B99 and B103 still print "holds". Set B9 the same way on `Evaluate`, enter $(4, 6)$, and the cost block confirms the figure from the other direction: \$1,650.00 ordering, \$3,651.67 holding and \$802.49 backorder.

What it is worth. The saving is $7{,}225.15 - 6{,}104.16 = \$1{,}120.99$ a year, which is 15.5% of what the item costs to carry now. Notice where it comes from. The holding cost falls by \$777 because θ halves and the reorder point follows it down, and the backorder cost falls by \$580 because the same policy now covers a shorter exposure. The ordering cost **rises** by \$236, because the optimal batch drops from seven to six. Thus, a shorter lead time is not simply less of everything, and a reader who priced it by scaling the current cost would have missed a term that runs the other way.

8.16.9. What the Worksheet Cannot Do

Two sections of this chapter have no sheet, and both for the same kind of reason.

The (s, S) policy of Section 8.12. Its inventory position is not uniform, and its distribution is the stationary vector of a Markov chain. Finding one means iterating until nothing moves, and a worksheet cannot iterate without either a macro or the circular-reference setting, which is a macro by another name. The figures in Table 8.23 were computed by solving that chain directly. Notice that the **approximation** in that section is perfectly sheetable: it is the `Evaluate` sheet with $r = s$ and $Q = S - s$, and Equation 8.59 replacing the order frequency.

The multi-item search of Section 8.14. Two nested searches over a portfolio of thousands of items is a program, not a sheet, and Section 8.17 is where it belongs.

Two conventions hold across every sheet.

There are no macros. Every number is produced by a formula in the cell that holds it.

Lead time demand is never modeled with a normal distribution. See Section 8.6.2.

Section 8.17 turns the same models into a program, which is what an item whose optimal batch runs to thousands of units requires.

8.17. Designing the Software

Section 5.9 had six recipes to unify and Section 7.12 had a formula to organize. This chapter has an algorithm, so the shape of the design is closer to the first. What makes it worth the walk is that the two hardest decisions here concern where a convention lives and how a fast search is kept exact, not which classes exist.

8.17.1. What the Software Must Do

Reading back over the chapter, we find that the software has to

1. hold a lead time demand distribution and evaluate its loss functions to the third order, including at arguments below zero,
2. evaluate any (r, Q) policy, reporting the measures of Section 8.2 separately, and the spread of the backorder level alongside its mean,
3. bound the answer before any search runs, as Section 8.9 does,
4. return the optimal integer (r, Q),
5. do that in reasonable time on an item whose optimal batch is in the thousands, and
6. accept a service target in place of a shortage cost.

Notice that requirement 1 reaches one order higher than the measures themselves do. Equation 8.18 needs G^2 and Equation 8.21 needs G^3, and the KSL supplies loss functions to the second order only, so the third is the one piece of distribution machinery this chapter has to build rather than borrow. Section C.3.6 gives the forms.

Requirement 5 is the one the worksheet of Section 8.16 cannot meet. That sheet unrolls one row per step of Algorithm 8.1, so it answers any item whose optimal batch fits in the rows laid down. An item whose batch is 2,842 does not fit, and that is the reason for writing a program at all.

8.17.2. Finding the Nouns

Underline the nouns in the requirements and four survive.

A **lead time demand** is a distribution with a quantile and two loss functions. Notice that this is not a new class but `LossFunctionDistributionIfc` of Section C.3.9, which the KSL already provides for every family Section 8.3 selects among, and the design's first decision is not to reimplement it. What the library does not provide is the one thing requirement 1 names: G^1 and G^2 below zero. A wrapper adds those and nothing else.

An **item** is λ, k, h, b and a lead time demand. It owns every formula in Section 8.2 and Section 8.5, which means it owns the cost of any policy, optimal or not. An item whose demand arrives in lots also carries its lot size distribution, which only the fill rate of Equation 8.53 reads.

A **policy** is a reorder point and an order quantity, and it is a pair of integers with a name.

An **optimizer** reads an item and returns a policy. It is the only thing in the chapter that searches.

Two candidate nouns fail, and each fails instructively.

Base-stock model looks like a class. Section 8.4 is a section of its own, it has its own cost function and its own optimality condition, and a reader who met it first would expect an object. It fails because Equation 8.39 makes it literal that the base-stock cost is the (r, Q) cost at $Q = 1$. In the code `baseStockCost(s)` is one line, and the line is the general window evaluated at a width of one unit. A second class would be a second implementation of an identity the chapter proves.

Service level also looks like a class, and it is the thing practitioners actually talk about. It fails for the reason Section 8.7.6 gives: a target is not an alternative to a shortage cost, it is a shortage cost stated without saying so. Thus, it belongs in the construction of an item, where Equation 8.33 converts it to a b that everything downstream can see and argue with. Making it a class would preserve the evasion the section exists to expose.

Figure 8.12 illustrates the four and how they stand to one another.

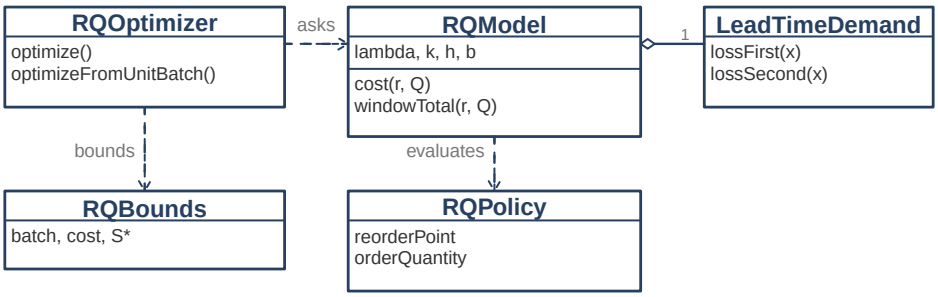


Figure 8.12.: The concepts of this chapter before any thought about implementation. An item holds a lead time demand distribution and evaluates any policy against it. An optimizer reads the item, asks it for the bounds of Section 8.9, and returns a policy. Notice that the optimizer holds no state of the item's: everything it needs it asks for, which is what allows two different searches to be run against one item and compared.

8.17.3. The Window Is the Unit of Work

The design's first real content is a decision about what a single step costs.

Equation 8.39 says a policy's cost is $k\lambda/Q$ plus the average of Q consecutive base-stock costs. Written that way it invites a loop, and a loop makes every policy cost Q evaluations of the lead time demand distribution. The search then costs Q times whatever it costs to walk, and the items that need a program are the items where Q is large.

Both sums have closed forms, and the chapter has already given them. Summing Equation 8.20 over the window is arithmetic, and summing Equation 8.18 telescopes by Equation C.10. Thus, one method replaces the loop:

```
fun windowTotal(r: Int, q: Int): Double {
    requirePolicy(r, q)
    val width = q.toDouble()
    val shortage = leadTimeDemand.lossSecond(r.toDouble()) -
        leadTimeDemand.lossSecond((r + q).toDouble())
    return fixedCostRate +
        holdingCost * width * (expectedPosition(r, q) -
 ↪  theta) +
        (holdingCost + backorderCost) * shortage
}
```

Notice that `q` appears in the arithmetic and not in a loop. A step of one thousand costs what a step of one costs, and that single fact is what makes the search of Section 8.10.3 worth writing. A step size that is free can double.

8.17.4. Two Searches, One Answer

The design's second real content is that the optimizer exposes two methods and not one.

`optimizeFromUnitBatch` is Algorithm 8.1 as printed: start at $Q = 1$, add the cheaper neighbor, stop when it is no longer below the average. It is exact by the argument of Section 8.10.1 and its loop runs Q^* times. `optimize` is the same algorithm warm started, searched in both directions, and stepped by doubling.

The doubling is fast and, on its own rather than exact. A step that commits on one edge of the window can build a window skewed to one side. It then carries Q past the optimum to a value that is still cheaper than where it started, and a search that only grows Q cannot walk back. `optimize` therefore finishes every doubling search with an exact per-unit search in both directions from wherever the doubling left it.

That is a claim, and a claim of that kind belongs in a test rather than a comment:

```
for ((name, model) in items()) {
    val exact = RQOptimizer(model).optimizeFromUnitBatch()
    val fast = RQOptimizer(model).optimize()
    assertEquals(exact.orderQuantity, fast.orderQuantity,
 ↪  "$name: Q")
    assertEquals(exact.reorderPoint, fast.reorderPoint,
 ↪  "$name: r")
    assertEquals(exact.cost, fast.cost, 1e-6, "$name: cost")
}
```

Keeping the slow method is the point. It is the thing the fast one is checked against, and a reader who distrusts the doubling can run it. On the items of Table 8.31 the two agree everywhere, and the cost of agreeing is visible: 74 evaluations against 5,684 on an item whose optimal batch is 2,842.

Table 8.31.: Window evaluations taken by the two searches. The transformer is the item of Example 8.1. The other two rows are test items chosen for their batch sizes: mid volume has $\lambda = 3{,}000$ a year, $k = \$150$, $h = \$12$, $b = \$90$ and $\theta = 230.77$; high volume has $\lambda = 12{,}000$, $k = \$900$, $h = \$3$, $b = \$25$ and $\theta = 1{,}500$, both Poisson. Notice the first row. On an item whose optimal batch is seven units the warm start costs more than the algorithm it improves, because there is nothing to warm start from.

Item	r^*	Q^*	Algorithm 8.1	Warm started
Transformer	8	7	14	19
Mid volume	196	295	590	75
High volume	1,195	2,842	5,684	74

8.17.5. Where the Conventions Live

Two places in this chapter have a discrete form and a continuous one, and both are easy to write wrongly.

Equation 8.16 puts the inventory position on the **integers** $r + 1$ through $r + Q$, whose mean is $\frac{1}{2}(Q + 1) + r$. A continuous family puts it on the **interval**, whose mean is $Q/2 + r$. Likewise Equation C.10 defines G^2 with $(X - b - 1)$ when X is discrete and with a square when it is continuous, so the two extensions below zero differ.

Each belongs in exactly one method. The position is one method that asks the distribution which kind it is; the loss function extension is one method that does the same. Everything else in the code, including the cost, the measures and both searches, calls those and never restates the rule. **Write a convention out at each place it is used and one of those places will eventually disagree with the others.** Half a unit of stock is $h/2$

per unit time on every policy the software will ever report, and nothing about the answer looks wrong when it is missing.

8.17.6. Running the Chapter's Examples

The code is in `code/`, package `inventory.continuousreview`, and four lines solve an item.

Example 8.13 (The transformer in code, and an item the worksheet cannot reach)**.** *Building the item.* The constructor takes the four cost and demand parameters and a lead time demand distribution. `LeadTimeDemand.matched` carries out step 2 of Section 8.3 rather than leaving the reader to choose a family by hand, and `holdingCostFrom` applies the storeroom's carrying charge to a unit cost the way Section 1.4.4 does.

```
val transformer = RQModel(
    demandRate = 45.0,
    orderCost = 220.0,
    holdingCost = RQModel.holdingCostFrom(carryingCharge =
 ↪  0.25, unitCost = 3800.0),
    backorderCost = 8550.0,
    leadTimeDemand = LeadTimeDemand.matched(mean = 7.5,
 ↪  variance = 7.5),
)

println(transformer.bounds())
val answer = RQOptimizer(transformer).optimize()
println(answer)
println(transformer.evaluate(answer.policy))
```

```
bounds: Q in [5, 11], S* = 11 costing 4859.83, C* in [4114.49, 8823.14]
```

```
(r = 8, Q = 7) costing 7225.15, found by warm start in 19 window evaluations
policy (r = 8, Q = 7)
  order frequency        6.4286 orders per unit time
  expected backorders    0.1617 units
  ready rate             0.8781
  fill rate              0.8781
  expected on hand       4.6617 units
  ordering             1414.29 per unit time
  holding              4428.59 per unit time
  backorder            1382.28 per unit time
  total                7225.15 per unit time
```

Every figure is the `Evaluate` sheet's, to the cent. Notice the order of the three calls. `bounds` is asked **before** the search runs and prints what the answer must satisfy, which is Section 8.9 used the way that section recommends: Q^* must fall in $[5, 11]$ and C^* between \$4,114.49 and \$8,823.14, and both hold. The solution line also reports what the search cost, which is 19 evaluations of Equation 8.39.

The lead time question of Example 8.12. Changing the mean of the lead time demand answers it, and the program agrees with the worksheet:

```
val shorter = transformer.copy(
    leadTimeDemand = LeadTimeDemand.matched(mean = 3.75,
 ↪  variance = 3.75)
)
println(shorter.bounds())
println(RQOptimizer(shorter).optimize())
```

```
bounds: Q in [5, 9], S* = 6 costing 3529.19, C* in [4114.49, 6883.92]
(r = 4, Q = 6) costing 6104.16, found by warm start in 16 window evaluations
```

Now the item the worksheet cannot take. The storeroom's ground rod moves $\lambda = 2{,}400$ a year at $c = \$14$, so $h = \$3.50$, with $k = \$41.25$, the shared backorder cost of \$287.50 from Section 8.14, and a two week lead time.

```
val rod = RQModel(
    demandRate = 2400.0,
    orderCost = 41.25,
    holdingCost = RQModel.holdingCostFrom(carryingCharge =
 ↪  0.25, unitCost = 14.0),
    backorderCost = 287.50,
    leadTimeDemand = LeadTimeDemand.matched(mean = 2400.0 /
 ↪  26.0, variance = 2400.0 / 26.0),
)
println(rod.bounds())
println(RQOptimizer(rod).optimize())
println("Algorithm Optimize_rq from a batch of one takes %d
 ↪  steps"
    .format(RQOptimizer(rod).optimizeFromUnitBatch().
     ↪  orderQuantity - 1))
```

```
bounds: Q in [239, 265], S* = 115 costing 90.83, C* in [827.44, 881.79]
(r = 94, Q = 244) costing 859.94, found by jump search in 65 window evaluations
Algorithm Optimize_rq from a batch of one takes 243 steps
```

Read the last two lines together. The `Optimize` sheet lays down thirty steps, and this item needs 243, so the worksheet cannot reach the answer and Section 8.16.9 is where that boundary was drawn. The program reaches it in 65 window evaluations, which is fewer steps than the worksheet lays down and a quarter of what the printed algorithm would take. Notice the method the solution reports: `jump search`, because Q^* is large enough for Section 8.17.4 to dispatch to it.

The same check as before. $94 < 115 \leq 338$ satisfies Equation 8.42 and \$859.94 lies inside \$827.44 to \$881.79, so the two tests of Section 8.9 pass on an answer no worksheet produced.

8.18. Summary

The policy is exposed to exactly one random quantity. Demand that arrives while stock is on the shelf is met from the shelf and costs nothing to have been uncertain about. Only demand arriving during a replenishment lead time can find the shelf empty, so every measure and every cost in this chapter is a functional of the distribution of $D(L)$ and of nothing else. Thus, two systems with different demand patterns and different lead times perform identically under the same policy whenever their lead time demand distributions coincide. The modeling effort belongs on $D(L)$ and not on the formula built over it. Section 8.13 showed how far that carries: removing continuous review does not overturn the claim, it lengthens the interval the demand is taken over.

The inventory position is what the policy controls and the net inventory is what it costs. Equation 8.17 separates the two, and Equation 8.16 says the position is uniform over the band $r + 1$ through $r + Q$ and independent of the lead time demand. That one fact carries Section 8.5 entire: every measure of a general batch size is a base-stock measure averaged over the band, and every such average telescopes into a difference of two loss functions.

Two moments do not determine a distribution, and the gap is where the work is. Table 8.3 gives the mean and variance of lead time demand for any combination of random demand and random lead time. The three cases it prints are one formula with terms switched off. A family still has to be chosen, and the variance to mean ratio of Equation C.1 chooses it. The normal is not among the candidates. Lead time demand is a non-negative count, the approximation is worst where items are slow-moving

and expensive, and Rossetti and Ünlü (2011) measures what assuming otherwise costs.

One-for-one replenishment is the general model at $Q = 1$, and its answer is the newsvendor's. The ordering cost drops out because an order is placed at every demand whatever the level is, so k cannot affect which level is best. What remains trades one unit too many against one unit too few, and S^* is the smallest level whose distribution function reaches $b/(b + h)$. Chapter 7 solved a problem with one period and no future, and this chapter solves one with an infinite horizon and a permanent stock. Both reach the same rule, and that is not a coincidence. In a one-for-one system the same trade is made afresh at every demand.

The classical approximation is wrong in two directions at once. It drops the backorder term from the on-hand level, prices shortages per cycle, and fits a continuous family to a count. On the transformer those assumptions cost -3.98%, the continuous family costs $+6.34\%$, and what is reported is $+2.11\%$ high. A method that comes close because two errors offset has not been validated by coming close, and nothing guarantees the offset on the next item. Section 8.5 is exact and costs no more to compute.

A service target is a shortage cost stated without saying so. Setting a fill rate does not remove b from the model; it fixes b at whatever value would have produced the same policy, and Equation 8.32 recovers the number. On the transformer a 95% fill rate implies \$919 per unit short against an emergency purchase that costs the utility about \$5,700. Thus, the target is not over-cautious, which is the usual accusation. It is holding far too little, and the useful question about any service level is what shortage cost would make it optimal.

The cost is an average of a convex sequence, and that is what makes the problem easy. By Equation 8.39 an (r, Q) policy costs $k\lambda/Q$ plus the average of Q consecutive base-stock costs. The best r for a given Q therefore takes the Q smallest values of $C(s)$, and convexity puts those next to one another. Adding one more value lowers the average exactly

when the value added is below it, which is a stopping rule rather than a convergence criterion. Thus, Algorithm 8.1 returns the exact integer optimum in a finite number of steps, with no search in two dimensions, no derivative, and nothing to converge.

A great deal about the answer is known before it is computed. The optimal band straddles the optimal base-stock level: $r^* < S^* \leq r^* + Q^*$, and the optimal cost lies between the deterministic cost and the two reference costs added in quadrature. Both are cheap, and both are checks you can run on an answer produced by software you did not write. Both also say that everything the deterministic chapters established about what drives performance continues to hold once demand is random.

Lumpiness changes the distribution and not the model. Demand arriving in lots makes lead time demand compound Poisson, raises the variance to mean ratio, and separates the fill rate from the ready rate. The backorder, ready rate and on-hand formulas of Section 8.5 carry over unchanged, because they never asked how the demand arrived. The fill rate is the exception: the ready rate now counts lots that find stock, and the unit fill rate of Equation 8.53, which needs the lot size as well as $D(L)$, is lower. The (s, S) policy needs one correction and only one: the undershoot below s, without which the approximation is biased in a direction that flatters the policy.

One service target does not mean one level of care. Applying the same fill rate across a storeroom sets a different implied shortage cost on every item, because the implied cost depends on the item's own unit cost and demand rate. Before raising service on everything, find out which items are carrying the average, because that is cheap to act on and almost never acted on.

Periodic review changes the interval and nothing else. Under an (R, S) policy the stock committed at a review must last until the order after next arrives, so the exposure is the protection interval $\tau = R + L$ rather than the lead time alone. Equation 8.62 is then Equation 8.16's dual: one policy randomizes the position over Q levels, the other randomizes

the exposure over R of time, and both are the base-stock model averaged over a uniform quantity. Thus, the four measures, the critical ratio and the newsvendor argument all survive with the averaged distribution $\bar{G}$ in place of G. The one new quantity is the $\lambda R/2$ of cycle stock the calendar creates, which is the EOQ's $Q/2$ in different units, and it is what makes Equation 8.74 set the review interval to the EOQ time supply.

Not watching an item costs money, and the amount is computable. On the transformer it is $960 a year, or 13.3%, at the same fill rate. The premium is not all safety stock: a longer exposure raises the safety stock, but the shorter review interval lowers the cycle stock, and the difference lands across all three cost terms. Thus, the choice between continuous and periodic review is a question about what the record-keeping discipline costs, not a question about the item.

Table 8.32 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 8.32.: Notation introduced in this chapter.

Symbol	Meaning
r, Q	Reorder point and order quantity, the two decisions
$D(L)$	Demand over a replenishment lead time, the chapter's one random quantity
θ	$E[D(L)]$, the mean lead time demand
w	$b/(b+h)$, the critical ratio the base-stock level is read against
S, S^*	Base-stock level, and the optimal one
$C(s)$	Cost rate of a base-stock policy at level s, holding plus backorder

Symbol	Meaning
s, S	Reorder point and order-up-to level of an (s, S) policy
U	Undershoot, the amount by which a demand carries the position below s
Q_k, C_k	Order quantity and cost of the deterministic model with backorders
C_∞	$\sqrt{bh}\,\sigma$, a reference cost built from lead time demand alone
Y, λ_e	Size of a demand lot, and the rate at which lots arrive
t_F, $\hat{D}_t$	Basic forecast period, and the forecast of demand in one
MAD_t	Smoothed mean absolute deviation of the forecast error
η	Exponent carrying a period standard deviation over a lead time
R	Review interval, the time between consecutive looks at the position
τ	$R + L$, the protection interval a periodic policy must cover
$\bar{G}$, $\overline{G^1}$	Distribution and first order loss function of $D(L + U)$, averaged over a review interval

Chapter 9 keeps the randomness and removes the single location. Stock is held at more than one place, a retailer is supplied by a warehouse that is itself supplied from outside, and the lead time one location faces is no longer a property of its supplier but a consequence of how much stock that supplier chose to hold. Thus, the $D(L)$ this chapter took as given becomes something the system decides, and that is the one change from which the

rest of that chapter follows.

8.19. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.8. Report reorder points and order quantities as whole units and state the unrounded value alongside. Take the storeroom's carrying charge as 0.25 a year throughout, so $h = 0.25c$, and take item data from Table 4.5. Where an exercise names a lead time, it is the supplier's quoted lead time and is constant unless the exercise says it is not.

8.19.1. Working the Model by Hand

Exercise 8.1. The utility will not name a backorder cost for the pad-mount transformer and sets a fill rate target instead. Take $\lambda = 45$ a year, $k = \$220$, $c = \$3{,}800$ and a constant two month lead time, so $\theta = 7.5$ units and $D(L)$ is Poisson.

Work entirely from the $\lambda = 7.5$ column of Table D.3, Table D.4 and Table D.5.

a. Compute h and the economic order quantity of Equation 3.17, and round it up to the next whole unit.
b. At that Q, find the smallest r meeting a fill rate of 98% using Equation 8.19. Show the column of G^1 values you walked.
c. Report $\bar{B}$ and $\bar{I}$ at your policy, and verify $\bar{I}$ against Equation 8.20 by the arithmetic check of Example 8.4.
d. Use Equation 8.32 to recover the shortage cost your target implies. Compare it with the \$5,700 of Example 8.6 and say which way the target errs.

Exercise 8.2. Continue with the policy you found in Exercise 8.1.

a. Compute the exact expected backorder level from Equation 8.18.
b. The holding term of Equation 8.23 prices an average on-hand level of $Q/2 + r - \theta$. Compute that figure and compare it with the exact $\bar{I}$ of part (a), reporting the difference in units and as a percentage of the exact level.
c. Split that difference into its two parts, and say which assumption of Section 8.6.1 each one comes from. One part is the same size whatever the item is; the other depends on the policy. Say which is which, and which of the two would grow if the backorder cost were cut in half.

Exercise 8.3. The transformer's supplier delivers in two months on average, but the storeroom's receiving records show a standard deviation of three weeks about that mean. Demand remains Poisson at 45 a year.

a. Using case 3 of Table 8.3, compute $E[D(L)]$ and $Var[D(L)]$. Work in years and show the unit conversion as a dimensional chain.
b. Report what fraction of the variance the supplier contributes, and compare it with the 65% of Example 8.2. Explain the direction of the change.
c. Compute the variance to mean ratio and say which family Equation C.1 selects. Match its moments.
d. The purchasing department believes it can halve the standard deviation of the lead time by paying a $400 a year premium for a dedicated slot. Recompute the variance, and say what further information you would need before advising whether to pay it.

Exercise 8.4. The storeroom currently reviews the fuse cutout continuously and orders up to a level rather than ordering a fixed quantity, so its policy is (s, S) rather than (r, Q). Take the lot size distribution of Table 8.20, with $\lambda_e = 400$ job lots a year and a two week lead time.

a. Compute $E[Y]$, $E[Y^2]$ and the demand rate λ from Equation 8.47.

b. Compute the equilibrium distribution of the undershoot from Equation 8.56 and report $E[U]$ from Equation 8.57.
c. The storeroom sets $s = 40$ and $S = 80$. Report the order frequency and the ordering cost rate from Equation 8.59, and then from the uncorrected $\lambda/(S - s)$ that Section 8.12.2 warns against.
d. State which of the two is biased and in which direction. The error is small on this item. Say what would have to change about Table 8.20 to make it large, and give a lot size distribution with the same mean of 2.0 units that raises $E[U]$ to at least 1.5 units.

Exercise 8.5. The storeroom moves the transformer of Example 8.10 from a monthly review to a review every **two** months, and raises the order-up-to level from 13 to 15 to compensate. The lead time is unchanged at two months, and demand is unchanged at $\lambda = 45$ a year, which is 3.75 a month.

Reviews now occur at the end of every even month. As in Example 8.10, an order placed at the review ending month t is received at the start of month $t + 3$. At the end of month 0 the shelf holds 6 transformers, 5 are due in month 1, 4 are due in month 2, and nothing is owed. The monthly demands for months 1 through 10 are

$$3,\ 5,\ 2,\ 6,\ 4,\ 3,\ 7,\ 2,\ 4,\ 3$$

which are the demands of Table 8.25, so that only the review interval and the level have changed.

a. Report the protection interval from Equation 8.60, the expected demand over it, and the safety stock $S - \lambda\tau$ that $S = 15$ provides.
b. Work the ledger for months 1 through 10, in the columns of Table 8.25. Note in a separate column which months carry a review.
c. Verify $IP = I + IO - B$ on every month in which a transformer is owed.

d. Verify Equation 8.61 at each of the four reviews that can be checked within the ledger, by comparing 15 against the demand of the following four months.
e. Table 8.25 was short in one month out of ten at $S = 13$. This ledger is short in three at $S = 15$. Using your answer to part (a), explain why raising the level by two units did not compensate for doubling the review interval, and state the level that would have restored the same safety stock.

Exercise 8.6. The utility's demand for the transformer has grown to $\lambda = 52$ units a year, which is one a week. The supplier now quotes a constant six week lead time, and the storeroom reviews the item every four weeks. Take $k = \$220$ an order, $c = \$3{,}800$ and $b = \$8,550$ per unit per year, and take demand to be Poisson.

This exercise is worked entirely from Table D.3 and Table D.4. The calendar has been chosen so that every mean you need is a printed column.

a. Report τ, and the three mean demands λL, $\lambda(L + R/2)$ and $\lambda\tau$ that Equation 8.70 calls for. Confirm that each is a column of the tables. Report λR and the ordering cost rate from Equation 8.64.
b. Using Equation 8.70 on the distribution function, compute $\bar{G}(S)$ for $S = 10, 11, 12, 13$, and apply Equation 8.72. Show the column you walked and state S^*.
c. At S^*, compute $\bar{B}$ from Equation 8.70, showing the three G^1 values you read.
d. Report $\bar{I}$ from Equation 8.66, the fill rate from Equation 8.69, and the cost from Equation 8.71.
e. Now read the critical ratio off the $\lambda = 10$ column of Table D.3 instead, which is demand over the full protection interval. Report the level that rule returns and the cost of running it. State, in one sentence, which question that rule answers.
f. Compute the review interval that Equation 8.74 recommends, in weeks, and say whether the storeroom's four week calendar needs changing.

Exercise 8.7. Each part concerns Section 8.13. Answer in a sentence or two, and name the equation or table that settles it.

a. True or false: under an (R, S) policy, halving the demand rate halves the number of orders placed per year, as it does under (r, Q).
b. True or false: doubling the review interval doubles the safety stock required to hold a given service level, all else equal.
c. A planner reports that an item reviewed monthly with a two month lead time "carries three months of stock on average". What has the planner confused, and what is the correct average exposure?
d. Which of the following is the reason $G^1_\tau(S)$ overstates the expected backorder level under an (R, S) policy? Choose one and say why each of the others is wrong.

   i. It uses demand over the protection interval rather than over the lead time.
   ii. It evaluates the backorder level at the worst instant of the review interval rather than averaging over the cycle.
   iii. It ignores the undershoot.
   iv. It assumes at most one order is outstanding.

e. An analyst proposes to model a periodically reviewed item as an (r, Q) system with $Q = \lambda R$, arguing that the two order the same amount per cycle. Name the quantity that substitution gets right and the quantity it gets wrong, and say in which direction the error runs.

8.19.2. Using the Software

The code of Section 8.17 is in `code/`, package `inventory.continuousreview`. Report the code you wrote alongside each answer.

Exercise 8.8. The storeroom's splice kit moves $\lambda = 1{,}500$ a year at $c = \$28$ and $k = \$55$. The supplier quotes three weeks. The utility puts the backorder cost at \$70 per unit per year, ten times the holding cost.

a. Build the item with `LeadTimeDemand.matched`. Report θ, the variance to mean ratio, and which family was selected. Say what would have had to be true of the data for a different family to be chosen.
b. Compute the economic order quantity of Equation 3.17 and round it. Holding that Q fixed, evaluate Equation 8.1 across r and report the table of Table 8.5 for the five reorder points either side of the best one.
c. Now run `RQOptimizer.optimize` and report the optimal policy and its cost. Express the cost of your answer to part (b) as a percentage above it, and relate that percentage to Equation 3.22.
d. Compute the cost of the best one-for-one policy, including the ordering term that Section 8.4 set aside. Compare it with part (c) and say in one sentence why the comparison comes out so differently here than it does for the transformer in Table 8.6.

Exercise 8.9. Section 8.14 applies one backorder cost of $287.50 per unit per year across the whole storeroom. Take that figure, a two week lead time, and Poisson lead time demand, and consider the meter socket, the ground rod and the warning tape.

a. For each item, report θ, the cost ratio w, and the bounds of Section 8.9.
b. Solve each item twice, with `optimizeFromUnitBatch` and with `optimize`. Confirm that the two agree on the policy and on the cost.
c. For each answer, verify Equation 8.42 and Equation 8.43 and report both as passing or failing.
d. Report `windowEvaluations` for both searches on all three items. Explain why the ratio between them grows with Q^*, using the closed form of Section 8.17.3.
e. Construct an item on which `optimize` costs **more** evaluations than `optimizeFromUnitBatch`, and say what property of the item makes that happen.

Exercise 8.10. The storeroom's smart meter moves $\lambda = 1{,}200$ a year at $c = \$95$ and $k = \$110$, with a two week lead time and Poisson lead time demand. The utility will not name a backorder cost.

a. For ready rate targets of 0.90, 0.95, 0.98 and 0.99, use `RQModel.fromReadyRateTarget` to build the item and report the implied b in each case.
b. Solve each and report the policy, the cost, and the fill rate the policy actually achieves.
c. The achieved fill rates fall short of the targets, and two separate things cause it. Set $Q = 1$ and evaluate the base-stock policy at the level Equation 8.15 gives. The shortfall is already there. Compare Equation 8.15, which Equation 8.33 inverts, against Equation 8.13, and say why the two are read one unit apart on the same distribution function. Then report, for each of the four targets, how the achieved rate at the optimal Q compares with the rate at $Q = 1$, and account for the fact that it does not move the same way every time.
d. Report the cost of moving from a 0.90 target to a 0.99 target, as a dollar figure and as a percentage. Then report the implied b at each, and say which of the two numbers a manager should be shown.

Exercise 8.11. Continue with the item of Exercise 8.6: the transformer at $\lambda = 52$ a year, a constant six week lead time, $k = \$220$, $h = \$950$ and $b = \$8,550$. Build it with `DemandOverInterval.poisson`.

a. Using `RQModel` and `RQOptimizer`, find the best (r, Q) policy under continuous review and report its cost, its fill rate, its safety stock $r - \theta$ and its cycle stock $\frac{1}{2}(Q + 1)$.
b. Using `RSModel`, find the best (R, S) policy, searching R over the range from one week to sixteen weeks. Report R^*, S^*, the cost, the fill rate, the safety stock and the cycle stock.
c. Report the premium in part (b) over part (a), in dollars and as a percentage, and confirm that the two policies deliver comparable fill rates before you quote it.

d. Decompose the premium across the three cost terms. State which of the two stock quantities rose, which fell, and why, referring to Equation 8.6 for the first and Equation 8.66 for the second.
e. The storeroom actually reviews every four weeks. Report what that calendar costs against your answer to part (b), and say whether it is worth changing.

Exercise 8.12. The storeroom's meter socket moves $\lambda = 900$ a year at $c = \$48$ and $k = \$82.50$, with a constant two week lead time. Take the storeroom-wide backorder cost of \$287.50 per unit per year from Section 8.14 and build the item with `DemandOverInterval.poisson`.

a. Report h, θ over the lead time, the economic order quantity of Equation 3.17, and the review interval of Equation 8.74 in weeks. Confirm that the second is the first divided by the demand rate.
b. For review intervals of one, two, four, six and eight weeks, report S^*, the fill rate and the three cost terms. Present it as Table 8.28 is presented.
c. Report the interval that minimizes cost to the nearest tenth of a week, and express the cost at the interval from part (a) as a percentage above it.
d. At the best interval, verify Equation 8.66 by confirming that $\bar{I}$ equals the safety stock plus $\lambda R/2$ plus $\bar{B}$.
e. Confirm that `readyRate` and `fillRate` agree at the best policy, then rebuild the item with `DemandOverInterval.stationary` at a variance rate three times the demand rate and report both again. Explain the direction of the gap using Section 8.2.2.

8.19.3. Using the Workbook

Exercise 8.13. Open `Chapter8Models.xlsx` and work entirely within it.

a. On the `Ledger` sheet, confirm that the inventory position column never falls below r except in the instant before an order is placed,

and say which column would have to change for Equation 8.17 to fail.

b. On the `Evaluate` sheet, reproduce Table 8.5 by changing only the reorder point in B13. Report the five rows.
c. Verify the arithmetic check of Example 8.4 at $r = 9$ by reading B15, B17 and B10, and confirm that B17 uses $\frac{1}{2}(Q + 1)$ and not $Q/2$.
d. Raise the demand rate in B5 from 45 to 300, leaving everything else alone. Report what the sheet now says for $\bar{B}$, the fill rate and $\bar{I}$. The answers are absurd rather than merely wrong. Diagnose why, naming the range the sheet is built over, and state the largest θ you would trust it at.

Exercise 8.14. Continue in `Chapter8Models.xlsx`, on the `Optimize` sheet.

a. Reproduce Table 8.19. Report the r column across the steps and confirm the property of Section 8.10.1 that it falls by at most one per step.
b. Read the two checks in B99 and B103 and confirm both hold.
c. Now change the item to a cheap one: set the holding cost in B7 to \$18 and the backorder cost in B8 to \$162, leaving λ, k and the lead time alone so that θ is still 7.5. Report r^*, Q^* and C^* as the sheet gives them, and read B99 and B103 again.
d. One of the two checks fails. Explain what the sheet has done wrong, using the number of steps the algorithm laid down against the Q^* this item needs.
e. Find the true optimum for the item of part (c) by any means you like, and report how much the sheet's answer costs above it. Then state the general lesson in one sentence: what the check of Equation 8.43 is for.

Exercise 8.15. Continue in `Chapter8Models.xlsx`, on the `Service` and `Approximate` sheets.

a. On `Service`, with both targets at 0.95 and $Q = 5$, reproduce the first two rows of Table 8.13. Report both reorder points and say which target is the more demanding on this item.
b. Raise the Type 2 target in B14 until the Type 2 reorder point equals the Type 1 reorder point of part (a). Report the target at which that happens, and state what it means to say that a 95% cycle service level is worth that much fill rate on this item.
c. On `Approximate`, confirm the iteration settles at the policy of Example 8.6 and report the three cost terms it gives.
d. Read the three diagnostics in B57, B58 and B59 and say what each measures. The chapter says two should be small and one below one. Report which condition this item fails and by how much.
e. Raising the demand rate alone will not fix the condition that fails. Show why, by writing θ/Q in terms of λ, k, h and L with Q taken from Equation 3.17, and saying which way it moves as λ rises.
f. Now enter the compression connector of Example 8.5 on the same sheet: $\lambda = 6{,}000$, $k = \$27.50$, $h = \$0.85$, $\pi = \$8.50$ and a two week lead time. Report the three diagnostics again and say, in one sentence, what kind of item the classical approximation is built for.

Exercise 8.16. Continue in `Chapter8Models.xlsx`, on the `Periodic` sheet. The review interval is in B13, the order-up-to level in B14, and the lead time in B9.

a. Reproduce the cost column of Table 8.27 by reading N41 through N45, which are the levels 12 through 16. Confirm that B93 gives $S^* = 14$. The costs will not match the chapter to the cent. Report the largest discrepancy across the five levels and say what causes it, naming the equation the sheet uses.
b. The three means Simpson's rule needs are in B23, B24 and B25. Report them, and report the three G^1 values the sheet reads at $S = 14$, which are D43, G43 and J43. Verify H23 against Equation 8.70 by hand.

c. Confirm the decomposition of Equation 8.66 by reading the safety stock in B19, the cycle stock in B20 and $\bar{B}$ in H23, and checking them against $\bar{I}$ in H24.

d. Set B13 to 0.001 years. Report S^* from B93 and compare it with the base-stock answer of Table 8.4. Then report the ordering cost and the total cost in L27, compare them with the base-stock policy's \$9,900 and \$14,760 a year from Table 8.6, and explain in one sentence why they converge as the review interval vanishes.

e. Set B13 back to one month, then work it up to 0.25, 0.5 and 1.0 years, reporting the level B93 gives and the sheet's $\bar{B}$ at it each time. The exact values, by fine quadrature, are 0.228690, 0.397945 and 0.370751 units at the levels the sheet selects. Report the percentage error at each, and account for its growth using the property of Simpson's rule named in Section 8.13.2. At one of the four intervals the error is large enough to move the level itself: say which, and report both levels. State the longest review interval at which you would trust this sheet.

9. Multi-Echelon Inventory Systems

Learning objectives

After reading this chapter you should be able to:

- state the stock positioning problem and explain why it is answered by an inventory policy rather than alongside one
- explain why expensive, slow moving, repairable items are managed one unit at a time
- apply Palm's theorem to obtain the distribution of the units in resupply at a location
- compute backorders, the ready rate, and on-hand stock for a single location operating a one-for-one policy
- work a coupled depot and base ledger by hand, and read the delay the depot imposes out of it
- apply METRIC to a two-echelon repairable item, and state what it approximates
- improve that approximation with VARI-METRIC, and say what carrying a second moment buys
- allocate a fixed budget across many items by marginal analysis

Section 4.6 introduced echelon inventory, echelon holding cost, and the reorder intervals that minimize cost when demand is known and constant. This chapter keeps the structures and removes that assumption. Echelon accounting carries over unchanged, which is most of the reason it was

worth defining; what does not carry over is the idea that a location can be described by a reorder interval alone, since with random demand it needs stock held against the variability as well.

Chapter 8 solved one location. It took the lead time as a property of the supplier, a number the purchasing department could quote. That is the assumption this chapter removes, and removing it is what makes a second location more than a second copy of the first. When the supplier is itself a stocking location, its lead time is not a quoted constant. It is an outcome of its own inventory policy, and it gets worse exactly when the supplier is short.

This chapter works that coupling in its simplest setting: repairable items managed one unit at a time. Recall from Chapter 8 that an order quantity exists because ordering costs money, so demand is accumulated until a batch is worth placing. For an expensive, slow moving repairable item that logic fails, and the location replenishes one unit for each unit demanded. Thus, the demand a location passes up to its supplier is its own demand, arrival for arrival, and nothing stands between the two levels except the supplier's ability to fill. That is the coupling, and this chapter studies nothing else.

9.1. Where Should the Stock Sit?

Large-scale commercial and military organizations stock items across many locations, at distribution centers and at the sites those centers support. A question arises before any policy can be set for an item: **where should it be stocked?** The item might be supplied from a single central location, or held at several locations closer to the people who need it. If it is held at several, then the support network between them matters, because some locations act as suppliers to others. This question is the **stock positioning problem**.

Positioning is not a question about how much inventory to hold but about **which locations hold any**, and which location replenishes which. Thus, it sits above the models of Chapter 8 rather than beside them.

Yet it cannot be separated from them. If an item is stocked at a location then that location needs an inventory policy, so positioning implies policy setting. The reverse implication matters more. Suppose we solve the policy problem for a forward location and the answer is that the location should hold no units at all. That is more than a small policy: it is the model telling us the location should not be a stocking point, because nothing wants to sit there.

Thus, **the answer to "how much" also answers "whether"**. That observation organizes the whole chapter. We are going to build machinery that sets policies at two coupled locations, and the reason to build it is that the policies, read correctly, decide the positioning.

9.1.1. Why Two Locations Are Enough

A support network can have many locations and many layers. Analyzing all of them at once is a large problem, and this book does not solve it. Notice that it does not have to in order to be useful.

The essential features of stock positioning appear in the comparison of **one location against two**, where one of the two supplies the other. That comparison contains the question of whether forward stock is worth holding, the question of what a supplying location owes its customers, and the interaction that makes the two inseparable. A network is then a collection of such pairs, analyzed one pair at a time.

That is the reduction this chapter takes, and it is the one such organizations take in practice. The two-location system has a name, and it is the subject of everything that follows: a **two-echelon inventory system**.

9.1.2. What Changes When Stock Moves Forward

Recall from Chapter 8 that a single location holds stock for one reason: demand arrives during a lead time and cannot be met from an empty shelf. Moving stock forward changes three things at once, and they do not point the same way.

Forward stock is closer to demand, so it is faster. A unit held at a supported site can be issued immediately. The same unit held at a distribution center has to be picked, packed and shipped first.

Forward stock is less pooled, so there is more of it. Recall from Equation B.12 that aggregating independent demand streams makes the total relatively less variable, since the mean grows with the number of streams and the standard deviation only with its square root. Section B.2.1 named this chapter as the place that fact is spent. For example, one location serving ten supported sites faces the pooled stream. Splitting that stock across ten locations gives up the pooling, and each location then covers its own variability separately.

Forward stock does not remove the need for stock behind it, either. The supplying location still faces demand, now from the locations it supports rather than from end customers, and it still needs a policy.

Thus, positioning trades responsiveness against the total investment needed to achieve it, and the trade is not obvious in either direction. Section 9.9 computes it on real items and finds the answer going both ways depending on the item.

One warning before the machinery starts. It is tempting to decide positioning by rules of thumb: stock fast movers forward, stock slow movers centrally, stock expensive items centrally. Those rules are usually right and they are not always right, and the cases where they fail are the ones to compute. You should treat them as a way to prioritize which items to analyze, and not as the analysis.

Section 9.2 now fixes the setting, and explains why an entire class of items is replenished one unit at a time.

9.2. Ordering One Unit at a Time

Section 8.4 already ran the storeroom's pad-mount transformer one for one. It set $Q = 1$, observed that the outstanding orders are the lead time demand, and read the whole model off Equation 8.9. This chapter keeps that policy and changes what stands behind it. Recall that Chapter 8 treated the transformer as **consumed**: a failed unit left the system and a replacement was purchased. That is not what a utility does with a transformer.

9.2.1. The Item Comes Back

A pad-mount transformer that fails is not scrapped. It is pulled from the pad, a serviceable unit is installed in its place, and the failed unit goes to a shop where it is rewound, retested and returned to stock. At $3,800 a unit that is worth doing, and the same is true of aircraft engines, circuit cards, pumps and locomotive traction motors. An item managed this way is a **repairable item**, and Sherbrooke (1968) calls it a **recoverable item**.

That single change has a consequence. A failure is simultaneously two events. It is a **demand** for a serviceable unit, and it is an **arrival** of an unserviceable one. The two happen at the same instant and they are the same physical event, so the quantity demanded and the quantity inducted into repair are never different.

Thus, one-for-one replenishment is not a policy choice here but what the item does. Section 8.4 presented $Q = 1$ as a decision the storeroom made, and for a consumable it is one. For a repairable there is nothing to decide, because the repair pipeline is fed by failures, one failure at a time, and no ordering cost stands between the failure and the induction that could

be amortized by waiting. That is why this chapter can carry two levels without carrying an order quantity, and it is the whole reason to start here.

> The transformer's economic order quantity as a consumable is not one. From Table 4.5 it moves $\lambda = 45$ units a year against an ordering cost of $k = \$220$ and a holding cost of $h = 0.25(\$3{,}800) = \$950$ per unit per year, so Equation 3.17 gives
>
> $$Q^* = \sqrt{\frac{2(220)(45)}{950}} = \sqrt{20.84} = 4.57$$
>
> which rounds to five. Notice that this does not contradict anything. As a consumable the transformer has a batching decision and the answer is five; as a repairable it has no batching decision at all. The population is fixed, and what circulates is the units themselves.

A fixed population is the second consequence. Because failures are repaired rather than replaced, the number of units the system owns does not change as demand is met. Buying more units is a separate and much slower decision, usually made once when the fleet is fielded. Thus, the day-to-day question is not how much to buy. It is **where the units the system already owns should sit**, which is the stock positioning problem of Section 9.1, and it is now the only question there is.

Two terms are used throughout. A **serviceable** unit is one that is ready to be issued. An **unserviceable** unit, often called a **carcass**, is one that has failed and is awaiting or undergoing repair. A unit is always exactly one of the two, and the sum over both categories and all locations is the fixed population.

9.2.2. The Two Levels

Recall from Section 9.1.1 that this book takes the two-location reduction. For a repairable item the two locations have standard names in the literature, which Sherbrooke (1968) established and everything since has followed: the forward locations are **bases** and the central location is the **depot**.

> ⚠ This book says **storeroom** where that literature says base, and reserves **base-stock** for the policy of Section 8.4. The two words are unrelated and using both would invite the confusion this chapter cannot afford. Thus, storerooms and a depot, and when you read Sherbrooke (1968) or Muckstadt and Sapra (2010), bases are storerooms.

Figure 9.1 draws the system.

Walk the figure before going on. Start at the bottom. Failures arrive at storeroom j at rate λ_j, and the storeroom issues a serviceable unit if one is on the shelf. Either way a carcass appears, and it is inspected. A fraction ϕ_j of carcasses can be rebuilt at the storeroom, and those travel the lower loop and come back to that storeroom's own stock after an average repair time T_j. The remaining fraction $1 - \phi_j$ cannot be rebuilt locally, so the carcass goes up the solid arrow to the depot and the storeroom asks the depot for a serviceable in exchange. Coming back down the dashed arrow, the depot ships one if it has one, and it arrives after an order and ship time O_j plus however long the request waited at the depot. The depot meanwhile rebuilds what it receives and returns it to depot stock after an average time T_0.

Notice that no arrow leaves the figure and none enters it. That is the fixed population. Contrast it with the consumable systems of Chapter 8, where every arrow eventually traces back to a purchase order. Here the units

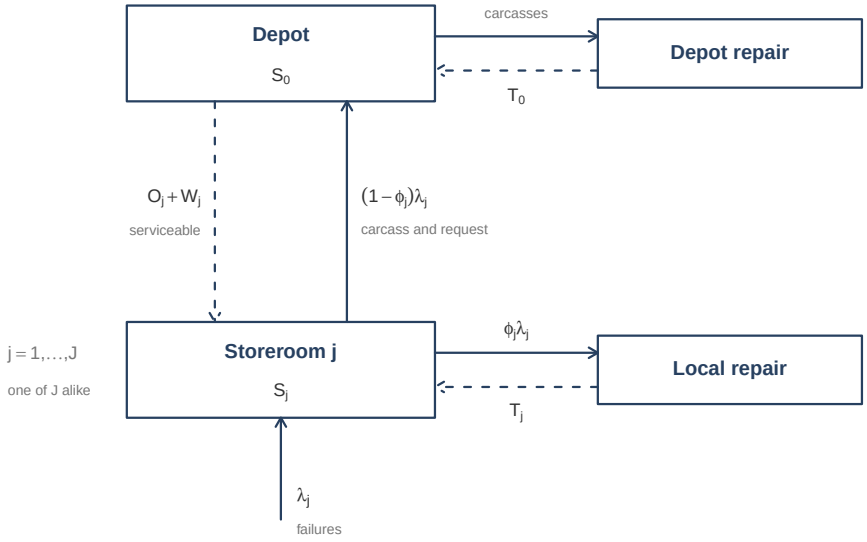


Figure 9.1.: A two-echelon repairable system, drawn for one storeroom of the J the depot supports. Notice that the loop is closed: no arrow leaves the figure and none enters it, because a failure creates a carcass rather than consuming a unit. A failure at the storeroom is met from its shelf if a serviceable unit is there. The carcass is inspected, and a fraction ϕ_j of failures can be rebuilt locally in an average time T_j. The rest go to the depot, and the storeroom asks the depot for a replacement serviceable, which arrives after an order and ship time O_j plus whatever the request waits at the depot. The depot rebuilds what it receives in an average time T_0 and returns it to depot stock.

circulate, and the only question is how many are sitting at each place at any moment.

Notice which delay is the hard one. O_j is a shipping time and T_j and T_0 are shop flow times, and all three are properties of the physical system that can be measured. W_j, the wait a request suffers at the depot, is not. It is an outcome of how much stock the depot chose to hold, and it gets longer exactly when the depot is short. That single quantity is what couples the two levels, and the rest of this chapter is built around computing it.

9.2.3. Notation

The chapter indexes the depot by 0 and the storerooms by $j = 1, \dots, J$. Let

- λ_j represent the failure rate at storeroom j, assumed Poisson,
- ϕ_j represent the fraction of failures at storeroom j that storeroom j can repair, so that $0 \le \phi_j \le 1$,
- T_j represent the average repair time at storeroom j for the carcasses it repairs,
- O_j represent the average order and ship time from the depot to storeroom j,
- T_0 represent the average repair time at the depot,
- S_j represent the stock level at storeroom j and S_0 the stock level at the depot, each being the number of units the location holds when nothing is in resupply.

Two rates follow immediately and both are used constantly. The rate at which storeroom j asks the depot for units is the rate of failures it cannot repair itself,

$$\lambda_{j0} = (1 - \phi_j)\lambda_j \tag{9.1}$$

and the depot's own demand rate is the sum of those over the storerooms it supports,

$$\lambda_0 = \sum_{j=1}^{J} \lambda_{j0} = \sum_{j=1}^{J} (1 - \phi_j)\lambda_j \tag{9.2}$$

Equation 9.1 is the payoff for starting with one-for-one ordering. The stream a storeroom sends up is its own failure stream, thinned by local repair and nothing else. That is, the requests arrive one at a time, at the instants the failures occur, and a thinned Poisson process is Poisson. Thus, the depot faces Poisson demand at rate λ_0, and Section 9.7 can treat the depot with the same machinery as a storeroom.

For example, take the utility's transformer with $\lambda_j = 45$ failures a year at a storeroom that rebuilds three in ten locally. Then $\phi_j = 0.3$ and Equation 9.1 gives $\lambda_{j0} = 0.7(45) = 31.5$ requests a year to the depot. If four such storerooms are supported and all look alike, then Equation 9.2 gives $\lambda_0 = 4(31.5) = 126$ requests a year.

Compare that with what batching would demand. When a location orders in batches the stream it sends up is not its demand stream at all. Its timing, its size and its variability all have to be derived, and none of them is Poisson. Equation 9.1 is one line because $Q = 1$.

9.2.4. What Is Being Measured

Chapter 8 chose a policy by minimizing cost, trading a holding rate h against a backorder rate b. That worked because the storeroom could put a dollar figure on owing a transformer to a waiting crew. For the systems this chapter is aimed at, that figure is usually not available.

The reason is that the item does not fail alone. A repairable part is a component of a system, an aircraft, a locomotive, a substation. For example, a missing part does not cost money directly; it grounds the

aircraft. What the organization cares about is **availability**, the fraction of its systems that are not down waiting for parts, and availability is driven by **backorders** at the locations where the systems are.

Thus, the objective in this chapter is stated in backorders rather than dollars, and the decision variable is a budget rather than a cost rate. We will compute the expected backorders a set of stock levels produces, and Section 9.9 will spend a fixed sum across many items so as to make the total as small as possible. Notice that money has not disappeared. It has moved from the objective into a constraint, which is the same move Section 4.2 made for the deterministic case.

One measure carries over unchanged, and we name it now. Let $\bar{B}_j(S_j)$ represent the expected number of backorders outstanding at storeroom j when it holds stock level S_j. Everything this chapter computes is in service of that one quantity, at the storerooms, where the systems are.

9.2.5. Where This Leaves Us

We have a closed system of circulating units, a policy that holds each location's inventory position fixed, and a request stream between the levels that is Poisson by Equation 9.1. What we do not yet have is the distribution of the number of units in resupply at a location, and Equation 8.9 says that is the only thing standing between us and every performance measure.

Section 9.3 supplies it, and supplies it exactly. The result is stronger than we have any right to expect: the distribution depends on the repair time only through its mean.

9.3. Palm's Theorem

Equation 8.9 says that the net inventory at a location is its stock level minus the number of units in resupply. Everything this chapter computes

follows from that one identity, so everything depends on knowing the **distribution of the number of units in resupply**. For a consumable with a constant lead time Equation 8.8 settled it in one line: the units on order are the lead time demand. A repair pipeline is not so obliging. Repair times vary by an order of magnitude between a quick reseat and a full teardown, and the shop does not finish jobs in the order it receives them.

It turns out none of that matters. Recall from Section B.3.3 the result that makes this chapter possible.

Theorem 9.1 (Palm's theorem)**.** *Suppose failures occur in a Poisson process at rate λ, and each failure starts a resupply activity whose duration is drawn independently from a distribution with mean T. Let X represent the number of activities in progress at a random instant in steady state. Then X has the Poisson distribution with mean λT,*

$$P\{X = x\} = \frac{e^{-\lambda T}(\lambda T)^x}{x!} \qquad x = 0, 1, 2, ... \tag{9.3}$$

whatever the distribution of the duration may be. (Palm 1938)

Section B.3.3 proves it by conditional uniformity and thinning, and the same statement is the $M/G/\infty$ occupancy distribution.

9.3.1. Why Distribution-Free Is the Whole Point

Only the mean repair time enters Equation 9.3. Not its variance, not its shape, not whether it is bimodal because half the carcasses need a rewind and half need a bushing. That is a remarkable license, and what it buys is plain once you remember that repair time data is the worst data in

this business. For example, a shop can usually tell you its average flow time; it can rarely tell you the distribution, and what it does tell you is contaminated by jobs that sat waiting for a part. Palm's theorem says we may use the average and stop.

> ⚠ **Do not read this as "lead time variability never matters".** Chapter 8 built the opposite lesson, and Equation B.30 inflates the lead time demand variance by $Var[L](E[D])^2$ when the lead time is random. Both are correct, and Section B.3.3 sets them side by side. The separating question is whether an activity started later can finish earlier. Under a single common lead time it cannot, and one long draw stretches everything in the pipe at once. In a repair shop it can, and routinely does, so a long teardown and a quick reseat overlap and the long one delays nothing. Thus, the variability averages away between units rather than accumulating across them.

That is a statement about the physics of the shop rather than a modeling convenience, and you should check that it describes yours before relying on it.

9.3.2. What the Theorem Requires

Three conditions carry Equation 9.3, and the third is the one that fails in practice.

Failures arrive in a Poisson process. Reasonable for a population of independently operating units, and it is the standard assumption for the items this chapter treats.

Durations are drawn independently, one per failure. This is what lets the thinning argument run.

Repair capacity never binds. The $M/G/\infty$ reading makes this explicit: there are infinitely many servers, so a carcass begins repair the instant it arrives and never queues behind another. Notice that if the shop has three test benches and twelve carcasses show up, the later ones wait, their durations are no longer independent of one another, and Palm's theorem does not apply. Thus, the model suits a shop sized comfortably above its workload, and it will understate the pipeline for one that is not.

9.3.3. The Pipeline at a Storeroom

Recall from Section 9.2.2 that a carcass at storeroom j takes one of two routes. Each route is a thinning of the failure stream, so by Section B.3.2 each is itself Poisson, and the two are independent.

The locally repaired units are in progress at rate $\phi_j\lambda_j$ for an average time T_j. The units sent to the depot are in progress at rate $(1-\phi_j)\lambda_j$ for however long it takes the depot to put a serviceable unit on the storeroom's shelf, which is the order and ship time O_j plus the wait W_j at the depot. We apply Equation 9.3 to each route and add, using the closure of the Poisson family under addition from Section B.2.3, and the number of units in resupply for storeroom j is Poisson with mean

$$\mu_j = \underbrace{\phi_j\lambda_j T_j}_{\text{local repair}} + \underbrace{(1-\phi_j)\lambda_j\left(O_j + \bar{W}_j\right)}_{\text{from the depot}} \tag{9.4}$$

where $\bar{W}_j = E[W_j]$ represents the average wait a request suffers at the depot.

Equation 9.4 is the chapter in one line. Every quantity in it is a measurable property of the physical system except one. $\bar{W}_j$ is not measurable in advance, because it is produced by how much stock the depot decided to hold. That is, the storeroom's pipeline, and therefore its backorders, and therefore the availability of the systems it supports, all depend on a decision

made at the depot. Thus, the two levels cannot be solved separately, which is what "multi-echelon" means.

> ⚠ **Equation 9.4 applies Palm's theorem slightly outside its warrant, and the rest of the chapter is about that.** The theorem wants durations drawn independently. The depot resupply durations are not independent of one another, because two requests arriving while the depot is short both wait on the same missing units. Section 9.7 adopts Equation 9.4 anyway, which is what METRIC does and what makes it an approximation rather than a result. Section 9.8 measures the damage and repairs it.

Example 9.1 (The transformer pipeline at one storeroom)**.** The utility of Example 8.3 now treats the pad-mount transformer as a repairable. A failed transformer is pulled, a serviceable one is installed, and the carcass is inspected. The storeroom's own shop can rewind three failures in ten; the rest go to the utility's central depot, which also holds serviceable stock.

What is stocked? One item, the pad-mount transformer, at one of four storerooms.

What is the demand process? Poisson at $\lambda_j = 45$ failures a year, one unit at a time.

When is inventory reviewed? Continuously.

What triggers replenishment, and how much? Every failure, for one unit, per Section 9.2.1.

What happens to unmet demand? It is backordered. A crew waits.

What costs are incurred, and when? None are needed here. This example computes a pipeline, not a policy.

Notation. Let $\phi_j = 0.3$ represent the fraction rebuilt locally, $T_j = 30$ days the average local repair time, and $O_j = 10$ days the average order and

ship time from the depot. Take $\bar{W}_j = 0$ for the moment, which describes a depot that never makes anyone wait.

The computation. The local stream contributes

$$\phi_j \lambda_j T_j = (0.3)(45)\left(\frac{30}{365}\right) = 13.5\left(\frac{30}{365}\right) = 1.1096 \text{ units}$$

and the depot stream contributes

$$(1-\phi_j)\lambda_j\left(O_j + \bar{W}_j\right) = (0.7)(45)\left(\frac{10}{365}\right) = 31.5\left(\frac{10}{365}\right) = 0.8630 \text{ units}$$

so Equation 9.4 gives $\mu_j = 1.1096 + 0.8630 = 1.9726$ units.

The check. Compute it the other way, as one rate times one average duration. The average resupply duration over both routes is

$$E[\text{duration}] = (0.3)(30) + (0.7)(10) = 9 + 7 = 16 \text{ days}$$

and $45(16/365) = 1.9726$ units, which agrees. That check is worth performing every time, because it catches a misplaced ϕ_j immediately, and putting ϕ_j on the wrong stream is the most common error here.

Interpretation. At any moment the storeroom has on average about two transformers away being fixed, and by Equation 9.3 the count is Poisson with that mean, so the standard deviation is $\sqrt{1.9726} = 1.4045$ units. Notice what that means for stocking. A storeroom that holds nothing serves no crew from the shelf, whatever the pipeline does. If it holds a single spare, that spare is on the shelf exactly when nothing is away being fixed, and $P\{X = 0\} = e^{-1.9726} = 0.1391$, so a crew arriving to a shelf that has a transformer on it happens fourteen times in a hundred. That is what one spare buys, and it is why the location stocks more than one.

What the depot's stock does to this. Now suppose the depot is thin enough that a request waits $\bar{W}_j = 15$ days on average. Only the second term moves:

$$(0.7)(45)\left(\frac{10+15}{365}\right) = 31.5\left(\frac{25}{365}\right) = 2.1575 \text{ units}$$

so $\mu_j = 1.1096 + 2.1575 = 3.2671$ units, an increase of 65.6 percent. The check holds: $(0.3)(30) + (0.7)(25) = 26.5$ days and $45(26.5/365) = 3.2671$.

Notice that the storeroom did nothing. Its failure rate did not change, its own shop did not slow down, and its stock level was never mentioned. A decision taken at the depot raised the number of transformers this storeroom has in the air by two thirds, and the storeroom will have to hold more units to absorb it. That is the coupling.

9.3.4. Where This Leaves Us

We have the distribution of the units in resupply, exactly, and it needs only mean repair times. Combined with Equation 8.9 that is enough to compute every performance measure at a single location, and Section 9.4 does so without introducing anything new.

The quantity we do not have is $\bar{W}_j$. Section 9.6 obtains it, and obtains it from the depot's own backorders by an argument the book has already made once.

9.4. A Single Location Under One-for-One Ordering

This section introduces no new mathematics, and saying so plainly is the point. Section 8.4 already derived every performance measure for a location holding its inventory position fixed, and Section 9.3 has just

supplied the one distribution those measures need. Putting the two together is a substitution.

9.4.1. The Substitution

Recall the derivation behind Equation 8.9. It used two facts and nothing else: the inventory position is held at S, and the units outstanding are those whose resupply has not yet finished. Notice that neither fact says what the resupply *is*. Chapter 8 had it be a purchase order in transit from a supplier; here it is a carcass on a workbench or a serviceable unit moving down from the depot. The accounting does not care.

Thus, writing X_j for the number of units in resupply for storeroom j,

$$IN_j = S_j - X_j \tag{9.5}$$

and Equation 9.3 says X_j is Poisson with the mean μ_j of Equation 9.4. That is the entire model, and Equation 9.5 is Equation 8.9 with the lead time demand replaced by the pipeline.

9.4.2. The Measures

Every result of Section 8.4.1 now transfers by changing the name of the random variable. Using G, G^0 and G^1 for the distribution function and the loss functions of the Poisson distribution with mean μ_j, as defined in Section C.3.1, the **expected backorder level** at storeroom j is

$$\bar{B}_j(S_j) = E\left[(X_j - S_j)^+\right] = G^1(S_j) \tag{9.6}$$

the **expected on-hand** is

$$\bar{I}_j(S_j) = S_j - \mu_j + G^1(S_j) \tag{9.7}$$

and the **ready rate**, the fraction of time a serviceable unit is on the shelf, is

$$\overline{RR}_j(S_j) = P\{X_j \le S_j - 1\} = G(S_j - 1) \tag{9.8}$$

Because failures arrive one at a time in a Poisson process, Equation 8.2 makes the fill rate equal to the ready rate, so the two need not be distinguished in this chapter.

Notice that Equation 9.7 is an identity and not an approximation. Rearranged it says $\bar{I}_j - \bar{B}_j = S_j - \mu_j$, which is Equation 9.5 in expectation. That rearrangement is the arithmetic check to run on any table of these quantities, and it catches a mis-indexed loss function at once.

9.4.3. What One More Unit Buys

Here is the one result this section does add, and the whole of Section 9.9 rests on it. We ask what the $(S+1)$st unit at a location is worth, measured in backorders removed, and write

$$\Delta_j(S_j) = \bar{B}_j(S_j) - \bar{B}_j(S_j + 1) \tag{9.9}$$

Recall from Equation C.11 that the first order loss function is a tail sum of the complementary distribution function, $G^1(b) = \sum_{x \ge b} G^0(x)$. We drop one term from the tail sum, which leaves that term, so

$$\Delta_j(S_j) = G^0(S_j) = P\{X_j > S_j\} \tag{9.10}$$

Equation 9.10 reads as a sentence. The next unit removes a backorder exactly when the pipeline would otherwise have exceeded the stock, and the value of that unit is simply the probability that it does. That is, you are paying for a unit and receiving, in expected backorders, the chance that you would have been short without it. Recall Equation 9.8 and notice the same object appearing twice: $\Delta_j(S_j) = 1 - \overline{RR}_j(S_j + 1)$.

Notice that G^0 is nonincreasing, because it is a complementary distribution function. Thus, $\Delta_j(S_j)$ shrinks as S_j grows: the first unit at a location buys more than the second, which buys more than the third. That is **diminishing returns**, and it is equivalent to saying $\bar{B}_j$ is **convex** in S_j.

Convexity matters here. Section 9.9 has to spend a fixed budget over many items at many locations, and a greedy rule that repeatedly buys whichever unit removes the most backorders per dollar works only if the units get worse as you buy them. Equation 9.10 is the guarantee that they do.

Example 9.2 (Stocking one storeroom with transformers)**.** The item, the storeroom and the pipeline are those of Example 9.1, and nothing about the system has changed, so the modeling questions there have the same answers. Take the depot to be generous, so that $\bar{W}_j = 0$ and Equation 9.4 gives $\mu_j = 1.9726$ transformers in resupply on average.

The question. How many transformers should this storeroom hold?

The computation. Equation 9.6 through Equation 9.10 applied to the Poisson distribution with mean 1.9726 give Table 9.1.

Table 9.1.: Performance at one storeroom against its stock level, for a pipeline that is Poisson with mean 1.9726 units.

S_j	$G(S_j)$	$\bar{B}_j$	$\bar{I}_j$	$\overline{RR}_j$	Δ_j
0	0.1391	1.9726	0.0000	0.0000	0.8609
1	0.4135	1.1117	0.1391	0.1391	0.5865
2	0.6841	0.5252	0.5526	0.4135	0.3159

S_j	$G(S_j)$	$\bar{B}_j$	$\bar{I}_j$	$\overline{RR}_j$	Δ_j
3	0.8620	0.2093	1.2367	0.6841	0.1380
4	0.9498	0.0713	2.0987	0.8620	0.0502
5	0.9844	0.0211	3.0485	0.9498	0.0156
6	0.9958	0.0055	4.0329	0.9844	0.0042

The check. Take the row $S_j = 3$. Then $\bar{I}_j - \bar{B}_j = 1.2367 - 0.2093 = 1.0274$, and $S_j - \mu_j = 3 - 1.9726 = 1.0274$. The identity behind Equation 9.7 holds, as it must, and it holds on every row.

A second check, on the marginal column. Equation 9.10 says $\Delta_j(2) = G^0(2) = 1 - G(2) = 1 - 0.6841 = 0.3159$, and the backorder column agrees: $0.5252 - 0.2093 = 0.3159$. Perform this one too, because it confirms the loss function and the distribution function were evaluated at the same index, which is the error Equation 8.11 warns about.

Interpretation. Read the Δ_j column down and the shape of the decision appears. The first transformer removes 0.86 units from the average number owed to waiting crews, the second 0.59, the third 0.32, and the fifth only 0.016. At \$3,800 a unit the fifth transformer is buying almost nothing, and the utility would do better spending that \$3,800 on a different item or a different storeroom. Notice that we have not needed a backorder cost to see this. We have a ranked list of what each unit buys, and Section 9.9 turns a list like this one into a decision.

Validation against a closed form. If a shortage cost were available the answer would be the critical ratio of Equation 8.15, since Equation 9.5 is Equation 8.9 and the base-stock problem is the newsvendor problem. For example, the $h = \$950$ and $b = \$8{,}550$ of Example 8.3 give a ratio of $8550/9500 = 0.90$, and reading down the $G(S_j)$ column the first value at or above 0.90 is 0.9498 at $S_j = 4$. Thus, four transformers, and the marginal reading agrees that the fourth is the last one that buys much.

Where that stops applying. The critical ratio needs b, and the systems this chapter is aimed at cannot state it, per Section 9.2.4. It also treats this storeroom alone, when in fact the \$3,800 competes against every other item at every other storeroom. Both objections are answered the same way, by ranking marginal buys against a budget rather than pricing shortages, and that is what Section 9.9 does.

9.4.4. Where This Leaves Us

One location is solved completely, provided μ_j is known. Every number in Table 9.1 followed from it, and the pipeline mean followed from Equation 9.4, which contains $\bar{W}_j$, which we have been setting to zero.

It is time to stop doing that. Section 9.5 runs a depot and a storeroom together by hand, one month at a time, and watches the wait appear.

9.5. Working Two Levels by Hand

Section 9.4 solved a storeroom completely, on the condition that μ_j was known, and Equation 9.4 says μ_j contains $\bar{W}_j$, the wait at the depot. Every example so far has set that wait to zero. This section stops doing that, and it does so before any formula for the wait is offered.

The reason is this. The formulas of Section 9.6 onward produce a number for $\bar{W}_j$ in one line, and a reader who meets that line first has no way to tell whether it is right or even what it is counting. You should be able to perform by hand the bookkeeping that the worksheet will otherwise perform for you. Thus, we are going to run a depot and a storeroom together and watch the wait appear as a consequence of the depot being empty.

9.5.1. The System and the Rules

The system is Figure 9.1 with one storeroom. The ledger follows the convention of Example 8.1 exactly: **one row per event, at the week the event occurs, with the state recorded after that event has been processed.** Review is continuous here as it was there, so there are no periods, no buckets and no rule about what happens first within a bucket. Something happens, the books are posted, and the next thing happens whenever it happens.

The stock levels. The storeroom holds $S_j = 2$ transformers and the depot holds $S_0 = 2$. That is the entire serviceable population when nothing is in resupply, and by Section 9.2.1 the population never changes.

The times. A carcass rebuilt at the storeroom takes $T_j = 4$ weeks. A carcass sent to the depot takes $T_0 = 8$ weeks to repair. A serviceable unit shipped from the depot takes $O_j = 2$ weeks to arrive.

The failures. Seven transformers fail over the twenty-six weeks observed, in weeks 2, 5, 8, 11, 12, 15 and 17. The two in weeks 5 and 15 are rebuilt at the storeroom; the other five go to the depot. The split is given rather than generated from ϕ_j, so the ledger contains no randomness the reader cannot see.

What the columns are. Let $I_j(t)$ and $B_j(t)$ represent the storeroom's on-hand and backorders, and $X_j(t)$ the units in resupply for the storeroom, counting local rebuilds in progress, shipments in transit, and requests the depot has not yet filled. Let $I_0(t)$, $B_0(t)$ and $X_0(t)$ be the same three at the depot, where B_0 is what the depot owes the storeroom and X_0 is what it has on the bench.

> **Every one of those is a function of time, not a number per week.** $B_0(t)$ steps up when a request cannot be filled and steps back down when it is, and it holds its value in between. The ledger records it at event epochs because that is where it changes; between two rows it is constant at the value in the upper row. That distinction is dead weight until Section 9.5.3 has to average one of these columns, at which point it decides the answer.

The check to run on every row. The inventory position is held at the stock level, so on-hand minus backorders plus resupply is S at all times. Thus,

$$X_j = S_j - I_j + B_j \qquad \text{and} \qquad X_0 = S_0 - I_0 + B_0 \tag{9.11}$$

Equation 9.11 is Equation 9.5 rearranged. Compute the resupply columns two ways on every row, once by counting what is actually out there and once from the identity, and stop when they disagree.

9.5.2. The Ledger

Both locations begin full: two serviceable transformers at the storeroom, two at the depot, nothing on a bench, nothing owed.

Week 2. A transformer fails. The storeroom has two on the shelf, issues one, and is left with one. The carcass goes to the depot, which is simultaneously a request. The depot has two serviceable units, ships one, and is left with one. That shipment will arrive in week 4. The carcass goes on the depot bench and will be repaired in week 10.

Week 4. The shipment arrives and the storeroom is back to two on the shelf. Notice that the storeroom's resupply column has returned to zero, and that the depot's has not: the carcass is still on the bench.

Week 5. A second transformer fails and is issued, leaving one on the shelf. This carcass is rebuilt locally, so it never reaches the depot and will be back on the storeroom's own shelf in week 9.

Week 8. A third failure empties the shelf. The storeroom still owes nobody anything; every failure so far has been met on the day it happened. The carcass goes to the depot, which has one serviceable unit, ships it, and is left with none. That shipment arrives in week 10, and the carcass will be repaired in week 16.

Week 9. The local rebuild started in week 5 returns, putting one transformer back on the storeroom shelf.

Week 10. Two unrelated things happen in the same week at different places. At the depot, the carcass inducted in week 2 finishes repair and becomes a serviceable unit in depot stock. At the storeroom, the shipment sent in week 8 arrives, restoring the shelf to two.

The table has been updated as follows.

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
0	start	2	0	0	2	0	0
2	failure, request to depot	1	0	1	1	0	1
4	shipment arrives	2	0	0	1	0	1
5	failure, rebuilt locally	1	0	1	1	0	1
8	failure, request to depot	0	0	2	0	0	2
9	local rebuild returns	1	0	1	0	0	2
10	depot repair completes	1	0	1	1	0	1
10	shipment arrives	2	0	0	1	0	1

Ten weeks in, three transformers have failed and every one was replaced from a shelf on the day it failed. Both locations look healthy. Notice, though, that the depot has been holding at most one spare since week 2, and that it has filled every request out of that one unit.

Week 11. A fourth failure. The shelf falls to one, and the carcass goes up. The depot has exactly one serviceable unit, ships it, and is empty. That shipment arrives in week 13; the carcass is due off the bench in week 19.

Week 12, where the luck runs out. A fifth failure, one week after the fourth. The storeroom issues its last transformer and the shelf is empty, but it still owes nobody. The carcass goes up as a request, and **the depot has nothing to ship**. It records a backorder. From this instant $B_0(t) = 1$, and it will stay 1 until the depot can put a unit against it.

Notice what is now in the storeroom's resupply column: one shipment in transit, and one request that no serviceable unit anywhere in the system has been assigned to. Equation 9.11 gives $2 - 0 + 0 = 2$, and both items are counted.

Week 13. The shipment sent in week 11 arrives, so the storeroom has one on the shelf again. The depot's backorder is untouched by this; the two are different units.

Week 15. A sixth failure takes the shelf back to zero. This one is rebuilt locally and will return in week 19.

Week 16. The carcass inducted in week 8 comes off the depot bench. The depot owes the storeroom one unit from week 12, so the repaired unit goes straight out as a shipment rather than onto the depot's shelf. $B_0(t)$ falls back to 0. **That backorder lasted from week 12 to week 16, four weeks.** The shipment will arrive at the storeroom in week 18.

Week 17. A seventh failure arrives at an empty shelf, and the storeroom records its **first backorder**. A crew is waiting. The carcass goes up, the depot is empty again, and it records a backorder too, so $B_0(t)$ returns to 1.

The table has been updated as follows.

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
0	start	2	0	0	2	0	0
2	failure, request to depot	1	0	1	1	0	1
4	shipment arrives	2	0	0	1	0	1
5	failure, rebuilt locally	1	0	1	1	0	1
8	failure, request to depot	0	0	2	0	0	2
9	local rebuild returns	1	0	1	0	0	2

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
10	depot repair completes	1	0	1	1	0	1
10	shipment arrives	2	0	0	1	0	1
11	failure, request to depot	1	0	1	0	0	2
12	failure, **request backordered**	0	0	2	0	**1**	3
13	shipment arrives	1	0	1	0	1	3

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
15	failure, rebuilt locally	0	0	2	0	1	3
16	depot repair completes, ships against the backorder	0	0	2	0	0	2
17	failure **backordered**, request backordered	0	**1**	3	0	**1**	3

Week 18. The shipment the depot sent in week 16 arrives. The storeroom owes one unit, so it goes straight to the waiting crew rather than onto the shelf. $B_j(t)$ returns to 0. **The storeroom's backorder lasted one**

week.

Week 19. Two things again. The carcass inducted in week 11 comes off the depot bench and goes straight out against the week-17 backorder, so $B_0(t)$ falls to 0 after two weeks. The local rebuild started in week 15 also returns, putting one transformer on the storeroom shelf.

Week 20. The carcass inducted in week 12 is repaired. Nothing is owed now, so for the first time since week 11 the depot has a unit on its own shelf.

Week 21. The shipment sent in week 19 arrives and the storeroom is back to two. **Week 25.** The last carcass is repaired and the depot is back to two. The system has returned to where it started, which it must, because the population is fixed.

The table has been updated as follows, and this is the completed ledger.

Table 9.2.: The completed ledger, twenty-six weeks at a storeroom and the depot that supplies it. Seven failures, five requests to the depot, two of which the depot could not fill on arrival.

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
0	start	2	0	0	2	0	0
2	failure, request to depot	1	0	1	1	0	1
4	shipment arrives	2	0	0	1	0	1

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
5	failure, rebuilt locally	1	0	1	1	0	1
8	failure, request to depot	0	0	2	0	0	2
9	local rebuild returns	1	0	1	0	0	2
10	depot repair completes	1	0	1	1	0	1
10	shipment arrives	2	0	0	1	0	1

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
11	failure, request to depot	1	0	1	0	0	2
12	failure, **request backordered**	0	0	2	0	1	3
13	shipment arrives	1	0	1	0	1	3
15	failure, rebuilt locally	0	0	2	0	1	3
16	depot repair completes, ships	0	0	2	0	0	2

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
17	failure **backordered**, request backordered	0	1	3	0	1	3
18	shipment arrives, clears the crew	0	0	2	0	1	3
19	depot repair completes, ships	0	0	2	0	0	2
19	local rebuild returns	1	0	1	0	0	2

Week	Event	I_j	B_j	X_j	I_0	B_0	X_0
20	depot repair completes	1	0	1	1	0	1
21	shipment arrives	2	0	0	1	0	1
25	depot repair completes	2	0	0	2	0	0

9.5.3. What the Ledger Establishes

The identity held on every row, at both levels. Read down X_j against $2 - I_j + B_j$, and X_0 against $2 - I_0 + B_0$. Forty checks, forty agreements. That is Equation 9.11 doing the work Equation 9.5 promised, and it is the only thing standing between a hand ledger and an undetected arithmetic error.

The storeroom was short once, and it was not its own doing. Run the ledger again with a depot that never makes anyone wait, so that every request ships the instant it is made. The week-12 request then arrives in week 14 instead of week 18, the shelf holds two transformers from week 14 and one from week 15, and the week-17 failure is met from stock. **The storeroom's only backorder disappears.** Its failures did not change,

its own shop did not change, and its stock level was never touched. The backorder was manufactured at the depot and delivered.

9.5.3.1. Averaging a Column That Is a Function of Time

Now measure the wait, and here the callout of Section 9.5.1 comes due.

$B_0(t)$ is a step function. It is 0 from week 0 to week 12, 1 from week 12 to week 16, 0 from week 16 to week 17, 1 from week 17 to week 19, and 0 from week 19 to the end. **Its average is an average over time, weighted by how long it held each value**, and not the average of the numbers appearing in the B_0 column. Those numbers are samples at the instants the process happened to change, and there are more of them in a busy fortnight than in a quiet month.

Weighting by duration,

$$\bar{B}_0 = \frac{1(16-12)+1(19-17)}{26} = \frac{4+2}{26} = \frac{6}{26} = 0.2308 \text{ units}$$

The numerator deserves a name. Six **backorder-weeks** were accumulated, in the sense that one unit was owed for four weeks and then one unit for two more.

Now we compute the wait a second way, by following the requests. Five requests reached the depot. Three were filled on arrival and waited nothing. The week-12 request waited until week 16, which is four weeks, and the week-17 request waited until week 19, which is two. Thus,

$$\bar{W}_j = \frac{0+0+0+4+2}{5} = \frac{6}{5} = 1.20 \text{ weeks}$$

Now we divide the time average by the arrival rate. The depot received five requests in twenty-six weeks, so $\lambda_0 = 5/26$ per week, and

$$\frac{\bar{B}_0}{\lambda_0} = \frac{6/26}{5/26} = \frac{6}{5} = 1.20 \text{ weeks}$$

The two agree, and the reason they agree is visible in the arithmetic. Both are the same six backorder-weeks divided by the same five requests. One route reached it by sweeping along the time axis and weighting each level by its duration; the other reached it by sweeping across the requests and adding each one's wait. The six is an area either way, counted in rows or in columns.

That is Little's Law, in the one setting where you can see the whole of it at once. Section 9.6 states it in general and uses it on quantities no ledger will ever produce, but it is the same identity, and nothing in the general statement is doing anything this ledger has not already done by hand.

> ⚠ **Had we averaged the B_0 column instead of the time axis, the answer would have been wrong**, and it would have been wrong in a way that is hard to notice. The column holds twenty entries, of which five are 1, giving 0.25 rather than 0.2308. That number is not an average of anything the system did. It is an average over the events the ledger happened to record, and a ledger with more rows in the quiet stretches would have produced a different one. Whenever a quantity carries (t), integrate it; do not average the rows.

9.5.4. Where This Leaves Us

The ledger has produced the missing quantity, but only for twenty-six weeks of one realization. We cannot run a ledger for every item at every storeroom, and we cannot run one at all for a system that has not been built yet.

Section 9.6 replaces it with the relation the two computations above just agreed on, which the book has already proved once and which holds for any arrival process at all.

9.6. The Delay the Depot Imposes

Section 9.5 produced $\bar{W}_j$ for twenty-six weeks of one realization by counting request-weeks on paper. This section obtains it for any system, from a formula, and the formula is one the book has already proved.

9.6.1. The Depot Is a Single Location

Before computing the wait, notice something about the depot that Figure 9.1 has been showing all along.

The depot's demand is Poisson. By Equation 9.1 each storeroom's requests are its own failure stream thinned by local repair, and a thinned Poisson process is Poisson. The depot sees the superposition of J such streams, and by Section B.3.2 a superposition of independent Poisson processes is Poisson at the sum of the rates. Thus the depot faces Poisson demand at the rate λ_0 of Equation 9.2.

The depot replenishes one for one. Every request arrives with a carcass, and the carcass goes on the bench the moment it arrives. That is a resupply activity started by a demand, one for one, as at a storeroom.

The depot waits for nobody. Its supplier is its own repair shop, which by the third condition of Section 9.3.2 has capacity to spare. There is no further echelon behind it and no wait to look up.

Thus Theorem 9.1 applies to the depot with no qualification whatever, and the number of units in resupply at the depot is Poisson with mean

$$\mu_0 = \lambda_0 T_0 \tag{9.12}$$

Everything in Section 9.4 now applies to the depot unchanged. Its expected backorders are $\bar{B}_0(S_0) = G^1(S_0)$ by Equation 9.6, evaluated on the Poisson distribution with mean μ_0.

> **Notice what is absent from Equation 9.12.** No S_j, no O_j, no T_j, no ϕ_j except through λ_0. The depot's own performance does not depend on how much stock the storerooms hold, or how fast they ship, or how well they are doing. Thus, **the depot can be solved first, once, and on its own**, and the storerooms then follow from it. There is no circle to iterate around.
> That is a property of one-for-one ordering, and batching would lose it: the stream a location passes up would then depend on the order quantity it chose, which depends in turn on the lead time the supplier gives it, and a fixed point would have to be found.

9.6.2. From Backorders to a Wait

Recall from Section B.4.3 that Equation B.24 relates the average number in a system, the arrival rate into it, and the average time an arrival spends there, and that it assumes almost nothing.

We take the system to be the set of requests the depot has accepted but not yet filled. Arrivals into that system are requests the depot cannot fill on the spot, departures are the shipments that clear them, the average number in it is the depot's average backorder level $\bar{B}_0$, and the rate at which requests present themselves is λ_0. Thus

$$\bar{W} = \frac{\bar{B}_0(S_0)}{\lambda_0} \tag{9.13}$$

This is the relation the ledger verified. Section 9.5.3.1 computed six backorder-weeks over twenty-six weeks against five requests and got 1.20 weeks twice, once by following requests and once by dividing a time average by a rate. Equation 9.13 is that second route, written for a system rather than for one realization.

The wait does not carry a storeroom subscript. The depot clears its backorders in the order they arrived, without regard to which storeroom raised them, so a request from a large storeroom waits no longer than one from a small storeroom. Thus $\bar{W}_j = \bar{W}$ for every j, and Equation 9.4 becomes

$$\mu_j = \phi_j \lambda_j T_j + (1 - \phi_j) \lambda_j \left(O_j + \bar{W} \right) \tag{9.14}$$

with a single $\bar{W}$ shared by all of them. The storerooms still differ, through ϕ_j, T_j, O_j and λ_j; what they do not differ in is the delay the depot imposes.

> ⚠ Equation 9.13 gives the average wait over **all** requests, including the ones filled immediately, which wait nothing. It is not the average wait of the requests that were delayed. In Section 9.5 two requests of five waited, for four weeks and two weeks, so the delayed requests averaged three weeks while Equation 9.13 reports 1.20. Both numbers are correct and they answer different questions. Equation 9.14 wants the first, because it is multiplying by the rate of all requests.

9.6.3. What a Unit at the Depot Buys

Recall from Equation 9.10 that one more unit at a location reduces its expected backorders by G^{0} evaluated at the current level. We apply that to

the depot and divide by λ_0: the reduction in the wait from the (S_0+1)st unit is

$$\bar{W}(S_0) - \bar{W}(S_0+1) = \frac{G^0(S_0)}{\lambda_0} \tag{9.15}$$

Thus, the delay falls with depot stock, and falls by less with every unit added. That is Section 9.4.3 read at the upper level, and it means the depot's stock level faces the same diminishing returns as a storeroom's.

It is not the same decision, however. A unit added at a storeroom helps that storeroom. A unit added at the depot shortens $\bar{W}$, which by Equation 9.14 shrinks the pipeline at **every storeroom at once**. That asymmetry is what makes multi-echelon stocking a subject of its own, and Section 9.9 is where the two kinds of unit are made to compete on even terms.

Example 9.3 (What the depot's stock does to four storerooms)**.** The utility of Example 9.1 supports four storerooms alike, each seeing $\lambda_j = 45$ failures a year and rebuilding $\phi_j = 0.3$ of them locally. The central depot repairs what it receives in $T_0 = 60$ days on average.

The depot's demand. By Equation 9.1 each storeroom sends up $0.7(45) = 31.5$ requests a year, and by Equation 9.2 the depot sees $\lambda_0 = 4(31.5) = 126$ requests a year.

The depot's pipeline. By Equation 9.12,

$$\mu_0 = 126\left(\frac{60}{365}\right) = 20.7123 \text{ units}$$

so the depot has about twenty-one transformers on benches at any moment. Notice that this is an order of magnitude larger than the 1.97 units in a storeroom's pipeline, which is what a central location looks like.

The delay, against depot stock. Equation 9.6 and Equation 9.13 applied to the Poisson distribution with mean 20.7123 give Table 9.3.

Table 9.3.: The depot's stock level, the delay it imposes, and what that delay does at one storeroom holding four transformers.

S_0	$\bar{B}_0$	$\bar{W}$ (days)	μ_j	$\bar{B}_j$ at $S_j = 4$
16	5.0201	14.54	3.2276	0.4055
18	3.4443	9.98	2.8337	0.2638
20	2.1763	6.30	2.5167	0.1748
22	1.2550	3.64	2.2863	0.1238
24	0.6565	1.90	2.1367	0.0964
26	0.3105	0.90	2.0502	0.0825

Reading the table across. At $S_0 = 16$ the depot makes a request wait 14.54 days on average, which is close to the fifteen days Example 9.1 supposed, so that illustration was a depot holding about sixteen units. Feeding 14.54 days into Equation 9.14,

$$\mu_j = 1.1096 + 31.5\left(\frac{10 + 14.54}{365}\right) = 1.1096 + 2.1180 = 3.2276$$

The check. Equation 9.15 says the seventeenth depot unit should shorten the wait by $G^0(16)/\lambda_0 = 0.8217/126$ years, which is 2.38 days. Reading the table, 14.54 at $S_0 = 16$ against 12.16 at $S_0 = 17$, not shown but computed the same way, is a drop of 2.38 days. The marginal relation and the table agree, as they must, and that check is worth running because it confirms the loss function and the distribution function were evaluated at the same index.

Interpretation. Six extra transformers at the depot, from sixteen to twenty-two, cut the delay from 14.5 days to 3.6 and cut a storeroom's pipeline from 3.23 units to 2.29. **Those six units did that at all four storerooms simultaneously**, so the backorders in the last column fall by about seventy percent at each of the four, for one purchase of six. Eight units bought at the storerooms, two apiece, would have done something comparable by

forward stocking alone, and six against eight is the pooling comparison that Section 9.9 makes exactly.

Where this stops being obvious. The same six units are the most expensive six in the table by the measure that matters, because they are the ones that sit at the depot doing nothing for anyone until a request arrives. Whether they belong there or at the storerooms is the stock positioning question of Section 9.1, and nothing so far decides it. Section 9.9 does, by making every candidate unit at every location compete on backorders removed per dollar.

9.6.4. Where This Leaves Us

Every quantity in Equation 9.14 is now available. Equation 9.12 gives the depot's pipeline from rates and a repair time, Section 9.4 turns that into $\bar{B}_0(S_0)$, Equation 9.13 turns that into $\bar{W}$, and Equation 9.14 turns that into each storeroom's pipeline.

Section 9.7 collects those four steps into a procedure, runs it on a system, and then does the thing this chapter has been postponing since Section 9.3.3: it examines the assumption the procedure rests on, and finds it wanting.

9.7. METRIC

Section 9.6 left four steps standing in a row. This section gives them a name, runs them, and then asks what they are worth.

The name is **METRIC**, the Multi-Echelon Technique for Recoverable Item Control, and it is due to Sherbrooke (1968). It is the earliest of the repairable-item models still in use, it is the ancestor of every model in this chapter, and Sherbrooke (2004) is the book-length treatment.

9.7.1. The Procedure

Algorithm 9.1 Algorithm Metric. Step 1 is Equation 9.1 and Equation 9.2, step 2 is Equation 9.12 with Equation 9.6, step 3 is Equation 9.13, and step 4 is Equation 9.14 with Equation 9.6.

```
metric(S_0, S_1 ... S_J):

    // Step 1, the demand the depot faces
    for each storeroom j:
        lambda_j0 <- (1 - phi_j) * lambda_j
    lambda_0 <- sum over j of lambda_j0

    // Step 2, the depot's pipeline, then its backorders
    mu_0   <- lambda_0 * T_0
    Bbar_0 <- G1(S_0), on the Poisson distribution with mean mu_0

    // Step 3, the delay the depot imposes on everybody
    Wbar <- Bbar_0 / lambda_0

    // Step 4, each storeroom in turn
    for each storeroom j:
        mu_j   <- phi_j*lambda_j*T_j + (1 - phi_j)*lambda_j*(O_j + Wbar)
        Bbar_j <- G1(S_j), on the Poisson distribution with mean mu_j

    return Bbar_0, Bbar_1 ... Bbar_J
```

Notice that Algorithm 9.1 has no loop in it. It is a straight line from rates to answers, and it runs in time proportional to the number of storerooms. That is the dividend of Section 9.6.1: the depot's own performance does

not depend on the storerooms, so step 2 never has to be revisited once step 4 has run. Nothing here is iterated to a fixed point, nothing is solved simultaneously, and there is no convergence to worry about.

Notice also what Algorithm 9.1 does not do. It does not choose S_0 or any S_j. It **evaluates** a stocking plan handed to it, which is why its arguments are stock levels and its returns are backorders. Section 9.9 supplies the search, and it will call Algorithm 9.1 many times.

Example 9.4 (METRIC on the utility's transformers)**.** Take the system of Example 9.3 entire. Four storerooms, each seeing $\lambda_j = 45$ failures a year and rebuilding $\phi_j = 0.3$ of them in $T_j = 30$ days, an order and ship time of $O_j = 10$ days, and a depot repairing what it receives in $T_0 = 60$ days. The utility proposes to hold $S_0 = 16$ transformers at the depot and $S_j = 4$ at each storeroom, twenty units in all.

What is stocked? One item at five locations, the pad-mount transformer.

What is the demand process? Poisson at 45 failures a year at each storeroom, one unit at a time.

When is inventory reviewed? Continuously, at every location.

What triggers replenishment, and how much? Every failure, for one unit, at both levels.

What happens to unmet demand? It is backordered, at the storeroom against a waiting crew and at the depot against a waiting storeroom.

What costs are incurred, and when? None are needed. Algorithm 9.1 returns backorders.

Step 1. Each storeroom passes up $0.7(45) = 31.5$ requests a year, so $\lambda_0 = 4(31.5) = 126$ a year.

Step 2. The depot's pipeline is $\mu_0 = 126(60/365) = 20.7123$ units, and the first order loss function of the Poisson distribution with that mean, evaluated at $S_0 = 16$, gives $\bar{B}_0 = 5.0201$ units.

Step 3. The delay is $\bar{W} = 5.0201/126 = 0.03984$ years, which is 14.54 days.

Step 4. Each storeroom's pipeline is

$$\mu_j = (0.3)(45)\left(\frac{30}{365}\right) + (0.7)(45)\left(\frac{10 + 14.54}{365}\right) = 1.1096 + 2.1180 = 3.2276 \text{ units}$$

and the loss function at $S_j = 4$ gives $\bar{B}_j = 0.4055$ backorders. Across four storerooms that is $4(0.4055) = 1.6218$ transformers owed to waiting crews at any moment (the product taken before rounding).

The check. Run Equation 9.7 on a storeroom: $\bar{I}_j = 4 - 3.2276 + 0.4055 = 1.1779$, so $\bar{I}_j - \bar{B}_j = 0.7724$ and $S_j - \mu_j = 4 - 3.2276 = 0.7724$. The identity holds, and it is worth running at both levels every time.

Interpretation. Twenty transformers, of which sixteen sit at a depot where no crew can reach them, leave about 1.6 crews waiting at any moment across the utility. Whether that is acceptable, and whether the twenty are in the right places, are the questions Section 9.9 answers. What we have is a number attached to a plan, which is what was missing.

9.7.2. What METRIC Approximates

Section 9.3.3 warned that Equation 9.4 applies Palm's theorem outside its warrant. That debt now comes due, and paying it turns out to be the most useful thing in the chapter, because the model fails in a specific and repairable way that has been understood since the model was published.

Every mean METRIC computes is exact, and that deserves establishing before anything is criticized. The depot's pipeline really is Poisson with mean $\lambda_0 T_0$, because Theorem 9.1 applies there without qualification. Thus $\bar{B}_0$ is exact, and $\bar{W}$ is exact because Equation B.24 assumes nothing. The storeroom's pipeline mean is exact too, for the same reason: Little's Law applied to the storeroom's own resupply says $E[X_j] = \lambda_j E[\text{resupply duration}]$, and

averaging that duration over the two routes gives Equation 9.14 with no independence assumed anywhere.

What is not exact is the claim that X_j is Poisson. A Poisson distribution has its variance equal to its mean, and it is here that the model asserts something it has not earned. Recall the reason from Section 9.3.2: Palm's theorem wants each unit's resupply duration drawn independently. The depot-routed durations are not independent. Two requests arriving while the depot is empty both wait for the same repairs to finish, so their delays move together. That is, the count is more variable than a Poisson count with the same mean.

None of this is new. It was known to the model's author at the time, and Sherbrooke (1968) says so plainly in the paper that introduced it. Looking back on it eighteen years later, Sherbrooke (1986) writes that "when the METRIC model was developed, it was clear that it understated base backorders. In most cases the error was not large, and the simplicity of METRIC seemed to overshadow the lack of precision."

The direction of the error is the unsafe one, and the reason is precise. Backorders are a tail quantity, so a distribution that is too tight produces too few of them. Sherbrooke (1986) states the consequence for the whole family of models built this way: they "understate the multi-echelon delay in the resupply of a base from a depot that has backorders", and thus "tend to understate expected backorders and overstate expected availability of repair items". A utility sizing its spares with Algorithm 9.1 will believe it is doing better than it is.

How much does it matter? Graves (1985) answers this by comparing the stock levels the two models choose rather than the backorders they report, which is the comparison a planner actually cares about. Sherbrooke (1986) reports the result: "Graves shows that in 11% of cases, the METRIC stock levels differ by at least one unit from the optimal results; the VARI-METRIC levels differ in only 1%." Notice that he immediately qualifies it, and the qualification is the more useful half: "the 10-fold improvement of VARI-METRIC is probably more meaningful than the actual percentages,

which depend on the number of bases, mean demands over the repair time, and other input values."

Thus, the size of the error is a property of METRIC applied to a particular system rather than of METRIC itself, and a system with more storerooms, or a leaner depot, will show more of it.

> ⚠ **The error is not in the mean, so it cannot be fixed by adjusting one.** A common instinct on meeting a gap of this kind is to inflate a lead time or pad a rate until the answer matches. Do not. The means above are already right; it is the second moment that is missing, and a fudge to the first moment will break the agreement that does exist while only approximately repairing the one that does not. It will also fail on the next item, for the reason Sherbrooke (1986) gives: the size of the gap depends on the system.

9.7.3. Where This Leaves Us

We have a procedure, it is fast, it has no loop, and every mean it produces is exact. What it does not produce is a variance, and that omission makes it optimistic about backorders by an amount that depends on the system.

The diagnosis names the cure, and Sherbrooke (1986) puts it in one sentence: "METRIC is a first-order model using mean values only, whereas VARI-METRIC is a second-order model incorporating variances as well." If the pipeline's variance is the thing METRIC is missing, then compute the variance and use a distribution that can hold a variance larger than its mean. Section 9.8 does that, following the derivation Graves (1985) published for the purpose.

9.8. VARI-METRIC

Section 9.7.2 left the model with exact means and a distribution that is too tight. The repair is to compute the variance the model was throwing away and to use a distribution that can hold it.

That repair is **VARI-METRIC**. It is due to Slay (1980), Graves (1985) published the short derivation this section follows, and Sherbrooke (1986) extended it and is the source for the numbered results quoted below.

9.8.1. Splitting the Depot's Backorders

The whole method turns on one question. The depot owes B_0 units at a random instant. **How many of them are owed to storeroom j?**

Recall from Section 9.6.2 that the depot fills requests in the order they arrive, without regard to who sent them. Thus, each outstanding backorder came from storeroom j with probability equal to storeroom j's share of the depot's demand, and the backorders are labeled independently of one another. Let

$$f_j = \frac{\lambda_{j0}}{\lambda_0} = \frac{(1-\phi_j)\lambda_j}{\sum_{k=1}^{J}(1-\phi_k)\lambda_k} \tag{9.16}$$

represent that share. Then, conditional on the depot owing B_0 units in total, the number owed to storeroom j is binomial with B_0 trials and success probability f_j.

This is a proved result rather than an assumption made for convenience. Sherbrooke (1986) records that "Simon (1971) and Graves have shown that the number of units in resupply at base I, conditional on the total number of units in resupply at all bases, has a binomial distribution when the original demand process is Poisson."

We write B_{0j} for the units the depot owes storeroom j. The binomial gives $E[B_{0j} \mid B_0] = f_j B_0$ and $Var[B_{0j} \mid B_0] = f_j(1-f_j)B_0$, and applying Equation B.8 we obtain

$$E[B_{0j}] = f_j \bar{B}_0, \qquad Var[B_{0j}] = f_j(1-f_j)\bar{B}_0 + f_j^2\, Var[B_0] \tag{9.17}$$

Read the two terms of that variance separately, because they are different things. The first is the randomness in *which* storeroom each backorder belongs to, given how many there are. The second is the randomness in *how many there are*, scaled down by f_j^2 because storeroom j receives only its share of any fluctuation. Notice that METRIC keeps neither.

9.8.2. The Pipeline's Two Moments

The storeroom's pipeline splits into two independent pieces. The units in local repair and the units in transit from the depot have independent durations and are thinnings of a Poisson process, so Theorem 9.1 applies to them exactly and their count is Poisson with mean equal to its variance. The units the depot owes but has not shipped are B_{0j}, and Equation 9.17 has just supplied its moments. Adding the two pieces, we obtain

$$\mu_j = \lambda_j \left[\phi_j T_j + (1-\phi_j)O_j\right] + f_j \bar{B}_0 \tag{9.18}$$

$$\sigma_j^2 = \lambda_j \left[\phi_j T_j + (1-\phi_j)O_j\right] + f_j(1-f_j)\bar{B}_0 + f_j^2\, Var[B_0] \tag{9.19}$$

Equation 9.18 is Equation 9.14 written differently. Recall that $\bar{W} = \bar{B}_0/\lambda_0$, so $f_j \bar{B}_0 = (\lambda_{j0}/\lambda_0)\bar{B}_0 = \lambda_{j0}\bar{W} = (1-\phi_j)\lambda_j \bar{W}$, which is the term METRIC already had. Thus VARI-METRIC changes no mean anywhere, and that is the reason the two models can be compared at all: they are fitted to the same first moment and differ only in what they do with the second.

Equation 9.19 is Equation 9.18 plus two terms. If we set those two terms to zero we get $\sigma_j^2 = \mu_j$, which is the Poisson assumption. Thus, METRIC is this model with the last two terms discarded rather than a different model, which is what Sherbrooke (1986) means by calling it "a first-order model using mean values only".

9.8.3. The Depot's Backorder Variance

Equation 9.19 needs $Var[B_0]$, which needs the second moment of the backorder function. We define $E[B^2(s)] = \sum_{x>s}(x-s)^2 g(x)$, the second moment of the shortfall at stock level s. Sherbrooke (1986) gives a recursion for it, which he attributes to Michael Konvalinka of the Logistics Management Institute:

$$E[B^2(s)] = E[B^2(s-1)] - E[B(s)] - E[B(s-1)], \qquad s \geq 1 \tag{9.20}$$

started from $E[B^2(0)] = E[X^2]$, which for the Poisson pipeline of the depot is $\mu_0 + \mu_0^2$. Then

$$Var[B_0] = E[B^2(S_0)] - \bar{B}_0^2 \tag{9.21}$$

Equation 9.20 is preferable to summing the series. It walks up from $s = 0$ one stock level at a time using only quantities the first order loss function has already produced, so the whole table of $E[B^2(s)]$ costs one pass. Notice that it subtracts, so it loses precision at large s where $E[B^2]$ is small; if you need it far out in the tail, sum directly and compare.

9.8.4. Fitting a Distribution to Two Moments

We now have μ_j and σ_j^2 with $\sigma_j^2 > \mu_j$, and we need a distribution. Recall from Section C.1.2 that the negative binomial is "the family to reach for when $VMR > 1$", with $VMR = 1/p$ for every admissible p. Matching Equation C.3 to our two moments,

$$p = \frac{\mu_j}{\sigma_j^2}, \qquad \beta = \frac{1-p}{p}, \qquad r = \frac{\mu_j}{\beta} \tag{9.22}$$

and X_j is taken to have that negative binomial distribution. Every measure of Section 9.4 then applies unchanged, evaluated on this distribution instead of a Poisson.

> **Why this family and not another with the right two moments?** Section C.1.2 gives the reading that answers it: a negative binomial is a Poisson whose rate is itself random, drawn from a gamma distribution. That is the situation here. Each storeroom's pipeline *would* be Poisson if the depot delay were a fixed quantity, and what spoils it is that the delay varies. Notice that this is not an analogy bolted on afterwards. It is the title of the paper that introduced the method: Slay (1980) is "An Approach to Modeling Multi-Echelon Resupply when the Demand Process is Poisson with a Gamma Prior".
> Sherbrooke (1986) adds the practical reasons, that the negative binomial "has a variance that is never less than its mean, is discrete and is easy to compute recursively".

A ratio near one is still fitted with a negative binomial. Equation 9.22 divides by $1 - p$, and $p \to 1$ as the variance approaches the mean, so a ratio of 1.001 produces a number of successes in the thousands. The VARI-METRIC code that the software of Section 9.11 was ported from therefore used a Poisson for any ratio between 0.9 and 1.1, on the grounds that the negative binomial is a Poisson in everything but arithmetic there. That holds at 1.001. It fails well before 1.1.

The ratio comes close to one at two quite different points: a depot so generous that it never delays anybody, and a depot so bare that its delay swamps everything else. For example, take Example 9.3 at $S_0 = 24$, where the ratio is 1.0641. A Poisson gives $\bar{B}_j = 0.0964$ at $S_j = 4$, which is METRIC's figure in Table 9.3, and the negative binomial gives 0.1095, fourteen percent more. A discrete-event simulation of the system, 30 replications of 1,000 years, gives 0.1101 ± 0.0007. Thus a variance only six percent above the mean still matters, because a storeroom's stock level sits out in the tail, where the two distributions differ most.

A band does a second kind of damage. Backorders jump where the ratio crosses its edge, and the allocation search of Section 9.9 finds the jump and buys to it. The software therefore fits the negative binomial whenever the ratio exceeds one, uses a Poisson only when the ratio is one to within rounding, and fits a gamma below one, a case that, as the warning below records, nobody has ruled out. Near one the negative binomial already agrees with the Poisson, so nothing is lost by fitting it there.

⚠ **One step here rests on evidence rather than proof, and the authors say so.** The fit needs $\sigma_j^2 > \mu_j$, or Equation 9.22 returns nonsense. Sherbrooke (1986) reports that "extensive calculations by us (and Graves) indicate that if x, the number of units in resupply/repair, is Poisson, then the function $Var[B(s)]/E[B(s)]$ is unimodal and greater than one, except for $s = 0$", and then states plainly: "neither Graves or we have been able to supply a proof."

Thus the method is known to work and is not known to be safe. In practice you should test the ratio before fitting rather than assume it: at one, which the authors' own statement says happens at $s = 0$, the Poisson of Section 9.7 is the fit, and below one a gamma carries the two moments.

9.8.5. The Procedure

Algorithm 9.2 Algorithm Varimetric. Step 3a is Equation 9.20 and Equation 9.21, step 4 is Equation 9.16, Equation 9.18, Equation 9.19 and Equation 9.22.

```
varimetric(S_0, S_1 ... S_J):

    // Steps 1 to 3 are METRIC, unchanged
    lambda_0 <- sum over j of (1 - phi_j)*lambda_j
    mu_0     <- lambda_0 * T_0
    Bbar_0   <- G1(S_0), on the Poisson distribution with mean mu_0

    // Step 3a, NEW: the depot's backorder variance
    EB2    <- mu_0 + mu_0^2                       // s = 0
    for s = 1 to S_0:
        EB2 <- EB2 - G1(s) - G1(s-1)              // recursion above
    VarB_0 <- EB2 - Bbar_0^2

    // Step 4, each storeroom, now to two moments
    for each storeroom j:
        f_j      <- (1 - phi_j)*lambda_j / lambda_0
        base     <- lambda_j*( phi_j*T_j + (1 - phi_j)*O_j )
        mu_j     <- base + f_j*Bbar_0
        sigma2_j <- base + f_j*(1 - f_j)*Bbar_0 + f_j^2*VarB_0

        c <- sigma2_j / mu_j
        if c > 1:
            fit negative binomial to (mu_j, sigma2_j)
        else if c = 1:
            use Poisson with mean mu_j             // to within rounding
        else:
            fit gamma to (mu_j, sigma2_j)          // under-
dispersed
        Bbar_j <- G1(S_j), on that distribution

    return Bbar_0, Bbar_1 ... Bbar_J
```

Notice how little changed. Algorithm 9.2 is Algorithm 9.1 with one extra loop over the depot's stock levels and two extra terms in the storeroom calculation. It still has no fixed point to find, it still runs in one pass, and it still evaluates a plan rather than choosing one. Sherbrooke (1986) makes the same point about cost: "the VARI-METRIC computation is only slightly more complicated than that of METRIC, since it requires an estimate of the variance as well of the mean of the backorders at each stage and the use of a negative binomial distribution rather than a Poisson."

Example 9.5 (VARI-METRIC on the utility's transformers)**.** Take the system of Example 9.4 unchanged: four storerooms at $\lambda_j = 45$ failures a year, $\phi_j = 0.3$ rebuilt locally in $T_j = 30$ days, $O_j = 10$ days to ship, a depot repairing in $T_0 = 60$ days, and the proposed plan $S_0 = 16$ with $S_j = 4$ at each storeroom.

The modeling questions have the same answers as in Example 9.4, since the system has not changed. Only the model has.

Steps 1 to 3, unchanged. $\lambda_0 = 126$ a year, $\mu_0 = 20.7123$ units, and $\bar{B}_0 = 5.0201$ units at $S_0 = 16$.

Step 3a, the depot's backorder variance. Starting from $E[B^2(0)] = 20.7123 + 20.7123^2 = 449.7129$ and running Equation 9.20 up to $s = 16$ gives $E[B^2(16)] = 41.8229$. Thus by Equation 9.21,

$$Var[B_0] = 41.8229 - (5.0201)^2 = 41.8229 - 25.2015 = 16.6215$$

Notice that the depot's backorders are far more variable than their mean, a variance-to-mean ratio of $16.6215/5.0201 = 3.31$. That is the quantity METRIC discards, and it is the reason discarding it matters.

Step 4, the storerooms. All four are alike, so $f_j = 31.5/126 = 0.25$. The Poisson part is

$$\lambda_j \left[\phi_j T_j + (1-\phi_j) O_j\right] = 45 \left[\frac{(0.3)(30) + (0.7)(10)}{365}\right] = 45 \left(\frac{16}{365}\right) = 1.9726$$

which is the pipeline of Example 9.1 when the depot never delayed, as it should be. Then Equation 9.18 gives

$$\mu_j = 1.9726 + (0.25)(5.0201) = 1.9726 + 1.2550 = 3.2276 \text{ units}$$

the same figure Example 9.4 reached, and Equation 9.19 gives

$$\sigma_j^2 = 1.9726 + (0.25)(0.75)(5.0201) + (0.25)^2(16.6215) = 1.9726 + 0.9413 + 1.0388 = 3.9527$$

The fit. By Equation 9.22, $p = 3.2276/3.9527 = 0.8166$, so $\beta = 0.1834/0.8166 = 0.2246$ and $r = 3.2276/0.2246 = 14.3674$. The variance-to-mean ratio is $1/p = 1.2246$.

The answer. The first order loss function of that negative binomial at $S_j = 4$ gives $\bar{B}_j = 0.4815$ backorders, against METRIC's 0.4055. Across four storerooms, 1.9259 transformers owed to waiting crews rather than 1.6218.

The check. Equation 9.7 holds whatever the distribution, since it is an identity: $\bar{I}_j = 4 - 3.2276 + 0.4815 = 1.2539$, and $\bar{I}_j - \bar{B}_j = 1.2539 - 0.4815 = 0.7724$, which is $S_j - \mu_j = 4 - 3.2276$. Run it on both models, because it is the one relation that must agree between them, the means being shared.

Interpretation. VARI-METRIC reports about **19 percent more backorders** than METRIC on the same plan, and the whole of that difference came from the two terms in Equation 9.19. Notice that the plan did not change and the system did not change. What changed is that the model now admits it does not know how long a request will wait, only what that wait averages.

The direction is not accidental. Sherbrooke (1986) states that VARI-METRIC "produces an estimate of backorders that exceeds that of METRIC in all cases except when stock levels are zero (when the two models agree)". Thus, wherever the two differ, VARI-METRIC is the pessimistic one, and the extra backorders it reports are ones METRIC was concealing rather than ones VARI-METRIC invented.

9.8.6. Where This Leaves Us

We can evaluate a stocking plan across two echelons, to two moments, in one pass, and we know from Section 9.7.2 roughly what that is worth against choosing the plan by METRIC instead.

What we still cannot do is choose the plan. Every example in this chapter has been handed S_0 and S_j from outside, and the interesting question has been sitting underneath them since Section 9.1: with a fixed sum to spend, how many units go to the depot and how many to each storeroom? Section 9.9 answers it, using the marginal result Equation 9.10 established for exactly this purpose.

9.9. Spending a Budget Across Items

Every example in this chapter has been handed its stock levels. Algorithm 9.1 and Algorithm 9.2 both take S_0 and the S_j as arguments and return backorders, which means they evaluate a plan rather than produce one. This section produces one.

9.9.1. What Is Being Chosen, and Against What

The decision variables are a stock level at the depot and one at each storeroom, **for every item in the range**. For example, a utility with

nine repairable items and four storerooms is choosing forty-five numbers, and they are not independent of one another because they compete for the same money.

The objective is the one argued for in Section 9.2.4: minimize the **total expected backorders at the storerooms**,

$$EBO = \sum_{\text{items}} \sum_{j=1}^{J} \bar{B}_j(S_j) \tag{9.23}$$

subject to the total cost of the units bought not exceeding a budget. Let c_j represent the unit cost of the item stocked at storeroom j, and c_0 its unit cost at the depot. Thus, one item's plan costs $c_0 S_0 + \sum_{j=1}^{J} c_j S_j$, and the budget applies to that sum taken over every item in the range.

> **Notice that $\bar{B}_0$ does not appear in Equation 9.23.** A backorder at the depot costs nothing directly, because no crew is waiting at a depot. It costs by lengthening $\bar{W}$, which by Equation 9.14 inflates every storeroom's pipeline, which raises the storeroom backorders that Equation 9.23 does count. Thus, depot stock is valued entirely for what it does four hundred miles away rather than for what it does at the depot, and that is what makes this a multi-echelon optimization rather than five separate ones.

9.9.2. Marginal Analysis

The technique is **marginal analysis**, and Sherbrooke (2004) reports that "while it is likely that the technique has been used for many years, the earliest published reference appears to be O. Gross (1956)" (Gross 1956).

For each item and each location, compute the **delta value**: the reduction in expected backorders that the next unit would buy, divided by what that

unit costs. Recall from Equation 9.10 that the reduction itself is $G^0(S)$, so for an item costing c the delta value at stock level S is

$$\delta(S) = \frac{\bar{B}(S) - \bar{B}(S+1)}{c} = \frac{G^0(S)}{c} \tag{9.24}$$

Then buy repeatedly, at each step taking the candidate with the largest delta value, until the budget runs out. Sherbrooke (2004) calls this "the increase in system effectiveness per dollar", and notes drily that "recent references sometimes refer to the technique as the 'greedy heuristic', hardly an improvement."

The procedure works only if the delta values decline as you buy. Section 9.4.3 established that they do at a single location, because G^0 is a complementary distribution function and therefore nonincreasing. That is the convexity of $\bar{B}(S)$, and Sherbrooke (2004) makes the same argument: "since the expected backorder function is convex, the marginal analysis values are non-increasing", and "it is easy to show that the sum of convex functions is convex."

Were that not so, the procedure would go wrong in a specific way. Sherbrooke states it: "the problem with marginal analysis that looks only one step ahead is that there may be other items whose values are between" the true two-step improvement and the understated one-step improvement, and "those item stock levels will be augmented prematurely."

9.9.3. Two Echelons Break the Convexity

At two echelons it is not so. This is the one place in the chapter where the machinery built so far does not simply extend.

Fix the depot's stock and vary a storeroom's, and everything is convex, for the single-location reason. But the depot's stock is a decision too, and moving it changes $\bar{W}$, which moves every storeroom's pipeline at once.

Sherbrooke (2004) puts it exactly: "each row of Table 3.3 is convex, because the marginal analysis procedure is identical with Chapter 2. But as we move from one level of depot stock to another in the optimal solutions, there is no assurance of convexity."

Table 9.4 shows it happening on the chapter's own item. For each total number of transformers bought, it reports the best split between the depot and the four storerooms, the expected backorders that split produces, and the reduction the last unit bought.

Table 9.4.: The best two-echelon split of a given number of transformers, computed with Algorithm 9.2. The two bold reductions are larger than the reduction above them, so the totals **above** them, 12 and 20, are the ones lying off the convex curve.

Units bought	S_0	Units at storerooms	EBO	Reduction
10	10	0	18.6080	0.9967
11	11	0	17.6154	0.9927
12	12	0	16.6303	0.9850
13	9	4	15.6433	**0.9870**
14	10	4	14.6587	0.9846
15	11	4	13.6817	0.9770
16	12	4	12.7166	0.9651
17	13	4	11.7696	0.9470
18	14	4	10.8490	0.9206
19	15	4	9.9647	0.8842
20	12	8	9.1088	0.8559
21	13	8	8.2462	**0.8627**

Read the Reduction column looking for an increase. The twelfth transformer removes 0.9850 backorders and the thirteenth removes 0.9870, which is more. The same thing happens at the twentieth and twenty-first. A

procedure that buys by comparing one-step improvements will undervalue this item at exactly those two points, and will spend money on something else instead.

Precision about which point is the bad one matters, because it is easy to name the wrong row. The reduction rising at total 13 means that total **12** lies above the straight line joining 11 and 13, so 12 is the plan that is never worth buying. Check it: $(17.6154 + 15.6433)/2 = 16.6294$, which is below the 16.6303 the table reports at 12.

The fix is to refuse to stop there. Sherbrooke (2004): the non-convex points "are easily dealt with by excluding them as potential solutions", and "by dropping the interior points, the marginal analysis will jump to the next convex point at the correct time, buying at least two more units of stock of the item because of the eliminated interior point or points." Applied to Table 9.4, totals of 12 and 20 are struck out, so the procedure moves from 11 units straight to 13 and from 19 straight to 21, buying two at a time and quoting the average of the two reductions as its delta value. This step is called **convexification**.

9.9.4. Flushout

Now read the same table's S_0 column, and notice something that ought to be impossible.

Units bought	10	11	12	**13**	…	19	**20**	21
Best S_0	10	11	12	**9**		15	**12**	13

The optimal depot stock falls as the total rises. For example, going from twelve transformers to thirteen, the depot's holding drops from twelve to nine, and four units appear at the storerooms. The utility bought one more transformer and the right answer was to move three of the ones it

already had out of the depot. It happens again at twenty, where the depot gives up three more.

Sherbrooke (2004) names this: "after three units of stock have been allocated optimally to depot and more stock is available for allocation, it becomes optimal to take some of that depot stock and flush it out to the bases." He calls the phenomenon **flushout**, and notes that it is most common when the bases are alike, which is why the four storerooms here were made identical.

The reason can be worked out rather than accepted. Depot stock is **pooled**, and Section 9.1.2 said pooled stock is efficient. Recall from Equation 9.13 that a unit at the depot buys $G^0(S_0)/\lambda_0$ off the delay, and the delay is divided among four storerooms, so each gets a quarter of it. A unit at a storeroom is not pooled and helps only that storeroom, but it helps immediately and it is on the right shelf. While the depot is nearly empty the pooling dominates, because the delay it removes is enormous. Once the depot is comfortable the delay it still has left to remove is small, and a unit does more good sitting where a crew can reach it.

Thus, **the model answers the stock positioning question of Section 9.1, and answers it as a function of the budget.** Not "should this item be forward stocked?" but "forward stocked at what level of total investment?", which is a better question and is the one a utility can act on. Notice also that the answer is not monotone, so it cannot be read off a rule of thumb of the kind Section 9.1.2 warned about.

9.9.5. Across Items

Everything so far concerned one item. Items are combined by the same rule, with the reductions divided by unit cost as Equation 9.24 requires. Sherbrooke (2004) is explicit that this division is what makes them comparable: "when we combine across items, the backorder reductions (first differences) must be divided by unit cost."

Suppose the utility also stocks a voltage regulator at $9,500, failing 12 times a year at each storeroom, of which it rebuilds a fifth locally in 45 days, with a depot repair time of 90 days and the same ten day shipment from the depot. Table 9.5 follows the merged buy sequence, each row taking whichever item offers the larger delta value.

Table 9.5.: Selected steps of the merged buy sequence for two items, as Algorithm 9.3 reports them. The delta column never rises, which is what convexification bought. Steps 12 and 19 buy two transformers at once, jumping the struck-out totals of Table 9.4.

Step	Bought	δ per $1,000	Cumulative spend	EBO, transformer	EBO, regulator
1	transformer	0.2632	$3,800	27.6027	11.7041
12	transformer, two	0.2595	$49,400	15.6433	11.7041
19	transformer, two	0.2261	$79,800	8.2462	11.7041
28	transformer	0.1148	$114,000	2.5852	11.7041
29	regulator	0.1053	$123,500	2.5852	10.7042
33	regulator	0.1009	$161,500	2.5852	6.7655
34	transformer	0.0988	$165,300	2.2096	6.7655
35	regulator	0.0958	$174,800	2.2096	5.8556

Step	Bought	δ per \$1,000	Cumulative spend	EBO, transformer	EBO, regulator
37	transformer	0.0834	\$188,100	1.8929	5.0230
39	regulator	0.0766	\$201,400	1.5952	4.2949
41	regulator	0.0734	\$214,700	1.3122	3.5973

Read the delta column down. It never rises, which is the whole point of convexification: without it, steps 12 and 19 would have quoted understated deltas and the regulator would have been bought too early.

Read the first twenty-eight steps, too. Not one regulator is purchased until \$114,000 has gone into transformers. That is arithmetic, not a judgment about regulators. The transformer removes about a quarter of a backorder per thousand dollars at the start and the regulator about a tenth, because the transformer fails nearly four times as often and costs less than half as much per unit. Notice that a planner reasoning by unit price alone would have bought regulators first, on the grounds that they are the expensive item and therefore the important one.

9.9.6. The Procedure

Algorithm 9.3 Algorithm Allocate. Step 1 calls Algorithm 9.2, step 2 is the convexification of Section 9.9.3, and step 3 is Equation 9.24. The procedure follows the eight steps Sherbrooke (2004) sets out for METRIC, with Algorithm 9.2 in place of Algorithm 9.1.

```
allocate(budget, items):

    for each item:
        // Step 1, the two-echelon curve for this item alone
        for S_0 = 0, 1, 2, ... up to a sensible bound:
            Wbar <- depot delay at S_0                  // alg-
varimetric steps 1-3a
            for n = 0, 1, 2, ... units across storerooms:
                spread n over the storerooms by marginal analysis
                EBO[S_0][n] <- total storeroom backorders

        // Step 2, best split for each total, then convexify
        for each total T:
            curve[T] <- min over S_0 + n = T of EBO[S_0][n]
        drop every T whose reduction exceeds the reduction before it

        // Step 3, delta values for what survives
        for each surviving step of curve:
            delta <- (EBO drop) / (units bought * unit cost)

    // Step 4, merge the items
    spent <- 0
    while spent + cost of the best next buy <= budget:
        take the item whose next delta is largest
        buy its next step; add its cost to spent

    return the stock levels reached, and the EBO-versus-
cost curve traced
```

What Algorithm 9.3 returns is not one answer but a curve. Every step of the merge is an efficient plan for the money spent up to that point, so the output is the whole schedule of backorders against budget. That is more useful than a single plan, because the budget is usually not fixed until someone has seen what the money buys. A utility that sees Table 9.5 can ask whether the step from $114,000 to $123,500 is worth it, which is a question it can answer and the model cannot.

> ⚠ **The last buy can overshoot the budget badly.** If the final unit selected is an expensive one, the plan costs materially more than the budget allowed. Sherbrooke (2004) gives the remedy in a footnote: take the stock levels reached after the overshoot as **minimum** levels, rerun the procedure, and repeat if necessary. Notice that this is the integrality of the problem rather than a defect in marginal analysis, and it is the same overshoot that made Equation 7.13 a rule about the first level at or above a ratio rather than an equality.

9.9.7. Where This Leaves Us

The chapter is now closed. Section 9.1 asked where stock should sit and argued that the answer would come out of a policy calculation rather than alongside one. Table 9.4 is that answer, arriving as a column of a table whose purpose was to choose stock levels, and behaving in a way no rule of thumb would have produced.

What remains is practice. Section 9.10 builds these calculations in a worksheet, and Section 9.11 says what changes when the same calculations have to run over forty thousand items.

9.10. Building These Models in a Worksheet

The workbook is `Chapter9Models.xlsx`. The sheets of Section 8.16 each held one location, and the difference here is that two locations have to agree with one another. Thus the sheets are ordered the way Section 9.6.1 says the chapter can solve them: the depot first and on its own, then a storeroom that reads from it, then the allocation across both.

Four working sheets, plus a `Notes` sheet that says what the others are for.

9.10.1. The Depot

`Depot` is Section 9.3 and Section 9.6. The rates block at the top turns the item and the network into λ_0 and $\mu_0 = \lambda_0 T_0$, and the block below walks one row per unit of pipeline.

Three of its four columns are the ones Section 8.16.2 already used, and they need no function Excel does not have. G comes from `POISSON.DIST` with the cumulative flag, G^0 is its complement, and G^1 is the recursion of Equation C.13 filled down.

The fourth column is new, and it is the one VARI-METRIC needs. Equation 9.20 is a recursion of the same shape, so it costs one more column:

```
E21:  =$B$13+$B$13^2
E22:  =E21-D22-D21
```

the first cell being $\mu_0 + \mu_0^2$ and every cell after it subtracting the two loss values on its own row and the row above. Notice that this is the whole of the extra work VARI-METRIC asks of a worksheet at the depot. Everything else on the sheet METRIC needed too.

The policy block then reads out of the columns by position, so the depot's stock level is the only thing typed:

Poisson at mu_0. G1 and E[B^2] are recursions; each row reads the row above.

x	G(x)	G0(x)	G1(x)	E[B^2(x)]
0	0.000000	1.000000	20.712329	449.712892
1	0.000000	1.000000	19.712329	409.288234
2	0.000000	1.000000	18.712329	370.863577
3	0.000002	0.999998	17.712329	334.438919
4	0.000009	0.999991	16.712331	300.014259
5	0.000042	0.999958	15.712340	267.589588
6	0.000152	0.999848	14.712382	237.164866
7	0.000481	0.999519	13.712534	208.739950
8	0.001330	0.998670	12.713015	182.314401
9	0.003284	0.996716	11.714345	157.887041
10	0.007333	0.992667	10.717629	135.455067
11	0.014955	0.985045	9.724962	115.012477
12	0.028113	0.971887	8.739917	96.547598
13	0.049075	0.950925	7.768030	80.039651
14	0.080088	0.919912	6.817105	65.454517

Figure 9.2.: The loss block on the `Depot` sheet. One row per unit of pipeline, and every row reads the row above it. Notice that the first order column steps down by G^0 of the row above, while the second moment column steps down by two loss values rather than one. Notice also that G^1 starts at μ_0 and $E[B^2]$ at $\mu_0+\mu_0^2$, which are the two quantities the recursions have to be told.

```
B17:  =INDEX($D$21:$D$81,$B$16+1)
B18:  =INDEX($E$21:$E$81,$B$16+1)-B17^2
```

which are $\bar{B}_0$ and $Var[B_0]$ at S_0, the offset of one turning a stock level into a row.

9.10.2. One Storeroom

`Storeroom` is Section 9.7 and Section 9.8 side by side. It reads $\bar{B}_0$ and $Var[B_0]$ off the `Depot` sheet, forms μ_j and σ_j^2 from Equation 9.18 and Equation 9.19, and fits the negative binomial by Equation 9.22.

METRIC closes in one cell. Recall that the Poisson first order loss function can be written with the distribution function twice over, $G^1(s) = \mu(1 - F(s-1)) - s(1-F(s))$, so no block is needed:

```
B33:  =IF(B32=0,B22,B22*(1-POISSON.DIST(B32-1,B22,TRUE))
              -B32*(1-POISSON.DIST(B32,B22,TRUE)))
```

The item and the network		
failures per storeroom	45.0000	*per year*
storerooms J	4	*locations*
fraction rebuilt locally	0.3000	*phi_j*
local repair time	30.0000	*days*
order and ship time	10.0000	*days*
depot repair time	60.0000	*days*
requests per storeroom	31.5000	*lambda_j0 = (1-phi_j)*lambda_j, equation 9.3*
depot demand rate	126.0000	*lambda_0, equation 9.4*
depot pipeline mu_0	20.7123	*Palm's theorem, equation 9.9*
Depot policy		
depot stock S_0	16	*units*
Bbar_0	5.0201	*units the depot owes, equation 9.11 at S_0*
Var[B_0]	16.6215	*Sherbrooke's recursion, equation 9.21*
Wbar, years	0.0398	*Little's Law, equation 9.12*
Wbar, days	14.5424	

Figure 9.3.: The `Depot` sheet above its loss block. The six blue cells are the only ones typed, and the depot stock level is the only decision among them. Notice that the four figures beneath it are the whole of what a storeroom needs from the depot: at $S_0 = 16$ the sheet reads 5.0201 units owed, a variance of 16.6215, and a delay of 14.54 days, which are the figures of Example 9.3. Notice too that nothing on this sheet mentions a storeroom.

Read from the Depot sheet		
Bbar_0	5.0201	
Var[B_0]	16.6215	
share f_j	0.2500	*equation 9.18, this storeroom's share of depot demand*
The storeroom's pipeline		
local and in transit	1.9726	*the part Palm's theorem covers exactly*
mu_j	3.2276	*equation 9.19; METRIC and VARI-METRIC share it*
sigma^2_j	3.9527	*equation 9.20; METRIC drops the last two terms*
variance to mean	1.2246	
Negative binomial fit, equation 9.23		
p	0.8166	
beta	0.2246	
r	14.3674	*not an integer, which is why the block below is a recursion*

Figure 9.4.: The moments block on the `Storeroom` sheet. Nothing here is typed; the two figures at the top are read across from the `Depot` sheet. Notice that the pipeline splits into the part Palm's theorem covers exactly, 1.9726 units, and the part the depot contributes, and that only the second carries any of the variance beyond its own mean. The variance-to-mean ratio of 1.2246 is the whole of the difference between the two models.

The guard on $S_j = 0$ is needed. `POISSON.DIST` rejects a negative first argument, and $G^1(0)$ is μ_j.

VARI-METRIC does not close in one cell, and the reason matters. The fitted r is 14.3674 and not an integer, and `NEGBINOM.DIST` truncates that argument to a whole number. A sheet that used it would quietly compute the wrong distribution and report a plausible answer. So the sheet does not use it. It builds the mass function from its own recursion instead, which needs nothing but arithmetic:

```
B38:  =$B$27^$B$29
B39:  =B38*($B$29+A39-1)/A39*(1-$B$27)
```

that is, $g(0) = p^r$ and $g(x) = g(x-1)\,\frac{r+x-1}{x}(1-p)$, and then walks G^0 and G^1 up the block as the Poisson block does. The sum of the mass column sits at the bottom and should read 1; that is the check that the block was walked far enough.

The negative binomial, built by its own recursion. No distribution function is used.

x	g(x)	G0(x)	G1(x)
0	0.054388	0.945612	3.227629
1	0.143341	0.802271	2.282017
2	0.202039	0.600232	1.479745
3	0.202203	0.398030	0.879513
4	0.161048	0.236982	0.481483
5	0.108524	0.128457	0.244502
6	0.064260	0.064197	0.116044
7	0.034298	0.029899	0.051847
8	0.016805	0.013095	0.021948
9	0.007661	0.005433	0.008854
10	0.003284	0.002149	0.003420
11	0.001334	0.000815	0.001271
12	0.000517	0.000297	0.000456
13	0.000193	0.000105	0.000158

Figure 9.5.: The negative binomial block on the `Storeroom` sheet. Compare it with Figure 9.2 and notice that only the first two columns differ. The mass column is a recursion in place of `POISSON.DIST`, and the two loss columns are then walked by the same rule at both levels, because the loss functions do not care which family produced the mass.

One cell on the sheet exists only to be looked at:

```
B37:  =B36-B34-(B32-B22)
```

which is Equation 9.7 rearranged. It is an identity, so it reads zero whatever the model, and if it ever does not, the fit or the block is wrong.

Storeroom policy		
storeroom stock S_j	4	*units*
METRIC Bbar_j	0.4055	*Poisson, closed in one cell; variance forced equal to the mean*
VARI-METRIC Bbar_j	0.4815	*read out of the block below, on both moments*
VARI over METRIC	1.1875	
Ibar_j, VARI-METRIC	1.2539	
check, Ibar-Bbar-(S-mu)	0.0000000000	*an identity, so this reads zero whatever the model*

Figure 9.6.: The two models on one screen. Notice that the ratio is 1.1875, so VARI-METRIC reports about nineteen percent more backorders on a plan neither model changed, and that the residual at the bottom reads zero to ten places. The residual is the reason to trust the nineteen percent: both models satisfy the same identity, so the difference between them is in the distribution and nowhere else.

9.10.3. The Allocation

`Allocate` is Section 9.9. Since the storerooms are alike, a plan is a depot level and one common storeroom level, so the sheet can lay the whole search out flat: **one row per total number of units, one column per way of splitting that total**.

Column B is the split that puts everything at the depot, column C gives each storeroom one unit, and so on. Each cell recomputes μ_j for its own depot level by reaching into the `Depot` sheet,

```
B8:   =Depot!$B$6*((Depot!$B$5*(Depot!$B$7*Depot!$B$8+(1-
Depot!$B$7)
        *Depot!$B$9)/365+INDEX(DepotLoss,1)/Depot!$B$6))
```

where `DepotLoss` is a defined name for the G^1 column. Then three cells do the choosing:

```
I8:   =MIN(B8:H8)
J8:   =MATCH(I8,B8:H8,0)-1
K8:   =A8-Depot!$B$6*J8
```

the best expected backorders for that total, the storeroom level that achieved it, and the depot level implied.

total	s_j = 0	s_j = 1	s_j = 2	s_j = 3	s_j = 4	s_j = 5	s_j = 6	best EBO	best s_j	best S_0	reduction	convex?	delta per $1,000
0	28.6027							28.6027	0	0			
1	27.6027							27.6027	0	1	1.0000		0.2632
2	26.6027							26.6027	0	2	1.0000		0.2632
3	25.6027							25.6027	0	3	1.0000		0.2632
4	24.6027	24.6059						24.6027	0	4	1.0000		0.2632
5	23.6028	23.6068						23.6028	0	5	1.0000		0.2632
6	22.6028	22.6079						22.6028	0	6	1.0000		0.2631
7	21.6029	21.6094						21.6029	0	7	0.9998		0.2631
8	20.6034	20.6113	20.6314					20.6034	0	8	0.9995		0.2630
9	19.6048	19.6137	19.6386					19.6048	0	9	0.9987		0.2628
10	18.6080	18.6169	18.6475					18.6080	0	10	0.9967		0.2623
11	17.6154	17.6210	17.6585					17.6154	0	11	0.9927		0.2612
12	16.6303	16.6266	16.6723	16.7372				16.6266	1	8	0.9888	drop	0.2602
13	15.6584	15.6345	15.6893	15.7663				15.6345	1	9	0.9921		0.2611
14	14.7075	14.6462	14.7104	14.8015				14.6462	1	10	0.9883		0.2601
15	13.7876	13.6643	13.7366	13.8437				13.6643	1	11	0.9819		0.2584
16	12.9105	12.6929	12.7692	12.8945	13.0342			12.6929	1	12	0.9714		0.2556
17	12.0889	11.7382	11.8101	11.9555	12.1147			11.7382	1	13	0.9547		0.2512

Figure 9.7.: The `Allocate` sheet. Each row is a total number of units and each of the seven columns is a way of splitting it, with the blank cells marking splits that would need a negative depot stock. **Read the best S_0 column down.** It climbs to 11 and then drops to 8, which is flushout: buying the twelfth transformer makes it right to move three of the eleven already at the depot out to the storerooms. Notice that total 12 is itself flagged "drop", because the reduction on the row beneath it, 0.9921, is the larger.

Flushout is on the sheet, in the best S_0 column, as Section 9.9.4 described it. Notice that it arrives one unit earlier than in Table 9.4, at a total of 12 rather than 13. The sheet runs METRIC, and METRIC understates the

backorders at a storeroom that holds stock, so it makes forward stock look slightly better than it is.

The last three columns are the reduction, a flag, and the delta value. The flag is the one piece of Section 9.9.3 a worksheet can do for you:

```
M9:   =IF(OR(L9="",L10=""),"",IF(L10>L9,"drop",""))
```

Notice which row the flag lands on. A total lies off the convex curve when the reduction on the row *below* it is the larger, so the formula looks forward, not back. Writing it the other way round marks the row after the offending plan, which reads plausibly and strikes out the wrong one. That is the non-convexity of Table 9.4 showing up as a word in a column. The sheet marks them; striking them out and merging across items is left to you, because a worksheet cannot sort a list it is also computing.

9.10.4. The Ledger

`Ledger` is Section 9.5 with the arithmetic wired in. The on-hand and backorder columns at both levels are typed and everything else is a formula, so Equation 9.11 is enforced rather than trusted:

```
E5:   =2-C5+D5
I5:   =E5-(2-C5+D5)
```

the first being the resupply column and the second a residual that reads zero on every row.

The sheet carries one column the chapter's table did not, and it is there for the reason Section 9.5.3.1 labored:

```
K5:   =A6-A5
B27:  =SUMPRODUCT(G5:G23,K5:K23)
```

`K` is how long each event's state was held, and the depot's backorder-weeks are the level column dotted with it. Notice that averaging column `G` instead would have been one shorter formula and the wrong answer. The sheet makes the weighting visible so that the wrong version is harder to write.

week	event	I_j	B_j	X_j	I_0	B_0	X_0	residual j	residual 0	weeks held
0	start	2	0	0	2	0	0	0	0	2
2	failure, request to depot	1	0	1	1	0	1	0	0	2
4	shipment arrives	2	0	0	1	0	1	0	0	1
5	failure, rebuilt locally	1	0	1	1	0	1	0	0	3
8	failure, request to depot	0	0	2	0	0	2	0	0	1
9	local rebuild returns	1	0	1	0	0	2	0	0	1
10	depot repair completes	1	0	1	1	0	1	0	0	0
10	shipment arrives	2	0	0	1	0	1	0	0	1
11	failure, request to depot	1	0	1	0	0	2	0	0	1
12	failure, request backordered	0	0	2	0	1	3	0	0	1
13	shipment arrives	1	0	1	0	1	3	0	0	2
15	failure, rebuilt locally	0	0	2	0	1	3	0	0	1
16	depot repair completes, ships	0	0	2	0	0	2	0	0	1
17	failure backordered, request backordered	0	1	3	0	1	3	0	0	1
18	shipment arrives, clears the crew	0	0	2	0	1	3	0	0	1
19	depot repair completes, ships	0	0	2	0	0	2	0	0	0
19	local rebuild returns	1	0	1	0	0	2	0	0	1
20	depot repair completes	1	0	1	1	0	1	0	0	1
21	shipment arrives	2	0	0	1	0	1	0	0	4
25	depot repair completes	2	0	0	2	0	0	0	0	

Figure 9.8.: The `Ledger` sheet. The blue columns are typed and the rest are formulas, so the two residual columns read zero on all twenty rows. Notice the weeks-held column on the right, and notice that it is not constant: week 10 holds its state for no time at all because a second event follows it, and week 21 holds its state for four weeks. Averaging the B_0 column would weight those two equally, which is the error Section 9.5.3.1 exists to prevent.

9.10.5. Solving a Problem on the Sheets

Example 9.6 (Pricing a repair contract)**.** The depot's repair shop offers the utility a contract. For a fee, it will hold the average turnaround on a pad-mount transformer at 45 days instead of the present 60. Nothing else about the system changes: the same four storerooms, the same 45

failures a year at each, the same three in ten rebuilt locally, the same ten day shipment. The utility currently holds $S_0 = 16$ at the depot and $S_j = 4$ at each storeroom.

What is the contract worth, and how would a utility find out?

The modeling questions have the answers of Example 9.4, since the system is that one. What follows is what you type and what comes back, not a derivation.

Step 1, the baseline. Open `Depot` and confirm cell `B16` holds 16, then open `Storeroom` and confirm `B32` holds 4. Read `B34`: **0.4815** backorders at this storeroom, so $4(0.4815) = 1.9259$ across the four.

Step 2, the contract. On `Depot`, type **45** into `B10`, the depot repair time. One number, on a sheet that never mentions a storeroom. Then read down both sheets:

Cell	What it is	Before	After
`Depot!B13`	μ_0	20.7123	15.5342
`Depot!B17`	$\bar{B}_0$	5.0201	1.3500
`Depot!B20`	$\bar{W}$, days	14.54	3.91
`Storeroom!B22`	μ_j	3.2276	2.3101
`Storeroom!B34`	$\bar{B}_j$	0.4815	0.1523

Across four storerooms the expected backorders fall from 1.9259 to 0.6091, a reduction of **68 percent**. Notice that a quarter cut in the depot's repair time removed two thirds of the waiting, and notice where the gain came from: the delay fell by a factor of nearly four, because Equation 9.13 divides a backorder level that is itself falling steeply.

Step 3, what the same improvement costs in stock. Put **60** back into `B10`. Now buy the reduction instead of contracting for it, by typing larger values into `Depot!B16` and watching `Storeroom!B34`.

Depot!B16	16	18	20	21	22
Depot!B20, days	14.54	9.98	6.30	4.84	3.64
Storeroom!B34	0.4815	0.3280	0.2199	0.1807	**0.1501**

At $S_0 = 22$ the storeroom reaches 0.1501, which is past the contract's 0.1523. **Six more transformers at the depot do what the contract does.**

Step 4, the price. Six transformers at \$3,800 is \$22,800 of capital, and at the carrying charge $\gamma = 0.25$ of Table 4.5 that is $6(0.25)(3800) = \$5{,}700$ a year to hold. Thus the contract is worth **up to \$5,700 a year**, and above that the utility should buy transformers instead.

The check. Two, and both are free. Equation 9.13 says the delay is the backorder level over the demand rate, so at the contract terms $1.3500/126 = 0.010714$ years, which is 3.91 days and is what `Depot!B20` reports. `Storeroom!B37` also holds Equation 9.7 rearranged; it read zero before any of this and reads zero after, so nothing above disturbed an identity that has to hold.

Interpretation. The answer is a rate, not a number, which is what makes it usable. A shop asking \$4,000 a year should be taken; one asking \$9,000 should be refused and the money spent on stock. Notice also what the exercise revealed about the system rather than the contract: at the present depot stock the utility is operating where the delay curve is steep, and either lever moves it a long way. A utility already holding 26 at the depot would find the same contract worth far less.

Where should the six go? That is the `Allocate` sheet's question, and the answer is not obvious. Adding six units brings the total to 38. Find the row whose total is 38, which is sheet row 46: the best split it reports is 18 at the depot and 5 at each storeroom, not 22 and 4.

The two sheets disagree, and Section 9.10.6 is why. `Allocate` runs METRIC and `Storeroom` runs VARI-METRIC. Judged on the `Storeroom` sheet,

METRIC's 18-and-5 gives 0.6157 against 0.6005 for 22-and-4, so taking the allocation sheet's advice costs about 2.5 percent more backorders than the model the chapter recommends would allow. That is small, it is in the direction Section 9.7.2 predicts, and it is the concrete form of a limitation stated one section later in the abstract.

9.10.6. What the Worksheet Cannot Do

The `Allocate` sheet runs METRIC, not VARI-METRIC, and the reason is the one difference between the two models that matters to a spreadsheet.

A Poisson first order loss function closes in one cell, so a grid of a few hundred candidate plans is a few hundred cells. A negative binomial one does not. Writing it the same way needs a distribution function evaluated at a non-integer r, which is Section 9.10.2's problem again, and the alternative is the recursion block, which is fifty cells per candidate plan rather than one. Fifty cells times a few hundred plans is not a worksheet.

Thus the `Allocate` sheet answers a slightly different question than Table 9.4 did, and its flushout and its dropped totals land at slightly different places. That the two agree on the shape and not on the digits is itself instructive.

The right reading is that the worksheet has reached its limit here rather than the method. Section 9.10 is where a chapter usually ends; this one does not, because a utility with nine repairable items and four storerooms is already past what the `Allocate` sheet can carry, and a defense establishment with forty thousand items is not in the same conversation. Section 9.11 takes that up.

9.11. Designing the Software

Section 8.17 had an algorithm to organize and one convention to place. This chapter has more moving parts: two locations that must agree with

one another, two models that share most of their arithmetic, and two searches only one of which may safely be run greedily. The package is `inventory.multiechelon`.

9.11.1. What the Software Must Do

Reading back over the chapter, the software has to

1. hold a depot and the storerooms it supplies, and derive the rates of Equation 9.1 and Equation 9.2 from the network,
2. evaluate the depot exactly, including the **variance** of its backorder level, which Equation 9.19 needs and which Section 8.17 never had to produce,
3. build a storeroom's pipeline under either model, to one moment or to two,
4. report the measures of Section 9.4 at every location,
5. find the best split of a given number of units across the two levels, discarding the non-convex plans of Section 9.9.3, and
6. spend a budget across items, returning the whole curve rather than one plan.

Requirements 5 and 6 are the ones the worksheet of Section 9.10 cannot meet, for the reason Section 9.10.6 gives. Requirement 2 is the one that looks hardest and turns out not to be.

9.11.2. Finding the Nouns

Underline the nouns in the requirements and four survive.

A **storeroom** is λ_j, ϕ_j, T_j and O_j. It owns Equation 9.1, and it owns the part of its own pipeline that Theorem 9.1 covers exactly, which is a property of the location and not of the system around it.

A **depot item** is one item's stock at the depot together with the storerooms it supplies. It owns λ_0, the shares f_j of Equation 9.16, and the depot's own pipeline distribution.

A **model** is a set of depot items, and it is what a search is handed.

A **search** reads a model and a budget and returns stock levels. It is the only thing in the chapter that chooses anything.

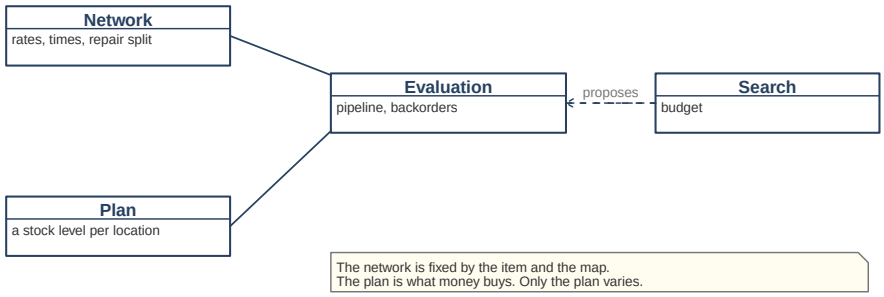


Figure 9.9.: The concepts of this chapter before any thought about implementation. A network of storerooms and a depot fixes the rates. A plan assigns stock levels to it. Evaluating the two together gives backorders, and a search is the thing that proposes plans until the money runs out. Notice that the network and the plan are separate: the chapter's whole optimization holds the first fixed and varies the second.

Three candidate nouns fail, and each fails instructively.

Depot looks like a class, and in a diagram it is one. It fails because everything the depot is asked for is a property of a **distribution**: a backorder level, a variance, a delay that is the first divided by a rate. Giving it a class of its own would give it a home for behavior it does not have. The depot's stock and its storerooms travel together, so they are one object.

METRIC and VARI-METRIC as two classes is the reading Section 9.8 almost invites, since they have separate names and separate sections. It fails on Equation 9.18. The two models agree on every mean in the chapter, and two classes would let them drift apart in a place where the chapter says they cannot.

A pipeline distribution of our own is the reflex, and it is wrong for the reason Section 8.17.2 gives: the library already has one.

9.11.3. What Each Thing Knows and Does

Class	Knows	Does
`BaseStockItem`	a stock level and a lead time demand distribution	the measures of Section 9.4, and the two moments of its backorder level
`VMBaseItem`	its failure rate, repair split, repair and ship times, and its depot	Equation 9.1; refits its own pipeline from Equation 9.18 and Equation 9.19
`VMItem`	the depot's stock and the storerooms beneath it	Equation 9.2 and Equation 9.12; tells every storeroom to refit when anything moves
`VariMetricModel`	a set of items	the totals of Equation 9.23
`MAFItemData`	one item's curve of backorders against units	builds the curve, takes the best split per total, and convexifies

Class	Knows	Does
`MarginalAnalysisAlgorithm`	a model and a budget	Algorithm 9.3: merges items by delta value until the money runs out
`LagrangeAlgorithm`	a model, a budget and an interval of multipliers	prices money and searches for the price that spends the budget

Notice that neither METRIC nor VARI-METRIC is a class. `VMBaseItem` builds a pipeline under either, and which one is a flag rather than a type. That is Equation 9.18 expressed as a design decision.

9.11.4. The Classes

Read the composition and the dependency against each other. A `VMItem` owns its storerooms, and each storeroom holds a reference back. Setting a storeroom's repair fraction recomputes λ_0 by Equation 9.2, refits the depot's Poisson, and then refits **every** storeroom. Setting the depot's own stock level does the same thing from the other side:

```
override var stockLevel: Int
    get() = super.stockLevel
    set(value) {
        super.stockLevel = value
        updateBaseLeadTimeDemands()
    }
```

That mutual reference should not be designed away. Equation 9.18 says a storeroom cannot be described without asking the depot how long it makes

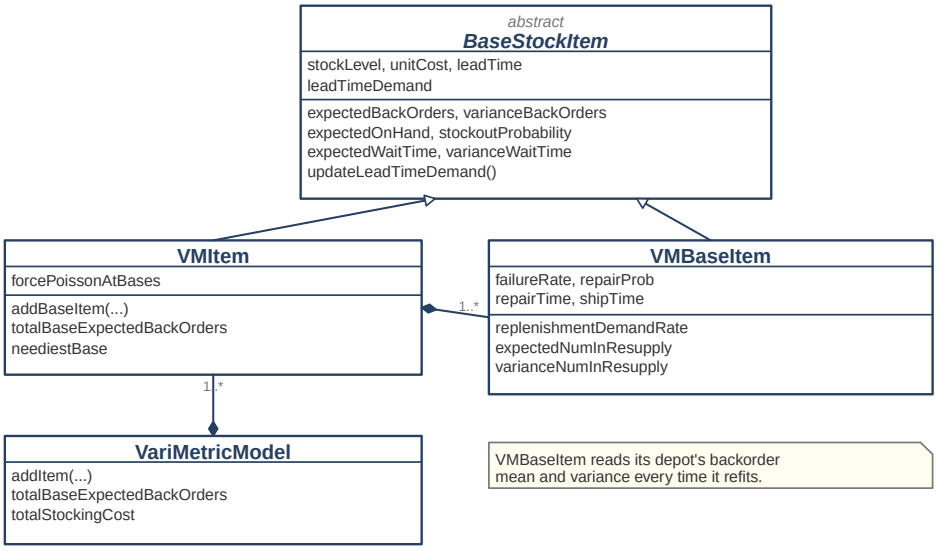


Figure 9.10.: The evaluation classes. Everything a location can be asked lives in BaseStockItem, because Equation 9.5 says the net inventory is the stock level minus the units in resupply and nothing else enters. A depot and a storeroom are both locations and differ only in how they obtain their lead time demand: the depot by Palm's theorem directly, the storeroom by fitting two moments that include the depot's backorder variance. The composition runs the other way from the dependency, which is the coupling drawn twice.

people wait. A design in which a storeroom could be changed without the depot noticing would be modeling something this chapter denies.

Mutability also suits what Section 9.9 does with these objects. Algorithm 9.3 moves stock levels thousands of times while searching, and rebuilding the network on every move would be the wrong shape for that.

9.11.5. What the Library Already Supplies

Three pieces of distribution machinery this chapter appears to need turn out to be available already, and recognizing that is most of the design.

The loss functions. Every location holds a `LossFunctionDistributionIfc` and asks it for `firstOrderLossFunction`. The measures of Section 9.4 are then written **once** and answered by whichever family the fit chose, which is Equation 9.5 saying that only the distribution of X_j matters.

The second moment of a backorder level. Equation 9.20 walks $E[B^2(s)]$ up one stock level at a time, and a worksheet needs that because it has rows and no second order loss function. A program has the reverse. We write the square as $(x-s)^2 = (x-s)(x-s-1) + (x-s)$ and take expectations over $x > s$, which gives

$$E[B^2(s)] = 2\,G^2(s) + G^1(s) \tag{9.25}$$

so Equation 9.19 needs one line:

```
val varianceBackOrders: Double
    get() {
        val ebo = expectedBackOrders
        val g2 = leadTimeDemand.secondOrderLossFunction(
         ↪ stockLevel.toDouble())
        return 2.0 * g2 - ebo * (ebo - 1.0)
```

```
    }
```

Notice that this does not make Equation 9.20 wasted. The two forms agree at every stock level, and each suits the artifact that carries it. A worksheet has rows and no second order loss function, so it climbs. A program has the loss function and no need of the rows, so it evaluates.

A negative binomial with a fractional number of successes. Section 9.10.6 left the worksheet stuck on $r = 14.3674$, because its own negative binomial function truncates that argument. The library takes it as a real number, so Equation 9.22 is a constructor call.

9.11.6. Choosing a Family

Equation 9.22 divides by $1 - p$, and $p \to 1$ as the variance approaches the mean. The original code answered that with a band, a Poisson for any ratio between 0.9 and 1.1, and Section 9.8.4 showed what the band costs. The port keys off the variance to mean ratio with no band:

```
val c = variance / mean
return when {
    c > 1.0 + RATIO_TOLERANCE  -> negativeBinomialOn(mean,
 ↪  variance)
    c >= 1.0 - RATIO_TOLERANCE -> Poisson(mean)
    else -> { val scale = variance / mean; Gamma(mean /
     ↪  scale, scale) }
}
```

Notice that `RATIO_TOLERANCE` is 10^{-9}, which is rounding and not a model. At an empty depot the two moments of Equation 9.19 agree exactly, but the variance is reached by subtracting numbers in the hundreds, so it lands

a few units in the last place away from the mean. The tolerance keeps that case a Poisson. Anything further from one is fitted as what it is.

The third branch is there for a reason. Section 9.8 records that neither Graves nor Sherbrooke could prove the variance always exceeds the mean, so a program that assumed it would be asserting what the literature declines to.

9.11.7. One Class, Two Models

METRIC and VARI-METRIC differ in one place. A `VMItem` carries a flag, and a storeroom consults it when it refits:

```
leadTimeDemand = if (depot.forcePoissonAtBases) {
    Poisson(expectedNumInResupply)
} else {
    fitTwoMoments(expectedNumInResupply,
 ↪  varianceNumInResupply)
}
```

Everything above that line is shared, which is what makes Equation 9.18 structural rather than merely asserted: the two models cannot disagree about a mean, because one object computes it.

> ⚠ A location reports the variance of the distribution **in use**, not the variance that Equation 9.19 computed. Under METRIC those differ, and a report showing the second while using the first would describe a model nobody ran.

9.11.8. Two Searches, Only One of Them Greedy

Section 9.9 is two searches nested, and the difference between them is the whole of Section 9.9.3.

The inner search is greedy and exact. At a fixed depot stock the storerooms' backorder functions are convex by Equation 9.10, so handing out units one at a time to whichever location gains most cannot be improved on. `MAFItemData` fills a table with one row per total and one column per depot level, spreading within a column that way.

The outer step is a minimum rather than a search. The best depot level for a given total is the row minimum, and taking minima across rows is exactly where convexity is lost.

The third step is convexification, and it is a lower convex hull reached by repeatedly jumping to whichever later total gives the steepest descent:

```
var slopeMax = 0.0
for (c in kIndex + 1 until maxRows) {
    val s = slope(kIndex, c)
    if (s >= slopeMax) { slopeMax = s; kLast = c }
}
```

The totals it skips are the ones Table 9.4 strikes out, and Equation 9.24 is then computed across the jump, over however many units it spans.

> **i** `neediestBase` ranks storerooms by exactly Equation 9.10, the $G^{0}(S_{j}) = P\{X_{j} > S_{j}\}$ of the next unit. The original code ranked by the stockout probability $P\{X_{j} \geq S_{j}\}$, one index off; the two agree on identical storerooms and need not agree in general, and the port corrects it so that the sentence above stays true.

Notice what is absent. There is no fixed point, no convergence test and no iteration count anywhere in the evaluation. Algorithm 9.1 has no loop and neither does the code, which is Section 9.6.1 cashed in. A model that allowed batching could not say that.

9.11.9. Pricing the Budget Instead of Enforcing It

Algorithm 9.3 is not the only way to spend a budget. A second family of searches solves the same program by relaxation, and the two fail in different places.

Attach a price to money and the problem falls apart into pieces. Let θ represent a charge per dollar held. We minimize

$$w(\theta) = \sum_j \bar{B}_j(S_j) + \theta \left(c_0 S_0 + \sum_j c_j S_j \right) \tag{9.26}$$

At a fixed θ the storerooms no longer compete for a budget, so each solves its own problem, and its problem is the newsvendor of Chapter 7: buy while the chance of needing the next unit exceeds its price. Thus

$$S_j = \min \left\{ S : G(S) \geq 1 - \theta c_j \right\} \tag{9.27}$$

and no search is needed at a storeroom at all. The depot still needs one, because moving it moves every storeroom, so `LagrangeAlgorithm` enumerates the depot over a window and reads Equation 9.27 off each storeroom at each trial level.

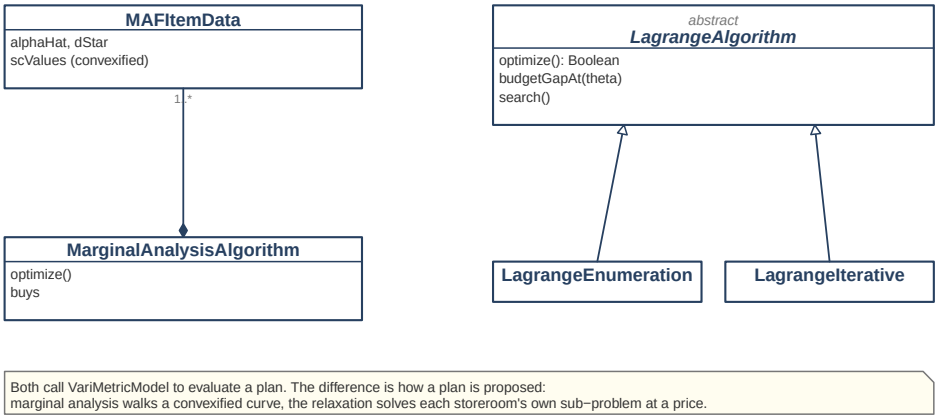


Figure 9.11.: The two families of search. Marginal analysis builds a curve for each item and then merges the curves; the relaxation prices money and hunts for the price that spends the budget. LagrangeAlgorithm holds everything except how the multiplier is chosen, which is the one method its two subclasses supply. Notice that both families end up calling the same evaluation, so neither is a different model.

⚠ **The multiplier lives on a scale set by the unit cost.** By Equation 9.27 a storeroom stocks nothing once $\theta \geq 1/c_j$, so the whole useful range is $(0, 1/c_j)$. For a transformer at \$3,800 that is 2.6×10^{-4}. Search an interval far above it and the algorithm reports, correctly and uselessly, that no root exists: every candidate multiplier prices money so dearly that no storeroom stocks anything.

On the two-item problem the three searches split two to one. At a budget of \$250,000:

Search	Spend	Storeroom backorders	Depot, transformer	Inside budget
Marginal analysis	\$247,000	2.8792	19	yes
Lagrangian, bisection	\$247,000	2.8792	19	yes
Lagrangian, enumeration	\$250,800	2.6840	20	**no, by \$800**

The stock levels are integers, so the cost is a step function of θ, and near the answer it jumps straight over the budget: at $\theta = 5.1356 \times 10^{-5}$ the plan costs \$250,800 and at 5.1361×10^{-5} it costs \$247,000. There is no multiplier that spends exactly \$250,000, so a search has to choose a side.

The bisection converges on the jump and returns the side that satisfies the constraint, landing on the same plan marginal analysis found. The enumeration keeps whichever candidate has the **smallest absolute gap**, and \$800 over is closer than \$3,000 under, so it takes the infeasible one.

Neither rule is wrong, but the difference matters before you use either. A planner who must not overspend wants the bisection or the marginal analysis. A planner asking what the budget nearly buys is better served by the enumeration, which here reports that $800 more would remove another 0.2 backorders. What you must not do is read the enumeration's answer as though it respected the budget.

The choice between the relaxation and marginal analysis is a different question again. Marginal analysis returns the whole curve of backorders against money and never overspends, and pays for that by enumerating every plan for every item first. The relaxation returns one plan and gets there without building a curve at all, which is what recommends it when the item count runs into the thousands and the budget is fixed.

9.11.10. Running the Chapter's Examples

`Section9Listings.kt` builds the utility of Example 9.4 and prints every figure the chapter quotes. Run it with

```
./gradlew run -PmainClass=inventory.multiechelon.Section9ListingsKt
```

The network is built depot first, with the storerooms hung off it, and the times are given in days at the call site so the conversion is visible:

```
val item = VMItem(1, 16, 3800.0, days(60.0))
repeat(4) { item.addBaseItem(45.0, days(30.0), 0.3,
 ↪  days(10.0), 4, 3800.0) }
```

Asking the same plan under both models prints

```
  METRIC       mu_j   3.2276  var   3.2276  Bbar_j   0.4055  Ibar_j   1.1778  system
  VARI-METRIC  mu_j   3.2276  var   3.9527  Bbar_j   0.4815  Ibar_j   1.2539  system
```

Read the two lines against each other. The `mu_j` column agrees to four decimals and the `var` column does not, which is Section 9.8's claim that no mean changes, enforced by the fact that one object produced both.

The allocation prints Table 9.4, including which totals it struck out:

```
 total        EBO    S_0  reduction    kept?
    11    17.6154     11     0.9927
    12    16.6303     12     0.9850  dropped
    13    15.6433      9     0.9870
    14    14.6587     10     0.9846
    ...
    19     9.9647     15     0.8842
    20     9.1088     12     0.8559  dropped
    21     8.2462     13     0.8627
```

Notice that the depot column falls from 12 to 9 at total 13 and from 15 to 12 at total 20. The flushout of Section 9.9.4 and the non-convexity of Section 9.9.3 are the same event seen from two columns.

9.11.11. Where This Leaves Us

Four classes evaluate either model at any plan, two more search a single item's curve, and three more spend a budget across items, with no iteration anywhere in the evaluation. Every figure this chapter quotes was produced by that code.

Section 9.12 collects what the chapter established.

9.12. Summary

A repairable item is not bought, so one-for-one replenishment is not a policy choice. A failure is simultaneously a demand for a serviceable unit and an arrival of an unserviceable one, and they are the same physical event. Thus the repair pipeline is fed one failure at a time and there is no ordering cost standing between the failure and the induction that waiting could amortize. Chapter 8 presented $Q = 1$ as a decision the storeroom made, and for a consumable it is one; here the item makes it. That is what allows this chapter to carry two locations without also carrying an order quantity, and it is the whole reason to meet multi-echelon inventory in this setting first.

The population is fixed, so the only question is where the units sit. Figure 9.1 has no arrow entering it and none leaving. Buying more units is a separate and much slower decision, usually taken once when a fleet is fielded. Thus the stock positioning problem of Section 9.1 stops being one question among several and becomes the only one there is.

Palm's theorem is the chapter's one exact result, and it asks for a mean and nothing else. With Poisson failures and independently drawn resupply times, the number of units in resupply is Poisson with mean λT whatever the resupply time distribution may be. That license is worth more here than it looks, because repair time data is the worst data in this business: a shop can usually report an average flow time and rarely more.
The condition that fails in practice is the third one. The $M/G/\infty$ reading makes it explicit that repair capacity never binds, and a shop with three benches and twelve carcasses breaks it. Notice also that Equation 9.3 and Equation B.30 do not contradict one another. The separating question is whether an activity started later can finish earlier, which is a fact about the shop and not about the model.

The depot's backorders are the storeroom's lead time. That sentence is the coupling, and Equation 9.13 is the arithmetic of it: divide the depot's average backorder level by its demand rate and a wait falls

out, by Little's Law, which assumes almost nothing. Section 9.5 obtained the same number twice on twenty-six weeks of one realization, once by following individual requests and once by weighting a level by how long it was held, and the two agreed because they are the same area counted along two different axes. **A quantity written $B(t)$ is averaged over time and not over the rows of a ledger**, and a table with more rows in its quiet stretches will mislead anyone who forgets it.

One-for-one ordering buys a system that solves in one pass. The depot's pipeline contains no storeroom stock level, so the depot is solved first, once, and on its own, and the storerooms follow from the delay it imposes. There is no circle to iterate around, no convergence to test, and no fixed point anywhere in Algorithm 9.1, Algorithm 9.2 or the package of Section 9.11. That is a property of $Q = 1$, and it does not survive batching.

METRIC gets every mean right and one distribution wrong. The depot's pipeline really is Poisson, so its backorders are exact; the delay is exact because Little's Law is; and the storeroom's pipeline **mean** is exact for the same reason, with no independence assumed anywhere. What is not earned is the claim that the storeroom's pipeline is Poisson, because two requests arriving while the depot is empty wait on the same repairs and their delays move together. The error runs the unsafe way, understating backorders and overstating availability, and it was known to the model's author at the time. Graves (1985) measured it where a planner would feel it, in stock levels rather than reported backorders.

VARI-METRIC changes no mean; it restores a variance that METRIC discarded. Conditional on the depot's backorder total, the number owed to any one storeroom is binomial, so Equation B.8 supplies two terms: the randomness in which storeroom each backorder belongs to, and the randomness in how many there are. Adding them to the Poisson part gives Equation 9.19, and setting them to zero gives METRIC back. Thus the two models are not rivals but one model read to one moment or to two, which is what Sherbrooke (1986) means by calling METRIC

a first-order model. **One step rests on evidence rather than proof**, and the authors say so: neither Graves nor Sherbrooke could show that the variance always exceeds the mean, so a program should test it rather than assume it.

Two echelons break the convexity that licenses marginal analysis, and the break is where positioning gets answered. Hold the depot's stock and every storeroom curve is convex, so the greedy rule is exact. Move the depot's stock and it is not, because one unit there shifts every storeroom at once; the remedy is to strike out the interior points so the procedure buys two units at a time. **Reading the same table's depot column produces flushout**: buying one more unit can make it right to move three units already held out of the depot and forward. Thus Section 9.1's claim is delivered, and delivered in a better form than it was made. The answer to "should this item be forward stocked" is a schedule against the budget rather than a yes or a no; it is not monotone, and no rule of thumb produces it.

Table 9.6 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 9.6.: Notation introduced in this chapter.

Symbol	Meaning
j, J	A storeroom and the number of them; the depot is location 0
λ_j	Failure rate at storeroom j, assumed Poisson
ϕ_j	Fraction of failures storeroom j repairs itself
T_j, T_0	Average repair time at storeroom j and at the depot
O_j	Average order and ship time from the depot to storeroom j

Symbol	Meaning
λ_{j0}	$(1-\phi_j)\lambda_j$, the requests storeroom j sends up
λ_0	$\sum_j \lambda_{j0}$, the depot's demand rate
S_j, S_0	Stock level at storeroom j and at the depot
X_j, X_0	Units in resupply for a storeroom and on the depot's benches
μ_j, μ_0	$E[X_j]$ and $E[X_0]$, the pipeline means
σ_j^2	$Var[X_j]$, which METRIC sets equal to μ_j
$\bar{W}$	Average wait a request suffers at the depot, the same for every storeroom
B_{0j}	Units the depot owes storeroom j, binomial given the total
f_j	λ_{j0}/λ_0, storeroom j's share of the depot's demand
$E[B^2(s)]$	Second moment of the backorder function, which VARI-METRIC needs
EBO	Total expected backorders at the storerooms, the objective
$\delta(S)$	Backorders removed per dollar by the next unit, the delta value

9.13. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.8. Report stock levels as whole units. Times are given in days and rates per year, so divide by 365 before combining them, and show the division. Take the

utility of Example 9.4 as the setting: four alike storerooms supplied by one depot, with the item data of Table 4.5 and a carrying charge of 0.25 a year.

9.13.1. Working the Model by Hand

Exercise 9.1. Answer each in a sentence, and say which section settles it.

a. True or false: because Palm's theorem needs only the mean repair time, the variability of a resupply time never affects an inventory model.
b. True or false: METRIC and VARI-METRIC report the same expected pipeline at a storeroom.
c. A depot's backorder column in an event ledger reads $0, 0, 1, 1, 0, 1$ on six rows. A student averages those six numbers and gets 0.5. What has been computed, and what was wanted?
d. Multiple choice. METRIC's storeroom backorders are wrong in which direction, and why?

   i. Too high, because it double counts the depot delay.
   ii. Too low, because it forces the pipeline variance to equal its mean.
   iii. Too low, because it uses the wrong mean for the pipeline.
   iv. Either way, depending on the item.

e. Multiple choice. A utility buys one more unit of an item and the optimal depot holding **falls**. This is

   i. an arithmetic error, since more stock cannot reduce a stock level.
   ii. possible only when the storerooms differ from one another.
   iii. flushout, and it is a property of the two-echelon trade.
   iv. a sign the convexification step was skipped.

Exercise 9.2. A storeroom stocks a recloser control, which it manages one for one. Failures are Poisson at $\lambda_j = 18$ a year. Half of them the storeroom rebuilds itself, in an average of 20 days; the other half go to the depot, which ships a replacement in an average of 14 days. **Assume for this exercise that the depot never makes anyone wait**, so $\bar{W} = 0$.

a. Write down the average resupply duration a failure faces, in days, and use Equation 9.3 to obtain the mean number of units in resupply. Give the unit conversion as a worked chain.
b. State the distribution of that number, and give its standard deviation.
c. The storeroom holds one control. What fraction of failures find it on the shelf?
d. Tabulate $\bar{B}_j$, $\bar{I}_j$ and the ready rate for $S_j = 0, 1, 2, 3$, and check every row against Equation 9.7.
e. The shop that rebuilds these controls is rebuilt itself, and the average repair time falls from 20 days to 8 while its **variability doubles**. Which of your answers change, and which do not?

Exercise 9.3. Run the ledger of Section 9.5 again with one rule changed: the storeroom now rebuilds **nothing** locally, so every failure sends a carcass and a request to the depot. Everything else is as it was, except that the storeroom is leaner.

Take $S_j = 1$ and $S_0 = 2$. The depot repairs a carcass in $T_0 = 8$ weeks and ships a serviceable unit in $O_j = 2$ weeks. Failures occur at the storeroom in weeks 1, 4, 7, 10, 13 and 16, and the observation runs for 28 weeks. Follow the convention of Example 8.1: one row per event, at the week it occurs, with the state recorded after that event.

a. Build the ledger, carrying I_j, B_j, X_j, I_0, B_0 and X_0. Check Equation 9.11 on every row at both levels.
b. Count the depot's backorder-weeks, weighting each level by how long it was held. State the time average of $B_0(t)$.
c. Obtain the average wait a second way, by following the six requests individually, and reconcile the two.

d. The depot is short four separate times and the storeroom's crews wait only three weeks in total. Explain why those numbers are not the same, in terms of Equation 9.14.
e. Rerun part (a) in your head with $S_j = 2$ instead of 1. How many storeroom backorder-weeks are there now, and what does that say about where a unit is worth more on this item?

Exercise 9.4. The utility also stocks a voltage regulator, one for one, at the same four storerooms. Each storeroom sees $\lambda_j = 12$ failures a year and rebuilds a fifth of them locally in 45 days. The rest go to the depot, which repairs in $T_0 = 90$ days and ships in $O_j = 10$ days. A regulator costs \$9,500.

a. Obtain λ_{j0} and λ_0 from Equation 9.1 and Equation 9.2, and the depot's pipeline mean from Equation 9.12.
b. For $S_0 = 6, 8, 10$ and 12, report the depot's expected backorders, the variance of its backorders, and the delay in days. Use Equation 9.25 for the variance rather than the recursion.
c. The utility will not tolerate a delay above three weeks. What is the smallest depot stock meeting that, and what does holding that much cost the utility each year at a carrying charge of 0.25?
d. Notice that the variance of the depot's backorders falls more slowly than their mean as S_0 rises. Report the variance to mean ratio at each level in part (b), and say what that does to Equation 9.19.

Exercise 9.5. Continue with the regulator of Exercise 9.4, at a depot stock of $S_0 = 8$.

a. Obtain the part of a storeroom's pipeline that Theorem 9.1 covers exactly, and then μ_j and σ_j^2 from Equation 9.18 and Equation 9.19. Report the variance to mean ratio.
b. Fit the negative binomial of Equation 9.22 and report p and r. Say why a worksheet cannot evaluate this fit with its own negative binomial function.

c. Report $\bar{B}_j$ under both models for $S_j = 1, 2$ and 3, and the ratio of the two at each level.
d. The ratio grows with S_j. Explain why, in terms of which part of the distribution a backorder calculation is reading.
e. The utility holds $S_j = 2$ at each storeroom. State the total expected backorders across the four under each model, and say which figure you would put in front of a manager and why.

Exercise 9.6. Equation 9.20 and Equation 9.25 compute the same quantity by different routes.

a. Prove Equation 9.25 by writing $(x-s)^2 = (x-s)(x-s-1) + (x-s)$ and taking expectations over $x > s$.
b. For the regulator's depot of Exercise 9.4 at $S_0 = 8$, evaluate $E[B^2(8)]$ both ways: walk Equation 9.20 up from $s = 0$, and evaluate Equation 9.25 directly. Report both to four decimals.
c. Section 9.10 uses the recursion and Section 9.11 uses the closed form. Say what each has that the other does not, and why neither choice is a mistake.
d. The recursion subtracts. At what sort of stock level would you expect it to lose precision, and how would you check?

9.13.2. Using the Software

Exercise 9.7. Example 9.6 priced a repair contract on the transformer. Do the same for the voltage regulator of Exercise 9.4, using `inventory.multiechelon` rather than the workbook.

Build the network with `VMItem` and four calls to `addBaseItem`, holding $S_0 = 8$ and $S_j = 2$.

a. Report the system's storeroom backorders as it stands.
b. The depot's shop offers to cut T_0 from 90 days to 70. Report the new delay and the new storeroom backorders.

c. Find the smallest depot stock that matches the contract's backorders without it, by raising S_0 and re-reading the model.
d. Price the contract as an annual figure, using the carrying charge of 0.25.
e. Compare the answer with the transformer's \$5,700 of Example 9.6, and say what about the two items makes them differ.

Exercise 9.8. Use `MAFItemData` to build the efficient curve for the regulator of Exercise 9.4 on its own, over depot levels 0 to 20 and totals up to 30.

a. Print the total, the best depot level, the expected storeroom backorders and the reduction, for totals 4 through 16.
b. Identify every total the convexification drops, and for each one show the chord test: the plan lies above the straight line joining its neighbors.
c. Identify every flushout, meaning a total at which the best depot level falls.
d. The regulator costs \$9,500 and the transformer \$3,800. Without running the merge, say which item the first dollar buys, and check your answer against Equation 9.24 at a stock level of zero for each.

9.13.3. Using the Workbook

Exercise 9.9. Open `Chapter9Models.xlsx` at the `Storeroom` sheet, which carries the transformer of Example 9.4.

a. With `Depot!B16` at 16, record μ_j, σ_j^2 and their ratio from cells `B22`, `B23` and `B24`.
b. Walk `Depot!B16` from 0 to 26 in steps of 2 and record the ratio at each. Plot it against the depot stock level.
c. Section 9.8.4 describes code that used a Poisson whenever this ratio lay between 0.9 and 1.1. At which of your depot stock levels would

that rule have used a Poisson? Then set `Depot!B16` to 24, read the storeroom's backorders under METRIC and VARI-METRIC from `B33` and `B34`, and use `B35` to say how much larger the second is, in percent.

d. The curve you plotted rises and then falls. Explain both ends: why the ratio is near one when the depot is nearly empty, and why it is near one again when the depot is generous.

9.13.4. Reading the Literature

Exercise 9.10. Read Sherbrooke (1986), which is nine pages, and Graves (1985) if you can obtain it.

a. Sherbrooke says the understatement of base backorders was known when METRIC was published. Quote the sentence and say why it was judged acceptable at the time.

b. Graves measures the improvement in stock levels rather than in reported backorders. State the two percentages, and say why measuring it that way is more useful to a planner.

c. Sherbrooke qualifies those percentages immediately. Quote the qualification and explain what it means for a utility deciding whether to move from METRIC to VARI-METRIC.

d. One step of VARI-METRIC rests on a claim the authors could not prove. State it, say where it is used, and say what a program should do about it.

e. In a paragraph, say what you would need to measure on a real range of items to decide whether the change is worth making.

10. Multi-Echelon Systems with Batch Ordering

Learning objectives

After reading this chapter you should be able to:

- explain why a location that orders in batches passes up a stream that is not its customers' demand
- compute a batching hub's measures when its storerooms order one at a time, and the distribution of the wait it imposes on them
- evaluate a two-level system exactly by Axsäter's recursion, and price the approximations of a storeroom's lead time demand against it
- count a batching storeroom's orders in a window from the Poisson table, both at a random moment and as one of its own orders sees them
- build a hub's lead time demand when storerooms batch and other demand arrives, exactly and by a two-moment recipe, and say why a common shortcut over-stocks
- handle a random supplier lead time by conditioning on it and averaging
- set both locations' policies with a two-level iteration, and price its answer against an exact cost

Chapter 9 solved two levels under one-for-one ordering, where every demand at a supported location triggers a request of exactly one unit. That assumption is what made the coupling tractable: the demand a location passes up is its own demand, arrival for arrival.

This chapter removes it. When a location orders in batches of Q, the stream it passes up is no longer its customers' demand stream. It is lumpier, it is steadier in its timing, and characterizing it is most of the work.

The chapter comes in three parts, and each earns the next. Part I lets only the hub batch. Its storerooms still order one unit at a time, so the hub sees their customers' demand unchanged, and everything can be computed exactly: the hub's measures, the wait it imposes, and, by a recursion due to Axsäter (1993), the storerooms' measures as well. Part II lets the storerooms batch and lets demand from elsewhere reach the hub. The machinery that handles it needs nothing beyond the Poisson table and chapter 8, and wherever Axsäter's recursion still applies the two agree exactly. Part III chooses both locations' policies together, by an iteration that solves one location at a time.

One move carries the whole chapter: condition on the position, then average. Section 8.5.1 obtained the measures of an (r, Q) location by averaging base-stock measures over its Q equally likely positions. The chapter makes that move four more times, and you should expect to recognize it before it is named.

10.1. What Batching Changes

A storeroom that orders one at a time sends its supplier one request for each customer, at the moment the customer arrives. Its supplier therefore sees the storeroom's customers. A storeroom that orders Q_j at a time sends one request for every Q_j customers, and the supplier sees something else: nothing at all for a while, then Q_j units at once.

Thus, batching changes three things at the hub, and each is a section of this chapter.

1. **The hub's demand is no longer the customers' demand.** It arrives in lumps of Q_j, and it arrives more regularly than the customers do, since the storeroom orders once per Q_j units of demand, nearly on a schedule. Section 10.8 counts those lumps.
2. **An order waits for its last unit.** A hub with two units on its shelf and an order of three ships two and owes one, and the order is not filled until the third arrives. Section 10.10 measures that wait.
3. **The two policies can no longer be set one after the other.** Under one-for-one ordering the depot of Chapter 9 could be solved first and on its own, because its demand contained no storeroom policy. Once the storeroom batches, the hub's demand depends on Q_j, and the storeroom's lead time depends on the hub. Section 10.12 handles the circle.

What does not change is the coupling itself. Recall from Section 9.6.2 that a supplier's backorders become a delay by Little's law. That move carries over unchanged, and so does the warning attached to it: the average it produces is over every request, including those filled at once.

10.1.1. Where This Leaves Us

The difficulty is in the hub's demand and in the storeroom's lead time. Part I sets both aside by letting only the hub batch, so that the chapter's arithmetic can first be done exactly on a system the reader already knows.

10.2. The Item and the Network

The item is the fuse cutout of Section 8.11, a cheap consumable that a utility stocks at a central hub and at the storerooms its crews draw from.

Section 8.11 took the cutout as drawn in job lots. **In Parts I and II each customer takes one unit**, and the job lots return in Section 10.14, where only the iteration of Part III can handle them. That is a simplification made for the mathematics, and it is stated here so that a reader who remembers the job lots does not think the book contradicted itself.

The running example of Part I has four identical storerooms supplied by one hub.

What is stocked? One item, the fuse cutout, at five locations: the hub and four storerooms.

What is the demand process? Poisson at one unit a week at each storeroom, one unit per customer.

When is inventory reviewed? Continuously, at every location.

What triggers replenishment, and how much? At a storeroom, every customer: the storerooms run base stock $S_j = 3$ and order one unit at a time. At the hub, its position reaching $r_0 = 9$, for $Q_0 = 8$ units.

What happens to unmet demand? It is backordered at both levels. A crew waits at the storeroom; a request waits at the hub, and the hub fills its requests first come, first served, shipping part of an order when it has only part of it.

What costs are incurred, and when? Holding at both levels at $h = \$28.75$ per unit per year, and a shortage cost at the storerooms of $b = \$287.50$ per unit per year owed. The hub carries no shortage cost of its own, since a unit short at the hub costs nothing until a crew is short of it.

The chapter works in weeks, so the costs are converted once:

$$h = \frac{\$28.75}{\text{unit} \cdot \text{year}} \times \frac{1 \text{ year}}{52 \text{ weeks}} = \$0.5529 \text{ per unit per week}, \qquad b = \frac{\$287.50}{52} = \$5.5288 \text{ per unit per week}$$

Notation. Let λ_j represent the demand rate at storeroom j and λ_0 the demand rate at the hub, in units per week. Let L_0 represent the hub's lead

time from its supplier and O_j the transit time from the hub to storeroom j, both constant and in weeks. Let (r_0, Q_0) and (r_j, Q_j) represent the two policies, a base-stock storeroom being $(S_j - 1, 1)$. Let $D_0(\ell)$ represent the hub's demand in a window of ℓ weeks. Let W represent the time a request waits at the hub before it ships, and $\bar{B}_0$, $\overline{RR}_0$ and $\bar{I}_0$ the hub's expected backorders, ready rate and on-hand stock. In the running example $\lambda_j = 1$, $\lambda_0 = 4$, $L_0 = 3$ and $O_j = 1$. The values $Q_0 = 8$ and $S_j = 3$ are stated rather than chosen here; Section 10.6 shows that, given $Q_0 = 8$, they and $r_0 = 9$ are the exact optimum.

10.2.1. Where This Leaves Us

Before any formula, Section 10.3 runs a hub and one storeroom by hand, so that every quantity the chapter computes has been seen on a posted row first.

10.3. Working Two Locations by Hand

The ledger follows the convention of Example 8.1 and Section 9.5.2 exactly: **one row per event, at the week it occurs, with the state recorded after the event is posted.** Two events can fall in the same week, and then the order matters, so it is fixed once: **the hub's receipts first, then the storeroom's, then demands in the order they arrive.**

You should be able to perform by hand the bookkeeping that the workbook and the code will otherwise perform for you. That is the reason the ledger comes first.

10.3.1. The System and the Rules

The system is one storeroom and its hub, with small numbers so that events are few. The hub runs $(r_0, Q_0) = (1, 4)$ with $L_0 = 3$ weeks and

starts with 5 units on its shelf. The storeroom runs base stock $S_j = 2$, is $O_j = 1$ week from the hub, and starts with 2 on its shelf. Failures occur in weeks 1, 3, 6, 8, 9, 10 and 14, and a storm in week 8 takes out three cutouts, so week 8 has three failures.

The columns. I_j, B_j and IP_j are the storeroom's on-hand, backorders and position, and "in transit" counts units shipped by the hub and not yet received. I_0, B_0, "on order" and IP_0 are the same four at the hub.

The check to run on every row. At the hub,

$$IP_0 = I_0 + \text{on order} - B_0, \tag{10.1}$$

and at the storeroom its position is on-hand less backorders plus what is in transit and what the hub still owes it. Compute the positions both ways on every row, and stop when they disagree.

10.3.2. The Ledger

Week 1. A cutout fails. The storeroom issues one of its two, leaving one, and sends a request to the hub. The hub has five, ships one, and is left with four. Its position falls from 5 to 4, which is above $r_0 = 1$, so it orders nothing.

Week 2. The shipment arrives and the storeroom is back to two.

Weeks 3 and 4, and 6 and 7. The same thing twice more: a failure, a request, a shipment that arrives a week later. The hub's shelf falls to 3 and then to 2.

The table has been updated as follows.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure, request to hub	1	0	2	1	4	0	0	4
2	storeroom receives 1	2	0	2	0	4	0	0	4
3	failure, request to hub	1	0	2	1	3	0	0	3
4	storeroom receives 1	2	0	2	0	3	0	0	3
6	failure, request to hub	1	0	2	1	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
7	storeroom receives 1	2	0	2	0	2	0	0	2

Notice what has not happened. Six weeks in, nobody has waited, and the hub has not ordered. It has been spending its opening stock one unit at a time, and its position has fallen from 5 to 2, one step above its reorder point.

Week 8, the storm. Three failures are reported, and they are posted one at a time.

The first takes the storeroom from two to one and sends a request. The hub ships its second-to-last unit, and its position falls to $1 = r_0$. **The hub orders 4**, due in week 11, and its position rises to 5.

The second takes the storeroom's last unit. The hub ships its last, and its position falls to 4.

The third finds the storeroom's shelf empty. A crew waits, and $B_j = 1$. Its request reaches a hub with nothing on the shelf, so **the hub records a backorder**, and $B_0 = 1$. The hub's position falls to 3.

The table has been updated as follows.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure, request to hub	1	0	2	1	4	0	0	4
2	storeroom receives 1	2	0	2	0	4	0	0	4
3	failure, request to hub	1	0	2	1	3	0	0	3
4	storeroom receives 1	2	0	2	0	3	0	0	3
6	failure, request to hub	1	0	2	1	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
7	store-room receives 1	2	0	2	0	2	0	0	2
8	failure, request to hub, **hub orders 4**	1	0	2	1	1	0	4	5
8	failure, request to hub	0	0	2	2	0	0	4	4
8	failure, request **back-ordered**	0	1	2	2	0	1	4	3

Check the last row both ways. At the hub, $0+4-1=3$. At the storeroom, on-hand less backorders is -1, two units are in transit and the hub owes one, so the position is $-1+2+1=2$. Both hold.

Week 9. The two units shipped in week 8 arrive. By the posting rule the receipt comes first: one goes straight to the waiting crew, whose wait was one week, and one goes on the shelf. Then a failure takes it, and its request finds the hub empty again. $B_0 = 2$.

Week 10. A failure finds the shelf empty, so a second crew waits. Its request is the hub's third backorder, and it carries the hub's position from 2 to 1, its reorder point again. **The hub orders another 4**, due in week 13, while the first is still on its way. On order now reads 8, the first time the reader has seen two hub orders outstanding.

Week 11, the payoff row. The hub's first order arrives. It owes three requests, from weeks 8, 9 and 10, and it fills them in that order. **One delivery clears three waiting requests at once**, and the fourth unit goes on the hub's shelf. The three shipments leave together and arrive together in week 12.

The table has been updated as follows.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure, request to hub	1	0	2	1	4	0	0	4

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
2	storeroom receives 1	2	0	2	0	4	0	0	4
3	failure, request to hub	1	0	2	1	3	0	0	3
4	storeroom receives 1	2	0	2	0	3	0	0	3
6	failure, request to hub	1	0	2	1	2	0	0	2
7	storeroom receives 1	2	0	2	0	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
8	failure, request to hub, hub orders 4	1	0	2	1	1	0	4	5
8	failure, request to hub	0	0	2	2	0	0	4	4
8	failure, request backordered	0	1	2	2	0	1	4	3

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
9	storeroom receives 2, serves the waiting crew	1	0	2	0	0	1	4	3
9	failure, request backordered	0	0	2	0	0	2	4	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
10	failure backordered, request backordered, hub orders 4	0	1	2	0	0	3	8	5
11	hub receives 4, **fills three waiting requests**	0	1	2	3	1	0	4	5

Week 12. The three units arrive at the storeroom. One goes to the crew waiting since week 10, who waited two weeks, and two go on the shelf.

Week 13. The hub's second order arrives. Nothing is owed, so all four go on its shelf, which now holds five.

Weeks 14 and 15. One more failure, filled at once, and its replacement arrives a week later. The system is quiet again.

The table has been updated as follows, and this is the completed ledger.

Table 10.1.: The completed ledger, fifteen weeks at a storeroom and the hub that supplies it. Nine failures, nine requests to the hub, three of which waited, and two crews who waited because of them.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure, request to hub	1	0	2	1	4	0	0	4
2	storeroom receives 1	2	0	2	0	4	0	0	4
3	failure, request to hub	1	0	2	1	3	0	0	3

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
4	storeroom receives 1	2	0	2	0	3	0	0	3
6	failure, request to hub	1	0	2	1	2	0	0	2
7	storeroom receives 1	2	0	2	0	2	0	0	2
8	failure, request to hub, hub orders 4	1	0	2	1	1	0	4	5

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
8	failure, request to hub	0	0	2	2	0	0	4	4
8	failure, request backordered	0	1	2	2	0	1	4	3
9	storeroom receives 2, serves the waiting crew	1	0	2	0	0	1	4	3
9	failure, request backordered	0	0	2	0	0	2	4	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
10	failure backordered, request backordered, hub orders 4	0	1	2	0	0	3	8	5
11	hub receives 4, fills three waiting requests	0	1	2	3	1	0	4	5

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
12	storeroom receives 3, serves the waiting crew	2	0	2	0	1	0	4	5
13	hub receives 4	2	0	2	0	5	0	0	5
14	failure, request to hub	1	0	2	1	4	0	0	4
15	storeroom receives 1	2	0	2	0	4	0	0	4

10.3.3. What the Ledger Establishes

The mean wait, two ways. Follow the nine requests. Six waited nothing, and those from weeks 8, 9 and 10 waited 3, 2 and 1 weeks, a total of 6 weeks, so the mean is $6/9 = 0.667$ weeks. Now weight the hub's backorder level by how long it was held: 1 from week 8 to 9, 2 from 9 to 10, 3 from 10 to 11, which is $1+2+3 = 6$ unit-weeks, and divide by the nine requests. The same 0.667. That is Little's law, as it was in Section 9.5.3, and the two agree because they count the same area along different axes.

The three waits of week 11 are one event. Three requests waited 3, 2 and 1 weeks, and all three ended at the same delivery. They are not three independent draws from a distribution of waits; they are one delay seen three times. That matters in Section 10.6, where we shall find that for a base-stock storeroom it turns out not to matter at all, and the reason why.

The crews waited because the hub did. The storeroom's stock never changed. Its two shortages were manufactured at the hub, by the storm draining a hub that had been spending its opening stock one unit at a time.

A request that arrives with the hub's position high waits nothing. In weeks 1 to 7 the hub's position was 4, 3 or 2, and every request shipped at once. The chance a request waits depends on where the hub's position happens to sit when it arrives, and Section 10.4 turns that observation into the hub's measures.

10.3.4. Where This Leaves Us

The ledger measured one wait by following requests. Section 10.4 computes the hub's measures for the running example, and Section 10.5 then gives the whole distribution of the wait, not only its mean.

10.4. The Hub Is a Chapter 8 Problem

When the storerooms order one unit at a time, each customer demand becomes one request at the hub the moment it occurs. The requests from four storerooms, each a Poisson stream, merge into a Poisson stream at $\lambda_0 = \sum_j \lambda_j$. Thus, **the hub is exactly the location of Section 8.5**: one location running (r_0, Q_0) against Poisson demand with a constant lead time. Its lead time demand $D_0(L_0)$ is Poisson with mean $\lambda_0 L_0$, and Equation 8.18, Equation 8.19 and Equation 8.20 give its measures with no change at all.

This is worth dwelling on for a moment, because it is where the chapter's one move comes from. Section 8.5.1 established that the position of an (r, Q) location is equally likely to sit at each of $r+1, \ldots, r+Q$ and is independent of the demand that follows. Its measures are therefore base-stock measures averaged over those Q values. **Condition on the position, then average**: that is how chapter 8 built $\bar{B}(r, Q)$ from G^1, and it is the move this chapter keeps making.

Example 10.1 (The hub of the running example)**.** The system is the one of Section 10.2, so the modeling questions have the answers given there.

The question. How often does the hub have stock, how much does it owe on average, and how much does it hold?

Notation. Let $D_0(L_0)$ represent the hub's demand over its three-week lead time, Poisson with mean $\theta_0 = \lambda_0 L_0 = 4 \times 3 = 12$ units. Let G^1 and G^2 represent its first and second order loss functions, read from Table D.4 and Table D.5 at mean 12.

The computation. With $(r_0, Q_0) = (9, 8)$, Equation 8.18 gives

$$\bar{B}_0 = \frac{G^2(9) - G^2(17)}{8} = \frac{8.3910 - 0.1726}{8} = 1.0273 \text{ units},$$

Equation 8.19 gives

$$\overline{RR}_0 = 1 - \frac{G^1(9) - G^1(17)}{8} = 1 - \frac{3.3212 - 0.1451}{8} = 0.6030,$$

and Equation 8.20 gives $\bar{I}_0 = (8+1)/2 + 9 - 12 + 1.0273 = 2.5273$ units.

The check. On-hand less backorders must be the average position less the mean lead time demand: $2.5273 - 1.0273 = 1.5000$, and $9 + 4.5 - 12 = 1.5$. That check is worth performing every time, because it catches most errors in reading a loss table at the wrong index.

Interpretation. The hub has stock only 60% of the time and holds about two and a half cutouts on average. That looks thin for a central warehouse, and it is deliberate. The hub carries no shortage cost of its own; its shortages matter only through the storerooms, and each storeroom holds three units of its own against them. Section 10.6 shows that this pair of policies is the exact optimum for $Q_0 = 8$.

Validation against a closed form, and its limit. At $Q_0 = 1$ the hub would run base stock, and Equation 8.18 would collapse to Equation 8.10 at $S_0 = r_0 + 1$: chapter 9's depot. The average over eight positions here is eight such depots, one for each place the position can sit. What has to break for this section to stop applying is the Poisson stream at the hub. The moment a storeroom orders more than one unit at a time, its requests are no longer its customers, and the hub is no longer a chapter 8 location.

10.4.1. Where This Leaves Us

The hub is solved. What it owes the storerooms is not yet a wait, and a storeroom needs a wait, since the wait is part of its lead time.

10.5. The Delay the Hub Imposes

10.5.1. The Mean

Recall from Section 9.6.2 that Little's law turns a supplier's backorders into a delay. Nothing in the argument used one-for-one ordering at the supplier, so it applies to the hub as it stands:

$$\bar{W} = \frac{\bar{B}_0}{\lambda_0}. \tag{10.2}$$

For the running example that is $1.0273/4 = 0.2568$ weeks, or $0.2568 \times 7 = 1.80$ days. Chapter 9's warning callout carries over word for word: this is the average over every request, including the 60% filled at once.

10.5.2. The Distribution

A storeroom's lead time is transit plus the wait, and a random lead time needs more than its mean. The distribution of the wait comes from one argument, the **lead-time shift** of van der Wal (1993).

Suppose the hub's supplier were w weeks faster. A request that waits at most w weeks in the real system is exactly a request that would be filled at once in that faster one. And a request is filled at once when the hub's position, one lead time before it arrives, covered all the demand since, the request's own unit included. In the faster system that lead time is $L_0 - w$. The position is uniform on $r_0 + 1, \ldots, r_0 + Q_0$ and independent of the demand after it, so

$$P(W \le w) = \frac{1}{Q_0} \sum_{u=r_0+1}^{r_0+Q_0} P(D_0(L_0 - w) \le u - 1), \qquad 0 \le w < L_0, \tag{10.3}$$

and $P(W \le L_0) = 1$.

Read Equation 10.3 against Equation 8.19 and something useful appears. At $w = 0$ it is the ready rate. At any other w **it is the ready rate the hub would have with a lead time w shorter.** That is the lead-time shift written as arithmetic, and it is how the workbook computes the whole distribution, one formula to a row.

The argument needs three things, and the text states them because each can fail. Requests are filled first come, first served. Deliveries from the supplier do not overtake one another. And $r_0 \geq 0$: with a negative reorder point a request can wait for an order that is placed after it arrives, and the shift no longer holds.

Example 10.2 (The wait at the running example's hub)**.** *The computation.* At $w = 0$ the window is the full lead time, and $D_0(3)$ is Poisson with mean 12. The positions are $u = 10, \ldots, 17$, so the entries of Table D.3 needed are $G(9)$ through $G(16)$:

$$P(W = 0) = \tfrac{1}{8}(0.2424{+}0.3472{+}0.4616{+}0.5760{+}0.6815{+}0.7720{+}0.8444{+}0.8987) = 0.6030.$$

At $w = 1$ the mean is $4 \times 2 = 8$, and the same eight entries of the mean 8 column average to 0.9117. At $w = 2$ the mean is 4, and they average to 0.9985.

The check. $P(W = 0)$ is the ready rate of Example 10.1, 0.6030, as it must be. A second check integrates the tail: the area under $P(W > w)$ from 0 to L_0 is the mean wait, and a day-wide grid in the workbook gives 0.2575 against the 0.2568 of Equation 10.2.

Interpretation. Six requests in ten ship at once; nine in ten within a week; almost none waits past two weeks. The mean of 1.8 days hides that shape. It is the average of a wait that is usually zero and occasionally a week or more.

10.5.3. The Second Moment

Little's law has a distributional form, due to Bertsimas and Nakazato (1995), that relates **factorial** moments when arrivals are Poisson and service is first come, first served: $E[B_0(B_0-1)] = \lambda_0^2 E[W^2]$. Subtracting the square of the mean,

$$Var[W] = \frac{Var[B_0] - \bar{B}_0}{\lambda_0^2}, \tag{10.4}$$

and $Var[B_0]$ is Equation 8.21, which chapter 8 built for exactly this use. Notice the subtraction. The variance of the wait is not the variance of the backorder level scaled; it is that variance less the mean. For the running example Equation 10.4 gives 0.1825 weeks squared, and integrating $2wP(W > w)$ from Equation 10.3 gives the same figure to nine decimals. Thus, the two routes check each other.

10.5.4. Where This Leaves Us

The wait is known exactly, its whole distribution and not only its mean. Section 10.6 turns it into the storeroom's measures, three ways, and measures the three against an exact answer.

10.6. Exact Storeroom Measures

10.6.1. The One-for-One Recursion

Begin with both levels ordering one unit at a time, the setting of Chapter 9. Axsäter (1990) gave a recursion for its exact cost, and Axsäter (1993) summarized it in an appendix that extends it to negative positions. Let $C(S_0, S_j)$ represent the system's cost per week when the hub's position is

held at S_0 and each storeroom's at S_j, counting holding at both levels and shortage at the storerooms.

The recursion rests on tracking a single unit. A unit the hub orders is used to fill the S_0th hub demand after the order, and at the storeroom it fills the S_jth demand after that storeroom's order. Its cost depends on when those demands arrive, and Poisson demand makes that computable. Let π^{S_j} represent the cost per unit at a storeroom whose supplier never runs out; it is chapter 8's base-stock cost per unit at lead time O_j. Let $\Pi^{S_j}(S_0)$ represent the storeroom's cost per unit when the hub's position is S_0, and $\gamma(S_0)$ the hub's holding cost per unit. Then $C(S_0, S_j) = \lambda_0(\Pi^{S_j}(S_0) + \gamma(S_0))$, and Π satisfies, for $S_0 > 0$,

$$\Pi^{S_j}(S_0 - 1) = \frac{\lambda_j}{\lambda_0}\Pi^{S_j-1}(S_0) + \frac{\lambda_0 - \lambda_j}{\lambda_0}\Pi^{S_j}(S_0) + \frac{\lambda_j}{\lambda_0}(1 - P(D_0(L_0) \geq S_0))(\pi^{S_j} - \pi^{S_j-1}), \tag{10.5}$$

started from $\Pi^{S_j}(S_0) = \pi^{S_j}$ at a hub position so high the hub never runs out, and from closed forms at $S_j \leq 0$. The chapter does not derive Equation 10.5; Axsäter (1990) does, and the interested reader should refer to it. What matters here is that it is exact and that it runs down a column, which is what the workbook's `Exact` sheet does.

The costs give the stock levels as well. Every term of the recursion is linear in the three cost rates. Thus, run it with the storerooms' shortage cost set to one and both holding costs to zero, and the "cost" it returns is the storerooms' expected backorders. The same trick returns either location's expected on-hand.

10.6.2. The Hub Batches

Now let the hub order Q_0 while the storerooms stay one for one. A hub order of Q_0 is Q_0 units, and the kth of them fills the $(r_0 + k)$th hub demand after the order, exactly as if the hub ran base stock at $S_0 = r_0 + k$.

Averaging over the units of a batch is averaging over the hub's positions, and Axsäter (1993)'s equation (1) is

$$C = \frac{1}{Q_0} \sum_{S_0=r_0+1}^{r_0+Q_0} C(S_0, S_j). \tag{10.6}$$

It is the chapter's move a second time: condition on the hub's position, then average.

Example 10.3 (Exact measures for the running example)**.** *The computation.* With $S_j = 3$, the one-for-one costs at the eight hub positions $S_0 = 10, \ldots, 17$ are 7.2105, 6.7516, 6.5493, 6.5587, 6.7352, 7.0373, 7.4296 and 7.8837 dollars a week, and their average is $56.1559/8 = \$7.0195$ a week. Run with the backorder-only rates, the same recursion gives 0.2906 backorders across the four storerooms, 0.0726 at each.

The check. The hub's expected on-hand by the recursion, 2.5273, is the $\bar{I}_0$ of Example 10.1 computed an entirely different way.

Interpretation. Searching every r_0 from -4 to 16 and every S_j from 0 to 6 with $Q_0 = 8$ finds $(9, 3)$ the cheapest, which is why the running example was set there. Each storeroom owes its crews 0.07 of a cutout on average, which at one failure a week is about one crew-week of waiting in fourteen weeks.

10.6.3. Three Routes to a Storeroom's Lead Time Demand

The storeroom's lead time is $O_j + W$, and chapter 8 already handles a random lead time. There are three ways to feed it.

1. **The mean wait only.** Treat the lead time as $O_j + \bar{W}$, so the storeroom's lead time demand is Poisson with mean $\lambda_j(O_j + \bar{W})$. This is METRIC's move, Section 9.7.

2. **Two moments.** Fit a negative binomial to the mean $\lambda_j(O_j + \bar{W})$ and the variance $\lambda_j(O_j + \bar{W}) + \lambda_j^2 \, Var[W]$, case 3 of Table 8.3. This is VARI-METRIC's move, Section 9.8.
3. **Conditioned on the wait.** For each value w the wait can take, the lead time demand is Poisson with mean $\lambda_j(O_j + w)$; average those over the wait's distribution. This is the chapter's move a third time.

Table 10.2 prices the three against the exact answer of Example 10.3.

Table 10.2.: A base-stock storeroom's expected backorders by three routes, against Axsäter's exact figure. Notice that the route that carries the whole distribution of the wait is not an approximation at all.

Route	Backorders per storeroom	Against exact
Exact, Equation 10.6	0.0726	
Conditioned on the wait	0.0726	exact
Two moments	0.0712	2% low
The mean wait only	0.0505	30% low

Conditioning on the wait is exact here, and the reason is worth stating. With first-come-first-served filling and $r_0 \geq 0$, the wait a request suffers is fixed by what happened before it arrived: the hub's position, its orders outstanding, and the queue ahead of the request. The storeroom's customers after the request are independent of all of that. So each unit's cost is the cost at lead time $O_j + w$, averaged over the wait, which is what route 3 computes. Notice what that says about the linked waits of Table 10.1. They are real, but for a base-stock storeroom they do not matter; what matters is that each unit's wait is independent of the demand it will meet.

Read the other two rows the way Section 9.8 read them. Little's law gets the mean right, and the mean alone understates the storeroom's backorders

by 30%, because a wait that is usually zero and occasionally long hurts more than a steady wait of the same mean. Carrying the variance recovers nearly all of it.

10.6.4. Where This Leaves Us

With one-for-one storerooms everything is exact, and the chapter's own route agrees with Axsäter's. Section 10.7 lets the storerooms batch, which is where Axsäter's recursion does its real work.

10.7. When the Storerooms Batch Too

10.7.1. One Storeroom

Let a single storeroom run (r_j, Q_j), and let the hub order Q_w storeroom batches at a time with a reorder point of R_w batches. Axsäter (1993) counts the hub in batches, and so, where his results are used, does this section: $Q_0 = Q_w Q_j$ units and $r_0 = R_w Q_j$.

A hub order is Q_w sub-batches of Q_j units. The storeroom placed the order that triggered the hub's, so its position was just reset to $r_j + Q_j$, and its next orders come after exactly Q_j, $2Q_j$, … further demands. Thus the ith sub-batch is released against system demand iQ_j, and its units fill the storeroom's $(r_j + 1)$th to $(r_j + Q_j)$th demands. Averaging,

$$C = \frac{1}{Q_w Q_j} \sum_{i=R_w+1}^{R_w+Q_w} \sum_{k=r_j+1}^{r_j+Q_j} C(iQ_j, k). \tag{10.7}$$

Example 10.4 (One batching storeroom behind the hub)**.** A storeroom whose crews draw two cutouts a week orders $Q_j = 3$ at a time, a week

from a hub that orders $Q_w = 2$ batches, six units, with a three-week lead time. Costs are the running example's.

The computation. At $(R_w, r_j) = (0, 6)$ the double sum needs the hub positions $iQ_j = 3$ and 6 and the storeroom positions $k = 7, 8, 9$. The six one-for-one costs are 3.6962, 3.1290 and 3.0011 at $S_0 = 3$, and 3.1694, 3.5117 and 3.9591 at $S_0 = 6$, so

$$C = \frac{20.4665}{2 \times 3} = \$3.4111 \text{ a week.}$$

Interpretation. That is the exact optimum, and it holds almost nothing at the hub: $R_w = 0$ reorders when the hub's position reaches zero, and the recursion gives the hub 0.52 cutouts on hand on average against 4.13 at the storeroom. With one storeroom the hub is little more than a pass-through, and stock is worth more a week closer to the crews. Raising the hub's reorder point to one batch moves 1.5 units to the hub, the best storeroom policy falls to $r_j = 3$, and the cost rises to \$3.5354, 3.6% more.

10.7.2. Several Storerooms

With N identical storerooms the ith sub-batch is no longer released at a fixed system demand. Which demand triggers the next storeroom order depends on where the other storerooms' positions sit. Axsäter (1993) lets $p_{i,j}$ represent the probability that the ith system demand after a hub order triggers the jth storeroom order, and his equation (6) replaces the fixed iQ_j of Equation 10.7 with an average over $p_{i,j}$.

The idea behind $p_{i,j}$ is the chapter's move a third time. The storeroom that triggered the hub's order has just reset its position. **The other $N-1$ have positions equally likely to sit anywhere in their ranges**, and a storeroom whose position is uniform on Q_j values orders its jth time after i demands with probability k/Q_j or $1 - k/Q_j$ for the appropriate k: that is his equation (7). Splitting the system's demands binomially among the

storerooms and convolving gives the rest. The recursion is in the software; the chapter gives its structure, not its algebra.

Axsäter (1993) also offers three approximations. The first releases the ith sub-batch at system demand iQ_j, as if there were one storeroom; the second lets each demand trigger an order with probability $1/Q_j$, independently; the third weights them by $1/N$ and $(N-1)/N$. On the 32 test problems of Svoronos and Zipkin (1988) he reports all three close to exact, the third closest. For example, on his problem 4 the exact cost is 17.07 and the three approximations give 15.92, 17.33 and 16.98.

10.7.3. Where Exactness Ends

Table 10.3.: How far exact evaluation reaches. Every exact case assumes constant lead times and demand with no other source at the hub.

Case	Status	Source
Hub batches, storerooms one for one, Poisson	Exact	Axsäter (1993), equation (1)
Both batch, one storeroom	Exact	Axsäter (1993), equation (2)
Both batch, identical storerooms, Poisson	Exact	Axsäter (1993), equation (6)
Both batch, two storerooms that differ	Exact	Axsäter (1998)
Both batch, compound Poisson demand	Exact	Axsäter (2000)
The hub's demand as a renewal stream of batches	Approximate	Deuermeyer and Schwarz (1981), Svoronos and Zipkin (1988)

Table 10.3 is the boundary as the literature draws it. Notice the footnote to every exact row: no demand at the hub except the storerooms', and no random lead time. Those two are exactly what a real hub has, and they are the subject of Part II.

10.7.4. Where This Leaves Us

Axsäter's recursion is the chapter's yardstick. Part II builds a route that reaches past the boundary, and wherever the two overlap they agree exactly, which is how the reader will know the route is right.

10.8. Orders in a Window

Let the storeroom order Q_j at a time. The hub no longer sees its customers. It sees one order of Q_j each time the customers carry the storeroom's position down through r_j, and the question this section answers is how many such orders fall in a window of time.

The storeroom's orders are its customers' demand, counted in whole batches. Let u represent how far the storeroom's position sits above r_j when a window opens, so u is one of $1, \dots, Q_j$. Let D represent the customers' demand in the window and N the number of orders the storeroom places in it. The first order comes after u units of demand, and each later one after Q_j more, so

$$N \geq n \iff D \geq u + (n-1)Q_j.$$

Recall from Section 8.5.1 that u is equally likely to take each of its Q_j values. Averaging over them, which is the chapter's move again,

$$P(N \geq n) = \frac{1}{Q_j} \sum_{y=(n-1)Q_j+1}^{nQ_j} P(D \geq y), \qquad n \geq 1. \tag{10.8}$$

In other words, the chance of at least n orders is the average of Q_j consecutive entries of the demand's tail. With Poisson customers that is chapter 8's table and nothing else.

Two checks come with Equation 10.8, and both should be run. At $Q_j = 1$ it is $P(D \geq n)$: the hub sees the customers, the case of Part I. And $Q_j E[N] = E[D]$: on average the storeroom passes up exactly its customers' demand. Notice what Equation 10.8 is in Axsäter (1993)'s terms. His equation (7) is the same law counted in demands rather than in time, and mixing it over a Poisson number of demands gives Equation 10.8.

Seen from one of the storeroom's own orders, the window looks different. A window that ends at an order is not a random moment for that storeroom. Its position has just reached r_j, so u is known, not uniform. Let D' represent the customers' demand in the window before the demand that triggered the order. It is still Poisson, since demand has independent increments, and the storeroom's earlier orders in the window number $\lfloor D'/Q_j \rfloor$. That is the whole difference, and Section 10.10 needs it.

Example 10.5 (Counting a storeroom's orders)**.** A storeroom's crews draw two cutouts a week and it orders $Q_j = 3$ at a time.

The question. How many orders does the hub receive from it in three weeks?

Notation. D is Poisson with mean $2 \times 3 = 6$. From Table D.3, $P(D \geq y) = 1 - G(y-1)$ is 0.9975, 0.9826, 0.9380 for $y = 1, 2, 3$; 0.8488, 0.7149, 0.5543 for $y = 4, 5, 6$; 0.3937, 0.2560, 0.1528 for $y = 7, 8, 9$; and 0.0839, 0.0426, 0.0201 for $y = 10, 11, 12$.

The computation. Averaging in threes, Equation 10.8 gives $P(N \geq 1) = 0.9727$, $P(N \geq 2) = 0.7060$, $P(N \geq 3) = 0.2675$ and $P(N \geq 4) = 0.0489$. Differencing, $P(N = 0) = 0.0273$, $P(N = 1) = 0.2667$, $P(N = 2) = 0.4385$ and $P(N = 3) = 0.2186$, with 0.0489 left for four or more.

The check. Carrying the small masses for four, five and six orders, 0.0443, 0.0044 and 0.0002, $3E[N] = 3(0.2667 + 2 \times 0.4385 + 3 \times 0.2186 + 4 \times 0.0443 + 5 \times 0.0044 + 6 \times 0.0002) = 3(1.9999) = 6.0$, the customers' mean demand.

Seen from an order. $\lfloor D'/3 \rfloor$ is 0 when $D' \leq 2$, 1 when $3 \leq D' \leq 5$, and so on, so its masses are $G(2) = 0.0620$, $G(5) - G(2) = 0.3837$, $G(8) - G(5) = 0.4015$ and $G(11) - G(8) = 0.1327$. Notice how much more weight sits on few orders. The storeroom has just ordered, so its position is at the top of its range, the farthest it can be from its next order.

Interpretation. The customers' demand over three weeks runs from 0 to a dozen or more. The hub sees 0, 3, 6, 9 or 12 units, with two orders, six units, the most likely. That is the lumpiness, and the tightly bunched count is the regularity.

10.8.1. Working a Batching Storeroom by Hand

The second ledger lets the storeroom batch and lets other demand reach the hub. The hub runs $(r_0, Q_0) = (1, 6)$ with $L_0 = 3$ weeks and starts with 7 on its shelf. The storeroom runs $(r_j, Q_j) = (1, 3)$, a week away, and starts with 4. Its crews' failures fall in weeks 1, 3, 5, 7, 8, 9, 9, 10 and 11, and one unit of other demand reaches the hub in each of weeks 2 and 4. The columns and the posting rule are those of Section 10.3.

Weeks 1 to 4. The failures in weeks 1 and 3 take the storeroom from four to two, and its position with it, but they do not reach the hub at all. The hub's position moves only in weeks 2 and 4, when its other customers take a unit each.

Week 5. A failure takes the storeroom's position to 1, its reorder point, and it orders three. **An order of 3 arrives at the hub at once**, the moment the hub stops seeing customer demand. The hub has five, ships three, and its position falls from 5 to 2.

Week 6. The three arrive, and the storeroom holds four.

The table has been updated as follows.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure	3	0	3	0	7	0	0	7
2	other demand at the hub	3	0	3	0	6	0	0	6
3	failure	2	0	2	0	6	0	0	6
4	other demand at the hub	2	0	2	0	5	0	0	5
5	failure, order of 3 to hub, shipped	1	0	4	3	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
6	storeroom receives 3	4	0	4	0	2	0	0	2

Weeks 7 and 8. Two failures take the storeroom to two. The hub sees neither.

Week 9. A failure takes the storeroom's position to 1, and it orders three. The hub has two on its shelf. **It ships two and owes one**, and the order of 3 units takes the hub's position from 2 to -1, two below its reorder point. The hub orders six, due in week 12, and its position rises to 5. Notice the undershoot. A single order of three carried the hub's position past r_0 in one step, which is what a lumpy demand did in Section 8.11. A second failure in the same week takes the storeroom's last unit.

Week 10. The two shipped units arrive. The storeroom holds two, then a failure takes one.

The table has been updated as follows.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure	3	0	3	0	7	0	0	7

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
2	other demand at the hub	3	0	3	0	6	0	0	6
3	failure	2	0	2	0	6	0	0	6
4	other demand at the hub	2	0	2	0	5	0	0	5
5	failure, order of 3 to hub, shipped	1	0	4	3	2	0	0	2
6	storeroom receives 3	4	0	4	0	2	0	0	2
7	failure	3	0	3	0	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
8	failure	2	0	2	0	2	0	0	2
9	failure, order of 3: 2 shipped, 1 backordered, hub orders 6	1	0	4	2	0	1	6	5
9	failure	0	0	3	2	0	1	6	5
10	storeroom receives 2	2	0	3	0	0	1	6	5
10	failure	1	0	2	0	0	1	6	5

Week 11. A failure takes the storeroom's position to 1 once more, and it orders three. The hub's shelf is empty, so the whole order waits, and the

hub owes four.

Week 12, the payoff row. The hub's six arrive. First in line is the week-9 order's last unit, then the three of week 11, and two go on the shelf. **The week-9 order is complete only now.** Two thirds of it left at once, and it still waited three weeks, because an order is filled when its last unit is. All four units reach the storeroom together in week 13.

The table has been updated as follows, and this is the completed ledger.

Table 10.4.: The second ledger, thirteen weeks of a storeroom ordering three at a time and a hub that also serves other demand. Notice that customer failures in weeks 1 and 3 leave the hub's position untouched, and that the week-9 order ships two units at once and its third three weeks later.

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
1	failure	3	0	3	0	7	0	0	7
2	other demand at the hub	3	0	3	0	6	0	0	6
3	failure	2	0	2	0	6	0	0	6

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
4	other demand at the hub	2	0	2	0	5	0	0	5
5	failure, order of 3 to hub, shipped	1	0	4	3	2	0	0	2
6	storeroom receives 3	4	0	4	0	2	0	0	2
7	failure	3	0	3	0	2	0	0	2
8	failure	2	0	2	0	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
9	failure, order of 3: 2 shipped, 1 backordered, hub orders 6	1	0	4	2	0	1	6	5
9	failure	0	0	3	2	0	1	6	5
10	storeroom receives 2	2	0	3	0	0	1	6	5
10	failure	1	0	2	0	0	1	6	5

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
11	failure, order of 3 to hub, back-ordered	0	0	4	0	0	4	6	2
12	hub receives 6, completes the week-9 order, fills the week-11 order	0	0	4	4	2	0	0	2

Week	Event	I_j	B_j	IP_j	In transit	I_0	B_0	On order	IP_0
13	storeroom receives 4	4	0	4	0	2	0	0	2

The count, checked. The storeroom ordered in weeks 5, 9 and 11, so the three weeks 9 to 11 hold two orders and the three weeks 6 to 8 hold none. Both are values Example 10.5 gives appreciable probability, and the second is the regularity: a storeroom that has just ordered does not order again for a while.

10.8.2. Where This Leaves Us

The storeroom's orders are countable from the Poisson table. Section 10.9 adds them to the hub's other demand and recovers the hub's lead time demand.

10.9. The Hub With Other Demand

A real hub serves more than one storeroom and usually some customers of its own. Let $X_o(\ell)$ represent the other demand in a window of ℓ weeks, Poisson with mean $\lambda_o \ell$, and let $N_j(\ell)$ count storeroom j's orders by Equation 10.8. Then

$$D_0(\ell) = X_o(\ell) + \sum_j Q_j N_j(\ell), \tag{10.9}$$

and the terms are independent, since each storeroom's position is uniform and independent of every other's. The distribution of Equation 10.9 is a convolution: a short table by hand for small numbers, a worksheet column otherwise. Every hub measure then follows from chapter 8's formulas with no change. Notice that nothing here needs the storerooms to be identical. Each adds one term.

The hub's position is uniform on $r_0 + 1, \dots, r_0 + Q_0$ provided some of its demand comes one unit at a time. When its only demand is storeroom orders of a common size Q_j, the position moves in steps of Q_j and is uniform on the multiples of Q_j in that band instead. That is why Axsäter (1993) counts the hub in batches.

10.9.1. A Two-Moment Recipe

For realistic volumes you may want two moments and chapter 8's family selection rather than a convolution. The mean is $\lambda_0 \ell$. For the variance, write the storeroom's orders over a window as its customers' demand plus the change in its position: if U_0 and U_ℓ are how far the position sits above r_j at the start and the end, then $Q_j N_j = D + U_\ell - U_0$. Over a long window the two positions are independent and uniform on $1, \dots, Q_j$, with variance $(Q_j^2 - 1)/12$ each. Thus

$$Var[D_0(\ell)] \approx \lambda_o \ell + \sum_j \Big(\lambda_j \ell + \frac{Q_j^2}{6} \Big), \tag{10.10}$$

where the exact long-window term for one-unit customers is $(Q_j^2 - 1)/6$ and Equation 10.10 uses its continuous version, Q_j^2 times the average of $x(1-x)$ for x uniform on $[0, 1]$. The difference is a sixth of a unit squared, and the recipe errs on the safe side.

The shortcut to warn against treats each storeroom's orders as Poisson arrivals of Q_j units at rate λ_j / Q_j. That gives the storeroom's term the variance $\lambda_j Q_j \ell$, Q_j times its customers' own. The storeroom's orders are

not random in that way. It orders once per Q_j units of demand, nearly on a schedule, and the shortcut charges the hub for a randomness that is not there.

Example 10.6 (A hub with other demand)**.** The hub now serves two cutouts a week of its own customers and the storeroom of Example 10.5. It runs $Q_0 = 8$ with $L_0 = 3$ weeks.

What is stocked? The cutout, at the hub and one storeroom. *What is the demand process?* Poisson at two units a week at the storeroom and two at the hub, one unit per customer. *When is inventory reviewed?* Continuously. *What triggers replenishment, and how much?* At the storeroom, its position reaching $r_j = 3$, for $Q_j = 3$; at the hub, its position reaching r_0, for $Q_0 = 8$. *What happens to unmet demand?* It is backordered at both levels. *What costs are incurred, and when?* For this example, the hub is priced as a chapter 8 location with h and b per week on its own on-hand and backorders.

Notation. $X_o(3)$ is Poisson with mean 6, and $3N(3)$ has the masses of Example 10.5 on 0, 3, 6, 9 and 12. $D_0(3) = X_o(3)+3N(3)$ by Equation 10.9.

The computation. The convolution has mean 12 and variance 13.333. Its masses peak at $P(D_0 = 11) = 0.1083$ and $P(D_0 = 12) = 0.1087$. The recipe gives $6 + 6 + 9/6 = 13.5$, and the shortcut $6 + 2 \times 3 \times 3 = 24$. With $Q_0 = 8$, the reorder point that minimizes the hub's holding and backorder cost is 13 under the exact convolution, 13 under the recipe, and 15 under the shortcut. Priced under the exact convolution, the first two cost \$4.4054 a week and the third \$4.6533.

The check. The mean is $\lambda_0 L_0 = 4 \times 3 = 12$, and the exact variance exceeds the customers' $6 + 6 = 12$ by 1.333, which is $(3^2 - 1)/6$.

Interpretation. The shortcut believes the hub faces nearly twice the variance it does, holds two cutouts too many, and costs 5.6% more. The recipe is within a sixth of a unit squared and chooses the same reorder point as the exact convolution. Notice that the shortcut is not an eccentric one. It is

the natural reading of "orders of Q_j arriving at rate λ_j/Q_j", and it is the one to check for in any planning system.

Validation, and its limit. At $Q_j = 1$ the convolution is Poisson with mean 12, the hub of Part I. The recipe's error stays bounded as the window grows but not as it shrinks: over a window holding one order or none, the rounding term is far from its long-window value, and the convolution is the thing to use.

10.9.2. Where This Leaves Us

The hub's lead time demand is known exactly, and closely by two moments. What a storeroom needs from the hub is a wait, and a storeroom's order waits differently from a customer's unit.

10.10. The Storeroom Behind a Batching Hub

10.10.1. The Order's Wait

The lead-time shift still applies. An order of Q_j units placed at time A is complete by $A + w$ exactly when, with the supplier w faster, it would have been complete at A. That happens when the hub's position $\ell = L_0 - w$ weeks earlier covered all the demand since, the order's own Q_j units included. Three things differ from a customer's single unit, and each is visible in Table 10.4. The order needs Q_j units, not one. The other demand and the other storerooms are seen as a random moment sees them. And the storeroom's own earlier orders in the window are seen from the order, so they number $\lfloor D'_j(\ell)/Q_j \rfloor$. Thus, with u averaged over the hub's

positions,

$$P(W_j \le w) = \frac{1}{Q_0} \sum_{u=r_0+1}^{r_0+Q_0} P\Big(X_o(\ell) + \sum_{i \ne j} Q_i N_i(\ell) + Q_j \lfloor D'_j(\ell)/Q_j \rfloor \le u - Q_j\Big), \qquad \ell = L_0 - w, \tag{10.11}$$

for $0 \le w < L_0$. At $Q_j = 1$ it reduces to Equation 10.3.

Conditioning on this wait is exact, for the reason of Section 10.6.3. The order's wait is fixed by the history up to its placement, and the storeroom's customers afterwards are independent of that history. The kth unit of the order fills the $(r_j + k)$th demand after it, so each unit's cost is chapter 8's at lead time $O_j + W_j$, averaged over W_j. The code checks this against Axsäter (1993) wherever both apply: 144 policies with one storeroom and 96 with four identical storerooms agree with his equations (2) and (6) to six decimals or better.

The order's own view is not a refinement. Using a customer's wait in its place treats the order as arriving at a random moment, and on one of those test systems it overstates the cost by 14%.

Example 10.7 (The storeroom of the hub with other demand)**.** The system is that of Example 10.6, with the hub at $(r_0, Q_0) = (12, 8)$ and the storeroom at $(r_j, Q_j) = (3, 3)$, one week from the hub.

The computation. A customer of the hub's own waits nothing with probability 0.8228, which is the hub's ready rate, and at most a week with probability 0.9797; its mean wait is 0.0874 weeks. The storeroom's order waits nothing with probability 0.7626, at most a week with probability 0.9668, and 0.1264 weeks on average, with a variance of 0.0894. Conditioning the storeroom's lead time demand on that wait gives 0.0823 backorders; two moments give 0.0783, 5% low, and the mean alone 0.0559, 32% low.

The check. At $Q_j = 1$ the order's wait and a customer's coincide; the code asserts it.

Interpretation. The storeroom's orders wait 45% longer on average than the hub's own customers. Each needs three units rather than one, and each arrives just after a draw of three the hub has had no time to recover from. With the batches held at $Q_0 = 8$ and $Q_j = 3$, the pair $(r_0, r_j) = (12, 3)$ is the cheapest for the whole system, at \$5.6675 a week, counting the hub's customers' waits at b.

10.10.2. Where This Leaves Us

For customers who take one unit each, constant lead times and $r_0 \geq 0$, the chapter's route is exact, and it reaches past Axsäter's boundary to other demand and unlike storerooms. What it has assumed so far is a supplier that delivers in exactly L_0 weeks.

10.11. Random Lead Times

Let the supplier's lead time L_0 take values $\ell_1, \dots, \ell_K$ with probabilities $p_1, \dots, p_K$. **Condition on the lead time, then average**: build the hub's lead time demand at each value by Equation 10.9, and average the distributions,

$$P(D_0(L_0) \leq x) = \sum_k p_k \, P(D_0(\ell_k) \leq x). \tag{10.12}$$

The measures average the same way, since each is an expectation.

The order matters. Both streams are counted over the same lead time, so they are independent only once its value is known. Averaging each stream over the lead time on its own and then adding them loses the covariance a shared lead time creates, $2\lambda_o\lambda_j \mathrm{Var}[L_0]$, and understates the variance.

The wait averages the same way. When lead times vary without orders overtaking one another and independently of demand, which is how Zipkin

(1986) describes a random lead time and the condition Section 8.5.4 already imposes, the lead-time shift gives

$$P(W \le w) = \sum_k p_k \, P(W \le w \mid L_0 = \ell_k). \tag{10.13}$$

Example 10.8 (A supplier that takes two, three or five weeks)**.** The system of Example 10.7, but the supplier now takes two, three or five weeks with probabilities 0.4, 0.4 and 0.2. The mean is still three weeks, and $\mathrm{Var}[L_0] = 0.4(1) + 0.4(0) + 0.2(4) = 1.2$ weeks squared.

The computation. Built at each value and averaged, the hub's lead time demand has mean 12 and variance 32.535; averaged one stream at a time, 22.935. At $(r_0, Q_0) = (12, 8)$ the hub's ready rate falls from 0.8228 to 0.7656 and its backorders rise from 0.3497 to 0.9948. The storeroom order's wait now has mean 0.3037 weeks and variance 0.4396. Planned on the mean wait alone, the storeroom chooses $r_j = 3$; planned on two moments or on the whole distribution, $r_j = 4$. Priced by conditioning, the first owes 0.239 on average at \$2.7771 a week, the second 0.143 at \$2.7472.

The check. The two variances differ by $32.535 - 22.935 = 9.6$, and $2 \times 2 \times 2 \times 1.2 = 9.6$.

Interpretation. The supplier's variability multiplies the variance the hub covers by two and a half and carries down to the storeroom through the wait. Planned on the mean wait, the storeroom owes its crews 67% more than it needs to, at a cost only 1.1% higher; both numbers are worth stating, because a utility that watches cost will not notice what its crews do. The mean-only error grows with the supplier's spread, and the exercises show a wider supplier costing 4.7%.

What to measure, and where the model stops. When a supplier delivers in the sequence orders were placed, the lead time to use is the time from placement to receipt, order by order, since that is what Equation 10.12 conditions on. With data, a handful of values at its quantiles, each with

equal weight, is enough; with only a mean and a standard deviation, the values must come from a family you choose, and you should say so. **When orders overtake one another**, the model over-protects, as Section 8.5.4 warned, and correcting for it is beyond this chapter.

10.11.1. Where This Leaves Us

Part II can evaluate any pair of policies. It has not said how to choose them, and choosing them is Part III.

10.12. Setting Both Policies

Everything needed to evaluate two locations is in place, and none of it can yet choose a policy. The storeroom's policy needs the hub's, through the wait. The hub's policy needs the storeroom's, through the orders it passes up. Neither can be computed first.

10.12.1. What Each Location Minimizes

Each location minimizes chapter 8's cost,

$$C_i(r_i, Q_i) = \frac{k_i \lambda_i}{Q_i} + h\,\bar{I}_i(r_i, Q_i) + b_i\,\bar{B}_i(r_i, Q_i), \tag{10.14}$$

with the storeroom's b_j the shortage cost of its crews. **The hub has no shortage cost of its own**, as Section 10.2 said, yet Equation 10.14 needs one. Recall from Chapter 4 that a service constraint and a penalty are the same thing seen from two sides. Thus the hub's b_0 is set from a **ready rate target** γ_0 for the hub, by Equation 8.33, which is the critical ratio of Equation 8.15 read backward:

$$\frac{b_0}{b_0 + h} = \gamma_0 \quad \Longleftrightarrow \quad b_0 = \frac{\gamma_0\, h}{1 - \gamma_0}. \tag{10.15}$$

This chapter uses $\gamma_0 = 0.8$, so $b_0 = 4h = \$2.2115$ per unit per week. It is a stated input, and Section 10.13 prices it.

Notice what the target is and what it is not. It is the ready rate a base-stock hub facing single-unit Poisson demand would run at with this b_0. That is, it is a device for pricing b_0, not a service level the hub is held to: the ready rate the hub actually reaches is whatever its (r_0, Q_0) delivers. Nor is it a fill rate. The hub's customers include storeroom orders of Q_j units, and recall from Section 8.11.4 that demand arriving in lots is filled at a lower rate than the ready rate.

10.12.2. The Iteration

The way out is the one Chapter 4 took for a different coupling: **hold one side fixed, solve the other, and alternate.** Guess the wait at the hub. With a guess, the storeroom's lead time is known, so the storeroom is a chapter 8 location. With the storeroom's policy, the hub's demand is known, so the hub is one too. Solving the hub gives a wait, which replaces the guess.

Algorithm 10.1 Algorithm TwoLevel. Step 1 is case 3 of Table 8.3 or Section 10.6.3, step 2 is Equation 10.10 or Equation 10.9 with Equation 10.15, and step 3 is Equation 10.2 with Equation 10.4, or Equation 10.11.

```
twoLevel(start):

    // Step 0, the guess: the hub never makes anyone wait
    E[W] <- 0,  Var[W] <- 0
    previous <- none

    loop:
        // Step 1, the storeroom against the wait
        lead time demand at the storeroom: mean lambda_j (O_j + E[W]),
            variance lambda_j (O_j + E[W]) + lambda_j^2 Var[W],
            or conditioned on the wait's distribution
        (r_j, Q_j) <- optimize, chapter 8

        // Step 2, the hub against the storeroom's orders
        lead time demand at the hub: mean lambda_0 L_0,
            variance by the recipe, or the exact convolution
        (r_0, Q_0) <- optimize, chapter 8, with b_0 from gamma_0

        // Step 3, the wait the hub's policy implies
        E[W]   <- Bbar_0 / lambda_0
        Var[W] <- (Var[B_0] - Bbar_0) / lambda_0^2
            or the order's wait by the lead-time shift

        // Step 4, stop when the four integers repeat
        if (r_j, Q_j, r_0, Q_0) == previous: return them
        previous <- (r_j, Q_j, r_0, Q_0)
```

Read step 0, because it is the only guess in the procedure. A wait

of zero says the hub always has stock, the assumption a planner makes by default when modeling a location alone. The first pass is therefore the uncoupled planner's answer, and every pass after it is a correction.

And read step 4, because it is weaker than it looks. The test is that the four integers stopped moving, not that any optimality condition holds. The iteration finds a **fixed point of a heuristic**: a pair of policies each optimal given the other. Nothing shows the pair is the best pair, and a fixed point of a biased map is a biased answer that has stopped moving.

Each of steps 1 to 3 can be done two ways, and Algorithm 10.1 names both. The **moment route** uses the recipe at the hub and two moments at the storeroom; it works for any customers. The **exact route** uses the convolution, the order's own wait and conditioning; it needs customers who take one unit each.

Example 10.9 (Both policies for the hub with other demand)**.** The system of Example 10.7, but now both batches are chosen. Both locations pay $k = \$82.50$ an order, and the hub's target is $\gamma_0 = 0.8$.

The trace, by the moment route.

Table 10.5.: The iteration on the hub with other demand. Pass 1 assumes the hub never delays the storeroom, and passes 2 and 3 correct that. The policies stop moving at pass 2, and pass 3 is the confirmation step 4 requires.

Pass	Store-room (r_j, Q_j)	Store-room cost	Hub (r_0, Q_0)	Hub cost	$E[W]$, weeks	σ_W
1	$(-1, 26)$	\$13.09	$(2, 47)$	\$20.99	0.5634	1.5988
2	$(0, 29)$	\$14.35	$(2, 48)$	\$21.71	0.6166	1.8033
3	$(0, 29)$	\$14.65	$(2, 48)$	\$21.71	0.6166	1.8033

Reading the trace. Pass 1 is the planner who ignores the hub: with no wait, the storeroom holds a reorder point of -1 against a transit of one week. The hub's answer to its orders implies a wait of 0.56 weeks with a standard deviation of 1.6, and pass 2 raises the storeroom's reorder point by one and its batch by three. Pass 3 changes nothing. Notice that the storeroom's cost at pass 3 differs from pass 2 although its policy does not. Each pass prices the storeroom against the wait the pass before it left, so pass 2 pays against a mean wait of 0.56 weeks and pass 3 against 0.62.

The exact route reaches hub $(2, 48)$ and storeroom $(3, 28)$, also in three passes. Its storeroom holds three more units, because its orders of 28 wait for their last unit, 1.75 weeks on average, where the moment route carries the per-unit wait of 0.62.

Priced exactly. The chapter's exact route can evaluate any policy for this system, and it prices the moment route's answer at \$34.35 a week and the exact route's at \$33.54.

10.12.3. Where This Leaves Us

The iteration converges quickly and to an answer that does not depend on where it starts. Whether that answer is any good is a separate question, and the chapter can answer it exactly.

10.13. Validating the Iteration

An iteration that has never been checked against an exact answer is an iteration the reader is asked to trust. This chapter does not have to ask. In Axsäter's setting there is an exact cost, and with one-unit customers the chapter's own route gives one everywhere.

Part I, with both batches held. With $Q_0 = 8$ and the storerooms one for one, every route of Algorithm 10.1 settles on $(r_0, S_j) = (11, 3)$.

Axsäter's recursion prices it at \$7.4337 a week, against \$7.0195 at the exact optimum $(9, 3)$: 5.9% more.

One batching storeroom. With $Q_j = 3$ and the hub's batch held at two storeroom batches, the fixed point has the hub reordering at one batch and the storeroom at $r_j = 3$, which Example 10.4 priced at \$3.5354, 3.6% above an optimum that keeps the hub nearly empty.

Part II, both batches chosen. The best policy the exact cost finds nearby is hub $(2, 43)$ and storeroom $(4, 26)$, at \$33.3955 a week. The exact route's fixed point is within 0.4% of it, and the moment route's within 2.9%.

Read the three gaps in domain terms. In each case the fixed point holds more at the hub than the exact optimum does. The exact optima run the hub at a low ready rate, 0.60 in Part I, because a unit at the hub protects every storeroom only a little and a unit at a storeroom protects its own crews a lot. A hub target of 0.8 already asks the hub for more protection than the system wants, and a higher target would ask for more still. Thus the target is the input to choose with care; the route matters less. That is the one comparison this chapter makes, and it is the one to repeat on your own items whenever an exact cost is available.

10.13.1. Where This Leaves Us

The iteration is trustworthy where it can be checked. Section 10.14 uses it where it cannot.

10.14. The Cutout in Job Lots

The cutout of Section 8.11 is drawn in job lots, and nothing exact in Parts I and II reaches it. The storeroom's orders are still its customers' demand in whole batches, but a lot can carry the position past the reorder point,

and the undershoot varies. The moment route of Algorithm 10.1 needs only two moments at each location, and those are available.

Example 10.10 (Both policies for the cutout in job lots)**.** A storeroom's crews draw cutouts in lots of 1, 2 or 4 units with probabilities 0.40, 0.40 and 0.20, the lots of Section 8.11. Lots arrive at the storeroom at 150 a year, and the hub serves 250 lots a year of its own from the same distribution. The hub's supplier takes two months and the transfer to the storeroom one month. Both locations pay \$82.50 an order and \$28.75 a unit a year to hold; the storeroom's crews are owed at \$287.50, and the hub's target is 0.8, so $b_0 = \$115$ a year. This example works in years.

What is stocked? The cutout, at the hub and one storeroom. *What is the demand process?* Compound Poisson at both locations, with lots of mean $E[Y] = 2.0$ and $E[Y^2] = 5.2$, so 300 units a year at the storeroom and 500 of the hub's own. *When is inventory reviewed?* Continuously. *What triggers replenishment, and how much?* Each location's position reaching its reorder point, for its batch. *What happens to unmet demand?* It is backordered. *What costs are incurred, and when?* Ordering, holding and shortage as stated.

The moments. A lot is at most four units and the storeroom's batch will be near fifty, so every storeroom order is exactly Q_j, and undershoot never changes an order's size. At the hub, Equation 10.10 with compound Poisson customers gives mean $800 \times 2/12 = 133.33$ units and variance $(250+150)(2/12)(5.2)+Q_j^2/6$, which at $Q_j = 49$ is $346.67+400.17 = 746.83$. At the storeroom, case 3 of Table 8.3 over the transfer plus the wait.

The trace. Pass 1 gives the storeroom $(22, 48)$ and the hub $(120, 92)$. Pass 2 raises the storeroom to $(25, 49)$ and leaves the hub alone; pass 3 confirms. The storeroom costs \$1,371.26 a year and the hub \$2,279.69, and the hub's orders wait 0.0054 years on average, about two days.

Interpretation. The coupling moves the storeroom's reorder point by three units, from the planner who ignores the hub to the converged answer. The

hub's policy does not move at all, because the storeroom's batch barely changes and the hub's lead time demand barely changes with it. Notice what drives the hub's variance: the storeroom's batch contributes 400 of the 747, more than all the customers together. A hub serving a storeroom that orders fifty at a time is facing a demand its customers never place.

Validation, and its limit. Nothing exact covers this case, and the chapter has said which assumption breaks: customers who take one unit each. What remains is to check the answer by simulation, which Chapter 11 builds.

10.14.1. Where This Leaves Us

The chapter's methods are complete. Section 10.15 puts them on a worksheet, and Section 10.16 in code.

10.15. Building These Models in a Worksheet

The workbook is `Chapter10Models.xlsx`. Its sheets follow the chapter's three parts in order, and a `Notes` sheet says which sheet serves which section. Time is in weeks inside the sheets, and the `Hub` sheet converts the costs from dollars a year. There are no macros; every number can be traced by clicking the cell that made it.

10.15.1. Part I on the Sheets

`Hub` is Section 10.4. Its blue cells are the item, the network and the hub's policy, and its loss block walks G^1 and G^2 down from the Poisson distribution.

The measures read out of the loss block by position, as in chapter 9:

The item and the network		
demand per storeroom	1.0000	*units a week*
storerooms N	4	*locations*
hub lead time L_0	3.0000	*weeks*
transit O_j	1.0000	*weeks*
holding cost h	$28.75	*dollars a unit a year*
backorder cost b	$287.50	*dollars a unit a year*
weeks a year	52	*weeks*
lambda_0	4.0000	*units a week reaching the hub*
mean lead time demand	12.0000	*Poisson, lambda_0 L_0*
h per week	$0.55	*$/unit/yr x 1 yr/52 wk*
b per week	$5.53	
The hub's policy		
reorder point r_0	9	*units*
order quantity Q_0	8	*units*
Its measures, averaged over the positions r_0+1 .. r_0+Q_0		
expected backorders	1.0273	
ready rate	0.6030	
expected on-hand	2.5273	
E[B(B-1)]	3.9760	
Var[B]	3.9480	
mean wait, weeks	0.2568	*Little's law, over every request*
Var[W], weeks squared	0.1825	*the distributional Little's law, in factorial moments*

Figure 10.1.: The `Hub` sheet. The blue cells are the only ones typed. Notice that the four measures beneath the policy are Example 10.1's: 1.0273 backorders, a ready rate of 0.6030, 2.5273 on hand, and a mean wait of 0.2568 weeks by Little's law. The last line is Equation 10.4, the wait's variance by the distributional form.

```
B22:  =(INDEX(HubLossTwo,B18+1)-INDEX(HubLossTwo,B18+B19+1))/B19
```

which is Equation 8.18 with the offset of one turning a reorder point into a row.

`Delay` is Section 10.5. Each row is one day and one formula: the ready rate the hub would have with a lead time that many days shorter.

What the grid gives								
P(W = 0), the ready rate	0.6030							
mean wait, the grid	0.2575							
day k	**w, weeks**	**mean over L_0 - w**	**G1(r_0)**	**G1(r_0+Q_0)**	**P(W <= w)**	**P(k-1 < W <= k)**	**midpoint**	**P(W > w)**
0	0.0000	12.0000	3.321196	0.145065	0.602984	0.602984	0.0000	0.397016
1	0.1429	11.4286	2.849747	0.095487	0.655718	0.052734	0.0714	0.344282
2	0.2857	10.8571	2.403961	0.060306	0.707043	0.051326	0.2143	0.292957
3	0.4286	10.2857	1.988301	0.036360	0.756007	0.048964	0.3571	0.243993
4	0.5714	9.7143	1.607230	0.020807	0.801697	0.045690	0.5000	0.198303
5	0.7143	9.1429	1.264902	0.011223	0.843290	0.041593	0.6429	0.156710
6	0.8571	8.5714	0.964784	0.005659	0.880109	0.036819	0.7857	0.119891
7	1.0000	8.0000	0.709240	0.002642	0.911675	0.031566	0.9286	0.088325
8	1.1429	7.4286	0.499120	0.001128	0.937751	0.026076	1.0714	0.062249
9	1.2857	6.8571	0.333444	0.000434	0.958374	0.020623	1.2143	0.041626
10	1.4286	6.2857	0.209250	0.000148	0.973862	0.015488	1.3571	0.026138
11	1.5714	5.7143	0.121690	0.000043	0.984794	0.010932	1.5000	0.015206
12	1.7143	5.1429	0.064428	0.000011	0.991948	0.007154	1.6429	0.008052
13	1.8571	4.5714	0.030319	0.000002	0.996210	0.004263	1.7857	0.003790
14	2.0000	4.0000	0.012264	0.000000	0.998467	0.002257	1.9286	0.001533
15	2.1429	3.4286	0.004060	0.000000	0.999493	0.001025	2.0714	0.000507
16	2.2857	2.8571	0.001020	0.000000	0.999873	0.000380	2.2143	0.000127
17	2.4286	2.2857	0.000171	0.000000	0.999979	0.000106	2.3571	0.000021
18	2.5714	1.7143	0.000015	0.000000	0.999998	0.000020	2.5000	0.000002
19	2.7143	1.1429	0.000000	0.000000	1.000000	0.000002	2.6429	0.000000
20	2.8571	0.5714	0.000000	0.000000	1.000000	0.000000	2.7857	0.000000
21	3.0000	0.0000	0.000000	0.000000	1.000000	0.000000	2.9286	0.000000
integral of P(W > w)	0.2575	*trapezoids, a day wide*						
Little's law, B_0/lambda_0	0.2568							
difference	0.000633							

Figure 10.2.: The `Delay` sheet, a row a day over the three-week lead time. Notice the third column, the hub's mean demand over the shortened window, falling by 4/7 each day, and the sixth, $P(W \le w)$, climbing from 0.6030 to 0.9117 at a week and 0.9985 at two. Notice too the midpoint column, which places each day's probability in the middle of the day; the grid's mean wait, 0.2575, is within a day's rounding of Little's 0.2568.

`Exact` runs Equation 10.5 down a grid, one column per storeroom stock level, then averages each column over the hub's positions by Equation 10.6. A second copy with the shortage cost at one and both holding costs at zero returns the storerooms' backorders.

Two sets of cost rates, per week								
	costs	backorders only						
h_j	0.5529	0						
h_0	0.5529	0						
beta	5.5288	1						
Every term is linear in the three rates, so the right-hand column prices only the storerooms' backorders and returns their expected number.								
Equation (1): the cost at each S_j, averaged over S_0 = r_0+1 .. r_0+Q_0								
S_j	**0**	**1**	**2**	**3**	**4**	**5**	**6**	
cost per week	29.1925	14.5152	8.2391	7.0195	8.0203	9.8401	11.9329	
storeroom backorders	5.0273	2.2503	0.8547	0.2906	0.0915	0.0271	0.0075	
per storeroom	1.2568	0.5626	0.2137	0.0726	0.0229	0.0068	0.0019	
At the storeroom stock above								
system cost per week	$7.02							
backorders per storeroom	0.0726							

Figure 10.3.: The top of the `Exact` sheet. Notice the row of costs: $7.0195 a week at $S_j = 3$, the cheapest in the row, and the backorders beneath it falling steeply with each unit at the storeroom. The block of cost rates above the row is the linearity of Section 10.6.1 put to work: its second column sets both holding costs to zero and the shortage cost to one, so the same recursion returns the storerooms' backorders.

`Storeroom` sets the three routes of Table 10.2 side by side, with the exact figure read from `Exact`.

10.15.2. Part II on the Sheets

`Orders` is Equation 10.8: a column of the demand's tail and, beside it, each $P(N \geq n)$ as an average of Q_j consecutive entries of that column,

```
F21:  =AVERAGE(INDEX(OrdersTail,(1-1)*$B$6+1):INDEX(OrdersTail,1*$B$6))
```

Read from the other sheets				
lambda_j	1.0000			
O_j	1.0000			
mean wait	0.2568			
Var[W]	0.1825			
S_j	3			
The two moment routes' parameters				
mean lead time demand	1.2568			
its variance	1.4394			
negative binomial p	0.8732			
negative binomial r	8.6534			
Backorders and on-hand per storeroom				
	mean only	**two moments**	**conditioned**	**exact**
expected backorders	0.0505	0.0712	0.0728	0.0726
expected on-hand	1.7937	1.8144	1.8153	1.8158
On-hand less backorders is S_j - lambda_j (O_j + E[W]) in every route with the exact mean wait.				

Figure 10.4.: The `Storeroom` sheet. Notice the four backorder figures: 0.0505 by the mean wait, 0.0712 by two moments, 0.0728 conditioned on the day grid, and 0.0726 exact. The conditioned figure differs from the exact one only by the grid, a day wide.

and the seen-from-order masses beside those. `HubGeneral` convolves the order count with the other demand, one helper column per number of orders, and prices the exact convolution against the recipe and the shortcut.

Three lead time demands			
	exact	**recipe**	**shortcut**
mean	12.0000	12.0000	12.0000
variance	13.3333	13.5000	24.0000
negative binomial p		0.8889	0.5000
negative binomial r		96.0000	12.0000
second moment E[X^2]	157.3333	157.5000	168.0000
r_0 it would choose	13	13	15
cost it believes	$4.41	$4.48	$5.88
true cost of that r_0	$4.41	$4.41	$4.65
The hub at r_0 from the Orders sheet, exactly			
expected backorders	0.3497		
ready rate	0.8228		
expected on-hand	4.8497		

Figure 10.5.: The comparison block of the `HubGeneral` sheet. Notice the variances, 13.3333 exact, 13.5 by the recipe and 24 by the shortcut, and the reorder points each would choose: 13, 13 and 15. Notice last the row of true costs, where the shortcut's choice costs $4.65 a week against $4.41.

`OrderDelay` gives Equation 10.11 and the customer's wait at whole weeks, and `RandomLead` the two variances of Example 10.8 and their difference.

10.15.3. Part III on the Sheets

`Iterate` is Algorithm 10.1 by hand. A sheet cannot run chapter 8's search by itself without a block per pass, so the reader types each pass's policies into the yellow cells, and the sheet prices each policy and its four neighbors and hands the wait to the next pass.

Costs and the hub's target									
order cost k, both locations	$82.50	*dollars an order*							
hub fill rate target	0.8000	*gamma_0*							
h per week	$0.55								
b per week, storeroom	$5.53								
b_0 per week, hub	$2.21	*gamma_0 h/(1 - gamma_0), so b_0/(b_0 + h) = gamma_0*							
lambda_j	2.0000								
lambda_o	2.0000								
O_j	1.0000								
L_0	3.0000								
pass	**storeroom r_j**	**storeroom Q_j**	**storeroom cost**	**hub r_0**	**hub Q_0**	**hub cost**	**E[W]**	**sd W**	**a neighbour cheaper?**
1	-1	26	$13.09	2	47	$20.99	0.5634	1.5988	no
2	0	29	$14.35	2	48	$21.71	0.6166	1.8033	no
3	0	29	$14.65	2	48	$21.71	0.6166	1.8033	no
Pass 1 starts from no wait, which is the planner who ignores the hub. The policies stop moving at pass 2; pass 3 is the confirmation.									

Figure 10.6.: The summary of the `Iterate` sheet. Notice the last column, "no" on every pass: no neighbor of any typed policy is cheaper. Notice also that passes 2 and 3 agree, which is the stopping test of Algorithm 10.1 read off a sheet.

10.15.4. Solving a Problem on the Sheets

Example 10.11 (What a larger hub batch does)**.** The utility is offered a price break for ordering cutouts twelve at a time instead of eight. Before weighing the price, it wants to know what the larger batch does to the system with everything else held.

The modeling questions have the answers of Section 10.2, with $Q_0 = 12$. What follows is what you type and what comes back.

Step 1, the baseline. On `Hub`, confirm `B18` holds 9 and `B19` holds 8. `Exact` reads $7.0195 a week in `B28`.

Step 2, the larger batch. Type **12** into `Hub!B19`. Then read down the sheets:

Cell	What it is	$Q_0 = 8$	$Q_0 = 12$
`Hub!B22`	$\bar{B}_0$	1.0273	0.6984
`Hub!B23`	$\overline{RR}_0$	0.6030	0.7242
`Hub!B27`	$\bar{W}$, weeks	0.2568	0.1746

Cell	What it is	$Q_0 = 8$	$Q_0 = 12$
`Storeroom!E19`	backorders per storeroom	0.0726	0.0567
`Exact!B28`	system cost per week	\$7.0195	\$7.7367

The larger batch holds more at the hub, so the hub owes less and the storerooms' crews wait less, and the system costs \$0.72 a week more. The row of costs on `Exact` still has its minimum at $S_j = 3$.

Step 3, adjust the reorder point. Holding more on average at the hub is more protection than the system wants, so lower `Hub!B18`. At 7, `Exact!B28` reads \$7.5379, the cheapest reorder point for a batch of twelve.

The check. On `Storeroom`, the exact and conditioned backorders at $Q_0 = 12$ are 0.0567 and 0.0568, apart only by the day grid, as they were at $Q_0 = 8$.

Interpretation. Without an ordering cost the larger batch cannot pay for itself, since it only adds stock. Its best version costs \$0.52 a week more than the present policy, \$27 a year. Thus the price break is worth taking if it saves more than \$27 a year on the cutouts the hub buys, about 208 a year: a thirteen-cent discount would do it. Notice that the answer came from two cells typed, and that the reorder point had to move with the batch. A planner who changed the batch and left the reorder point would have priced the offer at \$0.72 a week instead of \$0.52.

10.15.5. Where This Leaves Us

The sheets reproduce every Part I number and the Part II comparisons, and the iteration by hand. What they cannot do is search on their own or condition on a fine grid of waits, and Section 10.16 is where that is done.

10.16. Designing the Software

The package is `inventory.multiechelon`, extending what chapter 9 built, with one addition to `inventory.continuousreview`.

10.16.1. What the Software Must Do

Reading back over the chapter, the software has to

1. evaluate a two-level system exactly where Axsäter (1993) applies, for checking everything else against,
2. build a hub's lead time demand from other demand and storerooms that batch, exactly and by the recipe,
3. give the wait at the hub as a distribution, for a customer and for a storeroom's order, at a constant or a random lead time,
4. turn a wait into a storeroom's lead time demand by any of the three routes, and
5. run the iteration by either route and price its answer exactly.

10.16.2. The Classes

Class	Knows	Does
`OneForOneTwoLevel`	the rates, lead times and three cost rates	Equation 10.5; the stock levels by linearity
`AxsaterBatchOrdering`	a one-for-one system and the two batches	Equation 10.6, Equation 10.7 and his equation (6), and his three approximations

Class	Knows	Does
`OrderCountLaw`	a storeroom's rate and batch	Equation 10.8, at a random moment and seen from an order
`BatchedHub`	the other demand, the storerooms and the hub's policy	Equation 10.9, Equation 10.10, the shortcut, Equation 10.12, and the hub's measures over its positions
`HubDelay`	a hub, and whose wait	Equation 10.3 and Equation 10.11, their moments, and a midpoint grid
`MixedWait`	waits at several lead times	Equation 10.13
`StoreroomWithDelay`	a storeroom's rate, its transit and a wait	the three routes of Section 10.6.3
`TwoLevelIteration`	a problem and the two routes	Algorithm 10.1, and the exact price of any policy
`TwoLevelLedger`	two policies and a script of demands	posts the ledgers of Section 10.3, checking the positions on every row

Notice that no class is a "storeroom" in the sense of chapter 9. A storeroom here is a demand stream as the hub sees it, `StoreStream`, and a lead time demand as the storeroom sees it, `StoreroomWithDelay`; what it is to itself is chapter 8's `RQModel`. `LeadTimeDemand` gains two factories, `tabulated` and `mixture`, since a convolution and a mixture are distributions no family describes.

10.16.3. Running the Chapter's Examples

The hub of Part I and its wait:

```
val hub = BatchedHub(
    otherRate = 0.0,
    stores = List(4) { StoreStream(lambda = 1.0, q = 1) },
    reorder = 9, batch = 8, lead = 3.0, costs = costs,
)
val delay = HubDelay(hub)
```

`BatchedHub` holds Equation 10.9 for whatever streams it is given, and with four one-for-one storerooms it recognizes a Poisson hub and uses chapter 8's `LeadTimeDemand.poisson`. `HubDelay` defaults to a customer's wait.

```
### the hub of Part I and the wait it imposes
  ready rate 0.6030, backorders 1.0273, Little 0.2568 weeks
  P(W <= 0) = 0.6030
  P(W <= 1) = 0.9117
  P(W <= 2) = 0.9985
  E[W] 0.2568, Var[W] 0.1825
```

The exact yardstick and the three routes:

```
val oneForOne = OneForOneTwoLevel(storeRate = 1.0, stores =
 ↪  4, transit = 1.0, hubLead = 3.0,
    storeHolding = h, hubHolding = h, shortage = b)
val exact = AxsaterBatchOrdering(oneForOne, storeBatch = 1,
 ↪  hubBatches = 8)
val store = StoreroomWithDelay(lambda = 1.0, transit = 1.0,
 ↪  delay = delay.fineLaw(), costs = costs)
```

`exact.cost(hubReorder = 9, storeReorder = 2)` is Equation 10.6; the storeroom's reorder point is $S_j - 1$. `delay.fineLaw()` tabulates the wait on a grid of 3000 intervals, fine enough that conditioning on it reproduces the continuous wait.

```
### Axsater's exact cost, and the storeroom three ways
  exact cost 7.0195 a week, backorders per storeroom 0.0726
  MEAN_ONLY    backorders 0.0505
  TWO_MOMENT   backorders 0.0712
  CONDITIONED  backorders 0.0726
```

Part II, with a storeroom that batches:

```
val law = OrderCountLaw(lambda = 2.0, q = 3)
val hub2 = BatchedHub(otherRate = 2.0, stores =
 ↪  listOf(StoreStream(2.0, 3)),
    reorder = 12, batch = 8, lead = 3.0, costs = costs)
val order = HubDelay(hub2, store = 0)
```

`HubDelay(hub2, store = 0)` is the wait of storeroom 0's orders, Equation 10.11, rather than a customer's.

```
### Part II: the order-count law and the hub with other demand
  at a random moment: 0.0273, 0.2667, 0.4385, 0.2186, 0.0443
  seen from an order: 0.0620, 0.3837, 0.4016, 0.1327, 0.0187
  variance: exact 13.3333, recipe 13.5000, shortcut 24.0000
  the order's wait: P(W = 0) 0.7626, mean 0.1264
  storeroom backorders, conditioned: 0.0823
```

And the iteration, by both routes:

```
for ((hr, sr) in listOf(HubRoute.MOMENTS to
 ↪  StoreRoute.MOMENTS, HubRoute.EXACT to
 ↪  StoreRoute.CONDITIONED)) {
    val result = TwoLevelIteration(problem, hr, sr).solve()
    println(TwoLevelIteration.exactSystemCost(problem,
 ↪  result.policy, b))
}
```

`problem` is a `TwoLevelProblem` holding the rates, lot sizes, lead times and the two locations' costs, the hub's backorder cost set by `TwoLevelProblem.backorderCostForTarget(0.8, h)`.

```
### Part III: the two-level iteration
  moments route, 3 passes:
    1  hub (2, 47), storeroom (-1, 26)  E[W] 0.5634
    2  hub (2, 48), storeroom (0, 29)  E[W] 0.6166
    3  hub (2, 48), storeroom (0, 29)  E[W] 0.6166
    priced exactly: 34.3462 a week
  exact route, 3 passes:
    1  hub (2, 48), storeroom (-1, 26)  E[W] 1.6250
    2  hub (2, 48), storeroom (3, 28)  E[W] 1.7500
    3  hub (2, 48), storeroom (3, 28)  E[W] 1.7500
    priced exactly: 33.5370 a week
```

Notice the wait in the two routes' traces. The moment route carries a customer's wait, 0.62 weeks; the exact route carries the wait of an order of 28, 1.75 weeks, since an order waits for its last unit. The complete program is `Section10Listings.kt`, and the package's tests assert every number above.

10.16.4. Where This Leaves Us

Every number the chapter prints has a line of code behind it and a test pinning it, and every claim of exactness has been checked against Axsäter (1993)'s published figures wherever his setting applies.

10.17. Summary

A storeroom that orders in batches passes up its customers' demand counted in whole batches. Its supplier sees nothing for a while and then Q_j units at once, and it sees them more regularly than the customers arrive. Counting the orders in a window needs nothing but the Poisson table: the chance of at least n orders is an average of Q_j consecutive entries of the demand's tail, because the storeroom's position is equally likely to sit anywhere in its range.

One move carries the chapter: condition on the position, then average. Chapter 8 built the (r, Q) measures that way. The chapter used it again for Axsäter's equation (1), for the order-count law, and for a random supplier lead time, and each time a problem that looked new turned out to be an average of problems already solved.

When the storerooms order one at a time, everything is exact. The hub is a chapter 8 location. Its wait has a distribution, not only Little's mean: the chance of waiting at most w is the ready rate the hub would have with a lead time w shorter. And Axsäter's recursion gives the storerooms' exact measures, against which the chapter measured every approximation it made.

Conditioning a storeroom's lead time demand on the wait is exact. A request's wait is fixed by what happened before it, so it is independent of the demand the storeroom meets afterwards. For a batching storeroom the wait is the one its own order sees, which waits for its last unit and sees fewer of its storeroom's orders ahead of it than a random moment does.

With one-unit customers, constant lead times and a hub reorder point of zero or more, the chapter's route agrees with Axsäter wherever both apply and reaches past him to other demand and unlike storerooms.

Little's law gets the mean wait right, and the mean alone is not enough. On the running example it understated the storeroom's backorders by 30%. Two moments recovered nearly all of it, which is chapter 9's VARI-METRIC lesson made exact.

The recipe is sound and the shortcut is not. Two moments with $Q_j^2/6$ for the rounding to whole batches choose the same hub policy as the exact convolution. Treating a storeroom's orders as Poisson arrivals of Q_j units charges the hub for a randomness the orders do not have, and it over-stocks.

A random supplier lead time is handled by building at each value, then averaging. Never average each stream alone: both are counted over the same lead time, and adding their separate variances loses the covariance that sharing it creates.

The two policies are set by an iteration whose answer depends on the hub's target. Hold one location, solve the other, alternate; it converges in a few passes and its answer does not depend on the start. But the hub carries no shortage cost of its own, and the one it is given decides the fixed point more than any route does. The exact optima run the hub at a low ready rate, and a hub target near a customer's service level over-stocks it.

Table 10.6.: The chapter's notation.

Symbol	Meaning
λ_j, λ_o, λ_0	demand rate at storeroom j, of other demand at the hub, and at the hub in total

Symbol	Meaning
L_0, O_j	the hub's lead time from its supplier, and the transit from the hub to storeroom j
(r_0, Q_0), (r_j, Q_j)	the hub's and a storeroom's policies; base stock S_j is $(S_j - 1, 1)$
$D_0(\ell)$, $X_o(\ell)$	the hub's demand, and its other demand, in a window of ℓ
$N_j(\ell)$	the orders storeroom j places in a window of ℓ
$D'_j(\ell)$	storeroom j's demand in a window ending at one of its orders, before it
W, W_j	a customer's wait at the hub, and the wait of storeroom j's order
$\bar{B}_0$, $\overline{RR}_0$, $\bar{I}_0$	the hub's expected backorders, ready rate and on-hand
$C(S_0, S_j)$	the exact cost per unit time of one-for-one policies at both levels
R_w, Q_w	Axsäter's hub reorder point and batch, counted in storeroom batches
γ_0, b_0	the hub's ready rate target and the backorder cost it implies

10.18. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.8. Work in weeks. The cutout costs \$28.75 per unit per year to hold and \$287.50 per unit per year to owe a customer, so divide each by 52 and show the division before using it. Customers take one unit each and arrive in a Poisson process. Report probabilities to four decimals, stock levels as whole units,

and costs to the cent. A hub fills its requests first come, first served, and ships part of an order when it has only part of it.

10.18.1. Working the Model by Hand

Exercise 10.1. Answer each in a sentence.

a. True or false: when a hub serves four storerooms whose requests are each one unit, a request from the busiest storeroom waits no longer on average than a request from the quietest.
b. True or false: the hub's expected backorders divided by its demand rate, $\bar{B}_0/\lambda_0$, is the average wait of the requests that had to wait.
c. Multiple choice. A hub's supplier takes two, three or five weeks. A planner averages the other demand's lead time demand over those three values, does the same for the storeroom's orders, and adds the two variances. The result is

    i. right, because the two streams are independent.
    ii. too high, because each stream is counted over the longest lead time.
    iii. too low, because both streams grow together when the lead time is long, and adding the separate variances drops that.
    iv. too low, because the storeroom's orders are lumpier than its customers.

d. Multiple choice. A storeroom orders three units, and the hub has two on its shelf. The order's wait at the hub is

    i. zero, since most of it ships at once.
    ii. the average wait of a unit at the hub, $\bar{B}_0/\lambda_0$.
    iii. the time until its last unit leaves the hub.
    iv. the hub's lead time, since the hub must reorder.

e. Multiple choice. Treating a storeroom's orders of Q_j units as if they arrived at the hub in a Poisson process makes the variance of the hub's lead time demand

i. right, since the orders' average rate is right.
ii. too high, by roughly a factor of Q_j in the storeroom's term.
iii. too low, since the orders are counted as fewer events.
iv. right over a long lead time and wrong over a short one.

Exercise 10.2. Continue the two-location ledger. The hub runs $(r_0, Q_0) = (1, 4)$ with a supplier lead time of $L_0 = 3$ weeks. The storeroom runs base stock $S_j = 2$ and is $O_j = 1$ week from the hub. At the end of week 15 the hub holds 4 units, has nothing on order and owes nothing; the storeroom holds 2 units with nothing in transit. Failures occur at the storeroom in weeks 16, 17, 17, 18, 19 and 21. Within a week, post the hub's receipts first, then the storeroom's, then the failures in the order listed, and follow the convention of Example 8.1: one row per event, with the state recorded after the event.

a. Post the ledger from week 16 until nothing is outstanding. Carry the storeroom's on-hand, backorders, position and units in transit, and the hub's on-hand, backorders, units on order and position. Check on every row that the hub's position equals its on-hand plus on order less backorders.
b. Give each of the six requests' wait at the hub.
c. Count the hub's backorder-weeks from week 15 on, weighting each level by how long it was held, and obtain the mean wait over the six requests by Little's law. Reconcile it with part (b).
d. The hub was short, yet no crew at the storeroom waited. Explain why, in terms of the storeroom's stock and its transit time.

Exercise 10.3. Four storerooms each see Poisson demand of one unit a week and order one at a time, so the hub's demand is Poisson at $\lambda_0 = 4$ a week. The hub runs $(r_0, Q_0) = (10, 4)$ with $L_0 = 3$ weeks. Let $D_0(\ell)$ represent the hub's demand in ℓ weeks. A request waits at most w weeks

with probability

$$P(W \le w) = \frac{1}{Q_0} \sum_{u=r_0+1}^{r_0+Q_0} P(D_0(L_0 - w) \le u - 1), \qquad 0 \le w < L_0.$$

a. Obtain $P(W = 0)$ from Table D.3.
b. Obtain $P(W \le 1)$ and $P(W \le 2)$ the same way.
c. Obtain the hub's expected backorders by Equation 8.18 from Table D.5, then the mean wait $\bar{B}_0/\lambda_0$ in weeks and in days, as a worked chain.
d. The chapter's hub runs $(9, 8)$ against the same demand, with $P(W = 0) = 0.6030$ and a mean wait of 0.2568 weeks. Your hub has the higher reorder point and makes requests wait longer. Explain why, in terms of where the hub's position sits on average.

Exercise 10.4. One storeroom sees Poisson demand of one unit a week and is supplied by a hub. Both lead times are one week. Holding costs \$1 per unit per week at either location, and each unit owed at the storeroom costs \$10 per week. If both locations ordered one unit at a time, with the hub's position held at S_0 and the storeroom's at S_j, the exact cost per week $C(S_0, S_j)$ of Axsäter (1993) would be:

S_0	$S_j = 1$	$S_j = 2$	$S_j = 3$	$S_j = 4$
2	5.8617	3.5533	3.4159	4.0972
4	7.0797	5.1558	5.2624	6.0495
6	9.0474	7.1403	7.2568	8.0479

Now let the storeroom run $(r_j, Q_j) = (1, 2)$, and let the hub order two storeroom batches at a time with a reorder point of one batch. In Axsäter (1993)'s units that is $R_w = 1$ and $Q_w = 2$, or $r_0 = 2$ and $Q_0 = 4$ in units. His equation (2) gives the cost as

$$C = \frac{1}{Q_w Q_j} \sum_{i=R_w+1}^{R_w+Q_w} \sum_{k=r_j+1}^{r_j+Q_j} C(iQ_j, k).$$

a. List the four terms the double sum needs, read them from the table, and give the cost per week.
b. Explain in a sentence why the ith sub-batch of a hub order is released against system demand iQ_j when there is only one storeroom.
c. Which entry of the table is the cost of ordering one unit at a time at both levels with $S_0 = 4$ and $S_j = 2$? Is the batching policy dearer or cheaper than it, before any ordering cost, and why?

Exercise 10.5. A storeroom's customers take two units a week, and it orders $Q_j = 4$ units each time its position reaches its reorder point. Let N represent the number of orders it places in a window of two weeks, and let D represent its customers' demand in the window.

a. The position is equally likely to sit 1, 2, 3 or 4 units above the reorder point when the window opens, so $P(N \geq n) = \frac{1}{4}\sum_{y=4n-3}^{4n} P(D \geq y)$. Obtain $P(N \geq 2)$, $P(N \geq 3)$ and then $P(N = 2)$ from Table D.3.
b. Now let the window end at one of the storeroom's own orders. The storeroom's demand strictly before that order is still Poisson with mean 4, and its earlier orders number $\lfloor D'/4 \rfloor$. Obtain the chance of exactly two earlier orders.
c. Obtain $P(N = n)$ for $n = 0, 1, 2, 3, 4$ and check that four times the mean number of orders is the customers' mean demand in the window.

Exercise 10.6. A storeroom orders $Q_j = 3$ units at a time against customers who take two units a week. Let U_0 and U_ℓ represent how far its position sits above its reorder point at the start and the end of a window of ℓ weeks, and let D represent the customers' demand in the window. Then the units it orders are $Q_j N = D + U_\ell - U_0$, and over a long window U_0 and U_ℓ are each uniform on $\{1, 2, 3\}$ and independent of each other and of D.

a. List the values of U_0 with their probabilities and obtain its variance.

b. Show that the variance of the units ordered exceeds the customers' variance by $(Q_j^2 - 1)/6$, and evaluate it.
c. The two-moment recipe adds $Q_j^2/6$ instead. By how much is it high, and what does that difference come from?
d. Using the order-count law in the software, obtain the variance of the units ordered over $\ell = 30$ weeks and confirm part (b).

10.18.2. Using the Software

Exercise 10.7. Four storerooms each see one unit a week and run base stock $S_j = 3$. The hub runs $(r_0, Q_0) = (9, 8)$ with $L_0 = 3$ weeks. Take the transit time from the hub to a storeroom as $O_j = 2$ weeks, twice the chapter's.

a. Obtain the exact expected backorders at one storeroom from `AxsaterBatchOrdering`.
b. Obtain the same figure by the three routes of `StoreroomWithDelay`: the mean wait only, two moments, and conditioned on the wait's distribution.
c. At one week's transit the mean wait alone was 30% low and two moments 2% low. Give the two errors at two weeks, and explain why the longer transit shrinks them.

Exercise 10.8. A hub serves two units a week of other demand and one storeroom whose customers take two units a week. The hub runs $Q_0 = 8$ with $L_0 = 3$ weeks and is charged the cutout's holding cost and \$287.50 a year for every unit it owes. For $Q_j = 3$, 6 and 12:

a. Obtain the variance of the hub's lead time demand by the exact convolution, by the recipe (adding $Q_j^2/6$), and by treating the storeroom's orders as a Poisson stream of Q_j units.
b. For each of the three, obtain the reorder point that minimizes the hub's holding and backorder cost with $Q_0 = 8$, and the true cost of that reorder point under the exact convolution.

c. At which Q_j does the recipe's variance fall below the exact one, and what does its choice cost there?

Exercise 10.9. Take the hub of Exercise 10.8 at $(r_0, Q_0) = (12, 8)$ with $Q_j = 3$, the storeroom one week away. The supplier now takes one, three or five weeks with probabilities 0.25, 0.5 and 0.25.

a. Obtain the variance of the hub's lead time demand built at each value and averaged, and the variance obtained by averaging each stream alone. Show that they differ by $2\lambda_o\lambda_j \mathrm{Var}[L_0]$.
b. Obtain the mean and variance of the storeroom order's wait.
c. Choose the storeroom's reorder point with $Q_j = 3$ three ways: planned on the mean wait only, on two moments, and conditioned on the wait's distribution. Price each choice at the conditioned route's cost and backorders.

Exercise 10.10. Take the hub and storeroom of Exercise 10.8, but let the other demand be four units a week. Both locations pay \$82.50 an order, the storeroom's customers are owed at \$287.50 a year per unit, and the hub's backorder cost is set from a ready rate target of 0.8 by Equation 10.15. Let both batches be chosen.

a. Run the two-level iteration by the moment route and by the exact route. Give each fixed point and the number of passes.
b. Price both fixed points with `TwoLevelIteration.exactSystemCost`, charging the hub's other customers \$287.50 a year per unit owed. Which route's answer is cheaper, and by how much?

10.18.3. Using the Workbook

Exercise 10.11. In `Chapter10Models.xlsx`, change the transit time on the `Hub` sheet from one week to two.

a. Which figures on the `Hub` and `Delay` sheets move, and why?

b. Give the new system cost and backorders per storeroom on the `Exact` sheet.
c. Give the four backorder figures on the `Storeroom` sheet, and say which one the workbook's daily grid makes approximate.

Exercise 10.12. On the `Iterate` sheet, pass 1 starts from a wait of zero. Overwrite pass 1's mean wait with 2 weeks and its variance with 1, then find each pass's policies, typing into the yellow cells a policy that no neighbor undercuts.

a. Give the storeroom's and the hub's policies on each pass until they repeat.
b. How many passes does it take, and where does it end?
c. What does the answer say about where the iteration starts?

10.18.4. Reading the Literature

Exercise 10.13. Read section 3.2 of Axsäter (1993).

a. State approximation 1 and approximation 2 in a sentence each, in terms of which system demand triggers a storeroom's order.
b. Why is approximation 1 exact for a single storeroom, and approximation 2 exact when the storerooms order one unit at a time?
c. Approximation 3 weights approximation 1 by $1/N$ and approximation 2 by $(N-1)/N$. Explain the weights.

Exercise 10.14. Take the cutout in job lots: lots of 1, 2 or 4 units with probabilities 0.4, 0.4 and 0.2, 150 lots a year at the storeroom and 250 a year of other demand at the hub, a two-month supplier lead time, and a storeroom batch of $Q_j = 49$. The recipe gives the hub's lead time demand a mean of 133.33 units and a variance of 746.83.

a. Obtain the quantity a of Equation C.34 and the family Table C.8 selects.

b. Obtain that family's parameters by Equation C.36, and check that the mixture reproduces the mean and variance.
c. The rule of Section 8.3.2, which the chapter uses, fits a different family to the same two moments. Which, and why?
d. With $Q_0 = 92$, a holding cost of \$28.75 a year and a hub backorder cost of \$115 a year, obtain the reorder point each fit chooses and its cost under the other fit.

11. Simulating Inventory Systems

Learning objectives

After reading this chapter you should be able to:

- recognize when an inventory question requires simulation rather than analysis, and say what a simulated answer costs
- represent the state variables of an inventory policy as a sequence of events
- compute time averages from a sample path as a simulation computes them
- validate a simulation against an exact result before trusting it beyond one
- apply appropriate output analysis to a non-terminating inventory simulation
- simulate continuous review, periodic review and two-level systems, and compare policies using common random numbers
- build a small supply network with the KSL's supply chain package

11.1. When the Formulas Stop

Ten chapters have computed the performance of inventory policies, and every computation has had the same shape. A model was stated, its

assumptions were listed, and a number came out that was exact when the assumptions held. Where they did not hold, an approximation took over, and the chapter priced what the approximation cost. This chapter adds a third route to the same numbers, and the only one that reaches every case.

11.1.1. Three Routes to One Number

Recall from Section 7.10 that the book has simulated once already. The newsvendor's expected profit was estimated by sampling 100,000 demands at each of five quantities, and every estimate in Table 7.8 covered the exact value. That section drew a conclusion that this chapter adopts as its method: simulation belongs after the closed form, not instead of it.

The reason is that a simulation is a program, and a program can be wrong without saying so. An exact formula is checked once, by derivation. A simulation is checked by running it on a case whose answer is already known and seeing the answer come back. That is, the agreement in Table 7.8 was not a curiosity; it was the evidence that the sampling code computed the newsvendor's cost and not something near it. Section 7.12.4 made the same point about the software: two routes that share one cost function can be checked against each other, and two that do not, cannot.

A simulation is trusted only after it reproduces a number the book has already derived. Every model in this chapter is built to that rule. It first reproduces a result from an earlier chapter, inside its half-width, and only then is one assumption behind that result removed. For example, Section 11.3 simulates the transformer at its optimal policy $(8, 7)$ and checks the estimate against the exact cost of \$7,225.15 from Example 8.7. Section 11.5 then lets the transformer's orders cross, and no formula in the book is available to check that case.

11.1.2. Where the Exact Route Ends

Each earlier chapter said where its exactness ended. Four of those boundaries are crossed by nothing in the book except this chapter.

Lost sales. Recall from Section 1.3.2 that a demand which cannot be filled may depart rather than wait. Every formula of Chapter 8 assumes that it waits. The usual remedy is to charge a lost sale as though it were a backorder, and whether that remedy is close for a given item is a question only a model of the lost-sales system can answer.

Orders that overtake one another. Section 8.5.4 showed that a random lead time leaves the (r, Q) formulas intact, provided orders arrive in the sequence they were placed. Section 10.11 repeated the condition and called the correction beyond its scope. A simulation has no such condition. It delivers each order when that order's own lead time ends, in whatever sequence results.

Job lots at a batching hub. Example 10.10 found policies for a two-level system whose customers take one, two or four units at a time. Nothing exact covers that case, and the example closed by handing it to this chapter.

How wrong METRIC is for a particular system. Example 9.4 predicted 0.4055 backorders per storeroom for the transformer, and Example 9.5 predicted 0.4815 for the same system. The two differ by 19%, and Graves (1985) tells us how often each one errs across many systems. Neither tells us which one is closer for this one. A simulation of the two-level system does.

In addition, some answers in the book are exact but were hard to reach. The (s, S) costs of Section 8.12 came from solving a Markov chain, and a simulation of the same policy is a second, independent route to them. That is the agreement rule used in reverse: an exact result the reader cannot easily recompute is checked by a simulation they can.

11.1.3. What a Simulated Answer Costs

A simulation answers questions no formula reaches, and it charges for the service in three ways.

Every answer is an estimate. A simulated cost is a sample average with a half-width, and a cost reported without its half-width reports a number that is not there. Recall from Section 7.10 that the half-width shrinks with the square root of the run. Thus, ten times the precision costs a hundred times the run.

Every answer is for one policy. The exact route of Section 8.8 searched a cost function it could evaluate exactly at any policy it chose. A simulation evaluates one policy per run, with noise, so finding the best policy is a search over noisy estimates. That is a harder problem than the one it replaces, and Section 11.7 treats it only briefly.

The run is measured in demands, not in years. The transformer sees 45 demands a year. A thousand simulated years is 45,000 demands, which is a modest sample for estimating a backorder that occurs a few times a year. Section 11.4 turns that into a run length.

None of these costs applies to the formulas, which is why the formulas come first wherever they exist.

11.1.4. What This Chapter Assumes

The chapter builds its models with the KSL, the Kotlin Simulation Library, and it assumes the reader has met discrete-event simulation. The mechanics of the event calendar, of replications, and of the statistical analysis of simulation output are the subject of Rossetti (2026), and this chapter refers to that book rather than repeating it. Its chapter on output analysis and its chapter on event modeling are the two this chapter leans on most. Appendix E shows how to run the companion code.

What this chapter adds is the inventory side. That is, it says which events an inventory policy consists of, which statistics a stock point needs, and how each model is checked against the analysis in the earlier chapters. The next section starts with the events.

11.2. The Inventory Processes as Events

Recall from Section 8.1 that an (r, Q) system carries five state variables: $I(t)$ on hand, $IO(t)$ on order, $B(t)$ backordered, the net inventory $IN(t) = I(t)-B(t)$, and the inventory position $IP(t) = I(t)+IO(t)-B(t)$. Table 8.2 traced them through thirteen weeks at the storeroom. This section traces the same thirteen weeks again, this time the way a simulation does.

11.2.1. What Changes the State

An event is an instant at which the state of the system changes. Between two events nothing changes, so each state variable is constant from one event to the next. Thus, the five variables are step functions of time, and a simulation needs to compute them only at the events.

An (r, Q) system with backorders has two events.

- **A demand.** One unit is requested. If the shelf has stock, I falls by one; otherwise B rises by one. Either way, IP falls by one.
- **A receipt.** An order of Q units arrives. IO falls by Q and I rises by Q, and then the backorders are filled from the new stock, first come first served. IP does not change.

Notice what is missing from the list: placing an order. Placing an order changes IO and IP, but it never happens at an instant of its own. It happens inside a demand, at the moment the demand lowers the position to r. That is, an order is a **decision** taken during an event, not an event.

What the decision does is schedule one. An order placed at time t creates a receipt at time $t + L$, and that receipt waits in the **event calendar**, the list of future events in time order. The simulation proceeds by removing the earliest event from the calendar, advancing the clock to its time, and carrying out its changes, which may add new events to the calendar. Table 11.1 lists both events, the changes each one makes, and what each one schedules.

Table 11.1.: The events of an (r, Q) system with backorders. A periodic review policy adds a third event, the review, in Section 11.6.

Event	Changes to the state	Decision taken	Schedules
Demand	$I \leftarrow I - 1$ if $I > 0$, otherwise $B \leftarrow B + 1$; $IP \leftarrow IP - 1$	If $IP \leq r$, order Q: $IO \leftarrow IO + Q$, $IP \leftarrow IP + Q$	The next demand; a receipt at $t + L$ if an order was placed
Receipt	$IO \leftarrow IO - Q$, $I \leftarrow I + Q$, then fill $\min(B, I)$ backorders	None	Nothing

Notice that the demand event schedules its own successor. Demands arrive in a Poisson process, so the time to the next demand is exponential with mean $1/\lambda$, and the simulation samples it when the current demand occurs. In Table 8.2 the demand times were simply given. A simulation draws them.

11.2.2. Averages Over Time, Accumulated at Events

The measures of Section 8.2 are time averages. Recall from Section 9.5.3.1 that a time average weights each value by how long it held, and that

averaging the rows of a ledger instead gives a number that is not an average of anything the system did. A simulation never makes that mistake, because it never sees the rows. It sees the intervals between them.

Let $t_0 = 0 < t_1 < t_2 < \cdots$ represent the times of successive events, and let $A_I(t) = \int_0^t I(u)\,du$ represent the area under the on-hand curve up to time t. Since I is constant between events, the area grows by a rectangle at each one:

$$A_I(t_n) = A_I(t_{n-1}) + I(t_{n-1})\,(t_n - t_{n-1}) \tag{11.1}$$

The order of operations is the whole of the method. **The area is updated before the event changes the state**, using the value that held during the interval just ended. Updating it afterward would credit the interval with the new value, which the system did not hold.

The same rule accumulates $A_B(t)$, the area under $B(t)$. Let $A_+(t)$ represent the total time up to t during which $I > 0$, accumulated the same way with $I(t_{n-1})$ replaced by 1 when the shelf held stock and by 0 when it did not. At the end of an observation of length T, the averages are areas divided by time:

$$\bar{I} = \frac{A_I(T)}{T}, \qquad \bar{B} = \frac{A_B(T)}{T}, \qquad \overline{RR} = \frac{A_+(T)}{T} \tag{11.2}$$

Rossetti (2026) calls a quantity averaged this way **time-persistent**, and the KSL's `TWResponse` class computes it with exactly the update of Equation 11.1. A quantity recorded once per occurrence, such as whether a demand was filled, is **observation-based**, and its average is a plain average over the occurrences; the KSL's `Response` class collects it. The fill rate is the second kind. Let $N_D(t)$ represent the number of demands up to time t and $N_F(t)$ the number filled from the shelf on arrival. Then

$$\overline{FR} = \frac{N_F(T)}{N_D(T)} \tag{11.3}$$

Notice that the two service measures of Section 8.2.2 come out of different machinery. The **ready rate** is a time-persistent average and the **fill rate** is an observation-based one. That is why they can differ on any finite run, whatever the demand process.

11.2.3. The Storeroom's Thirteen Weeks as Events

Example 11.1 (The storeroom ledger, accumulated as a simulation would)**.** We return to the storeroom of Example 8.1. The transformer has demand of $\lambda = 45$ units a year arriving one at a time, an eight-week lead time, and the policy $(r, Q) = (4, 5)$. The observation starts with 6 on hand, none on order and nothing owed. The ten demands arrive in weeks 1, 2, 3, 5, 6, 7, 8, 9, 11 and 13, exactly as before, and the observation closes at week 13.

What is stocked? One item, the transformer.

What is the demand process? Single-unit demands; in this example their times are given rather than sampled.

When is inventory reviewed? Continuously, at every demand.

What triggers replenishment, and how much? The position reaching $r = 4$ or below, and the order is for $Q = 5$.

What happens to unmet demand? It is backordered and filled first come first served.

What costs are incurred, and when? None is needed here. The example computes the averages that Equation 8.1 prices.

Notation. The ledger carries I, B and IP after each event, the areas A_I, A_B and A_+ through each event's time, and the calendar's receipts. IO is

five times the number of receipts on the calendar, so it is not carried as a column.

We begin at $t_0 = 0$ with $I = 6$, $B = 0$, $IP = 6$, all three areas at zero, and an empty calendar.

The demand at week 1 is the first event. Before the state changes, the interval from week 0 to week 1 is credited: A_I grows by $6 \times 1 = 6$, and A_+ grows by 1 because the shelf held stock. Then the demand is filled, so I and IP fall to 5.

The demand at week 2 credits 5×1 to A_I, making 11, and one more week to A_+. Then I and IP fall to 4, the position has reached r, and the decision is taken: order 5. The position rises to 9, and a receipt is scheduled for week $2 + 8 = 10$.

The table has been updated as follows.

Table 11.2.: The event ledger through the first order. Each area is the total through the week on its row, credited before that row's event changed the state.

Week	Event	I	B	IP	A_I	A_B	A_+	Receipts due
0	start	6	0	6	0	0	0	none
1	demand	5	0	5	6	0	1	none
2	demand, **order 5**	4	0	9	11	0	2	week 10

The demands at weeks 3, 5, 6 and 7 are each filled. The interval before week 5 is two weeks long, so that event credits $3 \times 2 = 6$ to A_I, which is where row averaging and time averaging part company. By week 7 the shelf is empty and $A_I = 24$. The interval from week 7 to week 8 is then credited with nothing, since $I = 0$ throughout it, and A_+ stops growing at 7.

The demand at week 8 finds the shelf empty and is backordered, so B becomes 1 and the position falls to 4. The decision is taken again: order 5, the position returns to 9, and a second receipt joins the calendar for week 16. The demand at week 9 first credits one backorder-week to A_B, since $B = 1$ held from week 8 to week 9, and is then backordered itself, so B becomes 2.

The table has been updated as follows.

Table 11.3.: The event ledger through the second backorder. Notice that the shelf has been empty since week 7, but the second order waited until the position reached r at week 8.

Week	Event	I	B	IP	A_I	A_B	A_+	Receipts due
0	start	6	0	6	0	0	0	none
1	demand	5	0	5	6	0	1	none
2	demand, **order 5**	4	0	9	11	0	2	week 10
3	demand	3	0	8	15	0	3	week 10

Week	Event	I	B	IP	A_I	A_B	A_+	Receipts due
5	demand	2	0	7	21	0	5	week 10
6	demand	1	0	6	23	0	6	week 10
7	demand	0	0	5	24	0	7	week 10
8	demand backordered, **order 5**	0	1	9	24	0	7	weeks 10, 16
9	demand backordered	0	2	8	24	1	7	weeks 10, 16

The receipt at week 10 is the earliest event on the calendar, ahead of the demand at week 11, so the clock advances to it. It first credits the interval from week 9 to week 10, during which $B = 2$, so A_B becomes 3. Then the five units arrive, the two backorders are filled from them, and $I = 3$ with $B = 0$. The position stays at 8, as a receipt's always does. The demands at weeks 11 and 13 are filled, the second one crediting $2 \times 2 = 4$ for the two weeks before it.

Table 11.4.: The complete event ledger. The state columns repeat Table 8.2; the area columns are what a simulation adds.

Week	Event	I	B	IP	A_I	A_B	A_+	Receipts due
0	start	6	0	6	0	0	0	none
1	demand	5	0	5	6	0	1	none
2	demand, **order 5**	4	0	9	11	0	2	week 10
3	demand	3	0	8	15	0	3	week 10
5	demand	2	0	7	21	0	5	week 10
6	demand	1	0	6	23	0	6	week 10
7	demand	0	0	5	24	0	7	week 10
8	demand backordered, **order 5**	0	1	9	24	0	7	weeks 10, 16

Week	Event	I	B	IP	A_I	A_B	A_+	Receipts due
9	demand backordered	0	2	8	24	1	7	weeks 10, 16
10	**receipt of 5**	3	0	8	24	3	7	week 16
11	demand	2	0	7	27	3	8	week 16
13	demand	1	0	6	31	3	10	week 16

With $T = 13$ weeks, Equation 11.2 and Equation 11.3 give

$$\bar{I} = \frac{31}{13} = 2.3846, \qquad \bar{B} = \frac{3}{13} = 0.2308, \qquad \overline{RR} = \frac{10}{13} = 0.7692, \qquad \overline{FR} = \frac{8}{10} = 0.8000$$

and the storeroom placed two orders, a rate of $2/13 = 0.1538$ a week.

The check. Two identities must hold, and both catch a misplaced credit. First, the shelf was empty from week 7 to week 10, so $A_+(13)$ plus those three weeks must be the whole observation: $10 + 3 = 13$. Second, the area under B is the total time demands spent waiting. The week 8 demand waited two weeks and the week 9 demand waited one, which is 3 backorder-weeks, and $A_B(13) = 3$. That is Little's law, exactly as Section 9.5.3.1 found it, and it gives the average wait per demand as $3/10 = 0.3$ weeks either way. That check is worth performing on every event ledger, because an area credited after the state changed fails it at once.

Interpretation. Averaging the rows of Table 11.4 would give $27/12 = 2.25$ for on hand and $3/12 = 0.25$ for backorders. Both are wrong, and the second is wrong by the same 0.25 against 0.2308 that Section 9.5.3.1 warned about. Notice also that the ready rate, 0.7692, and the fill rate, 0.8000, disagree although nothing here is unusual. Ten demands is a small sample of the shelf's state, and Equation 8.2 is a statement about the long run.

Validation, and its limit. Given these demand times, there is nothing random left, so a correct simulation must reproduce Table 11.4 **exactly**, row for row, and not merely within a half-width. That makes the ledger the first test of the companion code: the model of Section 11.3 is driven by these ten demand times and its trace is compared with the table.

The companion code passes that test. Driven by these ten demand times, the model of Section 11.3 posts eleven events whose states and areas equal Table 11.4 to the last digit, and it ends at week 13 with $A_I = 31$, $A_B = 3$ and $A_+ = 10$. Its two orders and its receipt fall on the same rows as the ledger's.

What the ledger cannot test is everything that depends on chance: whether the demand times are sampled correctly, and whether the long-run averages approach the values of Chapter 8. In addition, the observation starts with 6 on hand and nothing on order, which is not where the system usually is, and a thirteen-week average is dominated by that start. Section 11.3 takes up the first question and Section 11.4 the second.

The event view adds nothing to the policy. The changes in Table 11.1 are the rows of Table 8.2, stated once as rules instead of once per week. What it adds is a way of computing averages that cannot be done by hand for a year of demands, and does not need to be: Equation 11.1 is the same rectangle each time. Section 11.3 turns these rules into a KSL model and asks it the question the ledger could not, which is whether its long-run averages agree with the exact ones.

11.3. Building the (r, Q) Model

The model of this section is the one Rossetti (2026) builds in its chapter on event modeling. The companion code copies it rather than depending on it, so that its names can follow this book's notation and it can carry the options the later sections need. Its structure is Table 11.1 turned into code. The demand event is a method that fills the unit from the shelf or owes it, and then takes the decision. The receipt event is a method that moves units from on order to on hand and fills what is owed. The decision is a third method, which compares the position with r and orders the fewest whole batches that lift it above r. With demand one unit at a time that is always one batch; with demand in lots it can be more, and Section 11.5 needs it to be right.

Notice what the stock point does not know: how its orders are filled. It hands each order to a supplier, an object whose only obligation is to deliver the order eventually, and the supplier decides when. Here the supplier waits a constant two months. Section 11.5 replaces it with one whose lead times vary, and Section 11.8 with another stock point, and the stock point's own code changes in neither case. Section 11.10 lists the classes.

The decision is the shortest of the three, and it is the policy:

```
override fun checkPosition() {
    if (inventoryPosition <= reorderPoint) {
        val batches = (reorderPoint - inventoryPosition) /
         ↪  orderQuantity + 1
        placeOrder(batches * orderQuantity)
    }
}
```

Here `inventoryPosition` is $IP = I + IO - B$, and `reorderPoint` and `orderQuantity` are r and Q. The division is between whole numbers, so

`batches` is $\lfloor (r - IP)/Q \rfloor + 1$, the fewest batches that lift the position above r. `placeOrder` adds the order to IO and hands it to the supplier.

The demand event takes what the shelf can give and owes the rest:

```
val now = minOf(onHand, amount)
if (now > 0) {
    myOnHand.decrement(now.toDouble())
    receiver?.receive(now)
}
val rest = amount - now
// ...
myBackordered.increment(rest.toDouble())
myOwed.enqueue(Owed(rest, receiver, time))
// ...
checkPosition()
```

`myOnHand` and `myBackordered` are I and B, each a KSL `TWResponse`, so every change to them closes a rectangle of Equation 11.1. `myOwed` is the queue of backordered demands, filled first come first served. `receiver` is null for a customer and is the ordering stock point when the demand is another location's order, which is the seam Section 11.10.2 builds on. The decision comes last, after the state has changed.

The receipt reverses the order of the demand's bookkeeping:

```
fun receiveReplenishment(amount: Int) {
    myOnOrder.decrement(amount.toDouble())
    myOnHand.increment(amount.toDouble())
    fillOwed()
    checkPosition()
    // ...
}
```

`myOnOrder` is IO. `fillOwed` gives the new stock to the backorder queue in its order, part of a lot if only part is on hand. Notice that the receipt calls the decision too, although under (r, Q) it can never order: a receipt leaves the position where it was. The call costs nothing and keeps every policy free to react to a receipt.

Example 11.2 (The transformer at its optimal policy, simulated)**.** The storeroom runs the transformer at the policy Example 8.7 found, $(r, Q) = (8, 7)$. Demand is $\lambda = 45$ units a year, the supplier takes exactly two months, an order costs $k = \$220$, a unit held costs $h = \$950$ a year, and a unit owed costs $b = \$8{,}550$ a year. Every measure of this policy is known exactly, so the simulation has nothing to discover. Its purpose is to show that it can reproduce what is known.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Continuously, at every demand.

What triggers replenishment, and how much? The position reaching $r = 8$ or below, and the order is for $Q = 7$.

What happens to unmet demand? It is backordered and filled first come first served.

What costs are incurred, and when? k for each order placed, h per unit per year on hand, and b per unit per year owed, as Equation 8.1 prices them.

Notation. The model works in years, so the lead time is $L = 1/6$ year and the time between demands is exponential with mean $1/45$ year. The run settings are the chapter's standard ones from Section 11.4.3: $n = 30$ replications of $T = 1{,}000$ years after a warm-up of $T_w = 20$ years. Each replication starts with $r + Q = 15$ on hand and nothing on order.

The values to reproduce. Table 11.5 lists every measure the model reports, with its exact value from Example 8.7 and Example 8.12.

Table 11.5.: The exact measures of the transformer at $(8, 7)$ and the simulated ones, each with its 95% half-width, from 30 replications of 1,000 years, the demand on stream 1 of KSL R1.7.1.

Measure	Exact	Simulated
$\bar{I}$, units	4.6617	4.6570 ± 0.0107
$\bar{B}$, units	0.1617	0.1628 ± 0.0020
$\overline{RR}$	0.8781	0.8779 ± 0.0009
$\overline{FR}$	0.8781	0.8776 ± 0.0010
$\overline{OF}$, orders a year	6.4286	6.4333 ± 0.0105
Ordering cost	\$1,414.29	\$1,415.32 $\pm$ 2.30
Holding cost	\$4,428.59	\$4,424.13 $\pm$ 10.20
Backorder cost	\$1,382.28	\$1,392.05 $\pm$ 16.94
Total cost	\$7,225.15	\$7,231.50 $\pm$ 15.42

The check. Each exact value should lie inside its interval. With nine measures at 95% each, one miss is not alarming and two are worth a look; the intervals are not independent, since all nine come from the same runs. Two identities must also hold within each replication, whatever the random numbers: $\bar{I} - \bar{B}$ must equal the average net inventory, and the ready rate and fill rate must agree to within their half-widths, by Equation 8.2.

Every one of the nine intervals covers its exact value, and the ready rate and fill rate agree to the third decimal, 0.8779 against 0.8776, as Equation 8.2 requires.

Interpretation. A storeroom running $(8, 7)$ holds about four and two thirds transformers on average, owes about one sixth of one, and orders six or seven times a year, at a cost the simulation puts at \$7,231.50 $\pm$ \$15.42 a year. None of that is news; chapter 8 said it exactly. What the agreement licenses is the machinery: the events, the decision, the supplier and the areas all do what Section 11.2 says they do, on a run of 45,000 demands per replication in which a systematic error of a fraction of a unit would have shown.

Validation, and its limit. Agreement here establishes the model under two assumptions, a constant lead time and single-unit Poisson demand, and under no others. Section 11.5 removes them one at a time, and from then on there is no exact column to compare with.

11.3.1. Where This Leaves Us

The model reproduces chapter 8 where chapter 8 is exact. Before it is trusted anywhere else, the run settings it used need justifying: how long to run, where to start, and how many replications to make. Section 11.4 does that.

11.4. Output Analysis for a Stock Point

Example 11.1 produced four averages from thirteen weeks, and none of them is an estimate of anything Chapter 8 computed. The run was too short, it started from a state chosen for convenience, and it was one run. This section says how long to run, how to begin, how many runs to make, and how to report what comes out. The methods are those of Rossetti (2026), whose chapter on the analysis of simulation output treats them in full. What follows is their use on a stock point, and the two places where a stock point needs more care than a queue.

11.4.1. A Stock Point Never Terminates

Rossetti (2026) separates simulations whose runs have a natural end from those that do not. A bank that closes at five o'clock is the first kind. A storeroom is the second: the item is stocked permanently and demand arrives forever, so there is no last customer to stop at. Every measure in Chapter 8 is a **long-run average**, the value the time average approaches as the observation grows without bound. Thus, a simulation of a stock

point is a **non-terminating** simulation, and what it estimates is the steady state.

That has two consequences. The run must be long enough for its average to be close to the long-run one, and it must not be distorted by where it began. The second is the one specific to inventory, so we take it first.

11.4.2. Where to Start the Run

A run has to begin somewhere, and no state the modeler writes down is typical. Example 8.1 began with 6 on hand and nothing on order. The KSL book's inventory model begins by default with $r + Q$ on hand and nothing on order. Both are states the system passes through, but in the long run it is rarely in either. For the transformer at $(8, 7)$, starting with 15 on hand and nothing on order puts the shelf far above its long-run average of 4.66 units. With nothing in transit, the whole position is on the shelf, and it stays there until demand draws it down.

The bias of the start is temporary, and for an (r, Q) policy its length can be reasoned out. Recall Equation 8.17: the net inventory at time t is the position at time $t - L$ less the demand in between. Thus, the start is forgotten once two things have happened. A lead time must have passed, so that every unit in transit was ordered by the policy rather than assumed at the start. In addition, the position must have cycled through its band of Q values often enough that its value is no longer predictable from the start, which takes a few multiples of Q/λ, the time to consume one batch.

For the transformer at $(8, 7)$ that is a lead time of one sixth of a year and a cycle of $7/45 = 0.16$ years. A year of simulated time, about 45 demands, is several of each, and the start should be forgotten well within it.

That argument is particular to this policy. For a system that offers no such argument, the general method is the **Welch plot** of Rossetti (2026): plot the on-hand level averaged across replications at each instant, smooth

it with a moving window, and choose the **warm-up** T_w as the time after which the smoothed curve no longer trends. Statistics collected before T_w are discarded, and the averages are taken over the remainder of the run.

Choose the warm-up long rather than short. The cost of discarding too much is run time, which is cheap for a stock point. The cost of discarding too little is a bias that no number of replications removes, because every replication carries the same one. The chapter's runs use $T_w = 20$ years for the transformer, which is far longer than the reasoning above requires, and it costs 2% of the run.

11.4.3. Replications and Their Length

The method used throughout this chapter is **replication-deletion**. Let n represent the number of replications, let T represent the length of each replication after its warm-up, and let Y_j represent the time average of some measure over replication j, for $j = 1, \ldots, n$. The replications use independent random numbers, so $Y_1, \ldots, Y_n$ are independent and identically distributed, and the textbook confidence interval applies to them:

$$\bar{Y} = \frac{1}{n} \sum_{j=1}^{n} Y_j, \qquad hw = t_{n-1,\,\alpha/2} \, \frac{s_Y}{\sqrt{n}} \tag{11.4}$$

Here s_Y is the sample standard deviation of the Y_j and hw is the **half-width** of a $100(1-\alpha)\%$ confidence interval, $\bar{Y} \pm hw$. Recall from Section B.2.2 that the half-width falls with $\sqrt{n}$.

Notice what is being averaged. Within one replication, successive values of $I(t)$ are strongly dependent: the shelf an hour from now is nearly the shelf now. Equation 11.4 never uses those values directly. It uses one number per replication, and the dependence within a replication is absorbed into that number's variance. That is why the method is safe, and it is also why

each replication must be long. A short replication produces an average dominated by a few lead times, and the Y_j then scatter widely.

The alternative is **batch means**, one long run cut into consecutive batches whose averages are treated as nearly independent. It spends the warm-up once rather than n times. For a stock point, whose warm-up is short, that saving is small, and replication-deletion is the simpler of the two to report. The interested reader should refer to Rossetti (2026) for batch means and for the tests that justify a batch size.

Measure the run in demands, not in years. Information arrives with demands. The transformer sees 45 a year, so a run of $T = 1{,}000$ years is 45,000 demands, about 6,400 orders, and about 5,500 backordered demands at the optimal policy's fill rate of 0.8781. The fuse cutout of Section 8.11 sees 400 job lots a year, so the same number of demands takes about 112 years. Thus, run lengths are stated per item, and a run length copied from one item to another is a guess.

The chapter's standard settings for the transformer are $n = 30$ replications of $T = 1{,}000$ years after a warm-up of $T_w = 20$ years. The rest of this section explains how to check whether they are enough.

11.4.4. Three Fill Rates, and How to Average Each

Recall from Equation 11.3 that a replication's fill rate is a ratio of two counts, $N_F(T)/N_D(T)$. Across replications there are two ways to combine those ratios. One averages the n ratios. The other adds up the counts first and divides once, $\sum N_F / \sum N_D$. When every replication sees nearly the same number of demands, the two agree to several decimals. The interval should be computed from the per-replication ratios, since those are the independent observations Equation 11.4 needs.

The larger question is which ratio to collect, and it arises as soon as a demand can be for more than one unit. Recall from Section 8.11.4 that job lots split the fill rate in two. The *unit fill rate* is the proportion of

units demanded that are filled from the shelf on arrival, and the *lot fill rate* is the proportion of lots filled complete on arrival. The *ready rate*, the proportion of time with stock on the shelf, is a third measure. Under single-unit Poisson demand all three coincide by Equation 8.2. Under job lots they separate. For example, a lot of four that finds two on the shelf counts as unfilled for the lot fill rate and as two of four units for the unit fill rate, while it arrived at an instant that counts toward the ready rate.

Chapter 8 computed all three exactly for the fuse cutout at $(r, Q) = (25, 75)$: a ready rate of 0.9060, a unit fill rate of 0.8978 and a lot fill rate of 0.8957 (Table 8.21). A simulation can collect all three at no extra cost: a counter for lots filled complete, a counter for units filled, and a time-persistent indicator of stock on hand. **Name the one you report.** Section 11.5 runs the cutout, reproduces those three figures, and then measures them under an (s, S) policy, where chapter 8 did not compute them.

11.4.5. How Many Replications

Fix the precision first and let it set n. Suppose we want the cost of the transformer at $(8, 7)$ to within 1% of its value, that is, a half-width of about \$72 on \$7,225.15. A pilot run of n_0 replications gives a sample standard deviation s_0 and a half-width hw_0. Since the half-width falls with $\sqrt{n}$, the number of replications needed for a target half-width hw^* is approximately

$$n \approx n_0 \left(\frac{hw_0}{hw^*} \right)^2 \tag{11.5}$$

rounded up. Rossetti (2026) gives the more careful version, which iterates on the t quantile; at the sample sizes used here the difference is a replication or two.

For the transformer at $(8, 7)$, a pilot of $n_0 = 10$ replications of $T = 1{,}000$ years gives $s_0 = \$33.69$ and $hw_0 = \$24.10$. That is already a third of the target, and Equation 11.5 gives

$$n \approx 10 \left(\frac{24.10}{72} \right)^2 = 1.1$$

so the pilot itself meets the 1% target with room, and the standard $n = 30$, whose half-width is \$15.42 in Table 11.5, meets it easily. Shorter replications change the answer. With $T = 100$ years the pilot gives $s_0 = \$120.41$ and $hw_0 = \$86.14$, and

$$n \approx 10 \left(\frac{86.14}{72} \right)^2 = 14.3$$

so 15 replications of a hundred years are needed for the same precision.

Notice that n and T trade against each other. Doubling T roughly halves the variance of each Y_j, which does what doubling n does, at the same total run time. The trade is not quite even, because each replication spends a warm-up, but for a stock point the warm-up is short and either lever works.

11.4.6. The Measure That Needs the Longest Run

A single run length does not deliver the same precision for every measure. The cost and the on-hand average are estimated well by a modest run, because they change at every demand. The backorder average is the hard one.

Recall from Example 8.4 that at $(9, 5)$ the mean backorder level is 0.1202 units but its standard deviation is 0.582, nearly five times larger. The typical state is no backorders at all, interrupted by occasional episodes of several. A time average of such a process is driven by how many episodes

the run contains, and episodes are rare. Thus, the **relative half-width** $hw/\bar{Y}$ of $\bar{B}$ is far larger than that of $\bar{I}$ from the same run, and it grows as the policy improves. At $(11, 5)$, where the fill rate is 0.9683, a demand is backordered about a third as often as at $(9, 5)$.

The standard runs show it. At $(8, 7)$ the relative half-widths are 0.23% for $\bar{I}$, 1.22% for $\bar{B}$, 0.12% for the fill rate and 0.21% for the cost. At $(11, 5)$ they are 0.20%, 2.24%, 0.06% and 0.11%. Thus, the backorder level is the least precise measure by a factor of five or more, and it is the only one that gets less precise as the service improves.

The rule that follows is simple. **Set the run length by the hardest measure you will quote**, which is usually the backorder level of a well-served item, and check its relative half-width before quoting it. At $(11, 5)$ the cost is known to about a tenth of a percent while the backorder level is known only to about two percent, because backorders are a small share of the cost, and a report that quotes both should say so.

None of this can be checked against a formula until a model exists to run. Section 11.3 builds it, and the first thing it does is run the transformer at $(8, 7)$ with these settings and compare every measure with Example 8.7.

11.5. Where the (r, Q) Formulas Stop

Example 11.2 rests on three assumptions that the formulas of Section 8.5 need and the model does not: orders arrive in the sequence they were placed, demand arrives one unit at a time, and a demand that finds the shelf empty waits. This section removes them one at a time. Each example changes one component of the validated model and nothing else: the supplier, then the demand process with the policy that suits it, then the rule for an unmet demand. Thus, whatever changes in the results is the effect of the assumption removed, and not of a new model.

11.5.1. Orders That Overtake One Another

Example 11.3 (The transformer from an unreliable supplier)**.** The transformer's supplier no longer delivers in exactly two months. As in case 3 of Example 8.2, each order passes through four stages averaging half a month each, a purchase order, a production slot, a shipment and a receiving inspection, so its lead time is Erlang with shape 4 and rate 24 a year: two months on average, with a standard deviation of one month. Table 8.7 found the best policy for this supplier at $(r, Q) = (10, 8)$, costing \$11,079.73 a year, and Section 8.5.4 stated the condition under which that figure holds: orders must arrive in the sequence they were placed.

Two supply arrangements give each order the same Erlang lead time and differ only in that condition.

- **Independent lead times.** Each order takes its own lead time, whatever happened to earlier orders, so a later order can arrive first.
- **One supply line.** Orders queue for the four stages in sequence, so an order is received at its own lead time or at the receipt of the order before it, whichever is later. No order overtakes another, and an order can be delayed by its predecessor.

Neither arrangement is the one Table 8.7 assumed, since independent lead times and an unbroken sequence cannot both hold. The question is how far each is from the figure chapter 8 computed, and in which direction.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Continuously, at every demand.

What triggers replenishment, and how much? The position reaching $r = 10$ or below, and the order is for $Q = 8$.

What happens to unmet demand? It is backordered and filled first come first served.

What costs are incurred, and when? $k = \$220$ an order, $h = \$950$ per unit per year on hand and $b = \$8{,}550$ per unit per year owed.

Notation. Let L_m represent the lead time drawn for the m-th order and t_m the time it was placed. Under independent lead times the m-th order is received at $t_m + L_m$. Under one supply line it is received at $\max(t_m + L_m,$ the receipt time of order $m-1)$. The model records, for each order, whether it arrived ahead of an order placed before it. The standard run settings apply.

The prediction. With lead time demand negative binomial with $r = 4$ and $p = 0.3478$, exact for an Erlang lead time by Example 8.2, the formulas of Section 8.5 give Table 11.6.

Both arrangements are run with common random numbers. They face the same demands, and since the decision never looks at when an order will arrive, they place the same orders and draw the same lead times for them. Only the rule for delivering an order differs.

Table 11.6.: What Table 8.7's policy delivers under chapter 8's formulas and under the two supply arrangements, simulated.

Measure	Chapter 8	Independent lead times	One supply line
$\bar{I}$, units	7.3360	7.1860 ± 0.0250	7.0717 ± 0.0257
$\bar{B}$, units	0.3360	0.2047 ± 0.0044	0.3652 ± 0.0079
$\overline{FR}$	0.8913	0.9053 ± 0.0012	0.8822 ± 0.0015
$\overline{OF}$, orders a year	5.6250	5.6290 ± 0.0092	5.6290 ± 0.0092
$\overline{IO}$, units	7.5	7.5183 ± 0.0271	7.7930 ± 0.0309
Orders that overtake another	none	9.11% ± 0.16%	none
Total cost	\$11,079.73	\$9,815.07 ± 31.47	\$11,078.65 ± 55.13

The check. Two relations must hold under both arrangements. The order rate is $\lambda/Q = 5.625$ a year whatever the lead times do, since every order is for eight units and all demand is eventually filled. In addition, the average net inventory is the average position less the average number of units on order: $\bar{I} - \bar{B} = r + (Q+1)/2 - \overline{IO}$, which is $14.5 - \overline{IO}$ here. By Little's law $\overline{IO}$ is λ times the average time an order spends outstanding. Under independent lead times that time is 1/6 year and $\overline{IO} = 7.5$; under one supply line it is longer, because an order can wait for its predecessor, and the model measures it.

Both relations hold. The order rates cover 5.625. Under independent lead times $\bar{I} - \bar{B} = 6.9813$ against $14.5 - 7.5183 = 6.9817$, and an order is outstanding 0.1670 year on average. Under one supply line $\bar{I} - \bar{B} = 6.7065$ against $14.5 - 7.7930 = 6.7070$, and an order is outstanding 0.1731 year, about two days longer, because one order in eleven waits for the order ahead of it.

Interpretation. Under independent lead times, about one order in eleven overtakes an earlier one, and the policy runs \$1,264.66 a year, or 11.4%, cheaper than chapter 8 predicts, with 39% fewer backorders. Section 10.11 said the model over-protects when orders overtake one another, and this is by how much. A supplier whose orders cross is better than the formula gives it credit for, because a late order is often covered by the next one arriving early.

Under one supply line, the cost matches chapter 8 to within its half-width, but not because the formula describes that arrangement. The supply line holds orders back, so there are fewer units on hand and more owed than the formula predicts, and the two errors nearly cancel in the cost. Thus, the agreement of a total is weak evidence that a model is right. Here the components disagree, and the components are what the storeroom would see.

Validation, and its limit. The lead time machinery has its own exact check. At base stock 11, which is $(r, Q) = (10, 1)$, Section B.3.3 says the number of units on order is Poisson with mean 7.5 whatever the lead time distribution,

so the measures are those of Example 8.3 even with Erlang lead times and orders crossing. The simulation gives 7.5049 ± 0.0128 on order, a ready rate of 0.8621 ± 0.0014 against 0.8622, backorders of 0.1616 ± 0.0024 against 0.1616, and on hand of 3.6567 ± 0.0114 against 3.6616. Nothing exact covers $Q = 8$, which is the point of the example. In addition, the policy $(10, 8)$ was chosen for chapter 8's formula; under independent lead times a lower reorder point may be cheaper still, and Exercise 11.6 asks a related question.

11.5.2. Job Lots and the (s, S) Policy

Example 11.4 (The cutout under an order-up-to rule, simulated)**.** The fuse cutout of Example 8.8 is drawn in job lots of one, two or four units, with probabilities 0.40, 0.40 and 0.20, at 400 lots a year. The supplier takes two weeks. An order costs $k = \$82.50$, a unit held costs $h = \$28.75$ a year and a unit owed costs $b = \$287.50$ a year. Example 8.9 priced two (s, S) policies exactly, by solving the Markov chain of the inventory position, and this example reproduces those prices by a different route.

What is stocked? One item, the fuse cutout.

What is the demand process? Lots arrive in a Poisson process at 400 a year, each of one, two or four units.

When is inventory reviewed? Continuously, at every lot.

What triggers replenishment, and how much? The position reaching s or below, and the order brings it up to S, so the order size is random.

What happens to unmet demand? A lot is filled from what is on the shelf and the remainder is backordered, first come first served, as Section 8.11 assumed.

What costs are incurred, and when? k for each order placed, h per unit per year on hand, and b per unit per year owed.

Notation. The model works in years, with $L = 1/26$ year. Let U represent the undershoot of Equation 8.55, recorded at every order. The run is measured in lots, so $T = 112$ years gives about the 45,000 demands of the transformer's standard run; $n = 30$ and $T_w = 5$ years.

The values to reproduce. Table 11.7 collects what Example 8.9 computed at two policies: one where the spread $S-s$ is wide and the (r, Q) approximation is good, and one where it is narrow and the correction of Equation 8.59 is needed.

Table 11.7.: The exact figures for the cutout under (s, S), from Table 8.23 and Example 8.9, and the simulated ones, from 30 replications of 112 years after a 5-year warm-up.

	$(36, 56)$ exact	simulated	$(25, 100)$ exact	simulated
$S - s$	20		75	
Cost, a year	\$3,755.69	\$3,755.96 ± 5.88	\$2,010.10	\$2,009.86 ± 2.66
Orders a year	38.462	38.466 ± 0.066	10.554	10.558 ± 0.018
$E[U]$, units	0.8	0.8015 ± 0.0068	0.8	0.7966 ± 0.0086

The undershoot has the distribution Example 8.9 gave: 0 half the time, 1 unit three times in ten, and 2 or 3 units one time in ten each. The simulation observes 0.5009, 0.2975, 0.1008 and 0.1008 at $(36, 56)$, and 0.4995, 0.3021, 0.1007 and 0.0977 at $(25, 100)$.

Table 11.8.: The three service measures for the cutout in job lots. The exact row is Table 8.21's.

	Ready rate	Unit fill rate	Lot fill rate
$(r, Q) = (25, 75)$, exact	0.9060	0.8978	0.8957
$(r, Q) = (25, 75)$, simulated	0.9059 ± 0.0010	0.8977 ± 0.0010	0.8958 ± 0.0010
$(s, S) = (25, 100)$, simulated	0.9072 ± 0.0011	0.8989 ± 0.0011	0.8968 ± 0.0011
$(s, S) = (36, 56)$, simulated	0.9219 ± 0.0009	0.9107 ± 0.0012	0.9076 ± 0.0012

The check. The observed undershoot distribution should match the equilibrium distribution of Equation 8.56 closely, value by value, since both spreads are wide compared with a lot of at most four, and the mean order size must be $S - s + E[U]$ by Equation 8.58. Both are checks on how the model handles a lot that carries the position past s.

Both hold. The observed frequencies match the equilibrium distribution to the second decimal at both spreads, and the mean order size is 20.8015 at $(36, 56)$ and 75.7966 at $(25, 100)$, which is $S - s$ plus the observed undershoot in each case.

Interpretation. Every exact figure is covered: the two costs, the two order rates, the undershoots, and in Table 11.8 all three of chapter 8's service measures at $(25, 75)$. The three measures, which coincided for the transformer, separate here by about a point: the ready rate exceeds the unit fill rate, which exceeds the lot fill rate, at every policy. Thus, a

storeroom quoting its ready rate of 0.92 at $(36, 56)$ is filling 0.91 of the units its crews ask for and 0.91 of their job lots complete, and the difference is the lots that arrive to a shelf holding some cutouts but not enough. Notice also that $(s, S) = (25, 100)$ and $(r, Q) = (25, 75)$ are indistinguishable on service and on cost; Table 8.23 said as much, since a spread of 75 against lots of at most four leaves the undershoot nothing to do.

Validation, and its limit. The costs are exact, and so are the three service measures at $(r, Q) = (25, 75)$, so together they validate the model under job lots. The service measures under (s, S) are where the simulation reports something Chapter 8 did not compute.

11.5.3. Demand That Is Lost

Example 11.5 (The transformer when a crew will not wait)**.** A crew that finds no transformer on the shelf now buys one from another utility at \$9,500, against a stock price of \$3,800, and the storeroom's demand is lost. That is the shortage cost $\pi =$ \$5,700 per unit short of Example 8.6, and with it chapter 8's exact search found the best (r, Q) at $(13, 6)$, costing \$11,892.91 a year. That figure treats a unit short as owed. This example asks what happens when it is lost instead.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Continuously, at every demand that is filled.

What triggers replenishment, and how much? The position reaching r or below, and the order is for $Q = 6$.

What happens to unmet demand? It is lost. The demand departs and nothing is owed, so a lost demand does not lower the inventory position.

What costs are incurred, and when? $k =$ \$220 for each order placed, $h =$ \$950 per unit per year on hand, and $\pi =$ \$5,700 for each unit lost.

Notation. Let $N_L(T)$ represent the number of demands lost by time T, so the lost sales rate is $N_L(T)/T$. Under lost sales $B(t) = 0$ throughout, and the position is $I(t) + IO(t)$. The standard run settings apply, at two policies: $(13, 6)$, where a shortage is rare, and $(8, 6)$, where it is not.

The prediction. Charging a lost unit as though it were backordered gives the figures of Table 11.9, computed exactly from Equation 8.19 and Equation 8.20 with shortages counted at the rate $\lambda(1 - \overline{FR})$.

Table 11.9.: The backorder model's figures at two policies, with each unit short charged π, beside the lost-sales system simulated.

	$(13, 6)$, backorder model	lost sales, simulated	$(8, 6)$, backorder model	lost sales, simulated
$\overline{FR}$	0.9934	0.9944 ± 0.0002	0.8595	0.9070 ± 0.0006
Units short a year	0.2961	0.2530 ± 0.0077	6.3214	4.1872 ± 0.0334
$\bar{I}$, units	9.0052	9.0469 ± 0.0131	4.1873	4.7400 ± 0.0087
Orders a year	7.5	7.4633 ± 0.0122	7.5	6.8075 ± 0.0086
Ordering cost	\$1,650.00	\$1,641.92 ± 2.69	\$1,650.00	\$1,497.64 ± 1.89
Holding cost	\$8,554.91	\$8,594.53 ± 12.45	\$3,977.94	\$4,503.04 ± 8.29
Shortage cost	\$1,688.00	\$1,442.29 ± 43.84	\$36,032.23	\$23,867.23 ± 190.34
Total cost	\$11,892.91	\$11,678.73 ± 46.09	\$41,660.17	\$29,867.92 ± 186.80

The check. Every demand is either filled or lost, so the fill rate and the lost

fraction must add to one exactly in each replication. In addition, orders are placed at the rate filled demand consumes stock, so the order rate must be $\lambda\,\overline{FR}/Q$, which is below the $\lambda/Q = 7.5$ a year the backorder model charges for.

Both hold. At $(13, 6)$ the lost fraction is $0.2530/45 = 0.0056$, which is $1-0.9944$, and $45(0.9944)/6 = 7.458$ orders a year lies inside the simulated 7.4633 ± 0.0122. At $(8, 6)$ the lost fraction is $4.1872/45 = 0.0930 = 1 - 0.9070$, and $45(0.9070)/6 = 6.8025$ lies inside 6.8075 ± 0.0086.

Interpretation. At $(13, 6)$ the backorder model is close. It overstates the cost by \$214, or 1.8%, almost all of it in the shortage term, because fewer units go short when a lost demand leaves the position where it was. Shortages are rare at this policy, so it hardly matters what happens to them. At $(8, 6)$ it is far off: the backorder model charges 6.3 units short a year, the lost-sales system loses 4.2, and the total is overstated by \$11,792, or 39% of the true cost. The mechanism is the one the check exposed. A lost demand never lowers the position, so the system orders less often, the shelf recovers before the next demand finds it empty, and the fill rate rises from the backorder model's 0.8595 to 0.9070. Thus, charging lost sales as backorders is safe for a policy that seldom runs short, which is the only kind worth running at $\pi =$ \$5,700, and badly pessimistic for one that runs short often.

Validation, and its limit. Nothing in the book computes a lost-sales system, so the only checks are the identities above and the agreement of the lost-sales model with the validated backorder model when r is high enough that no demand is short. The example is the case Section 1.3.2 anticipated, and the remedy it tests is the one Section 11.1 named.

11.5.4. Where This Leaves Us

Each of the three examples changed one component and kept the rest, which is the discipline that lets a simulated difference be read as the effect of

one assumption. Section 11.6 changes a different thing: not what happens to demand, but when anyone looks at the stock.

11.6. Periodic Review

Periodic review changes when the decision is taken. Under (r, Q) the decision rides on the demand event. Under periodic review it rides on a **review**, a third event that the policy schedules for itself every R time units, starting when the replication starts. A demand still changes I, B and IP, but it takes no decision. That is the only change to the model. Thus, Example 11.6 can validate it against the exact formulas of Section 8.13 before the next two examples use it for questions those formulas answer only by comparison or by approximation.

Example 11.6 (The transformer reviewed monthly, simulated)**.** The storeroom reviews the transformer once a month and orders up to $S = 14$, the level Example 8.11 found optimal. The supplier takes exactly two months, demand is 45 a year, and the costs are those of Example 11.2.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Once a month, at the start of each month.

What triggers replenishment, and how much? Every review, for $S - IP$ units, when that is positive.

What happens to unmet demand? It is backordered and filled first come first served.

What costs are incurred, and when? $k = \$220$ for each order placed, $h = \$950$ per unit per year on hand and $b = \$8{,}550$ per unit per year owed.

Notation. Let $R = 1/12$ year represent the review interval and $L = 1/6$ year the lead time. The model schedules a review at times $0, R, 2R, ...$, and

the review event observes the position and places the order. Demands and receipts are the events of Table 11.1, except that a demand no longer takes a decision. The standard run settings apply.

The values to reproduce. From Example 8.11:

Table 11.10.: The exact measures of the transformer under (R, S) with R one month and $S = 14$, and the simulated ones.

Measure	Exact	Simulated
$\bar{I}$, units	4.7751	4.7696 ± 0.0143
$\bar{B}$, units	0.1501	0.1507 ± 0.0025
$\overline{RR}$	0.8928	0.8925 ± 0.0012
$\overline{FR}$	0.8928	0.8929 ± 0.0013
Orders a year, Equation 8.64	11.7178	11.7194 ± 0.0068
Ordering cost	\$2,577.91	\$2,578.28 $\pm$ 1.50
Total cost	\$8,397.22	\$8,397.75 $\pm$ 19.49

The check. The simulation counts reviews and orders separately, and the two need not agree. A month with no demand at all leaves the position at S and nothing to order, which Equation 8.64 counts: with Poisson demand it happens with probability $e^{-\lambda R} = e^{-3.75} = 0.0235$, so the order rate is $12(1 - 0.0235) = 11.72$ a year. The interval for the order rate should cover 11.7178, and the fraction of reviews that place no order should cover 0.0235.

Both do. The simulation reviews twelve times a year and orders 11.7194 times, and the fraction of reviews that place an order is 0.9767 ± 0.0006 against $1 - 0.0235 = 0.9765$.

Interpretation. Every measure is covered. The review event behaves as the analysis of Section 8.13 assumes, down to the one review in forty-three that finds nothing to order.

Validation, and its limit. Agreement validates the review event. The policy it validates is the one Section 8.13 analyzed exactly, so the next two examples use it to answer questions that section answered only by comparison or approximation.

Example 11.7 (What continuous review is worth, measured directly)**.** Table 8.29 priced continuous review on the transformer at \$961.77 a year, 13.3% of the continuous cost, by comparing two exact costs: the continuous optimum $(r, Q) = (8, 7)$ at \$7,225.15, and the periodic optimum, $R = 0.120$ year (about 6.2 weeks) with $S = 15$, at \$8,186.92. This example measures the same difference by simulating both policies against the same demands.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Continuously under the first policy, and every 0.120 year under the second.

What triggers replenishment, and how much? Under the first, the position reaching 8 or below, for 7 units. Under the second, every review, up to 15.

What happens to unmet demand? It is backordered under both.

What costs are incurred, and when? Those of Example 11.2, for both.

Notation. Let C_c and C_p represent the simulated cost rates of the continuous and periodic policies in one replication, and let $\Delta = C_p - C_c$ represent their difference. The two models are driven by the same stream of demand times, which is what common random numbers means here, so the n values of Δ are paired. The standard run settings apply.

The values to reproduce. From Table 8.29: $\Delta =$ \$961.77 a year, and $\Delta/C_c = 13.3\%$. The two fill rates, 0.8781 and 0.8819, are equal to within half a percentage point, which is what makes the difference a price rather than a trade.

Table 11.11.: The two policies simulated against the same demands and against independent demands.

	Common random numbers	Independent streams
C_c	\$7,231.50 ± 15.42	\$7,231.50 ± 15.42
C_p	\$8,191.96 ± 24.05	\$8,182.21 ± 19.06
$\Delta = C_p - C_c$	\$960.47 ± 12.34	\$950.71 ± 27.61
Variance of Δ	1,092.73	5,468.41
Fill rate, continuous	0.8776 ± 0.0010	0.8776 ± 0.0010
Fill rate, periodic	0.8817 ± 0.0015	0.8825 ± 0.0014

The check. The interval for Δ should cover \$961.77, and the two fill rates should differ by about the 0.0038 chapter 8 computes; if they differ by much more, the two policies are not delivering the same service and the premium is not a like-for-like price.

Both hold. With common random numbers, Δ is \$960.47 ± \$12.34, and the fill rates differ by 0.0041.

Interpretation. The paired interval puts the price of not watching the transformer between \$948 and \$973 a year. The two separate intervals in the first column, read on their own, would only say that the premium lies somewhere between about \$920 and \$1,000. Run against independent demands, the difference is estimated with a variance five times larger, and a half-width of \$27.61 rather than \$12.34, from the same amount of simulation. That factor of five is what common random numbers buy, and it costs nothing but giving each source of randomness its own stream.

Validation, and its limit. Both costs are exact, so this example validates the use of common random numbers rather than either model. Section 11.7 relies on that.

Example 11.8 (How far the position falls past the trigger)**.** Section 8.13.5 showed that under periodic review the position passes below the reorder

point even when demand arrives one unit at a time. It put the mean undershoot at $\lambda R/2 = 1.875$ units for the transformer reviewed monthly when the spread $S - s$ is wide compared with a review interval's demand, and at exactly 2.8403 units at the narrow extreme, the (R, S) policy itself. Between the two, chapter 8 said only that the spread decides. This example measures the undershoot at three spreads to find out how quickly.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Once a month.

What triggers replenishment, and how much? A review finding the position at or below s, and the order brings it up to $S = 14$.

What happens to unmet demand? It is backordered.

What costs are incurred, and when? None are needed. The example measures the undershoot.

Notation. As in Example 11.6, with s set to 13, 10 and 6. Let $U = s - IP$ represent the undershoot of Equation 8.55, recorded at every review that places an order. The standard run settings apply.

The predictions. At $s = 13$ the policy is (R, S), and Section 8.13.5 gives $E[U] = 2.8403$ units exactly. At $s = 10$ and $s = 6$, spreads of four and eight units against 3.75 units of demand a month, Equation 8.75 predicts 1.875 units.

Table 11.12.: The undershoot at review for the transformer reviewed monthly, with $S = 14$.

s	$S - s$	Prediction	Mean undershoot	Reviews that order
13	1	2.8403, exact	2.8422 ± 0.0062	0.9767 ± 0.0006
10	4	1.875	1.7981 ± 0.0081	0.6472 ± 0.0009
6	8	1.875	1.8796 ± 0.0077	0.3798 ± 0.0006

The check. At $s = 13$ the prediction is exact, so the interval must cover 2.8403. That validates the recording of U before it is used where the prediction is only a limit. It does: 2.8422 ± 0.0062.

Interpretation. At a spread of eight units, about two months of demand, Equation 8.75 is right to the second decimal: 1.8796 against 1.875. At a spread of four, about one month of demand, it is 4% high, and the error is not in the direction a reader might guess: the undershoot is smaller than the long-run figure, not larger. Only at the narrowest spread, where every review orders, does it jump to the 2.84 of (R, S). Thus, the formula serves any (R, s, S) policy whose spread holds two review intervals of demand or more, which is the case Section 8.13.5 said the policy is for.

Validation, and its limit. Section 8.13.5 left the exact treatment of (R, s, S) to Zipkin (2000). The simulation does not replace it; it measures how good the approximation is on this item, which is the question a planner actually has.

11.6.1. Where This Leaves Us

Example 11.7 compared two policies by running them against the same demands, and the comparison was sharper for it. Section 11.7 makes that a method, and asks what it takes to choose among policies whose costs are close.

11.7. Comparing and Choosing Policies

Choosing a policy by simulation means comparing estimates, and an estimate carries noise. Two devices from Rossetti (2026) keep the noise from deciding the comparison: common random numbers, which make the policies face the same demands, and multiple comparison with the best, which says which policies can be told apart and which cannot. This section shows both on a comparison whose answer is already known.

11.7.1. Common Random Numbers

Let C_1 and C_2 represent the simulated costs of two policies in one replication. The quantity of interest is their difference, and its variance is

$$Var[C_1 - C_2] = Var[C_1] + Var[C_2] - 2\,Cov[C_1, C_2] \tag{11.6}$$

Run independently, the two policies have no covariance, and the difference is noisier than either cost. Run against the same demands, a replication with unusually heavy demand is expensive for both, the covariance is positive, and the difference is quieter than either. That is, common random numbers do not make either estimate more precise; they make the comparison more precise, which is the thing being asked. Recall from Section 7.10 that the newsvendor did the same with `resetStartStream`, so that every quantity faced the same hundred thousand demands.

An inventory model needs one precaution the newsvendor did not. Two policies place different numbers of orders, so if demand times and lead times are drawn from one stream, the policies' draws drift apart after the first order that one places and the other does not, and the demands are no longer common. **Give each source of randomness its own stream.** The demand process then supplies the same demand times to every policy, whatever the policies do with them. Under a constant lead time, as in the transformer examples, the lead times are common automatically.

11.7.2. Choosing the Best

With more than two policies, the question becomes which of them is best, and how sure we can be. Multiple comparison with the best, which Rossetti (2026) develops, compares each policy with the best of the others and returns the subset of policies that cannot be ruled out. Let δ represent the **indifference zone**, the smallest difference in cost worth distinguishing.

A difference below δ is one the planner has declared not to matter, and the procedure is entitled to call it a tie.

Choosing δ is a decision about the item, not about statistics. The transformer's costs run to about \$7,400 a year, so a difference of \$75, 1% of it, is a fair threshold: a planner would not reorganize a storeroom for less. Example 11.9 runs the procedure where the true ranking is known.

Example 11.9 (Five reorder points near the optimum)**.** Table 8.5 evaluated the transformer at $Q = 5$ for reorder points 7 through 11 and found $r = 9$ best. The costs near the optimum are close: $r = 8$ costs \$9.17 a year more than $r = 9$, about 0.12%. This example asks what it takes to see that by simulation.

What is stocked? One item, the pad-mount transformer.

What is the demand process? Poisson at 45 a year, one unit per demand.

When is inventory reviewed? Continuously.

What triggers replenishment, and how much? The position reaching r or below, for $Q = 5$, with r from 7 to 11.

What happens to unmet demand? It is backordered.

What costs are incurred, and when? Those of Example 11.2.

Notation. Let C_r represent the simulated cost rate at reorder point r in one replication. All five policies face the same demand times. Let δ represent the indifference zone of the multiple comparison, the smallest difference in cost worth distinguishing, set here at \$75 a year, about 1% of the cost. The standard run settings apply.

The values to reproduce.

Table 11.13.: The exact costs of the five policies, from Table 8.5, to the cent, and the simulated costs with common random numbers. An MCB interval that contains zero marks a policy that cannot be ruled out as the best.

r	7	8	**9**	10	11
Exact cost, a year	\$8,016.43	\$7,405.97	**\$7,396.80**	\$7,792.96	\$8,442.11
Above the best	\$619.63	\$9.17		\$396.16	\$1,045.31
Simulated cost	\$8,033.53	\$7,419.78	**\$7,405.71**	\$7,798.73	\$8,445.30
Half-width	\$29.03	\$19.41	\$13.38	\$9.93	\$9.32
MCB interval, $\delta = \$75$	$[0, 702.82]$	$[-60.92, 89.08]$	$[-89.08, 60.92]$	$[0, 468.02]$	$[0, 1114.59]$

The check. Every interval should cover its exact cost, and the procedure should return a subset containing $r = 9$.

Both hold. The five intervals cover the five exact costs, the smallest average is at $r = 9$, and the MCB intervals of $r = 8$ and $r = 9$ are the two that contain zero.

Interpretation. With $\delta = \$75$, multiple comparison with the best keeps $r = 8$ and $r = 9$ and rules out the other three. That is the right answer to the question the planner asked, which was which policies are worth considering: the two differ by \$9.17 a year, far inside the \$75 the planner said was not worth distinguishing.

The paired difference says something sharper, and it is worth seeing why it does not change the answer. With common random numbers, the cost at $r = 8$ less the cost at $r = 9$ is \$14.08 $\pm$ \$9.45, an interval that excludes zero, so even 30 replications can tell the two apart. The paired differences have a standard deviation of \$25.31, so by Equation 11.5 about 32 replications would put the half-width at \$9.17, the size of the true difference. Thus, separating the two policies is cheap. It is also pointless. A statistically significant difference of nine dollars a year on a \$7,400 item is not a reason to prefer one reorder point to the other, and the indifference zone is how the planner says so before the run rather than after it.

Validation, and its limit. With the exact costs known, the example shows what the comparison procedure does when the answer is a near tie. When no exact cost is available, as under lost sales, the same procedure is all there is.

11.7.3. Searching for a Policy

Comparing a handful of policies is enumeration. When the candidates number in the hundreds, or the cost surface has no formula to guide the search, the problem is simulation optimization: a search over policies in which every evaluation is a noisy estimate. The solvers for it, among them cross-entropy, simulated annealing and R-SPLINE, are the subject of a chapter of Rossetti (2026), which builds a problem definition for the (r, Q) policy and runs each solver on it. The interested reader should refer to it.

Two pieces of advice carry over from the earlier chapters of this book. Start the search from the exact answer to the nearest model the book can solve: for the transformer under lost sales, Example 11.5, that is the backorder optimum $(13, 6)$. And confirm the search's final candidates with common random numbers and a multiple comparison, since a search over noisy estimates reports the candidate that was luckiest as readily as the one that is best. Exercise 11.15 does the search by enumeration.

11.7.4. Where This Leaves Us

Every model so far has had one stock point. Section 11.8 builds the two-level systems of chapters 9 and 10 from the same classes, which is where the approximations those chapters needed can finally be tested.

11.8. Multi-Echelon Systems

Chapters 9 and 10 needed approximations for one reason. The upper level's backorders make each lower location's resupply time random, and the waits of different requests move together, and no formula carries that coupling whole. A simulation carries it because it has no choice: a storeroom's request waits at the depot exactly as long as the depot's queue makes it wait, and the storeroom's stock reflects that wait. Thus, a two-level model is the single-location model twice, with one location as the other's supplier, as Section 11.10.2 explains.

The validation follows the chapter's pattern. Example 11.10 first reproduces every quantity chapter 9 computed exactly and then settles the one in dispute. Example 11.11 reproduces chapter 10's exact system with batching at the hub. Example 11.12 then runs the case nothing exact covers.

Example 11.10 (METRIC or VARI-METRIC for the utility's transformers)**.** The utility of Example 9.4 runs four storerooms supported by a central depot. Each storeroom sees 45 failures a year, rebuilds three in ten in its own shop in 30 days, and sends the rest to the depot, from which a serviceable unit takes 10 days to arrive. The depot repairs what it receives in 60 days. The plan holds $S_0 = 16$ transformers at the depot and $S_j = 4$ at each storeroom. METRIC predicted 0.4055 backorders at each storeroom and VARI-METRIC predicted 0.4815, and this example measures which is closer.

What is stocked? One item at five locations, the pad-mount transformer.

What is the demand process? Poisson at 45 failures a year at each storeroom, one unit at a time.

When is inventory reviewed? Continuously, at every location.

What triggers replenishment, and how much? Every failure, for one unit, at both levels.

What happens to unmet demand? It is backordered, at the storeroom against a waiting crew and at the depot against a waiting storeroom, first come first served at each.

What costs are incurred, and when? None are needed. The example measures backorders.

Notation. As in Example 9.4. The model works in days, with the repair and shipping times taken as constants: 30 days for a local repair, 10 for a shipment and 60 for a depot repair. Every mean in Table 11.14 holds for any repair time distribution with these means, by Theorem 9.1 and Little's law. The disputed rows may depend on the shapes of those times, which Example 9.4 did not state and neither model uses; constants are the simplest choice consistent with the stated averages. Each failure is sent to the depot with probability 0.7, independently. The run is $n = 30$ replications of $T = 1{,}000$ years after $T_w = 5$ years; the system sees 180 failures a year.

The values to reproduce, and the value in dispute. Recall from Section 9.7.2 that every mean METRIC computes is exact, and that only the storeroom's backorders depend on the shape of its pipeline.

Table 11.14.: The two models on the same plan, from Example 9.4 and Example 9.5, and the simulation. The four storerooms are alike, so the simulated rows give the range across them.

Measure	METRIC	VARI-METRIC	Status	Simulated
$\bar{B}_0$, depot, units	5.0201	5.0201	exact	5.0098 ± 0.0216
$\bar{W}$, days	14.54	14.54	exact	14.52 ± 0.05
μ_j, storeroom pipeline, units	3.2276	3.2276	exact	3.2234 to 3.2282, each ± 0.008
$\bar{I}_j - \bar{B}_j$, units	0.7724	0.7724	exact	0.7719 to 0.7767
$\bar{B}_j$, per storeroom	0.4055	0.4815	in dispute	0.4813 to 0.4825, each ± 0.004
$\bar{I}_j$, per storeroom	1.1779	1.2539	in dispute	1.2537 to 1.2580, each ± 0.005
Backorders across four storerooms	1.6218	1.9259	in dispute	1.9269

The check. The four exact rows must be covered. If they are not, the model is wrong, and the disputed rows mean nothing.

They are: the depot's backorders and wait, and each storeroom's pipeline, cover the values both models share.

Interpretation. Each storeroom owes its crews 0.4817 transformers on average, across the four. VARI-METRIC's 0.4815 lies inside every storeroom's interval, and METRIC's 0.4055 lies far below all of them. Thus, on this plan METRIC understates the storerooms' backorders by about 16%, and VARI-METRIC is right to the third decimal. For the utility that is the difference between planning for 1.6 crews waiting at any moment and finding 1.9. Sherbrooke (1986) said METRIC understates; this plan shows by how much, and shows no sign that VARI-METRIC overstates.

Validation, and its limit. This is one plan on one system. Graves (1985) is the reference for how often each model chooses the wrong stock levels across many systems, and the example does not generalize from one case to that.

Example 11.11 (The cutout's hub and storerooms, simulated)**.** The running example of Section 10.2 has four storerooms, each seeing one fuse cutout demanded a week and running base stock $S_j = 3$, supplied by a hub that runs $(r_0, Q_0) = (9, 8)$ against a supplier taking three weeks. A shipment from the hub to a storeroom takes one week. Holding costs $h = \$0.5529$ per unit per week at both levels, and a storeroom's crews are owed at $b = \$5.5288$ per unit per week. Example 10.3 computed every measure of this system exactly.

What is stocked? One item at five locations, the fuse cutout.

What is the demand process? Poisson at one unit a week at each storeroom.

When is inventory reviewed? Continuously, at every location.

What triggers replenishment, and how much? At a storeroom, every demand, for one unit. At the hub, its position reaching 9 or below, for 8 units.

What happens to unmet demand? It is backordered at both levels. The hub fills its requests first come first served.

What costs are incurred, and when? Holding at both levels and backorders at the storerooms, at the rates above. There is no ordering cost in this example.

Notation. As in Section 10.2. Let W represent the time a storeroom's request waits at the hub, recorded for every request. The model works in weeks. The hub sees four requests a week, so a run of $T = 12{,}000$ weeks gives about the transformer's 45,000 demands at the hub; $n = 30$ and $T_w = 50$ weeks.

The values to reproduce.

Table 11.15.: The exact measures of the running example, from Example 10.1, Example 10.2 and Example 10.3, and the simulated ones, from 30 replications of 12,000 weeks after a 50-week warm-up.

Measure	Exact	Simulated
$\bar{B}_0$, hub, units	1.0273	1.0173 ± 0.0087
$\overline{RR}_0$	0.6030	0.6045 ± 0.0021
$\bar{I}_0$, units	2.5273	2.5334 ± 0.0124
$\bar{W}$, weeks	0.2568	0.2547 ± 0.0018
$P(W = 0)$	0.6030	0.6051 ± 0.0020
$P(W \leq 1)$	0.9117	0.9129 ± 0.0009
$P(W \leq 2)$	0.9985	0.9985 ± 0.0001
$\bar{B}_j$, per storeroom	0.0726	0.0707 to 0.0724, each ± 0.0016
System cost, a week	\$7.0195	\$7.0081

The check. $P(W = 0)$ must equal the hub's ready rate, as it did in Example 10.2, since a request ships at once exactly when the hub has stock. In addition, Little's law gives $\bar{W} = \bar{B}_0/4$ within each replication, and the model reports both sides.

Both hold: $P(W = 0)$ is 0.6051 against a ready rate of 0.6045, and $\bar{B}_0/4 = 0.2543$ against a mean wait of 0.2547.

Interpretation. This run is worth reading slowly, because four of its intervals miss: the hub's backorders, its mean wait, $P(W = 0)$ and $P(W \leq 1)$, together with one storeroom's backorders. They miss together, and in one direction: the hub had slightly more stock than its long-run average across these thirty replications, and every measure that depends on it moved the same way. That is what correlated intervals from one set of runs do, and it is why Section 11.10.4 checks such a family at the Bonferroni level, at which this run passes. Two further runs of 300 replications on other streams put the hub's backorders at 1.0283 ± 0.0027 and 1.0263 ± 0.0027, both covering 1.0273. The model is right; the first thirty replications were a little lucky.

The storerooms' backorders average 0.0717 across the four, beside the routes of Table 10.2: exact 0.0726, conditioned on the wait 0.0726, two moments 0.0712, and the mean wait only 0.0505. The simulation sits with the first three and far from the last, which is chapter 10's lesson that the mean wait alone understates a storeroom's backorders by 30%, seen from outside the mathematics.

Validation, and its limit. Every figure here is exact, so this example validates the two-level model with batching at the hub. The next example keeps the model and changes only the customers.

Example 11.12 (The cutout in job lots, the case chapter 10 handed on)**.** Example 10.10 set both policies for the cutout drawn in job lots, by the iteration of Algorithm 10.1, and closed by saying that nothing exact covers the case. A storeroom's crews draw lots of one, two or four units, with probabilities 0.40, 0.40 and 0.20, at 150 lots a year. The hub serves 250 lots a year of its own from the same distribution and also supplies the storeroom. The hub's supplier takes two months, and a transfer from the hub to the storeroom takes one. Both locations pay $k = \$82.50$ an order and $h = \$28.75$ a unit a year to hold; the storeroom's crews are owed at

$b = \$287.50$ a unit a year, and the hub carries $b_0 = \$115$, from its target of 0.8. The iteration converged to $(25, 49)$ at the storeroom and $(120, 92)$ at the hub.

What is stocked? One item at two locations, the fuse cutout.

What is the demand process? Compound Poisson at both locations: lots at 150 a year at the storeroom and 250 a year of the hub's own, each of one, two or four units.

When is inventory reviewed? Continuously, at both locations.

What triggers replenishment, and how much? At the storeroom, its position reaching 25 or below, for 49 units. At the hub, its position reaching 120 or below, for 92 units.

What happens to unmet demand? It is backordered at both. The hub fills its own customers and the storeroom's orders in one first-come-first-served queue, and ships part of a storeroom order when it has only part, as Section 10.2 specified.

What costs are incurred, and when? At each location, Equation 10.14: ordering, holding, and backorders at that location's rate.

Notation. The model works in years. Let W represent the time from a storeroom order's arrival at the hub until its last unit ships. The storeroom sees 150 lots a year, so $T = 300$ years gives 45,000 of them; $n = 30$ and $T_w = 5$ years.

The predictions. From Example 10.10: the storeroom costs \$1,371.26 a year and the hub \$2,279.69, and a request at the hub waits 0.0054 year on average, about two days. These come from the moment route, which carries two moments of each location's lead time demand and no more.

Table 11.16.: The cutout in job lots across two levels: the iteration's predicted costs from Example 10.10, and the simulation, 30 replications of 300 years after a 5-year warm-up. The hub's service measures are for its own customers.

	Storeroom, predicted	simulated	Hub, predicted	simulated
Ordering		\$504.52 ± 0.94		\$717.22 ± 0.87
Holding		\$686.16 ± 1.66		\$1,059.20 ± 4.52
Backorder		\$123.00 ± 2.95		\$423.61 ± 4.79
Total, a year	\$1,371.26	\$1,313.68 ± 2.46	\$2,279.69	\$2,200.02 ± 3.69
Orders a year	6.12	6.1153 ± 0.0114	8.70	8.6936 ± 0.0106
Ready rate		0.9227 ± 0.0011		0.8083 ± 0.0017
Unit fill rate		0.9134 ± 0.0012		0.8018 ± 0.0016
Lot fill rate		0.9108 ± 0.0011		0.8001 ± 0.0017

The hub's own customers wait 0.0043 ± 0.0001 year on average. A storeroom's order waits 0.0109 ± 0.0001 year for its last unit to ship, and 56.9% of the storeroom's orders ship complete the moment they arrive.

The check. Each location's order rate must be its demand in units divided by its batch: $300/49 = 6.12$ orders a year at the storeroom and $(500 + 300)/92 = 8.70$ at the hub, since every storeroom order is for exactly 49 units and so the hub sees 300 units a year from it.

The order rates cover 6.12 and 8.70, as the check requires.

Interpretation. This is the paragraph that closes the validation Example 10.10 left open. The iteration's policies cost less than the iteration said: $57.58 a year less at the storeroom and $79.67 less at the hub, an overstatement of about 4% at each. The moment route is conservative here, which is the safe direction for a planner, and the policies it chose are reasonable ones to run.

The wait shows where the moment route simplified. It carried one wait for every request at the hub, 0.0054 year. The hub's own customers, whose lots are small, wait less than that, 0.0043. A storeroom's order of 49 waits for its last unit, and nearly half of them cannot ship complete on arrival, so it waits twice as long, 0.0109. The storeroom's 25-unit reorder point covers the difference, and its crews still find a cutout on the shelf 92% of the time.

Validation, and its limit. Nothing exact covers this case. The model was validated on the same structure in Example 11.11, and under job lots in Example 11.4, so what is new here is only the combination.

11.8.1. Where This Leaves Us

The chapter's own classes have now reached every system the book analyzed, and one it could not. Section 11.9 builds the same two-level system with the KSL's supply chain package, which is what you would use for a network of realistic size.

11.9. The KSL Supply Chain Package

The chapter's own classes show the mechanism, and they are small enough to read in full. A network of realistic size needs more: several items at each location, transport between locations, cost accounting by location and by cost line, and a report a manager can read. The KSL's supply chain package, `ksl.modeling.supplychain`, supplies those. This section builds

one small network with it, checks the result against a system the book has already solved, and says what the package does not yet do.

> The package is released as experimental, and its interface may change between versions of the KSL. Everything in this section was written against KSL R1.7.1, the version the companion code declares. Check the release notes before relying on a name or a default with a later version.

11.9.1. What the Package Provides

A network in the package is a tree of **inventory holding points**, each holding an inventory of one or more items, each item's inventory running its own policy. The root of the tree is an external supplier. Customer demand is attached to the holding points that face customers, and each edge of the tree carries a transport time. Three policies are provided: (r, Q), (s, S), and periodic (s, S) with a constant review interval. A base-stock policy at level S is the (r, Q) policy with $r = S - 1$ and $Q = 1$, as Section 10.2 wrote it.

A network can be described in three ways, which produce the same model: a Kotlin specification language, a specification file in TOML or JSON, or calls that build it object by object. The specification language is the readable one, and it is the one Example 11.13 uses. At the end of a run, the network produces a structured report with the layout of the network, the costs by location and by cost line, and each holding point's on-hand level, fill rate and waiting time.

11.9.2. The Package's Names for the Book's Quantities

The package names its quantities for supply chains in general, and this book named them for the analysis of a stock point. Table 11.17 translates.

Table 11.17.: The package's names for the quantities of this book.

This book	The package
A stock point	An inventory holding point, holding one `Inventory` per item
(r, Q)	the `sQ(s, Q)` policy
(s, S)	the `sS(s, S)` policy
(R, s, S)	the `sSPeriodic(s, S, reviewInterval)` policy
Base stock at level S	`sQ(s = S - 1, Q = 1)`
The outside supplier and its lead time	the external supplier
The transit time O_j	the transport time from the parent node
$\bar{I}$, $\overline{IO}$, $\bar{B}$	`On Hand`, `On Order`, `Amt BackLogged`
$1 - \overline{RR}$	`Fraction of Time w/o Stock`
The lot fill rate	`First Fill Rate`
The unit fill rate	`Unit Fill Rate`

Notice the two fill rates. The package's `First Fill Rate` scores each demand filled complete or not, which is the lot fill rate of Section 8.11.4, and its `Unit Fill Rate` is the unit fill rate. With demand one unit at a time they agree, as Section 11.4.4 said they must.

Example 11.13 (The cutout's hub and storerooms, built with the package)**.** Rebuild the running example of Example 11.11 with the package: four storerooms, each seeing one fuse cutout demanded a week and running base stock $S_j = 3$, supplied over a one-week transit by a hub that runs $(r_0, Q_0) = (9, 8)$ against a supplier taking three weeks.

The modeling questions have the answers given in Example 11.11, since the system has not changed. Only the software has.

Notation. In the package's terms, the hub is a holding point attached to the external supplier with an `sQ(s = 9, Q = 8)` inventory, and each storeroom is a holding point beneath it with an `sQ(s = 2, Q = 1)` inventory, a transport time of one week from its parent, and demand arriving with exponential times between demands of mean one week. The run settings are those of Example 11.11, and each source of randomness has its own stream.

The values to reproduce. Every row of Table 11.15, and the simulated figures of Example 11.11 from the chapter's own classes.

The specification, in the package's Kotlin language:

```
supplyChain("CutoutNetwork") {
    transportStrategy = perIHPTimeBased
    val cutout = item("Cutout", leadTime = constant(3.0),
     ↪  unitCost = 115.0)
    holdingPoint("Hub") {
        attachedToExternalSupplier()
        inventory(cutout) { sQ(s = 9, Q = 8, initialOnHand =
 ↪  17) }
        tier(count = 4, namePrefix = "Store", transportTime
 ↪  = constant(1.0)) {
            inventory(cutout) { sQ(s = 2, Q = 1,
 ↪  initialOnHand = 3) }
            demand(cutout, exponential(mean = 1.0, stream =
 ↪  autoStream()))
        }
    }
}
```

`item` declares the cutout and the external supplier's lead time of three weeks. `holdingPoint` and `tier` lay out the tree: the hub beneath the external supplier, and four storerooms named `Store1` to `Store4` beneath

the hub, each a week's transport away. `sQ` is the (r, Q) policy, and `autoStream` gives each storeroom's demand its own random number stream. The run is `SupplyChainBuilder.build(model, spec)` followed by `model.simulate()`.

The structured report's summary of the holding points, pasted as it prints:

```
|IHP| Level| On hand| On order| Backordered| Fill rate| Avg wait|
|:---| ---:| ---:| ---:| ---:| :---| ---:|
|Hub| 1| 2.53| 11.98| 1.02| 60.5%| 0.64|
|Store1| 2| 1.82| 1.25| 0.07| 85.9%| 0.51|
|Store2| 2| 1.82| 1.25| 0.07| 86.0%| 0.51|
|Store3| 2| 1.82| 1.25| 0.07| 86.2%| 0.51|
|Store4| 2| 1.82| 1.25| 0.07| 85.9%| 0.51|
```

Notice that the report's "Avg wait" is the package's `Backorder Wait Time`, the wait of the demands that had to wait, and not the mean over all demands that Table 11.15 reports.

Table 11.18.: The running example three ways. The hub's fill rate is its `First Fill Rate` in the package and $P(W = 0)$ in the chapter's classes.

Measure	Exact	The chapter's classes	The package
Hub on hand, units	2.5273	2.5334 ± 0.0124	2.5334 ± 0.0124
Hub backorders, units	1.0273	1.0173 ± 0.0087	1.0173 ± 0.0087
Hub fill rate	0.6030	0.6051 ± 0.0020	0.6051 ± 0.0020

Measure	Exact	The chapter's classes	The package
Storeroom 1 backorders, units	0.0726	0.0721 ± 0.0016	0.0721 ± 0.0016

The check. At each storeroom, demand is single-unit Poisson, so `First Fill Rate`, `Unit Fill Rate` and one less `Fraction of Time w/o Stock` must agree within their half-widths. At the hub, the requests are single units too, so the same holds there.

They do: at storeroom 1, `First Fill Rate` and `Unit Fill Rate` are both 0.8592 ± 0.0019, and one less `Fraction of Time w/o Stock` is $1 - 0.1403 = 0.8597$.

Interpretation. The package and the chapter's own classes agree not to within their half-widths but exactly, to every digit printed. They draw the four demand streams from the same stream numbers, they implement the same assumptions, and so they follow the same sample path, event for event. That is the strongest agreement two independent programs can show, and it is why the package's figures inherit the reading Example 11.11 gave its own: the run is slightly lucky at the hub, and longer runs cover every exact value.

Validation, and its limit. Agreement with the exact figures validates the package on this system, and agreement with the chapter's own model shows that two independent implementations of the same assumptions give the same answer. It says nothing about the parts of the package this system does not use, such as shipment formation and cost lines beyond holding and backorders.

11.9.3. What the Package Does Not Do Yet

Four limits matter for the systems of this book, and they are stated here so that you meet them in a paragraph rather than in a failed run.

Lost sales are not available through a specification. Demand in a network built from a specification is backordered. A lost-sales system can be built object by object, but the package's fill rates then count only the demands that were not lost, so they overstate service. Example 11.5 uses the chapter's own classes for that reason.

A node has one supplier. The network is a tree, so a storeroom that sends some requests to its own repair shop and the rest to a depot, as in Example 11.10, is outside what a specification can describe.

A periodic review interval is a constant. That covers every periodic example in this book.

Random number streams are the modeler's responsibility. Every random quantity in a specification names its stream, and identical streams with an identical structure give an identical sample path. That is what makes common random numbers possible, and it is also why two runs that were meant to be independent must be given different streams.

11.9.4. Where This Leaves Us

The package reproduces the system the chapter's own classes reproduced, by a different route, and with far less code for the modeler to write. The next section sets out how the chapter's own classes are organized, which is also a description of what the package does at a much larger scale.

11.10. Designing the Software

The package is `inventory.simulation`. Its core is the event model of the (r, Q) policy in Rossetti (2026), copied into the companion code and adapted: the names follow this book's notation, the reorder decision orders as many batches as it takes to clear the reorder point, and the model gains the options the chapter's examples need. Every other package in the companion code computes; this one is the first that runs a model through time, and it is built on the KSL's `ModelElement` rather than on its distributions alone.

11.10.1. What the Software Must Do

Reading back over the examples, the software has to

1. run one stock point under (r, Q), (s, S) or (R, s, S), which includes (R, S) as $s = S - 1$, with demand either backordered or lost,
2. accept demand in lots of a stated distribution, filling a lot partly when the shelf holds part of it and owing the rest first come first served,
3. draw each lead time from any distribution, deliver orders either independently or in one supply line, and record when an order overtakes another,
4. be driven by a script of demand times as well as by a sampled demand process, so that Example 11.1 is a test and not only an illustration,
5. collect the time-persistent measures of Equation 11.2 and the observation-based measures of Section 11.4.4, together with the undershoot, the order sizes, the waits and the cost terms of Equation 8.1, or a cost per unit lost under lost sales,
6. let one stock point be another's supplier over a transit time, filling the lower location's orders first come first served and shipping part of an order when it holds only part of it, and let a lower location send

some of its requests elsewhere, as a storeroom with its own repair shop does, and

7. report each measure as an interval and compare it, field by field, with the exact model of the same policy.

11.10.2. The Nouns

Three nouns become classes: the stock point, the policy it runs, and the supplier that fills its orders. Several others do not, and the reasons are the design.

The event is not a class. A demand and a receipt are methods of the stock point, scheduled on the KSL's calendar. Table 11.1 lists two events and one decision, and the code has two event methods and one decision method in the same places.

Lost sales is not a subclass. Whether an unmet demand waits or departs is a property of the demand, not a different kind of stock point, so it is a flag on the stock point. Every policy runs under either setting without being written twice.

Periodic review is a policy, not a stock point. It is the one policy that owns an event of its own, the review, which it schedules on the calendar when the replication starts. A demand under periodic review takes no decision, so the stock point's decision method simply does nothing for it.

A two-level system is not a class. The KSL book's model gives a stock point one way to obtain stock: it asks an `InventoryFillerIfc` to fill an order. A supplier with a lead time is one kind of filler, and so is another stock point. Thus a storeroom supplied by a hub is a stock point whose filler is the hub, seen through a transit link that adds O_j, and a storeroom that rebuilds some of its failures is a stock point whose filler sends each request one way or the other. The two-level examples of Section 11.8 are assembled from the same three classes as the single

location of Example 11.2. Notice that this is the coupling chapters 9 and 10 worked so hard to approximate: here it is simply the order of the calls.

11.10.3. The Classes

Class	Knows	Does
`InventoryFillerIfc`	nothing	accepts a request for a quantity; the one thing a supplier must do
`SimInventory`	I, IO and B as time-weighted responses, the backorder queue, the lost-sales flag, the cost rates	the demand and receipt events of Table 11.1; Equation 11.1, through `TWResponse`; the unit fill rate, lot fill rate and ready rate; the cost terms at the end of each replication; fills a lower location's orders as an `InventoryFillerIfc`
`RQSimInventory`	r and Q	the decision of Table 11.1, ordering $\lfloor (r - IP)/Q \rfloor + 1$ batches
`SSSimInventory`	s and S	orders $S - IP$, and records the undershoot of Equation 8.55

Class	Knows	Does
`PeriodicSimInventory`	R, s and S	schedules the review; counts reviews and orders separately, as Equation 8.64 requires; records the undershoot
`LeadTimeSupplier`	a lead time distribution, and whether orders queue	fills after L, or at the later of $t + L$ and the previous receipt; records crossings and the time each order is outstanding
`TransitLink`	a stock point and a transit time	makes a stock point a supplier, adding O_j after it ships, and records each request's wait
`RoutingFiller`	several fillers and the probability of each	sends each request to one of them, as the storeroom's own shop and the depot of Example 11.10 share its failures
`SimStockPoint`	an inventory, a filler, and a demand process given by a rate and a lot size distribution, or by a script	assembles one location as a KSL `ModelElement`, the role of the KSL book's `RQInventorySystem`
`Estimate`	an average, a half-width and a count	whether the interval covers a value

Class	Knows	Does
`SimulatedPerformance`	an `Estimate` for each measure	compares itself with chapter 8's `RQPerformance` or `RSPerformance`, measure by measure

Notice what is missing: a class for the two-level system, for the hub, or for the depot. A hub is `SimInventory` running (r, Q) with a `LeadTimeSupplier` behind it and `TransitLink`s in front of it. The depot of Example 11.10 is the same thing running $(S_0 - 1, 1)$, since a one-for-one stock point replenished after a constant repair time behaves exactly as one replenished by a supplier with that lead time.

11.10.4. The Two Routes Must Agree

Recall from Section 7.12.4 that two routes sharing one cost structure can be checked against each other. Here the exact route is the code of chapters 8 to 10 and the simulated route is this package, and the check is the chapter's thesis written as tests.

There is one test for every example that has an exact column, and each test asserts the agreement the example states. The ledger test of Example 11.1 asserts equality, row by row, since nothing in it is random. Every other test asserts that each interval covers the exact value: Table 11.5, Table 11.10, the costs and service measures of Table 11.7, the order rates and undershoots of the periodic examples, the four exact rows of Table 11.14, and Table 11.15. The tests run at a fixed seed, so a test that passes once passes every time.

One detail of the tests deserves stating, because a test written naively would fail a correct model. A test that checks eleven measures from the same runs, each at 95%, will find a miss among them more often than not,

and the measures of one run tend to miss together. Each test therefore builds its intervals at the Bonferroni level for the number of measures it checks, $1 - 0.05/k$ for k measures, so that a correct model passes the whole family at 95%. The chapter prints ordinary 95% intervals, and Example 11.2 said what one miss among them does and does not mean.

The tests do not pin the simulated figures the chapter prints. Those figures are reported with their half-widths, at the seed and the KSL version stated with each run, and a later version of the library may move a printed digit within its half-width. What the tests guarantee is the thing that matters: that every interval still covers the exact value it was printed beside.

11.10.5. Running the Chapter's Examples

The transformer of Example 11.2 is three objects and a model:

```
val m = Model("TransformerRQ")
val supplier = LeadTimeSupplier(m, ConstantRV(1.0 / 6.0),
 ↪  name = "Supplier")
val store = RQSimInventory(m, 8, 7, filler = supplier, name =
 ↪  "Transformer").apply {
    orderCost = 220.0; holdingCost = 950.0; backorderCost =
 ↪  8550.0
}
CustomerDemand(m, store, ExponentialRV(1.0 / 45.0, streamNum
 ↪  = 1), name = "Crews")
m.numberOfReplications = 30
m.lengthOfReplication = 1020.0
m.lengthOfReplicationWarmUp = 20.0
m.simulate()
```

`LeadTimeSupplier` is the supplier, here with a constant lead time of $L = 1/6$ year. `RQSimInventory` is the stock point running $(r, Q) = (8, 7)$, with

the three cost rates of Equation 8.1. `CustomerDemand` sends it demands at exponential intervals of mean 1/45 year, on stream 1. The last four lines are the run settings of Section 11.4.3.

The comparison is one call, against chapter 8's `RQModel` evaluated at the same policy:

```
SimulatedPerformance.of(store).against(exact).forEach(::println)
```

```
on hand              exact       4.6617   simulated       4.6570 +/- 0.0107     cove
backorders           exact       0.1617   simulated       0.1628 +/- 0.0020     cove
ready rate           exact       0.8781   simulated       0.8779 +/- 0.0009     cove
fill rate            exact       0.8781   simulated       0.8776 +/- 0.0010     cove
order frequency      exact       6.4286   simulated       6.4333 +/- 0.0105     cove
ordering cost        exact    1414.2857   simulated    1415.3187 +/- 2.3038     cove
holding cost         exact    4428.5863   simulated    4424.1250 +/- 10.2001    cove
backorder cost       exact    1382.2771   simulated    1392.0530 +/- 16.9403    cove
total cost           exact    7225.1492   simulated    7231.4967 +/- 15.4165    cove
```

A two-level system is assembled from the same classes. The job-lot example of Example 11.12, in years:

```
val hub = RQSimInventory(m, 120, 92,
    filler = LeadTimeSupplier(m, ConstantRV(2.0 / 12.0), name = "Supplier"), name = "Hub")
val store = RQSimInventory(m, 25, 49,
    filler = TransitLink(m, hub, ConstantRV(1.0 / 12.0), name = "Transfer"), name = "Storeroom")
CustomerDemand(m, hub, ExponentialRV(1.0 / 250.0, streamNum = 1), lots(2), name = "HubCrews")
CustomerDemand(m, store, ExponentialRV(1.0 / 150.0, streamNum = 3), lots(4), name = "StoreCrews")
```

The storeroom's filler is a `TransitLink` to the hub, which turns each of the storeroom's orders into a demand at the hub and delivers what the hub ships a month later. `lots` is the lot-size distribution of Table 8.20, on its own stream. Nothing else distinguishes a hub from a storeroom.

```
### the cutout in job lots: storeroom (25, 49), hub (120, 92), in years
 Storeroom:
  ordering                    504.5150 +/- 0.9402
  holding                     686.1641 +/- 1.6630
  backorder                   123.0035 +/- 2.9464
  total                       1313.6826 +/- 2.4588
  orders a year               6.1153 +/- 0.0114
```

Every example of the chapter is a function of the program `Section11Listings.kt`, which runs them all in about fifteen seconds:

```
./gradlew run
 ↪  -PmainClass=inventory.simulation.Section11ListingsKt
```

The agreement tests of Section 11.10.4 are in the package's test directory and run with the rest of the companion code's tests.

11.10.6. Where This Leaves Us

Every example in the chapter is an assembly of the same few classes, and every one that has an exact answer is checked against it by a test. That is what allows the examples without an exact answer to be believed.

11.11. Summary

A simulation is trusted only after it reproduces a number the book has already derived. A simulation is a program, and a program can be wrong without saying so. Every model in the chapter therefore first reproduces a result from chapters 8 to 10 inside its half-width, and only then is one assumption behind that result removed. Simulation belongs after the closed form, not instead of it, and the closed form is what makes the simulation believable where no closed form exists.

An inventory policy is two events and one decision. A demand and a receipt change the state; placing an order is a decision taken inside a demand, which schedules a receipt one lead time later. Periodic review adds a third event, the review, which the policy schedules for itself and which takes the decision in the demand's place. Everything the chapter simulates is assembled from those events.

A time average is an area, accumulated at the events. Between events the state is constant, so the area under each state variable grows by one rectangle per event, credited with the value that held before the event changed it. The ready rate is an average of that kind; the fill rates are averages over demands. Averaging the rows of a ledger gives neither, and Example 11.1 showed by how much on chapter 8's own thirteen weeks.

A stock point never terminates, and its run is measured in demands. The target is the long-run average, so the start is discarded after a warm-up, which for an (r, Q) policy can be reasoned out from Equation 8.17 and is then confirmed by a Welch plot. Replications are long and independent, and their number follows from a pilot run and the precision wanted. The backorder level needs the longest run, because backorders come in rare episodes, and the run should be set by the hardest measure the report will quote.

Under job lots there are three service measures, and a report must name the one it quotes. The ready rate counts time, the unit

fill rate counts units and the lot fill rate counts lots. They coincide for single-unit Poisson demand and separate under lots, and a simulation collects all three at no extra cost.

Each assumption is removed by changing one component of a validated model. Orders that overtake one another change the supplier; job lots change the demand process and the policy that suits it; lost sales change what happens to an unmet demand. Because nothing else changes, a difference in the results is the effect of the assumption removed and not of a new model.

Policies are compared against the same demands. Common random numbers make the difference between two policies more precise without making either cost more precise, and in an inventory model they need a separate stream for each source of randomness, since policies place different numbers of orders. Near an optimum the cost surface is flat, and a multiple comparison with an indifference zone says which policies are worth telling apart. A difference can be cheap to detect and still not worth acting on.

A two-level system is two stock points, one supplying the other. The coupling chapters 9 and 10 had to approximate, a lower location's resupply delayed by the upper location's backorders, is in the model simply because one location's order waits in the other's queue. That is what lets the simulation reproduce the exact figures where they exist, settle how far METRIC errs on one system, and check the job-lot case chapter 10 handed on.

The KSL's supply chain package builds the same systems at a larger scale. It provides holding points, policies, transport and cost accounting, and a structured report, and on the chapter 10 example it is checked against the same exact figures as the chapter's own classes. It is experimental, a node has one supplier, and lost sales are not available through a specification, so a model should be checked against the limits of Section 11.9.3 before it is built.

Table 11.19.: The chapter's notation, beyond that of Table 8.32, Table 9.6 and Table 10.6.

Symbol	Meaning
t_n	the time of the n-th event
$A_I(t)$, $A_B(t)$	the areas under I and B from time 0 to t
$A_+(t)$	the total time up to t with stock on the shelf
$N_D(T)$, $N_F(T)$, $N_L(T)$	the demands, the demands filled on arrival, and the demands lost, by time T
T, T_w	the length of a replication after its warm-up, and the warm-up
n, Y_j	the number of replications, and a measure's time average over replication j
s_Y, hw	the sample standard deviation of the Y_j, and the half-width of the confidence interval
n_0, hw_0, hw^*	a pilot's replications and half-width, and the half-width wanted
L_m, t_m	the lead time drawn for the m-th order, and the time it was placed
C_c, C_p, Δ	the simulated cost rates of a continuous and a periodic policy, and their difference
C_r, δ	the simulated cost rate at reorder point r, and the indifference zone
W	the time a request waits at the upper location before it ships

11.12. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.8 and the chapter's standard run settings of Section 11.4.3: $n = 30$ replications, each of $T = 1{,}000$ years after a warm-up of $T_w = 20$ years, with the KSL's default random number streams, so that a student and the instructor who use the companion code reach the same intervals. Demand is backordered and filled first come first served unless the exercise says it is lost. Report every simulated figure as an average with its 95% half-width, probabilities and rates to four decimals, and costs to the cent.

11.12.1. Working the Model by Hand

Exercise 11.1. Answer each in a sentence or two, naming the section or equation that settles it.

a. True or false: averaging the on-hand column of an event ledger such as Table 11.4 gives the time average of the on-hand level.
b. True or false: under single-unit Poisson demand, a simulation's ready rate and fill rate must agree in every replication.
c. True or false: lengthening the warm-up reduces the bias of an estimate, and it costs nothing but run time.
d. True or false: common random numbers reduce the variance of each policy's estimated cost.
e. A simulation of an (r, Q) system reports a cost with a relative half-width of 0.8% and a backorder level with a relative half-width of 12%, from the same runs. Which is the most likely reason? (i) The warm-up is too short. (ii) Backorders come in rare episodes, so few independent episodes inform their average. (iii) The model is wrong. (iv) The replications are not independent. Say why each of the others is wrong.
f. A supplier's lead times are random, and its orders can overtake one another. In which direction does Section 8.5.4's treatment of a

random lead time err, and which example of this chapter measures it?

Exercise 11.2. Continue Example 11.1 past week 13, in the same storeroom and under the same policy. Further demands arrive in weeks 14, 15 and 18, and the order placed in week 8 is received in week 16. The observation now closes at week 18.

a. Post one row for each event, carrying I, B, IP, the areas A_I, A_B and A_+, and the receipts due, and say at each row whether an order is placed. Remember that each area is credited before the event changes the state.
b. Compute $\bar{I}$, $\bar{B}$, $\overline{RR}$ and $\overline{FR}$ over the eighteen weeks.
c. Verify the two identities of Example 11.1's check: the time with stock plus the time without is eighteen weeks, and A_B equals the total time demands spent waiting.
d. Average the rows of the on-hand and backorder columns instead, and compare.

Exercise 11.3. Five replications of the cutout under job lots, each of 112 years after its warm-up, recorded the counts below.

Table 11.20.: Counts from five replications.

Replication	Lots	Lots filled complete	Units demanded	Units filled on arrival
1	44,802	40,165	89,644	80,527
2	44,795	40,212	89,519	80,410
3	44,910	40,071	89,876	80,333
4	44,688	40,104	89,402	80,409
5	44,851	40,138	89,723	80,546

a. Compute the lot fill rate and the unit fill rate of each replication.

b. Combine the five replications in the two ways of Section 11.4.4, as the average of the five ratios and as the ratio of the totals, for each measure. How far apart are the two?
c. Give a 95% confidence interval for each measure from the five per-replication ratios, using $t_{4,\,0.025} = 2.776$, and say why the interval is built from those and not from the totals.
d. Which of the two fill rates is larger here? A large lot is both more likely to go unfilled and worth more units; say how each of those pulls the two measures apart.

11.12.2. Using the Software

Exercise 11.4. Example 8.6 used the approximate model of Section 8.6 on the transformer, with $\lambda = 45$ a year, $k = \$220$, $h = \$950$ per unit per year, a two-month lead time, and a shortage cost of $\pi = \$5{,}700$ per unit short. The approximation recommended $(r, Q) = (14, 6)$ and reported a cost of \$12,174.25 a year, against \$11,922.90 from the exact formulas.

a) Simulate the policy $(14, 6)$ and estimate its cost rate, k times the order rate plus $h\bar{I}$ plus π times the number of units backordered per year, with a 95% confidence interval. Which of the two reported costs does the interval cover?
b) Estimate the fill rate, and compare it with the fill rate the approximate model implies and with the one Equation 8.19 gives.
c) Account for the discrepancy in (b) with Table 8.12. Which of the approximation's assumptions does the simulation make, and which does it not?

Exercise 11.5. Table 8.5 gives the exact cost of four policies for the transformer with $Q = 5$ and $r = 8$, 9, 10 and 11, under the costs of Example 8.4: $k = \$220$, $h = \$950$ and $b = \$8{,}550$ per unit per year.

a) Simulate the four policies with independent random numbers, and report each policy's cost with a 95% confidence interval and the six pairwise differences, each with its interval.
b) Repeat (a) with common random numbers, so that every policy faces the same demands, and report the same intervals.
c) For each pairwise difference, report the ratio of its variance under (a) to its variance under (b). Which differences gain the most from common random numbers, and why those?
d) Which policy do you recommend, and is the recommendation the one Table 8.5 makes? Which pairs could the simulation not tell apart?

Exercise 11.6. Example 11.3 gave the transformer an Erlang lead time, two months on average with a standard deviation of one month, and ran the policy $(10, 8)$ under two supply arrangements. Keep everything else and give the lead time a lognormal distribution with the same mean and standard deviation.

a. Simulate both arrangements, and report each measure of Table 11.6 and the fraction of orders that overtake an earlier one.
b. Set the results beside Example 11.3's. Chapter 8's treatment uses only the lead time's mean and variance, through case 3 of Table 8.3, so it predicts the same figures for both shapes. Does the shape matter, and under which arrangement does it matter more?
c. A lognormal has a longer right tail than an Erlang of the same mean and variance. Say how that tail shows up in the fraction of orders that overtake another.

Exercise 11.7. Take the cutout of Example 11.4, in job lots, with the same costs and lead time, and use common random numbers throughout.

a. Simulate $(s, S) = (36, 56)$ and $(r, Q) = (36, 20)$, the policy Section 8.12.2 approximates it with. Report each cost, the paired difference with its interval, and each policy's mean order size. Table 8.23

gives the exact costs, \$3,755.69 and \$3,876.25; does the interval for the difference cover \$120.56?

b. Explain the difference using the undershoot. Which policy pays for it, and in which cost term?
c. Repeat (a) for $(s, S) = (25, 100)$ against $(r, Q) = (25, 75)$, whose exact costs differ by \$0.12. Can the simulation tell them apart, and should it need to?
d. At both pairs, report the three service measures of Section 11.4.4 and say which of them the undershoot affects.

Exercise 11.8. Run a 2^2 factorial experiment on the transformer of Example 11.2, with the reorder point at 7 and 9 and the order quantity at 5 and 7, using common random numbers across the four design points.

a. Report the simulated cost at each design point, and compute each exact cost with `RQModel`.
b. Fit the regression of cost on the two factors and their interaction, and report the coefficients with their intervals.
c. Compute the exact cross-difference $C(9,7) - C(7,7) - C(9,5) + C(7,5)$, and compare its sign and size with the fitted interaction, which estimates a quarter of it. What does the interaction say about choosing r and Q one at a time?

Exercise 11.9. Take four items of Table 4.5 with demand arriving one unit at a time: the transformer ($\lambda = 45$, $c =$ \$3,800, $k =$ \$220, a two-month lead time), the fuse cutout ($\lambda = 800$, $c =$ \$115, $k =$ \$82.50, two weeks), the ground rod ($\lambda =$ 2,400, $c =$ \$14, $k =$ \$41.25, two weeks) and the compression connector ($\lambda =$ 6,000, $c =$ \$3.40, $k =$ \$27.50, two weeks). Holding costs $h_j = 0.25 c_j$ a year, and the storeroom charges one backorder cost, $b =$ \$287.50 per unit per year, to every item, as Section 8.14.3 supposes.

a. Optimize each item's (r, Q) with `RQOptimizer`, and report each item's exact fill rate and the system fill rate, the demand-weighted average

$\sum_j \lambda_j \overline{FR}_j / \sum_j \lambda_j$.

b. The utility commits to a system fill rate of at least 0.99. Starting from (a), raise one reorder point by one unit at a time, each time choosing the item that adds the most exact system fill rate per dollar of added exact cost, until the target is met. Report the final policies and their cost.
c. Simulate the four items under the final policies and report the system fill rate with its interval. Does the interval lie above 0.99? If it straddles 0.99, say what that means for the commitment.
d. Compare each item's fill rate with the implied service of Table 8.30, and explain the pattern.

Exercise 11.10. Take the running example of Example 11.11, the hub at $(9, 8)$ and four storerooms at base stock 3, but suppose a crew that finds its storeroom empty draws a cutout from a neighboring utility instead of waiting, so the storerooms' demand is lost. The hub still backorders the storerooms' requests.

a. Simulate the system and report each storeroom's lost sales rate, ready rate and on-hand level, and the hub's backorders, ready rate and the mean wait of a request.
b. Compare each with Table 11.15, the same system with backorders. Which measures move, in which direction, and why does the hub improve when the storerooms lose sales?
c. Nothing exact covers this system. Which of the chapter's validated examples does its model share every component with, and what does that agreement license here?

Exercise 11.11. This exercise needs sampling but no event model. Take the transformer of case 3 of Example 8.2: Poisson demand at 45 a year over a lead time that is Erlang with shape 4 and rate 24 a year.

a. Sample 100,000 lead time demands, each by drawing a lead time and then a Poisson count over it. Compare the observed frequencies

of 0 through 20 units with the negative binomial with $r = 4$ and $p = 0.3478$, and report the largest difference.

b. Estimate $P\{D(L) \geq 12\}$ with a 95% interval, and compare it with the exact negative binomial value.

c. Repeat (a) with a lognormal lead time of the same mean and standard deviation. Does the negative binomial still describe the result, and what does that say about Section 8.3.2?

Exercise 11.12.

a. Reproduce Example 11.7 and report the premium of periodic review over continuous review as a paired interval.

b. Repeat at a review interval of two weeks, with $S = 12$, the level Table 8.28 gives there. Compute the exact premium from that table and Table 8.29, and say whether your interval covers it.

c. Report the fraction of two-week reviews that place no order, and compare it with Equation 8.64.

Exercise 11.13. Take the transformer reviewed every two weeks, so that $\lambda R = 1.73$ units, with $S = 14$.

a. Simulate (R, s, S) at $s = 13$, 10 and 6, and report the mean undershoot and the fraction of reviews that place an order at each.

b. Compare each mean undershoot with $\lambda R/2$ from Equation 8.75 and with the (R, S) value derived in Section 8.13.5.

c. Compare the results with Example 11.8, where a review interval held 3.75 units. How does the length of the review interval change how wide the spread must be before $\lambda R/2$ applies?

Exercise 11.14. Take the utility of Example 11.10, with $S_j = 4$ at each storeroom.

a. For $S_0 = 16$, 18, 20, 22, 24 and 26, simulate the system and report each storeroom's backorders. Set them beside METRIC's column of Table 9.3 and VARI-METRIC's, computed with the chapter 9 code.

b. For each S_0, find the smallest S_j that holds a storeroom's backorders at or below 0.10 by each of the three routes. At which depot stock levels does METRIC choose a different S_j from the simulation?
c. Relate the finding to the 11% of Graves (1985) that Section 9.7.2 quotes.

Exercise 11.15. Take the transformer of Example 11.5, with demand lost and a cost of $\pi =$ \$5,700 per unit lost.

a. With $Q = 6$, simulate every reorder point from 10 to 15 using common random numbers, and find the best by the method of Section 11.7.
b. Compare it with $(13, 6)$, the best policy when shortages are charged as backorders. Is the difference in cost worth anything?
c. Using Example 11.5, say when charging a lost sale as a backorder misleads a planner and when it does not.

Exercise 11.16. Take the transformer at $Q = 5$ with $r = 8$ and $r = 9$, whose exact costs differ by \$9.17 a year.

a. Run a pilot of $n_0 = 10$ replications of each, with common random numbers, and report the variance of the paired difference in cost.
b. Using Equation 11.5, how many replications would give a 95% interval for the difference that excludes zero, if the true difference is \$9.17?
c. Repeat (a) and (b) with independent random numbers.
d. Is it worth running? Say what a planner gains from knowing which of the two policies is cheaper.

Exercise 11.17. Rebuild the running example of Example 11.11 with the KSL's supply chain package, as Section 11.9 does, but with a transit time of two weeks from the hub to each storeroom.

a. Report the hub's backorders and each storeroom's backorders from the package's results report.

b. Compute the exact figures for a two-week transit with the chapter 10 code, `AxsaterBatchOrdering`, and say whether the package's intervals cover them.
c. Repeat (a) with the chapter's own classes, using the same transit time, and compare the two simulations with each other.

11.12.3. Reading the Literature

Exercise 11.18. Read Graves (1985).

a. State how the comparison was designed: what was compared with what, and how the reference answer was obtained.
b. State the conditions under which METRIC's stock levels were most often wrong.
c. Take one of the test systems the paper describes. Compute its backorders by METRIC and by VARI-METRIC with the chapter 9 code, and estimate them by simulation with this chapter's classes. Which model is closer, and is the result consistent with the paper's finding?

A. Notation

A single reference for the symbols used throughout the book. Each entry gives the symbol, what it denotes, its unit of measure, and where it is defined.

A.1. Conventions

Time. The models are written in a generic time unit. Whether that unit is a day, a week, or a year is a matter of convention; only two rules matter. Every rate in a model must use the same unit, and every price quoted per period must be converted to that unit before it enters the model. Carrying charges and positioning costs are quoted per year by universal accounting convention, so a model running in days divides them by 365 on entry.

Time averages. An overbar denotes a time average of a time-persistent variable:

$$\overline{X} \;=\; \frac{1}{T}\int_0^T X(t)\,\mathrm{d}t$$

Under the conditions discussed in Section 1.3, these averages converge as $T \to \infty$ to the corresponding steady-state quantities. An overbar on a quantity indexed by demand transaction rather than by time denotes an average over transactions, and the definition is given where that quantity is introduced.

Functions of time. A symbol written $X(t)$ is a quantity that varies over time and can, in principle, be observed at any instant. A symbol without the argument denotes either a parameter or the corresponding average.

Positive and negative parts. For a real quantity x, $[x]^+ = \max(x, 0)$ and $[x]^- = \max(-x, 0)$.

A.2. State Variables

Table A.1.: State variables of an inventory system.

Symbol	Meaning	Unit	Defined in
$I(t)$	Inventory on hand	units	Section 1.3
$IO(t)$	Inventory on order	units	Section 1.3
$B(t)$	Units backordered	units	Section 1.3
$I_a(t)$	Inventory actually on hand	units	Section 2.3.2
$I_r(t)$	Inventory on hand according to the record	units	Section 2.3.2
$D(t)$	Record discrepancy, $I_a(t) - I_r(t)$	units	Section 2.3.2
$IN(t)$	Net inventory, $I(t) - B(t)$	units	Section 1.3
$IP(t)$	Inventory position, $I(t) + IO(t) - B(t)$	units	Section 1.3

Symbol	Meaning	Unit	Defined in
$SO(t)$	Stockout indicator, 1 when $I(t) \leq 0$	dimensionless	Section 1.3
$RR(t)$	Ready indicator, $1 - SO(t)$	dimensionless	Section 1.3

Table A.2.: Time averages of the state variables.

Symbol	Meaning	Unit	Defined in
$\bar{I}$	Average inventory on hand	units	Section 1.3
$\bar{B}$	Average units backordered	units	Section 1.3
$\overline{N}$	Average order rate	orders per unit time	Section 1.6.1
$\overline{IO}$	Average inventory on order	units	Section 1.3
$\overline{IN}$	Average net inventory	units	Section 1.3
$\overline{IP}$	Average inventory position	units	Section 1.3
$\overline{SO}$	Proportion of time out of stock	dimensionless	Section 1.3

A.3. Demand and Time

Table A.3.: Demand and timing quantities.

Symbol	Meaning	Unit	Defined in
λ	Demand rate	units per unit time	Section 1.3
D_i	Size of the ith demand transaction	units	Section 1.3
n	Number of demand transactions observed	transactions	Section 1.3
L	Replenishment lead time	time	Section 1.3
p	Production or replenishment rate	units per unit time	Chapter 3
T_p	Time to replenish, Q/p	time	Chapter 3
$\overline{L}$	Average lead time	time	Section 1.3
$\overline{W}$	Average time a customer waits	time	Section 1.3
$\overline{T}_s$	Average time a unit spends on hand	time	Section 1.3
TO	Inventory turnover ratio, $\lambda/\bar{I}$	per unit time	Section 1.3
λ_ℓ	Rate of lost demand	units per unit time	Section 1.3

Symbol	Meaning	Unit	Defined in
T	Length of the observation interval	time	Section 1.3
t_F	Basic forecast period, the interval a forecasting system works in	time	Section 8.15
$\hat{D}_t$	Forecast of D_t, the demand in one forecast period, revised at the end of period t	units	Equation 8.79
MAD_t	Smoothed mean absolute deviation of the one period ahead forecast error	units	Equation 8.80
η	Scaling exponent carrying a period standard deviation over a lead time, $\frac{1}{2} \leq \eta \leq 1$	dimensionless	Equation 8.81

A.4. Cost Parameters

Table A.4.: Cost parameters. Note that h and b are themselves rates and therefore multiply a level, while k and π multiply rates.

Symbol	Meaning	Unit	Defined in
c	Unit cost	dollars per unit	Section 1.4.3
i	Carrying charge	dollars per dollar of value per unit time	Section 1.4.4
h	Holding cost, $h = ic$	dollars per unit per unit time	Section 1.4.4
k	Fixed order cost	dollars per order	Section 1.4.5
b	Backorder cost	dollars per unit per unit time	Section 1.4.6
π	Stockout cost	dollars per unit	Section 1.4.6
f	Stocking position cost	dollars per item per location per unit time	Section 1.4.7
γ	Carrying charge taken common to every item in an aggregate development	dollars per dollar of value per unit time	Section 4.4
K	Major setup cost, incurred once per order regardless of its contents	dollars per order	Section 4.5

A.5. Performance Measures

Table A.5.: Service and performance measures.

Symbol	Meaning	Unit	Defined in
$\overline{RR}$	Ready rate, the proportion of time stock is on hand	dimensionless	Section 1.6
$\overline{FR}$	Transaction fill rate	dimensionless	Section 1.6
$\overline{FR}_u$	Unit fill rate	dimensionless	Section 1.6
Y_i	Indicator that the ith demand is filled immediately	dimensionless	Section 1.6
CSL	Cycle service level	dimensionless	Section 1.6
TC	Total cost rate, Equation 1.15	dollars per unit time	Section 1.6.1
C	Relevant cost rate, the part of TC that varies with the policy, Equation 1.16	dollars per unit time	Section 1.6.1
α	Type 1 service target, the cycle service level a policy is set to reach	dimensionless	Section 8.7
β	Type 2 service target, the fill rate a policy is set to reach	dimensionless	Section 8.7

Symbol	Meaning	Unit	Defined in
π^*	Implied stockout cost, the π at which the cost-minimizing policy meets a stated target	dollars per unit	Section 8.7.6

A.6. Policy Parameters

These are the decision variables. Each is introduced with the policy that uses it; they are collected here for reference.

Table A.6.: Policy parameters.

Symbol	Meaning	Unit	Defined in
Q	Order quantity	units	Chapter 3
r	Reorder point	units	Chapter 8
s	Reorder point in an (s, S) policy	units	Chapter 8
S	Order-up-to level, and the base-stock level when $Q = 1$	units	Section 8.4
R	Review interval	time	Section 8.13

A.7. Demand Classification and Record Quantities

Table A.7.: Quantities for classifying demand and auditing records.

Symbol	Meaning	Unit	Defined in
ADI	Average demand interval, periods per period with a demand	periods	Section 2.2.6
CV^2	Squared coefficient of variation of the non-zero demands, with the sample standard deviation	dimensionless	Section 2.2.6
p	Probability that a verified record becomes wrong in one period. Not the replenishment rate of Table A.3	per period	Section 2.3.7
n, $A(n)$	Periods between counts, and the average record accuracy over that count cycle	periods, dimensionless	Section 2.3.7

Symbol	Meaning	Unit	Defined in
m, e, $\hat{a}$	Records to count in a sample, the margin of error, and a prior estimate of accuracy	records, dimensionless, dimensionless	Section 2.3.4

A.8. Deterministic Lot Sizing Quantities

Table A.8.: Quantities of the deterministic cycle and of price schedules.

Symbol	Meaning	Unit	Defined in
$\hat{I}$, $\hat{B}$	Peak on-hand level and maximum backorder level in a cycle	units	Section 3.1
$T_1, \ldots, T_4$	Lengths of the four segments of a cycle, summing to $T = Q/\lambda$	time	Section 3.1.2
m	Number of price levels in a discount schedule	levels	Section 3.8.1

Symbol	Meaning	Unit	Defined in
q_j, c_j	Break point beginning level j, and the unit price on that level	units, dollars per unit	Section 3.8.1
R_j	Under an incremental schedule, the cost of filling every interval below level j	dollars	Section 3.8.2
k_j	Effective ordering cost on level j of an incremental schedule, $k + R_j - c_j q_j$	dollars per order	Section 3.8.2
H_j	The fixed component $R_j - c_j q_j$, as a worksheet carries it	dollars	Section 3.10.4

A.9. Single Period Quantities

Table A.9.: Quantities of the newsvendor model.

Symbol	Meaning	Unit	Defined in
X, $F(x)$	Demand over the single period, and its distribution function $P\{X \leq x\}$	units	Section 7.5
c_o	Overage cost, the loss per unit left over, $c - u$	dollars per unit	Section 7.5
c_u	Underage cost, the loss per unit of unmet demand, $s - c$	dollars per unit	Section 7.5
s, u	Selling price and salvage value per unit. Not the s of an (s, S) policy	dollars per unit	Section 7.5.1
Q^*, S^*	Optimal order quantity, and the order-up-to level when stock is already on hand	units	Section 7.5
$c_u/(c_u + c_o)$	The critical ratio, the probability of no shortage at the optimum	dimensionless	Section 7.5

A.10. Continuous and Periodic Review Quantities

Table A.10.: Quantities of the reorder point policies and their periodic counterparts.

Symbol	Meaning	Unit	Defined in
θ	$E[D(L)]$, the mean lead time demand. Not the multiplier of Table A.11	units	Section 8.3
$\sigma_{D(L)}$	Standard deviation of lead time demand	units	Section 8.3
w	$b/(b+h)$, the critical ratio the base-stock level is read against	dimensionless	Section 8.4
$C(s)$	Cost rate of a base-stock policy at level s, holding plus backorder	dollars per unit time	Section 8.4
ss	Safety stock, $r-\theta$	units	Section 8.7
z	Safety factor, safety stock in standard deviations of lead time demand	dimensionless	Section 8.7

Symbol	Meaning	Unit	Defined in
Q_k, C_k	Order quantity and cost of the deterministic model with backorders, the lower bounds of the search	units, dollars per unit time	Section 8.9
C_∞	$\sqrt{bh}\,\sigma_{D(L)}$, a reference cost built from lead time demand alone	dollars per unit time	Section 8.9
Y, λ_e	Size of a demand lot, and the rate at which lots arrive	units, lots per unit time	Section 8.11
U	Undershoot, the amount by which a demand carries the position below s	units	Section 8.12
τ	$R + L$, the protection interval a periodic policy must cover	time	Section 8.13

$\bar{G}$, $\overline{G^1}$	Distribution and first order loss function of $D(L+U)$, averaged over a review interval	Section 8.13.2

A.11. Multi-Item Quantities

Items in a multi-item problem are indexed by j, and any symbol of the previous tables carries that subscript when it refers to one item. In Chapter 4 the letter n counts items rather than demand transactions.

Table A.11.: Quantities that appear only when items are decided together.

Symbol	Meaning	Unit	Defined in
n	Number of items sharing the resource or the setup; in Section 4.6 it counts stages or regional warehouses instead	items, or locations	Chapter 4
a_j	Amount of the shared resource one unit of item j consumes	resource units per unit	Section 4.2

Symbol	Meaning	Unit	Defined in
A	Amount of the shared resource available	resource units	Section 4.2
θ	Lagrange multiplier on the shared-resource constraint. See Section A.18	varies with the resource	Section 4.2.2
$R(\theta)$	Resource function, the amount consumed at θ less the amount available	resource units	Section 4.2.3
v_j	Volume one unit of item j occupies, the space-constraint case of a_j	cubic units per unit	Section 4.2
$\overline{OF}$	Order frequency for one item, λ/Q	orders per unit time	Section 4.4.1
$\bar{I}^a$	Aggregate investment in cycle stock	dollars	Section 4.4.2
N^a	Aggregate replenishments across all items	orders per unit time	Section 4.4.2
$\overline{OC}^a$	Aggregate ordering cost rate	dollars per unit time	Section 4.4.4

Symbol	Meaning	Unit	Defined in
J^*	Variety index, the effective number of items	items	Section 4.4.4
C^a	Aggregate purchase cost rate, $\sum_j c_j \lambda_j$	dollars per unit time	Section 4.4.4
λ^a	Aggregate demand rate, $\sum_j \lambda_j$	units per unit time	Section 4.4.4
c^w	Demand-weighted average unit cost	dollars per unit	Section 4.4.4
K^w	Purchase-weighted average ordering cost	dollars per order	Section 4.4.4
K	Major setup cost, incurred once whenever an order is placed at all	dollars per order	Section 4.5
k_j	Minor setup cost, incurred for each item included on that order	dollars per order	Section 4.5
T	Base period between order opportunities	time	Section 4.5.1

Symbol	Meaning	Unit	Defined in
m_j	Number of base periods between replenishments of item j	dimensionless	Section 4.5.1
ℓ_j	Exponent in a power-of-two multiplier, $m_j = 2^{\ell_j}$; the subscript is dropped while following one item	dimensionless	Section 4.5.2
$C(T_j)$	Relevant cost rate written as a function of the reorder interval	dollars per unit time	Section 4.5.3
g	Holding cost coefficient in the interval form, $h\lambda/2$	dollars per unit time squared	Section 4.5.3

A.12. Multi-Echelon Quantities

Stages in a serial system are numbered so that stage i supplies stage $i-1$, with stage 1 facing the customer. In a distribution system the central warehouse is indexed 0.

Table A.12.: Quantities for systems in which one stock replenishes another.

Symbol	Meaning	Unit	Defined in
h_i	Installation holding cost rate at stage i, the total value accumulated through that stage	dollars per unit per unit time	Section 4.6.1
h_i'	Echelon holding cost rate, the value added at stage i alone. In a serial chain $h_i' = h_i - h_{i+1}$; in a distribution system $h_i' = h_i - h_0$	dollars per unit per unit time	Section 4.6.1
k_i	Fixed ordering cost at stage i. There is no major setup in these sections, so this is the same k as Section 1.4.5	dollars per order	Section 4.6.2

Symbol	Meaning	Unit	Defined in
g_i	The echelon holding coefficient at stage i: $\frac{1}{2}\lambda h_i'$ in a serial chain, where every stage sees the same rate, and $\frac{1}{2}\lambda_i h_i'$ in a distribution system	dollars per unit time squared	Section 4.6.2
T_i	Reorder interval at stage i	time	Section 4.6.2
$\bar{I}_i$	Average on-hand stock at stage i	units	Section 4.6.1
$\bar{I}_i^e$	Average echelon stock at stage i	units	Section 4.6.1
G_r	The rth block of consecutive stages sharing one reorder interval		Section 4.6.2
M	Number of blocks in the partition	blocks	Section 4.6.2
$k(G_r)$, $g(G_r)$	Block totals, $\sum_{i \in G_r} k_i$ and $\sum_{i \in G_r} g_i$		Section 4.6.2
$T(r)$	The interval shared by every stage in block G_r	time	Section 4.6.2

Symbol	Meaning	Unit	Defined in
$\mathcal{C}^0$	The pinned set: the central warehouse together with the regions sharing its interval. Set in a calligraphic capital, to keep it apart from the cost rates written C		Section 4.6.3

A.13. Repairable Item Quantities

Chapter 9 holds a fixed population of repairable units circulating between a depot and the storerooms it supports. The depot is indexed 0 and the storerooms $j = 1, \dots, J$, which continues the convention of Section A.12. This book says **storeroom** where the repairable literature says *base*, because *base-stock* is already the name of a policy; see Section 9.2.2.

Table A.13.: Quantities for a fixed population of repairable units circulating between two levels.

Symbol	Meaning	Unit	Defined in
J	Number of storerooms one depot supports	locations	Section 9.2.3

Symbol	Meaning	Unit	Defined in
λ_j	Failure rate at storeroom j, assumed Poisson. A failure is simultaneously a demand and a carcass	units per unit time	Section 9.2.3
ϕ_j	Fraction of failures at storeroom j that storeroom j repairs itself. Written r_I in Sherbrooke (1986), which collides with the reorder point		Section 9.2.3
T_j	Average repair time at storeroom j, for the carcasses it keeps	time	Section 9.2.3
T_0	Average repair time at the depot	time	Section 9.2.3
O_j	Average order and ship time, depot to storeroom j, when the depot has stock	time	Section 9.2.3

Symbol	Meaning	Unit	Defined in
λ_{j0}	$(1-\phi_j)\lambda_j$, the rate at which storeroom j asks the depot for units	units per unit time	Section 9.2.3
λ_0	$\sum_j \lambda_{j0}$, the depot's demand rate. Poisson, by thinning and superposition	units per unit time	Section 9.2.3
S_j, S_0	Stock level at storeroom j and at the depot, the units held when nothing is in resupply	units	Section 9.2.3
X_j, X_0	Units in resupply for storeroom j, and units on the depot's benches	units	Section 9.3
μ_j, μ_0	$E[X_j]$ and $E[X_0]$. Both are exact; only the distribution of X_j is approximated	units	Section 9.3.3
σ_j^2	$Var[X_j]$. METRIC sets this equal to μ_j; VARI-METRIC does not	units squared	Section 9.8.2

Symbol	Meaning	Unit	Defined in
W_j, $\bar{W}$	Wait a request suffers at the depot, and its mean. The mean is the same for every storeroom	time	Section 9.6.2
B_{0j}	Units the depot owes storeroom j. Binomial, conditional on the depot's total	units	Section 9.8.1
f_j	λ_{j0}/λ_0, storeroom j's share of the depot's demand, and the binomial's success probability		Section 9.8.1
$E[B^2(s)]$	Second moment of the backorder function at stock level s. Not a loss function, though Equation 9.25 builds it from two	units squared	Section 9.8.3

Symbol	Meaning	Unit	Defined in
EBO	Total expected backorders at the storerooms, the quantity Section 9.9 minimizes	units	Section 9.9.1
$\delta(S)$	Delta value: backorders removed per dollar by the next unit	units per dollar	Section 9.9.2

A.14. Dynamic Lot Sizing Quantities

Chapter 5 plans one item over a finite horizon of N periods indexed $t = 1, \ldots, N$. Every quantity below carries a period subscript, and the subscript is dropped in the many cases where the costs hold still. Notice that t is a period number here and a point in continuous time elsewhere in the book.

Table A.14.: Quantities that appear when the demand rate varies by period.

Symbol	Meaning	Unit	Defined in
N	Number of periods in the planning horizon	periods	Section 5.1.1

Symbol	Meaning	Unit	Defined in
d_t	Requirement in period t. This is the discrete counterpart of the rate λ, and it replaces it for the whole of Chapter 5	units	Section 5.1.1
$d[t, u]$	Requirement of periods t through u inclusive, and zero when $u < t$	units	Section 5.1.1
$\bar{d}$	Average requirement per period over the horizon	units per period	Section 5.6.1
k_t	Order cost in period t. The same k as Section 1.4.5, given a period subscript	dollars per order	Section 5.1.1
c_t	Unit purchase cost in period t	dollars per unit	Section 5.1.1
h_t	Cost of carrying one unit from the end of period t into period $t + 1$	dollars per unit per period	Section 5.1.1

Symbol	Meaning	Unit	Defined in
Q_t	Order quantity received in period t	units	Section 5.2
I_t	Inventory at the **end** of period t, with $I_0 = 0$. Unlike the $I(t)$ of Section A.2 this is a period-end balance and not a continuous level	units	Section 5.2
y_t	One when an order is placed in period t, zero otherwise		Section 5.2
$\hat{c}(t, u)$	Cost of one unit bought in period t and used in period u, which is c_t plus the holding rates of periods t through $u - 1$	dollars per unit	Section 5.3.2

Symbol	Meaning	Unit	Defined in
$w(t, u)$	Window cost: the cost of a replenishment placed in period t that covers the requirements of periods t through u	dollars	Section 5.3.2
$V(u)$	Least cost of covering periods 1 through u and arriving at period $u + 1$ empty, with $V(0) = 0$	dollars	Section 5.4.2
$S(u)$	The period in which the last order of that least-cost plan starts	period	Section 5.4.2
M	The large constant in the linking constraint of the mixed integer program. Not the block count of Table A.12	units	Section 5.4.3

Symbol	Meaning	Unit	Defined in
VC	Variability coefficient, the variance of the requirements over the square of their mean		Section 5.6.1
W	Length of the planning window under a rolling horizon	periods	Section 5.7.1

Relevant cost is written TRC throughout Chapter 5 and means setup plus carrying, with the purchase term left out. Section 5.2.1 gives the one case in which it cannot be left out.

A.15. Requirements Planning Quantities

Chapter 6 plans every item of a product structure over a common horizon of periods. Three of its quantities are the same objects met earlier under other names, and the table says so where that is the case.

Table A.15.: Quantities that appear when requirements are computed rather than forecast.

Symbol	Meaning	Unit	Defined in
G_t	Gross requirement in period t: what the item is needed for, from a master schedule for an end item and from Equation 6.1 for everything else	units	Section 6.3
SR_t	Scheduled receipt in period t, an order already placed in a previous planning cycle. Not the $S(u)$ of Table A.14, which is a Wagner-Whitin pointer	units	Section 6.3

Symbol	Meaning	Unit	Defined in
I_t	Projected on hand at the **end** of period t. Deliberately the same symbol as Table A.14, because it is the same quantity: Equation 6.2 is Equation 5.1 with the scheduled receipts added	units	Section 6.3
NR_t	Net requirement in period t, floored at zero. Not the N of Table A.14, which is a horizon length	units	Section 6.3
POR_t	Planned order receipt in period t: what the lot sizing rule decides should arrive	units	Section 6.3

Symbol	Meaning	Unit	Defined in
Rel_t	Planned order release in period t, the receipt moved back by the lead time. This row is the record's output and the next level's input	units	Section 6.3
L	Lead time, in periods. The same L as Table A.1	periods	Section 6.3
SS	Safety stock, added to the gross requirement before netting	units	Section 6.3.2
q_{ij}	Quantity per: units of item j in one unit of item i. Equal to one throughout a distribution network		Section 6.2.2

The cumulative lead time is the sum of L down the longest path of a product structure. Section 6.3.3 shows why a horizon shorter than it cannot produce a plan.

A.16. Distribution and Loss Function Quantities

From Chapter 7 onward demand is a random variable, and the cost expressions need the expected amount by which it exceeds a stock level rather than its plain expectation. Section C.3 derives these functions and tabulates them for the distributions this book uses.

Table A.16.: Quantities that appear once demand is random.

Symbol	Meaning	Unit	Defined in
X	A random variable, usually a demand over a stated interval	units	Section C.3
$(X-b)^+$	$\max(X-b,0)$, the amount by which X exceeds b	units	Section C.3
$g(x)$	Probability mass function when X is discrete, probability density function when it is continuous		Section C.3
$G(b)$	$P\{X \le b\}$, the cumulative distribution function		Section C.3

Symbol	Meaning	Unit	Defined in
$G^0(b)$	$1 - G(b)$. For a discrete X this is $P\{X > b\}$ and not $P\{X \geq b\}$		Equation C.9
$G^1(b)$	First order loss function, $E[(X-b)^+]$: the expected shortage against a stock level of b	units	Equation C.9
$G^2(b)$	Second order loss function. Defined with a factor of $\frac{1}{2}$ and differently for discrete and continuous X	units squared	Equation C.10
$G^3(b)$	Third order loss function. Needed for the variance of a shortage when the stock level is itself random	units cubed	Equation C.22
$H^1(b)$, $H^2(b)$, $H^3(b)$	Partial expectations, the integrals of $xg(x)$, $x^2g(x)$ and $x^3g(x)$ above b	units, units squared, units cubed	Equation C.17

Symbol	Meaning	Unit	Defined in
c_X	Coefficient of variation, $\sigma_X / E[X]$	dimensionless	Section B.1.1
VMR	Variance to mean ratio, $Var[X]/E[X]$. The diagnostic that selects a discrete family	units	Equation C.1
S	A random sum, $\sum_{i=1}^{N} X_i$, of a random number of random quantities	units	Equation B.26
N, $N(t)$	The number of terms in a random sum. $N(t)$ counts demand occurrences in an interval of length t		Section B.5
$D(L)$	Demand over a lead time, the random sum $\sum_{i=1}^{L} D_i$. Written LTD in much of the literature	units	Equation B.30

Symbol	Meaning	Unit	Defined in
$\phi(z)$, $\Phi(z)$	Standard normal density and distribution function		Section C.3.4
$\Phi^0(z)$, $\Phi^1(z)$, $\Phi^2(z)$, $\Phi^3(z)$	The standard normal complement and loss functions, all dimensionless. Scaled by σ, σ^2 and σ^3 to give G^1, G^2 and G^3		Equation C.19

Notice the superscripts. They are orders of a single family and not powers: G^0 is a probability, G^1 is in units, G^2 is in units squared and G^3 is in units cubed, each obtained by integrating or summing the one before it. Thus, the unit of G^n is the unit of the demand raised to the nth power, which is the quickest check that a loss function has been scaled correctly.

A.17. Stochastic Process Quantities

Section B.3 and Section B.4 model demand as a stream of events in time rather than as a quantity, and Chapter 8 works with both views. The symbols below belong to the stream.

Table A.17.: Quantities for demand modeled as a stream of events. Notice that λ is the demand rate in units per unit time throughout this book, so for a stream of single-unit demands it is also the event rate, and for batched demand it is not.

Symbol	Meaning	Unit	Defined in
$N(t)$	Number of demand events in an interval of length t	events	Equation B.15
T_i	Time between the $(i-1)$st and ith events	time	Section B.3.1
μ_T, σ_T^2	Mean and variance of the time between events	time, time squared	Section B.4
$m(t)$	Renewal function, $E[N(t)]$	events	Equation B.21
S_n	Time of the nth event, $T_1 + \cdots + T_n$	time	Equation B.20
$D(L)$	Demand over a lead time. Written LTD in much of the literature	units	Equation B.30

Two of these are worth keeping apart from symbols that look like them. $\overline{L}$ in Table A.3 is the average lead time and $\overline{IO}$ in Table A.2 is the average number of units on order; neither is the $\overline{L}$ that the queueing literature writes for the average number in a system, which this book writes as

a named average instead. And $N(t)$ counts demand events, while $\overline{N}$ in Table A.2 is the average rate at which *orders are placed*. The two are different streams.

A.18. Symbols That Carry More Than One Meaning

A book this long cannot give every quantity its own letter, and five symbols carry more than one meaning. The context always decides, and the collisions are listed here so that nobody has to discover them.

- θ is the Lagrange multiplier of Section 4.2, the mean lead time demand $E[D(L)]$ throughout Chapter 8, the price of money in the relaxation of Section 9.11.9, and the error factor on a parameter in Section 1.5. The second is by far the most frequent.
- β is the Type 2 service target in Section 8.7, the scale parameter of the gamma distribution in Section C.2, and $(1-p)/p$ in the negative binomial fit of Section 9.8.4.
- α is the Type 1 service target in Section 8.7, the shape parameter of the gamma distribution, and the smoothing constant of Section 8.15.
- T is the observation interval in Section 1.3, the cycle length Q/λ in Chapter 3, the base period in Section 4.5, and a repair time when it carries a location subscript in Chapter 9.
- S is an order-up-to or base-stock level in every policy chapter, a random sum in Section B.5, and, written $S(u)$, the traceback pointer of Section 5.4.2.

A.19. A Note on Other Sources

Inventory theory has an unfortunate amount of notational variation, and a reader moving between this book and the literature will meet the same quantity under several names. Two differences recur.

The fixed order cost is written k here and A in much of the classical literature; the unit cost is written c here and v in Silver et al. (2016) and the practitioner literature that follows it. And *Simulation Modeling using the KSL* writes $BO(t)$ for the quantity this book calls $B(t)$; the definition is identical.

Three more recur throughout Chapter 8. The replenishment lead time is L here and τ in a large part of the stochastic inventory literature; this book uses τ for the protection interval $R + L$ of Section 8.13. Demand over that lead time is $D(L)$ here and LTD or X elsewhere. And the backorder cost is b here, which collides with the scale parameter some sources write b for in the gamma distribution; this book writes that scale as β throughout Section C.2.

The forecasting literature differs again. Section 8.15 writes the one period forecast as $\hat{D}_t$ and the scaling exponent as η, where Silver et al. (2016) writes $\hat{a}_t$ and c. The exponent is renamed here because c is the unit cost throughout this book. You should check the symbol list of any source before carrying a formula across.

B. Probability and Stochastic Processes Review

Learning objectives

After reading this chapter you should be able to:

- compute expectations and variances, and condition on a second random variable
- find the mean and the variance of a sum, and say when the variances add
- state what the central limit theorem gives and what it does not
- work with the Poisson and renewal processes used in the inventory models
- explain why the fill rate and the ready rate coincide only under Poisson demand
- say when the variance of a duration matters to a total and when only its mean does
- compute the expectations of random sums that arise in lead-time demand

B.1. Random Variables and Expectation

This section and the four that follow are a review for a reader who has had a probability course, not a substitute for one. They collect the results the inventory models use, in the notation the book uses, so that a chapter can point here rather than rederive them.

A **random variable** assigns a number to the outcome of an experiment whose result is not known in advance. Let X represent one. The **cumulative distribution function**

$$F(x) = P\{X \le x\} \tag{B.1}$$

determines everything about X, and it is the object the inventory models actually use. Recall from Equation 7.12 that the newsvendor quantity is a quantile of F, and from Section C.3 that both loss functions are integrals of it.

A **discrete** random variable takes values in a countable set and carries a probability mass function $g(x) = P\{X = x\}$. A **continuous** one carries a density $f(x)$ with $F(x) = \int_{-\infty}^{x} f(u)\,du$. Demand arrives in whole units, so the natural default is discrete and a continuous model is an approximation made for convenience. Section 7.8 says when that approximation is safe and what it costs when it is not.

B.1.1. Expectation, Variance, and Two Ratios

The **expectation** of X is its probability-weighted average,

$$E[X] = \sum_{x} x\, g(x) \quad \text{or} \quad \int_{-\infty}^{\infty} x f(x)\,dx \tag{B.2}$$

and the **variance** is the expected squared deviation from it,

$$Var[X] = E[(X - E[X])^2] = E[X^2] - \left(E[X]\right)^2 \tag{B.3}$$

The second form is the one to compute with, and it is how Equation B.31 below collapses to a second moment. The **standard deviation** $\sigma_X = \sqrt{Var[X]}$ carries the units of X and is therefore the one to report.

Two dimensionless ratios appear throughout the book and they are not the same thing. The **coefficient of variation** $c_X = \sigma_X / E[X]$ measures spread relative to level, and Table C.1 reads a normal model's defect off it directly. The **variance to mean ratio** of Equation C.1 selects among discrete families and carries the units of X. Notice that c_X compares a standard deviation with a mean and VMR compares a variance with a mean, so the two answer different questions and only one of them is a pure number.

For a **linear** transformation, with a and b constants,

$$E[aX + b] = aE[X] + b, \qquad Var[aX + b] = a^2\, Var[X] \tag{B.4}$$

Notice that b moves the mean and does not touch the variance, and that a enters the variance squared. That is why a change of units rescales a standard deviation linearly, and it is the whole of the argument in Section 1.4.1 that a constant added to every alternative cannot change which alternative is least.

B.1.2. A Function of a Random Variable

For any function h,

$$E[h(X)] = \sum_x h(x)\, g(x) \quad \text{or} \quad \int_{-\infty}^{\infty} h(x) f(x)\, dx \tag{B.5}$$

so the expectation is taken against the distribution of X and no distribution of $h(X)$ has to be found. Equation 7.10 is Equation B.5 applied to the newsvendor cost, and Equation C.9 is it applied to $(X-b)^+$.

The result that matters here is the one about what you may not do. In general

$$E\big[h(X)\big] \neq h\big(E[X]\big) \tag{B.6}$$

and the two sides differ by more than a rounding. Replacing a random demand by its average and evaluating the cost there is the commonest modeling error in inventory work, and it has a name outside this book: the flaw of averages. Thus, a model that plans against average demand is not a conservative approximation of a stochastic model. It is a different model, and usually an optimistic one.

For example, take the five-point demand sketch of Table 7.1, whose mean is 485 belts, with $c_o = \$38$ and $c_u = \$45$. Order the average, $Q = 485$. Substituting the average demand into Equation 7.4 gives

$$C(485, 485) = 38(485-485)^+ + 45(485-485)^+ = \$0$$

which reports that ordering the mean costs nothing at all. Now let's take the expectation properly. We compute the expected leftover as 49.0 belts and the expected shortage as also 49.0 belts, so by Equation 7.10

$$E\big[C(485)\big] = 38(49.0) + 45(49.0) = 49.0(83) = \$4{,}067$$

The flaw of averages understated the cost of that decision by the entire cost of it. Notice that the two expectations came out equal, and that this is not a coincidence: Equation 7.9 says their difference is $Q - E[X]$, which is zero at $Q = E[X]$. That identity is worth using as an arithmetic check every time a loss function is evaluated at the mean.

B.1.3. Conditioning

We derive most of the results in this book by conditioning on a second random variable and then averaging over it. Let Y represent that second variable. The **conditional expectation** $E[X \mid Y]$ is the expectation of X computed within each value of Y, and it is itself a random variable, because it depends on which value Y takes. Averaging it over Y recovers the unconditional mean,

$$E[X] = E\big[E[X \mid Y]\big] \tag{B.7}$$

which is the **law of total expectation**. Equation B.27 is Equation B.7 with Y the number of terms in a sum, and Chapter 8 conditions on the inventory position in the same way.

The variance needs both a within and a between term. The **law of total variance** is

$$Var[X] = E\big[Var[X \mid Y]\big] + Var\big[E[X \mid Y]\big] \tag{B.8}$$

That is, the total variability is the average variability within a value of Y plus the variability of the averages across values of Y. Both terms are non-negative, so conditioning on anything can only reduce the first term at the price of raising the second. Equation B.29 is Equation B.8 worked through for a random sum, and the asymmetry that section makes so much of is visible here already: the first term averages a variance and the second takes the variance of a mean.

Almost every quantity the inventory models need is a sum of the quantities defined here, so the next section takes up what addition does to a mean, to a variance, and to a distribution.

B.2. Sums of Random Variables

Almost every quantity this book models is a sum. Demand over a lead time is a sum of period demands, demand over a season is a sum of weeks, and an aggregate investment is a sum across items. This section gives what happens to the moments and to the distribution when quantities are added.

Let $X_1, X_2, \ldots, X_n$ represent random variables and let $S_n = X_1 + X_2 + \cdots + X_n$.

B.2.1. The Mean and the Variance of a Sum

The mean of a sum is the sum of the means, **always**, whatever the dependence among the terms:

$$E[S_n] = \sum_{i=1}^{n} E[X_i] \tag{B.9}$$

The variance is not so obliging. In general we have

$$Var[S_n] = \sum_{i=1}^{n} Var[X_i] + 2\sum_{i<j} Cov[X_i, X_j] \tag{B.10}$$

and the covariance terms vanish only when the terms are uncorrelated. Thus, the familiar statement that variances add is a statement about **independence** and not about addition. Notice the direction of the error when independence is assumed wrongly. Demand that runs in streaks has positive covariances, so Equation B.10 without its second term understates the spread, and understating the spread understates the safety stock. Section B.5.5 makes the same point for random sums.

When the terms are independent and identically distributed, with mean μ and variance σ^2,

$$E[S_n] = n\mu, \qquad Var[S_n] = n\sigma^2, \qquad \sigma_{S_n} = \sigma\sqrt{n} \tag{B.11}$$

Notice that the mean grows linearly and the standard deviation grows only as $\sqrt{n}$. Thus, the coefficient of variation of a sum is

$$c_{S_n} = \frac{\sigma\sqrt{n}}{n\mu} = \frac{c_X}{\sqrt{n}} \tag{B.12}$$

so aggregation makes a quantity **relatively** less variable even as it makes it absolutely more variable. That single fact is behind a great deal of inventory practice: it is why pooling stock across locations reduces the safety stock needed, which Chapter 9 takes up, and it is why a demand that is far too variable to model in one bucket can be tractable in a larger one.

For example, weekly demand for the belt of Section 7.3 has a mean of 33.2404 belts and a standard deviation of 24.7596 belts, so $c_X = 0.7449$. Over the fourteen remaining weeks,

$$E[X] = 14(33.2404) = 465.37 \text{ belts}, \qquad Var[X] = 14(613.0387) = 8582.54 \text{ belts}^2$$

so $\sigma = 92.64$ belts and $c_X = 92.64/465.37 = 0.1991$. Let's check that against Equation B.12: $0.7449/\sqrt{14} = 0.7449/3.7417 = 0.1991$. The two agree, and that check is worth performing every time, because it catches a variance that was added when it should have been multiplied.

B.2.2. Averages, and Why Precision Is Expensive

The **sample mean** of n independent observations is $\bar{X} = S_n/n$. Applying Equation B.4 to Equation B.11 with $a = 1/n$,

$$E[\bar{X}] = \mu, \qquad Var[\bar{X}] = \frac{\sigma^2}{n}, \qquad \sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}} \tag{B.13}$$

The quantity $\sigma_{\bar{X}}$ is the **standard error** of the mean, and an approximate confidence interval for μ is $\bar{X} \pm z\sigma/\sqrt{n}$, whose **half-width** is $z\sigma/\sqrt{n}$.

Read the exponent before using it. The half-width falls with $\sqrt{n}$, so halving it costs four times the observations and dividing it by ten costs a hundred times. Section 7.10 reports every simulated figure with its half-width for this reason, and the arithmetic above is why a simulated answer is expensive to sharpen and an exact one is not.

B.2.3. Families Closed Under Addition

Equation B.11 gives two moments and not a distribution. For a few families the distribution of the sum stays in the family, and those cases turn an approximation into an identity.

Table B.1.: Families closed under addition. In every row the parameter that is shared must be shared, and the parameter that adds is the one that counts.

If $X_1, \ldots, X_n$ are independent and	then S_n is
normal, $N(\mu_i, \sigma_i^2)$	normal, $N(\sum \mu_i, \sum \sigma_i^2)$
Poisson, rate λ_i	Poisson, rate $\sum \lambda_i$
gamma with a **common** scale, shapes α_i	gamma, same scale, shape $\sum \alpha_i$

If $X_1, \dots, X_n$ are independent and	then S_n is
negative binomial with a **common** p, sizes r_i	negative binomial, same p, size $\sum r_i$
binomial with a **common** p, trials n_i	binomial, same p, trials $\sum n_i$

Three notes on Table B.1. The normal row holds without the terms being identically distributed, which is the reason the normal is so often reached for. The gamma and negative binomial rows require the common parameter, so a sum of gammas with different scales is **not** a gamma and has to be approximated. And the exponential is the gamma with shape one, so a sum of n independent exponentials with the same rate is the Erlang of Section C.2.2, which is the fact Chapter 8 needs when a lead time is built from stages.

Closure is the exception and not the rule. For every other combination the distribution of the sum has to be obtained by convolution, by a transform, by simulation, or by matching moments to a chosen family as Section C.4.2 does.

B.2.4. The Central Limit Theorem, and What It Does Not Say

If $X_1, X_2, \dots$ are independent and identically distributed with mean μ and finite variance σ^2, then

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} \longrightarrow N(0,1) \qquad \text{as } n \to \infty \tag{B.14}$$

whatever the distribution of the terms. Thus, a quantity aggregated over many independent contributions tends toward the normal, which is the justification Section C.2.3 gives for a family that is otherwise indefensible for demand.

Three qualifications, and you should apply all three before treating an aggregate as normal.

It is a statement about the limit, not about your n. How large n must be depends on how skewed the terms are. A symmetric term converges in a handful of observations and a heavily right-skewed one may need hundreds.

The approximation is worst in the tail, which is where inventory lives. Recall from Section 7.6 that the quantity of interest is a quantile, often far from the mean. A central limit argument is strongest at the center, so agreement of the means says nothing about agreement at the 95th percentile.

The support is wrong. The normal puts mass below zero and demand cannot be negative. Table C.1 prices that defect against c_X, and combining it with Equation B.12 gives the practical test: aggregate until c_X falls far enough that the mass below zero is negligible, then check rather than assume.

For example, the belt's weekly demand at $c_X = 0.7449$ would put about 9% of a normal model below zero, which is not a small error in a tail but a model asserting that demand is frequently negative. Aggregated over fourteen weeks, $c_X = 0.1991$ and the mass below zero is about one in four million. Notice that the family did not change and the aggregation did. The normal was wrong weekly and safe over fourteen weeks for the reason Equation B.12 gives, and the test that settles it is the coefficient of variation of the quantity actually being modeled.

Equation B.11 assumed a fixed number of terms. In an inventory system the number of terms is usually random, because demands arrive at random times, so the next two sections build the process that counts them before Section B.5 sums over it.

B.3. The Poisson Process

The sections above treated demand as a quantity. This one treats it as a stream of events arriving in time, which is what the continuous review models of Chapter 8 actually face. The Poisson process is the standard model for such a stream, and the inventory results lean on it harder than on anything else in this appendix.

Let $N(t)$ represent the number of events in the interval $(0, t]$, with $N(0) = 0$. The collection $\{N(t), t \geq 0\}$ is a **counting process**. It is a **Poisson process with rate** λ when three properties hold.

1. **Independent increments.** The numbers of events in non-overlapping intervals are independent.
2. **Stationary increments.** The number of events in an interval depends only on the interval's length and not on where it starts.
3. **Orderliness.** Events do not occur simultaneously. In an interval of length h the probability of exactly one event is $\lambda h + o(h)$ and the probability of two or more is $o(h)$.

From those three,

$$P\{N(t) = x\} = \frac{e^{-\lambda t}(\lambda t)^x}{x!}, \qquad E[N(t)] = Var[N(t)] = \lambda t \tag{B.15}$$

so the count over any interval is the Poisson distribution of Section C.1.1 with mean λt. Notice that the mean and the variance are equal, which is Equation C.1 equal to one, and that this is forced by the three properties rather than assumed separately.

You should read the three properties as assumptions to be checked and not as technical preliminaries. Independent increments is the one that fails most often in demand data, because a promotion, a shortage at a competitor, or a shared weather event correlates one interval with the next. Figure 7.3 is that check performed on a real series.

B.3.1. Interarrival Times

Let T_i represent the time between the $(i-1)$st and ith events. The interarrival times of a Poisson process are independent and exponentially distributed with rate λ, and the converse holds as well, so the two characterizations are the same process described from different ends. Thus, a mean interarrival time of $1/\lambda$ and a mean rate of λ carry the same information, and which one to work with is a matter of convenience.

For example, a spare part demanded on average once every 5 weeks has

$$\lambda = \frac{1 \text{ demand}}{5 \text{ weeks}} = 0.2 \text{ demands per week}$$

and over a lead time of 3 weeks the demand count is Poisson with mean $0.2(3) = 0.6$ demands. Notice that the rate and the interval must be measured in the same time unit, which is the same caution Equation B.30 carries.

The exponential is **memoryless**, so the time until the next demand does not depend on how long we have already waited. That is, a Poisson process has no notion of being overdue. It is the strongest statement in this appendix and the reason the Poisson process is both tractable and often wrong.

B.3.2. Superposition, Thinning, and Conditioning

Three operations preserve the process, and all three are used later.

Superposition. Merging independent Poisson processes with rates $\lambda_1, \ldots, \lambda_n$ gives a Poisson process with rate $\sum \lambda_i$. Thus, demand at a warehouse serving n independent retailers is Poisson when each retailer's is, which is what lets Chapter 9 treat an upstream location as facing one stream.

Thinning. If each event is independently retained with probability p, the retained events form a Poisson process with rate $p\lambda$ and the discarded events form an independent Poisson process with rate $(1-p)\lambda$. For example, if one demand in ten is for an emergency order, the emergency stream is Poisson with rate $\lambda/10$ and is independent of the routine stream.

Conditional uniformity. Given $N(t) = n$, the n event times are distributed as the order statistics of n independent observations drawn uniformly on $(0, t)$. That is, conditional on how many events occurred, the process says nothing about when they occurred beyond that they are scattered at random. Section B.3.3 uses this property together with thinning, and the result it gives is the most surprising one in this appendix.

B.3.3. Palm's Theorem

Suppose events occur in a Poisson process at rate λ, and each event starts an activity whose duration is drawn independently from a distribution with mean $E[L]$. Let IO represent the number of activities in progress at a random instant in steady state. Then

$$P\{IO = x\} = \frac{e^{-\lambda E[L]} \left(\lambda E[L]\right)^x}{x!} \tag{B.16}$$

Poisson with mean $\lambda E[L]$, whatever the distribution of the duration is. Only its mean enters. That is **Palm's theorem**, and the same result is known as the $M/G/\infty$ occupancy distribution.

The argument uses the two properties above and nothing else. Condition on the number of events in $(0, t]$. Given that there were n, their times are independent and uniform on $(0, t)$ by conditional uniformity. An event at time u is still in progress at t with probability $1 - G_L(t - u)$, and those outcomes are independent across events, so the events still in progress are a **thinning** of the n with retention probability

$$q = \frac{1}{t}\int_0^t [1 - G_L(v)]\, dv \tag{B.17}$$

A thinned Poisson count is Poisson with mean $\lambda t q$, and as $t \to \infty$ the integral in Equation B.17 tends to $E[L]$, which gives Equation B.16. For a compound process in which each event carries a quantity, the mean becomes $\lambda E[L]E[Y]$ by Equation B.27.

Read Equation B.16 against Equation B.30 before using either. They appear to contradict one another. Equation B.30 says that when the duration is random its variance inflates the total, through the term $Var[L](E[D])^2$, and Equation B.16 says the variance does not matter at all. Both are correct, and they describe two different kinds of randomness.

- Equation B.16 is about **variability between activities**. Each draws its own duration independently, so a long one and a short one overlap, and the long one delays nothing. Averaged over many, only the mean survives.
- Equation B.30 is about the total accumulated over **one** duration. If that duration is a single unknown common to everything in progress, then a long draw stretches all of it at once and the variance is not averaged away.

Notice that the two are not alternative analyses of one situation. They are analyses of two situations, and the question that separates them is whether activities started later can finish earlier. Section 8.5.4 applies this distinction to a replenishment system, where the answer decides which formula is the right one.

B.3.4. PASTA

An inventory system is measured in two different ways and the difference matters. The **ready rate** is the fraction of *time* that stock is on the shelf.

The **fill rate** is the fraction of *demands* that are filled from the shelf. The first is an average over time and the second is an average over customers, and in general they are different numbers.

They coincide under one condition. **Poisson Arrivals See Time Averages**, or PASTA, says that when arrivals form a Poisson process and do not anticipate the future of the system, the distribution of the system state found by an arriving customer equals the long-run time-stationary distribution. Thus, under Poisson demand the fraction of demands arriving to an empty shelf equals the fraction of time the shelf is empty, and the fill rate equals the ready rate.

Notice the size of what that assumption is doing. Under any other arrival process an arriving demand is more likely to find the system in some states than a randomly chosen instant is, and the two service measures separate. Chapter 8 keeps them apart for exactly this reason, and reports which one a formula computes.

B.3.5. Demand Over a Random Lead Time

We can now settle a question Chapter 8 will ask directly: if demand is Poisson and the lead time is itself random, what distribution does the lead time demand have?

Let D represent the demand over a lead time L, where L is independent of the demand process. Conditioning on $L = \ell$ in the manner of Equation B.7, the demand is Poisson with mean $\lambda\ell$, so the probability generating function of D is

$$\tilde{g}_D(z) = E\left[e^{-\lambda L(1-z)}\right] = \tilde{f}_L(\lambda(1-z)) \tag{B.18}$$

where $\tilde{f}_L$ is the Laplace transform of the lead time. That is, the lead time demand is obtained by evaluating the lead time's own transform at $\lambda(1-z)$, and no new machinery is needed.

Take L to be Erlang with shape n and rate μ, which by Section C.2.2 is the sum of n independent exponential stages. Its transform is $\tilde{f}_L(s) = (\mu/(\mu+s))^n$, so

$$\tilde{g}_D(z) = \left(\frac{\mu}{\mu+\lambda(1-z)}\right)^n = \left(\frac{p}{1-(1-p)z}\right)^n, \qquad p = \frac{\mu}{\lambda+\mu} \quad \text{(B.19)}$$

which is the generating function of the negative binomial of Equation C.3 with $r = n$ and that p. **Thus, Poisson demand over an Erlang lead time is exactly negative binomial.** Section C.1.2 reaches for the negative binomial because real demand has $VMR > 1$; Equation B.19 says where that extra variability comes from when the lead time is the source of it.

A warning about p. Recall from Section C.1.2 that two conventions are in circulation. Equation B.19 is in the convention of Equation C.3, where p is the probability of success, so $p = \mu/(\lambda+\mu)$. The failure convention gives $\lambda/(\lambda+\mu)$ instead, and at $\lambda = 3$ and $\mu = 2$ those are 0.400 and 0.600. Notice that both are admissible values, so a routine handed the wrong one returns an answer rather than an error.

The arithmetic check. Take $\lambda = 3$ demands per week, $n = 4$ and $\mu = 2$ per week, so $p = 2/5 = 0.400$ and $r = 4$. From Equation C.3, $\beta = (1-p)/p = 1.5$, so $E[D] = 4(1.5) = 6$ units and $Var[D] = 4(1.5)/0.4 = 15$ units squared. Now compute the same two moments from Equation B.30, using $E[L] = n/\mu = 2$ weeks and $Var[L] = n/\mu^2 = 1$ week squared:

$$E[D] = 3(2) = 6, \qquad Var[D] = 3(2) + 3^2(1) = 6 + 9 = 15$$

The two routes agree. That check is worth performing every time a transform argument produces a named family, because it catches an inverted convention immediately. Passing $p = 0.600$ to Equation C.3 instead would

have reported a mean of 2.667 units and a variance of 4.444, and neither figure announces itself as wrong.

The Poisson process assumes exponential interarrival times, and real demand rarely obliges. The next section keeps the counting and drops that assumption.

B.4. Renewal Processes

A renewal process keeps the counting of Section B.3 and drops the exponential interarrival time. Let $T_1, T_2, ...$ represent independent and identically distributed positive interarrival times with mean $\mu_T = E[T]$ and variance σ_T^2, let $S_n = T_1 + \cdots + T_n$ be the time of the nth event, and let

$$N(t) = \max\{n : S_n \leq t\} \tag{B.20}$$

count the events in $(0, t]$. The Poisson process is the special case in which T is exponential. Thus, everything below specializes to Equation B.15, and we use that as the check on each result.

Notice the relationship Equation B.20 encodes: $N(t) \geq n$ exactly when $S_n \leq t$. That is, a statement about a count is a statement about a sum, so Section B.2 supplies the distribution of S_n and this section translates it into statements about $N(t)$.

B.4.1. How Many Events, and How Variable

The **renewal function** $m(t) = E[N(t)]$ has no convenient closed form for most interarrival distributions, and for large t it does not need one. The **elementary renewal theorem** states

$$\lim_{t\to\infty} \frac{m(t)}{t} = \frac{1}{\mu_T} \tag{B.21}$$

so events accumulate at the reciprocal of the mean interarrival time. For example, a part demanded on average once every 5 weeks generates about $t/5$ demands in t weeks, which is the statement a planner would have made without any of this machinery. Equation B.21 is the guarantee that the obvious statement is correct, and that only the **mean** of the interarrival time enters it.

The variance is where the interarrival distribution reappears. For large t,

$$Var[N(t)] \approx \frac{\sigma_T^2}{\mu_T^3}\, t \tag{B.22}$$

Check it against the Poisson process, where T is exponential with $\mu_T = 1/\lambda$ and $\sigma_T^2 = 1/\lambda^2$. Then Equation B.22 gives $(1/\lambda^2)(\lambda^3)t = \lambda t$, which is Equation B.15 exactly. That agreement is the check worth performing on any renewal formula, because the Poisson case is the one whose answer is already known.

Equation B.22 is a rate and not the whole answer. The exact variance differs from it by a bounded amount that does not grow with t, and that correction depends on the **third** moment of the interarrival time and not only on its first two. That is why a routine that computes the variance of the number of demands in an interval asks for three moments of the time between demands, which will look like an error to a reader who has only seen Equation B.22. For a Poisson process the correction is exactly zero, which is another way of saying the exponential is the case in which two moments suffice.

B.4.2. A Random Horizon

Chapter 8 counts demands over a **lead time**, which is itself random. Let L represent the lead time, independent of the demand process. Applying Equation B.8 to $N(L)$, with Equation B.21 for the conditional mean and Equation B.22 for the conditional variance,

$$E[N(L)] \approx \frac{E[L]}{\mu_T}, \qquad Var[N(L)] \approx \frac{\sigma_T^2}{\mu_T^3} E[L] + \frac{Var[L]}{\mu_T^2} \tag{B.23}$$

Read the two variance terms the way Section B.5.2 reads its own. The first is the variability of the demand **stream**, accumulated over the expected length of the lead time. The second is the variability of the **lead time**, converted into a count by dividing by the mean interarrival time twice. Thus, a supplier who is unreliable and a demand stream that is erratic enter the same expression through different doors, and computing the two terms separately tells you which door to close.

The check. Take demand Poisson at rate λ, so $\mu_T = 1/\lambda$ and $\sigma_T^2 = 1/\lambda^2$. Then Equation B.23 gives

$$Var[N(L)] \approx \lambda E[L] + \lambda^2 \, Var[L]$$

which is Equation B.30 for demands of one unit each. For example, at $\lambda = 3$ per week and an Erlang lead time with $E[L] = 2$ weeks and $Var[L] = 1$ week squared, the variance is $3(2) + 9(1) = 15$, which is the figure Equation B.19 produced exactly. Three routes, one answer.

B.4.3. Little's Law

One result about long-run averages holds with almost no assumptions and is used throughout Chapter 8. Let λ represent the long-run arrival rate

into a system, let $\overline{W}$ represent the average time an arrival spends in it, and let $\overline{L}$ represent the average number in it. Then

$$\overline{L} = \lambda \overline{W} \tag{B.24}$$

Little's Law requires only that the averages exist and that the system is in steady state. It does not require Poisson arrivals, independence, or any distributional assumption, and that generality is what makes it useful.

Its inventory reading is Equation 1.6, which Section 1.6 obtained by this argument before the machinery for it existed. Take the system to be the set of backordered demands. Then $\overline{L}$ is the average backorder level $\overline{B}$, λ is the demand rate, and $\overline{W}$ is the average time a demanded unit waits. Thus, a constraint on the average number of units backordered is the same constraint as one on how long a customer waits, and two service targets an organization would argue about separately are one target in two units. Chapter 8 uses Equation 1.6 in both directions.

B.4.4. Demands That Arrive in Batches

Nothing so far has said how **much** is demanded at an event. Let D_i represent the size of the ith demand. The total demand over an interval of length t is

$$\sum_{i=1}^{N(t)} D_i \tag{B.25}$$

a **compound renewal process**, and a random sum in which the count comes from Equation B.20. When the counting process is Poisson this is the compound Poisson of Section C.4.3.

Equation B.25 is the object Section B.5 takes up, and it now has both of its ingredients: this section supplies $E[N(t)]$ and $Var[N(t)]$, and the next supplies what to do with them once the sizes are random too.

B.5. Random Sums

A **random sum** adds up a random number of random quantities. Let N represent a non-negative integer random variable, let $X_1, X_2, ...$ represent identically distributed random variables with the same distribution as some X, and let

$$S = \sum_{i=1}^{N} X_i, \qquad S = 0 \text{ when } N = 0 \tag{B.26}$$

Assume throughout that the X_i are independent of one another **and of** N. That last requirement is the one that fails in practice, and Section B.5.5 says what it costs.

Random sums appear twice in this book, and both times the quantity being summed is demand.

Demand over a random lead time. Chapter 8 needs the demand that occurs while a replenishment order is outstanding. Let D_i represent the demand in period i and let L represent the lead time measured in whole periods. Then the lead time demand is $D(L) = \sum_{i=1}^{L} D_i$, a random sum in which N is the lead time.

Demand that arrives in batches. A slow-moving item is not demanded every period. Let $N(t)$ represent the number of demand occurrences in an interval of length t and let D_i represent the size of the ith one. Then the demand over the interval is $\sum_{i=1}^{N(t)} D_i$, a random sum in which N counts occurrences rather than periods. Section C.4.3 uses this representation for intermittent demand.

B.5.1. The Mean

Condition on N. Given $N = n$ the sum has exactly n terms, so $E[S \mid N = n] = nE[X]$. Thus, $E[S \mid N] = NE[X]$, and taking the expectation of both sides,

$$E[S] = E\big[E[S \mid N]\big] = E\big[NE[X]\big] = E[N]\,E[X] \tag{B.27}$$

Equation B.27 is Wald's equation, and it says the obvious thing: the expected total is the expected count times the expected size. Notice that it needs only the *means* of N and X, which is why it is the one result in this section that survives almost any weakening of the assumptions.

B.5.2. The Variance

The variance is where the interesting behavior is, and the obvious guess is wrong. Conditioning again, and using the law of total variance,

$$Var[S] = E\big[Var[S \mid N]\big] + Var\big[E[S \mid N]\big] \tag{B.28}$$

Take the two terms in turn. Given $N = n$ the sum is of n independent terms, so $Var[S \mid N] = N\,Var[X]$ and the first term is $E[N]\,Var[X]$. And $E[S \mid N] = NE[X]$, so the second term is $Var[NE[X]] = (E[X])^2\,Var[N]$. Thus,

$$Var[S] = E[N]\;Var[X] + Var[N]\,\big(E[X]\big)^2 \tag{B.29}$$

The two terms are not symmetric, and that asymmetry is the point. The variability of the *sizes* enters multiplied by the expected count. The variability of the *count* enters multiplied by the square of the expected size. That is, uncertainty in how many contributes a term that grows

quadratically in the typical size, while uncertainty in how much contributes only linearly in the typical count.

For example, an item demanded five units at a time is four times as exposed to uncertainty in the number of demands as an item demanded two and a half units at a time, all else equal. Recall from Section 8.3.2 that a supplier's reliability and a customer's predictability are different problems. Equation B.29 is why they are also different sizes of problem.

B.5.3. Lead Time Demand

Specializing Equation B.27 and Equation B.29 to $D(L) = \sum_{i=1}^{L} D_i$ gives the two results Section C.4.2 and Chapter 8 both use:

$$E[D(L)] = E[L]\,E[D], \qquad Var[D(L)] = E[L]\;Var[D] + Var[L]\,(E[D])^2 \tag{B.30}$$

Notice the units. $E[L]$ is in periods and $E[D]$ in units per period, so the product is in units. In the variance, $E[L]\,Var[D]$ is periods times units squared per period squared, and $Var[L](E[D])^2$ is periods squared times units squared per period squared. Both come to units squared, which is the check that the two terms belong in the same sum. Notice also that L and D must be measured in the *same* period. A lead time in days and a demand per week is the most common error in applying Equation B.30, and it is invisible in the answer.

For example, take an item averaging 5 units a week with a variance of 16, and a supplier whose lead time averages 10 weeks with a variance of 25. Then

$$E[D(L)] = 10(5) = 50 \text{ units}$$

$$Var[D(L)] = 10(16) + 25(5)^2 = 160 + 625 = 785 \text{ units}^2$$

so the standard deviation of lead time demand is $\sqrt{785} = 28.02$ units.

Read the two terms before moving on. The demand contributes 160 of the 785 and the lead time contributes 625, so about 80% of the variance the safety stock must cover comes from the supplier rather than from the customer. That split is typical, and it is the argument for spending on lead time reliability before spending on forecasting. You should compute both terms separately every time, because the total alone does not say where to act.

B.5.4. Batches, and the Compound Poisson

When N counts demand *occurrences* rather than periods, Equation B.29 applies unchanged with $N = N(t)$. What changes is that $E[N(t)]$ and $Var[N(t)]$ now have to come from somewhere, and Section B.4 is where.

One case is clean enough to state here. If demands occur according to a Poisson process with rate λ, then $N(t)$ is Poisson with $E[N(t)] = Var[N(t)] = \lambda t$, and the two terms of Equation B.29 collapse:

$$Var[S] = \lambda t \, (\, Var[X] + (E[X])^2) = \lambda t \, E[X^2] \tag{B.31}$$

Thus, a compound Poisson demand needs only the *second moment* of the batch size, not its variance and mean separately. For example, at three demand occurrences on average, each averaging 5 units with a variance of 16, the total averages $3(5) = 15$ units with a variance of $3(16)+3(25) = 123$, which Equation B.31 gives directly as $3(41) = 123$. The two routes agree, and that agreement is the arithmetic check for this section.

Notice what Equation B.31 implies. The variance to mean ratio of a compound Poisson demand is $E[X^2]/E[X]$, which exceeds one whenever

the batch size is ever larger than one unit. That is the reason Section C.1 sends batched demand to the negative binomial rather than the Poisson, and it is why a Poisson *process* does not produce Poisson *period demand* unless every demand is for a single unit.

B.5.5. What This Does and Does Not Give You

Equation B.27 and Equation B.29 give two moments. They do not give a distribution, and the inventory models of Chapter 8 onward need one. Section C.4 is the bridge, and it is an assumption rather than a derivation: two numbers do not determine a distribution, so a family has to be chosen.

Two assumptions stand behind the results above, and both fail in real data.

The count must not depend on the sizes. Suppose a large demand triggers an expedite that shortens the lead time. Then L and the D_i are dependent. Equation B.27 survives that under weaker conditions; Equation B.29 does not, and the true variance is usually smaller than it gives.

The sizes must not depend on one another. Demand that is autocorrelated, a run of heavy weeks followed by a run of light ones, breaks the first term of Equation B.29. Recall from Section 2.2.6 that burstiness is one of the things a demand history is characterized for. An item that tests as bursty is an item for which Equation B.29 understates the spread, and understating the spread understates the safety stock.

C. Common Probability Distributions

Learning objectives

After reading this chapter you should be able to:

- identify the distributions used to model demand and lead time
- compute the loss functions that appear in inventory cost expressions
- explain why a variance of a shortage needs a loss function one order higher

C.1. Discrete Distributions

Demand arrives in whole units, so a discrete distribution is the natural default and a continuous one is an approximation made for convenience. This section gives the four families the book uses, what each assumes, and the single diagnostic that usually decides between them. Section C.3 gives their loss functions.

The diagnostic is the variance to mean ratio. Let X represent a demand over some interval, and let

$$VMR = \frac{Var[X]}{E[X]} \tag{C.1}$$

Recall from Section 2.2.6 that a demand history is characterized before it is modeled. Equation C.1 is the characterization that selects among the families below, because each of them fixes the ratio or bounds it on one side. Notice that VMR is not the variability coefficient of Equation 5.10 or the squared coefficient of variation of Equation 2.3. Those are dimensionless; this one carries the units of X and is therefore a statement about a particular time bucket.

C.1.1. The Poisson Distribution

$$g(x) = \frac{\lambda^x e^{-\lambda}}{x!}, \quad x = 0, 1, 2, \ldots, \qquad E[X] = Var[X] = \lambda \tag{C.2}$$

The Poisson is what demand looks like when customers arrive independently at a constant average rate and each takes one unit. That is the standard model for a slow-moving part, and it is the distribution to reach for first when the item is demanded singly.

Notice what Equation C.2 fixes. The mean and the variance are the same parameter, so $VMR = 1$ exactly. Thus, the Poisson is not a family you fit to a variance; it is a family whose variance is decided the moment you estimate its mean. A demand history whose VMR is far from one is telling you that the Poisson is the wrong family, and no choice of λ will repair it.

C.1.2. The Geometric and Negative Binomial Distributions

The negative binomial is the family to reach for when $VMR > 1$, which is the usual case for real demand. Let $\beta = (1 - p)/p$. Then

$$g(x) = \binom{x+r-1}{r-1} p^r (1-p)^x, \quad x = 0, 1, 2, \ldots, \qquad E[X] = r\beta, \qquad Var[X] = \frac{r\beta}{p} \tag{C.3}$$

so that $VMR = 1/p > 1$ for every admissible p. The geometric is the special case $r = 1$, with $E[X] = \beta$ and $Var[X] = \beta/p$.

Two readings of Equation C.3 are useful and they are different. Counting failures before the rth success is the textbook reading and explains the binomial coefficient. The reading that matters here is that the negative binomial is a Poisson whose rate is itself random, drawn from a gamma distribution. That is, it is what you get when demand is Poisson but the rate varies from interval to interval, which is the situation an inventory analyst meets. Thus, the negative binomial is the Poisson with the constant-rate assumption relaxed, and the extra variance is the price of relaxing it.

A warning about p. Two conventions are in circulation. Equation C.3 makes p the probability of *success*, so the mean is $r(1-p)/p$. Much of the fitting literature, including the selection rules of Section C.4, makes p the probability of *failure*, so the mean is $rp/(1-p)$. The two are exchanged by replacing p with $1-p$, and a fit that comes out inverted is almost always this and not an error in the data. You should check which convention a routine uses before passing it a parameter.

C.1.3. The Binomial Distribution

$$g(x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, \ldots, n, \qquad E[X] = np, \qquad Var[X] = np(1-p) \tag{C.4}$$

Here $VMR = 1 - p < 1$, so the binomial is the family for demand that is *less* variable than Poisson. That is rarer than the other direction, and

when it appears it usually has a structural cause: a fixed population of n machines each of which may or may not need the part this period, or a contract that caps the quantity. Notice that the support is bounded above, which is the strongest statement any of these families makes and should be believed only when something in the situation actually enforces it.

C.1.4. The Discrete Empirical Distribution

Let $S = \{x_1, \dots, x_k\}$ represent the distinct values observed and p_j their relative frequencies. The empirical distribution is the data used as its own model, with no family assumed.

It is the right choice more often than its simplicity suggests, and Chapter 7 uses it. Thus, when the history is long enough to populate the range and the decision is a single quantile, fitting a family buys nothing and risks a poor fit. Two costs come with it. The support is bounded by what happened to be observed, so a quantile beyond the largest observation cannot be produced at all, and a long tail is exactly where inventory decisions live. And the estimate inherits every irregularity of the sample, including the ones that are noise.

The rule of thumb. Use the empirical distribution when the question is inside the data and a fitted family when it is outside. A service level of 90% on two hundred observations is inside. A service level of 99% is not.

C.2. Continuous Distributions

A continuous distribution is an approximation to a demand that arrives in whole units, and it is made for two reasons. The arithmetic is easier, and once demand is aggregated over a lead time or a season the granularity stops mattering. This section gives the four families the book uses and, for each, the condition under which the approximation is safe. Section C.3 gives their loss functions.

The diagnostic here is the coefficient of variation. Let

$$c_X = \frac{\sqrt{Var[X]}}{E[X]}, \qquad c_X^2 = \frac{Var[X]}{(E[X])^2} \tag{C.5}$$

Unlike the variance to mean ratio of Equation C.1, c_X is dimensionless, so it compares across items and across time buckets. Each family below either fixes c_X or restricts it, and that is what selects among them.

C.2.1. The Exponential Distribution

$$g(x) = \lambda e^{-\lambda x}, \quad x \geq 0, \qquad E[X] = \frac{1}{\lambda}, \qquad Var[X] = \frac{1}{\lambda^2} \tag{C.6}$$

Thus, $c_X = 1$ exactly. The exponential is the continuous counterpart of the Poisson: it is the time between Poisson arrivals, and it appears in this book as a lead time rather than as a demand. Like the Poisson it has no free parameter for variability, so it is a hypothesis to be tested rather than a family to be fitted to a spread.

Notice also that it is memoryless. A lead time modeled as exponential is one for which an order already three weeks late is no more likely to arrive tomorrow than one placed today. That is a strong claim about a supplier, and it is usually false.

C.2.2. The Gamma and Erlang Distributions

$$g(x) = \frac{x^{\alpha-1} e^{-x/\beta}}{\beta^{\alpha} \Gamma(\alpha)}, \quad x \geq 0, \qquad E[X] = \alpha\beta, \qquad Var[X] = \alpha\beta^2 \tag{C.7}$$

with shape $\alpha > 0$ and scale $\beta > 0$. Then $c_X = 1/\sqrt{\alpha}$, so the shape parameter alone sets the variability and the scale sets the size. The gamma is the flexible workhorse of this appendix: it covers every $c_X > 0$, it lives

on the non-negative half line where demand lives, and it reduces to the exponential at $\alpha = 1$.

The **Erlang** is the gamma with integer α, which is the sum of α independent exponentials. That buys a closed form for the distribution function and nothing else, and Table C.5 shows the loss functions are the gamma's unchanged. Its importance here is structural rather than computational: because an Erlang is a sum of exponentials, mixtures of Erlangs can be built to match any mean and variance, which is the procedure of Section C.4.2.

Notice what Equation C.7 does *not* have. There is no closed form for $G(x)$, so a quantile requires a numerical routine. That is rarely an obstacle, and it is not an argument for the normal.

C.2.3. The Normal Distribution

The normal is the most used and the least defensible of the four. It is used because Section B.2 makes it the limit of a sum, so a demand aggregated over many periods or many customers tends toward it, and because its loss functions are the tabulated ones of Table C.2.

Its defect is the one every inventory text mentions and few quantify. The support is the whole real line, so a normal demand model assigns positive probability to negative demand, and the amount is governed entirely by c_X.

Table C.1.: The probability a normal model puts below zero, as a function of the coefficient of variation. Notice the shape of the failure. Below about $c_X = 0.3$ the defect is invisible, and past $c_X = 1$ a sixth of the distribution is nonsense.

c_X	$P\{X < 0\}$ under a normal model
0.2	0.0000
0.4	0.0062

c_X	$P\{X < 0\}$ under a normal model
0.6	0.0478
1.0	0.1587
1.5	0.2525

Table C.1 is the rule in practice. Thus, a normal model is safe for demand whose coefficient of variation is below roughly 0.3, questionable to about 0.5, and wrong above it. Recall from Example 2.4 that intermittent items routinely run past $c_X = 1$, which is precisely where the normal fails, and why Chapter 7 does not reach for it there.

C.2.4. The Lognormal Distribution

If $\ln X$ is normal with mean μ and variance σ^2, then X is lognormal. It is the natural repair of the normal's defect: the support is strictly positive, the shape is right-skewed as demand data usually is, and it accommodates any c_X.

The parameterization is the trap. A fitted demand distribution reports the mean and variance of X, while the density is written in terms of the mean and variance of $\ln X$. The conversion is

$$\sigma^2 = \ln\left(\frac{Var[X]}{(E[X])^2} + 1\right) = \ln\left(1 + c_X^2\right), \qquad \mu = \ln(E[X]) - \frac{\sigma^2}{2} \tag{C.8}$$

and the lognormal rows of Table C.5 and Table C.6 are written in terms of $E[X]$ and $Var[X]$ for that reason. Notice that σ^2 depends on the data only through c_X, so the shape of a lognormal is set by the coefficient of variation as the gamma's is by α.

Choosing between the gamma and the lognormal is usually not worth a long argument. Both live on the positive half line, both take any

c_X, and on demand data they rarely separate by enough to change a policy. The lognormal has the heavier right tail, so it gives the larger answer at a high service level, and that difference grows with c_X. Where the two disagree enough to matter, the disagreement is the finding, and Section C.4 says what to do about it.

C.3. Loss Functions

Two quantities appear in almost every stochastic inventory cost expression in this book, and neither is an ordinary expectation. How much demand goes unmet when b units are on hand, and how much stock is left over when b units were provided. Both are expectations of a *truncated* difference, and the functions that give them are called **loss functions**.

Let X represent a random variable, usually a demand over some interval. Let b represent a stock level the demand is compared against. The notation $(X-b)^+ = \max(X-b, 0)$ is used throughout, so $(X-b)^+$ is the amount by which demand exceeds b, and zero when it does not.

Four functions carry almost all of the work, and a fifth is needed once, in Section C.3.6. Let $G(b) = P\{X \le b\}$ represent the cumulative distribution function of X. Then

$$G^0(b) = 1 - G(b), \qquad G^1(b) = E\left[(X-b)^+\right] \tag{C.9}$$

define the **complementary distribution function** and the **first order loss function**. The second order loss function is defined differently for the two cases, and the difference is not cosmetic:

$$G^2(b) = \begin{cases} \frac{1}{2} E\left[(X-b)^+(X-b-1)^+\right] & X \text{ discrete} \\ \frac{1}{2} E\left[\left((X-b)^+\right)^2\right] & X \text{ continuous} \end{cases} \tag{C.10}$$

Read G^0 carefully in the discrete case. Because $G(b) = P\{X \le b\}$, the complement is $G^0(b) = P\{X > b\}$, which is $P\{X \ge b+1\}$ and not $P\{X \ge b\}$. Every formula in this section depends on that convention, and using $P\{X \ge b\}$ instead produces answers that are correct only where the two happen to coincide. You should check which convention a source uses before taking a formula from it.

The first order function is the one with the obvious meaning, and the second order function has a use of its own. Chapter 8 needs the expected backorder level of a policy that orders Q at a time, and that level is a *difference* of second order loss functions evaluated at r and $r+Q$. Chapter 7 needs only the first. The *variance* of that backorder level needs one order higher again, which is what Section C.3.6 supplies.

C.3.1. Discrete Random Variables

Let $g(x) = P\{X = x\}$ represent the probability mass function. Writing the expectation as a sum over the range gives

$$G^1(b) = \sum_{x>b} (x-b)g(x) = \sum_{x \ge b} G^0(x) \tag{C.11}$$

$$G^2(b) = \tfrac{1}{2} \sum_{x>b} (x-b)(x-b-1)g(x) = \sum_{x>b} G^1(x) \tag{C.12}$$

The second form of each is the one to notice. A loss function is a tail sum of the function one order below it, which is what makes both computable by a single downward pass over the support.

For a non-negative X the pass can start from a moment. Notice that $G^1(0) = E[X]$ and that $G^2(0) = \frac{1}{2}E[X(X-1)]$, the latter being the **second binomial moment**. Thus, for $b \ge 0$,

$$G^1(b) = E[X] - \sum_{0 \le x < b} G^0(x), \qquad G^2(b) = \tfrac{1}{2}E[X(X-1)] - \sum_{0 < x \le b} G^1(x) \tag{C.13}$$

Equation C.13 is how these are computed in practice. That is, one accumulating loop produces G^0, G^1 and G^2 at every b from a single evaluation of the mass function at each point.

C.3.2. The Discrete Empirical Distribution

A demand history that is used directly, without fitting, is a discrete empirical distribution. Let $S = \{x_1, x_2, \dots, x_k\}$ represent its ordered set of distinct values and let $g(x_j) = p_j$ represent the mass at each. Inside the support the sums of Equation C.11 apply unchanged. Outside it they need care, because the support is finite in both directions.

$$G^1(b) = \begin{cases} E[X] - b & b \le x_1 \\ \sum_{x>b,\ x \in S} (x-b)g(x) & x_1 < b < x_k \\ 0 & b \ge x_k \end{cases} \tag{C.14}$$

Notice the first line. Below the smallest possible value every unit of demand exceeds b, so the truncation never binds and $G^1(b)$ is the plain expectation $E[X] - b$. It is *not* constant at $G^1(x_1)$, which is the value it takes only at $b = x_1$ itself. The same reasoning gives

$$G^2(b) = \tfrac{1}{2}\left(E\left[X^2\right] - (2b+1)E[X] + b(b+1)\right) \qquad \text{for } b \le x_1 \tag{C.15}$$

and zero for $b \ge x_k$. For example, take $S = \{2, 5, 9, 14\}$ with masses 0.20, 0.30, 0.35 and 0.15, so that $E[X] = 7.15$ units. At $b = 0$ the first order loss function is $7.15 - 0 = 7.15$ units, not the $7.15 - 2 = 5.15$ that treating

it as constant below the support would give. The two agree at $b = 2$ and nowhere else.

C.3.3. Continuous Random Variables

Let $g(x)$ represent the probability density function. The sums become integrals and the tail relations survive:

$$G^1(b) = \int_b^\infty (x-b)g(x)\,dx = \int_b^\infty G^0(x)\,dx, \qquad G^2(b) = \tfrac{1}{2}\int_b^\infty (x-b)^2 g(x)\,dx = \int_b^\infty G^1(x)\,dx \tag{C.16}$$

For a non-negative X and $b \leq 0$ the truncation again never binds, so $G^1(b) = E[X] - b$ and $G^2(b) = \frac{1}{2}\left(E[X^2] - 2bE[X] + b^2\right)$. Thus, $G^1(0) = E[X]$ and $G^2(0) = \frac{1}{2}E[X^2]$.

The closed forms for the continuous families are all obtained the same way, through the **partial expectation functions**. Let

$$H^1(b) = \int_b^\infty x\, g(x)\,dx, \qquad H^2(b) = \int_b^\infty x^2 g(x)\,dx \tag{C.17}$$

Expanding $(x-b)$ and $(x-b)^2$ inside the integrals of Equation C.16 and collecting terms gives

$$G^1(b) = H^1(b) - bG^0(b), \qquad G^2(b) = \tfrac{1}{2}\left(H^2(b) - 2bH^1(b) + b^2G^0(b)\right) \tag{C.18}$$

That is, a loss function for a new distribution follows from its partial expectations, and those are often the easier integrals. The lognormal entries of Table C.5 and Table C.6 are derived exactly this way.

C.3.4. The Standard Normal Case

The normal distribution is used often enough that its loss functions carry their own notation. Let $\phi(z)$ and $\Phi(z)$ represent the standard normal density and distribution function, and let $\Phi^0(z) = 1 - \Phi(z)$. Then

$$\Phi^1(z) = \phi(z) - z\Phi^0(z), \qquad \Phi^2(z) = \tfrac{1}{2}\left[(z^2+1)\,\Phi^0(z) - z\phi(z)\right] \quad \text{(C.19)}$$

and for $X \sim N(\mu, \sigma^2)$ with $z = (x-\mu)/\sigma$,

$$G^0(x) = \Phi^0(z), \qquad G^1(x) = \sigma\Phi^1(z), \qquad G^2(x) = \sigma^2\Phi^2(z) \qquad \text{(C.20)}$$

Notice the units. Φ^1 and Φ^2 are dimensionless, and the scaling by σ and σ^2 is what puts G^1 in units and G^2 in units squared. A loss function computed without that scaling is one of the more common errors in this material, and the units are what catch it.

Table C.2 gives the three functions at the values of z that arise most often.

Table C.2.: The standard normal density, complementary distribution function, and loss functions of the first three orders. Notice that Φ^1 is strictly decreasing and stays positive, which is what makes the expected shortage fall as the stock level rises without ever reaching zero. The third order column is defined in Section C.3.6 and is placed here so the normal quantities stay in one place.

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
-2.00	0.0540	0.9772	2.0085	2.4971	2.3342
-1.00	0.2420	0.8413	1.0833	0.9623	0.6819

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
0.00	0.3989	0.5000	0.3989	0.2500	0.1330
0.50	0.3521	0.3085	0.1978	0.1048	0.0485
1.00	0.2420	0.1587	0.0833	0.0377	0.0152
1.28	0.1758	0.1003	0.0475	0.0197	0.0074
1.50	0.1295	0.0668	0.0293	0.0114	0.0041
1.645	0.1031	0.0500	0.0209	0.0078	0.0027
2.00	0.0540	0.0228	0.0085	0.0029	0.0009
2.33	0.0264	0.0099	0.0034	0.0010	0.0003
3.00	0.0044	0.0013	0.0004	0.0001	0.0000

For example, take demand over a lead time as $N(50, 12^2)$ units and a stock level of 62 units. Then $z = (62 - 50)/12 = 1.00$, so from Table C.2 the probability of a shortage is $\Phi^0(1.00) = 0.1587$ and the expected shortage is $\sigma\Phi^1(1.00) = 12(0.0833) = 0.9996$ units, i.e. about one unit short on average.

That answer is worth checking a second way, and the partial expectation gives one. Reading $\phi(1.00) = 0.2420$ from the same table,

$$H^1(62) = \mu\Phi^0(z) + \sigma\phi(z) = 50(0.1587) + 12(0.2420) = 10.8390 \text{ units}$$

so from Equation C.18, $G^1(62) = 10.8390 - 62(0.1587) = 0.9996$ units. The two routes agree. Carried to full precision both give 0.999786 units, and the difference is the four decimal places Table C.2 is printed to.

That check is worth performing whenever a loss function is computed for the first time in a new setting, because it catches a missing σ immediately. Without the scaling the first route returns 0.0833 units while the second still returns 0.9996, and a disagreement by a factor of σ is unmistakable.

C.3.5. Loss Functions for Common Distributions

The closed forms are collected below, one table per order so that each formula has the width of the page to itself. Throughout, g is the mass or density function and G^0 is the complementary distribution function, so every loss function is expressed in terms of quantities the distribution already provides. The parameters and moments are not repeated here; they are in Section C.1 and Section C.2 with the family that owns them.

Table C.3.: First order loss functions for the discrete distributions, with $\beta = (1-p)/p$ as in Equation C.3. The binomial support is finite, so its sums terminate and Equation C.13 computes every order in one pass.

Distribution	$G^1(x)$
Poisson, mean λ	$-(x-\lambda)G^0(x) + \lambda g(x)$
Geometric	$\beta(1-p)^x$
Negative binomial, size r	$-(x-r\beta)G^0(x) + (x+r)\beta g(x)$
Binomial	No simpler form than Equation C.11

Table C.4.: Second order loss functions for the discrete distributions. Notice that the Poisson and negative binomial forms have the same shape at both orders, a multiple of G^0 plus a multiple of g, which is what the downward recursion of Equation C.13 produces in closed form.

Distribution	$G^2(x)$
Poisson, mean λ	$\frac{1}{2}\left\{\left[(x-\lambda)^2 + x\right]G^0(x) - \lambda(x-\lambda)g(x)\right\}$
Geometric	$\beta^2(1-p)^x$
Negative binomial, size r	$\frac{1}{2}\left\{\left[r(r+1)\beta^2 - 2r\beta x + x(x+1)\right]G^0(x) + \left[(r+1)\beta - x\right](x+r$

Distribution	$G^2(x)$
Binomial	No simpler form than Equation C.12

Table C.5.: First order loss functions for the continuous distributions. The Erlang is the gamma with an integer shape, which buys a closed form for $G(x)$ and changes nothing here. The lognormal row uses the μ and σ of $\ln X$ given by Equation C.8, and $E[X]$ is the mean of X itself.

Distribution	$G^1(x)$
Exponential, rate λ	$\frac{1}{\lambda}e^{-\lambda x}$
Gamma, shape α and scale β	$\beta\left[\left(\alpha-\frac{x}{\beta}\right)G^0(x)+xg(x)\right]$
Erlang	The gamma form, unchanged
Normal	$\sigma\Phi^1(z)$, with $z=(x-\mu)/\sigma$ and Φ^1 from Equation C.19
Lognormal	$E[X]\,\Phi(\sigma-z)-x\Phi^0(z)$, with $z=(\ln x-\mu)/\sigma$

Table C.6.: Second order loss functions for the continuous distributions. Notice that the gamma has no closed form for $G(x)$ while its loss functions *are* closed once G^0 is available numerically. That is the usual situation: a loss function is rarely harder to obtain than the distribution function it is built from.

Distribution	$G^2(x)$
Exponential, rate λ	$\frac{1}{\lambda^2}e^{-\lambda x}$
Gamma, shape α and scale β	$\frac{1}{2}\beta^2\left\{\left[\left(\alpha-\frac{x}{\beta}\right)^2+\alpha\right]G^0(x)+\left(\alpha-\frac{x}{\beta}+1\right)xg(x)\right\}$

Distribution	$G^2(x)$
Erlang	The gamma form, unchanged
Normal	$\sigma^2\Phi^2(z)$, with Φ^2 from Equation C.19
Lognormal	$\frac{1}{2}\left[\left(Var[X]+(E[X])^2\right)\Phi(2\sigma-z)-2xE[X]\,\Phi(\sigma-z)+x^2\Phi^0(z)\right.$

The lognormal rows deserve a word, because they are the ones most often misquoted. They are Equation C.18 applied to the partial expectations $H^1(x) = E[X]\Phi(\sigma - z)$ and $H^2(x) = \left(Var[X] + (E[X])^2\right)\Phi(2\sigma - z)$. It is easy to quote one of those partial expectations as though it were the loss function itself, and the resulting figure is too large by roughly a factor of three at stock levels near the mean.

C.3.6. The Third Order Loss Function

Everything above computes an **average**. Chapter 8 also needs the variability of a shortage, and that is where a third function enters.

Start with what the first two already give. For a **fixed** stock level b, the shortage $(X-b)^+$ is a random variable whose mean is $G^1(b)$, and its second moment follows directly from Equation C.10:

$$E\left[\left((X-b)^+\right)^2\right] = \begin{cases} 2G^2(b) + G^1(b) & X \text{ discrete} \\ 2G^2(b) & X \text{ continuous} \end{cases} \tag{C.21}$$

so $Var\left[(X-b)^+\right]$ needs nothing beyond G^2. Notice the extra G^1 in the discrete case. It is there because $(X-b)^+(X-b-1)^+ = \left((X-b)^+\right)^2 - (X-b)^+$ when X and b are whole numbers, which is the same convention difference Equation C.10 records.

The stock level is not always fixed, and that is what forces a third order function. In the policies of Chapter 8 the quantity the demand is

compared against is itself random, and the shortage has to be averaged over its distribution. Averaging G^1 over a range produces a difference of G^2, by the tail relations of Equation C.12 and Equation C.16. Averaging Equation C.21 over the same range produces a difference of the function one order higher again, and that function is not yet defined.

Thus, define the loss function of order n by extending Equation C.10 in the obvious way:

$$G^n(b) = \begin{cases} \dfrac{1}{n!} E\left[(X-b)^+(X-b-1)^+ \cdots (X-b-n+1)^+\right] & X \text{ discrete} \\ \dfrac{1}{n!} E\left[((X-b)^+)^n\right] & X \text{ continuous} \end{cases} \tag{C.22}$$

and in both cases each order is the tail accumulation of the one below it:

$$G^n(b) = \sum_{x>b} G^{n-1}(x) \qquad \text{or} \qquad G^n(b) = \int_b^\infty G^{n-1}(x)\,dx \tag{C.23}$$

The book needs $n = 3$ and no higher. Setting $n = 3$ in Equation C.22 gives the **third order loss function**, and Equation C.23 gives the identity that makes it useful:

$$\sum_{j=b+1}^{c} G^2(j) = G^3(b) - G^3(c) \tag{C.24}$$

That is, a sum of second order loss functions across a range of stock levels collapses to a difference of third order loss functions at its endpoints, in the way a sum of first order functions collapses to a difference of second order ones. Chapter 8 reports the mean backorder level with the second identity and its variance with Equation C.24.

C.3.7. Closed Forms for the Third Order

Table C.7 gives G^3 for the families whose lower orders appear in Table C.4 and Table C.6. The standard normal values are in the last column of Table C.2.

Table C.7.: Third order loss functions. In the gamma row $\alpha^{(k)} = \alpha(\alpha+1)\cdots(\alpha+k-1)$ and $G^0_{\alpha+k}$ is the complementary distribution function of a gamma with the shape raised by k and the same scale. Notice that the exponential row is the gamma row at $\alpha = 1$, and that the geometric and exponential both simply gain another factor of the mean.

Distribution	$G^3(x)$
Geometric	$\beta^3(1-p)^x$
Poisson, negative binomial	Computed by the recursion of Equation C.26
Exponential	$\frac{1}{\lambda^3}e^{-\lambda x}$
Gamma, shape α and scale β	$\frac{1}{6}\left[\alpha^{(3)}\beta^3 G^0_{\alpha+3}(x) - 3x\,\alpha^{(2)}\beta^2 G^0_{\alpha+2}(x) + 3x^2\alpha\beta\, G^0_{\alpha+1}(x) - x^3 G^0_{\alpha}(x)\right]$
Normal	$\sigma^3\Phi^3(z)$, with $\Phi^3(z) = \frac{1}{6}\left[(z^2+2)\phi(z) - z(z^2+3)\Phi^0(z)\right]$
Lognormal	$\frac{1}{6}\left[H^3(x) - 3xH^2(x) + 3x^2H^1(x) - x^3G^0(x)\right]$, with $H^3(x) = E[X^3]\,\Phi(3\sigma - z)$

Two of those rows generalize a pattern already visible. The geometric has $G^1 = \beta(1-p)^x$ and $G^2 = \beta^2(1-p)^x$, and the exponential has $1/\lambda$ and $1/\lambda^2$ against the same exponential factor, so in both families each order contributes one more factor of the mean and nothing else. Thus, for those two distributions G^n is known at every order without any further work.

The gamma and lognormal rows are Equation C.18 carried one order up. Expanding $(u-x)^3$ inside the integral of Equation C.22 and collecting terms gives

$$G^3(x) = \tfrac{1}{6}\left[H^3(x) - 3xH^2(x) + 3x^2H^1(x) - x^3G^0(x)\right], \qquad H^3(x) = \int_x^\infty u^3 g(u)\,du \tag{C.25}$$

so a third order loss function for any continuous family follows from its third partial expectation. For the gamma, $u\,g_{\alpha,\beta}(u) = \alpha\beta\,g_{\alpha+1,\beta}(u)$ applied three times gives $H^3(x) = \alpha^{(3)}\beta^3 G^0_{\alpha+3}(x)$, which is the gamma row.

For the discrete families the practice of Equation C.13 extends unchanged. Notice that $G^3(0) = \frac{1}{6}E[X(X-1)(X-2)]$, the **third binomial moment**, so for $b \geq 0$

$$G^3(b) = \tfrac{1}{6}E[X(X-1)(X-2)] - \sum_{0<x\leq b} G^2(x) \tag{C.26}$$

and the same accumulating loop that produces G^0, G^1 and G^2 produces G^3 at no extra cost in evaluations of the mass function. The third binomial moment is $\lambda^3/6$ for the Poisson and $r(r+1)(r+2)\beta^3/6$ for the negative binomial.

For example, take demand over a lead time as $N(50, 12^2)$ units and a stock level of 62 units, the case worked in Section C.3.4. Then $z = 1.00$ and from Table C.2, $\Phi^3(1.00) = 0.0152$, so

$$G^3(62) = \sigma^3\Phi^3(z) = 12^3(0.0152) = 1728(0.0152) = 26.27 \text{ units}^3$$

Carried to full precision that is 26.29 units cubed, and the difference is again the four decimal places Table C.2 is printed to.

Notice the units. G^1 is in units, G^2 in units squared and G^3 in units cubed, and the scaling by σ, σ^2 and σ^3 is what produces them. That check catches a missing power of σ in the same way Section C.3.4 catches a missing σ.

C.3.8. Mixtures

Chapter 7 describes demand that is intermittent, and a mixture is one way to represent it. Let X be distributed as a mixture of X_1 and X_2 with weights p_1 and $p_2 = 1 - p_1$, so that

$$g_X(x) = p_1 g_{X_1}(x) + p_2 g_{X_2}(x), \qquad G_X(x) = p_1 G_{X_1}(x) + p_2 G_{X_2}(x) \tag{C.27}$$

Because the loss functions are expectations and the expectation operator is linear, they mix the same way:

$$G^1_X(x) = p_1 G^1_{X_1}(x) + p_2 G^1_{X_2}(x), \qquad G^2_X(x) = p_1 G^2_{X_1}(x) + p_2 G^2_{X_2}(x) \tag{C.28}$$

and so do the raw moments, $E[X^n] = p_1 E[X_1^n] + p_2 E[X_2^n]$. The variance does not, and the extra term is the reason mixtures are useful for intermittent demand:

$$Var[X] = p_1 \, Var[X_1] + p_2 \, Var[X_2] + p_1 p_2 \left(E[X_1] - E[X_2]\right)^2 \tag{C.29}$$

Notice that last term. Two components with identical variances still produce a mixture more variable than either, and the excess grows with the square of the distance between their means. Thus, a mixture can reach a variability the component families cannot, which is what an intermittent demand series demands of a model.

What a mixture does not inherit is an inverse. There is no closed form for $G_X^{-1}(\cdot)$, which the newsvendor solution of Chapter 7 needs. However, the quantile is bracketed by the component quantiles. For any $p \in [0, 1]$, with $x_1 = G_{X_1}^{-1}(p)$ and $x_2 = G_{X_2}^{-1}(p)$,

$$\min\{x_1, x_2\} \le G_X^{-1}(p) \le \max\{x_1, x_2\} \tag{C.30}$$

Thus, the search has a starting interval that is free. For a discrete mixture, step up from $\min\{x_1, x_2\}$; for a continuous one, use the bracket as the initial interval of a bisection search. Either way the bracket is what turns an unbounded search into a bounded one.

C.3.9. Computing Them

The KSL implements these functions directly, so none of the closed forms in Section C.3.5 has to be coded by hand. A distribution that provides them implements `LossFunctionDistributionIfc`, which adds two methods:

```
fun firstOrderLossFunction(x: Double): Double
fun secondOrderLossFunction(x: Double): Double
```

Nine distributions implement it, and they are the ones tabulated above: `Poisson`, `Geometric`, `NegativeBinomial`, `Binomial` and `DEmpiricalCDF` for the discrete case, and `Exponential`, `Gamma`, `Normal` and `Lognormal` for the continuous case. `ShiftedLossFunctionDistribution` wraps any of them when the support has to be moved off the origin.

```
val ltd = Normal(mean = 50.0, variance = 144.0)

println(ltd.complementaryCDF(62.0))        // 0.158655, the
 ↪  shortage probability
println(ltd.firstOrderLossFunction(62.0))  // 0.999786, the
 ↪  expected shortage in units
println(ltd.secondOrderLossFunction(62.0)) // 5.424464
```

The Erlang is the `Gamma` with an integer shape parameter, and needs no separate class.

Check any implementation before trusting it, including this one. Two identities make that cheap, and both come from earlier in this section. For a non-negative random variable,

$$G^1(0) = E[X], \qquad G^2(b) \geq 0 ext for every b \tag{C.31}$$

The first is Equation C.13 and its continuous counterpart read at $b = 0$. The second holds because G^2 is the expectation of a product of two non-negative quantities. Thus, an implementation whose $G^1(0)$ differs from the distribution's own mean has its parameters crossed somewhere, and one that returns a negative G^2 is wrong outright. That check is worth performing every time a loss function is used from a new source, because it is one line and it catches the two errors that actually happen: a rate used where a mean belongs, and a truncation applied at the wrong end.

C.4. Choosing a Demand Distribution

Every model from Chapter 7 onward takes a demand distribution as an input, and the distribution carries more of the answer than the policy formula built on top of it does. This section covers the two routes to one. **Fitting** uses a sample of demand. **Moment matching** uses only a mean and a variance, which is the situation whenever the quantity of interest was derived rather than observed.

C.4.1. Fitting to a Sample

The sequence is the same one *Simulation Modeling using the KSL* gives at length in its distribution modeling appendix, compressed here to what an inventory analyst needs.

1. **Characterize before hypothesizing.** Plot the series in time order, then as a histogram. Compute the mean, the variance, and the diagnostic of Equation C.1 or Equation C.5. Count the zeros.
2. **Hypothesize from the characterization, not from habit.** A variance to mean ratio near one points at the Poisson, above one at the negative binomial, below one at the binomial. A coefficient of variation past 0.5 rules out the normal by Table C.1.
3. **Estimate the parameters**, by maximum likelihood or by matching moments.
4. **Test the fit**, with a chi-squared test for a discrete family or a Kolmogorov-Smirnov test for a continuous one.
5. **Look at the fit**, as a density over a histogram and as a quantile plot. The tail is what an inventory decision uses, and a test statistic averages over the whole distribution.

The KSL does the last four steps. `PDFModeler` for continuous data and `PMFModeler` for discrete data estimate parameters across a set of families, score the candidates, and rank them:

```
val results = PDFModeler(data).estimateAndEvaluateScores()
```

Three cautions are worth more than the mechanics.

Failing to reject is not proof. A goodness-of-fit test asks whether the data are inconsistent with a family. With two hundred observations it will fail to reject several families at once, and with twenty thousand it will reject all of them. Thus, the test narrows the field and does not choose.

A sales history is not a demand history. Sales are censored at whatever was on the shelf, so fitting to sales understates both the mean and the variance, and understates them most in the periods when the item mattered. Chapter 7 returns to this.

The reporting bucket changes the answer. Recall from Example 2.4 that the same transactions bucketed daily rather than monthly show more zeros

and a longer interval between demands, which can move an item from one demand class to another. Thus, fit at the bucket the decision is made in, and state which one it was.

C.4.2. Matching the First Two Moments

Often there is no sample. The demand during a lead time, which Chapter 8 needs, is a random sum whose mean and variance follow from the moments of demand and of lead time by Section B.5, and no observations of it exist at all. What is available is $\hat{\mu}$ and $\hat{\sigma}^2$, and what is wanted is a distribution.

For example, take demand averaging 5 units a week with a variance of 16, and a lead time averaging 10 weeks with a variance of 25. Then

$$E[D(L)] = 10(5) = 50 \text{ units}, \qquad Var[D(L)] = 10(16) + 25(5)^2 = 785 \tag{C.32}$$

so $c^2 = 785/2500 = 0.3140$ and $VMR = 15.70$ units. Notice that the lead time variance enters multiplied by the *square* of the demand rate, which is why an unreliable supplier dominates a variable demand in almost every real instance.

The gamma match is the default. The gamma can match any mean and variance, and inverting Equation C.7 gives the parameters directly:

$$\hat{\beta} = \frac{\hat{\sigma}^2}{\hat{\mu}}, \qquad \hat{\alpha} = \frac{\hat{\mu}^2}{\hat{\sigma}^2} \tag{C.33}$$

On Equation C.32 that gives $\hat{\beta} = 785/50 = 15.70$ and $\hat{\alpha} = 2500/785 = 3.1847$. The check is immediate: $\alpha\beta = 50.0000$ and $\alpha\beta^2 = 785.0000$. That check is worth performing every time, because a matching procedure that does not reproduce its own targets is the most common failure in this material.

A mixture of two Erlangs is the alternative, and it is the one to use when the distribution must be discrete in spirit or when a phase-type representation is wanted. Algorithm C.1 returns Erlang$(k_1, \lambda_1, k_2, \lambda_2, p_1, p_2)$ matching $E[X]$ and $Var[X]$.

Algorithm C.1 The Erlang mixture procedure. Two branches, because below a coefficient of variation of one the mixture needs shapes above one to be tight enough, and at or above it two exponentials with different rates are already spread enough. Notice `disc`, which is exactly zero when c^2 is the reciprocal of an integer and can go slightly negative in floating point, so it is clamped rather than trusted. At that same point p_1 evaluates to one, and to a shade above one in floating point, so it is clamped as well. Neither clamp changes an answer; both stop the procedure returning something that is not a probability.

```
erlangMixture(E[X], Var[X]):

    c2 = Var[X] / (E[X] * E[X])

    if c2 < 1:                                  // both components are Erlang
        k1 = floor(1 / c2)
        k2 = k1 + 1
        disc = max(0, k2*(1 + c2) - k2*k2*c2)     // exactly 0 when c2 = 1/k
        p1 = min(1, (k2*c2 - sqrt(disc)) / (1 + c2))
        lambda1 = (k2 - p1) / E[X]
        lambda2 = lambda1
    else:                                       // both components are exponential
        k1 = 1
        k2 = 1
        lambda1 = (2 / E[X]) * (1 + sqrt((c2 - 1/2) / (c2 + 1)))
        lambda2 = 4 / E[X] - lambda1
        p1 = lambda1 * (lambda2 * E[X] - 1) / (lambda2 - lambda1)

    p2 = 1 - p1
    return (k1, lambda1, k2, lambda2, p1, p2)
```

On Equation C.32, $c^2 = 0.3140$ falls in the first branch, giving $k_1 = 3$,

$k_2 = 4$, $p_1 = 0.5893$ and a common rate of $\lambda = 0.06821$ per unit. The mixture's mean is 50.0000 and its variance 785.0000, matching to six figures.

One implementation note. When c^2 is exactly $1/k$ for an integer k, the quantity under the square root in the first branch is exactly zero, and floating point arithmetic can make it very slightly negative. Thus, clamp it at zero rather than letting the square root fail.

Adan's rule selects a family rather than assuming one, and it is the procedure to use when the quantity being represented is known to be discrete. Let

$$a = c^2 - \frac{1}{\mu} = \frac{VMR - 1}{\mu} \tag{C.34}$$

Notice that a is a rescaled statement about how the variance compares with the mean, so $a = 0$ is the Poisson case. Table C.8 gives the rule, in the form of Adan et al. (1995) as extended by Rossetti and Ünlü (2011) to cover $a < -1$, where no discrete family can match.

Table C.8.: Adan's rule. Notice that the five cases partition the whole range of a, so the rule always returns something, and that the ordering is by increasing variability: bounded support, then equidispersion, then two degrees of overdispersion.

Condition	Family
$a < -1$	Gamma, by Equation C.33
$-1 \le a < 0$	A mixture of $\mathrm{Bin}(n, p)$ and $\mathrm{Bin}(n+1, p)$
$a = 0$	Poisson with mean μ
$0 < a < 1$	A mixture of $\mathrm{NB}(r, p)$ and $\mathrm{NB}(r+1, p)$

Condition	Family
$a \geq 1$	A mixture of $\text{Geom}(p_1)$ and $\text{Geom}(p_2)$

In every mixture below, q is the weight on the first component.

Mixed binomial, for $-1 \leq a < 0$:

$$n = \left\lfloor \frac{-1}{a} \right\rfloor, \qquad q = \frac{1 + a(n+1) + \sqrt{-an(n+1) - n}}{1+a}, \qquad p = \frac{\mu}{n + 1 - q} \tag{C.35}$$

Mixed negative binomial, for $0 < a < 1$:

$$r = \left\lfloor \frac{1}{a} \right\rfloor, \qquad q = \frac{a(r+1) - \sqrt{(r+1)(1-ar)}}{1+a}, \qquad p = \frac{\mu}{r + 1 - q + \mu} \tag{C.36}$$

Mixed geometric, for $a \geq 1$:

$$q = \frac{1}{1 + a + \sqrt{a^2 - 1}}, \qquad p_1 = \frac{\mu}{2q + \mu}, \qquad p_2 = \frac{\mu}{2(1-q) + \mu} \tag{C.37}$$

In Equation C.36 and Equation C.37, p is the probability of a failure, so that $\text{NB}(r, p)$ has mean $rp/(1-p)$ and $\text{Geom}(p)$ has mean $p/(1-p)$. That is the opposite of the convention in Equation C.3, and it is the single most likely thing to go wrong when these are implemented. The check of Equation C.33 applies here too and catches it at once: compute the mixture's mean and variance from its components and compare them with the targets.

On Equation C.32, $a = 0.3140 - 1/50 = 0.2940$, which falls in the mixed negative binomial branch. Then $r = \lfloor 1/0.2940 \rfloor = 3$, $q = 0.37788$ and $p = 0.93245$, and the resulting mixture has mean 50.0000 and variance 785.0000.

Notice that all three routes reproduce the same two moments and disagree about everything else. That is the lesson of moment matching: two numbers do not determine a distribution, and the choice of family is an assumption that the procedure hides rather than removes. Where the answer matters, compute it under two of them and report the spread.

C.4.3. When the Demand Is Intermittent

A series in which most periods have no demand at all is not served well by any of the above, because a single family has to explain both how often demand occurs and how large it is when it does. Two responses are available.

The first is a **zero-modified** family, which mixes a point mass at zero with one of the families of Section C.1. Rossetti and Ünlü (2011) recommend a zero-modified form of Adan's rule when the probability of a zero period exceeds one half, and switching back to the ordinary rule below that.

The second is a **compound representation**, in which the time between demands and the size of a demand are modeled separately and the period demand is the sum of a random number of sizes. Section B.5 gives the moments. That representation is the more faithful one, and it is the one that connects to the demand classification of Section 2.2.6, where the interval between demands and the variability of the sizes are the two axes.

Both are beyond what Chapter 7 needs, and both matter from Chapter 8 onward, where the quantity being modeled is demand over a lead time and the zeros no longer conveniently aggregate away.

D. Statistical Tables

Learning objectives

After reading this chapter you should be able to:

- read a standard normal probability from a table to four decimal places
- read the standard normal loss functions and scale them to a demand distribution
- read cumulative Poisson probabilities and Poisson loss functions for hand work

These tables exist so that the hand computations of Chapter 7 and Chapter 8 can be worked without software. Every figure in them was generated at full precision and rounded for printing, so a value computed with a calculator may differ from the table in the last place shown. Where a hand result and a computed result disagree by more than that, the disagreement is a mistake and not a rounding.

D.1. Standard Normal Distribution

Table D.1 gives $\Phi(z) = P\{Z \le z\}$ for a standard normal Z. Read the first decimal place of z down the left column and the second across the top. For example, $\Phi(1.64) = 0.9495$. For a negative argument use the symmetry of the density, $\Phi(-z) = 1 - \Phi(z)$, so $\Phi(-1.64) = 0.0505$.

Table D.1.: The standard normal cumulative distribution function $\Phi(z)$. Notice that the table runs only over non-negative z, because the symmetry of the normal density makes a second half unnecessary.

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	0.5000	0.5040	0.5080	0.5120	0.5160	0.5199	0.5239	0.5279	0.5319	0.5359
0.1	0.5398	0.5438	0.5478	0.5517	0.5557	0.5596	0.5636	0.5675	0.5714	0.5753
0.2	0.5793	0.5832	0.5871	0.5910	0.5948	0.5987	0.6026	0.6064	0.6103	0.6141
0.3	0.6179	0.6217	0.6255	0.6293	0.6331	0.6368	0.6406	0.6443	0.6480	0.6517
0.4	0.6554	0.6591	0.6628	0.6664	0.6700	0.6736	0.6772	0.6808	0.6844	0.6879
0.5	0.6915	0.6950	0.6985	0.7019	0.7054	0.7088	0.7123	0.7157	0.7190	0.7224
0.6	0.7257	0.7291	0.7324	0.7357	0.7389	0.7422	0.7454	0.7486	0.7517	0.7549
0.7	0.7580	0.7611	0.7642	0.7673	0.7704	0.7734	0.7764	0.7794	0.7823	0.7852
0.8	0.7881	0.7910	0.7939	0.7967	0.7995	0.8023	0.8051	0.8078	0.8106	0.8133
0.9	0.8159	0.8186	0.8212	0.8238	0.8264	0.8289	0.8315	0.8340	0.8365	0.8389
1.0	0.8413	0.8438	0.8461	0.8485	0.8508	0.8531	0.8554	0.8577	0.8599	0.8621
1.1	0.8643	0.8665	0.8686	0.8708	0.8729	0.8749	0.8770	0.8790	0.8810	0.8830
1.2	0.8849	0.8869	0.8888	0.8907	0.8925	0.8944	0.8962	0.8980	0.8997	0.9015
1.3	0.9032	0.9049	0.9066	0.9082	0.9099	0.9115	0.9131	0.9147	0.9162	0.9177
1.4	0.9192	0.9207	0.9222	0.9236	0.9251	0.9265	0.9279	0.9292	0.9306	0.9319
1.5	0.9332	0.9345	0.9357	0.9370	0.9382	0.9394	0.9406	0.9418	0.9429	0.9441
1.6	0.9452	0.9463	0.9474	0.9484	0.9495	0.9505	0.9515	0.9525	0.9535	0.9545
1.7	0.9554	0.9564	0.9573	0.9582	0.9591	0.9599	0.9608	0.9616	0.9625	0.9633
1.8	0.9641	0.9649	0.9656	0.9664	0.9671	0.9678	0.9686	0.9693	0.9699	0.9706
1.9	0.9713	0.9719	0.9726	0.9732	0.9738	0.9744	0.9750	0.9756	0.9761	0.9767
2.0	0.9772	0.9778	0.9783	0.9788	0.9793	0.9798	0.9803	0.9808	0.9812	0.9817
2.1	0.9821	0.9826	0.9830	0.9834	0.9838	0.9842	0.9846	0.9850	0.9854	0.9857
2.2	0.9861	0.9864	0.9868	0.9871	0.9875	0.9878	0.9881	0.9884	0.9887	0.9890
2.3	0.9893	0.9896	0.9898	0.9901	0.9904	0.9906	0.9909	0.9911	0.9913	0.9916
2.4	0.9918	0.9920	0.9922	0.9925	0.9927	0.9929	0.9931	0.9932	0.9934	0.9936
2.5	0.9938	0.9940	0.9941	0.9943	0.9945	0.9946	0.9948	0.9949	0.9951	0.9952
2.6	0.9953	0.9955	0.9956	0.9957	0.9959	0.9960	0.9961	0.9962	0.9963	0.9964

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
2.7	0.9965	0.9966	0.9967	0.9968	0.9969	0.9970	0.9971	0.9972	0.9973	0.9974
2.8	0.9974	0.9975	0.9976	0.9977	0.9977	0.9978	0.9979	0.9979	0.9980	0.9981
2.9	0.9981	0.9982	0.9982	0.9983	0.9984	0.9984	0.9985	0.9985	0.9986	0.9986
3.0	0.9987	0.9987	0.9987	0.9988	0.9988	0.9989	0.9989	0.9989	0.9990	0.9990
3.1	0.9990	0.9991	0.9991	0.9991	0.9992	0.9992	0.9992	0.9992	0.9993	0.9993
3.2	0.9993	0.9993	0.9994	0.9994	0.9994	0.9994	0.9994	0.9995	0.9995	0.9995
3.3	0.9995	0.9995	0.9995	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9997
3.4	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9998

D.2. Standard Normal Loss Functions

Table D.2 gives the density, the complementary distribution function, and the loss functions of the first three orders, defined in Section C.3.4 and Section C.3.6. To use them for a demand $X \sim N(\mu, \sigma^2)$ at a stock level x, form $z = (x - \mu)/\sigma$ and scale by Equation C.20: $G^1(x) = \sigma\Phi^1(z)$, $G^2(x) = \sigma^2\Phi^2(z)$ and $G^3(x) = \sigma^3\Phi^3(z)$.

Unlike Table D.1 this table carries negative arguments, because a stock level below the mean is an ordinary situation in inventory work and the loss functions have no symmetry to exploit.

Table D.2.: The standard normal density, complementary distribution function, and loss functions of orders one through three. Notice that every column is strictly decreasing in z and that each order falls faster than the one before it, because each is a tail accumulation of its predecessor.

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
−3.0	0.0044	0.9987	3.0004	4.9999	6.0000
−2.9	0.0060	0.9981	2.9005	4.7049	5.5149

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
−2.8	0.0079	0.9974	2.8008	4.4198	5.0587
−2.7	0.0104	0.9965	2.7011	4.1447	4.6306
−2.6	0.0136	0.9953	2.6015	3.8796	4.2295
−2.5	0.0175	0.9938	2.5020	3.6244	3.8543
−2.4	0.0224	0.9918	2.4027	3.3792	3.5042
−2.3	0.0283	0.9893	2.3037	3.1438	3.1782
−2.2	0.0355	0.9861	2.2049	2.9184	2.8751
−2.1	0.0440	0.9821	2.1065	2.7029	2.5942
−2.0	0.0540	0.9772	2.0085	2.4971	2.3342
−1.9	0.0656	0.9713	1.9111	2.3011	2.0944
−1.8	0.0790	0.9641	1.8143	2.1149	1.8737
−1.7	0.0940	0.9554	1.7183	1.9383	1.6711
−1.6	0.1109	0.9452	1.6232	1.7712	1.4857
−1.5	0.1295	0.9332	1.5293	1.6136	1.3166
−1.4	0.1497	0.9192	1.4367	1.4653	1.1627
−1.3	0.1714	0.9032	1.3455	1.3262	1.0232
−1.2	0.1942	0.8849	1.2561	1.1961	0.8972
−1.1	0.2179	0.8643	1.1686	1.0749	0.7837
−1.0	0.2420	0.8413	1.0833	0.9623	0.6819
−0.9	0.2661	0.8159	1.0004	0.8582	0.5909
−0.8	0.2897	0.7881	0.9202	0.7622	0.5100
−0.7	0.3123	0.7580	0.8429	0.6740	0.4382
−0.6	0.3332	0.7257	0.7687	0.5935	0.3749
−0.5	0.3521	0.6915	0.6978	0.5202	0.3193
−0.4	0.3683	0.6554	0.6304	0.4538	0.2707
−0.3	0.3814	0.6179	0.5668	0.3940	0.2283
−0.2	0.3910	0.5793	0.5069	0.3403	0.1917
−0.1	0.3970	0.5398	0.4509	0.2925	0.1601
0.0	0.3989	0.5000	0.3989	0.2500	0.1330
0.1	0.3970	0.4602	0.3509	0.2125	0.1099
0.2	0.3910	0.4207	0.3069	0.1797	0.0903
0.3	0.3814	0.3821	0.2668	0.1510	0.0738

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
0.4	0.3683	0.3446	0.2304	0.1262	0.0600
0.5	0.3521	0.3085	0.1978	0.1048	0.0485
0.6	0.3332	0.2743	0.1687	0.0865	0.0389
0.7	0.3123	0.2420	0.1429	0.0710	0.0311
0.8	0.2897	0.2119	0.1202	0.0578	0.0246
0.9	0.2661	0.1841	0.1004	0.0468	0.0194
1.0	0.2420	0.1587	0.0833	0.0377	0.0152
1.1	0.2179	0.1357	0.0686	0.0301	0.0118
1.2	0.1942	0.1151	0.0561	0.0239	0.0092
1.3	0.1714	0.0968	0.0455	0.0188	0.0070
1.4	0.1497	0.0808	0.0367	0.0147	0.0054
1.5	0.1295	0.0668	0.0293	0.0114	0.0041
1.6	0.1109	0.0548	0.0232	0.0088	0.0031
1.7	0.0940	0.0446	0.0183	0.0067	0.0023
1.8	0.0790	0.0359	0.0143	0.0051	0.0017
1.9	0.0656	0.0287	0.0111	0.0039	0.0012
2.0	0.0540	0.0228	0.0085	0.0029	0.0009
2.1	0.0440	0.0179	0.0065	0.0021	0.0007
2.2	0.0355	0.0139	0.0049	0.0016	0.0005
2.3	0.0283	0.0107	0.0037	0.0012	0.0003
2.4	0.0224	0.0082	0.0027	0.0008	0.0002
2.5	0.0175	0.0062	0.0020	0.0006	0.0002
2.6	0.0136	0.0047	0.0015	0.0004	0.0001
2.7	0.0104	0.0035	0.0011	0.0003	0.0001
2.8	0.0079	0.0026	0.0008	0.0002	0.0001
2.9	0.0060	0.0019	0.0005	0.0001	0.0000
3.0	0.0044	0.0013	0.0004	0.0001	0.0000
3.1	0.0033	0.0010	0.0003	0.0001	0.0000
3.2	0.0024	0.0007	0.0002	0.0000	0.0000
3.3	0.0017	0.0005	0.0001	0.0000	0.0000
3.4	0.0012	0.0003	0.0001	0.0000	0.0000
3.5	0.0009	0.0002	0.0001	0.0000	0.0000

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$	$\Phi^3(z)$
3.6	0.0006	0.0002	0.0000	0.0000	0.0000
3.7	0.0004	0.0001	0.0000	0.0000	0.0000
3.8	0.0003	0.0001	0.0000	0.0000	0.0000
3.9	0.0002	0.0000	0.0000	0.0000	0.0000
4.0	0.0001	0.0000	0.0000	0.0000	0.0000

D.3. Cumulative Poisson Probabilities

Table D.3 gives $G(x) = P\{X \leq x\}$ for a Poisson random variable with mean λ. Recall from Equation C.9 that the complementary function used throughout the loss function material is $G^0(x) = 1 - G(x) = P\{X > x\}$, which is $P\{X \geq x+1\}$ and not $P\{X \geq x\}$. Subtracting a table entry from one gives G^0 directly.

For example, the discrete newsvendor rule of Equation 7.13 asks for the smallest x whose cumulative probability reaches a critical ratio. At $\lambda = 6$ and a ratio of 0.6628, reading down the $\lambda = 6$ column gives $G(6) = 0.6063$, which is short, and $G(7) = 0.7440$, which clears it.

Table D.3.: The Poisson cumulative distribution function $G(x)$. An entry printed as 1.0000 is not exactly one; it is a probability that rounds to one at four decimal places.

x	0.5	1	2	3	4	5	6	7.5	8	10	12	15
0	0.6065	0.3679	0.1353	0.0498	0.0183	0.0067	0.0025	0.0006	0.0003	0.0000	0.0000	0.0000
1	0.9098	0.7358	0.4060	0.1991	0.0916	0.0404	0.0174	0.0047	0.0030	0.0005	0.0001	0.0000
2	0.9856	0.9197	0.6767	0.4232	0.2381	0.1247	0.0620	0.0203	0.0138	0.0028	0.0005	0.0000
3	0.9982	0.9810	0.8571	0.6472	0.4335	0.2650	0.1512	0.0591	0.0424	0.0103	0.0023	0.0002
4	0.9998	0.9963	0.9473	0.8153	0.6288	0.4405	0.2851	0.1321	0.0996	0.0293	0.0076	0.0009
5	1.0000	0.9994	0.9834	0.9161	0.7851	0.6160	0.4457	0.2414	0.1912	0.0671	0.0203	0.0028

x	0.5	1	2	3	4	5	6	7.5	8	10	12	15
6	1.0000	0.9999	0.9955	0.9665	0.8893	0.7622	0.6063	0.3782	0.3134	0.1301	0.0458	0.0076
7	1.0000	1.0000	0.9989	0.9881	0.9489	0.8666	0.7440	0.5246	0.4530	0.2202	0.0895	0.0180
8	1.0000	1.0000	0.9998	0.9962	0.9786	0.9319	0.8472	0.6620	0.5925	0.3328	0.1550	0.0374
9	1.0000	1.0000	1.0000	0.9989	0.9919	0.9682	0.9161	0.7764	0.7166	0.4579	0.2424	0.0699
10	1.0000	1.0000	1.0000	0.9997	0.9972	0.9863	0.9574	0.8622	0.8159	0.5830	0.3472	0.1185
11	1.0000	1.0000	1.0000	0.9999	0.9991	0.9945	0.9799	0.9208	0.8881	0.6968	0.4616	0.1848
12	1.0000	1.0000	1.0000	1.0000	0.9997	0.9980	0.9912	0.9573	0.9362	0.7916	0.5760	0.2676
13	1.0000	1.0000	1.0000	1.0000	0.9999	0.9993	0.9964	0.9784	0.9658	0.8645	0.6815	0.3632
14	1.0000	1.0000	1.0000	1.0000	1.0000	0.9998	0.9986	0.9897	0.9827	0.9165	0.7720	0.4657
15	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	0.9995	0.9954	0.9918	0.9513	0.8444	0.5681
16	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9998	0.9980	0.9963	0.9730	0.8987	0.6641
17	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	0.9992	0.9984	0.9857	0.9370	0.7489
18	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9997	0.9993	0.9928	0.9626	0.8195
19	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	0.9997	0.9965	0.9787	0.8752
20	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	0.9984	0.9884	0.9170
21	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9993	0.9939	0.9469
22	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9997	0.9970	0.9673
23	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9999	0.9985	0.9805
24	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9993	0.9888
25	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.9997	0.9938

D.4. Poisson Loss Functions

Chapter 8 works the Poisson case by hand, and doing so needs the loss functions rather than the probabilities. Table D.4 and Table D.5 give them at the same values of λ.

Read Table D.4 as the expected number of units of demand that go unmet when x units are available. Notice that the entry at $x = 0$ is $G^1(0) = \lambda$, the whole of expected demand, which is the arithmetic check on any column.

Table D.4.: The Poisson first order loss function $G^1(x)$, the expected shortage against a stock level of x.

x	0.5	1	2	3	4	5	6	7.5	8	10	12	15
0	0.5000	1.0000	2.0000	3.0000	4.0000	5.0000	6.0000	7.5000	8.0000	10.0000	12.0000	15.0000
1	0.1065	0.3679	1.1353	2.0498	3.0183	4.0067	5.0025	6.5006	7.0003	9.0000	11.0000	14.0000
2	0.0163	0.1036	0.5413	1.2489	2.1099	3.0472	4.0198	5.5053	6.0034	8.0005	10.0001	13.0000
3	0.0019	0.0233	0.2180	0.6721	1.3480	2.1718	3.0818	4.5255	5.0171	7.0033	9.0006	12.0000
4	0.0002	0.0043	0.0751	0.3194	0.7815	1.4368	2.2330	3.5847	4.0595	6.0137	8.0029	11.0003
5	0.0000	0.0007	0.0225	0.1346	0.4103	0.8773	1.5181	2.7167	3.1591	5.0429	7.0105	10.0011
6	0.0000	0.0001	0.0059	0.0507	0.1954	0.4933	0.9637	1.9582	2.3504	4.1100	6.0308	9.0039
7	0.0000	0.0000	0.0014	0.0172	0.0848	0.2555	0.5700	1.3363	1.6637	3.2401	5.0767	8.0115
8	0.0000	0.0000	0.0003	0.0053	0.0336	0.1221	0.3140	0.8609	1.1167	2.4604	4.1662	7.0295
9	0.0000	0.0000	0.0001	0.0015	0.0123	0.0540	0.1613	0.5229	0.7092	1.7932	3.3212	6.0670
10	0.0000	0.0000	0.0000	0.0004	0.0041	0.0222	0.0773	0.2993	0.4259	1.2511	2.5636	5.1368
11	0.0000	0.0000	0.0000	0.0001	0.0013	0.0085	0.0347	0.1616	0.2417	0.8341	1.9108	4.2553
12	0.0000	0.0000	0.0000	0.0000	0.0004	0.0030	0.0146	0.0823	0.1298	0.5309	1.3724	3.4401
13	0.0000	0.0000	0.0000	0.0000	0.0001	0.0010	0.0058	0.0397	0.0660	0.3225	0.9484	2.7077
14	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0022	0.0181	0.0318	0.1869	0.6299	2.0709
15	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0008	0.0078	0.0146	0.1035	0.4019	1.5365
16	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0032	0.0064	0.0547	0.2464	1.1046
17	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0013	0.0026	0.0277	0.1451	0.7688
18	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0005	0.0010	0.0134	0.0821	0.5176
19	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0002	0.0004	0.0062	0.0447	0.3371
20	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0001	0.0028	0.0234	0.2123
21	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0012	0.0118	0.1293
22	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0005	0.0057	0.0762
23	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0002	0.0027	0.0435
24	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0012	0.0240
25	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0005	0.0128

Table D.5.: The Poisson second order loss function $G^2(x)$. The entry at $x = 0$ is $\frac{1}{2}E[X(X-1)] = \lambda^2/2$, which is the check on this table, and the values run in units squared.

x	0.5	1	2	3	4	5	6	7.5	8	10	12	15
0	0.1250	0.5000	2.0000	4.5000	8.0000	12.5000	18.0000	28.1250	32.0000	50.0000	72.0000	112.5000
1	0.0185	0.1321	0.8647	2.4502	4.9817	8.4933	12.9975	21.6242	24.9999	41.0000	61.0000	98.5000
2	0.0021	0.0285	0.3233	1.2013	2.8718	5.4461	8.9777	16.1192	18.9963	32.9994	50.9998	85.5000
3	0.0002	0.0051	0.1053	0.5292	1.5238	3.2743	5.8959	11.5937	13.9792	25.9964	41.9997	73.4999
4	0.0000	0.0008	0.0302	0.2098	0.7423	1.8374	3.6629	8.0090	9.9197	19.9824	33.9966	62.4997
5	0.0000	0.0001	0.0077	0.0752	0.3320	0.9601	2.1448	5.2923	6.7606	14.9392	26.9859	52.4986
6	0.0000	0.0000	0.0018	0.0245	0.1366	0.4668	1.1811	3.3342	4.4102	10.8292	20.9554	43.4947
7	0.0000	0.0000	0.0004	0.0073	0.0518	0.2113	0.6111	1.9978	2.7465	7.5894	15.8783	35.4831
8	0.0000	0.0000	0.0001	0.0020	0.0182	0.0892	0.2970	1.1369	1.6298	5.1291	11.7122	28.4536
9	0.0000	0.0000	0.0000	0.0005	0.0059	0.0352	0.1358	0.6140	0.9206	3.3359	8.3910	22.3866
10	0.0000	0.0000	0.0000	0.0001	0.0018	0.0130	0.0584	0.3147	0.4947	2.0848	5.8274	17.2498
11	0.0000	0.0000	0.0000	0.0000	0.0005	0.0045	0.0237	0.1531	0.2530	1.2507	3.9166	12.9945
12	0.0000	0.0000	0.0000	0.0000	0.0001	0.0015	0.0091	0.0708	0.1231	0.7197	2.5442	9.5544
13	0.0000	0.0000	0.0000	0.0000	0.0000	0.0005	0.0033	0.0311	0.0571	0.3973	1.5958	6.8468
14	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0011	0.0130	0.0253	0.2103	0.9659	4.7759
15	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0004	0.0052	0.0107	0.1069	0.5640	3.2393
16	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0020	0.0043	0.0521	0.3176	2.1347
17	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0007	0.0017	0.0244	0.1726	1.3659
18	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0006	0.0110	0.0905	0.8483
19	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0001	0.0002	0.0048	0.0458	0.5113
20	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0020	0.0224	0.2990
21	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0008	0.0106	0.1696
22	0.0000	0.0000	0.0000	- 0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0003	0.0048	0.0934

x	0.5	1	2	3	4	5	6	7.5	8	10	12	15
23	0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0001	0.0021	0.0499
24	0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0009	0.0259
25	0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0004	0.0131

E. Kotlin and KSL Setup

> **Learning objectives**
>
> After reading this appendix you should be able to:
>
> - install the tools needed to run the simulation examples
> - open the course code project and run an example
> - add your own model to the project

E.1. What You Need

Two things, and nothing else:

- **A JDK, version 21 or later.** Temurin is a good default. Verify the installation with `java -version` at a terminal.
- **IntelliJ IDEA**, Community Edition. It has Kotlin support built in and understands Gradle projects without configuration.

You do not need to install Kotlin, Gradle, or the KSL separately. The course code project carries a Gradle wrapper that downloads the correct versions of Gradle and Kotlin on first use, and Gradle downloads the KSL from Maven Central.

E.2. Getting the Course Code Project

The Kotlin examples live in their own repository, separate from the book:

https://github.com/rossetti/RossettiInventoryBookCode

Clone it if you have Git. An updated example then costs you one command rather than a second download:

```
git clone
 ↪  https://github.com/rossetti/RossettiInventoryBookCode.git
cd RossettiInventoryBookCode
```

If you do not have Git, use the **Code** button on that page and choose **Download ZIP**, then unpack it. Either way you end up with one directory, and that directory is the Gradle project. Open it in IntelliJ with **File**, then **Open**, and select the directory itself rather than a file inside it. IntelliJ recognizes the Gradle build and configures the rest on its own.

When a chapter says that its code is in `code/`, it means this project. The book uses the short name because the directory you unpack it under is yours to name.

E.3. What Is in It

The project's structure mirrors the book:

```
build.gradle.kts       build configuration; the KSL dependency lives here
settings.gradle.kts
gradlew, gradle/       the Gradle wrapper
src/main/kotlin/inventory/
  <topic>/             one package per subject
```

```
src/test/kotlin/       the checks that pin what the book prints
data/                  the demand histories used for fitting
```

A package is named for its subject, so `inventory.dynamiclotsizing` holds the code of Chapter 5 and `inventory.newsvendor` holds the code of Chapter 7. Chapter numbers move when a book is revised and subjects do not, which is the same reason the section labels you see in cross-references are semantic rather than numbered.

Notice the test directory. Every number this book prints from a program is asserted there, so `./gradlew test` is a check that the copy you have agrees with the copy the book was written against. Run it once after you clone. If it passes, any later failure is yours and not the book's, which is worth knowing before you start changing things.

E.4. Running Your First Model

From a terminal, in the project directory:

```
./gradlew run
```

The first run takes a minute while Gradle, Kotlin, and the KSL are downloaded; later runs start immediately. The default example evaluates a newsvendor order quantity by Monte Carlo simulation, using the final buy of the mower deck belt from Chapter 7 with its fitted normal demand, and prints expected profit across a grid of order quantities:

```
Expected profit by order quantity (10,000 replications each)
         Q     avg profit   half-width
       400       16905.21        52.62
       420       17402.01        61.77
```

```
440       17708.05        72.49
460       17861.59        85.22
480       17817.87        96.08
500       17792.08       105.77
520       17514.23       115.26
540       17088.56       124.98
560       16740.80       132.01

Best quantity on this grid: Q = 460, expected profit 17861.59
Compare against the critical ratio solution -- see @sec-
newsvendor.
```

Notice two things about that output before you go further. The critical ratio puts the optimal buy at 475 belts, and the grid's flat top from 440 to 500 brackets it; the best point on the grid, 460, differs from its neighbors by less than their half-widths, so the simulation cannot tell 460 from 480 at ten thousand replications and does not pretend to. That agreement with the closed form is how you gain confidence that a simulation is built correctly. And the half-widths grow with the order quantity, because profit becomes more variable as more units are exposed to demand risk. A single replication would have told you neither.

To run it from IntelliJ instead, open the project directory as described in Section E.2 and use the gutter icon beside `fun main`.

E.5. Adding Your Own Model

Put the file in the package of the subject it belongs to, so a model of a continuous review policy goes in `src/main/kotlin/inventory/continuousreview/` beside the chapter's own examples. Give it a top-level `fun main()`. Kotlin compiles the top-level functions of a file named `MyPolicy.kt` into a class named `MyPolicyKt`, and that class name is what Gradle runs. To run your file from the terminal, pass the class as the `mainClass` property:

```
./gradlew run
 ↪  -PmainClass=inventory.continuousreview.MyPolicyKt
```

To make it the default instead, point `application { mainClass }` in `build.gradle.kts` at it; the block already reads the property and falls back to the newsvendor example when none is given. From IntelliJ, the gutter icon beside `fun main` runs the file with no configuration at all.

Keep the model's inputs at the top of `main` as named values, as every example in the project does, so that the next reader can see what was assumed without reading the code that uses it. Thus, a model of your own is one file, one `main`, and one property on the command line.

E.6. The KSL Itself

The KSL is a large library, and this book uses a small part of it. For the library on its own terms, its packages, its process view, and its output analysis machinery, see *Simulation Modeling using the KSL*.

The version this course pins is declared in the project's `build.gradle.kts`:

```
implementation("io.github.rossetti:KSLCore:R1.7.1")
```

References

Abaydulla, Anvar. 2016. "Grouping Techniques to Manage Large-Scale Multi-Item Multi-Echelon Inventory Systems." PhD thesis, University of Arkansas.

Adan, Ivo, Michel van Eenige, and Jacques Resing. 1995. "Fitting Discrete Distributions on the First Two Moments." *Probability in the Engineering and Informational Sciences* 9 (4): 623–32.

APICS. 1980. *Cycle Counting Training Guide*. American Production; Inventory Control Society.

Axsäter, Sven. 1990. "Simple Solution Procedures for a Class of Two-Echelon Inventory Problems." *Operations Research* 38: 64–69.

Axsäter, Sven. 1993. "Exact and Approximate Evaluation of Batch-Ordering Policies for Two-Level Inventory Systems." *Operations Research* 41 (4): 777–85.

Axsäter, Sven. 1998. "Evaluation of Installation Stock Based (R, Q)-Policies for Two-Level Inventory Systems with Poisson Demand." *Operations Research* 46 (3, Supplement): S135–45.

Axsäter, Sven. 2000. "Exact Analysis of Continuous Review (R, Q) Policies in Two-Echelon Inventory Systems with Compound Poisson Demand." *Operations Research* 48 (5): 686–96.

References

Axsäter, Sven. 2006. *Inventory Control*. 2nd ed. Kluwer Academic Publishers.

Bertsimas, Dimitris, and Daisuke Nakazato. 1995. "The Distributional Little's Law and Its Applications." *Operations Research* 43 (2): 298–310.

Brooks, Roger B., and Larry W. Wilson. 1995. *Inventory Record Accuracy: Unleashing the Power of Cycle Counting*. John Wiley; Sons.

Buck, James R., and Randall P. Sadowski. 1983. "Optimum Stratified Cycle Counting." *IIE Transactions* 15 (2): 119–26.

Croston, J. D. 1972. "Forecasting and Stock Control for Intermittent Demands." *Operational Research Quarterly* 23 (3): 289–303.

DeHoratius, Nicole, and Ananth Raman. 2004. "Inventory Record Inaccuracy: An Empirical Analysis." *Management Science*.

Deuermeyer, B., and L. B. Schwarz. 1981. "A Model for the Analysis of System Service Level in Warehouse/Retailer Distribution Systems: The Identical Retailer Case." In *Multi-Level Production/Inventory Control Systems*, edited by L. B. Schwarz. Studies in the Management Sciences 16. North-Holland.

Erlebacher, Steven J. 2000. "Optimal and Heuristic Solutions for the Multi-Item Newsvendor Problem with a Single Capacity Constraint." *Production and Operations Management* 9 (3): 303–18.

Ernst, Ricardo, and Morris A. Cohen. 1990. "Operations Related Groups (ORGs): A Clustering Procedure for Production/Inventory Systems." *Journal of Operations Management* 9 (4): 574–98.

Federgruen, Awi, and Michal Tzur. 1991. "A Simple Forward Algorithm to Solve General Dynamic Lot Sizing Models with n Periods in o(n Log

n) or o(n) Time." *Management Science* 37 (8): 909–25.

Flores, Benito E., and D. Clay Whybark. 1986. "Multiple Criteria ABC Analysis." *International Journal of Operations and Production Management* 6 (3): 38–46.

Graves, Stephen C. 1985. "A Multi-Echelon Inventory Model for a Repairable Item with One-for-One Replenishment." *Management Science* 31 (10): 1247–56.

Gross, Oliver. 1956. *A Class of Discrete-Type Minimization Problems*. RM-1655-PR. RAND Corporation.

Hadley, G., and T. M. Whitin. 1963. *Analysis of Inventory Systems*. Prentice-Hall.

Hopp, Wallace J., and Mark L. Spearman. 2011. *Factory Physics*. 3rd ed. Waveland Press.

Iglehart, Donald L., and Richard C. Morey. 1971. "Inventory Systems with Imperfect Asset Information." *Management Science* 18 (8): B388–94.

Johnston, F. R., and John E. Boylan. 1996. "Forecasting for Items with Intermittent Demand." *Journal of the Operational Research Society* 47: 113–21.

Kampen, Tim J. Van, Renzo Akkerman, and Dirk Pieter van Donk. 2012. "SKU Classification: A Literature Review and Conceptual Framework." *International Journal of Operations and Production Management* 32 (7): 850–76.

Kumar, Sameer, and Sant Arora. 1992. "Effects of Inventory Miscount and Non-Inclusion of Lead Time Variability on Inventory Performance." *IIE Transactions* 24 (2): 96–103.

Lau, Hon-Shiang, and Amy Hing-Ling Lau. 1996. "The Newsstand Problem: A Capacitated Multiple-Product Single-Period Inventory Problem." *European Journal of Operational Research* 94 (1): 29–42.

Lenard, J. D., and B. Roy. 1995. "Multi-Item Inventory Control: A Multicriteria View." *European Journal of Operational Research* 87 (3): 685–92.

Morey, Richard C. 1985. "Estimating Service Level Impacts from Changes in Cycle Count, Buffer Stock, or Corrective Action." *Journal of Operations Management* 5 (4): 411–18.

Muckstadt, John A., and Amar Sapra. 2010. *Principles of Inventory Management: When You Are down to Four, Order More*. Springer Series in Operations Research and Financial Engineering. Springer.

Nahmias, Steven, and Tava Lennon Olsen. 2015. *Production and Operations Analysis*. 7th ed. Waveland Press.

Neeley, Paul. 1987. "A Framework for Cycle Counting." *Production and Inventory Management* 24 (3): 64–69.

Orozco, R. E. 1993. "Is ABC Cycle Counting a Wasteful Approach." *American Production and Inventory Control Society International Conference Proceedings*, 536–37.

Palm, Conny. 1938. "Analysis of the Erlang Traffic Formula for Busy-Signal Arrangements." *Ericsson Technics* 4: 39–58.

Ramanathan, Ramakrishnan. 2006. "ABC Inventory Classification with Multiple-Criteria Using Weighted Linear Optimization." *Computers and Operations Research* 33 (3): 695–700.

Rossetti, Manuel D. 2026. *Simulation Modeling using the Kotlin Simulation*

Library (KSL). Self-published. https://rossetti.github.io/KSLBook/.

Rossetti, Manuel D., and A. V. Achlerkar. 2011. "Evaluation of Segmentation Techniques for Inventory Management in Large Scale Multi-Item Inventory Systems." *International Journal of Logistics Systems and Management* 8 (4): 403–24.

Rossetti, Manuel D., Nebil Buyurgan, Amit Bhonsle, Seda Gumrukcu, and Kiran Chittoori. 2008. "An Analysis of the Effect of Inventory Record Inaccuracy in a Two-Echelon Supply Chain." *International Journal of Inventory Research*.

Rossetti, Manuel D., Terry Collins, and Ravi Kurgund. 2001. "Inventory Cycle Counting: A Review." *Proceedings of the 2001 Industrial Engineering Research Conference*.

Rossetti, Manuel D., John English, Nebil Buyurgan, Ronald Walker, Seda Gumrukcu, and Long Yu. 2006. *Improving Inventory Record Accuracy Within Retail Store Operations*. {CELDi} Project Report. Center for Engineering Logistics; Distribution, University of Arkansas.

Rossetti, Manuel D., and Yasin Ünlü. 2011. "Evaluating the Robustness of Lead Time Demand Models." *International Journal of Production Economics* 134 (1): 159–76.

Roundy, Robin. 1985. "98%-Effective Integer-Ratio Lot-Sizing for One-Warehouse Multi-Retailer Systems." *Management Science* 31 (11): 1416–30.

Scarf, Herbert. 1960. "The Optimality of (s, S) Policies in the Dynamic Inventory Problem." In *Mathematical Methods in the Social Sciences*, edited by Kenneth J. Arrow, Samuel Karlin, and Patrick Suppes. Stanford University Press.

Sheppard, G. M., and K. A. Brown. 1993. "Predicting Inventory Record-Keeping Errors with Discriminant Analysis: A Field Experiment." *International Journal of Production Economics* 32: 39–51.

Sherbrooke, Craig C. 1968. "METRIC: A Multi-Echelon Technique for Recoverable Item Control." *Operations Research* 16 (1): 122–41.

Sherbrooke, Craig C. 1986. "VARI-METRIC: Improved Approximations for Multi-Indenture, Multi-Echelon Availability Models." *Operations Research* 34 (2): 311–19.

Sherbrooke, Craig C. 2004. *Optimal Inventory Modeling of Systems: Multi-Echelon Techniques*. 2nd ed. International Series in Operations Research and Management Science. Kluwer Academic Publishers.

Silver, Edward A., David F. Pyke, and Douglas J. Thomas. 2016. *Inventory and Production Management in Supply Chains*. 4th ed. CRC Press.

Slay, Michael. 1980. *VARI-METRIC: An Approach to Modeling Multi-Echelon Resupply When the Demand Process Is Poisson with a Gamma Prior*. Working Paper. Logistics Management Institute.

Stahl, Robert A. 1998. "Cycle Counting: A Quality Assurance Process." *Hospital Material Management Quarterly* 20 (2): 22–28.

Svoronos, Antony, and Paul Zipkin. 1988. "Estimating the Performance of Multi-Level Inventory Systems." *Operations Research* 36 (1): 57–72.

Syntetos, Aris A., and John E. Boylan. 2001. "On the Bias of Intermittent Demand Estimates." *International Journal of Production Economics* 71 (1–3): 457–66.

Syntetos, Aris A., and John E. Boylan. 2005. "The Accuracy of Intermittent Demand Estimates." *International Journal of Forecasting* 21 (2):

303–14.

Syntetos, Aris A., John E. Boylan, and J. D. Croston. 2005. "On the Categorization of Demand Patterns." *Journal of the Operational Research Society* 56: 495–503.

Tersine, Richard J. 1994. *Principles of Inventory and Materials Management*. 4th ed. Prentice Hall.

Teunter, Ruud H., M. Zied Babai, and Aris A. Syntetos. 2010. "ABC Classification: Service Levels and Inventory Costs." *Production and Operations Management* 19 (3): 343–52.

van der Wal, J. 1993. *The Lead Time Shift Theorem for the Continuous Review* (s, Q) *and* (s, S) *Inventory Systems*. Memorandum COSOR 93-34. Eindhoven University of Technology, Department of Mathematics and Computing Science.

Varghese, Vijith, Manuel D. Rossetti, Edward Pohl, Server Apras, and Douglas Marek. 2012. "Applying Actual Usage Inventory Management Best Practice in a Health Care Supply Chain." *Working Paper, University of Arkansas*.

Wagner, Harvey M., and Thomson M. Whitin. 1958. "Dynamic Version of the Economic Lot Size Model." *Management Science* 5 (1): 89–96.

Young, S., and W. Nie. 1992. "A Cycle-Count Model Considering Inventory Policy and Record Variance." *Production and Inventory Management* 33 (1): 11–16.

Zhang, R. Q., Wallace J. Hopp, and C. Supatgiat. 2001. "Spreadsheet Implementable Inventory Control for a Distribution Center." *Journal of Heuristics* 7 (2): 185–203.

Zipkin, Paul. 1986. "Stochastic Leadtimes in Continuous-Time Inventory Models." *Naval Research Logistics Quarterly* 33 (4): 763–74.

Zipkin, Paul H. 2000. *Foundations of Inventory Management*. McGraw-Hill.